

Lenguaje de fórmulas M de Power Query

Microsoft Power Query proporciona una experiencia de importación de datos eficaz que abarca muchas características. Power Query funciona con libros de Escritorio Analysis Services, Excel y Power BI, además de muchos servicios en línea, como Fabric, servicio Power BI, Power Apps, Microsoft 365 Customer Insights, etc. Una funcionalidad básica de Power Query es filtrar y combinar, es decir, combinar datos de uno o varios de una colección enriquecida de orígenes de datos admitidos. Cualquier mashup de datos de este tipo se expresa mediante el lenguaje de fórmulas M de Power Query. El lenguaje M es un lenguaje funcional y distingue mayúsculas de minúsculas similar a F#.

Comienza

INICIO RÁPIDO

[Paseo rápido de M](#)

CONCEPTO

[Expresiones, valores y expresión Let](#)

[Modelo de evaluación](#)

[Tipos y conversión de tipos](#)

[Cómo afecta la referencia cultural al formato de texto](#)

CURSOS

[Documentación de Power Query](#)

[Descripción de las funciones de M de Power Query](#)

Especificación

ARQUITECTURA

[Especificación del lenguaje M de Power Query](#)

[Sistema de tipos M de Power Query](#)

Referencia

REFERENCIA

[Referencia de la función M de Power Query](#)

[Referencia de enumeración M de Power Query](#)

[Referencia constante de M de Power Query](#)

[Referencia de valor dinámico de Power Query M](#)

Paseo rápido por el lenguaje de fórmulas M de Power Query

Artículo • 05/02/2025

En este recorrido rápido se describe cómo crear consultas de lenguaje de fórmulas de Power Query M.

⚠ Nota

M es un lenguaje que distingue mayúsculas de minúsculas.

Creación de una consulta con el editor de Power Query

Para crear una consulta avanzada, use el editor avanzado de [Power Query](#). Una consulta mashup se compone de variables, expresiones y valores encapsulados por una expresión de `let`. Una variable puede contener espacios mediante el identificador `#` con el nombre entre comillas como en `#"Variable name"`.

Una expresión `let` sigue esta estructura:

Power Query M

```
let
    Variablename = expression,
    #"Variable name" = expression2
in
    Variablename
```

Para crear una consulta M en el editor avanzado, siga este proceso básico:

1. Cree una serie de pasos de fórmula de consulta que comiencen con la instrucción `let`. Cada paso se define mediante un nombre de variable de paso. Una variable de M puede incluir espacios mediante el carácter `#`, como `#"Step Name"`. Un paso de fórmula puede ser una fórmula personalizada. Tenga en cuenta también que el lenguaje de fórmulas de Power Query distingue mayúsculas de minúsculas.
2. Cada paso de fórmula de consulta se construye a partir del paso anterior, haciendo referencia a este mediante su nombre de variable.

3. Genera un paso de fórmula de consulta usando la instrucción `in`. Por lo general, el último paso de consulta se usa como resultado del conjunto de datos final `in`.

Para obtener más información sobre las expresiones y los valores, vaya a [Expresiones, valores y expresión let](#).

Pasos sencillos de la fórmula M de Power Query

Supongamos que ha creado la siguiente transformación en el editor de Power Query. Esta consulta convierte los nombres de producto de forma adecuada, en este caso, con todas las iniciales en mayúscula.

Power Query

Home

Transform

Add column

View

Help

Get data

Enter data

New query

Manage connections

Data sources

Options

Options

Manage parameters

Parameters

Refresh

Query

Properties

Advanced editor

Manage

Table.TransformColumns(Orders, {"Item", Text.Proper})

OrderID

CustomerID

Item

Price

1

1

1

Fishing Rod

100

2

2

1

1 Lb. Worms

5

3

3

2

Fishing Net

25

Queries [1]

Para empezar, tiene una tabla similar a la siguiente:

[Expandir tabla](#)

OrderID	CustomerID	Item	Price
1	1	fishing rod	100
2	1	1 lb. worms	5
3	2	fishing net	25

Además, desea poner en mayúsculas la primera letra de cada palabra de la columna Item para generar la tabla siguiente:

 Expandir tabla

OrderID	CustomerID	Item	Price
1	1	Fishing Rod	100
2	1	1 lb. Gusanos	5
3	2	Fishing Net	25

Los pasos de la fórmula M para proyectar la tabla original en la tabla de resultados tienen este aspecto en el editor avanzado de Power Query:

Advanced editor ×

```
1 let Orders = Table.FromRecords({
2   [OrderID = 1, CustomerID = 1, Item = "fishing rod", Price = 100.0],
3   [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
4   [OrderID = 3, CustomerID = 2, Item = "fishing net", Price = 25.0]}),
5   #"Capitalized Each Word" = Table.TransformColumns(Orders, {"Item", Text.Proper})
6 in
7   #"Capitalized Each Word"
```

OK

Cancel

Este es el código que puede pegar en el editor avanzado de Power Query:

Power Query M

```
let Orders = Table.FromRecords({
    [OrderID = 1, CustomerID = 1, Item = "fishing rod", Price = 100.0],
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
    [OrderID = 3, CustomerID = 2, Item = "fishing net", Price = 25.0]}),
    #"Capitalized Each Word" = Table.TransformColumns(Orders, {"Item",
Text.Proper})
in
    #"Capitalized Each Word"
```

Vamos a revisar cada paso de fórmula.

1. **Pedidos:** cree una tabla con los datos de Pedidos.
2. **#"Poner En Mayúsculas Cada Palabra":** para poner en mayúsculas cada palabra, use [Table.TransformColumns](#).
3. En **#"Poner En Mayúsculas Cada Palabra":** se genera la tabla con la primera letra de cada palabra en mayúscula.

Contenido relacionado

- [Expresiones, valores y expresión let](#)
- [Operadores](#)
- [Conversión de tipos](#)

Comentarios

¿Le ha resultado útil esta página?



Sí



No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Especificación del lenguaje M de Power Query

La especificación describe los valores, expresiones, entornos y variables, identificadores y el modelo de evaluación que forman los conceptos básicos del lenguaje M de Power Query.

La especificación se incluye en los temas siguientes.

- [Introduction](#)
- [Estructura léxica](#)
- [Conceptos básicos](#)
- [Valores](#)
- [Tipos](#)
- [Operators](#)
- [Let](#)
- [Condicionales](#)
- [Funciones](#)
- [Tratamiento de errores](#)
- [Secciones](#)
- [Gramática consolidada](#)

Last updated on 22/11/2025

Introducción

26/08/2025

Información general

Microsoft Power Query proporciona una experiencia eficaz de "obtener datos" que abarca muchas características. Una funcionalidad básica de Power Query es filtrar y combinar, es decir, los datos "mash-up" de una o varias de una colección enriquecida de orígenes de datos admitidos. Cualquier mashup de datos de este tipo se expresa mediante el lenguaje de fórmulas de Power Query (conocido informalmente como "M"). Power Query inserta documentos M en una amplia gama de productos de Microsoft, incluidos Excel, Power BI, Analysis Services y Dataverse, para habilitar el mashup repetible de datos.

En este documento se proporciona la especificación de M. Después de una breve introducción que tiene como objetivo construir una primera intuición y familiaridad con el lenguaje, el documento trata precisamente el idioma en varios pasos progresivos:

1. La *estructura léxica* define el conjunto de textos que son léxicos de validez léxica.
2. Los valores, expresiones, entornos y variables, identificadores y el modelo de evaluación forman los *conceptos básicos* del lenguaje.
3. La especificación detallada de *valores*, tanto primitivos como estructurados, define el dominio de destino del lenguaje.
4. Los valores tienen *tipos*, en sí mismos un tipo especial de valor, que caracterizan los tipos fundamentales de valores y llevan metadatos adicionales específicos de las formas de valores estructurados.
5. El conjunto de *operadores* de M define qué tipos de expresiones se pueden formar.
6. *Las funciones*, otro tipo de valores especiales, proporcionan la base para una biblioteca estándar enriquecida para M y permiten la adición de nuevas abstracciones.
7. Se pueden producir errores al aplicar operadores o funciones durante la evaluación de expresiones. Aunque los errores no son valores, hay maneras de *controlar los errores* que asignan los errores a los valores.
8. *Permitir que las expresiones* permitan la introducción de definiciones auxiliares usadas para crear expresiones complejas en pasos más pequeños.
9. *Si las expresiones* admiten la evaluación condicional.

10. *Las secciones* proporcionan un mecanismo de modularidad simple. (Power Query aún no aprovecha las secciones).
11. Por último, una *gramática consolidada* recopila los fragmentos de gramática de todas las demás secciones de este documento en una única definición completa.

En el caso de los teóricos del lenguaje informático: el lenguaje de fórmulas especificado en este documento es un lenguaje funcional parcialmente diferido, de orden superior, puro y de orden superior.

Expresiones y valores

La construcción central de M es la *expresión*. Se puede evaluar una expresión (calculada), lo que produce un *único valor*.

Aunque muchos valores se pueden escribir literalmente como una expresión, un valor no es una expresión. Por ejemplo, la expresión `1` se evalúa como el valor `1`; las expresiones `1+1` se evalúan como el valor `2`. Esta distinción es sutil, pero importante. Las expresiones son recetas para la evaluación; los valores son los resultados de la evaluación.

En los ejemplos siguientes se muestran los distintos tipos de valores disponibles en M. Como convención, un valor se escribe con el formato literal en el que aparecerían en una expresión que se evalúa como solo ese valor. (Tenga en cuenta que `//` indica el inicio de un comentario que continúa hasta el final de la línea).

- Un valor *primitivo* es un valor de una sola parte, como un número, lógico, texto o null. Se puede usar un valor NULL para indicar la ausencia de datos.

Power Query M

```
123           // A number
true          // A logical
"abc"         // A text
null          // null value
```

- Un valor de *lista* es una secuencia ordenada de valores. M admite listas infinitas, pero si se escribe como literal, las listas tienen una longitud fija. Los caracteres `{` de llave y `}` denotan el principio y el final de una lista.

Power Query M

```
{123, true, "A"} // list containing a number, a logical, and
                  //      a text
{1, 2, 3}         // list of three numbers
```

- Un *registro* es un conjunto de *campos*. Un campo es un par nombre-valor donde el nombre es un valor de texto que es único dentro del registro del campo. La sintaxis literal de los valores de registro permite escribir los nombres sin comillas, un formulario también denominado *identificadores*. A continuación se muestra un registro que contiene tres campos denominados "A", "B" y "C", que tienen valores 1, 2 y 3.

```
Power Query M

[
    A = 1,
    B = 2,
    C = 3
]
```

- Una *tabla* es un conjunto de valores organizados en columnas (que se identifican por nombre) y filas. No hay ninguna sintaxis literal para crear una tabla, pero hay varias funciones estándar que se pueden usar para crear tablas a partir de listas o registros.

Por ejemplo:

```
Power Query M

#table( {"A", "B"}, { {1, 2}, {3, 4} } )
```

Esto crea una tabla de la forma siguiente:

A	B
1	2
3	4

- Una *función* es un valor que, cuando se invoca con argumentos, genera un nuevo valor. Para escribir una función, se enumeran los *parámetros* de la función entre paréntesis, seguidos del símbolo `=>` de paso a , seguido de la expresión que define la función. Esa expresión normalmente hace referencia a los parámetros (por nombre).

```
Power Query M

(x, y) => (x + y) / 2`
```

Evaluation

El modelo de evaluación del lenguaje M se modela después de que el modelo de evaluación se encuentre normalmente en hojas de cálculo, donde el orden de cálculo se puede determinar en función de las dependencias entre las fórmulas de las celdas.

Si ha escrito fórmulas en una hoja de cálculo como Excel, es posible que reconozca las fórmulas del resultado izquierdo en los valores de la derecha cuando se calcula:

	A		A
1	=A2 * 2	1	4
2	=A3 + 1	2	2
3	1	3	1

En M, las partes de una expresión pueden hacer referencia a otras partes de la expresión por nombre y el proceso de evaluación determina automáticamente el orden en el que se calculan las expresiones a las que se hace referencia.

Puede usar un registro para generar una expresión equivalente al ejemplo de hoja de cálculo anterior. Al inicializar el valor de un campo, puede hacer referencia a otros campos del registro mediante el nombre del campo, como se muestra en el ejemplo siguiente:

```
Power Query M

[
    A1 = A2 * 2,
    A2 = A3 + 1,
    A3 = 1
]
```

La expresión anterior es equivalente al ejemplo siguiente (en que ambas se evalúan como valores iguales):

```
Power Query M

[
    A1 = 4,
    A2 = 2,
    A3 = 1
]
```

Los registros se pueden contener dentro o *anidar* dentro de otros registros. Puede usar el *operador de búsqueda* ([[]]) para acceder a los campos de un registro por nombre. Por ejemplo,

el registro siguiente tiene un campo denominado `Sales` que contiene un registro y un campo denominado `Total` que tiene acceso a los `FirstHalf` campos y `SecondHalf` del `Sales` registro:

Power Query M

```
[
    Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
    Total = Sales[FirstHalf] + Sales[SecondHalf]
]
```

La expresión anterior es equivalente al ejemplo siguiente cuando se evalúa:

Power Query M

```
[
    Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
    Total = 2100
]
```

Los registros también se pueden contener en listas. Puede usar el *operador de índice posicional* (`{}`) para tener acceso a un elemento de una lista por su índice numérico. Los valores de una lista se conocen mediante un índice de base cero desde el principio de la lista. Por ejemplo, los índices `0` y `1` se usan para hacer referencia a los elementos primero y segundo de la lista siguiente:

Power Query M

```
[
    Sales =
        {
            [
                Year = 2007,
                FirstHalf = 1000,
                SecondHalf = 1100,
                Total = FirstHalf + SecondHalf // 2100
            ],
            [
                Year = 2008,
                FirstHalf = 1200,
                SecondHalf = 1300,
                Total = FirstHalf + SecondHalf // 2500
            ]
        },
    TotalSales = Sales{0}[Total] + Sales{1}[Total] // 4600
]
```


Las expresiones de miembro de lista y registro (así como [las expresiones let](#)) se evalúan mediante *la evaluación diferida*, lo que significa que solo se evalúan según sea necesario. Todas las demás expresiones se evalúan mediante *la evaluación diligente*, lo que significa que se evalúan inmediatamente cuando se encuentran durante el proceso de evaluación. Una buena manera de pensar en esto es recordar que la evaluación de una expresión de lista o registro devuelve un valor de lista o registro que recuerda cómo deben calcularse sus elementos de lista o campos de registro, cuando se solicitan (por operadores de búsqueda o índice).

Functions

En M, una *función* es una asignación de un conjunto de valores de entrada a un único valor de salida. Para escribir una función, primero asigna un nombre al conjunto necesario de valores de entrada (los parámetros a la función) y, a continuación, proporciona una expresión que calcula el resultado de la función utilizando esos valores de entrada (el cuerpo de la función) después del símbolo de paso a (`=>`). Por ejemplo:

Power Query M

```
(x) => x + 1 // function that adds one to a value
(x, y) => x + y // function that adds two values
```

Una función es un valor como un número o un valor de texto. En el ejemplo siguiente se muestra una función que es el valor de un campo Agregar, que luego se *invoca* o se ejecuta, desde varios otros campos. Cuando se invoca una función, se especifica un conjunto de valores que se sustituyen lógicamente por el conjunto necesario de valores de entrada dentro de la expresión del cuerpo de la función.

Power Query M

```
[
    Add = (x, y) => x + y,
    OnePlusOne = Add(1, 1), // 2
    OnePlusTwo = Add(1, 2) // 3
]
```

Biblioteca

M incluye un conjunto común de definiciones disponibles para su uso desde una expresión denominada *biblioteca estándar* o simplemente *biblioteca* para abreviar. Estas definiciones constan de un conjunto de valores con nombre. Los nombres de valores proporcionados por

una biblioteca están disponibles para su uso dentro de una expresión sin haber sido definidos explícitamente por la expresión. Por ejemplo:

Power Query M

```
Number.E // Euler's number e (2.7182...)
Text.PositionOf("Hello", "ll") // 2
```

Operadores

M incluye un conjunto de operadores que se pueden usar en expresiones. *Los operadores* se aplican a *los operandos* para formar expresiones simbólicas. Por ejemplo, en la expresión `1 + 2` los números `1` y `2` son operandos `1` y el operador es el operador de suma (`+`).

El significado de un operador puede variar en función del tipo de valores que son sus operandos. Por ejemplo, el operador más se puede usar con otros tipos de valores además de números:

Power Query M

```
1 + 2 // numeric addition: 3
#time(12,23,0) + #duration(0,0,2,0)
// time arithmetic: #time(12,25,0)
```

Otro ejemplo de un operador con un significado en función del operando es el operador de combinación (`&`):

Power Query M

```
"A" & "BC" // text concatenation: "ABC"
{1} & {2, 3} // list concatenation: {1, 2, 3}
[ a = 1 ] & [ b = 2 ] // record merge: [ a = 1, b = 2 ]
```

Tenga en cuenta que algunos operadores no admiten todas las combinaciones de valores. Por ejemplo:

Power Query M

```
1 + "2" // error: adding number and text isn't supported
```

Expresiones que, cuando se evalúan, encuentran condiciones de operador no [definidas](#) se evalúan como errores.

Metadatos

Los *metadatos* son información sobre un valor asociado a un valor. Los metadatos se representan como un valor de registro, denominado *registro de metadatos*. Los campos de un registro de metadatos se pueden usar para almacenar los metadatos de un valor.

Cada valor tiene un registro de metadatos. Si no se ha especificado el valor del registro de metadatos, el registro de metadatos está vacío (no tiene campos).

Los registros de metadatos proporcionan una manera de asociar información adicional con cualquier tipo de valor de forma discreta. La asociación de un registro de metadatos con un valor no cambia el valor ni su comportamiento.

Un valor `y` de registro de metadatos está asociado a un valor `x` existente mediante la sintaxis `x meta y`. Por ejemplo, el código siguiente asocia un registro de metadatos con `Rating` los campos y `Tags` con el valor `"Mozart"` de texto :

Power Query M

```
"Mozart" meta [ Rating = 5, Tags = {"Classical"} ]
```

En el caso de los valores que ya contienen un registro de metadatos no vacío, el resultado de aplicar `meta` es el de calcular la combinación de registros del registro existente y el nuevo registro de metadatos. Por ejemplo, las dos expresiones siguientes son equivalentes entre sí y a la expresión anterior:

Power Query M

```
("Mozart" meta [ Rating = 5 ]) meta [ Tags = {"Classical"} ]  
"Mozart" meta ([ Rating = 5 ] & [ Tags = {"Classical"} ])
```

Se puede tener acceso a un registro de metadatos para un valor determinado mediante la [Value.Metadata](#) función . En el ejemplo siguiente, la expresión del `ComposerRating` campo obtiene acceso al registro de metadatos del valor en el `Composer` campo y, a continuación, accede al `Rating` campo del registro de metadatos.

Power Query M

```
[  
    Composer = "Mozart" meta [ Rating = 5, Tags = {"Classical"} ],  
    ComposerRating = Value.Metadata(Composer)[Rating] // 5  
]
```

Expresión Let

Muchos de los ejemplos mostrados hasta ahora han incluido todos los valores literales de la expresión en el resultado de la expresión. La `let` expresión permite calcular, asignar nombres y, a continuación, usar un conjunto de valores en una expresión posterior que precede a `in`. Por ejemplo, en nuestro ejemplo de datos de ventas, podría hacer lo siguiente:

Power Query M

```
let
    Sales2007 =
        [
            Year = 2007,
            FirstHalf = 1000,
            SecondHalf = 1100,
            Total = FirstHalf + SecondHalf // 2100
        ],
    Sales2008 =
        [
            Year = 2008,
            FirstHalf = 1200,
            SecondHalf = 1300,
            Total = FirstHalf + SecondHalf // 2500
        ],
    TotalSales = Sales2007[Total] + Sales2008[Total]
in
    TotalSales // 4600
```

El resultado de la expresión anterior es un valor numérico (4600) que se calcula a partir de los valores enlazados a los nombres `Sales2007` y `Sales2008`.

Expresión If

La `if` expresión selecciona entre dos expresiones basadas en una condición lógica. Por ejemplo:

Power Query M

```
if 2 > 1 then
    2 + 2
else
    1 + 1
```

La primera expresión (2 + 2) se selecciona si la expresión lógica (2 > 1) es true y la segunda expresión (1 + 1) está seleccionada si es false. La expresión seleccionada (en este caso 2 + 2)

se evalúa y se convierte en el resultado de la `if` expresión (4).

Errors

Un *error* es una indicación de que el proceso de evaluación de una expresión no pudo generar un valor.

Los operadores y las funciones producen errores que encuentran condiciones de error o mediante la expresión de error. Los errores se controlan mediante la `try` expresión. Cuando se produce un error, se especifica un valor que se puede usar para indicar por qué se produjo el error.

Power Query M

```
let Sales =  
    [  
        Revenue = 2000,  
        Units = 1000,  
        UnitPrice = if Units = 0 then error "No Units"  
                     else Revenue / Units  
    ],  
    UnitPrice = try Number.ToText(Sales[UnitPrice]),  
    Result = "Unit Price: " &  
        (if UnitPrice[HasError] then UnitPrice[Error][Message]  
         else UnitPrice[Value])  
in  
    Result
```

En el ejemplo anterior se obtiene acceso al `Sales[UnitPrice]` campo y se da formato al valor que genera el resultado:

Power Query M

```
"Unit Price: 2"
```

Si el `Units` campo hubiera sido cero, el `UnitPrice` campo habría generado un error, que habría sido controlado por `try`. A continuación, el valor resultante habría sido:

Power Query M

```
"Unit Price: No Units"
```

Una `try` expresión convierte los valores y errores adecuados en un valor de registro que indica si la `try` expresión controló un error, o no, y el valor adecuado o el registro de error que

extraído al controlar el error. Por ejemplo, considere la siguiente expresión que genera un error y, a continuación, la controla inmediatamente:

Power Query M

```
try error "negative unit count"
```

Esta expresión se evalúa como el siguiente valor de registro anidado, explicando las [HasError] búsquedas de campos, [Error] y [Message] en el ejemplo de precio unitario anterior.

Power Query M

```
[
  HasError = true,
  Error =
    [
      Reason = "Expression.Error",
      Message = "negative unit count",
      Detail = null
    ]
]
```

Un caso común es reemplazar los errores por valores predeterminados. La try expresión se puede usar con una cláusula opcional otherwise para lograr solo eso en un formato compacto:

Power Query M

```
try error "negative unit count" otherwise 42
// 42
```

Estructura léxica

Documentos

Un *documento* de M es una secuencia ordenada de caracteres Unicode. En M se permiten diferentes clases de caracteres Unicode en diferentes partes de un documento de M. Para obtener información sobre las clases de caracteres Unicode, vea el *Estándar Unicode*, versión 3.0, sección 4.5.

Un documento consta de exactamente una *expresión*, o bien de grupos de *definiciones* organizadas en *secciones*. Las secciones se describen en detalle en el Capítulo 10. En términos conceptuales, se usan los pasos siguientes para leer una expresión de un documento:

1. El documento se descodifica según su esquema de codificación de caracteres en una secuencia de caracteres Unicode.
2. Se realiza el análisis léxico, con lo que se traduce la secuencia de caracteres Unicode en una secuencia de tokens. En las subsecciones restantes de esta sección se describe el análisis léxico.
3. Se realiza el análisis sintáctico, con lo que se traduce el flujo de tokens en un formato que se puede evaluar. Este proceso se describe en secciones posteriores.

Convenciones gramaticales

Las gramáticas léxicas y sintácticas se presentan mediante *producciones gramaticales*. En cada producción gramatical se define un símbolo que no es de terminal y las posibles expansiones de ese símbolo en secuencias de símbolos terminales o no terminales. En las producciones gramaticales, los símbolos *no terminales* se muestran en cursiva y los símbolos *terminales* se muestran en una fuente de ancho fijo.

La primera línea de una producción gramatical es el nombre del símbolo que no es de terminal que se define, seguido de un signo de dos puntos. Cada línea con sangría sucesiva contiene una posible expansión del símbolo de no terminal proporcionada como una secuencia de símbolos terminales o no terminales. Por ejemplo, la producción:

if-expression:

```
if if-condition then true-expression else false-expression
```

define *if-expression* que consta del token `if`, seguido de *if-condition*, seguido del token `then`, seguido de *true-expression*, seguido del token `else`, seguido de *false-expression*.

Cuando hay más de una expansión posible de un símbolo que no es de terminal, las alternativas se enumeran en líneas independientes. Por ejemplo, la producción:

variable-list:

variable

variable-list , *variable*

define una *variable-list* que consta de una *variable* o de una *variable-list* seguida de una *variable*. En otras palabras, la definición es recursiva y especifica que una lista de variables consta de una o más variables, separadas por comas.

Se usa un sufijo de subíndice "opt" para indicar un símbolo opcional. La producción:

field-specification:

optional _{opt} *field-name* = *tipo de campo*

es la abreviatura de:

especificación de campo:

field-name = *field-type*

optional *field-name* = *tipo de campo*

y define una instancia de *field-specification* que, de manera opcional, puede empezar con el símbolo de terminal *optional*, seguido de un valor *field-name*, el símbolo de terminal = y un valor *field-type*.

Las alternativas se muestran normalmente en líneas independientes, aunque en casos donde hay muchas alternativas, la frase "uno de" puede preceder a una lista de expansiones indicadas en una sola línea. Esto es simplemente una abreviatura para enumerar cada una de las alternativas en una línea independiente. Por ejemplo, la producción:

decimal-digit: uno de

0 1 2 3 4 5 6 7 8 9

es la abreviatura de:

decimal-digit:

0

1

2

3

4

5

6

Análisis léxico

En la producción *lexical-unit* se define la gramática léxica para un documento de M. Cada documento de M válido se ajusta a esta gramática.

unidad léxica:

*lexical-elements*_{opt}

lexical-elements:

lexical-element

lexical-element

lexical-elements

lexical-element:

whitespace

comentario de token

En el nivel léxico, un documento de M está formado por una secuencia de elementos *whitespace*, *comment* y *token*. Cada una de estas producciones se explica en las secciones siguientes. Solo los elementos *token* son significativos en la gramática sintáctica.

Espacio en blanco

El espacio en blanco se usa para separar los comentarios y los tokens dentro de un documento de M. El espacio en blanco incluye el carácter de espacio (que forma parte de la clase Zs de Unicode), así como las secuencias de caracteres de tabulación horizontal y vertical, avance de página y nueva línea. Las secuencias de caracteres de nueva línea incluyen retorno de carro, avance de línea y retorno de carro seguido de los caracteres separadores de avance de línea, línea siguiente y párrafo.

whitespace:

Cualquier carácter con la clase Zs de Unicode

Carácter de tabulación horizontal (U+0009)

Carácter de tabulación vertical (U+000B)

Carácter de avance de página (U+000C)

Carácter de retorno de carro (U+000D) seguido de un carácter de avance de línea (U+000A)

new-line-character

new-line-character:

Carácter de retorno de carro (U+000D)

Carácter de avance de línea (U+000A)

Carácter de línea siguiente (U+0085)

Carácter de línea siguiente (U+2028)

Carácter separador de párrafo (U+2029)

Por compatibilidad con las herramientas de edición de código fuente que agregan marcadores de fin de archivo y para permitir que un documento se vea como una secuencia de líneas terminadas correctamente, se aplican las transformaciones siguientes, en orden, a un documento de M:

- Si el último carácter del documento es un carácter Control-Z (U+001A), se elimina.
- Un carácter de retorno de carro (U+000D) se agrega al final del documento si ese documento no está vacío y si el último carácter del documento no es un retorno de carro (U+000D), un avance de línea (U+000A), un separador de línea (U+2028) o un separador de párrafo (U+2029).

Comentarios

Se admiten dos formatos de comentarios: de una sola línea y delimitados. Los *comentarios de una sola línea* comienzan con los caracteres `//` y se extienden hasta el final de la línea de código fuente. Los *comentarios delimitados* comienzan con los caracteres `/*` y terminan con los caracteres `*/`.

Los comentarios delimitados pueden abarcar varias líneas.

comentario:

single-line-comment

delimited-comment

single-line-comment:

`//` *single-line-comment-characters*_{opt}

single-line-comment-characters:

single-line-comment-character *single-line-comment-characters*_{opt}

single-line-comment-character:

Cualquier carácter Unicode excepto un *carácter de nueva línea*

delimited-comment:

`/*` *delimited-comment-text*_{opt} *asteriscos* `/`

delimited-comment-text:

delimited-comment-section *delimited-comment-text*_{opt}

delimited-comment-section:

/

asteriscos_{opt} not-slash-or-asterisk

Asteriscos:

* *asteriscos_{opt}*

not-slash-or-asterisk:

Cualquier carácter Unicode menos * o /

Los comentarios no se anidan. Las secuencias de caracteres /* y */ no tienen ningún significado especial en un comentario de una sola línea y las secuencias de caracteres // y /* tampoco lo tienen dentro de un comentario delimitado.

Los comentarios no se procesan dentro de los literales de texto. En el ejemplo

Power Query M

```
/* Hello, world
*/
    "Hello, world"
```

se incluye un comentario delimitado.

En el ejemplo

Power Query M

```
// Hello, world
//
"Hello, world" // This is an example of a text literal
```

se muestran varios comentarios de una línea.

Tokens

Un *token* es un identificador, palabra clave, literal, operador o signo de puntuación. Los espacios en blanco y los comentarios se usan para separar los tokens, pero no se consideran tokens.

seña:

identificador

palabra clave

Literal

operator-or-punctuator

Secuencias de escape de caracteres

Los valores de texto de M pueden contener caracteres Unicode arbitrarios. Pero los literales de texto se limitan a los caracteres gráficos y requieren el uso de *secuencias de escape* para los caracteres no gráficos. Por ejemplo, para incluir un carácter de retorno de carro, avance de línea o tabulación en un literal de texto, se pueden usar las secuencias de escape `#(cr)`, `#(lf)` y `#(tab)`, respectivamente. Para insertar los caracteres de inicio de secuencia de escape `#(` en un literal de texto, se debe aplicar la secuencia de escape al propio carácter `#`:

Power Query M

```
#(#)(
```

Las secuencias de escape también pueden contener valores de punto de código de Unicode cortos (cuatro dígitos hexadecimales) o largos (ocho dígitos hexadecimales). Por tanto, las tres secuencias de escape siguientes son equivalentes:

Power Query M

```
#(000D)      // short Unicode hexadecimal value
#(0000000D)  // long Unicode hexadecimal value
#(cr)        // compact escape shorthand for carriage return
```

Se pueden incluir varios códigos de escape en una sola secuencia de escape, separados por comas; por tanto, las dos secuencias siguientes son equivalentes:

Power Query M

```
#(cr,lf)
#(cr)#(lf)
```

A continuación se describe el mecanismo estándar de escape de caracteres en un documento de M.

character-escape-sequence:

```
#( escape-sequence-list )
```

escape-sequence-list:

single-escape-sequence

single-escape-sequence-sequence-sequence-list ,

single-escape-sequence:

long-unicode-escape-sequence

short-unicode-escape-sequence

control-character-escape-sequence

escape-escape

long-unicode-escape-sequence:

hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit

short-unicode-escape-sequence:

hexadecimal de dígito hexadecimal de dígito hexadecimal

control-character-escape-sequence:

control-character

carácter de control:

`cr`

`lf`

`tab`

escape-escape:

`#`

Literales

Un *literal* es una representación de código fuente de un valor.

literal:

logical-literal

number-literal

text-literal

null-literal

literal textual

Literales NULL

El literal NULL se usa para escribir el valor `null`. El valor `null` representa un valor ausente.

null-literal:

`null`

Literales lógicos

Un literal lógico se usa para escribir los valores `true` y `false`, y genera un valor lógico.

logical-literal:

`true`

`false`

Literales numéricos

Un literal numérico se usa para escribir un valor numérico y genera un valor de número.

number-literal:

decimal-number-literal

hexadecimal-number-literal

decimal-number-literal:

decimal-digits decimal-digits . exponent-part_{opt}

. decimal-digits exponent-part_{opt}

decimal-digits exponent-part_{opt}

decimal-digits:

decimal-digit decimal-digits_{opt}

decimal-digit: uno de

0 1 2 3 4 5 6 7 8 9

exponent-part:

e sign_{opt} decimal-digits

E sign_{opt} decimal-digits

sign: uno de

+ -

hexadecimal-number-literal:

0x dígitos hexadecimales

0X dígitos hexadecimales

hex-digits:

hex-digits_{opt}

dígito hexadecimal: uno de

0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

Se puede especificar un número en formato hexadecimal si se colocan los caracteres delante de `0x`. Por ejemplo:

Power Query M

```
0xff // 255
```

Tenga en cuenta que si se incluye un separador decimal en un literal numérico, debe tener al menos un dígito después. Por ejemplo, `1.3` es un literal numérico, pero `1.` y `1.e3` no lo son.

Literales de texto

Un literal de texto se usa para escribir una secuencia de caracteres Unicode y genera un valor de texto.

text-literal:

`" text-literal-charactersopt"`

text-literal-characters:

text-literal-character text-literal-characters_{opt}

text-literal-character:

single-text-character

character-escape-sequence

double-quote-escape-sequence

carácter de texto único:

Cualquier carácter excepto `"` (U+0022) o `#` (U+0023) seguido de `(` (U+0028)

double-quote-escape-sequence:

`"" (U+0022, U+0022)`

Para incluir comillas en un valor de texto, las comillas se repiten de la manera siguiente:

Power Query M

```
"The ""quoted"" text" // The "quoted" text
```

La producción *character-escape-sequence* se puede usar para escribir caracteres en valores de texto sin tener que codificarlos directamente como caracteres Unicode en el documento. Por ejemplo, un retorno de carro y un avance de línea se pueden escribir en un valor de texto como:

Power Query M

```
"Hello world#(cr,lf)"
```

Literales textuales

Los literales textuales se usan para almacenar una secuencia de caracteres Unicode que un usuario escribió como código, pero que no se puede analizar correctamente como código. En tiempo de ejecución, generan un valor de error.

verbatim-literal:

`#!" text-literal-charactersopt"`

Identificadores

Un *identificador* es un nombre que se usa para hacer referencia a un valor. Los identificadores pueden ser normales o entre comillas.

identificador:

regular-identifier

quoted-identifier

regular-identifier:

available-identifier

available-identifier dot-character regular-identifier

available-identifier:

Un *identificador* o *palabra clave* que no es una *palabra clave*

keyword-or-identifier:

*identifier-start-character identifier-part-characters*_{opt}

identifier-start-character:

letter-character

underscore-character

identifier-part-characters:

*identifier-part-character identifier-part-part-characters*_{opt}

identifier-part-character:

letter-character

decimal-digit-character

underscore-character

connecting-character

combining-character

formatting-character

dot-character:

 (U+002E)

underscore-character:

 (U+005F)

letter-character:

Un carácter Unicode de las clases Lu, Ll, Lt, Lm, Lo o Nl

combining-character:

Un carácter Unicode de las clases Mn o Mc

decimal-digit-character:

Un carácter Unicode de la clase Nd

connecting-character:

Un carácter Unicode de la clase Pc

formatting-character:

Un carácter Unicode de la clase Cf

Se puede usar un *identificador entre comillas* para permitir que cualquier secuencia de cero o más caracteres Unicode se use como identificador, incluidas las palabras clave, los espacios en blanco, los comentarios, los operadores y los signos de puntuación.

quoted-identifier:

```
#" text-literal-charactersopt "
```

Tenga en cuenta que las secuencias de escape y las comillas dobles para aplicar secuencias de escape a las comillas se pueden usar en una *identificador entre comillas*, al igual que en un *literal de texto*.

En el ejemplo siguiente se usa el identificador de comillas para los nombres que contienen un carácter de espacio:

Power Query M

```
[
    #"1998 Sales" = 1000,
    #"1999 Sales" = 1100,
    #"Total Sales" = #"1998 Sales" + #"1999 Sales"
]
```

En el ejemplo siguiente se usa el identificador de comillas para incluir el operador `+` en un identificador:

Power Query M

```
[
    #"A + B" = A + B,
    A = 1,
    B = 2
]
```

Identificadores generalizados

Hay dos lugares en M en los que no se generan ambigüedades por los identificadores que contienen espacios en blanco o que son palabras clave o literales numéricos. Estos lugares son los nombres de los campos de registro de un literal de registro y de un operador de acceso de campo (`[ ]`). Allí, M permite esos identificadores sin tener que usar identificadores entre comillas.

Power Query M

```
[
    Data = [ Base Line = 100, Rate = 1.8 ],
```

```
Progression = Data[Base Line] * Data[Rate]
```

```
]
```

Los identificadores que se usan para asignar nombres a los campos y acceder a ellos se denominan *identificadores generalizados* y se definen de la siguiente manera:

identificador generalizado:

generalized-identifier-part

identificador generalizado separado solo por espacios en blanco (`U+0020`)

generalized-identifier-part

generalized-identifier-part:

generalized-identifier-segment

decimal-digit-character generalized-identifier-segment

generalized-identifier-segment:

keyword-or-identifier

keyword-or-identifier dot-character keyword-or-identifier

Palabras clave

Una *palabra clave* es una secuencia de caracteres similar a un identificador que está reservada y no se puede usar como identificador excepto cuando se utiliza el [mecanismo de comillas de identificador](#) o cuando [se permite un identificador generalizado](#).

keyword: una de

```
and as each else error false if in is let meta not null or otherwise
```

```
section shared then true try type #binary #date #datetime
```

```
#datetimezone #duration #infinity #nan #sections #shared #table #time
```

Operadores y signos de puntuación

Hay varios tipos de operadores y signos de puntuación. Los operadores se usan en expresiones para describir las operaciones con uno o varios operandos implicados. Por ejemplo, la expresión `a + b` usa el operador `+` para agregar los dos operandos `a` y `b`. Los signos de puntuación se usan para agrupar y separar.

operator-or-punctuator: uno de

```
, ; = < <= > >= <> + - * / & ( ) [ ] { } @ ! ? ?? => .. ...
```

Conceptos básicos

Artículo • 20/11/2024

En esta sección se describen los conceptos básicos que aparecen en las secciones siguientes.

Valores

Un único fragmento de datos se denomina *valor*. En general, hay dos categorías generales de valores: *valores primitivos*, que son atómicos, y *valores estructurados*, que se crean a partir de valores primitivos y otros valores estructurados. Por ejemplo, los valores

Power Query M

```
1
true
3.14159
"abc"
```

son primitivos ya que no se componen de otros valores. Por otra parte, los valores

Power Query M

```
{1, 2, 3}
[ A = {1}, B = {2}, C = {3} ]
```

se construyen con valores primitivos y, en el caso del registro, otros valores estructurados.

Expresiones

Una *expresión* es una fórmula que se usa para construir valores. Una expresión se puede formar mediante una variedad de construcciones sintácticas. Los siguientes son algunos ejemplos de expresiones. Cada línea es una expresión independiente.

Power Query M

```
"Hello World"           // a text value
123                      // a number
1 + 2                    // sum of two numbers
{1, 2, 3}                 // a list of three numbers
```

```
[ x = 1, y = 2 + 3 ]      // a record containing two fields:
                          //      x and y
(x, y) => x + y           // a function that computes a sum
if 2 > 1 then 2 else 1    // a conditional expression
let x = 1 + 1 in x * 2    // a let expression
error "A"                // error with message "A"
```

La forma más sencilla de expresión, como se ha mencionado antes, es un literal que representa un valor.

Las expresiones más complejas se crean a partir de otras expresiones, llamadas *subexpresiones*. Por ejemplo:

Power Query M

1 + 2

La expresión anterior se compone realmente de tres expresiones. Los literales **1** y **2** son subexpresiones de la expresión primaria **1 + 2**.

La ejecución del algoritmo definido por las construcciones sintácticas que se usan en una expresión se denomina *evaluación* de la expresión. Cada tipo de expresión tiene reglas sobre cómo se evalúa. Por ejemplo, una expresión literal como **1** generará un valor constante, mientras que la expresión **a + b** tomará los valores resultantes generados al evaluar otras dos expresiones (**a** y **b**), y los sumará en función de un conjunto de reglas.

Entornos y variables

Las expresiones se evalúan dentro de un entorno determinado. Un *entorno* es un conjunto de valores con nombre, denominados *variables*. Cada variable de un entorno tiene un nombre único dentro del entorno, denominado *identificador*.

Una expresión de nivel superior (o *raíz*) se evalúa dentro del *entorno global*. El evaluador de expresiones proporciona el entorno global, en lugar de determinarse a partir del contenido de la expresión que se evalúa. El contenido del entorno global incluye las definiciones de la biblioteca estándar y se puede ver afectado por las exportaciones de secciones de algún conjunto de documentos. (para simplificar, en los ejemplos de esta sección se asumirá un entorno global vacío. Es decir, se supone que no hay ninguna biblioteca estándar y que no hay otras definiciones basadas en secciones).

La expresión primaria determina el entorno que se usa para evaluar una subexpresión. La mayoría de los tipos de expresión primaria evaluarán una subexpresión en el mismo

entorno en el que se hayan evaluado, pero algunas usarán otro. El entorno global es el *entorno primario* en el que se evalúa la expresión global.

Por ejemplo, *record-initializer-expression* evalúa la subexpresión para cada campo con un entorno modificado. El entorno modificado incluye una variable para cada uno de los campos del registro, excepto el que se está inicializando. La inclusión de los demás campos del registro permite que los campos dependan de los valores de los campos. Por ejemplo:

Power Query M

```
[
    x = 1,          // environment: y, z
    y = 2,          // environment: x, z
    z = x + y       // environment: x, y
]
```

Del mismo modo, *let-expression* evalúa la subexpresión para cada variable con un entorno que contenga cada una de las variables de *let*, excepto la que se está inicializando. *let-expression* evalúa la expresión que sigue a "in" con un entorno que contenga todas las variables:

Power Query M

```
let
    x = 1,          // environment: y, z
    y = 2,          // environment: x, z
    z = x + y       // environment: x, y
in
    x + y + z       // environment: x, y, z
```

(Tanto *record-initializer-expression* como *let-expression* definen realmente *dos* entornos, y uno incluye la variable que se inicializa. Esto resulta útil para las definiciones recursivas avanzadas y se describe en [Referencias de identificador](#)).

Para crear los entornos de las subexpresiones, las variables nuevas se "mezclan" con las del entorno primario. En el ejemplo siguiente se muestran los entornos para los registros anidados:

Power Query M

```
[
    a =
    [
```

```

        x = 1,      // environment: b, y, z
        y = 2,      // environment: b, x, z
        z = x + y    // environment: b, x, y
    ],
    b = 3            // environment: a
]

```

En el ejemplo siguiente se muestran los entornos para un registro anidado dentro de una instrucción `let`:

Power Query M

```

Let
    a =
    [
        x = 1,      // environment: b, y, z
        y = 2,      // environment: b, x, z
        z = x + y    // environment: b, x, y
    ],
    b = 3            // environment: a
in
    a[z] + b        // environment: a, b

```

La combinación de variables con un entorno puede introducir un conflicto entre las variables (ya que cada variable de un entorno debe tener un nombre único). El conflicto se resuelve de la siguiente manera: si el nombre de una variable nueva que se va a combinar es igual que el de una variable existente en el entorno primario, la variable nueva tendrá prioridad en el nuevo entorno. En el ejemplo siguiente, la variable interna `x` (la que está más profundamente anidada) tendrá prioridad sobre la variable externa `x`.

Power Query M

```

[
    a =
    [
        x = 1,      // environment: b, x (outer), y, z
        y = 2,      // environment: b, x (inner), z
        z = x + y    // environment: b, x (inner), y
    ],
    b = 3,          // environment: a, x (outer)
    x = 4           // environment: a, b
]

```

Referencias de identificador

Una *referencia de identificador* se usa para hacer referencia a una variable dentro de un entorno.

identifier-expression:

identifier-reference

identifier-reference:

exclusive-identifier-reference

inclusive-identifier-reference

La forma más sencilla de referencia de identificador es *exclusive-identifier-reference*:

exclusive-identifier-reference:

identifier

Es un error que un elemento *exclusive-identifier-reference* haga referencia a una variable que no forma parte del entorno de la expresión en la que aparece el identificador.

Es un error que un elemento *exclusive-identifier-reference* haga referencia a un identificador que se está inicializando actualmente, si el identificador al que se hace referencia está definido en un elemento *record-initializer-expression* o *let-expression*. En su lugar, se puede usar *inclusive-identifier-reference* para obtener acceso al entorno que incluye el identificador que se va a inicializar. Si se usa un elemento *inclusive-identifier-reference* en cualquier otra situación, este equivale a un elemento *exclusive-identifier-reference*.

inclusive-identifier-reference:

@ *identificador*

Esto resulta útil a la hora de definir funciones recursivas, ya que el nombre de la función normalmente no se encontraría en el ámbito.

Power Query M

```
[
    Factorial = (n) =>
        if n <= 1 then
            1
        else
            n * @Factorial(n - 1), // @ is scoping operator

    x = Factorial(5)
]
```

Como sucede con *record-initializer-expression*, *inclusive-identifier-reference* se puede usar dentro de *let-expression* para acceder al entorno que incluye el identificador que se inicializa.

Orden de evaluación

Considere la expresión siguiente que inicializa un registro:

```
Power Query M

[
    C = A + B,
    A = 1 + 1,
    B = 2 + 2
]
```

Cuando se evalúa, esta expresión genera el siguiente valor de registro:

```
Power Query M

[
    C = 6,
    A = 2,
    B = 4
]
```

La expresión indica que para realizar el cálculo `A + B` para el campo `C`, se deben conocer los valores de los campos `A` y `B`. Este es un ejemplo de una *ordenación de dependencias* de cálculos proporcionada por una expresión. El evaluador de M se atiene a la ordenación de dependencias proporcionada por las expresiones, pero es libre de realizar los cálculos restantes en el orden que elija. Por ejemplo, el orden de cálculo podría ser este:

```
Power Query M

A = 1 + 1
B = 2 + 2
C = A + B
```

O bien, podría ser:

```
Power Query M

B = 2 + 2
A = 1 + 1
C = A + B
```

O bien, ya que como `A` y `B` no dependen entre sí, se pueden calcular de manera simultánea:

$B = 2 + 2$ simultáneamente con $A = 1 + 1$

$C = A + B$

Efectos secundarios

Al permitir que un evaluador de expresiones calcule de forma automática el orden de los cálculos para los casos en los que no hay dependencias explícitas indicadas por la expresión, es un modelo de cálculo sencillo y eficaz.

Pero se basa en la capacidad de reordenar los cálculos. Como las expresiones pueden llamar a funciones, y esas funciones podrían observar un estado externo a la expresión mediante la emisión de consultas externas, es posible construir un escenario en el que el orden del cálculo sea importante, pero que no se capture en el orden parcial de la expresión. Por ejemplo, una función puede leer el contenido de un archivo. Si se llama a esa función varias veces, se pueden observar cambios externos en ese archivo y, por tanto, la reordenación puede producir diferencias tangibles en el comportamiento del programa. En función del orden de evaluación observado para la corrección de una expresión de M, se produce una dependencia de determinadas opciones de implementación que pueden variar de un evaluador a otro, o incluso dentro del mismo evaluador en circunstancias variables.

Inmutabilidad

Una vez que se ha calculado un valor, es *inmutable*, lo que significa que ya no se puede cambiar. Esto simplifica el modelo para evaluar una expresión y facilita el razonamiento sobre el resultado, ya que no es posible cambiar un valor una vez que se haya usado para evaluar un elemento posterior de la expresión. Por ejemplo, un campo de registro solo se calcula cuando es necesario. Pero una vez calculado, permanece fijo durante la vigencia del registro. Incluso si se ha producido un error al intentar calcular el campo, ese mismo error se volverá a producir en cada intento de acceso a ese campo de registro.

Una excepción importante a la regla inmutable una vez calculada se aplica a los valores de lista, tabla y binarios, que tienen *semántica de transmisión*. La semántica de transmisión permite a M transformar todos a la vez conjuntos de datos que no caben en la memoria. Con la transmisión por secuencias, los valores devueltos al enumerar una tabla, lista o valor binario determinado se producen según demanda cada vez que se solicitan. Dado que las expresiones que definen los valores enumerados se evalúan cada vez que se enumeran, el resultado que producen puede ser diferente en varias enumeraciones. Esto no significa que varias enumeraciones siempre den como resultado

valores diferentes, solo que pueden ser diferentes si la fuente de datos o la lógica M que se utiliza no es determinista.

Además, tenga en cuenta que la aplicación de funciones *no* es lo mismo que la construcción de valores. Las funciones de biblioteca pueden exponer el estado externo (como la hora actual o los resultados de una consulta en una base de datos que evoluciona con el tiempo) y representarlos como *no deterministas*. Aunque las funciones definidas en M, como tales, no exponen ningún comportamiento no determinista, pueden hacerlo si se definen para invocar a otras funciones que sean no deterministas.

En M, un origen final del no determinismo son los *errores*. Cuando se producen, los errores detienen las evaluaciones (hasta el nivel en el que se controlan mediante una expresión `try`). Normalmente, no se puede ver si `a + b` ha provocado la evaluación de `a` antes de `b` o de `b` antes de `a` (se omite la simultaneidad por motivos de simplicidad). Pero si la subexpresión que se ha evaluado en primer lugar genera un error, se puede determinar cuál de las dos expresiones se ha evaluado en primer lugar.

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Valores

21/08/2025

Un valor son los datos que se generan al evaluar una expresión. En esta sección se describen los tipos de valores del lenguaje M. Cada tipo de valor está asociado a una sintaxis literal, un conjunto de valores de ese tipo, un conjunto de operadores definidos sobre ese conjunto de valores y un tipo intrínseco asignado a los valores recién construidos.

 Expandir tabla

Variante	Literal
<i>Null</i>	<code>null</code>
<i>Lógico</i>	<code>true</code> <code>false</code>
<i>Number</i>	<code>0</code> <code>1</code> <code>-1</code> <code>1.5</code> <code>2.3e-5</code>
<i>Time</i>	<code>#time(09,15,00)</code>
<i>Date</i>	<code>#date(2013,02,26)</code>
<i>DateTime</i>	<code>#datetime(2013,02,26, 09,15,00)</code>
<i>DateTimeZone</i>	<code>#datetimezone(2013,02,26, 09,15,00, 09,00)</code>
<i>Duration</i>	<code>#duration(0,1,30,0)</code>
<i>Texto</i>	<code>"hello"</code>
<i>Binario</i>	<code>#binary("AQID")</code>
<i>Lista</i>	<code>{1, 2, 3}</code>
<i>Registro</i>	<code>[A = 1, B = 2]</code>
<i>Tabla</i>	<code>#table({"X","Y"},{{0,1},{1,0}})</code>
<i>Function</i>	<code>(x) => x + 1</code>
<i>Tipo</i>	<code>type { number }</code> <code>type table [A = any, B = text]</code>

En las secciones siguientes se describe con detalle cada tipo de valor. Los tipos y atribución de tipos se definen formalmente en [Tipos](#). Los valores de función se definen en [Funciones](#). En las secciones siguientes se enumeran los operadores definidos para cada tipo de valor y se proporcionan ejemplos. La definición completa de la semántica de los operadores continúa en [Operadores](#).

Null

Un *valor NULL* se usa para representar la ausencia de un valor, o bien un valor indeterminado o desconocido. Un valor NULL se escribe mediante el literal `null`. Los operadores siguientes se definen para los valores NULL:

 Expandir tabla

Operador	Resultado
<code>x > y</code>	Mayor que
<code>x >= y</code>	Mayor o igual que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x ?? y</code>	Fusionar

El tipo nativo del valor `null` es el tipo intrínseco `null`.

Lógico

Un *valor lógico* se usa para las operaciones booleanas y tiene el valor `true` o `false`. Un valor lógico se escribe mediante los literales `true` y `false`. Los operadores siguientes se definen para los valores lógicos:

 Expandir tabla

Operador	Resultado
<code>x > y</code>	Mayor que
<code>x >= y</code>	Mayor o igual que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual

Operador	Resultado
<code>x or y</code>	OR lógico condicional
<code>x ?? y</code>	Fusionar
<code>x and y</code>	AND lógico condicional
<code>not x</code>	NOT lógico

El tipo nativo de los dos valores lógicos (`true` y `false`) es el tipo intrínseco `logical`.

Número

Se usa un *valor número* para las operaciones numéricas y aritméticas. Los siguientes son ejemplos de literales de número:

Power Query M

```
3.14 // Fractional number
-1.5 // Fractional number
1.0e3 // Fractional number with exponent
123 // Whole number
1e3 // Whole number with exponent
0xff // Whole number in hex (255)
```

Un número se representa con al menos la precisión de un *doble* (pero puede conservar más precisión). La representación *doble* es congruente con el estándar de precisión doble de 64 bits de IEEE para la aritmética de punto flotante binaria definida en [IEEE 754-2008]. (La representación *doble* tiene un intervalo dinámico aproximado de $5,0 \times 10^{324}$ a $1,7 \times 10^{308}$ con una precisión de 15-16 dígitos).

Los valores especiales siguientes también se consideran valores *número*:

- Cero positivo y cero negativo. En la mayoría de los casos, cero positivo y cero negativo se comportan exactamente igual que el valor cero simple, pero [ciertas operaciones distinguen entre ambos](#).
- Infinito positivo (`#infinity`) e infinito negativo (`-#infinity`). Los infinitos se generan mediante operaciones como la división por cero de un número distinto de cero. Por ejemplo, `1.0 / 0.0` produce infinito positivo y `-1.0 / 0.0` produce infinito negativo.
- El valor *No es un número* (`#nan`), a menudo abreviado como NaN. Los Nan se generan mediante operaciones de punto flotante no válidas, como la división de cero por cero.

Las operaciones matemáticas binarias se realizan mediante una *precisión*. La precisión determina el dominio al que se redondean los operandos y el dominio en el que se realiza la operación. En ausencia de una precisión especificada de forma explícita, estas operaciones se realizan con *precisión doble*.

- Si el resultado de una operación matemática es demasiado pequeño para el formato de destino, el resultado de la operación se convierte en cero positivo o cero negativo.
- Si el resultado de una operación matemática es demasiado grande para el formato de destino, el resultado de la operación se convierte en infinito positivo o infinito negativo.
- Si una operación matemática no es válida, el resultado de la operación se convierte en NaN.
- Si uno o los dos operandos de una operación de punto flotante son NaN, el resultado de la operación se convierte en NaN.

Los operadores siguientes se definen para los valores numéricos:

 Expandir tabla

Operador	Resultado
<code>x > y</code>	Mayor que
<code>x >= y</code>	Mayor o igual que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x + y</code>	Suma
<code>x - y</code>	Diferencia
<code>x * y</code>	Producto
<code>x / y</code>	Cociente
<code>x ?? y</code>	Fusionar
<code>+x</code>	Suma unaria
<code>-x</code>	Negación

El tipo nativo de los valores numéricos es el tipo intrínseco `number`.

Tiempo

Un *valor de hora* almacena una representación opaca de la hora del día. Una hora se codifica como el número de *tics desde la medianoche*, que cuenta el número de tics de 100 nanosegundos que han transcurrido en un reloj de 24 horas. El número máximo de *tics desde la medianoche* corresponde a las 23:59:59.9999999 horas.

Aunque no hay ninguna sintaxis literal para las horas, se proporcionan varias funciones estándar de biblioteca para construirlas. Las horas también pueden construirse mediante la función intrínseca `#time`:

Power Query M

#time(hour, minute, second)

Se debe contener lo siguiente o se produce un error con el código de motivo

`Expression.Error`:

- 0 ≤ hora ≤ 24
- 0 ≤ minuto ≤ 59
- 0 ≤ segundo ≤ 59

Además, si hora = 24, el minuto y el segundo deben ser cero.

Los operadores siguientes se definen para los valores de hora:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

Los operadores siguientes permiten que uno o los dos operandos sean una fecha:

 Expandir tabla

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>time</code>	<code>duration</code>	Desplazamiento de fecha por duración
<code>x + y</code>	<code>duration</code>	<code>time</code>	Desplazamiento de fecha por duración
<code>x - y</code>	<code>time</code>	<code>duration</code>	Desplazamiento de fecha por duración negada
<code>x - y</code>	<code>time</code>	<code>time</code>	Duración entre fechas
<code>x & y</code>	<code>date</code>	<code>time</code>	Fecha y hora combinadas

El tipo nativo de los valores de hora es el tipo intrínseco `time`.

Fecha

Un *valor de fecha* almacena una representación opaca de un día concreto. Una fecha se codifica como un número de *días desde el comienzo de la era*, a partir del 1 de enero del año 1 de la Era Común en el calendario gregoriano. El número máximo de días transcurridos desde la época es 3 652 058, que se corresponde al 31 de diciembre de 9999.

Aunque no hay ninguna sintaxis literal para las fechas, se proporcionan varias funciones estándar de biblioteca para construirlas. Las fechas también pueden construirse mediante la función intrínseca `#date`:

Power Query M

#date(year, month, day)

Debe cumplirse lo siguiente o se generará un error con el código de motivo `Expression.Error`:

- $1 \leq \text{año} \leq 9999$
- $1 \leq \text{mes} \leq 12$
- $1 \leq \text{día} \leq 31$

Además, el día debe ser válido para el mes y año elegidos.

Los operadores siguientes se definen para los valores de fecha:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

Los operadores siguientes permiten que uno o los dos operandos sean una fecha:

 Expandir tabla

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>date</code>	<code>duration</code>	Desplazamiento de fecha por duración
<code>x + y</code>	<code>duration</code>	<code>date</code>	Desplazamiento de fecha por duración
<code>x - y</code>	<code>date</code>	<code>duration</code>	Desplazamiento de fecha por duración negada
<code>x - y</code>	<code>date</code>	<code>date</code>	Duración entre fechas
<code>x & y</code>	<code>date</code>	<code>time</code>	Fecha y hora combinadas

El tipo nativo de los valores de fecha es el tipo intrínseco `date`.

FechaHora

Un *valor de fecha y hora* contiene una fecha y una hora.

Aunque no hay ninguna sintaxis literal para los valores de fecha y hora, se proporcionan varias funciones estándar de biblioteca para construirlos. Los valores de fecha y hora también pueden construirse mediante la función intrínseca `#datetime`:

Power Query M

#datetime(year, month, day, hour, minute, second)

Se debe contener lo siguiente o se produce un error con el código de motivo Expression.Error:

$1 \leq \text{año} \leq 9999$

$1 \leq \text{mes} \leq 12$

$1 \leq \text{día} \leq 31$

$0 \leq \text{hora} \leq 23$

$0 \leq \text{minuto} \leq 59$

$0 \leq \text{segundo} \leq 59$

Además, el día debe ser válido para el mes y año elegidos.

Los operadores siguientes se definen para los valores de fecha y hora:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

Los operadores siguientes permiten que uno o los dos operandos sean un valor de fecha y hora:

 Expandir tabla

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Desplazamiento de fecha y hora por duración
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración negada
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Duración entre fechas y horas

El tipo nativo de los valores de fecha y hora es el tipo intrínseco `datetime`.

Fecha, hora y zona horaria

Un valor de *fecha, hora y zona horaria* contiene una fecha, una hora y una zona horaria. Un valor *zona horaria* se codifica como un número de *desplazamiento de minutos con respecto a la hora UTC*, que cuenta el número de minutos que la parte de hora del valor *fecha y hora* se debe desplazar con respecto a la hora universal coordinada (UTC). El número mínimo de *desplazamiento de minutos con respecto a la hora UTC* es -840, que representa un desplazamiento con respecto a la hora UTC de -14:00 o 14 horas antes que la hora UTC. El número máximo de *desplazamiento de minutos con respecto a la hora UTC* es 840, que corresponde a un desplazamiento con respecto a la hora UTC de 14:00.

Aunque no hay ninguna sintaxis literal para los valores de fecha, hora y zona horaria, se proporcionan varias funciones estándar de biblioteca para construirlos. Los valores de fecha, hora y zona horaria también pueden construirse mediante la función intrínseca `#datetimezone`:

Power Query M

```
#datetimezone(  
    year, month, day,  
    hour, minute, second,  
    offset-hours, offset-minutes)
```

Debe cumplirse lo siguiente, de lo contrario, se generará un error con el código de motivo `Expression.Error`.

$1 \leq \text{año} \leq 9999$

$1 \leq \text{mes} \leq 12$

$1 \leq \text{día} \leq 31$

$0 \leq \text{hora} \leq 23$

$0 \leq \text{minuto} \leq 59$

$0 \leq \text{segundo} \leq 59$

$-14 \leq \text{offset-hours} \leq 14$

$-59 \leq \text{offset-minutes} \leq 59$

Además, el día debe ser válido para el mes y año elegidos y, si `offset-hours = 14`, entonces `offset-minutes <= 0` y, si `offset-hours = -14`, entonces `offset-minutes >= 0`.

Los operadores siguientes se definen para los valores de fecha, hora y zona horaria:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

Los operadores siguientes permiten que uno o los dos operandos sean un valor de fecha, hora y zona horaria:

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetimezone</code>	<code>duration</code>	Desplazamiento de zona horaria de fecha según la duración
<code>x + y</code>	<code>duration</code>	<code>datetimezone</code>	Desplazamiento de zona horaria de fecha según la duración
<code>x - y</code>	<code>datetimezone</code>	<code>duration</code>	Desplazamiento de zona horaria de fecha según la duración negada
<code>x - y</code>	<code>datetimezone</code>	<code>datetimezone</code>	Duración entre zonas horarias de fecha

El tipo nativo de los valores de fecha, hora y zona horaria es el tipo intrínseco `datetimezone`.

Duración

Un *valor de duración* almacena una representación opaca de la distancia entre dos puntos en una escala de tiempo, medidos en tics de 100 segundos. La magnitud de una *duración* puede ser positiva o negativa; los valores positivos indican el progreso hacia delante en el tiempo y los negativos el progreso hacia atrás. El valor mínimo que se puede almacenar en una *duración* es -9 223 372 036 854 775 808 tics, o 10 675 199 días, 2 horas, 48 minutos y 05,4775808 segundos hacia atrás en el tiempo. El valor máximo que se puede almacenar en

una *duración* es 9 223 372 036 854 775 807 ticks, o 10 675 199 días, 2 horas, 48 minutos y 05,4775807 segundos hacia el futuro.

Aunque no hay ninguna sintaxis literal para las duraciones, se proporcionan varias funciones estándar de la biblioteca para construirlas. Las duraciones también pueden construirse mediante la función intrínseca `#duration`:

Power Query M

```
#duration(0, 0, 0, 5.5)           // 5.5 seconds
#duration(0, 0, 0, -5.5)          // -5.5 seconds
#duration(0, 0, 5, 30)            // 5.5 minutes
#duration(0, 0, 5, -30)           // 4.5 minutes
#duration(0, 24, 0, 0)            // 1 day
#duration(1, 0, 0, 0)             // 1 day
```

Los operadores siguientes se definen para los valores de duración:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

Además, los operadores siguientes permiten que uno o los dos operandos sean un valor de duración:

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Desplazamiento de fecha y hora por duración

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>duration</code>	<code>duration</code>	Suma de las duraciones
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración negada
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Duración entre fechas y horas
<code>x - y</code>	<code>duration</code>	<code>duration</code>	Diferencia de duraciones
<code>x * y</code>	<code>duration</code>	<code>number</code>	N veces una duración
<code>x * y</code>	<code>number</code>	<code>duration</code>	N veces una duración
<code>x / y</code>	<code>duration</code>	<code>number</code>	Fracción de una duración
<code>x / y</code>	<code>duration</code>	<code>duration</code>	Cociente numérico de duraciones

El tipo nativo de los valores de duración es el tipo intrínseco `duration`.

Texto

Un valor de *texto* representa una secuencia de caracteres Unicode. Los valores de texto tienen una forma literal compatible con la gramática siguiente:

_text-literal:

`" text-literal-charactersopt"`

text-literal-characters:

text-literal-character text-literal-characters_{opt}

text-literal-character:

single-text-character

character-escape-sequence

double-quote-escape-sequence

single-text-character:

Cualquier carácter excepto `"` (U+0022) o `#` (U+0023) seguido de `(` (U+0028)

double-quote-escape-sequence:

`""` (U+0022, U+0022)

A continuación se muestra un ejemplo de un valor de *texto*:

Power Query M
<code>"ABC" // the text value ABC</code>

Los operadores siguientes se definen para los valores de *texto*:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x & y</code>	Concatenación
<code>x ?? y</code>	Fusionar

El tipo nativo de los valores de texto es el tipo intrínseco `text`.

Binario

Un *valor binario* representa una secuencia de bytes.

Aunque no hay ninguna sintaxis literal para los valores binarios, se proporcionan varias funciones estándar de biblioteca para construirlos. Los valores binarios también pueden construirse mediante la función intrínseca `#binary`.

En el ejemplo siguiente se construye un valor binario a partir de una lista de bytes:

Power Query M

```
#binary( {0x00, 0x01, 0x02, 0x03} )
```

Los operadores siguientes se definen para los valores *binarios*:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual

Operador	Resultado
<code>x <> y</code>	No es igual
<code>x >= y</code>	Mayor o igual que
<code>x > y</code>	Mayor que
<code>x < y</code>	Menor que
<code>x <= y</code>	Menor o igual que
<code>x ?? y</code>	Fusionar

El tipo nativo de los valores binarios es el tipo intrínseco *binary*.

Lista

Un *valor de lista* es un valor que genera una secuencia de valores cuando se enumera. Un valor generado por una lista puede contener cualquier tipo de valor, incluida una lista. Las listas se pueden construir mediante la sintaxis de inicialización, como se indica a continuación:

list-expression:

`{ item-listopt }`

lista de ítems:

artículo

lista de artículos ,

artículo:

expresión

expresión .. *expresión*

En el ejemplo siguiente de *list-expression* se define una lista con tres valores de texto: `"A"`, `"B"` y `"C"`.

Power Query M

```
{ "A", "B", "C" }
```

El valor `"A"` es el primer elemento de la lista y el valor `"C"` es el último.

- Los elementos de una lista no se evalúan hasta que se accede a ellos.
- Aunque los valores de lista contruidos con la sintaxis de lista producirán elementos en el orden en que aparecen en *item-list*, en general, las listas devueltas por las funciones de

biblioteca pueden generar otro conjunto u otro número de valores cada vez que se enumeran.

Para incluir una secuencia de números enteros en una lista, se puede usar el formato `a..b`:

Power Query M

```
{ 1, 5..9, 11 }      // { 1, 5, 6, 7, 8, 9, 11 }
```

El número de elementos de una lista, conocido como *recuento de lista*, se puede determinar mediante la función `List.Count`.

Power Query M

```
List.Count({true, false}) // 2
List.Count({})            // 0
```

Una lista puede tener un número infinito de elementos; para estas listas, no se define `List.Count` y puede producir un error o no finalizar.

Si una lista no contiene ningún elemento, se denomina *lista vacía*. Una lista vacía se escribe de esta forma:

Power Query M

```
{ } // empty list
```

Los operadores siguientes se definen para las listas:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x & y</code>	Concatenar
<code>x ?? y</code>	Fusionar

Por ejemplo:

Power Query M

```
{1, 2} & {3, 4, 5}    // {1, 2, 3, 4, 5}
{1, 2} = {1, 2}        // true
{2, 1} <> {1, 2}      // true
```

El tipo nativo de los valores de lista es el tipo intrínseco `list`, que especifica un tipo de elemento de `any`.

Registro

Un *valor de registro* es una secuencia ordenada de campos. Un *campo* está formado por un *nombre de campo*, que es un valor de texto que identifica de forma única el campo dentro del registro, y por un *valor de campo*. El valor del campo puede ser cualquier tipo de valor, incluido el registro. Los registros se pueden construir mediante la sintaxis de inicialización, como se indica a continuación:

record-expression:

[*field-list*_{opt}]

lista de campos:

campo

campo, *lista de campos*

campo:

field-name = *expresión*

nombre de campo:

identificador-generalizado

quoted-identifier

En el ejemplo siguiente se crea un registro con un campo denominado `x` con el valor `1` y un campo denominado `y` con el valor `2`.

Power Query M

```
[ x = 1, y = 2 ]
```

En el ejemplo siguiente se crea un registro con un campo denominado `a` con un valor de registro anidado. El registro anidado tiene un campo denominado `b` con el valor `2`.

Power Query M

```
[ a = [ b = 2 ] ]
```

Cuando se evalúa una expresión de registro, sucede lo siguiente:

- Se usa la expresión asignada a cada nombre de campo para determinar el valor del campo asociado.
- Si la expresión asignada a un nombre de campo genera un valor cuando se evalúa, se convierte en el valor del campo del registro resultante.
- Si la expresión asignada a un nombre de campo genera un error cuando se evalúa, el hecho de que se haya producido un error se registra con el campo junto con el valor de error que se haya producido. El acceso posterior a ese campo hará que se vuelva a generar un error con el valor de error registrado.
- La expresión se evalúa en un entorno como el entorno primario solo con variables combinadas en que corresponden al valor de cada campo del registro, excepto el que se inicializa.
- Un valor de un registro no se evalúa hasta que se accede al campo correspondiente.
- Un valor de un registro se evalúa como máximo una vez.
- El resultado de la expresión es un valor de registro con un registro de metadatos vacío.
- El orden de los campos dentro del registro se define mediante el orden en que aparecen en *record-initializer-expression*.
- Cada nombre de campo que se especifica debe ser único dentro del registro o es un error. Los nombres se comparan mediante una comparación ordinal.

Power Query M

```
[ x = 1, x = 2 ] // error: field names must be unique
[ X = 1, x = 2 ] // OK
```

Un registro sin campos se denomina *registro vacío* y se escribe de esta manera:

Power Query M

```
[] // empty record
```

Aunque el orden de los campos de un registro no es significativo al acceder a un campo o comparar dos registros, es significativo en otros contextos, como cuando se enumeran los campos de un registro.

Los dos mismos registros producen resultados diferentes cuando se obtienen los campos:

Power Query M

```
Record.FieldNames([ x = 1, y = 2 ]) // [ "x", "y" ]
Record.FieldNames([ y = 1, x = 2 ]) // [ "y", "x" ]
```

El número de campos de un registro se puede determinar mediante la función

`Record.FieldCount`. Por ejemplo:

Power Query M

```
Record.FieldCount([ x = 1, y = 2 ]) // 2
Record.FieldCount([]) // 0
```

Además de usar la sintaxis de inicialización de registros `[ ]`, se pueden construir registros a partir de una lista de valores y una lista de nombres de campo o un tipo de registro. Por ejemplo:

Power Query M

```
Record.FromList({1, 2}, {"a", "b"})
```

Lo anterior equivale a:

Power Query M

```
[ a = 1, b = 2 ]
```

Los operadores siguientes se definen para los valores de registro:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x & y</code>	Combinar
<code>x ?? y</code>	Fusionar

En los ejemplos siguientes se ilustran los operadores anteriores. Tenga en cuenta que la combinación de registros usa los campos del operando derecho para invalidar los campos del operando izquierdo, si hay una superposición en los nombres de campo.

Power Query M

```
[ a = 1, b = 2 ] & [ c = 3 ]    // [ a = 1, b = 2, c = 3 ]
[ a = 1, b = 2 ] & [ a = 3 ]    // [ a = 3, b = 2 ]
[ a = 1, b = 2 ] = [ b = 2, a = 1 ]    // true
[ a = 1, b = 2, c = 3 ] <> [ a = 1, b = 2 ] // true
```

El tipo nativo de los valores de registro es el tipo intrínseco `record`, que especifica una lista vacía abierta de campos.

Tabla

Un *valor de tabla* es una secuencia ordenada de filas. Una *fila* es una secuencia ordenada de valores de columna. El tipo de la tabla determina la longitud de todas las filas de la tabla, los nombres y los tipos de las columnas de la tabla, y la estructura de las claves de la tabla (si existen).

Aunque no hay ninguna sintaxis literal para las tablas, se proporcionan varias funciones estándar de biblioteca para construirlas. Las tablas también pueden construirse mediante la función intrínseca `#table`.

En el ejemplo siguiente se construye una tabla a partir de una lista de nombres de columna y una lista de filas. La tabla resultante contendrá dos columnas de `type any` y tres filas.

Power Query M

```
#table({"x", "x^2"}, {{1,1}, {2,4}, {3,9}})
```

`#table` también se puede usar para especificar un tipo de tabla completo:

Power Query M

```
#table(
    type table [Digit = number, Name = text],
    {{1,"one"}, {2,"two"}, {3,"three"}}
)
```

Aquí, el nuevo valor de tabla tiene un tipo de tabla que especifica los nombres y los tipos de columna.

Los operadores siguientes se definen para los valores de tabla:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No es igual
<code>x & y</code>	Concatenación
<code>x ?? y</code>	Fusionar

La concatenación de tablas alinea las columnas con el mismo nombre y rellena con `null` las columnas que aparecen solo en una de las tablas involucradas. En el ejemplo siguiente se muestra la concatenación de tablas:

Power Query M

```
#table({"A","B"}, {{1,2}})
& #table({"B","C"}, {{3,4}})
```

 Expandir tabla

A	B	C
1	2	null
null	3	4

El tipo nativo de los valores de tabla es un tipo de tabla personalizado (derivado del tipo intrínseco `table`) que enumera los nombres de columna, especifica todos los tipos de columna como any y no tiene claves. (Vaya a [Tipos de tabla](#) para obtener más información sobre los tipos de tabla).

Función

Un *valor de función* es un valor que asigna un conjunto de argumentos a un valor único. Los detalles de los valores de *función* se describen en [Funciones](#).

Tipo

Un *valor de tipo* es un valor que clasifica otros valores. Los detalles de los valores de *tipo* se describen en [Tipos](#).

Tipos

Artículo • 29/01/2025

Un *valor de tipo* es un valor que *clasifica* otros valores. Se dice que un valor que está clasificado por un tipo *se ajusta* a ese tipo. El sistema de tipos de M consta de los tipos siguientes:

- Tipos primitivos, que clasifican valores primitivos (`binary`, `date`, `datetime`, `datetimezone`, `duration`, `list`, `logical`, `null`, `number`, `record`, `text`, `time`, `type`) e incluyen también una serie de tipos abstractos (`function`, `table`, `any`, `anynonnull` y `none`)
- Tipos de registro, que clasifican valores de registro en función de nombres de campo y tipos de valor
- Tipos de lista, que clasifican listas mediante un tipo base de un solo elemento
- Tipos de función, que clasifican valores de función según los tipos de sus parámetros y los valores devueltos
- Tipos de tabla, que clasifican valores de tabla en función de nombres de columna, tipos de columna y claves
- Tipos que aceptan valores NULL, que clasifican el valor NULL, además de todos los valores clasificados por un tipo base
- Tipos de tipo, que clasifican valores que son tipos

El conjunto de *tipos primitivos* incluye los tipos de valores primitivos, y un número de *tipos abstractos*, que son tipos que no clasifican de forma única los valores: `function`, `table`, `any`, `anynonnull` y `none`. Todos los valores de función se ajustan al tipo abstracto `function`, todos los valores de tabla al tipo abstracto `table`, todos los valores al tipo abstracto `any`, todos los valores que no son null al tipo abstracto `anynonnull` y ningún valor al tipo abstracto `none`. Una expresión de tipo `none` debe generar un error o no finalizar porque no se ha podido generar ningún valor que se ajuste al tipo `none`. Tenga en cuenta que los tipos primitivos `function` y `table` son abstractos porque ninguna función o tabla es directamente de esos tipos, respectivamente. Los tipos primitivos `record` y `list` no son abstractos porque representan un registro abierto sin campos definidos y una lista de tipo `any`, respectivamente.

Todos los tipos que no son miembros del conjunto cerrado de tipos primitivos más sus homólogos que aceptan valores null se denominan colectivamente *tipos personalizados*.

Los tipos personalizados se pueden escribir mediante `type-expression`:

type-expression:

primary-expression

`type` *primary-type*

primary-type:

primitive-or-nullable-primitive-type

record-type

list-type

function-type

table-type

nullable-type

primitive-or-nullable-primitive-type:

`nullable` *opt* *primitive-type*

primitive-type: uno de

`any anynonnull binary date datetime datetimezone duration function list logical
none null number record table text time type`

Los nombres de *primitive-type* son *palabras clave contextuales* que solo se reconocen en el contexto de un *tipo*. El uso de paréntesis en el contexto de un *tipo* devuelve la gramática al contexto de una expresión regular, lo que requiere el uso de la palabra clave `type` para volver al contexto de un tipo. Por ejemplo, para invocar una función en el contexto de un *tipo*, se pueden usar paréntesis:

Power Query M

```
type nullable ( Type.ForList({type number}) )  
// type nullable {number}
```

Los paréntesis también se pueden usar para acceder a una variable cuyo nombre entra en conflicto con un nombre de *tipo primitivo*:

Power Query M

```
let record = type [ A = any ] in type {(record)}  
// type {[ A = any ]}
```

En el ejemplo siguiente se define un tipo que clasifica una lista de números:

Power Query M

```
type { number }
```


Del mismo modo, en el ejemplo siguiente se define un tipo personalizado que clasifica los registros con los campos obligatorios `X` e `Y` cuyos valores son números:

Power Query M

```
type [ X = number, Y = number ]
```

El tipo descrito de un valor se obtiene mediante la función de biblioteca estándar `Value.Type`, como se muestra en los ejemplos siguientes:

Power Query M

```
Value.Type( 2 )           // type number
Value.Type( {2} )         // type list
Value.Type( [ X = 1, Y = 2 ] ) // type record
```

El operador `is` se usa para determinar si el tipo de un valor es compatible con un tipo determinado, como se muestra en los ejemplos siguientes:

Power Query M

```
1 is number      // true
1 is text        // false
{2} is list      // true
```

El operador `as` comprueba si el valor es compatible con el tipo especificado y genera un error si no lo es. De lo contrario, devuelve el valor original.

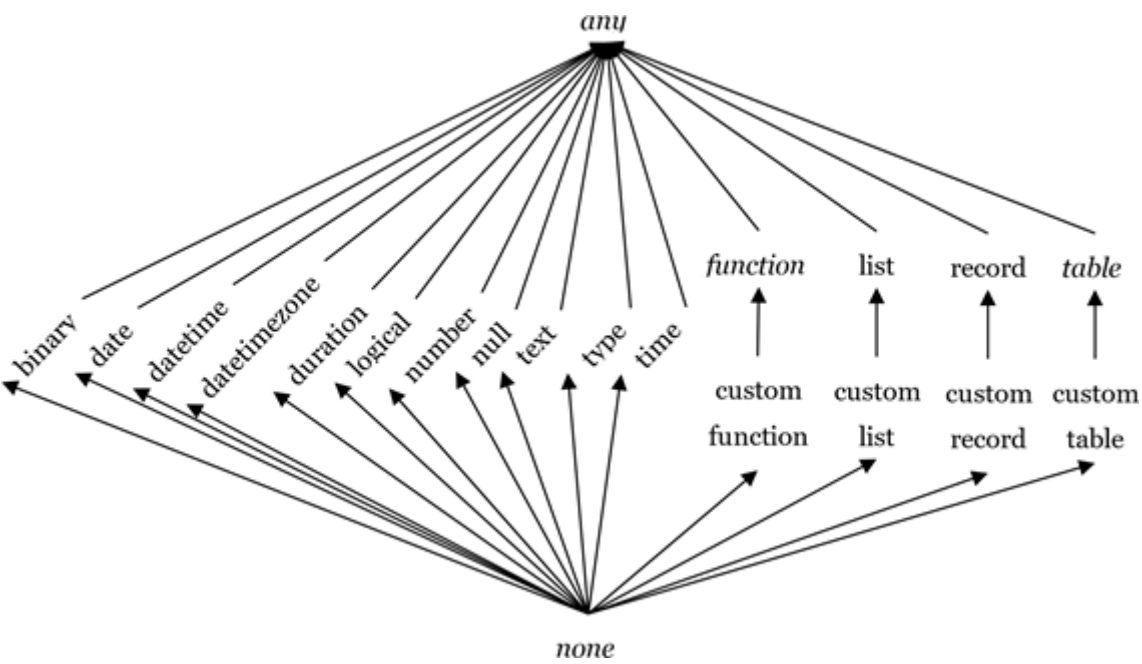
Power Query M

```
Value.Type( 1 as number ) // type number
{2} as text                // error, type mismatch
```

Tenga en cuenta que los operadores `is` y `as` solo aceptan un tipo primitivo o un tipo primitivo que acepta valores `NULL` (es decir, un tipo no personalizado) como operando derecho. En M no se proporcionan medios para comprobar la conformidad con tipos personalizados.

Un tipo `X` es compatible con un tipo `Y` solo si todos los valores que se ajustan a `X` también se ajustan a `Y`. Todos los tipos son compatibles con el tipo `any` y ningún tipo (menos el propio `none`) es compatible con el tipo `none`. En el gráfico siguiente se muestra la relación de compatibilidad. (La compatibilidad de tipos es reflexiva y

transitiva. Forma un enlace con el tipo `any` en la parte superior y el tipo `none` como valor inferior). Los nombres de los tipos abstractos se establecen en *cursiva*.



Los operadores siguientes se definen para los valores de tipo:

[Expandir tabla](#)

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No igual a
<code>x ?? y</code>	Coalesce

El tipo nativo de los valores de tipo es el tipo intrínseco `type`.

Tipos primitivos

Los tipos del lenguaje M forman una jerarquía no contigua cuya raíz es el tipo `any`, el que clasifica todos los valores. Cualquier valor de M se ajusta a exactamente un subtipo primitivo de `any`. A continuación se muestra el conjunto cerrado de tipos primitivos que derivan del tipo `any`:

- `type null`, que clasifica el valor NULL.
- `type logical`, que clasifica los valores true y false.
- `type number`, que clasifica valores numéricos.
- `type time`, que clasifica valores de hora.
- `type date`, que clasifica valores de fecha.

- `type datetime`, que clasifica valores de fecha y hora.
- `type datetimezone`, que clasifica valores de fecha, hora y zona horaria.
- `type duration`, que clasifica valores de duración.
- `type text`, que clasifica valores de texto.
- `type binary`, que clasifica valores binarios.
- `type type`, que clasifica valores de tipo.
- `type list`, que clasifica valores de lista.
- `type record`, que clasifica valores de registro.
- `type table`, que clasifica valores de tabla.
- `type function`, que clasifica valores de función.
- `type anynonnull`, que clasifica todos los valores menos NULL.
- `type none`, que no clasifica valores.

Tipo any

El tipo `any` es abstracto, clasifica todos los valores de M y todos los tipos de M son compatibles con `any`. Las variables de tipo `any` se pueden enlazar a todos los valores posibles. Como `any` es abstracto, no se puede atribuir a valores; es decir, ningún valor es directamente de tipo `any`.

Tipos de lista

Cualquier valor que sea una lista se ajusta al tipo intrínseco `list`, que no impone ninguna restricción sobre los elementos dentro de un valor de lista.

list-type:

```
{ item-type }
```

item-type:

```
type
```

El resultado de evaluar *list-type* es un *valor de tipo de lista* cuyo tipo base es `list`.

En los ejemplos siguientes se muestra la sintaxis para declarar tipos de lista homogéneos:

Power Query M

```
type { number }      // list of numbers type
    { record }       // list of records type
    {{ text }}       // list of lists of text values
```

Un valor se ajusta a un tipo de lista si el valor es una lista y cada elemento de ese valor de lista se ajusta al tipo de elemento del tipo de lista.

El tipo de elemento de un tipo de lista indica un enlace: todos los elementos de una lista conforme se ajustan al tipo de elemento.

Tipos de registro

Cualquier valor que sea un registro se ajusta al tipo intrínseco `record`, que no impone ninguna restricción sobre los nombres de campo ni los valores dentro de un valor de registro. Un *valor de tipo de registro* se usa para restringir el conjunto de nombres válidos, así como los tipos de valores que se pueden asociar a esos nombres.

record-type:

```
[ open-record-marker ]  
[ field-specification-listopt ]  
[ field-specification-list , open-record-marker ]
```

field-specification-list:

```
field-specification  
field-specification , field-specification-list
```

field-specification:

```
optionalopt field-name field-type-specificationopt
```

field-type-specification:

```
= field-type
```

field-type:

```
type
```

open-record-marker:

```
...
```

El resultado de evaluar *record-type* es un valor de tipo cuyo tipo base es `record`.

En los ejemplos siguientes se muestra la sintaxis para declarar tipos de registro:

Power Query M

```
type [ X = number, Y = number ]  
type [ Name = text, Age = number ]  
type [ Title = text, optional Description = text ]  
type [ Name = text, ... ]
```

De forma predeterminada, los tipos de registro son *cerrados*, lo que significa que no se permiten campos adicionales que no están presentes en *fieldspecification-list* en los

valores de conformidad. Al incluir *openrecord-marker* en el tipo de registro, se declara que el tipo es *abierto*, lo que permite campos no presentes en la lista de especificación de campos. Las dos expresiones siguientes son equivalentes:

Power Query M

```
type record    // primitive type classifying all records
type [ ... ]   // custom type classifying all records
```

Un valor se ajusta a un tipo de registro si el valor es un registro y se cumple cada especificación de campo en el tipo de registro. Se cumple una especificación de campo si se cumple alguna de las condiciones siguientes:

- Un nombre de campo que coincide con el identificador de la especificación existe en el registro y el valor asociado se ajusta al tipo de la especificación.
- La especificación está marcada como opcional y no se encuentra ningún nombre de campo correspondiente en el registro

Un valor compatible puede contener nombres de campo que no aparecen en la lista de especificación de campos si el tipo de registro está abierto.

Tipos de función

Cualquier valor de función se ajusta al tipo primitivo `function`, que no impone ninguna restricción en los tipos de los parámetros formales de la función ni en el valor que devuelve. Se usa un *valor de tipo de función* personalizado para imponer restricciones de tipo en las firmas de los valores de función compatibles.

tipo de función: `_`

`function ( parameter-specification-listopt ) return-type`

parameter-specification-list:

required-parameter-specification-list

required-parameter-specification-list , *optional-parameter-specification-list*

optional-parameter-specification-list

required-parameter-specification-list:

required-parameter-specification

required-parameter-specification , *required-parameter-specification-list*

required-parameter-specification:

parameter-specification

optional-parameter-specification-list:

optional-parameter-specification

optional-parameter-specification , *optional-parameter-specification-list*

optional-parameter-specification:

optional *parameter-specification*

parameter-specification:

parameter-name *parameter-type*

tipo de parámetro:

type-assertion

type-assertion:

as *type*

El resultado de evaluar *function-type* es un valor de tipo cuyo tipo base es *function*.

En los ejemplos siguientes se muestra la sintaxis para declarar tipos de función:

Power Query M

```
type function (x as text) as number
type function (y as number, optional z as text) as any
```

Un valor de función se ajusta a un tipo de función si el tipo del valor devuelto de función es compatible con el tipo de valor devuelto del tipo de función y cada especificación de parámetro del tipo de función es compatible con el parámetro formal correspondiente posicionalmente de la función. Una especificación de parámetro es compatible con un parámetro formal si el tipo *parameter-type* especificado es compatible con el tipo del parámetro formal y la especificación de parámetro es opcional si el parámetro formal es opcional.

Los nombres de parámetros formales se omiten con el fin de determinar la compatibilidad del tipo de función.

Especificar un parámetro como opcional implícitamente hace que su tipo acepte valores null. La acción siguiente crea tipos de función idénticos:

Power Query M

```
type function (optional x as text) as any
type function (optional x as nullable text) as any
```

Tipos de tablas

Un *valor de tipo de tabla* se usa para definir la estructura de un valor de tabla.

table-type:

`table` *row-type*

row-type:

[*field-specification-list*_{opt}]

El resultado de evaluar *table-type* es un valor de tipo cuyo tipo base es `table`.

El valor *row type* de una tabla especifica los nombres y los tipos de columna de la tabla como un tipo de registro cerrado. Para que todos los valores de tabla se ajusten al tipo `table`, su tipo de fila es `record` (el tipo de registro abierto vacío). Por tanto, el tipo de tabla es abstracto ya que ningún valor de tabla puede tener el tipo de fila de `table` del tipo (pero todos los valores de tabla tienen un tipo de fila que es compatible con el tipo de fila de `table` del tipo). En el ejemplo siguiente se muestra la construcción de un tipo de tabla:

Power Query M

```
type table [A = text, B = number, C = binary]
// a table type with three columns named A, B, and C
// of column types text, number, and binary, respectively
```

Un valor de tipo de tabla también incluye la definición de las *claves* de un valor de tabla. Una clave es un conjunto de nombres de columna. Como máximo, se puede designar una clave como *clave principal* de la tabla. (En M, las claves de tabla no tienen ningún significado semántico. Pero es habitual que los orígenes de datos externos, como las bases de datos o las fuentes de OData, definan claves antes que tablas. En Power Query se usa la información de clave para mejorar el rendimiento de las funciones avanzadas, como las operaciones de combinación entre orígenes).

Las funciones de la biblioteca estándar `Type.TableKeys`, `Type.AddTableKey` y `Type.ReplaceTableKeys` se pueden usar para obtener las claves de un tipo de tabla, agregar una clave a un tipo de tabla y reemplazar todas las claves de un tipo de tabla, respectivamente.

Power Query M

```
Type.AddTableKey(tableType, {"A", "B"}, false)
// add a non-primary key that combines values from columns A and B
Type.ReplaceTableKeys(tableType, {})
// returns type value with all keys removed
```

Tipos que aceptan valores NULL

Para cualquier `type T`, se puede derivar una variante que acepta valores NULL mediante *nullable-type*:

nullable-type:

```
nullable type
```

El resultado es un tipo abstracto que permite valores de tipo `T` o el valor `null`.

Power Query M

```
42 is nullable number      // true null is
nullable number           // true
```

La atribución de `type nullable T` se reduce a la de `type null` o `type T`. (Recuerde que los tipos que aceptan valores NULL son abstractos y que ningún valor puede ser directamente de tipo abstracto).

Power Query M

```
Value.Type(42 as nullable number)    // type number
Value.Type(null as nullable number)   // type null
```

Las funciones de la biblioteca estándar `Type.IsNullable` y `Type.NonNullable` se pueden usar para probar la nulabilidad de un tipo y para quitarla de un tipo.

Se aplica lo siguiente (para cualquier `type T`):

- `type T` es compatible con `type nullable T`
- `Type.NonNullable(type T)` es compatible con `type T`

Los elementos siguientes son equivalentes en pares (para cualquier `type T`):

```
type nullable any
```

```
any
```

```
Type.NonNullable(type any)
```

```
type anynonnull
```

```
type nullable none
```

```
type null
```

```
Type.NonNullable(type null)
```

```
type none
```



```
type nullable nullable T
```

```
type nullable T
```

```
Type.NonNullable(Type.NonNullable(type T))
```

```
Type.NonNullable(type T)
```

```
Type.NonNullable(type nullable T)
```

```
Type.NonNullable(type T)
```

```
type nullable (Type.NonNullable(type T))
```

```
type nullable T
```

Tipo atribuido de un valor

El *tipo atribuido* de un valor es el tipo al que se *declara* que el valor se ajusta.

Un valor se puede atribuir a un tipo mediante la función de biblioteca

`Value.ReplaceType`. Esta función devuelve un nuevo valor con el tipo atribuido o genera un error si el nuevo tipo es incompatible con el valor.

Cuando un valor se atribuye a un tipo, solo se produce una comprobación de compatibilidad limitada:

- El tipo que se describe debe ser no abstracto, que no acepta valores NULL y compatible con el *tipo primitivo* intrínseco (nativo) del valor.
- Cuando se describe un tipo personalizado que define la estructura, debe coincidir con la estructura del valor.
 - Para los registros: el tipo debe cerrarse y definir el mismo número de campos que el valor, y no debe contener ningún campo opcional. (Los nombres de los campos y los tipos de campo del tipo reemplazarán los que actualmente están asociados con el registro. Sin embargo, los valores de campo existentes no se comprobarán con los nuevos tipos de campo).
 - Para las tablas: el tipo debe definir el mismo número de columnas que el valor. (Los nombres de las columnas y los tipos de las columnas del tipo reemplazarán a los que están actualmente asociados con la tabla. Sin embargo, los valores de columna existentes no se comprobarán con los nuevos tipos de columna).
 - Para las funciones: el tipo debe definir el mismo número de parámetros necesarios, así como el mismo número de parámetros opcionales, que el valor. (El parámetro del tipo y las aserciones de devolución, así como sus nombres de parámetro, reemplazarán los asociados con el tipo actual del valor de la función. Sin embargo, las nuevas aserciones no tendrán ningún efecto en el comportamiento real de la función).

- Para las listas: el valor debe ser una lista. (Pero los elementos de lista existentes no se comprobarán con el nuevo tipo de elemento).

Las funciones de biblioteca pueden optar por calcular y atribuir tipos complejos a los resultados en función de los tipos atribuidos de los valores de entrada.

El tipo atribuido de un valor se puede obtener mediante la función de biblioteca

`Value.Type`. Por ejemplo:

Power Query M

```
Value.Type( Value.ReplaceType( {1}, type {number} )  
// type {number}
```

Equivalencia y compatibilidad de tipos

La equivalencia de tipos no está definida en M. Opcionalmente, una implementación M puede optar por usar sus propias reglas para realizar comparaciones de igualdad entre valores de tipo. La comparación de la igualdad de dos valores de tipo debe evaluarse como `true` si la implementación los considera idénticos o, en caso contrario, como `false`. En cualquier caso, la respuesta devuelta debe ser coherente si se comparan dos valores iguales de forma repetida. Tenga en cuenta que, dentro de una implementación determinada, comparar algunos valores de tipo idénticos (como `(type text) = (type text)`) puede devolver `true`, pero no si se comparan otros valores (como `(type [a = text]) = (type [a = text])`).

La compatibilidad entre un tipo determinado y un tipo primitivo o un tipo primitivo que acepta valores NULL se puede determinar mediante la función de biblioteca `Type.Is`, que acepta un valor de tipo arbitrario como primer argumento y un valor primitivo o de tipo primitivo que acepta valores NULL como segundo argumento:

Power Query M

```
Type.Is(type text, type nullable text) // true  
Type.Is(type nullable text, type text) // false  
Type.Is(type number, type text)       // false  
Type.Is(type [a=any], type record)    // true  
Type.Is(type [a=any], type list)      // false
```

En M no hay ninguna forma de determinar la compatibilidad de un tipo determinado con un tipo personalizado.

La biblioteca estándar incluye una colección de funciones para extraer las características de definición de un tipo personalizado, de modo que se pueden implementar pruebas de compatibilidad específicas como expresiones de M. A continuación se muestran algunos ejemplos; consulte la especificación de la biblioteca de M para obtener todos los detalles.

Power Query M

```
Type.ListItem( type {number} )
// type number
Type.NonNullable( type nullable text )
// type text
Type.RecordFields( type [A=text, B=time] )
// [ A = [Type = type text, Optional = false],
//   B = [Type = type time, Optional = false] ]
Type.TableRow( type table [X=number, Y=date] )
// type [X = number, Y = date]
Type.FunctionParameters(
    type function (x as number, optional y as text) as number)
// [ x = type number, y = type nullable text ]
Type.FunctionRequiredParameters(
    type function (x as number, optional y as text) as number)
// 1
Type.FunctionReturn(
    type function (x as number, optional y as text) as number)
// type number
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)
- [Tipos del lenguaje de fórmulas M de Power Query](#)

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Comportamiento del operador

En esta sección se define el comportamiento de los distintos operadores de M.

Prioridad de los operadores

Cuando una expresión contiene varios operadores, su *precedencia* controla el orden en el que se evalúan los operadores individuales. Por ejemplo, la expresión `x + y * z` se evalúa como `x + (y * z)` porque el operador `*` tiene mayor precedencia que el operador binario `+`. La precedencia de un operador se establece mediante la definición de su producción gramatical asociada. Por ejemplo, una instancia de *additive-expression* consta de una secuencia de *multiplicative-expression* separadas por los operadores `+` o `-`, por lo que se asigna a los operadores `+` y `-` una menor precedencia que la de los operadores `*` y `/`.

La producción *parenthesized-expression* se puede usar para cambiar el orden de precedencia predeterminado.

parenthesized-expression:

`( expresión )`

Por ejemplo:

Power Query M

1 + 2 * 3 // 7
(1 + 2) * 3 // 9

En la tabla siguiente se resumen los operadores de M y se enumeran las categorías de operador en orden de prioridad, de mayor a menor. Los operadores de la misma categoría tienen la misma precedencia.

[Expandir tabla](#)

Categoría	Expresión	Descripción
Principal	<i>i</i> @ Yo	Expresión de identificador
	(x)	Expresión entre paréntesis
	x[l]	Lookup
	x{y}	Acceso a elementos
	x(...)	Invocación de funciones

	$\{x, y, \dots\}$	Inicialización de listas
	$[i = x, \dots]$	Inicialización de registros
	...	No implementado
Unario	$+ x$	Identidad
	$- x$	Negación
	<code>not</code> x	Negación lógica
Metadatos	$x_{\text{meta}} y$	Asociación de metadatos
Multiplicativa	$x * y$	Multiplicación
	x / y	División
Aditiva	$x + y$	Suma
	$x - y$	Resta
Relacional	$x < y$	Menor que
	$x > y$	Mayor que
	$x \leq y$	Menor o igual que
	$x \geq y$	Mayor o igual que
Igualdad	$x = y$	Igual
	$x \neq y$	No igual a
Aserción de tipos	$x \text{ as } y$	Es un tipo primitivo o un error primitivos que aceptan valores NULL compatibles
Conformidad de tipos	$x \text{ is } y$	Prueba si el tipo es compatible con el tipo primitivo o el tipo primitivo que acepta valores NULL
Y lógico	$x \text{ and } y$	Conjunción de cortocircuito
O lógico	$x \text{ or } y$	Disyunción de cortocircuito
Coalesce	$x ?? y$	Operador de fusión de NULL

Operadores y metadatos

Cada valor tiene un valor de registro asociado que puede contener información adicional sobre el valor. Este registro se conoce como el *registro de metadatos* de un valor. Un registro de

metadatos se puede asociar a cualquier tipo de valor, incluso `null`. El resultado de este tipo de asociación es un nuevo valor con los metadatos especificados.

Un registro de metadatos es simplemente un registro normal y puede contener todos los campos y valores de un registro normal, y a su vez puede ser un registro de metadatos. La asociación de un registro de metadatos con un valor es "no intrusiva". No cambia el comportamiento del valor en las evaluaciones, excepto el de las que inspeccionan registros de metadatos de forma explícita.

Cada valor tiene un registro de metadatos predeterminado, incluso si no se ha especificado ninguno. El registro de metadatos predeterminado está vacío. En los ejemplos siguientes se muestra el acceso al registro de metadatos de un valor de texto mediante la función de la biblioteca estándar `Value.Metadata`:

Power Query M

```
Value.Metadata( "Mozart" ) // []
```

Generalmente, los registros de metadatos *no se conservan* cuando un valor se usa con un operador o una función que crea un valor. Por ejemplo, si se concatenan dos valores de texto con el operador `&`, los metadatos del valor de texto resultante es el registro vacío `[]`. Las expresiones siguientes son equivalentes:

Power Query M

```
"Amadeus " & ("Mozart" meta [ Rating = 5 ])
"Amadeus " & "Mozart"
```

Se pueden usar las funciones de biblioteca estándar `Value.RemoveMetadata` y `Value.ReplaceMetadata` para quitar todos los metadatos de un valor y para reemplazarlos (en lugar de combinarlos en metadatos que ya existan).

El único operador que devuelve resultados que contienen metadatos es el [operador meta](#).

Operadores recursivos estructuralmente

Los valores pueden ser *cíclicos*. Por ejemplo:

Power Query M

```
let l = {0, @l} in l
// {0, {0, {0, ... }}}
[A={B}, B={A}]
// [A = {{ ... }}, B = {{ ... }}]
```

Para controlar los valores cíclicos en M, la construcción de registros, listas y tablas es diferida. Un intento de construir un valor cíclico que no se beneficie de los valores estructurados intercalados en diferido produce un error:

Power Query M

```
[A=B, B=A]
// [A = Error.Record("Expression.Error",
//      "A cyclic reference was encountered during evaluation"),
//   B = Error.Record("Expression.Error",
//      "A cyclic reference was encountered during evaluation"),
// ]
```

Algunos operadores de M se definen mediante la recursión estructural. Por ejemplo, la igualdad de registros y listas se define mediante la igualdad conjunta de los campos de registro y listas de elementos correspondientes, respectivamente.

Para los valores que no son cíclicos, la aplicación de recursión estructural genera una *expansión finita* del valor: los valores anidados compartidos se recorren repetidamente, pero el proceso de recursión siempre finaliza.

Un valor cíclico tiene una *expansión infinita* cuando se aplica la recursión estructural. En la semántica de M no hay disposiciones especiales para estas expansiones infinitas; un intento de comparar la igualdad de los valores cíclicos, por ejemplo, se quedará sin recursos y finalizará de forma excepcional.

Operadores de selección y proyección

Los operadores de selección y proyección permiten extraer datos de valores de lista y de registro.

Acceso a elementos

Se puede seleccionar un valor de una lista o una tabla en función de su posición de base cero dentro de esa lista o tabla mediante *item-access-expression*.

item-access-expression:

item-selection

optional-item-selection

item-selection:

primary-expression { *item-selector* }

optional-item-selection:

primary-expression { *item-selector* } ?

selector de elementos:

expresión

item-access-expression `x{y}` devuelve lo siguiente:

- En el caso de una lista `x` y un número `y`, el elemento de lista `x` en la posición `y`. El primer elemento de una lista se considera que tiene un índice ordinal de cero. Si la posición solicitada no existe en la lista, se produce un error.
- En el caso de una tabla `x` y un número `y`, la fila de la tabla `x` en la posición `y`. La primera fila de una tabla se considera que tiene un índice ordinal de cero. Si la posición solicitada no existe en la tabla, se produce un error.
- En el caso de una tabla `x` y un registro `y`, la fila de la tabla `x` que coincide con los valores de campo de registro `y` para los campos con nombres de campo que coinciden con los nombres de columna de tabla correspondientes. Si no hay ninguna fila coincidente única en la tabla, se produce un error.

Por ejemplo:

Power Query M

```
{ "a", "b", "c" } { 0 } // "a"
{ 1, [A=2], 3 } { 1 } // [A=2]
{ true, false } { 2 } // error
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { 0 } // [A=0, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [A=2] } // [A=2, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=3] } // error
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=1] } // error
```

item-access-expression también admite el formato `x{y}?`, que devuelve `null` cuando la posición (o coincidencia) `y` no existe en la lista o la tabla `x`. Si hay varias coincidencias para `y`, se producirá un error.

Por ejemplo:

Power Query M

```
{ "a", "b", "c" } { 0 } ? // "a"
{ 1, [A=2], 3 } { 1 } ? // [A=2]
{ true, false } { 2 } ? // null
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { 0 } ? // [A=0, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [A=2] } ? // [A=2, B=1]
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=3] } ? // null
#table({ "A", "B" }, { { 0, 1 }, { 2, 1 } }) { [B=1] } ? // error
```


El acceso a los elementos no fuerza la evaluación de elementos de lista o tabla distintos del elemento al que se accede. Por ejemplo:

Power Query M

```
{ error "a", 1, error "c" }{1} // 1  
{ error "a", error "b" }{1} // error "b"
```

Cuando se evalúa el operador de acceso a elementos `x{y}`, sucede lo siguiente:

- Se propagan los errores generados durante la evaluación de expresiones `x` o `y`.
- La expresión `x` genera un valor de lista o tabla.
- La expresión `y` genera un valor numérico o, si `x` genera un valor de tabla, un valor de registro.
- Si `y` genera un valor numérico y el valor de `y` es negativo, se genera un error con el código de motivo `"Expression.Error"`.
- Si `y` genera un valor numérico y el valor de `y` es mayor o igual que el recuento de `x`, se genera un error con el código de motivo `"Expression.Error"`, a menos que se use el formato de operador opcional `x{y}?`, en cuyo caso se devuelve el valor `null`.
- Si `x` genera un valor de tabla y `y` genera un valor de registro, y no hay coincidencias con `y` en `x`, se genera un error con el código de motivo `"Expression.Error"`, a menos que se use el formato de operador opcional `x{y}?`, en cuyo caso se devuelve el valor `null`.
- Si `x` genera un valor de tabla y `y` genera un valor de registro, y hay varias coincidencias con `y` en `x`, se genera un error con el código de motivo `"Expression.Error"`.

No se evalúa ningún elemento en `x` que no sea el que se encuentra en la posición `y` durante el proceso de selección del elemento. (Para las listas o tablas de streaming, se omiten los elementos o las filas anteriores a la posición `y`, lo que puede provocar su evaluación, en función del origen de la lista o de la tabla).

Acceso a campos

field-access-expression se usa para *seleccionar* un valor de un registro o para *proyectar* un registro o una tabla en otro registro u otra tabla uno con menos campos o columnas, respectivamente.

field-access-expression:

field-selection

implicit-target-field-selection

proyección

implicit-target-projection

field-selection:

primary-expression field-selector

field-selector:

required-field-selector

optional-field-selector

required-field-selector:

[*field-name*]

optional-field-selector:

[*field-name*] ?

field-name:

generalized-identifier

quoted-identifier

implicit-target-field-selection:

field-selector

projection:

primary-expression required-projection

primary-expression optional-projection

required-projection:

[*required-selector-list*]

optional-projection:

[*required-selector-list*] ?

required-selector-list:

required-field-selector

required-selector-list , *required-field-selector*

implicit-target-projection:

required-projection

proyección opcional

La forma más sencilla de acceso a campos es la *selección de campos obligatorios*. Usa el operador `x[y]` para buscar un campo en un registro por nombre de campo. Si un campo `y` no existe en `x`, se produce un error. El formato `x[y]?` se usa para realizar la selección de campos *opcionales* y devuelve `null` si el campo solicitado no existe en el registro.

Por ejemplo:

```
[A=1,B=2][B]           // 2
[A=1,B=2][C]           // error
[A=1,B=2][C]?          // null
```

El acceso colectivo de varios campos es compatible con los operadores para la *proyección de registros obligatorios* y la *proyección de registros opcionales*. El operador `x[[y1],[y2],...]` proyecta el registro en un registro nuevo con menos campos (seleccionados por `y1`, `y2`, `...`). Si un campo seleccionado no existe, se produce un error. El operador `x[[y1],[y2],...]?` proyecta el registro en un registro nuevo con los campos seleccionados por `y1`, `y2`, `...`; si falta un campo, se usa `null` en su lugar. Por ejemplo:

Power Query M

```
[A=1,B=2][[B]]          // [B=2]
[A=1,B=2][[C]]          // error
[A=1,B=2][[B],[C]]?     // [B=2,C=null]
```

Se admiten los formatos `[y]` y `[y]?` como una referencia *abreviada* al identificador `_` (carácter de subrayado). Las dos expresiones siguientes son equivalentes:

Power Query M

```
[A]
_[A]
```

En el ejemplo siguiente se muestra el formato abreviado del acceso a campos:

Power Query M

```
let _ = [A=1,B=2] in [A] //1
```

También se admiten los formatos `[[y1],[y2],...]` y `[[y1],[y2],...]?` como abreviatura y las dos expresiones siguientes también son equivalentes:

Power Query M

```
[[A],[B]]
_[[A],[B]]
```

El formato abreviado es especialmente útil si se combina con la abreviatura `each`, una manera de introducir una función de un único parámetro denominado `_` (para obtener más información, vea [Declaraciones simplificadas](#)). Juntas, las dos abreviaturas simplifican las expresiones funcionales de orden superior comunes:

Power Query M

```
List.Select( {[a=1, b=1], [a=2, b=4]}, each [a] = [b])  
// {[a=1, b=1]}
```

La expresión anterior es equivalente a la siguiente, más extensa y de aspecto críptico:

Power Query M

```
List.Select( {[a=1, b=1], [a=2, b=4]}, (_ ) => _[a] = _[b])  
// {[a=1, b=1]}
```

El acceso a campos no fuerza la evaluación de otros distintos al que se accede. Por ejemplo:

Power Query M

```
[A=error "a", B=1, C=error "c"] [B] // 1  
[A=error "a", B=error "b"] [B] // error "b"
```

Cuando se evalúa un operador de acceso a campos `x[y]`, `x[y]?`, `x[[y]]` o `x[[y]]?`, sucede lo siguiente:

- Se propagan los errores generados durante la evaluación de la expresión `x`.
- Los errores que se producen al evaluar el campo `y` están asociados permanentemente al campo `y` y luego se propagan. Cualquier acceso futuro al campo `y` producirá el mismo error.
- La expresión `x` genera un valor de registro o tabla, o bien se produce un error.
- Si el identificador `y` menciona a un campo que no existe en `x`, se genera un error con el código de motivo `"Expression.Error"`, a menos que se use el formato de operador opcional `...?`, en cuyo caso se devuelve el valor `null`.

En el proceso de acceso a campos, no se evalúa ningún campo de `x` que no sea el que menciona `y`.

Operador de metadatos

El registro de metadatos de un valor se modifica mediante el *operador meta* (`x meta y`).

metadata-expression:

unary-expression

unary-expression meta unary-expression

En el ejemplo siguiente se crea un valor de texto con un registro de metadatos mediante el operador `meta` y, después, se accede al registro de metadatos del valor resultante mediante `Value.Metadata`:

Power Query M

```
Value.Metadata( "Mozart" meta [ Rating = 5 ] )  
// [Rating = 5 ]  
Value.Metadata( "Mozart" meta [ Rating = 5 ] )[Rating]  
// 5
```

Cuando se aplica el operador de combinación de metadatos `x meta y`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- La expresión `y` debe ser un registro, o bien se produce un error con el código de motivo `"Expression.Error"`.
- El registro de metadatos resultante es el de `x` combinado con `y`. (Para obtener la semántica de la combinación de registros, vea [Combinación de registros](#)).
- El valor resultante es el de la expresión `x`, sin sus metadatos, con el registro de metadatos recién calculado adjunto.

Se pueden usar las funciones de biblioteca estándar `Value.RemoveMetadata` y `Value.ReplaceMetadata` para quitar todos los metadatos de un valor y para reemplazarlos (en lugar de combinarlos en metadatos que ya existan). Las expresiones siguientes son equivalentes:

Power Query M

```
x meta y  
Value.ReplaceMetadata(x, Value.Metadata(x) & y)  
Value.RemoveMetadata(x) meta (Value.Metadata(x) & y)
```

Operadores de igualdad

El *operador de igualdad* `=` se usa para determinar si dos valores son iguales. El *operador de desigualdad* `<>` se usa para determinar si dos valores no son iguales.

equality-expression:

relational-expression

relational-expression `=` *equality-expression*

relational-expression `<>` *equality-expression*

Por ejemplo:

Power Query M

```
1 = 1           // true
1 = 2           // false
1 <> 1          // false
1 <> 2          // true
null = true     // false
null = null     // true
```

Los metadatos no forman parte de la comparación de igualdad o desigualdad. Por ejemplo:

Power Query M

```
(1 meta [ a = 1 ]) = (1 meta [ a = 2 ]) // true
(1 meta [ a = 1 ]) = 1                  // true
```

Cuando se aplican los operadores de igualdad `x = y` y `x <> y`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- El operador `=` tiene un resultado de `true` si los valores son iguales y de `false` en caso contrario.
- El operador `<>` tiene un resultado de `false` si los valores son iguales y de `true` en caso contrario.
- Los registros de metadatos no se incluyen en la comparación.
- Si los valores generados al evaluar las expresiones `x` e `y` no tienen el mismo tipo de valor, los valores no son iguales.
- Si los valores generados al evaluar las expresiones `x` e `y` son el mismo tipo de valor, hay reglas específicas para determinar si son iguales, como se define a continuación.
- Lo siguiente se cumple siempre:

Power Query M

```
(x = y) = not (x <> y)
```

Los operadores de igualdad se definen para los tipos siguientes:

- El valor `null` solo es igual a sí mismo.

Power Query M

```
null = null    // true
null = true    // false
null = false   // false
```

- Los valores lógicos `true` y `false` solo son iguales a sí mismos. Por ejemplo:

Power Query M

```
true = true    // true
false = false   // true
true = false    // false
true = 1        // false
```

- Los números se comparan con la precisión especificada:
 - Si uno es `#nan`, los números no son los mismos.
 - Si ninguno es `#nan`, los números se comparan mediante una comparación de bits del valor numérico.
 - `#nan` es el único valor que no es igual a sí mismo.

Por ejemplo:

Power Query M

```
1 = 1,          // true
1.0 = 1         // true
2 = 1           // false
#nan = #nan     // false
#nan <> #nan     // true
```

- Dos duraciones son iguales si representan el mismo número de tics de 100 nanosegundos.
- Dos horas son iguales si las magnitudes de sus partes (hora, minuto, segundo) son iguales.
- Dos fechas son iguales si las magnitudes de sus partes (año, mes, día) son iguales.
- Dos valores de fecha y hora son iguales si las magnitudes de sus partes (año, mes, día, hora, minuto, segundo) son iguales.
- Dos valores de fecha, hora y zona horaria son iguales si los valores de fecha y hora UTC correspondientes son iguales. Para llegar a la fecha y hora UTC correspondiente, el

desplazamiento de horas/minutos se resta del componente de fecha y hora del valor de fecha, hora y zona horaria.

- Dos valores de texto son iguales si al usar una comparación ordinal, con distinción de mayúsculas y minúsculas y sin distinción de referencia cultural tienen la misma longitud y caracteres iguales en las posiciones correspondientes.
- Dos valores de lista son iguales si se cumple todo lo siguiente:
 - Las dos listas contienen el mismo número de elementos.
 - Los valores de cada elemento posicionalmente correspondiente en las listas son iguales. Esto significa que las listas no solo deben contener elementos iguales, sino que deben estar en el mismo orden.

Por ejemplo:

Power Query M

```
{1, 2} = {1, 2}      // true  
{2, 1} = {1, 2}      // false  
{1, 2, 3} = {1, 2}  // false
```

- Dos registros son iguales si se cumple todo lo siguiente:
 - El número de campos es el mismo.
 - Cada nombre de campo de un registro también está presente en el otro registro.
 - El valor de cada campo de un registro es igual al campo con el mismo nombre del otro registro.

Por ejemplo:

Power Query M

```
[ A = 1, B = 2 ] = [ A = 1, B = 2 ]      // true  
[ B = 2, A = 1 ] = [ A = 1, B = 2 ]      // true  
[ A = 1, B = 2, C = 3 ] = [ A = 1, B = 2 ] // false  
[ A = 1 ] = [ A = 1, B = 2 ]              // false
```

- Dos tablas son iguales si se cumple todo lo siguiente:
 - El número de columnas es el mismo.
 - Cada nombre de columna de una tabla también está presente en la otra.
 - El número de filas es el mismo.

- Cada fila tiene los mismos valores en las celdas correspondientes.

Por ejemplo:

Power Query M

```
#table({"A","B"},{{1,2}}) = #table({"A","B"},{{1,2}}) // true
#table({"A","B"},{{1,2}}) = #table({"X","Y"},{{1,2}}) // false
#table({"A","B"},{{1,2}}) = #table({"B","A"},{{2,1}}) // true
```

- Un valor de función es igual a sí mismo, pero puede o no ser igual a otro valor de función. Si dos valores de función se consideran iguales, se comportarán de la misma manera cuando se invocan.

Dos valores de función dados siempre tendrán la misma relación de igualdad.

- Un valor de tipo es igual a sí mismo, pero puede o no ser igual a otro valor de tipo. Si dos valores de tipo se consideran iguales, se comportarán de la misma manera cuando se consulta su compatibilidad.

Dos valores de tipo dados siempre tendrán la misma relación de igualdad.

Operadores relacionales

Los operadores `<`, `>`, `<=` y `>=` se denominan *operadores relacionales*.

relational-expression:

additive-expression

additive-expression `<` *relational-expression*

additive-expression `>` *relational-expression*

additive-expression `<=` *relational-expression*

additive-expression `>=` *relational-expression*

Estos operadores se usan para determinar la relación de ordenación relativa entre dos valores, como se muestra en la tabla siguiente:

[Expandir tabla](#)

Operación	Resultado
<code>x < y</code>	<code>true</code> si <code>x</code> es menor que <code>y</code> , <code>false</code> en caso contrario
<code>x > y</code>	<code>true</code> si <code>x</code> es mayor que <code>y</code> , <code>false</code> en caso contrario
<code>x <= y</code>	<code>true</code> si <code>x</code> es menor o igual que <code>y</code> , <code>false</code> en caso contrario

Operación	Resultado
<code>x >= y</code>	<code>true</code> si <code>x</code> es mayor o igual que <code>y</code> , <code>false</code> en caso contrario

Por ejemplo:

Power Query M

```
0 <= 1           // true
null < 1         // null
null <= null     // null
"ab" < "abc"     // true
#nan >= #nan     // false
#nan <= #nan     // false
```

Cuando se evalúa una expresión que contiene los operadores relacionales, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones de operando `x` o `y`.
- Los valores generados al evaluar las expresiones `x` e `y` deben ser de binario, fecha, fecha y hora, fecha, hora y zona horaria, duración, lógicos, null, texto o de hora. De lo contrario, se genera un error con el código de motivo `"Expression.Error"`.
- Ambos operandos deben ser el mismo tipo de valor o `null`. De lo contrario, se genera un error con el código de motivo `"Expression.Error"`.
- Si uno o los dos operandos son `null`, el resultado es el valor `null`.
- Dos archivos binarios se comparan byte por byte.
- Para comparar dos fechas se comparan sus partes de año y, si son iguales, sus partes de mes y, si son iguales, sus partes de día.
- Para comparar dos valores de fecha y hora se comparan sus partes de año y, si son iguales, sus partes de mes y, si son iguales, sus partes de día y, si son iguales, sus partes de hora y, si son iguales, sus partes de minuto y, si son iguales, sus partes de segundo.
- Para comparar dos valores de fecha, hora y zona horaria, se normalizan a la hora UTC restando su desplazamiento por hora/minuto y comparando después sus componentes de fecha y hora.
- Dos duraciones se comparan según el número total de tics de 100 nanosegundos que representan.
- Se comparan dos elementos lógicos de forma que `true` se considera mayor que `false`.

- Dos números `x` e `y` se comparan según las reglas del estándar IEEE 754:
 - Si alguno de los operandos es `#nan`, el resultado es `false` para todos los operadores relacionales.
 - Cuando ninguno de los operandos es `#nan`, los operadores comparan los valores de los dos operandos de punto flotante con respecto a la ordenación `-∞ < -max < ... < -min < -0.0 = +0.0 < +min < ... < +max < +∞`, donde "min" y "max" son los valores finitos positivos más pequeños y más grandes que se pueden representar. En M, $-\infty$ y $+\infty$ se denominan "`-#infinity`" y "`#infinity`".

Los efectos importantes de este orden son:

- Los ceros negativos y positivos se consideran iguales.
 - Un valor `-#infinity` se considera menor que el resto de los valores numéricos, pero es igual a otro `-#infinity`.
 - Un valor `#infinity` se considera mayor que el resto de los valores numéricos, pero es igual a otro `#infinity`.
- Se comparan dos textos mediante una comparación sin distinción entre mayúsculas y minúsculas, sin distinción entre mayúsculas y minúsculas.
 - Para comparar dos horas se comparan sus partes de hora y, si son iguales, sus partes de minuto y, si son iguales, sus partes de segundo.

Operadores lógicos condicionales

Los operadores `and` y `or` se denominan operadores lógicos condicionales.

logical-or-expression:

logical-and-expression

logical-and-expression `or` *logical-or-expression*

logical-and-expression:

is-expression

is-expression `and` *logical-and-expression*

El operador `or` devuelve `true` cuando al menos uno de sus operandos es `true`. El operando derecho se evalúa solo si el operando izquierdo no es `true`.

El operador `and` devuelve `false` cuando al menos uno de sus operandos es `false`. El operando derecho se evalúa solo si el operando izquierdo no es `false`.

A continuación se muestran las tablas de verdad para los operadores `or` y `and`, con el resultado de evaluar la expresión de operando de la izquierda en el eje vertical y el de evaluar la expresión de operando de la derecha en el horizontal.

[Expandir tabla](#)

<code>and</code>	<code>true</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>true</code>	<code>true</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>false</code>	<code>false</code>	<code>false</code>	<code>false</code>	<code>false</code>
<code>null</code>	<code>null</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>error</code>	<code>error</code>	<code>error</code>	<code>error</code>	<code>error</code>

[Expandir tabla](#)

<code>or</code>	<code>true</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>or</code>	<code>true</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>true</code>	<code>true</code>	<code>true</code>	<code>true</code>	<code>true</code>
<code>false</code>	<code>true</code>	<code>false</code>	<code>null</code>	<code>error</code>
<code>null</code>	<code>true</code>	<code>null</code>	<code>null</code>	<code>error</code>
<code>error</code>	<code>error</code>	<code>error</code>	<code>error</code>	<code>error</code>

Cuando se evalúa una expresión que contiene operadores lógicos condicionales, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- Los operadores lógicos condicionales se definen antes que los tipos `logical` y `null`. Si los valores de operando no son de esos tipos, se genera un error con el código de motivo `"Expression.Error"`.
- El resultado es un valor lógico.
- En la expresión `x` o `y`, se evaluará la expresión `y` solo si `x` no se evalúa como `true`.
- En la expresión `x` e `y`, se evaluará la expresión `y` solo si `x` no se evalúa como `false`.

Las dos últimas propiedades proporcionan a los operadores lógicos condicionales su calificación de "condicionales"; las propiedades también se denominan "cortocircuito". Estas

propiedades son útiles para escribir *predicados protegidos*. Por ejemplo, las expresiones siguientes son equivalentes:

```
Power Query M

d <> 0 and n/d > 1 if d <> 0 then n/d > 1 else false
```

Operadores aritméticos

Los operadores `+`, `-`, `*` y `/` son los *operadores aritméticos*.

- additive-expression*:
 - multiplicative-expression*
 - additive-expression* `+` *multiplicative-expression*
 - additive-expression* `-` *multiplicative-expression*
- expresión-multiplicativa*:
 - metadata-expression*
 - multiplicative-expression* `*` *metadata-expression*
 - multiplicative-expression* `/` *metadata-expression*

Precisión

En M, los números se almacenan con diversas representaciones para conservar tanta información como sea posible sobre los números procedentes de diversos orígenes. Los números solo se convierten de una representación a otra según sea necesario para los operadores que se les aplica. En M se admiten dos precisiones:

 Expandir tabla

Precisión	Semántica
<code>Precision.Decimal</code>	Representación decimal de 128 bits con un intervalo de $\pm 1,0 \times 10^{-28}$ a $\pm 7,9 \times 10^{28}$ and 28-29 dígitos significativos.
<code>Precision.Double</code>	Representación científica con mantisa y exponente; se ajusta al estándar aritmético IEEE 754 de doble precisión de 64 bits IEEE 754-2008 .

Para realizar operaciones aritméticas, se elige una precisión, se convierten los dos operandos a esa precisión (si es necesario), después se realiza la operación real y, por último, se devuelve un número con la precisión elegida.

Los operadores aritméticos integrados (+, -, *, /) usan precisión doble. Las funciones de la biblioteca estándar (Value.Add, Value.Subtract, Value.Multiply, Value.Divide) se pueden usar para solicitar estas operaciones con un modelo de precisión específico.

- El desbordamiento numérico no es posible: #infinity o -#infinity representan valores de magnitudes demasiado grandes para representarlos.
- El subdesbordamiento numérico no es posible: 0 y -0 representan valores de magnitudes demasiado pequeñas para representarlos.
- El valor especial de IEEE 754 #nan (NaN: No es un número) se usa para cubrir los casos aritméticos no válidos, como la división de cero por cero.
- Para realizar la conversión de precisión decimal a doble se redondean los números decimales al valor doble equivalente más próximo.
- Para realizar la conversión de precisión doble a decimal se redondean los números dobles al valor decimal equivalente más cercano y, si es necesario, se desborda a valores #infinity o -#infinity.

Operador de suma

La interpretación del operador de suma (x + y) depende del tipo de valor de las expresiones x e y evaluadas, como se indica a continuación:

 Expandir tabla

x	y	Resultado	Interpretación
type number	type number	type number	Suma numérica
type number	null	null	
null	type number	null	
type duration	type duration	type duration	Suma numérica de magnitudes
type duration	null	null	
null	type duration	null	
type <i>datetime</i>	type duration	type <i>datetime</i>	Desplazamiento de fecha y hora según la duración
type duration	type <i>datetime</i>	type <i>datetime</i>	
type <i>datetime</i>	null	null	

x	y	Resultado	Interpretación
null	type datetime	null	

En la tabla, type datetime representa type date, type datetime, type datetimetimezone o type time. Al sumar una duración y un valor de algún tipo datetime, el valor resultante es del mismo tipo.

Para otras combinaciones de valores que no sean las enumeradas en la tabla, se genera un error con el código de motivo "Expression.Error". Cada combinación se describe en las secciones siguientes.

Se propagan los errores que se producen al evaluar cualquiera de los operandos.

Suma numérica

La suma de dos números se calcula mediante el *operador de suma*, lo que genera un número.

Por ejemplo:

Power Query M

```
1 + 1 // 2
#nan + #infinity // #nan
```

El operador de suma + en números usa la precisión doble; la función de la biblioteca estándar Value.Add se puede usar para especificar la precisión decimal. Al calcular una suma de números sucede lo siguiente:

- La suma en precisión doble se calcula de acuerdo a las reglas aritméticas de IEEE 754 de precisión doble binaria de 64 bits [IEEE 754-2008](#). En la tabla siguiente se enumeran los resultados de todas las posibles combinaciones de valores finitos distintos de cero, ceros, infinitos y NaN. En la tabla, x e y son valores finitos distintos de cero y z es el resultado de x + y. Si x e y tienen la misma magnitud pero signos opuestos, z es cero positivo. Si x + y es demasiado grande para representarse en el tipo de destino, z es un infinito con el mismo signo que x + y.

 Expandir tabla

+	y	+0	-0	+∞	-∞	NaN
x	z	x	x	+∞	-∞	NaN
+0	y	+0	+0	+∞	-∞	NaN

+	y	+0	-0	$+\infty$	$-\infty$	NaN
-0	y	+0	-0	$+\infty$	$-\infty$	NaN
$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	NaN	NaN
$-\infty$	$-\infty$	$-\infty$	$-\infty$	NaN	$-\infty$	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN

- La suma en precisión decimal se calcula sin perder precisión. La escala del resultado es la mayor de las escalas de los dos operandos.

Suma de las duraciones

La suma de dos duraciones es la duración que representa la suma del número de tics de 100 nanosegundos representados por las duraciones. Por ejemplo:

Power Query M

```
#duration(2,1,0,15.1) + #duration(0,1,30,45.3)
// #duration(2, 2, 31, 0.4)
```

Desplazamiento de fecha y hora según la duración

Se puede sumar un valor *datetime* **x** y una duración **y** con **x + y** para calcular un nuevo valor *datetime* cuya distancia con respecto a **x** en una escala de tiempo lineal sea exactamente la magnitud de **y**. Aquí, *datetime* equivale a **Date**, **DateTime**, **DateTimeZone** o **Time**, y un resultado no NULL será del mismo tipo. El desplazamiento de fecha y hora por duración se puede calcular de la manera siguiente:

- Si se especifican los días de la fecha y hora desde el valor de época, se construye un nuevo objeto *datetime* con los elementos de información siguientes:
 - Se calcula un nuevo número de días desde la época, equivalente a dividir la magnitud de **y** por el número de tics de 100 nanosegundos en un período de 24 horas, se trunca la parte decimal del resultado y se suma este valor a los días de **x** desde la época.
 - Se calcula un nuevo tic desde la medianoche equivalente a sumar la magnitud de **y** a los tics de **x** desde la medianoche, módulo el número de tics de 100 nanosegundos en un período de 24 horas. Si **x** no especifica un valor para los tics desde la medianoche, se supone un valor de 0.

- Se copia el valor de x para el desplazamiento de minutos con respecto a la hora UTC sin modificar.
- Si no se especifican los días de la fecha y hora desde el valor de época, se construye un nuevo objeto datetime con los elementos de información siguientes:
 - Se calcula un nuevo tic desde la medianoche equivalente a sumar la magnitud de y a los tics de x desde la medianoche, módulo el número de tics de 100 nanosegundos en un período de 24 horas. Si x no especifica un valor para los tics desde la medianoche, se supone un valor de 0.
 - Se copian los valores de x de los días desde la época y el desplazamiento de minutos con respecto a la hora UTC sin modificar.

En los ejemplos siguientes se muestra cómo calcular la suma temporal absoluta cuando la fecha y hora especifica los *días desde la época*:

Power Query M

```
#date(2010,05,20) + #duration(0,8,0,0)
//#datetime( 2010, 5, 20, 8, 0, 0 )
//2010-05-20T08:00:00

#date(2010,01,31) + #duration(30,08,0,0)
//#datetime(2010, 3, 2, 8, 0, 0)
//2010-03-02T08:00:00

#datetime(2010,05,20,12,00,00,-08) + #duration(0,04,30,00)
//#datetime(2010, 5, 20, 16, 30, 0, -8, 0)
//2010-05-20T16:30:00-08:00

#datetime(2010,10,10,0,0,0,0) + #duration(1,0,0,0)
//#datetime(2010, 10, 11, 0, 0, 0, 0, 0)
//2010-10-11T00:00:00+00:00
```

En el ejemplo siguiente se muestra cómo calcular el desplazamiento de fecha y hora por duración para una hora concreta:

Power Query M

```
#time(8,0,0) + #duration(30,5,0,0)
//#time(13, 0, 0)
//13:00:00
```

Operador de resta

La interpretación del operador de resta ($x - y$) depende del tipo de valor de las expresiones `x` e `y` evaluadas, como se indica a continuación:

[Expandir tabla](#)

x	Y	Resultado	Interpretación
type number	type number	type number	Diferencia numérica
type number	null	null	
null	type number	null	
type duration	type duration	type duration	Diferencia numérica de magnitudes
type duration	null	null	
null	type duration	null	
type datetime	type datetime	type duration	Duración entre fechas y horas
type datetime	type duration	type datetime	Desplazamiento de fecha y hora según la duración negada
type datetime	null	null	
null	type datetime	null	

En la tabla, `type datetime` representa `type date`, `type datetime`, `type datetimetimezone` o `type time`. Al restar una duración de un valor de algún tipo `datetime`, el valor resultante es del mismo tipo.

Para otras combinaciones de valores que no sean las enumeradas en la tabla, se genera un error con el código de motivo `"Expression.Error"`. Cada combinación se describe en las secciones siguientes.

Se propagan los errores que se producen al evaluar cualquiera de los operandos.

Diferencia numérica

La diferencia entre dos números se calcula mediante el *operador de resta*, lo que genera un número. Por ejemplo:

```
Power Query M

1 - 1           // 0
#nan - #infinity // #nan
```

El operador de resta `-` en números usa la precisión doble; la función de la biblioteca estándar `Value.Subtract` se puede usar para especificar la precisión decimal. Al calcular una diferencia de números sucede lo siguiente:

- La diferencia en precisión doble se calcula de acuerdo a las reglas aritméticas de IEEE 754 de precisión doble binaria de 64 bits [IEEE 754-2008](#). En la tabla siguiente se enumeran los resultados de todas las posibles combinaciones de valores finitos distintos de cero, ceros, infinitos y NaN. En la tabla, `x` e `y` son valores finitos distintos de cero y `z` es el resultado de `x - y`. Si `x` e `y` son iguales, `z` es cero positivo. Si `x - y` es demasiado grande para representarse en el tipo de destino, `z` es un infinito con el mismo signo que `x - y`.

[Expandir tabla](#)

-	y	+0	-0	+∞	-∞	NaN
x	z	x	x	-∞	+∞	NaN
+0	-y	+0	+0	-∞	+∞	NaN
-0	-y	-0	+0	-∞	+∞	NaN
+∞	+∞	+∞	+∞	NaN	+∞	NaN
-∞	-∞	-∞	-∞	-∞	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN

- La diferencia en precisión decimal se calcula sin perder precisión. La escala del resultado es la mayor de las escalas de los dos operandos.

Resta de las duraciones

La diferencia de dos duraciones es la duración que representa la diferencia entre el número de tics de 100 nanosegundos representados por cada duración. Por ejemplo:

```
Power Query M
```

```
#duration(1,2,30,0) - #duration(0,0,0,30.45)
// #duration(1, 2, 29, 29.55)
```

Desplazamiento de fecha y hora según la duración negativa

Se puede restar un valor *datetime* **x** y una duración **y** con **x - y** para calcular un nuevo valor *datetime*. Aquí, *datetime* representa **date**, **datetime**, **datetimezone** o **time**. El valor *datetime* resultante tiene una distancia con respecto a **x** en una escala de tiempo lineal que es exactamente la magnitud de **y**, en la dirección opuesta al signo de **y**. La resta de duraciones positivas genera resultados anteriores en el tiempo con respecto a **x**, mientras que la resta de valores negativos genera resultados posteriores en el tiempo.

Power Query M

```
#date(2010,05,20) - #duration(00,08,00,00)
// #datetime(2010, 5, 19, 16, 0, 0)
// 2010-05-19T16:00:00
#date(2010,01,31) - #duration( 30,08,00,00)
// #datetime(2009, 12, 31, 16, 0, 0)
// 2009-12-31T16:00:00
```

Duración entre valores datetime

Se pueden restar dos valores *datetime* **t** y **u** con **t - u** para calcular la duración entre ellos. Aquí, *datetime* representa **date**, **datetime**, **datetimezone** o **time**. La duración que se genera al restar **u** de **t** debe producir **t** cuando se suma a **u**.

Power Query M

```
#date(2010,01,31) - #date(2010,01,15)
// #duration(16,00,00,00)
// 16.00:00:00

#date(2010,01,15) - #date(2010,01,31)
// #duration(-16,00,00,00)
// -16.00:00:00

#datetime(2010,05,20,16,06,00,-08,00) -
#datetime(2008,12,15,04,19,19,03,00)
// #duration(521,22,46,41)
// 521.22:46:41
```

Al restar **t - u** cuando **u > t** se genera una duración negativa:

Power Query M

```
#time(01,30,00) - #time(08,00,00)
// #duration(0, -6, -30, 0)
```

Cuando se restan dos valores *datetime* mediante `t - u`, sucede lo siguiente:

- $u + (t - u) = t$

Operador de multiplicación

La interpretación del operador de multiplicación (`x * y`) depende del tipo de valor de las expresiones `x` e `y` evaluadas, como se indica a continuación:

[Expandir tabla](#)

X	Y	Resultado	Interpretación
type number	type number	type number	Producto numérico
type number	null	null	
null	type number	null	
type duration	type number	type duration	Múltiplo de duración
type number	type duration	type duration	Múltiplo de duración
type duration	null	null	
null	type duration	null	

Para otras combinaciones de valores que no sean las enumeradas en la tabla, se genera un error con el código de motivo `"Expression.Error"`. Cada combinación se describe en las secciones siguientes.

Se propagan los errores que se producen al evaluar cualquiera de los operandos.

Producto numérico

El producto de dos números se calcula mediante el *operador de multiplicación*, lo que genera un número. Por ejemplo:

Power Query M

```
2 * 4           // 8
6 * null        // null
```

```
#nan * #infinity // #nan
```

El operador de multiplicación `*` en números usa la precisión doble; la función de la biblioteca estándar `Value.Multiply` se puede usar para especificar la precisión decimal. Al calcular un producto de números sucede lo siguiente:

- El producto en precisión doble se calcula de acuerdo a las reglas aritméticas de IEEE 754 de precisión doble binaria de 64 bits [IEEE 754-2008](#). En la tabla siguiente se enumeran los resultados de todas las posibles combinaciones de valores finitos distintos de cero, ceros, infinitos y NaN. En la tabla, `x` e `y` son valores finitos positivos. `z` es el resultado de `x * y`. Si el resultado es demasiado grande para el tipo de destino, `z` es infinito. Si el resultado es demasiado pequeño para el tipo de destino, `z` es cero.

[Expandir tabla](#)

*	+y	-y	+0	-0	+∞	-∞	NaN
+x	+z	-z	+0	-0	+∞	-∞	NaN
-x	-z	+z	-0	+0	-∞	+∞	NaN
+0	+0	-0	+0	-0	NaN	NaN	NaN
-0	-0	+0	-0	+0	NaN	NaN	NaN
+∞	+∞	-∞	NaN	NaN	+∞	-∞	NaN
-∞	-∞	+∞	NaN	NaN	-∞	+∞	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- El producto en precisión decimal se calcula sin perder precisión. La escala del resultado es la mayor de las escalas de los dos operandos.

Múltiplos de duraciones

El producto de una duración y un número es la duración que representa el número de tics de 100 nanosegundos representados por el operando de duración multiplicado por el de número. Por ejemplo:

Power Query M

```
#duration(2,1,0,15.1) * 2  
// #duration(4, 2, 0, 30.2)
```

Operador de división

La interpretación del operador de división (x / y) depende del tipo de valor de las expresiones x e y evaluadas, como se indica a continuación:

 Expandir tabla

X	Y	Resultado	Interpretación
type number	type number	type number	Cociente numérico
type number	null	null	
null	type number	null	
type duration	type number	type duration	Fracción de duración
type duration	type duration	type number	Cociente numérico de duraciones
type duration	null	null	
null	type duration	null	

Para otras combinaciones de valores que no sean las enumeradas en la tabla, se genera un error con el código de motivo `"Expression.Error"`. Cada combinación se describe en las secciones siguientes.

Se propagan los errores que se producen al evaluar cualquiera de los operandos.

Cociente numérico

El cociente de dos números se calcula mediante el *operador de división*, lo que genera un número. Por ejemplo:

Power Query M

```
8 / 2           // 4
8 / 0           // #infinity
0 / 0           // #nan
0 / null        // null
#nan / #infinity // #nan
```

El operador de división `/` en números usa la precisión doble; la función de la biblioteca estándar `Value.Divide` se puede usar para especificar la precisión decimal. Al calcular un cociente de números sucede lo siguiente:

- El cociente en precisión doble se calcula de acuerdo a las reglas aritméticas de IEEE 754 de precisión doble binaria de 64 bits [IEEE 754-2008](#). En la tabla siguiente se enumeran los resultados de todas las posibles combinaciones de valores finitos distintos de cero, ceros, infinitos y NaN. En la tabla, **x** e **y** son valores finitos positivos. **z** es el resultado de x / y . Si el resultado es demasiado grande para el tipo de destino, **z** es infinito. Si el resultado es demasiado pequeño para el tipo de destino, **z** es cero.

[Expandir tabla](#)

/	+y	-y	+0	-0	+∞	-∞	NaN
+x	+z	-z	+∞	-∞	+0	-0	NaN
-x	-z	+z	-∞	+∞	-0	+0	NaN
+0	+0	-0	NaN	NaN	+0	-0	NaN
-0	-0	+0	NaN	NaN	-0	+0	NaN
+∞	+∞	-∞	+∞	-∞	NaN	NaN	NaN
-∞	-∞	+∞	-∞	+∞	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- La suma en precisión decimal se calcula sin perder precisión. La escala del resultado es la mayor de las escalas de los dos operandos.

Cociente de duraciones

El cociente de dos duraciones es el número que representa el cociente del número de tics de 100 nanosegundos representados por las duraciones. Por ejemplo:

Power Query M

```
#duration(2,0,0,0) / #duration(0,1,30,0)
// 32
```

Duraciones escaladas

El cociente de una duración **x** y un número **y** es la duración que representa el cociente del número de tics de 100 nanosegundos representados por la duración **x** y el número **y**. Por ejemplo:

Power Query M

```
#duration(2,0,0,0) / 32
// #duration(0,1,30,0)
```

Combinación de estructura

El operador de combinación (`x & y`) se define en los tipos de valores siguientes:

 Expandir tabla

X	Y	Resultado	Interpretación
type text	type text	type text	Concatenación
type text	null	null	
null	type text	null	
type date	type time	type datetime	Combinar
type date	null	null	
null	type time	null	
type list	type list	type list	Concatenación
type record	type record	type record	Combinar
type table	type table	type table	Concatenación

Concatenación

Dos valores de texto, dos listas o dos tablas se pueden concatenar mediante `x & y`.

En el ejemplo siguiente se muestra la concatenación de valores de texto:

Power Query M

```
"AB" & "CDE" // "ABCDE"
```

En el ejemplo siguiente se muestra la concatenación de listas:

Power Query M

```
{1, 2} & {3} // {1, 2, 3}
```

Cuando se concatenan dos valores mediante `x & y`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- No se propaga ningún error si un elemento de `x` o `y` contiene un error.
- El resultado de concatenar dos valores de texto es un valor de texto que contiene el valor de `x` inmediatamente seguido de `y`. Si alguno de los operandos es NULL y el otro es un valor de texto, el resultado es NULL.
- El resultado de la concatenación de dos listas es una lista que contiene todos los elementos de `x` seguidos de todos los elementos de `y`.
- El resultado de la concatenación de dos tablas es una tabla que tiene la unión de las columnas de las dos tablas de operandos. Se conserva el orden de las columnas de `x`, seguido de las columnas que solo aparecen en `y`, para conservar su orden relativo. En el caso de las columnas que solo aparecen en uno de los operandos, se usa `null` para rellenar los valores de celda para el otro operando.

Combinar

Combinación de registros

Dos registros se pueden combinar mediante `x & y`, lo que genera un registro que incluye los campos de `x` e `y`.

En los ejemplos siguientes se muestra la combinación de registros:

Power Query M

```
[ x = 1 ] & [ y = 2 ] // [ x = 1, y = 2 ]  
[ x = 1, y = 2 ] & [ x = 3, z = 4 ] // [ x = 3, y = 2, z = 4 ]
```

Cuando se combinan dos registros mediante `x + y`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- Si un campo aparece en `x` e `y`, se usa el valor de `y`.
- El orden de los campos en el registro resultante es el de `x`, seguido de los campos de `y` que no forman parte de `x`, en el mismo orden en que aparecen en `y`.
- La combinación de registros no provoca la evaluación de los valores.

- No se produce ningún error porque un campo contenga un error.
- El resultado es un registro.

Combinación de fecha y hora

Una fecha `x` se puede combinar con una hora `y` mediante `x & y`, lo que genera un valor de fecha y hora que combina las partes de `x` e `y`.

En el ejemplo siguiente se muestra cómo combinar una fecha y una hora:

Power Query M

```
#date(2013,02,26) & #time(09,17,00)
// #datetime(2013,02,26,09,17,00)
```

Cuando se combinan dos registros mediante `x + y`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar las expresiones `x` o `y`.
- El resultado es un valor de fecha y hora.

Operadores unarios

Los operadores `+`, `-` y `not` son operadores unarios.

unary-expression:

type-expression

`+` *expresión unaria*

`-` *expresión unaria*

`not` *expresión unaria*

Operador unario más

El operador unario más (`+x`) se define para los tipos de valores siguientes:

[Expandir tabla](#)

X	Resultado	Interpretación
<code>type number</code>	<code>type number</code>	Suma unaria
<code>type duration</code>	<code>type duration</code>	Suma unaria

X	Resultado	Interpretación
null	`null	

Para otros valores, se genera un error con el código de motivo "Expression.Error".

El operador unario más permite aplicar un + signo a un valor de número, fecha y hora, o NULL.

El resultado es ese mismo valor. Por ejemplo:

Power Query M

```
+ - 1           // -1
+ + 1           // 1
+ #nan          // #nan
+ #duration(0,1,30,0) // #duration(0,1,30,0)
```

Al evaluar el operador unario más +x, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar x.
- Si el resultado de evaluar x no es un valor numérico, se genera un error con el código de motivo "Expression.Error".

Operador unario menos

El operador unario menos (-x) se define para los tipos de valores siguientes:

[Expandir tabla](#)

X	Resultado	Interpretación
type number	type number	Negación
type duration	type duration	Negación
null	null	

Para otros valores, se genera un error con el código de motivo "Expression.Error".

El operador unario menos se usa para cambiar el signo de un número o de una duración. Por ejemplo:

Power Query M

```
- (1 + 1)       // -2
- - 1           // 1
```

```
- - - 1          // -1
- #nan          // #nan
- #infinity     // -#infinity
- #duration(1,0,0,0) // #duration(-1,0,0,0)
- #duration(0,1,30,0) // #duration(0,-1,-30,0)
```

Al evaluar el operador unario menos `-x`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar `x`.
- Si la expresión es un número, el resultado es el valor numérico de la expresión `x` con el signo cambiado. Si el valor es NaN, el resultado también es NaN.

Operador de negación lógica.

El operador de negación lógica (`not`) se define para los tipos de valores siguientes:

[Expandir tabla](#)

X	Resultado	Interpretación
<code>type logical</code>	<code>type logical</code>	Negación
<code>null</code>	<code>null</code>	

Este operador calcula la operación `not` lógica en un valor lógico determinado. Por ejemplo:

Power Query M

```
not true          // false
not false         // true
not (true and true) // false
```

Al evaluar el operador de negación lógica `not x`, sucede lo siguiente:

- Se propagan los errores que se producen al evaluar `x`.
- El valor generado de la evaluación de la expresión `x` debe ser un valor lógico, o bien se debe generar un error con el código de motivo `"Expression.Error"`. Si el valor es `true`, el resultado es `false`. Si el operando es `false`, el resultado es `true`.

El resultado es un valor lógico.

Operadores de tipo

Los operadores `is` y `as` se conocen como operadores de tipo.

Operador de compatibilidad de tipos

El operador de compatibilidad de tipos `x is y` se define para los tipos de valores siguientes:

[Expandir tabla](#)

X	Y	Resultado
<code>type any</code>	<i>primitive-or-nullable-primitive-type</i>	<code>type logical</code>

La expresión `x is y` devuelve `true` si el tipo descrito de `x` es compatible con `y` y devuelve `false` si no es compatible. `y` debe ser un tipo primitivo o un tipo primitivo que acepta valores NULL.

is-expression:
as-expression
is-expression `is` *primitive-or-nullable-primitive-type*
primitive-or-nullable-primitive-type:
`nullable` *opt* *primitive-type*

La compatibilidad de tipos, tal como la admite el operador `is`, es un subconjunto de la [compatibilidad con tipos generales](#) y se define mediante las reglas siguientes:

- Si `x` es null, es compatible si `y` es el tipo `any`, el tipo `null` o un tipo primitivo que acepta valores NULL.
- Si `x` no es null, es compatible si el tipo primitivo de `x` es el mismo que `y`.

Cuando se evalúa la expresión `x is y` sucede lo siguiente:

- Se propaga un error que se produce al evaluar la expresión `x`.

Operador de aserción de tipos

El operador de aserción de tipos `x as y` se define para los tipos de valores siguientes:

[Expandir tabla](#)

X	Y	Resultado
<code>type any</code>	<i>primitive-or-nullable-primitive-type</i>	<code>type any</code>

La expresión `x as y` afirma que el valor `x` es compatible con `y` según el operador `is`. Si no es compatible, se produce un error. `y` debe ser un tipo primitivo o un tipo primitivo que acepta valores NULL.

as-expression:

equality-expression

as-expression as primitive-or-nullable-primitive-type

primitive-or-nullable-primitive-type:

`nullable` *opt* *primitive-type*

La expresión `x as y` se evalúa de esta forma:

- Se realiza una comprobación de compatibilidad de tipos `x is y` y la aserción devuelve `x` sin modificar si la prueba se realiza correctamente.
- Si se produce un error en la comprobación de compatibilidad, se genera un error con el código de motivo `"Expression.Error"`.

Ejemplos:

Power Query M

```
1 as number           // 1
"A" as number         // error
null as nullable number // null
```

Cuando se evalúa la expresión `x as y` sucede lo siguiente:

- Se propaga un error que se produce al evaluar la expresión `x`.

Operador de fusión

El operador de fusión `??` devuelve el resultado de su operando izquierdo si no es NULL; de lo contrario, devolverá el resultado de su operando derecho. El operando derecho solamente se evalúa si el operando izquierdo no es NULL.

Let

Artículo • 20/11/2024

Expresión let

Una expresión let se puede usar para capturar un valor de un cálculo intermedio en una variable.

let-expression:

`let` *variable-list* `in` *expresión*

variable-list:

variable

variable `,` *variable-list*

variable:

expresión *variable-name* `=`

variable-name:

identifier

En el ejemplo siguiente se muestran los resultados intermedios que se calculan y se almacenan en las variables `x`, `y` y `z`, que después se usan en un cálculo `x + y + z` posterior:

Power Query M

```
let    x = 1 + 1,
      y = 2 + 2,
      z = y + 1
in
      x + y + z
```

El resultado de esta expresión es el siguiente:

Power Query M

```
11 // (1 + 1) + (2 + 2) + (2 + 2 + 1)
```

Al evaluar expresiones dentro de *let-expression*, sucede lo siguiente:

- Las expresiones de la lista variable definen un nuevo ámbito que contiene los identificadores de la producción *variable-list* y deben estar presentes al evaluar las

expresiones dentro de las producciones *variable-list*. Las expresiones dentro de *variable-list* pueden hacerse referencia entre ellas.

- Las expresiones dentro de *variable-list* se deben evaluar antes de que se evalúe la expresión de *let-expression*.
- A menos que se acceda a las expresiones de *variable-list*, no se deben evaluar.
- Los errores que se producen durante la evaluación de las expresiones de *let-expression* se propagan.

Una expresión *let* se puede considerar un revestimiento sintáctico sobre una expresión de registro implícita. La expresión siguiente es equivalente al ejemplo anterior:

Power Query M

```
[    x = 1 + 1,  
    y = 2 + 2,  
    z = y + 1,  
    result = x + y + z  
][result]
```

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Condicionales

14/06/2025

if-expression (expresión if) selecciona entre dos expresiones según un valor de entrada lógico y evalúa solo la expresión seleccionada.

if-expression:

```
if if-condition then true-expression else false-expression
```

if-condition:

expresión

expresión verdadera:

expresión

expresión falsa:

expresión

Los siguientes son ejemplos de *if-expression*:

Power Query M

```
if 2 > 1 then 2 else 1          // 2
if 1 = 1 then "yes" else "no"  // "yes"
```

Cuando se evalúa *if-expression* sucede lo siguiente:

- Si el valor generado mediante la evaluación de *if-condition* no es un valor lógico, se genera un error con el código `"Expression.Error"` de motivo.
- *true-expression* solo se evalúa si *if-condition* se evalúa como el valor `true`.
- *false-expression* solo se evalúa si *if-condition* se evalúa como el valor `false`.
- El resultado de *if-expression* es el valor de *true-expression* si *if-condition* es `true`, o bien el valor de *false-expression* si *if-condition* es `false`.
- Se propagan los errores producidos durante la evaluación de *if-condition*, *true-expression* o *false-expression*.

Funciones

Una *función* es un valor que representa una asignación de un conjunto de valores de argumento a un valor único. Una función se invoca mediante un conjunto de valores de entrada (los valores de argumento) y genera un único valor de salida (el valor devuelto).

Escritura de funciones

Las funciones se escriben mediante *function-expression* (expresión de función):

function-expression:

(*opción de*) lista de parámetros *return-type*_{opt} => *cuerpo de la función*

function-body:

expresión

parameter-list:

fixed-parameter-list

fixed-parameter-list , *optional-parameter-list*

optional-parameter-list

fixed-parameter-list:

parámetro

parameter , *fixed-parameter-list*

parameter:

parameter-name *parameter-type*_{opt}

parameter-name:

identificador

parameter-type:

primitive-or-nullable-primitive-type-assertion

tipo de valor devuelto:

primitive-or-nullable-primitive-type-assertion

primitive-or-nullable-primitive-type-assertion:

as *primitive-or-nullable-primitive-type*

optional-parameter-list:

optional-parameter

optional-parameter , *optional-parameter-list*

optional-parameter:

optional *parámetro*

primitive-or-nullable-primitive-type:

*nullable*_{opt} *primitive-type*

En el ejemplo siguiente se muestra una función que requiere exactamente dos valores `x` e `y`, y genera el resultado de aplicar el operador `+` a esos valores. `x` y `y` son *parámetros* que forman parte del elemento *parameter-list* de la función, y `x + y` es *function-body*:

Power Query M

```
(x, y) => x + y
```

El resultado de evaluar una *expresión de función* es para generar un valor de función (no para evaluar el *cuerpo de la función*). Como convención en este documento, los valores de función (en contraposición a las expresiones de función) se muestran con *formal-parameter-list* pero con puntos suspensivos (`...`) en lugar de *function-body*. Por ejemplo, una vez que se ha evaluado la expresión de función anterior, se mostraría como el siguiente valor de función:

Power Query M

```
(x, y) => ...
```

Los operadores siguientes se definen para los valores de función:

 Expandir tabla

Operador	Resultado
<code>x = y</code>	Igual
<code>x <> y</code>	No igual a

El tipo nativo de los valores de función es un tipo de función personalizado (derivado del tipo intrínseco `function`) que enumera los nombres de parámetro y especifica que todos los tipos de parámetro y el tipo de valor devuelto sean `any`. (Visite [Tipos de función](#) para saber más sobre los tipos de funciones).

Invocación de funciones

El elemento *function-body* de una función se ejecuta al *invocar* el valor de la función mediante *invoke-expression*. La invocación de un valor de función significa que el *cuerpo de la función* del valor de función se evalúa y se devuelve un valor, o bien se produce un error.

invoke-expression:

primary-expression (*argument-list*_{opt})

argument-list:

expression-list

Cada vez que se invoca un valor de función, se especifica un conjunto de valores como *argument-list* (lista de argumentos), denominados *argumentos* de la función.

argument-list se usa para especificar un número fijo de argumentos directamente como una lista de expresiones. En el ejemplo siguiente se define un registro con un valor de función en un campo y, después, se invoca la función desde otro campo del registro:

Power Query M

```
[
    MyFunction = (x, y, z) => x + y + z,
    Result1 = MyFunction(1, 2, 3)           // 6
]
```

Al invocar una función, sucede lo siguiente:

- El entorno que se usa para evaluar el *cuerpo de la función* de la función incluye una variable que se corresponde a cada parámetro, con el mismo nombre que el parámetro. El valor de cada parámetro se corresponde a un valor construido a partir de *argument-list* de *invoke-expression*, como se define en [Parámetros](#).
- Todas las expresiones correspondientes a los argumentos de la función se evalúan antes que el *cuerpo de la función*.
- Se propagan los errores que se producen al evaluar las expresiones de *expression-list* o *function-body*.
- El número de argumentos construido a partir de *argument-list* debe ser compatible con los parámetros de la función, o bien se produce un error con el código de motivo "Expression.Error". El proceso para determinar la compatibilidad se define en [Parámetros](#).

Parámetros

parameter-list puede contener dos tipos de parámetros:

- Un parámetro *obligatorio* indica que cuando se invoca una función siempre se debe especificar un argumento correspondiente al parámetro. Los parámetros obligatorios se deben especificar en primer lugar en *parameter-list*. En el ejemplo siguiente, la función define los parámetros obligatorios **x** e **y**:

Power Query M

```
[
    MyFunction = (x, y) => x + y,
```

```

    Result1 = MyFunction(1, 1),      // 2
    Result2 = MyFunction(2, 2)      // 4
]

```

- Un parámetro *opcional* indica que, cuando se invoca una función, se puede especificar un argumento correspondiente al parámetro, pero no es obligatorio hacerlo. Si no se especifica un argumento que se corresponda con un parámetro opcional al invocar la función, en su lugar se usa el valor `null`. Los parámetros opcionales deben aparecer después de los parámetros obligatorios en *parameter-list*. La función del ejemplo siguiente define un parámetro fijo `x` y un parámetro opcional `y`:

Power Query M

```

[
    MyFunction = (x, optional y) =>
        if (y = null) x else x + y,
    Result1 = MyFunction(1),          // 1
    Result2 = MyFunction(1, null),    // 1
    Result3 = MyFunction(2, 2),       // 4
]

```

El número de argumentos que se especifican al invocar una función debe ser compatible con la lista de parámetros. La compatibilidad de un conjunto de argumentos `A` para una función `F` se calcula de esta forma:

- Se permite que el valor `N` represente el número de argumentos `A` construidos a partir de *argument-list*. Por ejemplo:

Power Query M

```

MyFunction()           // N = 0
MyFunction(1)          // N = 1
MyFunction(null)        // N = 1
MyFunction(null, 2)     // N = 2
MyFunction(1, 2, 3)     // N = 3
MyFunction(1, 2, null)  // N = 3
MyFunction(1, 2, {3, 4}) // N = 3

```

- Se permita que el valor *Required* represente el número de parámetros fijos de `F` y *Optional* el número de parámetros opcionales de `F`. Por ejemplo:

Power Query M

```

()           // Required = 0, Optional = 0
(x)          // Required = 1, Optional = 0

```

```
(optional x) // Required = 0, Optional = 1  
(x, optional y) // Required = 1, Optional = 1
```

- Los argumentos **A** son compatibles con la función **F** si se cumple lo siguiente:
 - $(N \geq \text{Fijo})$ y $(N \leq (\text{Fijo} + \text{Opcional}))$
 - Los tipos de argumento son compatibles con los tipos de parámetro correspondientes de **F**
- Si la función tiene un tipo de valor devuelto declarado, el valor del resultado del cuerpo de la función **F** es compatible con el tipo de valor devuelto de **F** si se cumple lo siguiente:
 - El valor generado al evaluar el cuerpo de la función con los argumentos proporcionados para los parámetros de la función tiene un tipo que es compatible con el tipo de valor devuelto.
- Si el cuerpo de la función genera un valor incompatible con el tipo de valor devuelto de la función, se genera un error con el código de motivo `"Expression.Error"`.

Funciones recursivas

Para escribir un valor de función que sea recursivo, es necesario usar el operador de ámbito (**@**) para hacer referencia a la función dentro de su ámbito. Por ejemplo, el registro siguiente contiene un campo que define la función **Factorial** y otro campo que la invoca:

Power Query M

```
[  
    Factorial = (x) =>  
        if x = 0 then 1 else x * @Factorial(x - 1),  
    Result = Factorial(3) // 6  
]
```

De forma similar, se pueden escribir funciones mutuamente recursivas siempre que cada función a la que se deba acceder tenga un nombre. En el ejemplo siguiente, parte de la función **Factorial** se ha refactorizado en una segunda función **Factorial2**.

Power Query M

```
[  
    Factorial = (x) => if x = 0 then 1 else Factorial2(x),  
    Factorial2 = (x) => x * Factorial(x - 1),  
    Result = Factorial(3) // 6  
]
```

Cierres

Una función puede devolver otra función como un valor. A su vez, esta función puede depender de uno o más parámetros de la función original. En el ejemplo siguiente, la función asociada al campo `MyFunction` devuelve una función que devuelve el parámetro especificado para ella:

Power Query M

```
[
    MyFunction = (x) => () => x,
    MyFunction1 = MyFunction(1),
    MyFunction2 = MyFunction(2),
    Result = MyFunction1() + MyFunction2() // 3
]
```

Cada vez que se invoca la función, se devolverá un nuevo valor de función que mantiene el valor del parámetro de modo que, cuando se invoca, se devolverá el valor del parámetro.

Funciones y entornos

Además de parámetros, el *cuerpo de la función* de una *expresión de función* puede hacer referencia a las variables que se encuentran en el entorno cuando se inicializa la función. Por ejemplo, la función definida por el campo `MyFunction` accede al campo `C` del registro contenedor `A`:

Power Query M

```
[
    A =
        [
            MyFunction = () => C,
            C = 1
        ],
    B = A[MyFunction]() // 1
]
```

Cuando se invoca `MyFunction`, accede al valor de la variable `C`, aunque se invoque desde un entorno (`B`) que no contenga una variable `C`.

Declaraciones simplificadas

each-expression es una abreviatura sintáctica para declarar funciones sin tipo que toman un único parámetro denominado `_` (carácter de subrayado).

each-expression:

`each` *each-expression-body*

each-expression-body:

cuerpo de la función

Las declaraciones simplificadas se usan normalmente para mejorar la legibilidad de la invocación de funciones de orden superior.

Por ejemplo, los siguientes pares de declaraciones son semánticamente equivalentes:

Power Query M

```
each _ + 1  
(_) => _ + 1  
each [A]  
(_) => _[A]
```

```
Table.SelectRows( aTable, each [Weight] > 12 )  
Table.SelectRows( aTable, (_) => _[Weight] > 12 )
```

Last updated on 22/11/2025

Tratamiento de errores

Artículo • 09/03/2023

El resultado de la evaluación de una expresión de M genera uno de los resultados siguientes:

- Se genera un solo valor.
- Se genera un error, lo que indica que el proceso de evaluación de la expresión no ha podido generar un valor. Un error contiene un valor de registro único que se puede usar para proporcionar información adicional sobre la causa de la evaluación incompleta.

Los errores se pueden generar desde una expresión y se pueden controlar desde una expresión.

Generación de errores

La sintaxis para generar un error es la siguiente:

error-raising-expression:

```
error expression
```

Los valores de texto se pueden usar como abreviaturas para los valores de error. Por ejemplo:

Power Query M

```
error "Hello, world" // error with message "Hello, world"
```

Los valores de error completos son registros y se pueden construir con la función

`Error.Record`:

Power Query M

```
error Error.Record("FileNotFound", "File my.txt not found",  
    "my.txt")
```

La expresión anterior es equivalente a:

Power Query M

```
error [  
  Reason = "FileNotFound",  
  Message = "File my.txt not found",  
  Detail = "my.txt"  
]
```

La generación de un error detendrá la evaluación de la expresión actual y la pila de evaluación de la expresión se desenredará hasta que se produzca uno de los siguientes casos:

- Se alcanza un campo de registro, un miembro de sección o una variable `let` (colectivamente, una *entrada*). La entrada se marca como con un error, el valor de error se guarda con esa entrada y, después, se propaga. Cualquier acceso posterior a esa entrada hará que se produzca un error idéntico. Otras entradas del registro, la sección o la expresión `let` no se ven afectadas necesariamente (a menos que accedan a una entrada marcada previamente como con error).
- Se alcanza la expresión de nivel superior. En este caso, el resultado de la evaluación de la expresión de nivel superior es un error en lugar de un valor.
- Se alcanza una expresión `try`. En este caso, el error se captura y se devuelve como un valor.

Control de errores

Una *expresión de control de errores* (conocida informalmente como "try expression") se usa para controlar un error:

error-handling-expression:

`try` *protected-expression* *error-handler*_{opt}

protected-expression:

expresión

error-handler:

otherwise-clause

catch-clause

otherwise-clause:

`otherwise` *default-expression*

default-expression:

expression

catch-clause:

`catch` *catch-function*

catch-function:

(*parameter-name*_{opt}) => *function-body*

Al evaluar una *expresión de control de errores* sin un *controlador de errores*, sucede lo siguiente:

- Si la evaluación de la *expresión protegida* no produce un error y genera un valor *x*, el valor generado por la *expresión de control de errores* es un registro con el formato siguiente:

Power Query M

```
[ HasErrors = false, Value = x ]
```

- Si la evaluación de la *expresión protegida* genera un error, el resultado de la *expresión de control de errores* es un registro con el formato siguiente:

Power Query M

```
[ HasErrors = true, Error = e ]
```

Al evaluar una *expresión de control de errores* con un *controlador de errores*, sucede lo siguiente:

- Se debe evaluar la *expresión protegida* antes que el *controlador de errores*.
- Se debe evaluar el *controlador de errores* solo si la evaluación de la *expresión protegida* genera un error.
- Si la evaluación de la *expresión protegida* genera un error, el valor generado por la *expresión de control de errores* es el resultado de la evaluación del *controlador de errores*.
- Se propagan los errores generados durante la evaluación del *controlador de errores*.
- Cuando el *controlador de errores* que se evalúa es una *cláusula catch*, se invoca la *función catch*. Si esa función acepta un parámetro, el valor de error se pasará como su valor.

En el ejemplo siguiente se muestra una *expresión de control de errores* en un caso en el que no se produce ningún error:

Power Query M

```
let
    x = try "A"
in
    if x[HasError] then x[Error] else x[Value]
// "A"
```

En el ejemplo siguiente se muestra cómo generar un error y después controlarlo:

Power Query M

```
let
    x = try error "A"
in
    if x[HasError] then x[Error] else x[Value]
// [ Reason = "Expression.Error", Message = "A", Detail = null ]
```

El ejemplo anterior se puede reescribir con menos sintaxis mediante una *cláusula catch* con una *función catch* que acepta un parámetro:

Power Query M

```
let
    x = try error "A" catch (e) => e
in
    x
// [ Reason = "Expression.Error", Message = "A", Detail = null ]
```

Se puede usar una *cláusula otherwise* para reemplazar los errores controlados por una expresión try con un valor alternativo:

Power Query M

```
try error "A" otherwise 1
// 1
```

Una *cláusula catch* con una *función catch* de parámetro cero es eficazmente una sintaxis alternativa más larga para una *cláusula otherwise*:

Power Query M

```
try error "A" catch () => 1
// 1
```

Si el *controlador de errores* también produce un error, lo mismo sucede con la expresión try completa:

Power Query M

```
try error "A" otherwise error "B"  
// error with message "B"
```

Power Query M

```
try error "A" catch () => error "B"  
// error with message "B"
```

Power Query M

```
try error "A" catch (e) => error "B"  
// error with message "B"
```

Errores en registros e inicializadores let

En el ejemplo siguiente se muestra un inicializador de registros con un campo **A** que genera un error y al que acceden otros dos campos **B** y **C**. El campo **B** no controla el error generado **A**, pero **C** sí lo hace. El último campo **D** no accede a **A** y, por tanto, no se ve afectado por el error en **A**.

Power Query M

```
[  
    A = error "A",  
    B = A + 1,  
    C = let x =  
        try A in  
        if not x[HasError] then x[Value]  
        else x[Error],  
    D = 1 + 1  
]
```

El resultado de evaluar la expresión anterior es el siguiente:

Power Query M

```
[  
    A = // error with message "A"  
    B = // error with message "A"  
    C = "A",  
    D = 2  
]
```

En M, el control de errores se debe realizar cerca de la causa de los errores para tratar los efectos de la inicialización diferida de campos y las evaluaciones de cierre en diferido. En el ejemplo siguiente se muestra un intento incorrecto de control de un error mediante una expresión `try`:

Power Query M

```
let
    f = (x) => [ a = error "bad", b = x ],
    g = try f(42) otherwise 123
in
    g[a] // error "bad"
```

En este ejemplo, la definición `g` estaba pensada para controlar el error que se produce al llamar a `f`. Pero el error lo genera un inicializador de campo que solo se ejecuta cuando es necesario y, por tanto, después de que el registro se haya devuelto desde `f` y se haya pasado a través de la expresión `try`.

Error no implementado

Mientras se desarrolla una expresión, es posible que un autor quiera dejar de usar la implementación en algunas partes de la expresión, pero que pueda seguir ejecutándola. Una manera de controlar este caso es generar un error para las partes no implementadas. Por ejemplo:

Power Query M

```
(x, y) =>
    if x > y then
        x - y
    else
        error Error.Record("Expression.Error",
            "Not Implemented")
```

El símbolo de puntos suspensivos (`...`) se puede usar como método abreviado para `error`.

not-implemented-expression:

`...`

Por ejemplo, el ejemplo siguiente es equivalente al anterior:

Power Query M

```
(x, y) => if x > y then x - y else ...
```

Comentarios

¿Le ha resultado útil esta página?

 Yes

 No

[Obtener ayuda en Microsoft Q&A](#)

Secciones

Artículo • 06/04/2023

section-document es un programa de M que consta de varias expresiones con nombre.

section-document:

section

section:

*literal-attributes*_{opt} *section-name* ; *section-members*_{opt}

section-name:

identificador

section-members:

section-member *section-members*_{opt}

section-member:

*literal-attributes*_{opt} *shared*_{opt} *section-member-name* = *expression* ;

section-member-name:

identifier

En M, una sección es un concepto de organización que permite asignar un nombre a las expresiones relacionadas y agruparlas dentro de un documento. Cada sección tiene un nombre de sección (*section-name*), que identifica la sección y califica los nombres de los miembros de sección (*section-member*) declarados en la sección. Un *miembro de sección* consta de un *nombre de miembro* y una *expresión*. Las expresiones de miembro de sección pueden hacer referencia a otros miembros de sección dentro de la misma sección directamente por nombre de miembro.

En el siguiente ejemplo se muestra un elemento *section-document*:

Power Query M

```
section Section1;
```

```
A = 1; //1
```

```
B = 2; //2
```

```
C = A + B; //3
```

Las expresiones de miembro de sección pueden hacer referencia a miembros de sección ubicados en otras secciones por medio de *section-access-expression* (expresión de acceso a la sección), que califica un nombre de miembro de sección con el nombre de la sección contenedora.

section-access-expression:

identifier ! identifier

En el ejemplo siguiente se muestra un conjunto de dos documentos que contiene secciones que se hacen referencia mutuamente:

Power Query M

```
section Section1;
A = "Hello";           //"Hello"
B = 1 + Section2!A;     //3

section Section2;
A = 2;                 //2
B = Section1!A & " world!"; //Hello, world"
```

Los miembros de la sección se pueden declarar opcionalmente como `shared`, lo que omite el requisito de usar *section-access-expression* al hacer referencia a miembros compartidos fuera de la sección contenedora. Se puede hacer referencia a los miembros compartidos en secciones externas por su nombre de miembro no calificado siempre que no se declare ningún miembro del mismo nombre en la sección de referencia y que ninguna otra sección tenga un miembro compartido con un nombre similar.

En el ejemplo siguiente se muestra el comportamiento de los miembros compartidos cuando se usan en secciones del mismo conjunto de documentos:

Power Query M

```
section Section1;
shared A = 1;      // 1

section Section2;
B = A + 2;         // 3 (refers to shared A from Section1)

section Section3;
A = "Hello";       // "Hello"
B = A + " world!"; // "Hello world" (refers to local A)
C = Section1!A + 2; // 3
```

Si se define un miembro compartido con el mismo nombre en secciones diferentes, se generará un entorno global válido, pero el acceso al miembro compartido producirá un error.

Power Query M

```
section Section1;
shared A = 1;
```

```
section Section2;  
shared A = "Hello";  
  
section Section3;  
B = A;    //Error: shared member A has multiple definitions
```

Cuando se evalúa un conjunto de elementos section-document, sucede lo siguiente:

- Cada nombre de sección (*section-name*) debe ser único en el entorno global.
- Dentro de una sección, cada *section-member* debe tener un *section-member-name* (nombre de miembro de sección) único.
- Los miembros de sección compartidos con más de una definición generan un error cuando se accede al miembro compartido.
- El componente de expresión de un *section-member* no se debe evaluar antes de que se acceda al miembro de sección.
- Los errores que se producen mientras se evalúa el componente de expresión de un *section-member* se asocian a ese miembro de sección antes de propagarse hacia fuera y, después, se vuelven a generar cada vez que se accede al miembro de sección.

Vinculación de documentos

Se puede vincular un conjunto de documentos de sección de M a un valor de registro opaco que tenga un campo por cada miembro compartido de los documentos de la sección. Si los miembros compartidos tienen nombres ambiguos, se produce un error.

El valor del registro resultante se cierra por completo en el entorno global en el que se ha realizado el proceso de vínculo. Por tanto, estos registros son componentes adecuados para redactar documentos de M desde otros conjuntos (vinculados) de documentos de M. No hay ninguna oportunidad de conflictos de nomenclatura.

Las funciones de la biblioteca estándar `Embedded.Value` se pueden usar para recuperar esos valores de registro "insertados" que se corresponden a componentes de M reutilizados.

Introspección de documentos

M proporciona acceso mediante programación al entorno global por medio de las palabras clave `#sections` y `#shared`.

#sections

La variable intrínseca `#sections` devuelve todas las secciones del entorno global como un registro. Este registro tiene claves de nombre de sección y cada valor es una representación de registro de la sección correspondiente indexada por el nombre de miembro de la sección.

En el ejemplo siguiente se muestra un documento con dos secciones y el registro que se genera al evaluar la variable intrínseca `#sections` en el contexto de ese documento:

Power Query M

```
section Section1;
A = 1;
B = 2;

section Section2;
C = "Hello";
D = "world";

#sections
//[
//  Section1 = [ A = 1, B = 2],
//  Section2 = [ C = "Hello", D = "world" ]
//]
```

Cuando se evalúa `#sections` sucede lo siguiente:

- La variable intrínseca `#sections` conserva el estado de evaluación de todas las expresiones de miembro de sección dentro del documento.
- La variable intrínseca `#sections` no fuerza la evaluación de ningún miembro de sección no evaluado.

#shared

La variable intrínseca `#shared` devuelve el contenido del entorno global como un registro. (El entorno global consta de todos los miembros de sección compartidos, así como de los identificadores incluidos directamente en el entorno global por parte del evaluador de expresiones). Este registro posee una clave por nombre de identificador, y cada valor es el valor del identificador asociado.

En el ejemplo siguiente se muestra un documento con dos miembros compartidos y el registro correspondiente generado al evaluar la variable intrínseca `#shared` en el contexto de ese documento:

Power Query M

```
section Section1;
shared A = 1;
B = 2;

Section Section2;
C = "Hello";
shared D = "world";

//[
//  A = 1,
//  D = "world"
//]
```

Cuando se evalúa `#shared` sucede lo siguiente:

- La variable intrínseca `#shared` conserva el estado de evaluación del entorno global.
- La variable intrínseca `#shared` no fuerza la evaluación de ningún valor no evaluado.

Comentarios

¿Le ha resultado útil esta página?

 Yes

 No

[Obtener ayuda en Microsoft Q&A](#)

Gramática consolidada

Gramática léxica

unidad léxica:

lexical-elements_{opt}

lexical-elements:

lexical-element lexical-elements_{opt}

lexical-element:

whitespace

token

comentario

Espacio en blanco

whitespace:

Cualquier carácter con la clase Zs de Unicode

Carácter de tabulación horizontal (U+0009)

Carácter de tabulación vertical (U+000B)

Carácter de avance de página (U+000C)

Carácter de retorno de carro (U+000D) seguido del carácter de avance de línea (U+000A)

nuevo carácter de línea

new-line-character:

Carácter de retorno de carro (U+000D)

Carácter de avance de línea (U+000A)

Carácter de línea siguiente (U+0085)

Carácter de línea siguiente (U+2028)

Carácter separador de párrafo (U+2029)

Comentario

comentario:

single-line-comment

delimited-comment

single-line-comment:

// *single-line-comment-characters_{opt}*

single-line-comment-characters:

single-line-comment-character single-line-comment-characters_{opt}

single-line-comment-character:

Cualquier carácter Unicode excepto un *carácter de nueva línea*
delimited-comment:

`/` *delimited-comment-text*_{opt} *asteriscos* `/`

delimited-comment-text:

*delimited-comment-section delimited-comment-text*_{opt}

delimited-comment-section:

`/`

*asteriscos*_{opt} *not-slash-or-asterisk*

Asteriscos:

`*` *asteriscos*_{opt}

not-slash-or-asterisk:

Cualquier carácter Unicode menos `*` o `/`

Tokens

seña:

identificador

palabra clave

Literal

operator-or-punctuator

Secuencias de escape de caracteres

character-escape-sequence:

`#` `(` *escape-sequence-list* `)`

escape-sequence-list:

single-escape-sequence

escape-sequence-list `,` *single-escape-sequence*

single-escape-sequence:

long-unicode-escape-sequence

short-unicode-escape-sequence

control-character-escape-sequence

escape-escape

long-unicode-escape-sequence:

hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit

short-unicode-escape-sequence:

hexadecimal de dígito hexadecimal de dígito hexadecimal

control-character-escape-sequence:

control-character

carácter de control:

cr

lf

tab

escape-escape:

#

Literales

literal:

logical-literal

number-literal

text-literal

null-literal

verbatim-literal

logical-literal:

true

false

number-literal:

decimal-number-literal

hexadecimal-number-literal

decimal-digits:

decimal-digit decimal-digits_{opt}

decimal-digit: uno de

0 1 2 3 4 5 6 7 8 9

hexadecimal-number-literal:

0x dígitos hexadecimales

0x dígitos hexadecimales

hex-digits:

hex-digits_{opt}

dígito hexadecimal: uno de

0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

decimal-number-literal:

decimal-digits decimal-digits . exponent-part_{opt}

. *decimal-digits exponent-part_{opt}*

decimal-digits exponent-part_{opt}

exponent-part:

e *sign opt decimal-digits*

E *sign opt decimal-digits*

sign: uno de

+ -

text-literal:

`" text-literal-charactersopt "`

text-literal-characters:

text-literal-character text-literal-characters_{opt}

text-literal-character:

single-text-character

character-escape-sequence

double-quote-escape-sequence

carácter de texto único:

Cualquier carácter excepto `"` (U+0022) o `#` (U+0023) seguido de `(` (U+0028)

double-quote-escape-sequence:

`""` (U+0022, U+0022)

null-literal:

`null`

verbatim-literal:

`#!" text-literal-charactersopt "`

Identificadores

identificador:

regular-identifier

quoted-identifier

regular-identifier:

available-identifier

available-identifier dot-character regular-identifier

available-identifier:

Un *identificador* o *palabra clave* que no es una *palabra clave*

keyword-or-identifier:

letter-character

underscore-character

identifier-start-character identifier-part-characters

identifier-start-character:

letter-character

underscore-character

identifier-part-characters:

identifier-part-character identifier-part-part-characters_{opt}

identifier-part-character:

letter-character

decimal-digit-character

underscore-character

connecting-character

combining-character

formatting-character

generalized-identifier:

generalized-identifier-part

identificador generalizado separado solo por espacios en blanco (U+0020) *generalized-identifier-part*

generalized-identifier-part:

generalized-identifier-segment

decimal-digit-character generalized-identifier-segment

generalized-identifier-segment:

keyword-or-identifier

keyword-or-identifier dot-character keyword-or-identifier

dot-character:

`.` (U+002E)

underscore-character:

`_` (U+005F)

letter-character:

Un carácter Unicode de las clases Lu, Ll, Lt, Lm, Lo o Nl

combining-character:

Un carácter Unicode de las clases Mn o Mc

decimal-digit-character:

Un carácter Unicode de la clase Nd

connecting-character:

Un carácter Unicode de la clase Pc

formatting-character:

Un carácter Unicode de la clase Cf

quoted-identifier:

`#" text-literal-charactersopt "`

Palabras clave e identificadores predefinidos

Los identificadores predefinidos y las palabras clave no se pueden redefinir. Se puede usar un identificador entre comillas para controlar los identificadores que de otro modo entrarían en conflicto con las palabras clave o los identificadores predefinidos.

palabra clave: una de

`and as each else error false if in is let meta not null or otherwise`

section shared then true try type #binary #date #datetime

#datetimezone #duration #infinity #nan #sections #shared #table #time

Operadores y signos de puntuación

operator-or-punctuator: uno de

, ; = < <= > >= <> + - * / & () [] { } @ ? ?? =>

Gramática sintáctica

Documentos

documento:

section-document

expression-document

Documentos de sección

section-document:

section

section:

*literal-attributes*_{opt} **section** *section-name* ; *section-members* *opt*

section-name:

identificador

section-members:

section-member *section-members*_{opt}

section-member:

*literal-attributes*_{opt} *shared*_{opt} *section-member-name* = *expression* ;

section-member-name:

identificador

Documentos de expresión

Expresiones

expression-document:

expresión

expression:

logical-or-expression

each-expression

function-expression

let-expression

if-expression

error-raising-expression

error-handling-expression

Expresiones lógicas

logical-or-expression:

logical-and-expression

logical-and-expression **or** *logical-or-expression*

logical-and-expression:

is-expression

logical-and-expression *is-expression* **and**

Expresión is

is-expression:

as-expression

is-expression **is** *tipo-primitivo-o-nulable-primitivo*

Expresión as

as-expression:

equality-expression

as-expression **as** *tipo primitivo o primitivo anulable*

Expresión de igualdad

equality-expression:

relational-expression

relational-expression **=** *equality-expression*

relational-expression **<>** *equality-expression*

Expresión relacional

relational-expression:

additive-expression

additive-expression **<** *relational-expression*

additive-expression **>** *relational-expression*
additive-expression **<=** *relational-expression*
additive-expression **>=** *relational-expression*

Expresiones aritméticas

additive-expression:

multiplicative-expression
multiplicative-expression **+** *additive-expression*
multiplicative-expression **-** *additive-expression*
multiplicative-expression **&** *_additive-expression*

multiplicative-expression:

metadata-expression
metadata-expression ***** *multiplicative-expression*
metadata-expression **/** *multiplicative-expression*

Expresión de metadatos

metadata-expression:

unary-expression
unary-expression **meta** *unary-expression*

Expresión unaria

unary-expression:

type-expression
+ *unary-expression*
- *unary-expression*
not *unary-expression*

Expresión primaria

primary-expression:

literal-expression
list-expression
record-expression
identifier-expression
section-access-expression
parenthesized-expression

field-access-expression
item-access-expression
invoke-expression
not-implemented-expression

Expresión literal

literal-expression:
literal

Expresión de identificador

identifier-expression:
identifier-reference
identifier-reference:
exclusive-identifier-reference
inclusive-identifier-reference
exclusive-identifier-reference:
identificador
inclusive-identifier-reference:
@ identificador

Expresión de acceso de sección

section-access-expression:
identificador de identificador !

Expresión entre paréntesis

parenthesized-expression:
(expresión)

Expresión no implementada

not-implemented-expression:
...

Expresión de invocación

invoke-expression:

primary-expression (*argument-list*_{opt})

argument-list:

expresión

expresión , *argument-list*

Expresión de lista

list-expression:

{ *participación en la lista*_{de elementos} }

item-list:

item

item-list ,

item:

expresión

expresión ..

Expresión de registro

record-expression:

[*opción de lista*_{de campos}]

lista de campos:

campo

field-list ,

campo:

expresión field-name =

field-name:

generalized-identifier

quoted-identifier

Expresión de acceso a elementos

item-access-expression:

item-selection

optional-item-selection

item-selection:

primary-expression { *item-selector* }

optional-item-selection:

primary-expression { *item-selector* } ?

selector de elementos:

expresión

Expresión de acceso a campos

field-access-expression:

field-selection

implicit-target-field-selection

proyección

implicit-target-projection

field-selection:

primary-expression field-selector

field-selector:

required-field-selector

optional-field-selector

required-field-selector:

[*field-name*]

optional-field-selector:

[*field-name*] ?

implicit-target-field-selection:

field-selector

projection:

primary-expression required-projection

primary-expression optional-projection

required-projection:

[*required-selector-list*]

optional-projection:

[*required-selector-list*] ?

required-selector-list:

required-field-selector

required-field-selector , *required-selector-list*

implicit-target-projection:

required-projection

proyección opcional

Expresión de función

function-expression:

(*parameter-list*_{opt}) *return-type*_{opt} => *function-body*

function-body:

expresión

parameter-list:

fixed-parameter-list

fixed-parameter-list , *optional-parameter-list*

optional-parameter-list

fixed-parameter-list:

parámetro

parameter , *fixed-parameter-list*

parameter:

nombre-de-parámetro *tipo-de-parámetro*_{opcional}

parameter-name:

identificador

parameter-type:

aserción-de-tipo-primitivo-o-primitivo-anulable

return-type:

aserción-de-tipo-primitivo-o-primitivo-anulable

primitive-or-nullable-primitive-type-assertion:

as *tipo primitivo o primitivo anulable*

optional-parameter-list:

optional-parameter

optional-parameter , *optional-parameter-list*

optional-parameter:

optional *parámetro*

Expresión each

each-expression:

each *each-expression-body*

each-expression-body:

cuerpo de la función

Expresión let

let-expression:

let *expresión variable-list* **in**

variable-list:

variable

variable , *variable-list*

variable:

expresión variable-name **=**

variable-name:
identificador

Expresión if

if-expression:
`if if-condition then true-expression else false-expression`

if-condition:

expresión

true-expression:

expresión

false-expression:

expresión

Expresión de tipo

type-expression:

primary-expression

`type` *tipo principal*

type:

primary-expression

primary-type

primary-type:

tipo-primitivo-o-tipo-primitivo-anulable

record-type

list-type

function-type

table-type

tipo anulable

tipo primitivo o tipo primitivo anulable:

`nullable` *opt* *primitive-type*

tipo primitivo: uno de

`any anyonnull binary date datetime datetimezone duration function list logical`

`none null number record table text time type`

record-type:

`[ open-record-marker ]`

`[ field-specification-listopt ]`

`[ field-specification-list , open-record-marker ]`

field-specification-list:

field-specification

field-specification , *field-specification-list*

especificación de campo:

*optional*_{opt} *field-name* *field-type-specification*_{opt}

field-type-specification:

= *tipo de campo*

field-type:

tipo

open-record-marker:

...

list-type:

{ *item-type* }

item-type:

tipo

tipo de función:

function (*parameter-specification-list*_{opt}) *return-type*

parameter-specification-list:

required-parameter-specification-list

required-parameter-specification-list , *optional-parameter-specification-list*

optional-parameter-specification-list

required-parameter-specification-list:

required-parameter-specification

required-parameter-specification , *required-parameter-specification-list*

required-parameter-specification:

parameter-specification

optional-parameter-specification-list:

optional-parameter-specification

optional-parameter-specification , *optional-parameter-specification-list*

optional-parameter-specification:

optional *parameter-specification*

parameter-specification:

parameter-name *parameter-type*

parameter-type:

aseveración de tipo

type-assertion:

as *tipo*

tipo de tabla:

table *tipo de fila*

tipo de fila:

[*field-specification-list*_{opt}]

nullable-type:

`nullable` *tipo*

Expresión de generación de errores

error-raising-expression:

`error` *expresión_*

Expresión de control de errores

error-handling-expression:

`try` *protected-expression* *error-handler*_{opt}

protected-expression:

expresión

error-handler:

otherwise-clause

catch-clause

de lo contrario, cláusula:

`otherwise` *default-expression*

default-expression:

expression

catch-clause:

`catch` *catch-function*

catch-function:

`( )`

Atributos literales

literal-attributes:

record-literal

record-literal:

`[` *literal-field-list*_{opt} `]`

literal-field-list:

literal-field

literal-field , *literal-field-list*

literal-field:

field-name = *any-literal*

list-literal:

`{` *literal-item-list*_{opt} `}`

literal-item-list:

any-literal

any-literal-item-list ,

any-literal:

record-literal

list-literal

logical-literal

number-literal

text-literal

literal null

Last updated on 22/11/2025

Tipos del lenguaje de fórmulas M de Power Query

El lenguaje de fórmulas M de Power Query es un lenguaje de mashup de datos útil y expresivo. Pero tiene algunas limitaciones. Por ejemplo, no hay ninguna aplicación segura del sistema de tipos. En algunos casos, se necesita una validación más rigurosa. Afortunadamente, en M se proporciona una biblioteca integrada compatible con tipos para hacer factible una validación más segura.

Los desarrolladores deben tener un conocimiento exhaustivo del sistema de tipos para poder hacer esto con generalidad. Y, aunque en la especificación del lenguaje M de Power Query se explica correctamente el sistema de tipos, hay algunas sorpresas. Por ejemplo, para la validación de instancias de función se necesita una manera de comparar la compatibilidad de los tipos.

Si se explora el sistema de tipos de M con mayor detenimiento, muchos de estos problemas se pueden aclarar y los desarrolladores estarán capacitados para diseñar las soluciones que necesitan.

Para comprender la notación que se usa, debería bastar con conocimientos sobre el cálculo de predicados y la teoría informal de conjuntos.

PASOS PRELIMINARES

(1) $B := \{ \text{true}; \text{false} \}$

B es el conjunto típico de valores booleanos

(2) $N := \{ \text{identificadores válidos de M} \}$

N es el conjunto de todos los nombres válidos en M. Esto se define en otra parte.

(3) $P := \langle B, T \rangle$

P es el conjunto de parámetros de función. Cada uno es posiblemente opcional y tiene un tipo. Los nombres de parámetro son irrelevantes.

(4) $P^n := \bigcup_{0 \leq i \leq n} \langle i, P^i \rangle$

P^n es el conjunto de todas las secuencias ordenadas de n parámetros de función.

(5) $P := \bigcup P^i$

P^ es el conjunto de todas las secuencias posibles de parámetros de función, desde la longitud 0 en adelante.*

(6) $F := \langle B, N, T \rangle$

F es el conjunto de todos los campos de registro. Cada campo es posiblemente opcional, tiene un

nombre y un tipo.

$$(7) F^n := \prod_{0 \leq i \leq n} F$$

F^n es el conjunto de todos los n campos de registro.

$$(8) F := (\bigcup_{0 \leq i \leq \infty} F_i) \setminus \{F \mid \langle b^1, n_1, t_1 \rangle, \langle b_2, n_2, t_2 \rangle \in F \wedge =$$

F^ es el conjunto de todos los conjuntos (de cualquier longitud) de campos de registro, excepto los conjuntos en los que más de un campo tiene el mismo nombre.*

$$(9) C := \langle N, T \rangle$$

C es el conjunto de tipos de columna, para las tablas. Cada columna tiene un nombre y un tipo.

$$(10) C^n \subset \bigcup_{0 \leq i \leq n} \langle i, C \rangle$$

C^n es el conjunto de todas las secuencias ordenadas de n tipos de columna.

$$(11) C := (\bigcup_{0 \leq i \leq \infty} C_i) \setminus \{C^m \mid \langle a, \langle n_1, t_1 \rangle \rangle, \langle b, \langle n_2, t_2 \rangle \rangle \in C_m \wedge =$$

C^ es el conjunto de todas las combinaciones (de cualquier longitud) de tipos de columna, excepto aquellas en las que más de una columna tiene el mismo nombre.*

TIPOS DE M

$$(12) T_F := \langle P, P^* \rangle$$

Un tipo de función consta de un tipo de valor devuelto y una lista ordenada de cero o más parámetros de función.

$$(13) T_L := \llbracket T \rrbracket$$

Un tipo de lista se indica mediante un tipo determinado (denominado "tipo de elemento") incluido entre llaves. Puesto que las llaves se usan en el metalanguage, $\llbracket \rrbracket$ se usan corchetes en este documento.

$$(14) T_R := \langle B, F^* \rangle$$

Un tipo de registro tiene una marca que indica si está abierto y tiene cero o más campos de registro sin ordenar.

$$(15) T_R^0 := \langle \text{true}, F \rangle$$

$$(16) T_R^* := \langle \text{false}, F \rangle$$

T_R^0 and T_R^ son métodos abreviados de notación para los tipos de registro abiertos y cerrados, respectivamente.*

$$(17) T_T := C^*$$

Un tipo de tabla es una secuencia ordenada de cero o más tipos de columna, donde no hay conflictos de nombres.

(18) $T_P := \{ \text{any; none; null; logical; number; time; date; datetime; duration; text; binary; type; list; record; table; function; anynonnull} \}$

Un tipo primitivo es uno de esta lista de palabras clave de M.

(19) $T_N := \{ t_n, u \in T \mid t_n = u + \text{null} \} = \text{nullable } t$

Además, cualquier tipo se puede marcar como que acepta valores NULL mediante la palabra clave "nullable" .

(20) $T := T_F \cup T_L \cup T_R \cup T_{TTT} \cup T_P \cup T_N$

El conjunto de todos los tipos de M es la unión de estos seis conjuntos de tipos:

tipos de función, tipos de lista, tipos de registros, tipos de tabla, tipos primitivos y tipos que aceptan valores NULL.

FUNCIONES

Una función se debe definir: $\text{NonNullable} : T \leftarrow T$

Esta función toma un tipo y devuelve un tipo que es equivalente, salvo que no se ajusta al valor NULL.

IDENTIDADES

Algunas identidades son necesarias para definir algunos casos especiales y también pueden ayudar a aclarar lo anterior.

(21) acepta valores NULL cualquiera = any

(22) anynonnull que acepta valores NULL = any

(23) NULL que acepta valores NULL = NULL

(24) nullable none = NULL

(25) t que acepta valores NULL acepta valores NULL $\in T = t$ que acepta valores NULL

(26) $\text{NonNullable}(\text{nullable } t \in T) = \text{NonNullable}(t)$

(27) $\text{NonNullable}(\text{any}) = \text{anynonnull}$

COMPATIBILIDAD DE TIPOS

Como se ha definido en otro lugar, un tipo de M es compatible con otro tipo de M solo si todos los valores que se ajustan al primer tipo también se ajustan al segundo.

Aquí se define una relación de compatibilidad que no depende de los valores que se ajustan y se basa en las propiedades de los propios tipos. Se asume que esta relación, como se define en este documento, es totalmente equivalente a la definición semántica original.

La relación "es compatible con": $\leq : B \leftarrow T \times T$

En la sección siguiente, una t minúscula siempre representa un tipo de M , un elemento de T .

Un símbolo Φ representará un subconjunto de F^* , o bien de C^* .

(28) $t \leq t$

Esta relación es reflexiva.

(29) $t_a \leq t_b \wedge t_b \leq t_c \rightarrow t_a \leq t_c$

Esta relación es transitiva.

(30) ninguna $\leq no \leq$ ninguna

Los tipos de M forman un enlace sobre esta relación; nonees la parte inferior yany la superior.

(31) $t_a, t_b \in T_N \wedge t_a \leq t_b \rightarrow NonNullable(t_a) \leq NonNullable(t_b)$

Si dos tipos son compatibles, los equivalentes NonNullable también son compatibles.

(32) $\leq t \in T_N \text{ null}$

El tipo primitivo null es compatible con todos los tipos que aceptan valores NULL.

(33) $t \notin T_N \leq \text{anynonnull}$

Todos los tipos que no aceptan valores NULL son compatibles con anynonnull.

(34) $NonNullable(t) \leq t$

Un tipo que no acepta valores NULL es compatible con el equivalente que acepta valores NULL.

(35) $t \in \text{función } T_F \rightarrow t \leq$

Todos los tipos de función son compatibles con función.

(36) $t \in T_L \rightarrow t \leq \text{lista}$

Todos los tipos de lista son compatibles con lista.

(37) $t \in \text{registro } T_R \rightarrow t \leq$

Todos los tipos de registro son compatibles con registro.

(38) $t \in T_T \rightarrow t \leq \text{tabla}$

Todos los tipos de tabla son compatibles con tabla.

(39) $t_a \leq t_b \leftrightarrow \llbracket t_a \rrbracket \leq \llbracket t_b \rrbracket$

Un tipo de lista es compatible con otro tipo de lista si los tipos de elemento son compatibles y viceversa.

(40) $t_a \in T_F = \langle p_a, p^* \rangle, t_b \in T_F = \langle p_b, p^* \rangle \wedge p_a \leq p \rightarrow t_a \leq t_b$

Un tipo de función es compatible con otro tipo de función si los tipos devueltos son compatibles y las listas de parámetros son idénticas.

$$(41) t_a \in T_R^0, t_b \in T_R^* \rightarrow t_a \leq t_b$$

Un tipo de registro abierto nunca es compatible con un tipo de registro cerrado.

$$(42) t_a \in T_R^* = \langle \text{false}, N \rangle, t_b \in T_R^0 = \langle \text{true}, N \rangle \rightarrow t_a \leq t_b$$

Un tipo de registro cerrado es compatible con un tipo de registro abierto que, de lo contrario, es idéntico.

$$(43) t_a \in T_R^0 = \langle \text{true}, (N, \langle \text{true}, n, \text{any} \rangle) \rangle, t_b \in T_R^0 = \langle \text{true}, N \rangle \rightarrow t_a \leq t_b \wedge t_b \leq t_a$$

Se puede omitir un campo opcional con el tipo any al comparar dos tipos de registro abiertos.

$$(44) t_a \in T_R = \langle b, (WP, \langle \beta, n, u_a \rangle) \rangle, t_b \in T_R = \langle b, (WP, \langle \beta, n, u_b \rangle) \rangle \wedge u_a \leq u_b \rightarrow t_a \leq t_b$$

Dos tipos de registro que solo difieren en un campo son compatibles si el nombre y la opcionalidad del campo son idénticos, y los tipos de ese campo son compatibles.

$$(45) t_a \in T_R = \langle b, (WP, \langle \text{false}, n, u \rangle) \rangle, t_b \in T_R = \langle b, (WP, \langle \text{true}, n, u \rangle) \rangle \rightarrow t_a \leq t_b$$

Un tipo de registro con un campo no opcional es compatible con un tipo de registro idéntico, pero opcional para ese campo.

$$(46) t_a \in T_R^0 = \langle \text{true}, (WP, \langle b, n, u \rangle) \rangle, t_b \in T_R^0 = \langle \text{true}, WP \rangle \rightarrow t_a \leq t_b$$

Un tipo de registro abierto es compatible con otro tipo de registro abierto con un campo menos.

$$(47) t_a \in T = (WP, \langle i, \langle n, u_a \rangle \rangle), t_b \in T = (WP, \langle i, \langle n, u_b \rangle \rangle) \wedge u_a \leq u_b \rightarrow t_a \leq t_b$$

Un tipo de tabla es compatible con un segundo tipo de tabla, que es idéntico pero para una columna con otro tipo, cuando los tipos de esa columna son compatibles.

Contenido relacionado

- [Especificación del lenguaje M de Power Query](#)
- [Referencia de la función M de Power Query](#)
- [Tipos y de conversión de tipos](#)
- [Tipos](#)

Expresiones, valores y expresión Let

15/08/2025

Una consulta del lenguaje de fórmulas M de Power Query se compone de una serie de pasos de **expresión** de fórmula que crean una consulta de mashup. Una expresión de fórmula se puede evaluar (calcular), lo que produce un valor. La expresión **let** encapsula un conjunto de valores que se van a calcular, a los que se van a asignar nombres que y, por último, se van a usar en una expresión subsiguiente que sigue a la instrucción **in**. Por ejemplo, una expresión **let** podría contener una variable **Source** que es igual al valor de **Text.Proper** y produce un valor de texto en el caso adecuado.

Expresión let

Power Query M

```
let
    Source = Text.Proper("hello world")
in
    Source
```

En este ejemplo, `Text.Proper("hello world")` se evalúa como `"Hello World"`.

En las siguientes secciones se describen los tipos de valor en el lenguaje.

Valor primitivo

Un valor *primitivo* es un valor de una sola parte, como , `number` `logical`, `text` o `null` Se puede usar un `null` valor para indicar la ausencia de datos.

 Expandir tabla

Tipo	Valor de ejemplo
Binary	00 00 00 02 // número de puntos (2)
Date	23/5/2015
DateTime	23/5/2015 12:00:00 a.m.
Fecha, hora y zona horaria	23/5/2015 12:00:00 a.m. - 08:00
Duration	15:35:00

Tipo	Valor de ejemplo
Lógicos	true y false
Null	null
Número	0, 1, -1, 1.5, y 2.3e-5
Texto	"abc"
Hora	12:34:12 p.m.

Valor de función

Una *función* es un valor que, cuando se invoca con argumentos, genera un nuevo valor. Las funciones se escriben mediante la enumeración de los *parámetros* de la función en cuestión entre paréntesis, seguidos del signo igual =>, seguido de la expresión que define la función. Por ejemplo, para crear una función llamada **MyFunction** que tiene dos parámetros y realiza un cálculo en parameter1 y parameter2:

Power Query M

```
let
    MyFunction = (parameter1, parameter2) => (parameter1 + parameter2) / 2
in
    MyFunction
```

Al llamar a **MyFunction** se devuelve el resultado:

Power Query M

```
let
    Source = MyFunction(2, 4)
in
    Source
```

Este código genera un valor de 3.

Valores de datos estructurados

El lenguaje M admite los siguientes valores de datos estructurados:

- [Lista](#)
- [Registro](#)

- [Tabla](#)
- [Otros ejemplos de datos estructurados](#)

ⓘ Nota

Los datos estructurados pueden contener cualquier valor de M. Para ver un par de ejemplos, vaya a [Ejemplos de datos estructurados adicionales](#).

List

Una *lista* es una secuencia ordenada de base cero de valores delimitados por caracteres de llave { }. Los caracteres de llave { } también se usan para recuperar un elemento de una lista por su posición de índice. Para obtener más información, vaya a [Valores de lista](#).

ⓘ Nota

M de Power Query admite un tamaño de lista infinito, pero si una lista se escribe como un literal, debe tener una longitud fija. Por ejemplo, {1, 2, 3} tiene una longitud fija de 3.

Estos son algunos ejemplos de listas.

[Expandir tabla](#)

Valor	Tipo
{123, true, "A"}	Lista que contiene un número, un valor lógico y un texto.
{1, 2, 3}	Lista de números.
{ {1, 2, 3}, {4, 5, 6} }	Lista de una lista de números.
{ [CustomerID = 1, Name = "Bob", Phone = "123-4567"], [CustomerID = 2, Name = "Jim", Phone = "987-6543"] }	Lista de registros.
{123, true, "A"}{0}	Obtiene el valor del primer elemento de una lista. Esta expresión devuelve el valor 123.

Valor	Tipo
<pre>{ {1, 2, 3}, {4, 5, 6} }{0}{1}</pre>	Obtiene el valor del segundo elemento del primer elemento de lista. Esta expresión devuelve el valor 2.

Registro

Un *registro* es un conjunto de campos. Un *campo* es un par nombre-valor donde el nombre es un valor de texto que es único dentro del registro del campo. La sintaxis de los valores de registro permite escribir los nombres sin comillas, formato que también se conoce como *identificadores*. Un identificador puede tener las dos formas siguientes:

- nombre_identificador, como OrderID.
- #"nombre del identificador", como #"Today's data is:".

Aquí mostramos un registro que contiene los campos "OrderID", "CustomerID", "Item" y "Price" con los valores 1, 1, "Fishing rod" y 100.00. Los caracteres de corchete [] señalan el principio y el final de una expresión de registro, y se usan para obtener un valor de campo de un registro. En los siguientes ejemplos se muestra un registro y cómo obtener el valor del campo Item.

Este es un registro de ejemplo:

```
Power Query M

let
    Source =
        [
            OrderID = 1,
            CustomerID = 1,
            Item = "Fishing rod",
            Price = 100.00
        ]
in
    Source
```

Para obtener el valor de Item, use corchetes, como, por ejemplo, `Source[Item]`:

```
Power Query M

let Source =
    [
        OrderID = 1,
        CustomerID = 1,
        Item = "Fishing rod",
```

```
        Price = 100.00
    ],
    GetItem = Source[Item] //equals "Fishing rod"
in
    GetItem
```

Tabla

Una *tabla* es un conjunto de valores organizados en columnas y filas con nombre. El tipo de columna puede ser implícito o explícito. Puede usar `#table` para crear una lista de nombres de columna y una lista de filas. Una tabla de valores es una lista dentro de una [lista](#). Los caracteres de llave { } también se usan para recuperar una fila de una tabla por su posición de índice (vaya al [Ejemplo 3: obtener una fila de una tabla por su posición de índice](#)).

Ejemplo 1: crear una tabla con tipos de columna implícitos

Power Query M

```
let
    Source = #table(
        {"OrderID", "CustomerID", "Item", "Price"},
        {
            {1, 1, "Fishing rod", 100.00},
            {2, 1, "1 lb. worms", 5.00}
        }
    )
in
    Source
```

Ejemplo 2: crear una tabla con tipos de columna explícitos

Power Query M

```
let
    Source = #table(
        type table [OrderID = number, CustomerID = number, Item = text, Price =
number],
        {
            {1, 1, "Fishing rod", 100.00},
            {2, 1, "1 lb. worms", 5.00}
        }
    )
in
    Source
```

En los dos ejemplos anteriores se crea una tabla con la siguiente forma:

[Expandir tabla](#)

OrderID	CustomerID	Elemento	Precio
1	1	Caña de pescar	100,00
2	1	1 libra de gusanos	5.00

Ejemplo 3: obtener una fila de una tabla por su posición de índice

Power Query M

```
let
    Source = #table(
        type table [OrderID = number, CustomerID = number, Item = text, Price =
number],
        {
            {1, 1, "Fishing rod", 100.00},
            {2, 1, "1 lb. worms", 5.00}
        }
    )
in
    Source{1}
```

Esta expresión devuelve el siguiente registro:

[Expandir tabla](#)

Campo	Valor
OrderID	2
CustomerID	1
Elemento	1 libra de gusanos
Precio	5

Otros ejemplos de datos estructurados

Los datos estructurados pueden contener cualquier valor de M. A continuación se muestran algunos ejemplos:

Ejemplo 1: lista con valores [Primitive](#_Primitive_value_1), [Function](#_Function_value) y [Record](#_Record_value)

Power Query M

```
let
    Source =
    {
        1,
        "Bob",
        DateTime.ToText(DateTime.LocalNow(), "yyyy-MM-dd"),
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0]
    }
in
    Source
```

La evaluación de esta expresión se puede visualizar del siguiente modo:

A List containing a Record	
1	
"Bob"	
2015-05-22	
OrderID	1
CustomerID	1
Item	"Fishing rod"
Price	100.0

Ejemplo 2: registro que contiene valores primitivos y registros anidados

Power Query M

```
let
    Source = [CustomerID = 1, Name = "Bob", Phone = "123-4567", Orders =
        {
            [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
            [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0]
        }
    ]
```

in

Source

La evaluación de esta expresión se puede visualizar del siguiente modo:

A record containing a List of Records		
CustomerID	1	
Name	"Bob"	
Phone	"123-4567"	
Orders	OrderID	1
	CustomerID	1
	Item	"Fishing rod"
	Price	100.0
	OrderID	2
	CustomerID	1
	Item	"1 lb. worms"
	Price	5.0

ⓘ Nota

Aunque muchos valores se pueden escribir literalmente como una expresión, un valor no es una expresión. Por ejemplo, la expresión 1 se evalúa como el valor 1, mientras que la expresión 1 + 1 se evalúa como el valor 2. Esta distinción es sutil, pero importante. Las expresiones son las recetas de evaluación y los valores, los resultados de esa evaluación.

Expresión if

La expresión if opta por una de dos expresiones posibles en función de una condición lógica. Por ejemplo:

```
if 2 > 1 then
  2 + 2
else
  1 + 1
```

La primera expresión (2 + 2) está seleccionada si la expresión lógica (2 > 1) es true y la segunda expresión (1 + 1) está seleccionada si es false. La expresión seleccionada (en este caso, 2 + 2) se evalúa y se convierte en el resultado de la expresión **if** (4).

Comentarios

Puede agregar comentarios al código con comentarios `//` de una sola línea o comentarios de varias líneas que comiencen por `/*` y terminen con `*/`.

Ejemplo: comentario de una sola línea

Power Query M

```
let
    //Convert to proper case.
    Source = Text.Proper("hello world")
in
    Source
```

Ejemplo: comentario de varias líneas

Power Query M

```
/* Capitalize each word in the Item column in the Orders table. Text.Proper
is evaluated for each Item in each table row. */
let
    Orders = Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "fishing net", Price = 25.0]}),
    #"Capitalized Each Word" = Table.TransformColumns(Orders, {"Item",
Text.Proper})
in
    #"Capitalized Each Word"
```

Modelo de evaluación

El modelo de evaluación del lenguaje de fórmulas M de Power Query se modela después de que el modelo de evaluación se encuentre normalmente en hojas de cálculo, donde el orden de los cálculos se puede determinar en función de las dependencias entre las fórmulas de las celdas.

Si ha escrito fórmulas en una hoja de cálculo como Excel, puede reconocer que las fórmulas de la izquierda darán lugar a los valores de la derecha cuando se calculen:

	A
1	=A2 * 2
2	=A3 + 1
3	1

de

	A
1	4
2	2
3	1

En M, una expresión puede hacer referencia a expresiones anteriores por nombre y el proceso de evaluación determinará automáticamente el orden en el que se calculan las expresiones a las que se hace referencia.

Vamos a usar un registro para generar una expresión que es equivalente al ejemplo de hoja de cálculo anterior. Al inicializar el valor de un campo, se hace referencia a otros campos del registro por el nombre del campo, como se indica a continuación:

Power Query M

```
[  
    A1 = A2 * 2,  
    A2 = A3 + 1,  
    A3 = 1  
]
```

La expresión anterior se evalúa como el registro siguiente:

Power Query M

```
[  
    A1 = 4,  
    A2 = 2,  
    A3 = 1  
]
```

Los registros se pueden contener dentro de otros registros o **anidados**. Puede usar el **operador de búsqueda** ([]) para acceder a los campos de un registro por nombre. Por ejemplo, el siguiente registro tiene un campo denominado Sales que contiene un registro y un campo denominado Total que tiene acceso a los campos FirstHalf y SecondHalf del registro Sales:

Power Query M

```
[
    Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
    Total = Sales[FirstHalf] + Sales[SecondHalf]
]
```

La expresión anterior se evalúa como el registro siguiente:

Power Query M

```
[
    Sales = [ FirstHalf = 1000, SecondHalf = 1100 ],
    Total = 2100
]
```

Use el **operador de índice posicional** ({ }) para tener acceso a un elemento de una lista por su índice numérico. Los valores de una lista se conocen mediante un índice de base cero desde el principio de la lista. Por ejemplo, los índices 0 y 1 se usan para hacer referencia a los elementos primero y segundo de la lista siguiente:

Power Query M

```
[
    Sales =
        {
            [
                Year = 2007,
                FirstHalf = 1000,
                SecondHalf = 1100,
                Total = FirstHalf + SecondHalf // equals 2100
            ],
            [
                Year = 2008,
                FirstHalf = 1200,
                SecondHalf = 1300,
                Total = FirstHalf + SecondHalf // equals 2500
            ]
        },
    #"Total Sales" = Sales{0}[Total] + Sales{1}[Total] // equals 4600
]
```

Evaluación diferida y diligente

Las expresiones de miembro **List**, **Record** y **Table** , así como las expresiones **let** (Ir a [expresiones, valores y expresión let](#)), se evalúan mediante la **evaluación diferida**. Es decir, se evalúan cuando es necesario. Todas las demás expresiones se evalúan mediante la **evaluación diligente**. Es decir, se evalúan inmediatamente cuando se encuentran durante el proceso de evaluación. Una buena manera de pensar en esto es recordar que la evaluación de una expresión de lista o registro devolverá un valor de lista o registro que sepa cómo sus elementos de lista o campos de registro deben calcularse, cuando se solicitan (por operadores de búsqueda o índice).

Last updated on 22/11/2025

Operadores

14/06/2025

El lenguaje de fórmulas M de Power Query incluye un conjunto de operadores que se pueden usar en una expresión. Los **operadores** se aplican a los **operandos** para formar expresiones simbólicas. Por ejemplo, en la expresión `1 + 2` los números `1` y `2` son operandos `1` y el operador es el operador de suma `(+)`.

El significado de un operador puede variar en función del tipo de valores de operando. El idioma tiene los operadores siguientes:

Operador Plus (+)

[Expandir tabla](#)

Expresión	Iguals
<code>1 + 2</code>	Suma numérica: <code>3</code>
<code>#time(12,23,0) + #duration(0,0,2,0)</code>	Aritmética de tiempo: <code>#time(12,25,0)</code>

Operador de combinación (&)

[Expandir tabla](#)

Función	Iguals
<code>"A" & "BC"</code>	Concatenación de texto: <code>"ABC"</code>
<code>{1} & {2, 3}</code>	Concatenación de listas: <code>{1, 2, 3}</code>
<code>[a = 1] & [b = 2]</code>	Combinación de registros: <code>[a = 1, b = 2]</code>

Lista de operadores M

Operadores comunes que se aplican a `null`, `logical`, `number`, `time`, `date`, `datetime`, `datetimezone`, `duration`, `text`, `binary`

[Expandir tabla](#)

Operador	Descripción
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que
=	Igual
<>	No es igual
??	Fusión nula

Operadores lógicos (además de operadores comunes)

 Expandir tabla

Operador	Descripción
or	Lógico condicional OR
and	Lógico condicional AND
not	Lógico NOT

Operadores de número (además de operadores comunes)

 Expandir tabla

Operador	Descripción
+	Suma
-	Diferencia
*	Producto
/	Cociente
+x	Suma unaria
-x	Negación

Operadores de texto (además de operadores comunes)

[Expandir tabla](#)

Operador	Descripción
<code>&</code>	Concatenación

Enumeración, registro, operadores de tabla

[Expandir tabla](#)

Operador	Descripción
<code>=</code>	Igual
<code><></code>	No es igual
<code>&</code>	Concatenación

Operador de búsqueda de registros

[Expandir tabla](#)

Operador	Descripción
<code>[]</code>	Acceda a los campos de un registro por nombre.

Operador de indexador de lista

[Expandir tabla](#)

Operador	Descripción
<code>{}</code>	Acceda a un elemento de una lista por su índice numérico de base cero.

Compatibilidad de tipos y operadores de aserción

[Expandir tabla](#)

Operador	Descripción
<code>is</code>	La expresión <code>x is y</code> devuelve <code>true</code> si el tipo de <code>x</code> es compatible con <code>y</code> y devuelve <code>false</code> si el tipo de <code>x</code> no es compatible con <code>y</code> .
<code>as</code>	La expresión <code>x as y</code> afirma que el valor <code>x</code> es compatible con <code>y</code> según el <code>is</code> operador .

Operadores de fecha

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>time</code>	<code>duration</code>	Desplazamiento de fecha por duración
<code>x + y</code>	<code>duration</code>	<code>time</code>	Desplazamiento de fecha por duración
<code>x - y</code>	<code>time</code>	<code>duration</code>	Desplazamiento de fecha por duración negada
<code>x - y</code>	<code>time</code>	<code>time</code>	Duración entre fechas
<code>x & y</code>	<code>date</code>	<code>time</code>	Fecha y hora combinadas

Operadores de fecha y hora

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Desplazamiento de fecha y hora por duración
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración negada
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Duración entre fechas y horas

Operadores de fecha y hora con zona horaria

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetimezone</code>	<code>duration</code>	Desplazamiento de zona horaria de fecha según la duración
<code>x + y</code>	<code>duration</code>	<code>datetimezone</code>	Desplazamiento de zona horaria de fecha según la duración
<code>x - y</code>	<code>datetimezone</code>	<code>duration</code>	Desplazamiento de zona horaria de fecha según la duración negada
<code>x - y</code>	<code>datetimezone</code>	<code>datetimezone</code>	Duración entre zonas horarias de fecha

Operadores de duración

[Expandir tabla](#)

Operador	Operando izquierdo	Operando derecho	Significado
<code>x + y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración
<code>x + y</code>	<code>duration</code>	<code>datetime</code>	Desplazamiento de fecha y hora por duración
<code>x + y</code>	<code>duration</code>	<code>duration</code>	Suma de las duraciones
<code>x - y</code>	<code>datetime</code>	<code>duration</code>	Desplazamiento de fecha y hora por duración negada
<code>x - y</code>	<code>datetime</code>	<code>datetime</code>	Duración entre fechas y horas
<code>x - y</code>	<code>duration</code>	<code>duration</code>	Diferencia de duraciones
<code>x * y</code>	<code>duration</code>	<code>number</code>	N veces una duración
<code>x * y</code>	<code>number</code>	<code>duration</code>	N veces una duración
<code>x / y</code>	<code>duration</code>	<code>number</code>	Fracción de una duración

⚠ Nota

No todas las combinaciones de valores pueden ser compatibles con un operador. Expresiones que, cuando se evalúan, encuentran condiciones de operador no definidas se evalúan como errores. Para obtener más información sobre los errores en M, vaya a [Errores](#).

Ejemplo de error:

[Expandir tabla](#)

Función	Iguals
<code>1 + "2"</code>	Error: no se admite agregar el número y el texto

Tipos y conversión de tipos

Power Query M usa tipos para clasificar valores para tener un conjunto de datos más estructurado. En este artículo se describen los tipos de M más usados y cómo convertir un tipo a otro.

Tipos usados habitualmente

Los tipos de datos hacen referencia a cualquier tipo que se use para aclarar la estructura de datos específicos. Los tipos de datos más usados son tipos primitivos. Estos tipos incluyen:

- `type any`, que clasifica cualquier valor.
- `type null`, que clasifica el valor NULL.
- `type logical`, que clasifica los valores true y false.
- `type number`, que clasifica valores numéricos.
- `type time`, que clasifica valores de hora.
- `type date`, que clasifica valores de fecha.
- `type datetime`, que clasifica valores de fecha y hora.
- `type datetimezone`, que clasifica valores de fecha, hora y zona horaria.
- `type duration`, que clasifica valores de duración.
- `type text`, que clasifica valores de texto.
- `type binary`, que clasifica valores binarios.
- `type type`, que clasifica valores de tipo.
- `type list`, que clasifica valores de lista.
- `type record`, que clasifica valores de registro.
- `type table`, que clasifica valores de tabla.
- `type function`, que clasifica valores de función.
- `type anynonnull`, que clasifica todos los valores menos NULL.
- `type none`, que no clasifica valores.

Para obtener más información sobre estos tipos, vaya a [Tipos](#).

Además de estos tipos de datos comunes, también hay un conjunto de tipos de datos con el formato `*.Type`. Los tipos de datos más usados de este formato son:

- `Byte.Type`, que clasifica un valor numérico de 8 bits.
- `Int8.Type`, que clasifica un valor numérico de 8 bits.
- `Int16.Type`, que clasifica un valor numérico de 16 bits.
- `Int32.Type`, que clasifica un valor numérico de 32 bits.
- `Int64.Type`, que clasifica un valor numérico de 64 bits.

- `Single.Type`, que clasifica un valor de número flotante de 9 dígitos.
- `Double.Type`, que clasifica un valor de número flotante de 17 dígitos.
- `Decimal.Type`, que clasifica un valor de número flotante de 15 dígitos.
- `Currency.Type`, que clasifica un valor de número de 19 dígitos con cuatro dígitos a la derecha del separador ".".
- `Percentage.Type`, que clasifica un valor de número de 15 dígitos con una máscara para dar formato al valor como porcentaje.
- `Guid.Type`, que clasifica un valor de texto GUID.

Los tipos primitivos también se pueden escribir en el `*.Type` formato. Por lo tanto, puede escribir `number` como `Number.Type`, `record` como `Record.Type`, etc.

Cuando use cualquiera de estos tipos, tenga en cuenta que, al igual que todo el código M, estos tipos distinguen mayúsculas de minúsculas.

La tabla siguiente contiene más información sobre cada uno de estos tipos.

 Expandir tabla

Tipo de dato	Descripción
<code>any</code>	El <code>any</code> tipo de datos es el estado proporcionado cuando un valor no tiene una definición de tipo de datos explícita. El <code>any</code> tipo es el tipo de datos que clasifica todos los valores.
<code>binary</code>	El <code>binary</code> tipo de datos se puede usar para representar cualquier otro dato con un formato binario.
<code>type</code>	Valor que clasifica otros valores. Para obtener más información, vaya a Tipos .
<code>null</code>	Representa la ausencia de un valor o un valor de estado indeterminado o desconocido.
<code>anynonnull</code>	Representa cualquier tipo que no sea null.
<code>date</code>	Representa solo una fecha (sin parte de hora).
<code>time</code>	Representa just time (sin parte de fecha).
<code>datetime</code>	Representa un valor de fecha y hora. La parte de hora de una fecha se almacena como una fracción en múltiplos completos de 1/300 segundos (3,33 ms). Se admiten fechas entre los años 1900 y 9999.
<code>datetimezone</code>	Representa una fecha y hora UTC con un desplazamiento de zona horaria.
<code>duration</code>	Representa un período de tiempo. Este tipo se puede agregar o restar de un <code>datetime</code> campo con resultados correctos. Para obtener más información, vaya a Duración .

Tipo de dato	Descripción
<code>text</code>	Cadena de datos de caracteres Unicode. Puede ser cadenas, números o fechas representadas en un formato de texto. La longitud máxima de cadena es de 268 435 456 caracteres Unicode (donde cada carácter Unicode es de dos bytes) o 536 870 912 bytes.
<code>logical</code>	Valor booleano de <code>true</code> o <code>false</code> .
<code>list</code>	Valor que genera una secuencia de valores cuando se enumera. Para obtener más información, vaya a Tipos de lista y Valores de lista .
<code>record</code>	Secuencia ordenada de campos. Cada campo contiene un nombre de campo y un valor de campo. Para obtener más información, vaya a Tipos de registro y Valores de registro .
<code>table</code>	Secuencia ordenada de filas divididas en columnas. Para obtener más información, vaya a Tipos de tabla y Valores de tabla .
<code>function</code>	Valor que asigna un conjunto de argumentos a un único valor. Para obtener más información, vaya a Funciones y tipos de función .
<code>number</code>	Representa cualquier número usado para las operaciones numéricas y aritméticas. Para obtener más información, vaya a Número .
<code>Decimal.Type</code>	Representa un número de punto flotante de 64 bits (ocho bytes). Es el tipo de número más común y corresponde a los números como se suele pensar en ellos. Aunque está diseñado para controlar números con valores fraccionarios, también controla números enteros. <code>Decimal.Type</code> Puede controlar valores negativos de $-1,79\text{E} + 308$ a $-2,23\text{E} - 308$, 0 y valores positivos de $2,23\text{E} - 308$ a $1,79\text{E} + 308$. Por ejemplo, los números como 34, 34.01 y 34.000367063 son números decimales válidos. La precisión más grande que se puede representar en un <code>Decimal.Type</code> es de 15 dígitos. El separador decimal puede producirse en cualquier parte del número. <code>Decimal.Type</code> corresponde a cómo Excel almacena sus números. Tenga en cuenta que un número de punto flotante binario no puede representar todos los números dentro de su intervalo admitido con una precisión de 100%. Por lo tanto, pueden producirse pequeñas diferencias de precisión al representar determinados números decimales.
<code>Currency.Type</code>	Este tipo de datos tiene una ubicación fija para el separador decimal. El separador decimal siempre tiene cuatro dígitos a su derecha y permite 19 dígitos de importancia. El valor más grande que puede representar es 922.337.203.685.477.5807 (positivo o negativo). A diferencia <code>Decimal.Type</code> de , siempre <code>Currency.Type</code> es preciso y, por tanto, resulta útil en los casos en los que la imprecisión de la notación de punto flotante podría introducir errores.
<code>Percentage.Type</code>	Fundamentalmente igual que un <code>Decimal.Type</code> , pero tiene una máscara para dar formato a los valores como un valor de porcentaje.
<code>Int8.Type</code>	Representa un valor entero con signo de 8 bits (un byte). Dado que es un entero, no tiene dígitos a la derecha del separador decimal. Permite 3 dígitos; un número entero

Tipo de dato	Descripción
	positivo o negativo entre -128 y 127. Al igual que con <code>Currency.Type</code> , <code>Int8.Type</code> puede ser útil en los casos en los que necesita controlar el redondeo.
<code>Int16.Type</code>	Representa un valor entero con signo de 16 bits (dos bytes). Dado que es un entero, no tiene dígitos a la derecha del separador decimal. Permite 6 dígitos; un número entero positivo o negativo entre -32.768 (-2^{15}) y 32.767 ($2^{15}-1$). Al igual que con <code>Currency.Type</code> , <code>Int16.Type</code> puede ser útil en los casos en los que necesita controlar el redondeo.
<code>Int32.Type</code>	Representa un valor entero con signo de 32 bits (cuatro bytes). Dado que es un entero, no tiene dígitos a la derecha del separador decimal. Permite 10 dígitos; un número entero positivo o negativo entre -2,147,483,648 (-2^{31}) y 2,147,483,647 ($2^{31}-1$). Al igual que con <code>Currency.Type</code> , <code>Int32.Type</code> puede ser útil en los casos en los que necesita controlar el redondeo.
<code>Int64.Type</code>	Representa un valor entero con signo de 64 bits (ocho bytes). Dado que es un entero, no tiene dígitos a la derecha del separador decimal. Permite 19 dígitos; un número entero positivo o negativo entre -9,223,372,036,854,775,808 (-2^{63}) y 9,223,372,036,854,775,807 ($2^{63}-1$). Puede representar la mayor precisión posible de los distintos tipos de datos numéricos. Al igual que con <code>Currency.Type</code> , <code>Int64.Type</code> puede ser útil en los casos en los que necesita controlar el redondeo.
<code>Byte.Type</code>	Representa un valor entero de 8 bits (un byte) sin signo. Dado que es un entero sin signo, no tiene dígitos a la derecha de la posición decimal y solo puede contener valores positivos. Permite 3 dígitos; un número positivo entre 0 y 255.
<code>Single.Type</code>	Representa un número de punto flotante de precisión sencilla. Tiene un intervalo aproximado de $-3,99 \times 10^{38}$ a $3,99 \times 10^{38}$ y admite aproximadamente 9 dígitos de precisión. También puede representar infinito positivo y negativo, y NaN (no un número).
<code>Double.Type</code>	Representa un número de punto flotante de precisión doble. Tiene un intervalo aproximado de $-1,7976931348623158 \times 10^{307}$ a $1,7976931348623158 \times 10^{307}$ y admite aproximadamente 17 dígitos de precisión. También puede representar infinito positivo y negativo, y NaN (no un número).
<code>Guid.Type</code>	Representa un valor de texto de 128 bits que consta de 32 valores hexadecimales mediante el factor de forma de <8 valores> hexadecimales-4< valores> hexadecimales-4> << valores> hexadecimales-12< valores> hexadecimales, que componen el valor GUID.
<code>none</code>	Tipo de datos que clasifica ningún valor.

Los únicos otros valores usados `*.Type` habitualmente son enumeraciones. Para obtener más información, vaya a [Enumeraciones](#).

Conversión de tipos

El lenguaje de fórmulas M de Power Query tiene fórmulas para convertir entre tipos. Este es un resumen de las fórmulas de conversión en M.

Número

 Expandir tabla

Conversión de tipos	Descripción
Number.FromText(texto como texto) como número	Devuelve un valor numérico de un valor de texto.
Number.ToText(número como número) como texto	Devuelve un valor de texto de un valor numérico.
Number.From(valor como cualquiera) como número	Devuelve un valor numérico de un valor.
Byte.From(valor como cualquiera) como número	Devuelve un valor de número entero de 8 bits del valor especificado.
Int8.From(valor como cualquiera) como número	Devuelve un valor de número entero de 8 bits del valor especificado.
Int16.From(valor como cualquiera) como número	Devuelve un valor de número entero de 16 bits del valor especificado.
Int32.From(valor como cualquiera) como número	Devuelve un valor de número entero de 32 bits del valor especificado.
Int64.From(valor como cualquiera) como número	Devuelve un valor de número entero de 64 bits del valor especificado.
Single.From(valor como cualquiera) como número	Devuelve un valor numérico único del valor especificado.
Double.From(valor como cualquiera) como número	Devuelve un valor de número doble del valor especificado.
Decimal.From(valor como cualquiera) como número	Devuelve un valor de número decimal del valor especificado.
Currency.From(valor como cualquiera) como número	Devuelve un valor numérico monetario del valor especificado.
Percentage.From(valor como cualquiera) como número	Devuelve un valor de número de porcentaje del valor especificado.

Texto

[Expandir tabla](#)

Conversión de tipos	Descripción
Text.From(valor como cualquiera) como texto	Devuelve la representación textual de un valor binario, lógico, de número, de duración, de fecha, de hora, de fecha y hora o de fecha, hora y zona horaria.
Guid.From(valor como texto) como texto	Devuelve la representación GUID del texto especificado.

Lógicos

[Expandir tabla](#)

Conversión de tipos	Descripción
Logical.FromText(texto como texto) como lógico	Devuelve un valor lógico de true o false de un valor de texto.
Logical.ToText(lógico como lógico) como texto	Devuelve un valor de texto de un valor lógico.
Logical.From(valor como cualquiera) como lógico	Devuelve un valor lógico de un valor.

Date, Time, DateTime y DateTimeZone

[Expandir tabla](#)

Conversión de tipos	Descripción
FromText(texto como texto) como un valor Date, Time, DateTime o DateTimeZone	Devuelve un valor Date, Time, DateTime o DateTimeZone de un conjunto de formatos de fecha y un valor de referencia cultural.
. ToText(date, time, dateTime o dateTimeZone como date, time, datetime o datetimezone) como texto	Devuelve un valor de texto de un valor date, time, datetime o datetimezone.
.From(valor como cualquiera)	Devuelve un valor date, time, datetime o datetimezone de un valor.
.ToRecord(date, time, dateTime o dateTimeZone como date, time, datetime o datetimezone)	Devuelve un registro que contiene partes de un valor date, time, datetime o datetimezone.

Contenido relacionado

- [Tipos](#)
- [Sistema de tipos M de Power Query](#)

Last updated on 22/11/2025

Metadatos

Artículo • 20/11/2024

Los **metadatos** es información relativa un valor que está asociado a otro valor. Los **metadatos** se representan como un valor de registro, denominado registro de metadatos. Los campos de un **registro de metadatos** se pueden usar para almacenar los metadatos de un valor. Cada valor tiene un registro de metadatos. Si no se ha especificado el valor del registro de metadatos, el registro de metadatos estará vacío (no tendrá campos). La asociación de un registro de metadatos con un valor no cambia el comportamiento del valor en las evaluaciones, excepto las que inspeccionan explícitamente los registros de metadatos.

Registros de metadatos

Un valor de registro de metadatos se asocia a un valor x usando el valor de sintaxis "meta" [registro]. Por ejemplo, en el siguiente ejemplo se asocia un registro de metadatos con campos Rating y Tags que contienen el valor de texto "Mozart":

Power Query M

```
"Mozart" meta [ Rating = 5,
Tags = {"Classical"} ]
```

Se puede tener acceso a un registro de metadatos para un valor mediante la [función Value.Metadata](#) . En el siguiente ejemplo, la expresión del campo ComposerRating accede al registro de metadatos del valor en el campo Composer y, después, accede al campo Rating del registro de metadatos.

Power Query M

```
[
    Composer = "Mozart" meta [ Rating = 5, Tags = {"Classical"} ],
    ComposerRating = Value.Metadata(Composer)[Rating]    // 5
]
```

Los registros de metadatos no se conservan cuando se usa un valor con un operador o una función que crea un nuevo valor. Por ejemplo, si se concatenan dos valores de texto con el operador &, los metadatos del valor de texto resultante es un registro vacío [].

Las funciones [de biblioteca estándar Value.RemoveMetadata](#) y [Value.ReplaceMetadata](#) se pueden usar para quitar todos los metadatos de un valor y reemplazar los metadatos

de un valor.

Limitaciones

Algunos hosts que usan Power Query para transformar o mover datos no admiten el almacenamiento de metadatos personalizados en el almacenamiento. Los hosts siguientes no admiten el almacenamiento de los metadatos personalizados:

- Flujos de datos de Power BI
- Flujo de datos Gen2 de Fabric
- Flujos de datos de Power Platform

Comentarios

¿Le ha resultado útil esta página?



Sí



No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Errors

Un *error* en el lenguaje de fórmulas M de Power Query es una indicación de que el proceso de evaluación de una expresión no pudo generar un valor. Los operadores y las funciones producen errores que encuentran condiciones de **error** o mediante la expresión **de error** . Los errores se controlan mediante la expresión **try** . Cuando se produce un error, se especifica un valor que se puede usar para indicar por qué se produjo el error.

Probar expresión

Una expresión **try** convierte valores y errores en un valor de registro que indica si la expresión **try** controló un error, o no, y el valor adecuado o el registro de error que extrajo al controlar el error. Por ejemplo, considere la siguiente expresión que genera un error y, a continuación, la controla inmediatamente:

Power Query M

```
try error "negative unit count"
```

Esta expresión se evalúa como el siguiente valor de registro anidado, explicando las `[HasError]`, `[Error]` búsquedas de campos y `[Message]` en el ejemplo de precio unitario antes.

Registro de errores

Power Query M

```
[
  HasError = true,
  Error =
    [
      Reason = "Expression.Error",
      Message = "negative unit count",
      Detail = null
    ]
]
```

Un caso común es reemplazar los errores por valores predeterminados. La expresión **try** se puede usar con una cláusula opcional de lo contrario para lograr solo eso en un formato compacto:

Power Query M

```
try error "negative unit count" otherwise 42
```

Ejemplo de error

Power Query M

```
let Sales =  
    [  
        ProductName = "Fishing rod",  
        Revenue = 2000,  
        Units = 1000,  
        UnitPrice = if Units = 0 then error "No Units"  
                    else Revenue / Units  
    ],  
  
    //Get UnitPrice from Sales record  
    textUnitPrice = try Number.ToText(Sales[UnitPrice]),  
    Label = "Unit Price: " &  
        (if textUnitPrice[HasError] then textUnitPrice[Error][Message]  
        //Continue expression flow  
        else textUnitPrice[Value])  
in  
    Label
```

En el ejemplo anterior se obtiene acceso al `Sales[UnitPrice]` campo y se da formato al valor que genera el resultado:

Power Query M

```
"Unit Price: 2"
```

Si el campo Unidades hubiera sido cero, el `UnitPrice` campo habría generado un error que habría sido controlado por el intento. A continuación, el valor resultante habría sido:

Power Query M

```
"No Units"
```

Variantes locales, fijas y UTC de las funciones de hora actual

16/07/2025

Cuando trabaja con Power Query en herramientas como Excel y Power BI, controlar los valores de fecha y hora con precisión es esencial, especialmente cuando las transformaciones de datos dependen de la hora actual. Power Query ofrece varias funciones para recuperar la fecha y hora actuales:

- [DateTime.LocalNow](#)
- [DateTimeZone.LocalNow](#)
- [DateTime.FixedLocalNow](#)
- [DateTimeZone.FixedLocalNow](#)
- [DateTimeZone.UtcNow](#)
- [DateTimeZone.FixedUtcNow.](#)

En este artículo se exploran las diferencias entre estas funciones y se aclara cuándo y por qué usar cada una. Además, resalta un detalle crítico pero a menudo pasado por alto. Power Query Online siempre devuelve la hora UTC incluso cuando se usa una función etiquetada como "Local". Comprender estos matices puede ayudarle a evitar resultados inesperados, especialmente al compilar informes sensibles al tiempo o automatizar las actualizaciones de datos en aplicaciones como el servicio Power BI o Power Apps.

Diferencias entre funciones

Cada una de las funciones de hora actual tiene diferencias importantes. Estas funciones varían en términos de reconocimiento de zona horaria, volatilidad (si el valor cambia cuando se llama varias veces en la misma consulta) y cómo se comportan en diferentes entornos (escritorio frente a en línea). La tabla siguiente contiene un desglose de cada función.

 Expandir tabla

Función	Devoluciones	Volatilidad	Comportamiento del escritorio	Comportamiento en línea	Caso de uso típico
<code>DateTime.LocalNow</code>	Un <code>datetime</code> que representa la hora local actual	Dinámico: devuelve un nuevo valor cada vez que se invoca durante la evaluación de consultas.	Devuelve la hora del equipo local.	Devuelve la hora UTC.	Marca de tiempo local rápida sin contexto de zona horaria
<code>DateTimeZone.LocalNow</code>	Valor <code>datetimezone</code>	Dinámico: devuelve	Devuelve la hora local con	Devuelve la hora UTC con <code>+00:00</code>	Hora local con reconocimiento

Función	Devoluciones	Volatilidad	Comportamiento del escritorio	Comportamiento en línea	Caso de uso típico
	que representa la hora local actual con desplazamiento de zona horaria	un nuevo valor cada vez que se invoca durante la evaluación de consultas.	desplazamiento.	desplazamiento.	de zona horaria
<code>DateTime.FixedLocalNow</code>	Valor <code>datetime</code> que representa la hora local cuando se invoca por primera vez durante la evaluación de consultas.	Corregido: devuelve el mismo valor durante una única evaluación de consulta.	Captura la hora local cuando se llama por primera vez	Captura el tiempo UTC la primera vez que se llama.	Instantánea de la hora local sin zona horaria
<code>DateTimeZone.FixedLocalNow</code>	Valor <code>datetimezone</code> que representa la hora local con desplazamiento cuando se invoca por primera vez durante la evaluación de consultas.	Corregido: devuelve el mismo valor durante una única evaluación de consulta.	Captura la hora local con desplazamiento horario cuando se llama por primera vez	Captura la hora UTC con un desplazamiento de <code>+00:00</code> cuando se llama por primera vez	Instantánea de la hora local con zona horaria
<code>DateTimeZone.UtcNow</code>	Valor <code>datetimezone</code> que representa la hora UTC actual	Dinámico: devuelve un nuevo valor cada vez que se invoca durante la evaluación de consultas.	Devuelve la hora UTC actual.	Devuelve la hora UTC actual.	Marca de tiempo UTC coherente para escenarios dinámicos
<code>DateTimeZone.FixedUtcNow</code>	Valor <code>datetimezone</code> que representa la hora UTC cuando se invoca por primera vez durante la	Corregido: devuelve el mismo valor durante una única evaluación de consulta.	Captura el tiempo UTC la primera vez que se llama.	Captura el tiempo UTC la primera vez que se llama.	Marca de tiempo UTC fija para el registro o la auditoría

Función	Devoluciones	Volatilidad	Comportamiento del escritorio	Comportamiento en línea	Caso de uso típico
	evaluación de consultas.				

En Power Query M, elegir entre las funciones de fecha y hora locales y las funciones de fecha y hora basadas en UTC es una decisión de diseño crítica que afecta a la coherencia, la precisión y la portabilidad de las consultas. Las funciones como `DateTime.LocalNow` y `DateTime.FixedLocalNow` son útiles cuando la lógica depende de la hora del sistema local, como filtrar los registros que se produjeron "hoy" o generar marcas de tiempo para informes orientados al usuario. Estas funciones reflejan la zona horaria del entorno en el que se ejecuta la consulta, lo que las convierte en adecuadas para escenarios de Power Query Desktop en los que el contexto local está bien definido.

Sin embargo, en entornos distribuidos o basados en la nube, como Power Query Online, estas mismas funciones devuelven la hora UTC, no la hora local real del usuario. Esta discrepancia puede provocar incoherencias sutiles si la lógica asume un contexto de hora local. Por el contrario, `DateTimeZone.UtcNow` y `DateTimeZone.FixedUtcNow` proporcionan un punto de referencia neutral de zona horaria coherente entre entornos y que no se ve afectado por el horario de verano o la configuración regional. Estas funciones basadas en UTC son la opción preferida para escenarios que implican la integración de datos, el registro, la auditoría o cualquier lógica que se comporte de forma idéntica independientemente de dónde o cuando se ejecute la consulta.

Diferencias entre las funciones LocalNow y FixedLocalNow

Power Query M proporciona cuatro funciones para recuperar la hora local actual:

- `DateTime.LocalNow` devuelve el valor local `datetime` actual cada vez que se evalúa la expresión.
- `DateTime.FixedLocalNow` devuelve el valor local `datetime` una vez por evaluación de consultas, que actúa como una instantánea.
- `DateTimeZone.LocalNow` devuelve el valor local `datetimezone` actual cada vez que se evalúa la expresión.
- `DateTimeZone.FixedLocalNow` devuelve el valor local `datetimezone` una vez por evaluación de consultas, que actúa como una instantánea.

Para demostrar la diferencia, en el ejemplo siguiente se genera una tabla con varias filas. Cada fila captura un valor nuevo `DateTime.LocalNow` mediante un retraso para garantizar marcas de tiempo distintas, mientras que cada valor capturado `DateTime.FixedLocalNow` permanece constante en todas las filas.

⚠ Nota

Todas las fechas y horas de la salida de los ejemplos de este artículo dependen de cuándo se ejecutan las funciones. Las fechas y horas que se muestran en la salida son solo con fines de

Power Query M

```

let
    // Create a table with LocalNow and FixedLocalNow columns
    TableWithTimes = Table.FromList(
        {1..5},
        each {
            ~
            Function.InvokeAfter(() => DateTime.LocalNow(), #duration(0, 0, 0, 0.2)),
            Function.InvokeAfter(() => DateTime.FixedLocalNow(), #duration(0, 0, 0, 0.2))
        },
        {"Index", "LocalNow", "FixedLocalNow"}
    ),

    // Format both datetime columns
    FormatLocalNow = Table.TransformColumns(TableWithTimes,
        {"LocalNow", each DateTime.ToText(_, "yyyy-MM-ddThh:mm:ss.fff")}),
    FormatFixedNow = Table.TransformColumns(FormatLocalNow,
        {"FixedLocalNow", each DateTime.ToText(_, "yyyy-MM-ddThh:mm:ss.fff")}),

    // Change the table types
    FinalTable = Table.TransformColumnTypes(FormatFixedNow, {"Index", Int64.Type},
        {"LocalNow", type text}, {"FixedLocalNow", type text})

in
    FinalTable

```

La salida de este ejemplo es:

	1.2 Index	A _C LocalNow	A _C FixedLocalNow
1	1	2025-05-28T09:40:15.277	2025-05-28T09:40:14.976
2	2	2025-05-28T09:40:15.496	2025-05-28T09:40:14.976
3	3	2025-05-28T09:40:15.700	2025-05-28T09:40:14.976
4	4	2025-05-28T09:40:15.903	2025-05-28T09:40:14.976
5	5	2025-05-28T09:40:16.106	2025-05-28T09:40:14.976

Si observa la salida, es posible que pueda notar que aunque la `DateTime.LocalNow` función aparezca primero en el código, el valor devuelto para `DateTime.FixedLocalNow` muestra una hora que ocurre antes de la hora de `DateTime.LocalTime`. `DateTime.LocalNow` Aunque se muestra primero en la construcción de la tabla, no se garantiza que el orden de evaluación en Power Query M siga el orden de los campos de una tabla. En su lugar, Power Query utiliza un modelo de evaluación perezosa. El uso de este modelo significa que los campos solo se evalúan cuando es necesario y el motor determina el orden de evaluación, no el orden en el código. En este caso, la `DateTime.FixedLocalNow` función se evalúa primero, por lo que la primera vez que se devuelve para esta función se produce antes de la primera vez que se devuelve para `DateTime.LocalNow`.

En el ejemplo siguiente se muestra cómo generar resultados similares mediante `DateTimeZone.LocalNow` y `DateTimeZone.FixedLocalNow`.

Power Query M

```
let
    // Create a table with LocalNow and FixedLocalNow columns
    TableWithTimes = Table.FromList(
        {1..5},
        each {
            ~,
            Function.InvokeAfter(() => DateTimeZone.LocalNow(), #duration(0, 0, 0, 0.2)),
            Function.InvokeAfter(() => DateTimeZone.FixedLocalNow(), #duration(0, 0, 0,
0.2))
        },
        {"Index", "LocalNow", "FixedLocalNow"}
    ),

    // Format both datetimezone columns
    FormatLocalNow = Table.TransformColumns(TableWithTimes,
        {"LocalNow", each DateTimeZone.ToText(_, "yyyy-MM-ddThh:mm:ss.fff:zzz")}),
    FormatFixedNow = Table.TransformColumns(FormatLocalNow,
        {"FixedLocalNow", each DateTimeZone.ToText(_, "yyyy-MM-ddThh:mm:ss.fff:zzz")}),

    // Change the table types
    FinalTable = Table.TransformColumnTypes(FormatFixedNow,
        {"Index", Int64.Type}, {"LocalNow", type text}, {"FixedLocalNow", type text})
in
    FinalTable
```

La salida de este ejemplo en Power Query Desktop es:

	1 ² Index	A ^B _C LocalNow	A ^B _C FixedLocalNow
1	1	2025-05-30T12:34:02.616:-07:00	2025-05-30T12:34:02.382:-07:00
2	2	2025-05-30T12:34:03.026:-07:00	2025-05-30T12:34:02.382:-07:00
3	3	2025-05-30T12:34:03.436:-07:00	2025-05-30T12:34:02.382:-07:00
4	4	2025-05-30T12:34:03.846:-07:00	2025-05-30T12:34:02.382:-07:00
5	5	2025-05-30T12:34:04.257:-07:00	2025-05-30T12:34:02.382:-07:00

⚠ Nota

Si ejecuta este ejemplo en Power Query Online, la hora devuelta siempre es la hora UTC y la parte de zona horaria de los valores devueltos siempre es `+00:00`.

Diferencias entre las funciones `UtcNow` y `FixedUtcNow`

Power Query M proporciona dos funciones para recuperar la hora UTC actual:

- `DateTimeZone.UtcNow` devuelve la hora UTC `datetimezone` actual cada vez que se evalúa la expresión.
- `DateTimeZone.FixedUtcNow` devuelve el valor local `datetimezone` una vez por evaluación de consultas, que actúa como una instantánea.

Las diferencias entre estas dos funciones son similares a las funciones `LocalNow` y `FixedLocalNow`. Sin embargo, si las funciones se ejecutan en Power Query Desktop o Power Query Online, los valores devueltos siempre se devuelven como hora UTC. En el ejemplo siguiente se muestran las diferencias entre estas dos funciones.

Power Query M

```
let
    // Create a table with UtcNow and FixedUtcNow columns
    TableWithTimes = Table.FromList(
        {1..5},
        each {
            ~,
            Function.InvokeAfter(() => DateTimeZone.UtcNow(), #duration(0, 0, 0, 0.2)),
            Function.InvokeAfter(() => DateTimeZone.FixedUtcNow(), #duration(0, 0, 0, 0.2))
        },
        {"Index", "UtcNow", "FixedUtcNow"}
    ),

    // Format both datetimezone columns
    FormatLocalNow = Table.TransformColumns(TableWithTimes,
        {"UtcNow", each DateTimeZone.ToText(_, "yyyy-MM-ddThh:mm:ss.fff:zzz")}),
    FormatFixedNow = Table.TransformColumns(FormatLocalNow,
        {"FixedUtcNow", each DateTimeZone.ToText(_, "yyyy-MM-ddThh:mm:ss.fff:zzz")}),

    // Change the table types
    FinalTable = Table.TransformColumnTypes(FormatFixedNow,
        {"Index", Int64.Type}, {"UtcNow", type text}, {"FixedUtcNow", type text})
in
    FinalTable
```

La salida de este ejemplo en Power Query Desktop y Power Query Online es:

	1 ² ₃ Index	A ^B _C UtcNow	A ^B _C FixedUtcNow
1	1	2025-06-11T03:42:57.269:+00:00	2025-06-11T03:42:56.964:+00:00
2	2	2025-06-11T03:42:57.692:+00:00	2025-06-11T03:42:56.964:+00:00
3	3	2025-06-11T03:42:58.099:+00:00	2025-06-11T03:42:56.964:+00:00
4	4	2025-06-11T03:42:58.506:+00:00	2025-06-11T03:42:56.964:+00:00
5	5	2025-06-11T03:42:58.914:+00:00	2025-06-11T03:42:56.964:+00:00

Efectos en otras funciones

Otras funciones de Power Query M que dependen de la fecha y hora actuales también pueden verse afectadas por cómo se devuelve la hora local en Power Query Desktop o Power Query Online. Por ejemplo, si usa la función [DateTimeZone.ToLocal](#) para convertir la hora UTC a hora local, todavía devuelve la hora UTC en Power Query Online.

Otro ejemplo es cualquier función que pueda usar la hora actual del sistema como parámetro. Estas funciones incluyen [Date.Month](#), [Date.DayOfYear](#), [DateTime.IsInCurrentYear](#), [DateTimeZone.ZoneHour](#) o cualquier otra función que pueda evaluar la fecha y hora actuales.

En todas estas funciones, si la lógica depende de si un valor se encuentra dentro del día actual, la hora, el mes o el año, los resultados pueden diferir entre entornos. Estas diferencias entre entornos son especialmente perceptibles si la consulta se ejecuta cerca de un límite (por ejemplo, justo antes o después de medianoche, el inicio de un nuevo mes o un año nuevo). Si la coherencia es fundamental en distintos entornos, use las `DateTimeZone.UtcNow` funciones o `DateTimeZone.FixedUtcNow` para recuperar la fecha y hora.

Procedimientos recomendados y recomendaciones

Elegir la función de hora adecuada en Power Query depende de su caso de uso específico, el entorno en el que se ejecuta la consulta (escritorio frente a en línea) y si necesita una marca de tiempo dinámica o fija. Estos son algunos procedimientos recomendados para ayudar a guiar su decisión:

- **Ser explícito sobre las zonas horarias:** use las funciones `DateTimeZone` en lugar de las funciones `DateTime` cuando importa el contexto de zona horaria. Use `DateTimeZone.UtcNow` o `DateTimeZone.FixedUtcNow` para mantener la coherencia entre entornos, especialmente en soluciones basadas en la nube, como el servicio Power BI.
- **Usar funciones fijas para resultados repetibles:** use las variantes fijas (como `DateTimeZone.FixedUtcNow`) cuando desee que la marca de tiempo permanezca constante en las evaluaciones de consulta. Este método es especialmente útil para registrar, auditar o capturar el tiempo de ingesta de datos.
- **Evite las funciones locales en Power Query Online:** funciones como `DateTime.LocalNow` y `DateTimeZone.LocalNow` devuelven la hora UTC en soluciones basadas en la nube como el servicio Power BI, lo que puede provocar confusión o suposiciones incorrectas. Si necesita una hora local real en el servicio, considere la posibilidad de ajustar las UTC manualmente mediante desplazamientos conocidos (aunque este ajuste puede ser frágil, por ejemplo, debido al horario de verano o a la configuración regional).
- **Prueba en entornos de escritorio y en línea:** pruebe siempre las consultas tanto en Power Query Desktop como en Power Query Online si la lógica depende de la hora actual. Esta prueba ayuda a detectar discrepancias al principio, especialmente para escenarios de actualización programados.
- **Documentar la lógica de tiempo:** comenta o documenta claramente por qué se usa una función de hora específica, especialmente si usa una solución alternativa para el control de zonas horarias. Esta información ayuda a los colaboradores futuros a comprender la intención detrás de la lógica.
- **Usar UTC para flujos de trabajo programados:** para actualizaciones programadas o canalizaciones automatizadas, UTC es la opción más segura y predecible. Evita ambigüedades causadas por el horario de verano o los turnos de zona horaria regionales.

- **Valores de tiempo de caché cuando sea necesario:** si necesita usar la misma marca de tiempo en varios pasos de una consulta, asígnela a una variable en la parte superior de la consulta mediante una función fija. Esta variable garantiza la coherencia en toda la lógica de transformación.

Compatibilidad con la duración en Power Query M

15/08/2025

Una duración en Power Query M representa la diferencia entre dos puntos de tiempo, expresada en días, horas, minutos y segundos. Tanto si calcula el tiempo entre las interacciones del cliente, el filtrado de registros en función del tiempo transcurrido o la creación de lógica dinámica basada en tiempo, las duraciones son esenciales para crear modelos de datos sólidos e inteligentes.

En este artículo se explora la estructura, la creación y la manipulación de duraciones en Power Query M. Incluye ejemplos prácticos y sugerencias para compartir para ayudarle a usar duraciones de forma eficaz en sus propios flujos de trabajo de datos.

Crear una duración

La función define `#duration(<days>, <hours>, <minutes>, <seconds>)` una duración. Por ejemplo, `#duration(2, 3, 0, 0)` representa una duración de 2 días y 3 horas. Power Query M proporciona varias maneras de crear una duración, según el contexto y el nivel de precisión necesario.

Use la función #duration

La manera más directa de crear una duración es con la `#duration(<days>, <hours>, <minutes>, <seconds>)` sintaxis. Cada argumento debe ser un número y el resultado es un valor de duración.

```
#duration(2, 5, 30, 0) // 2 days, 5 hours, 30 minutes
```

Esta función también admite fracciones de segundo:

```
#duration(0, 0, 0, 1.75) // 1.75 seconds
```

Crear duraciones a partir de valores de fecha y hora

Las duraciones también se pueden crear restando un valor de fecha y hora de otro. El resultado es una duración que representa el intervalo de tiempo entre los dos.


```

let
    Source =
    {
        #date(2025, 7, 24) - #date(2025, 7, 23),
        // Result: #duration(1, 0, 0, 0)
        #time(12, 0, 0) - #time(11, 30, 30),
        // Result: #duration(0, 0, 29, 30)
        #datetime(2025, 7, 24, 12, 0, 0) - #datetime(2025, 7, 23, 12, 0, 0),
        // Result: #duration(1, 0, 0, 0)
        #datetimezone(2025, 7, 24, 12, 0, 0, 7, 0) - #datetimezone(2025, 7, 23,
10, 30, 0, 4, 0),
        // Result: #duration(0, 22, 30, 0)
        #datetime(2025, 7, 24, 12, 0, 0) - DateTime.From(#date(2025, 7, 23))
        // Result: #duration(1, 12, 0, 0)
    }
in
    Source

```

ⓘ Nota

Restar un tipo de fecha y hora de un tipo de fecha y hora diferente (por ejemplo, restando un `date` valor de un `datetime` valor) produce un error. Si debe usar diferentes tipos de fecha y hora para determinar una duración, use las [Date.From](#) funciones, [DateTime.From](#), [DateTimeZone.From](#) o [Time.From](#) para cambiar explícitamente uno de los tipos de fecha y hora.

Conversión de valores compatibles

La [Duration.From](#) función puede convertir valores compatibles en duraciones. Para obtener más información, vaya a [Duration.From\(valor\)](#).

Trabajar con duraciones

Una vez creada una duración en Power Query M, se puede manipular mediante varias operaciones y funciones. Estas funcionalidades hacen que las duraciones sean muy versátiles para la lógica y los cálculos basados en tiempo.

Operaciones aritméticas

Las duraciones admiten operaciones aritméticas estándar:

- Suma y resta: agregue o resta duraciones entre sí o de valores de fecha y hora.

Power Query M

```
let
    Source = {
        #duration(1, 2, 0, 0) + #duration(0, 3, 30, 0),
        // Result: #duration(1, 5, 30, 0)
        #duration(1, 2, 0, 0) - #duration(0, 3, 30, 0),
        // Result: #duration(0, 22, 30, 0)
        #datetime(2025, 7, 24, 12, 0, 0) + #duration(0, 2, 0, 0),
        // Result: #datetime(2025, 7, 24, 14, 0, 0)
        #datetime(2025, 7, 24, 12, 0, 0) - #duration(0, 2, 0, 0),
        // Result: #datetime(2025, 7, 24, 10, 0, 0)
        #time(12, 0, 0) - #duration(0, 3, 30, 0)
        // Result: #time(8, 30, 0)
    }
in
    Source
```

- Negación: una duración se puede negar para invertir su dirección.

Power Query M

```
let
    Source = {
        #datetime(2025, 7, 24, 12, 0, 0) + -#duration(0, 2, 0, 0),
        // Result (subtracts two hours): #datetime(2025, 7, 24, 10, 0, 0)
        #datetime(2025, 7, 23, 12, 0, 0) - #datetime(2025, 7, 24, 12, 0, 0)
        // Result: -#duration(1, 0, 0, 0)
    }
in
    Source
```

Multiplicación y división

Las duraciones se pueden multiplicar o dividir por valores numéricos:

Power Query M

```
let
    Source = {
        #duration(0, 2, 0, 0) * 2,
        // Result (4 hours): #duration(0, 4, 0, 0)
        #duration(1, 0, 0, 0) / 2
        // Result (12 hours): #duration(0, 12, 0, 0)
    }
in
    Source
```

Este cálculo es útil para las duraciones de escalado o el promedio de intervalos de tiempo.

Comparaciones

Las duraciones se pueden comparar mediante operadores de comparación estándar:

Power Query M

```
let
    Source = #duration(1, 0, 0, 0) > #duration(0, 23, 59, 59)
    // Result: true
in
    Source
```

Este cálculo permite usar duraciones en la lógica condicional, como el filtrado de filas en función del tiempo transcurrido.

Compatibilidad de tipos

Las duraciones son compatibles con valores de fecha y hora en expresiones aritméticas, pero no intercambiables con ellos. Por ejemplo, restar dos valores de fecha y hora produce un `duration`, pero agregar dos valores de fecha y hora no es válido.

Power Query M

```
let
    Source =
    {
        #datetime(2025, 7, 24, 12, 0, 0) - #datetime(2025, 7, 23, 12, 0, 0),
        // Result: #duration(1, 0, 0, 0)
        #datetime(2025, 7, 24, 12, 0, 0) + #datetime(2025, 7, 23, 12, 0, 0)
        // Result: Error
    }
in
    Source
```

Funciones de duración en M

Power Query M incluye un conjunto de funciones integradas para trabajar con duraciones. Estas funciones permiten la conversión, la extracción de componentes y la agregación de valores de duración, lo que las convierte en herramientas esenciales para las transformaciones basadas en el tiempo.

Duration.From(valor)

La [Duration.From](#) función convierte un valor compatible en una duración. Los valores compatibles constan de un número que se interpreta como una fracción de un día o una representación textual de una duración. Vaya a [Duration.FromText](#) para obtener información sobre los formatos de representación textual.

Power Query M

```
let
    Source =
    {
        Duration.From(1.5),
        // Result: 1.5 days = #duration(1, 12, 0, 0)
        Duration.From("2.05:55:20.242")
        // Result: #duration(2, 5, 55, 20.242)
    }
in
    Source
```

Descriptores de acceso de componente

Estas funciones extraen partes específicas de una duración:

- `Duration.Days(<duration>)`

Devuelve el número de días enteros en la duración.

- `Duration.Hours(<duration>)`

Devuelve el número de horas más allá de los días completos.

- `Duration.Minutes(<duration>)`

Devuelve el número de minutos más allá de las horas completas.

- `Duration.Seconds(<duration>)`

Devuelve el número de segundos más allá de los minutos completos.

Power Query M

```
let
    Source = #duration(2, 5, 30, 45),
    TextFormat = Text.Format(
        "Duration = #{0} days, #{1} hours, #{2} minutes, and #{3} seconds.",
        {
            Duration.Days(Source),
            Duration.Hours(Source),
            Duration.Minutes(Source),
            Duration.Seconds(Source)
        }
    )
in
    TextFormat
```

```

        Duration.Seconds(Source)
    }
)
// Results: "Duration = 2 days, 5 hours, 30 minutes, and 45 seconds."
in
    TextFormat

```

Funciones de valor total

Estas funciones devuelven el valor total de una duración en una sola unidad, incluidas las partes fraccionarias:

- `Duration.TotalDays(<duration>)`
- `Duration.TotalHours(<duration>)`
- `Duration.TotalMinutes(<duration>)`
- `Duration.TotalSeconds(<duration>)`

Power Query M

```

let
    Source =
    {
        Duration.TotalDays(#duration(1, 12, 0, 0)),    // 1.5 days
        Duration.TotalHours(#duration(1, 12, 0, 0)),    // 36 hours
        Duration.TotalMinutes(#duration(1, 12, 0, 0)), // 2160 minutes
        Duration.TotalSeconds(#duration(1, 12, 0, 0))  // 129600 seconds
    }
in
    Source

```

Normalización de la duración

En la mayoría de los casos, la duración se compone de días, horas (máximo 23 horas), minutos (máximo 59 minutos) y segundos (máximo 59,9999999 segundos). Sin embargo, en algunos casos puede superar los valores máximos en los parámetros de duración. En este caso, Power Query M normaliza automáticamente estos valores:

- Desbordamiento de segundos en minutos
- Desbordamiento de minutos en horas
- Desbordamiento de horas en días

Por ejemplo, supongamos que tiene una columna que proporciona la fecha y hora de inicio de un proceso en ejecución. Además, tiene una columna que muestra cuánto tiempo ha tardado

el proceso en completarse, en segundos. Desea crear una tercera columna que muestre la fecha y hora en que se completa el proceso.

```
Power Query M

let
    Source = #table(type table[StartTime = datetime, Seconds = Int64.Type],
        {
            {#datetime(2025, 7, 25, 8, 0, 0), 5400},
            {#datetime(2025, 7, 25, 13, 15, 0), 86400},
            {#datetime(2025, 7, 24, 22, 30, 0), 172800}
        }
    ),
    AddSeconds = Table.AddColumn(
        Source,
        "EndTime",
        each [StartTime] + #duration(0, 0, 0, [Seconds]),
        type datetime
    )
in
    AddSeconds
```

La tabla siguiente es el resultado de estos cálculos.

	 StartTime	 Seconds	 EndTime
1	7/25/2025, 8:00:00 AM	5400	7/25/2025, 9:30:00 AM
2	7/25/2025, 1:15:00 PM	86400	7/26/2025, 1:15:00 PM
3	7/24/2025, 10:30:00 PM	172800	7/26/2025, 10:30:00 PM

Por lo tanto, aunque solo tuviera el número de segundos que tuvo lugar un proceso, Power Query M acumula ese valor de duración en minutos, horas y días cuando se evalúa el resultado.

Representar semanas, meses y años

Dado que las duraciones se basan en unidades fijas (días, horas, minutos, segundos), no hay ningún concepto nativo de semanas, meses o años, que varían en longitud. Un `duration` tipo de Power Query M es una estructura fija que no tiene en cuenta las reglas de calendario. Durante los meses o años, resta una fecha y hora de otra en lugar de usar duraciones fijas. Este enfoque controla correctamente los años bisiestos, las distintas longitudes del mes y el horario de verano (DST). Sin embargo, tenga en cuenta también que algún comportamiento de fecha y hora puede diferir en función de si la consulta se ejecuta localmente (en Power Query Desktop) o en línea (en Power Query Online). Para obtener más información, vaya a [Variantes locales](#),

fijas y UTC de fecha y hora actuales. En general, evite confiar en duraciones fijas para los cálculos a largo plazo.

Cadenas con formato numérico estándar

Artículo • 18/12/2024

Las cadenas de formato numérico estándar se utilizan para dar formato a tipos numéricos comunes. La forma de una cadena de formato numérico estándar es `[format specifier][precision specifier]`, donde:

- El *especificador de formato* es un carácter alfabético único que especifica el tipo de formato numérico, como, por ejemplo, moneda o porcentaje. Cualquier cadena de formato numérico que contenga más de un carácter alfabético, incluido el espacio en blanco, se interpreta como una cadena de formato numérico personalizado. Para obtener más información, vaya a [Cadenas de formato numérico personalizado](#).
- El *especificador de precisión* es un entero opcional que afecta al número de dígitos de la cadena resultante. El especificador de precisión controla el número de dígitos en la representación de cadena de un número.

Cuando el especificador de precisión controla el número de dígitos fraccionarios en la cadena de resultado, la cadena de resultado refleja un número redondeado a un resultado representable más cercano al resultado infinitamente preciso.

⚠ Nota

El especificador de precisión determina el número de dígitos de la cadena de resultado. Para rellenar una cadena de resultado con espacios iniciales o finales u otros caracteres (como 0), use las funciones [Text.PadStart](#) y [Text.PadEnd](#) y use la longitud general `count` en estas funciones para rellenar la cadena de resultado.

Las cadenas de formato numérico estándar son compatibles con la función [Number.ToText](#).

Especificadores de formato estándar

En la tabla siguiente se describen los especificadores de formato numérico estándar y se muestran los resultados de ejemplo generados por cada especificador de formato. Vaya a la [sección Notas](#) para obtener información adicional sobre el uso de cadenas de

formato numérico estándar y la [sección Ejemplo de código](#) para obtener una ilustración completa de su uso.

ⓘ **Nota**

El resultado de una cadena con formato para una referencia cultural específica puede diferir de los ejemplos siguientes. La configuración del sistema operativo, la configuración del usuario, las variables de entorno y otras configuraciones del sistema que usa pueden afectar al formato.

 Expandir tabla

Especificador de formato	NOMBRE	Descripción	Ejemplos
"C" o "c"	Moneda	Resultado: un valor de divisa.	123.456 ("C", en-US) -> \ \$123.46
		Compatible con: todos los tipos numéricos.	123.456 ("C", fr-FR) -> 123,46 €
		Especificador de precisión: número de dígitos decimales.	123.456 ("C", ja-JP) -> ¥123
		Especificador de precisión predeterminado: definido por la referencia cultural.	-123.456 ("C3", en-US) -> (\ \$123.456)
		Más información: Especificador de formato de divisa ("C") .	-123.456 ("C3", fr-FR) -> -123,456 €
			-123.456 ("C3", ja-JP) -> -¥123.456
"D" o "d"	Decimal	Resultado: dígitos enteros con signo negativo opcional.	1234 ("D") -> 1234
		Compatible con: solo tipos enteros.	-1234 ("D6") -> -001234
		Especificador de precisión: número mínimo de dígitos.	
		Especificador de precisión predeterminado: número mínimo de dígitos necesario.	

Especificador de formato	NOMBRE	Descripción	Ejemplos
		Más información: Especificador de formato decimal ("D") .	
"E" o "e"	Exponencial (científico)	Resultado: notación exponencial. Compatible con: todos los tipos numéricos. Especificador de precisión: número de dígitos decimales. Especificador de precisión predeterminado: 6. Más información: Especificador de formato exponencial ("E").	1052.0329112756 ("E", en-US) -> 1.052033E+003 1052.0329112756 ("e", fr-FR) -> 1,052033e+003 -1052.0329112756 ("e2", en-US) -> -1.05e+003 -1052.0329112756 ("E2", fr-FR) -> -1,05E+003
"F" o "f"	Punto fijo	Resultado: dígitos integrales y decimales con signo negativo opcional. Compatible con: todos los tipos numéricos. Especificador de precisión: número de dígitos decimales. Especificador de precisión predeterminado: definido por la referencia cultural. Más información: Especificador de formato de punto fijo ("F").	1234.567 ("F", en-US) -> 1234.57 1234.567 ("F", de-DE) -> 1234,57 1234 ("F1", en-US) -> 1234.0 1234 ("F1", de-DE) -> 1234,0 -1234.56 ("F4", en-US) -> -1234.5600 -1234.56 ("F4", de-DE) -> -1234,5600
"G" o "g"	General	Resultado: notación de punto fijo o científica, la que sea más compacta. Compatible con: todos los tipos numéricos.	-123.456 ("G", en-US) -> -123.456 -123.456 ("G", sv-SE) -> -123,456

Especificador de formato	NOMBRE	Descripción	Ejemplos
		Especificador de precisión: número de dígitos significativos. Especificador de precisión predeterminado: depende del tipo numérico. Más información: Especificador de formato general ("G").	123.4546 ("G4", en-US) -> 123.5 123.4546 ("G4", sv-SE) -> 123,5 -1.234567890e-25 ("G", en-US) -> -1.23456789E-25 -1.234567890e-25 ("G", sv-SE) -> -1,23456789E-25
"N" o "n"	número	Resultado: dígitos integrales y decimales, separadores de grupos y un separador decimal con signo negativo opcional. Compatible con: todos los tipos numéricos. Especificador de precisión: número deseado de decimales. Especificador de precisión predeterminado: definido por la referencia cultural. Más información: Especificador de formato numérico ("N").	1234.567 ("N", en-US) -> 1,234.57 1234.567 ("N", ru-RU) -> 1 234,57 1234 ("N1", en-US) -> 1,234.0 1234 ("N1", ru-RU) -> 1 234,0 -1234.56 ("N3", en-US) -> -1,234.560 -1234.56 ("N3", ru-RU) -> -1 234,560
"P" o "p"	Porcentaje	Resultado: número multiplicado por 100 y mostrado con un símbolo de porcentaje. Compatible con: todos los tipos numéricos. Especificador de precisión: número	1 ("P", en-US) -> 100.00 % 1 ("P", fr-FR) -> 100,00 % -0.39678 ("P1", en-US)

Especificador de formato	NOMBRE	Descripción	Ejemplos
		deseado de decimales.	-> -39.7 %
		Especificador de precisión predeterminado: definido por la referencia cultural.	-0.39678 ("P1", fr-FR) -> -39,7 %
		Más información: Especificador de formato de porcentaje ("P") .	
"X" o "x"	Hexadecimal	Resultado: cadena hexadecimal. Compatible con: solo tipos enteros. Especificador de precisión: número de dígitos en la cadena de resultado. Más información: Especificador de formato hexadecimal (X) .	255 ("X") -> FF -1 ("x") -> ff 255 ("x4") -> 00ff -1 ("X4") -> 00FF
Cualquier otro carácter único	Especificador desconocido	Resultado: produce un error de expresión en tiempo de ejecución.	

Uso de Cadenas con formato numérico estándar

Se puede usar una cadena de formato numérico estándar para definir el formato de un valor numérico. Se puede pasar al parámetro `Number.ToText` `format`. En el ejemplo siguiente se da formato a un valor numérico como una cadena de divisa en la referencia cultural actual (en este caso, en-US).

Power Query M

```
Number.ToText(123.456, "C2")
// Displays $123.46
```

Opcionalmente, puede proporcionar un argumento `count` en las funciones `Text.PadStart` y `Text.PadEnd` para especificar el ancho del campo numérico y si su valor está alineado a la derecha o a la izquierda. Por ejemplo, el ejemplo siguiente alinea un valor de moneda en un campo de 28 caracteres y alinea a la derecha un valor de moneda en un campo de 14 caracteres (cuando se usa una fuente monoespacial).

```

let
    amounts = {16305.32, 18794.16},
    result = Text.Format("      Beginning Balance      Ending Balance#
(cr,lf)      #{0}#{1}",
    {
        Text.PadEnd(Number.ToText(amounts{0}, "C2"), 28),
        Text.PadStart(Number.ToText(amounts{1}, "C2"), 14)
    })
in
    result

// Displays:
//      Beginning Balance      Ending Balance
//      $16,305.32              $18,794.16

```

En las secciones siguientes se proporciona información detallada sobre cada una de las cadenas de formato numérico estándar.

Especificador de formato de moneda (C)

El especificador de formato "C" (divisa) convierte un número en una cadena que representa una cantidad de divisa. El especificador de precisión indica el número deseado de posiciones decimales en la cadena de resultado. Si se omite el especificador de precisión, el número predeterminado de posiciones decimales que se van a usar en los valores de moneda es 2.

Si el valor al que se va a dar formato tiene más posiciones decimales que el número especificado o predeterminado, el valor fraccionario se redondea en la cadena de resultado. Si el valor situado a la derecha del número de posiciones decimales especificadas es 5 o superior, el último dígito de la cadena de resultado se redondea desde cero.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a un valor con el especificador de formato de divisa:

```

let
    Source =
    {

```

```

        Number.ToText(12345.6789, "C"),
        Number.ToText(12345.6789, "C3"),
        Number.ToText(12345.6789, "C3", "da-DK")
    }
in
    Source

// The example displays the following list on a system whose
// current culture is English (United States):
//      $12,345.68
//      $12,345.679
//      12.345,679 kr.

```

Especificador de formato decimal (D)

El especificador de formato "D" (o decimal) convierte un número en una cadena de dígitos decimales (0-9), precedida por un signo menos si el número es negativo. Este formato sólo es compatible con los tipos enteros.

El especificador de precisión indica el número mínimo de dígitos deseado en la cadena resultante. Si es preciso, el número se rellena con ceros a la izquierda para generar el número de dígitos que aporta el especificador de precisión. Si no se indica ningún especificador de precisión, el valor predeterminado es el valor mínimo necesario para representar el entero sin ceros iniciales.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a un valor con el especificador de formato decimal.

Power Query M

```

let
    Source =
    {
        Number.ToText(12345, "D"),
        // Displays 12345

        Number.ToText(12345, "D8"),
        // Displays 00012345

        Number.ToText(-12345, "D"),
        // Displays -12345

        Number.ToText(-12345, "D8")
        // Displays -00012345
    }

```

in

Source

Especificador de formato exponencial (E)

El especificador de formato exponencial ("E") convierte un número en una cadena con el formato "-d.ddd...E+ddd" o "-d.ddd...e+ddd", donde cada "d" indica un dígito (0-9). La cadena comienza con un signo menos si el número es negativo. El separador decimal siempre va precedido por exactamente un dígito.

El especificador de precisión indica el número deseado de dígitos después del separador decimal. Si se omite el especificador de precisión, se emplea uno predeterminado que tiene seis dígitos después del separador decimal.

El modelo de mayúsculas o minúsculas del especificador de formato indica si se debe prefijar el exponente con una "E" o con una "e". El exponente siempre consta de un signo más o menos y de un mínimo de tres dígitos. El exponente se rellena con ceros para adaptarlo a este mínimo, si es necesario.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a un valor con el especificador de formato exponencial:

Power Query M

```
let
    Source =
    {
        Number.ToText(12345.6789, "E", ""),
        // Displays 1.234568E+004

        Number.ToText(12345.6789, "E10", ""),
        // Displays 1.2345678900E+004

        Number.ToText(12345.6789, "e4", ""),
        // 1.2346e+004

        Number.ToText(12345.6789, "E", "fr-FR")
        // Displays 1,234568E+004
    }
in
    Source
```

ⓘ Nota

El valor de texto en blanco ("") del último parámetro de [Number.ToText](#) del ejemplo anterior hace referencia a la referencia cultural invariable.

Especificador de formato de punto fijo (F)

El especificador de formato de punto fijo ("F") convierte un número en una cadena con el formato "-ddd.ddd...", donde cada "d" indica un dígito (0-9). La cadena comienza con un signo menos si el número es negativo.

El especificador de precisión indica el número deseado de cifras decimales. Si se omite el especificador de precisión, el número predeterminado de posiciones decimales que se van a usar en valores numéricos es 2.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a un valor double y un entero con el especificador de formato de punto fijo:

Power Query M

```
let
    Source =
    {
        Number.ToText(17843, "F", ""),
        // Displays 17843.00

        Number.ToText(-29541, "F3", ""),
        // Displays -29541.000

        Number.ToText(18934.1879, "F", ""),
        // Displays 18934.19

        Number.ToText(18934.1879, "F0", ""),
        // Displays 18934

        Number.ToText(-1898300.1987, "F1", ""),
        // Displays -1898300.2

        Number.ToText(-1898300.1987, "F3", "es-ES")
        // Displays -1898300,199
    }
in
    Source
```


Especificador de formato general (G)

El especificador de formato general ("G") convierte un número a la notación de punto fijo o científica más compacta, dependiendo del tipo del número y de si hay un especificador de precisión o no. El especificador de precisión define el número máximo de dígitos significativos que pueden aparecer en la cadena de resultado. Si el especificador de precisión se omite o es cero, el tipo del número determina la precisión predeterminada, como se indica en la tabla siguiente.

 Expandir tabla

Tipo numérico	Precisión predeterminada
Byte.Type o Int8.Type	3 dígitos
Int16.Type	5 dígitos
Int32.Type	10 dígitos
Int64.Type	19 dígitos
Single.Type	9 dígitos
Double.Type	17 dígitos
Decimal.Type	15 dígitos

La notación de punto fijo se utiliza si el exponente que resultaría de la expresión del número en notación científica es mayor que -5 y menor que el especificador de precisión, de lo contrario se utiliza la notación científica. El resultado contiene un separador decimal si es necesario y se omiten los ceros finales después de ese separador. Si el especificador de precisión está presente y el número de dígitos significativos del resultado supera la precisión especificada, los dígitos finales sobrantes se quitan mediante redondeo.

Sin embargo, si el número es `Decimal.Type` y se omite el especificador de precisión, siempre se usa la notación de punto fijo y se conservan los ceros finales.

Si se usa la notación científica, el exponente del resultado lleva el prefijo "E" si el especificador de formato es "G", o "e" si el especificador de formato es "g". El exponente contiene un mínimo de dos dígitos. Esto difiere del formato para la notación científica que genera el especificador de formato exponencial, que incluye un mínimo de tres dígitos en el exponente.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a valores de punto flotante ordenados con el especificador de formato general:

Power Query M

```
let
    Source =
    {
        Number.ToText(12345.6789, "G", ""),
        // Displays 12345.6789

        Number.ToText(12345.6789, "G", "fr-FR"),
        // Displays 12345,6789

        Number.ToText(12345.6789, "G7", ""),
        // Displays 12345.68

        Number.ToText(.0000023, "G", ""),
        // Displays 2.3E-06

        Number.ToText(.0000023, "G", "fr-FR"),
        // Displays 2,3E-06

        Number.ToText(.0023, "G", ""),
        // Displays 0.0023

        Number.ToText(1234, "G2", ""),
        // Displays 1.2E+03

        Number.ToText(Number.PI, "G5", "")
        // Displays 3.1416
    }
in
    Source
```

Especificador de formato numérico (N)

El especificador de formato numérico ("N") convierte un número en una cadena con el formato "-d,ddd,ddd.ddd...", donde "-" indica el símbolo de número negativo, si es necesario; "d", representa cada dígito (0-9); "," es el separador de grupo; y "." es el símbolo de punto decimal. El especificador de precisión indica el número deseado de dígitos después del separador decimal. Si se omite el especificador de precisión, la referencia cultural actual define el número de posiciones decimales.

La cadena de resultado se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a valores de punto flotante ordenados con el especificador de formato numérico:

Power Query M

```
let
    Source =
    {
        Number.ToText(-12445.6789, "N", ""),
        // Displays -12,445.68

        Number.ToText(-12445.6789, "N1", "sv-SE"),
        // Displays -12 445,7

        Number.ToText(123456789, "N1", "")
        // Displays 123,456,789.0
    }
in
    Source
```

Especificador de formato de porcentaje (P)

El especificador de formato de porcentaje ("P") multiplica un número por 100 y lo convierte en una cadena que representa un porcentaje. El especificador de precisión indica el número deseado de cifras decimales. Si se omite el especificador de precisión, se usa la precisión numérica predeterminada proporcionada por la referencia cultural actual.

En el ejemplo siguiente se da formato a valores de punto flotante con el especificador de formato de porcentaje:

Power Query M

```
let
    Source =
    {
        Number.ToText(.2468013, "P", ""),
        // Displays 24.68 %

        Number.ToText(.2468013, "P", "hr-HR"),
        // Displays 24,68 %

        Number.ToText(.2468013, "P1", "en-US")
        // Displays 24.7%
    }
in
    Source
```

in

Source

Especificador de formato hexadecimal (X)

El especificador de formato hexadecimal ("X") convierte un número en una cadena de dígitos hexadecimales. El modelo de mayúsculas y minúsculas del especificador de formato indica si se van a usar caracteres en mayúsculas o en minúsculas para los dígitos hexadecimales mayores de 9. Por ejemplo, use "X" para generar "ABCDEF" y "x" para generar "abcdef". Este formato sólo es compatible con los tipos enteros.

El especificador de precisión indica el número mínimo de dígitos deseado en la cadena resultante. Si es preciso, el número se rellena con ceros a la izquierda para generar el número de dígitos que aporta el especificador de precisión.

La cadena de resultado no se ve afectada por la información de formato de la referencia cultural actual.

En el ejemplo siguiente se da formato a valores con el especificador de formato hexadecimal.

Power Query M

```
let
    Source =
    {
        Number.ToText(0x2045e, "x"),
        // Displays 2045e

        Number.ToText(0x2045e, "X"),
        // Displays 2045E

        Number.ToText(0x2045e, "X8"),
        // Displays 0002045E

        Number.ToText(123456789, "X"),
        // Displays 75BCD15

        Number.ToText(123456789, "X2")
        // Displays 75BCD15
    }
in
    Source
```

Notas

Esta sección contiene información adicional sobre el uso de cadenas de formato numérico estándar.

Tipos numéricos enteros y de punto flotante

Algunas descripciones de especificadores de formato numérico estándar hacen referencia a tipos numéricos enteros o de punto flotante. Los tipos numéricos enteros son `Byte.Type`, `Int8.Type`, `Int16.Type`, `Int32.Type` y `Int64.Type`. Los tipos numéricos de punto flotante son `Decimal.Type`, `Single.Type` y `Double.Type`.

Infinitos de punto flotante y NaN

Independientemente de la cadena de formato, si el valor de un `Decimal.Type`, `Single.Type` o `Double.Type` tipo de punto flotante es infinito positivo, infinito negativo o no un número (NaN), la cadena con formato es el valor del `Number.PositiveInfinity` respectivo, `Number.NegativeInfinity` o `Number.NaN` constantes especificadas por la referencia cultural aplicable actualmente.

Ejemplo de código

En el ejemplo siguiente se da formato a un punto flotante y un valor numérico entero mediante la referencia cultural en-US y todos los especificadores de formato numérico estándar. En este ejemplo se usan dos tipos numéricos concretos (`Double.Type` y `Int32.Type`), pero produciría resultados similares para cualquiera de los otros tipos base numéricos (`Byte.Type`, `Decimal.Type`, `Int8.Type`, `Int16.Type`, `Int64.Type` y `Single.Type`).

Power Query M

```
let
    // Display text representations of numbers for en-US culture
    culture = "en-US",

    // Output floating point values
    floating = Double.From(10761.937554),
    #"Floating results" =
    {
        Text.Format("C: #{0}", {Number.ToText(floating, "C", culture)}),
    // Displays "C: $10,761.94"
        Text.Format("E: #{0}", {Number.ToText(floating, "E03", culture)}),
    // Displays "E: 1.076E+004"
        Text.Format("F: #{0}", {Number.ToText(floating, "F04", culture)}),
    // Displays "F: 10761.9376"
        Text.Format("G: #{0}", {Number.ToText(floating, "G", culture)}),
    // Displays "G: 10761.937554"
```

```

        Text.Format("N: #{0}", {Number.ToText(floating, "N03", culture)}),
// Displays "N: 10,761.938"
        Text.Format("P: #{0}", {Number.ToText(floating/10000, "P02",
culture)}) // Displays "P: 107.62%"
    },

    // Output integral values
    integral = Int32.From(8395),
    #"Integral results" =
    {
        Text.Format("C: #{0}", {Number.ToText(integral, "C", culture)}),
// Displays "C: $8,395.00"
        Text.Format("D: #{0}", {Number.ToText(integral, "D6", culture)}),
// Displays "D: 008395"
        Text.Format("E: #{0}", {Number.ToText(integral, "E03", culture)}),
// Displays "E: 8.395E+003"
        Text.Format("F: #{0}", {Number.ToText(integral, "F01", culture)}),
// Displays "F: 8395.0"
        Text.Format("G: #{0}", {Number.ToText(integral, "G", culture)}),
// Displays "G: 8395"
        Text.Format("N: #{0}", {Number.ToText(integral, "N01", culture)}),
// Displays "N: 8,395.0"
        Text.Format("P: #{0}", {Number.ToText(integral/10000, "P02",
culture)}), // Displays "P: 83.95%"
        Text.Format("X: 0x#{0}", {Number.ToText(integral, "X", culture)})
// Displays "X: 0x20CB"
    },
    results = #"Floating results" & #"Integral results"

in
    results

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Conversión de tipos de número](#)
- [Tipos de datos en Power Query](#)
- [Cadenas con formato numérico personalizado](#)

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Cadenas con formato numérico personalizado

Artículo • 20/11/2024

Puede crear una cadena de formato numérico personalizado, formada por uno o varios especificadores numéricos personalizados, para definir cómo debe darse formato a los datos numéricos. Una cadena de formato numérico personalizada es cualquier cadena de formato que no sea una [cadena](#) de formato numérico estándar.

En la tabla siguiente se describen los especificadores de formato numérico personalizado y se muestran las salidas de ejemplo generadas por cada especificador de formato. Vaya a la [sección Notas](#) para obtener información adicional sobre el uso de cadenas de formato numérico personalizado y la [sección Ejemplo](#) para obtener una ilustración completa de su uso.

[Expandir tabla](#)

Especificador de formato	NOMBRE	Descripción	Ejemplos
"0"	Marcador de posición cero	Reemplaza el cero con el dígito correspondiente si hay alguno presente; de lo contrario, el cero aparece en la cadena de resultado. Más información: Especificador personalizado "0" .	1234.5678 ("00000") -> 01235 0.45678 ("0.00", en-US) -> 0.46 0.45678 ("0.00", fr-FR) -> 0,46
"#"	Marcador de posición de dígito.	Reemplaza el símbolo "#" por el dígito correspondiente si hay alguno presente; de lo contrario, no aparece ningún dígito en la cadena de resultado. No aparece ningún dígito en la cadena de resultado si el dígito correspondiente de la cadena de entrada es un no significativo 0. Por ejemplo, 0003 ("#####") -> 3. Más información: Especificador personalizado "#" .	1234.5678 ("#####") -> 1235 0.45678 ("#.##", en-US) -> .46 0.45678 ("#.##", fr-FR) -> ,46
"."	Separador decimal	Determina la ubicación del separador decimal en la cadena de resultado.	0.45678 ("0.00", en-US) -> 0.46

Especificador de formato	NOMBRE	Descripción	Ejemplos
		Más información: El especificador personalizado "." .	0.45678 ("0.00", fr-FR) -> 0,46
","	Separador de grupos y escala numérica	Actúa como separador de grupos y como especificador de escala numérica. Como separador de grupos, inserta un carácter separador de grupos adaptado entre cada grupo. Como especificador de escala numérica, divide un número por 1000 por cada coma especificada. Más información: Especificador personalizado "," .	Especificador de separador de grupos: 2147483647 ("##,#", en-US) -> 2,147,483,647 2147483647 ("##,#", es-ES) -> 2.147.483.647 Especificador de escala: 2147483647 ("#,#,," , en-US) -> 2,147 2147483647 ("#,#,," , es-ES) -> 2.147
"%"	Marcador de posición de porcentaje.	Multiplica un número por 100 e inserta un símbolo de porcentaje adaptado en la cadena de resultado. Más información: Especificador personalizado "%" .	0.3697 ("%#0.00", en-US) -> %36.97 0.3697 ("%#0.00", el-GR) -> %36,97 0.3697 ("##.0 %", en-US) -> 37.0 % 0.3697 ("##.0 %", el-GR) -> 37,0 %
"‰"	Marcador de posición de "por mil"	Multiplica un número por 1000 e inserta un símbolo de "por mil" adaptado en la cadena de resultado. Más información: Especificador personalizado "‰" .	0.03697 ("#0.00‰", en-US) -> 36.97‰ 0.03697 ("#0.00‰", ru-RU) -> 36,97‰

Especificador de formato	NOMBRE	Descripción	Ejemplos
"E0" "E+0" "E-0" "E0" "E+0" "E-0"	Notación exponencial	Si va seguido al menos de un 0 (cero), da formato al resultado usando notación exponencial. El modelo de mayúsculas de "E" o "e" indica el modelo de mayúsculas del símbolo de exponente en la cadena de resultado. El número de ceros que siguen al carácter "E" o "e" determina el número mínimo de dígitos en el exponente. Un signo más (+) indica que un carácter de signo precede siempre al exponente. Un signo menos (-) indica que un carácter de signo solo precede a los exponentes negativos. Más información: Especificadores personalizados "E" y "e" .	987654 ("#0.0e0") -> 98.8e4 1503.92311 ("0.0##e+00") -> 1.504e+03 1.8901385E-16 ("0.0e+00") -> 1.9e-16
"\\", "''", "''''"	Carácter de escape	Hace que el siguiente carácter o caracteres se interpreten como literal en lugar de como especificador de formato personalizado. Más información: Caracteres de escape .	987654 ("\\###00\\#") -> #987654# 987654 ("'###00'#") -> #987654# 987654 ("''###00''''") -> #987654#
'cadena' "string"	Delimitador de cadena literal	Indica que los caracteres que encierra se deben copiar en la cadena de resultado sin modificar. Más información: Literales de carácter .	68 ("# 'grados") - > 68 grados 68 ("#' grados") - > 68 grados
;	Separador de secciones	Define secciones con cadenas de formato diferentes para los números positivos, negativos y cero. Más información: Separador de sección ";" .	12.345 ("#0.0#; (#0.0#);-\\0-") -> 12.35 0 ("#0.0#;(#0.0#);-\\0-") -> -0- -12.345 ("#0.0#; (#0.0#);-\\0-") -> (12.35) 12.345 ("#0.0#;

Especificador de formato	NOMBRE	Descripción	Ejemplos
			(#0.0#) -> 12.35 0 ("#0.0#;(#0.0#)") -> 0.0 -12.345 ("#0.0#;(#0.0#)") -> (12.35)
Otros	Todos los demás caracteres	El carácter se copia en la cadena de resultado sin modificar. Más información: Literales de carácter .	68 ("# °") -> 68 °

En las secciones siguientes se proporciona información detallada sobre cada uno de los especificadores de formato numérico personalizado.

Especificador personalizado "0"

El especificador de formato personalizado "0" actúa como un símbolo de marcador de posición cero. Si el valor al que se está dando formato tiene un dígito en la posición donde aparece el cero en la cadena de formato, se copia ese dígito a la cadena de resultado; de lo contrario, aparecerá un cero en la cadena de resultado. La posición del cero que aparece más a la izquierda antes del separador decimal y la del cero que está más a la derecha después del separador decimal determinan el intervalo de dígitos que están siempre presentes en la cadena de resultado.

El especificador "00" hace que el valor se redondee al dígito más próximo que precede al decimal, donde siempre se utiliza el redondeo para evitar el cero. Por ejemplo, al aplicar el formato a 34.5 con "00" el resultado del valor es 35.

En el ejemplo siguiente se muestran varios valores a los que se les ha aplicado cadenas de formato personalizado que incluyen marcadores de posición cero.

Power Query M

```
let
    Source =
    {
        Number.ToText(123, "00000", ""),
        // Displays 00123

        Number.ToText(1.2, "0.00", ""),
        // Displays 1.20
    }
```

```
Number.ToText(1.2, "00.00", ""),  
// Displays 01.20  
  
Number.ToText(1.2, "00.00", "da-DK"),  
// Displays 01,20  
  
Number.ToText(.56, "0.0", ""),  
// Displays 0.6  
  
Number.ToText(1234567890, "0,0", ""),  
// Displays 1,234,567,890  
  
Number.ToText(1234567890, "0,0", "el-GR"),  
// Displays 1.234.567.890  
  
Number.ToText(1234567890.123456, "0,0.0", ""),  
// Displays 1,234,567,890.1  
  
Number.ToText(1234.567890, "0,0.00", "")  
// Displays 1,234.57  
}  
in  
Source
```

ⓘ Nota

El valor de texto en blanco ("") del último parámetro de [Number.ToText](#) del ejemplo anterior hace referencia a la referencia cultural invariable.

[Volver a la tabla](#)

Especificador personalizado "#"

El especificador de formato personalizado "#" actúa como un símbolo de marcador de posición de dígitos. Si el valor al que se está dando formato tiene un dígito en la posición donde aparece el símbolo "#" en la cadena de formato, ese dígito se copia a la cadena de resultado. En caso contrario, no se almacena nada en esa posición de la cadena de resultado.

Tenga en cuenta que este especificador nunca muestra un cero que no es un dígito significativo, incluso si cero es el único dígito de la cadena. Solo muestra cero si es un dígito significativo en el número que se muestra.

La cadena de formato "###" hace que el valor se redondee al dígito más próximo que precede al decimal, donde siempre se utiliza el redondeo para evitar el cero. Por ejemplo, al aplicar el formato a 34.5 con "###" el resultado del valor es 35.

En el ejemplo siguiente se muestran varios valores a los que se les ha aplicado cadenas de formato personalizado que incluyen marcadores de posición de dígitos.

Power Query M

```
let
    Source =
    {
        Number.ToText(1.2, "#.##", ""),
        // Displays 1.2

        Number.ToText(123, "#####"),
        // Displays 123

        Number.ToText(123456, "[##-##-##]"),
        // Displays [12-34-56]

        Number.ToText(1234567890, "#"),
        // Displays 1234567890

        Number.ToText(1234567890, "(###) ###-#####")
        // Displays (123) 456-7890
    }
in
    Source
```

Para devolver una cadena de resultado en la que los dígitos ausentes o los ceros iniciales se reemplazan por espacios, use el [Text.PadStart](#) y especifique un ancho de campo, como se muestra en el ejemplo siguiente.

Power Query M

```
let
    Source = Text.Format("The value is: '#{0}'",
        {Text.PadStart(Number.ToText(.324, "#.###"), 5)})
in
    Source

// The example displays the following output if the current culture
// is en-US:
//      The value is: ' .324'
```

[Volver a la tabla](#)

Especificador personalizado "."

El especificador de formato personalizado "." inserta un separador decimal localizado en la cadena del resultado. El primer punto de la cadena de formato determina la ubicación

del separador decimal en el valor con formato y se omite cualquier punto adicional. Si el especificador de formato termina con "." solo se da formato a los dígitos significativos en la cadena de resultado.

El carácter que se usa como separador decimal en la cadena de resultado no siempre es un punto; viene determinado por la referencia cultural que controla el formato.

En el ejemplo siguiente se utiliza el especificador de formato "." para definir la ubicación del separador decimal en varias cadenas de resultado.

Power Query M

```
let
    Source =
    {
        Number.ToText(1.2, "0.00", ""),
        // Displays 1.20

        Number.ToText(1.2, "00.00", ""),
        // Displays 01.20

        Number.ToText(1.2, "00.00", "da-DK"),
        // Displays 01,20

        Number.ToText(.086, "#0.##%", ""),
        // Displays 8.6%

        Number.ToText(Double.From(86000), "0.###E+0", "")
        // Displays 8.6E+4
    }
in
    Source
```

[Volver a la tabla](#)

Especificador personalizado ","

El carácter "," actúa como separador de grupos y como especificador de escala numérica.

- Separador de grupos: si se especifican una o varias comas dos marcadores de posición de dígitos (0 o #) que dan formato a los dígitos enteros de un número, se insertará un carácter separador de grupos entre cada grupo de números en la parte entera de la salida.

La referencia cultural determina el carácter utilizado como separador de grupo de números y el tamaño de cada grupo de números. Por ejemplo, si se utiliza la

cadena "#,#" y la referencia cultural de todos los idiomas para dar formato al número 1000, el resultado será "1,000".

- Especificador de escala numérica: si se especifican una o varias comas inmediatamente a la izquierda del signo decimal explícito o implícito, el número al que se va a dar formato se divide por 1000 por cada coma. Por ejemplo, si se utiliza la cadena "0,," para dar formato al número 100 millones, el resultado será "100".

Puede usar especificadores de separador de grupos y de escala numérica en la misma cadena de formato. Por ejemplo, si se utiliza la cadena "#,0,," y la referencia cultural de todos los idiomas para dar formato al número mil millones, el resultado será "1,000".

En el ejemplo siguiente se muestra el uso de la coma como separador de grupos.

Power Query M

```
let
    Source =
    {
        Number.ToText(1234567890, "#,#", ""),
        // Displays 1,234,567,890

        Number.ToText(1234567890, "#,##0,,", "")
        // Displays, 1,235
    }
in
    Source
```

En el ejemplo siguiente se muestra el uso de la coma como especificador de escala numérica.

Power Query M

```
let
    Source =
    {
        Number.ToText(1234567890, "#,,", ""),
        // Displays 1235

        Number.ToText(1234567890, "#,,,", ""),
        // Displays 1

        Number.ToText(1234567890, "#,##0,,", "")
        // Displays 1,235
    }
in
    Source
```

[Volver a la tabla](#)

Especificador personalizado "%"

Un signo de porcentaje (%) en una cadena de formato hace que un número se multiplique por 100 antes de que tenga formato. El símbolo de porcentaje adaptado se inserta en el número en la ubicación donde aparece % en la cadena de formato. El carácter de porcentaje usado se define mediante la referencia cultural.

En el ejemplo siguiente se define una cadena de formato personalizada que incluye el especificador personalizado "%".

Power Query M

```
let
    Source = Number.ToText(.086, "#0.##%", "")
    // Displays 8.6%
in
    Source
```

[Volver a la tabla](#)

Especificador personalizado "%o"

Un carácter por mil (* o \u2030) en una cadena de formato hace que un número se multiplique por 1000 antes de que tenga formato. El símbolo de "por mil" adecuado se inserta en la cadena devuelta, en la ubicación de la cadena de formato en la que aparece el símbolo %o. El carácter por mil usado se define mediante la referencia cultural, que proporciona información de formato específica de la referencia cultural.

En el ejemplo siguiente se define una cadena de formato personalizado que incluye el especificador personalizado "%o".

Power Query M

```
let
    Source = Number.ToText(.00354, "#0.##" & Character.FromNumber(0x2030),
    "")
    // Displays 3.54%
in
    Source
```

[Volver a la tabla](#)

Especificadores personalizados "E" y "e"

Si alguna de las cadenas "E", "E+", "E-", "e", "e+", o "e-" está presente en la cadena de formato y va seguida inmediatamente de al menos un cero, se da formato al número mediante notación científica con una 'E' o una 'e' insertadas entre el número y el exponente. El número de ceros que hay a continuación del indicador de notación científica determina el número mínimo de dígitos para el exponente. Los formatos 'E+' y 'e+' indican que un signo más o un signo menos debe preceder siempre al exponente. Los formatos 'E', 'E-', 'e' o 'e-' indican que un carácter de signo debe preceder solo a exponentes negativos.

En el ejemplo siguiente se da formato a varios valores numéricos utilizando los especificadores de notación científica.

Power Query M

```
let
    Source =
    {
        Number.ToText(86000, "0.###E+0", ""),
        // Displays 8.6E+4

        Number.ToText(86000, "0.###E+000", ""),
        // Displays 8.6E+004

        Number.ToText(86000, "0.###E-000", "")
        // Displays 8.6E004
    }
in
    Source
```

[Volver a la tabla](#)

Carácter de escape

Los símbolos "#", "0", ".", ",", "%", y "%" en una cadena de formato se interpretan como especificadores de formato en lugar de como caracteres literales. Dependiendo de su posición en una cadena de formato personalizado, los símbolos "E" mayúsculas y minúsculas, así como los símbolos + y - también se pueden interpretar como especificadores de formato.

Para evitar que un carácter se interprete como especificador de formato, puede hacer lo siguiente:

- Precedítelo con una barra diagonal inversa.

- Envolverlo con una sola comilla.
- Envolverlo con dos comillas dobles.

Cada uno de estos caracteres actúa como caracteres de escape. El carácter de escape significa que el siguiente carácter es un carácter literal que se debe incluir en la cadena de resultado sin modificar.

Para incluir una barra diagonal inversa en una cadena de resultado, debe indicar su secuencia de escape con otra barra diagonal inversa (`\\`).

Para incluir una comilla simple en una cadena de resultado, debe escaparla con una barra diagonal inversa (`\'`). Si otra comilla simple que no tiene escape precede a la comilla simple de escape, se muestra la barra diagonal inversa en su lugar (`'\'` muestra `\`).

Para incluir una comilla doble en una cadena de resultado, debe escapar dos de ellas con una barra diagonal inversa (`\"`).

En el ejemplo siguiente se usan caracteres de escape para evitar que la operación de formato interprete los caracteres "#", "0" y "" como caracteres de escape o especificadores de formato.

Power Query M

```
let
    Source =
    {
        Number.ToText(123, "\#\#\# ##0 dollars and \0\0 cents \#\#\#"),
        // Displays ### 123 dollars and 00 cents ###

        Number.ToText(123, "'###' ##0 dollars and '00' cents '###'"),
        // Displays ### 123 dollars and 00 cents ###

        Number.ToText(123, "\"###\" ##0 dollars and \"00\" cents \"###\""),
        // Displays ### 123 dollars and 00 cents ###

        Number.ToText(123, "\\\\\\ \\ ##0 dollars and \\0\\0 cents \\\\\\\\"),
        // Displays \\ 123 dollars and 00 cents \\

        Number.ToText(123, "'\\\\' ##0 dollars and '00' cents '\\\\'"),
        // Displays \\ 123 dollars and 00 cents \\

        Number.ToText(123, "\"\\\\\" ##0 dollars and \"00\" cents \"\\\\\""),
        // Displays \\ 123 dollars and 00 cents \\
    }
in
    Source
```

Separador de sección ":"

El punto y coma (;) es un especificador de formato condicional que aplica distinto formato a un número dependiendo de si su valor es positivo, negativo o cero. Para generar este comportamiento, una cadena de formato personalizado puede contener hasta tres secciones separadas por signos de punto y coma. Estas secciones se describen en la siguiente tabla.

[Expandir tabla](#)

Número de secciones	Descripción
Una sección	La cadena de formato se aplica a todos los valores.
Dos secciones	La primera sección se aplica a valores positivos y ceros, y la segunda, sólo a valores negativos. Si el número al que se va a dar formato es negativo, pero se convierte en cero después de redondearlo según el formato de la segunda sección, se da formato al cero resultante según la primera sección.
Tres secciones.	La primera sección se aplica a valores positivos y ceros, la segunda, sólo a valores negativos, y la tercera, a ceros. La segunda sección se puede dejar vacía (no dejando nada entre los signos de punto y coma) y, en ese caso, la primera sección se aplica a los valores distintos de cero. Si el número al que se va a dar formato es distinto de cero, pero se convierte en cero después de redondearlo según el formato de la primera o la segunda sección, se da formato al cero resultante según la tercera sección.

Los separadores de sección omiten cualquier formato preexistente asociado a un número al dar formato al valor final. Por ejemplo, los valores negativos se muestran siempre con signo menos cuando se utilizan separadores de sección. Si se desea que el valor con formato final tenga un signo menos, debe incluir explícitamente el signo menos como parte del especificador de formato personalizado.

En el ejemplo siguiente se usa el especificador de formato ":" para aplicar un formato diferente a los números positivos, negativos y cero.

```

let
    Source =
    {
        Number.ToText(1234, "##;(##)",
            // Displays 1234

        Number.ToText(-1234, "##;(##)",
            // Displays (1234)

        Number.ToText(0, "##;(##);**Zero**")
            // Displays **Zero**
    }
in
    Source

```

[Volver a la tabla](#)

Literales de carácter

Los especificadores de formato que aparecen en una cadena de formato numérico personalizado siempre se interpretan como caracteres de formato y no como caracteres literales. Se incluyen los caracteres siguientes:

- 0
- #
- %
- ‰
- '
- \
- ""
- .
- ,
- E o e, según su posición en la cadena de formato.

Todos los demás caracteres se interpretan siempre como literales de carácter y, en una operación de formato, se incluyen en la cadena de resultado sin modificar. En una operación de análisis, deben coincidir exactamente con los caracteres de la cadena de entrada; la comparación distingue entre mayúsculas y minúsculas.

En el ejemplo siguiente se muestra un uso habitual de las unidades de carácter literal (en este caso, los millares):

```
let
    Source = Number.ToText(123.8, "#,##0.0K")
    // Displays 123.8K
in
    Source
```

Hay dos formas de indicar que los caracteres se han de interpretar como caracteres literales y no como caracteres de formato, para que se puedan incluir en una cadena de resultado o analizarse correctamente en una cadena de entrada:

- Mediante el escape de un carácter de formato. Para obtener más información, vaya a [Caracteres de escape](#).
- Mediante la inclusión de toda la cadena literal entre comillas o apóstrofes.

En el ejemplo siguiente se usan los dos enfoques para incluir caracteres reservados en una cadena de formato numérico personalizada.

Power Query M

```
let
    Source =
    {
        Number.ToText(9.3, "##.0\%"),
        // Displays 9.3%

        Number.ToText(9.3, "\"'##'\""),
        // Displays '9'

        Number.ToText(9.3, "\\##\\"),
        // Displays \9\

        Number.ToText(9.3, "##.0'%\""),
        // Displays 9.3%

        Number.ToText(9.3, "\"'\"'##'\""),
        // Displays \9\

        Number.ToText(9.3, "##.0\"\"%\"\"\""),
        // Displays 9.3%

        Number.ToText(9.3, "\"\"##\"\"\"")
        // Displays "9"
    }
in
    Source
```

Notas

Infinitos de punto flotante y NaN

Independientemente de la cadena de formato, si el valor de un `Decimal.Type`, `Single.Type` o `Double.Type` tipo de punto flotante es infinito positivo, infinito negativo o no un número (NaN), la cadena con formato es el valor del `Number.PositiveInfinity` respectivo, `Number.NegativeInfinity` o `Number.NaN` constantes especificadas por la referencia cultural aplicable actualmente.

Cadenas de formato de punto fijo y redondeo

Para las cadenas de formato de punto fijo (es decir, cadenas de formato que no contienen caracteres de formato de notación científica), los números se redondean a tantas posiciones decimales como hay marcadores de posición de dígitos a la derecha del separador decimal. Si la cadena de formato no contiene un separador decimal, el número se redondea al entero más cercano. Si el número tiene más dígitos que marcadores de posición de dígitos a la izquierda del separador decimal, los dígitos adicionales se copian en la cadena de resultado justo antes del primer marcador de posición de dígitos.

[Volver a la tabla](#)

Ejemplo

En el siguiente ejemplo se muestran dos cadenas de formato numérico personalizado. En ambos casos, el marcador de posición de dígitos (`#`) muestra los datos numéricos, y todos los demás caracteres se copian en la cadena de resultado.

Power Query M

```
let
    Source =
    {
        Number.ToText(1234567890, "(###) ###-####"),
        // Displays (123) 456-7890

        Number.ToText(42, "My Number = #")
        // Displays My number = 42
    }
in
    Source
```

[Volver a la tabla](#)

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Conversión de tipos de número](#)
- [Tipos de datos en Power Query](#)
- [Cadenas con formato numérico estándar](#)

Comentarios

¿Le ha resultado útil esta página?



[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Cadenas con formato de fecha y hora estándar

Artículo • 20/11/2024

Una cadena de formato de fecha y hora estándar usa un solo carácter como especificador de formato para definir la representación de texto de un valor de fecha y hora. Cualquier cadena con formato de fecha y hora que contenga más de un carácter, incluido un espacio en blanco, se interpreta como una [cadena con formato de fecha y hora personalizado](#). Se puede usar una cadena de formato estándar o personalizada para definir la representación de texto resultante de una operación de formato.

Tabla de especificadores de formato

En la tabla siguiente se describen los especificadores de formato de fecha y hora estándar.

 Expandir tabla

Especificador de formato	Descripción	Ejemplos
"d"	Patrón de fecha corta. Más información: Especificador de formato de fecha corta ("d") .	2009-06-15T13:45:30 -> 6/15/2009 (en-US) 2009-06-15T13:45:30 -> 15/06/2009 (fr-FR) 2009-06-15T13:45:30 -> 2009/06/15 (ja-JP)
"D"	Patrón de fecha larga. Más información: Especificador de formato de fecha larga ("D") .	2009-06-15T13:45:30 -> Monday, June 15, 2009 (en-US) 2009-06-15T13:45:30 -> понедельник, 15 июня 2009 г. (ru-RU) 2009-06-15T13:45:30 -> Montag, 15. Juni 2009 (de-DE)
"f"	Patrón de fecha y hora completa (hora corta). Más información: El especificador	2009-06-15T13:45:30 -> Monday, June 15, 2009 1:45 PM (en-US) 2009-06-15T13:45:30 -> den 15 juni 2009 13:45 (sv-SE)

Especificador de formato	Descripción	Ejemplos
	de formato de fecha completa y hora corta ("f").	2009-06-15T13:45:30 -> Δευτέρα, 15 Ιουνίου 2009 1:45 μμ (el-GR)
"F"	Patrón de fecha y hora completa (hora larga).	2009-06-15T13:45:30 -> Monday, June 15, 2009 1:45:30 PM (en-US)
	Más información: El especificador de formato de fecha completa y hora larga ("F") .	2009-06-15T13:45:30 -> den 15 juni 2009 13:45:30 (sv-SE)
		2009-06-15T13:45:30 -> Δευτέρα, 15 Ιουνίου 2009 1:45:30 μμ (el-GR)
"g"	Patrón de fecha y hora general (hora corta).	2009-06-15T13:45:30 -> 6/15/2009 1:45 PM (en-US)
	Más información: El especificador de formato de fecha general y hora corta ("g") .	2009-06-15T13:45:30 -> 15/06/2009 13:45 (es-ES)
		2009-06-15T13:45:30 -> 2009/6/15 13:45 (zh-CN)
"G"	Patrón de fecha y hora general (hora larga).	2009-06-15T13:45:30 -> 6/15/2009 1:45:30 PM (en-US)
	Más información: El especificador de formato de fecha general y hora larga ("G") .	2009-06-15T13:45:30 -> 15/06/2009 13:45:30 (es-ES)
		2009-06-15T13:45:30 -> 2009/6/15 13:45:30 (zh-CN)
"M", "m"	Patrón de mes/día.	2009-06-15T13:45:30 -> June 15 (en-US)
	Más información: El especificador de formato de mes ("M", "m") .	2009-06-15T13:45:30 -> 15. juni (da-DK)
		2009-06-15T13:45:30 -> 15 Juni (id-ID)
"O", "o"	Patrón de fecha y hora de ida y vuelta.	2009-06-15T13:45:30 (Local):> 2009-06-15T13:45:30.0000000-07:000
	Más información: El especificador de formato de operación de ida y vuelta ("O", "o") .	2009-06-15T13:45:30 (Utc) --> 2009-06-15T13:45:30.0000000+00:000
		2009-06-15T13:45:30 (Sin especificar):> 2009-06-15T13:45:30.00000000

Especificador de formato	Descripción	Ejemplos
"R", "r"	Patrón RFC1123. Más información: El especificador de formato RFC1123 ("R", "r").	2009-06-15T13:45:30 -> Mon, 15 jun 2009 13:45:30 GMT
"s"	Patrón de fecha y hora que se puede ordenar. Más información: El especificador de formato que se puede ordenar ("s").	2009-06-15T13:45:30 (Local) -> 2009-06-15T13:45:30 2009-06-15T13:45:30 (Utc) -> 2009-06-15T13:45:30
"t"	Patrón de hora corta. Más información: El especificador de formato de hora corta ("t").	2009-06-15T13:45:30 -> 1:45 PM (en-US) 2009-06-15T13:45:30 -> 13:45 (hr-HR) 2009-06-15T13:45:30 -> 01:45 μ (ar-EG)
"T"	Patrón de hora larga. Más información: El especificador de formato de hora larga ("T").	2009-06-15T13:45:30 -> 1:45:30 PM (en-US) 2009-06-15T13:45:30 -> 13:45:30 (hr-HR) 2009-06-15T13:45:30 -> 01:45:30 μ (ar-EG)
"u"	Patrón de fecha y hora universal que se puede ordenar. Más información: El especificador de formato universal que se puede ordenar ("u").	2009-06-15T13:45:30 -> 2009-06-15 13:45:30Z
"Y", "y"	Patrón de mes y año. Más información: El especificador de formato de mes y año ("Y").	2009-06-15T13:45:30 -> June 2009 (en-US) 2009-06-15T13:45:30 -> juni 2009 (da-DK) 2009-06-15T13:45:30 -> Juni 2009 (id-ID)
Cualquier otro carácter único	Especificador desconocido.	Produce un error de expresión en tiempo de ejecución.

Cómo funcionan las cadenas con formato estándar

En una operación de formato, una cadena de formato estándar es simplemente un alias de una cadena con formato personalizado. La ventaja de usar un alias para referirse a una cadena de formato personalizado es que, aunque el alias permanece invariable, la propia cadena de formato personalizado puede variar. Esto es importante, porque las representaciones de cadena de valores de fecha y hora suelen variar con las referencias culturales. Por ejemplo, la cadena de formato estándar "d" indica que un valor de fecha y hora se va a mostrar utilizando un patrón de fecha corta. En la referencia cultural de todos los idiomas, este patrón es "MM/dd/yyyy". En la referencia cultural fr-FR, es "dd/MM/yyyy". En la referencia cultural ja-JP, es "yyyy/MM/dd."

Si una cadena con formato estándar en una operación de formato se asigna a una cadena con formato personalizado de una referencia cultural específica, la aplicación puede definir la referencia cultural concreta cuyas cadenas con formato personalizado se usan de uno de los modos siguientes:

- Puede utilizar la referencia cultural predeterminada (o la actual). En el ejemplo siguiente se muestra una fecha con el formato de fecha abreviado de la referencia cultural. En este caso, la referencia cultural actual es en-US.

```
powerquery

let
    Source =
    {
        Date.ToText(#date(2024, 3, 15), [Format = "d"])
        //Displays 3/15/2024
    }
in
    Source
```

- Puede pasar una referencia cultural usada para dar formato a la fecha según las reglas de esa referencia cultural específica. En el ejemplo siguiente se muestra una fecha con el formato de fecha abreviado de la referencia cultural pt-BR.

```
Power Query M

let
    Source =
    {
        Date.ToText(#date(2024, 3, 15), [Format = "d", Culture = "pt-
BR"])
        //Displays 15/03/2024
    }
```

```
}  
in  
Source
```

En algunos casos, la cadena con formato estándar actúa como la abreviatura correspondiente de una cadena con formato personalizado más larga que es invariable. Hay cuatro cadenas de formato estándar que pertenecen a esta categoría: "O" (u "o"), "R" (o "r"), "s" y "u". Estas cadenas se corresponden con las cadenas de formato personalizado definidas en la referencia cultural de todos los idiomas. Generan representaciones de cadena de valores de fecha y hora que están pensados para que sean idénticos en todas las referencias culturales. En la tabla siguiente se proporciona información sobre estas cuatro cadenas de formato de fecha y hora estándar.

[Expandir tabla](#)

Cadena con formato estándar	Definido por	Cadena con formato personalizado
"O" u "o"	None	yyyy'-'MM'-'dd'T'HH':'mm':'ss'.fffffffK
"R" o "r"	Especificación RFC 1123 de IETF	ddd, dd MMM yyyy HH':'mm':'ss 'GMT'
"s"	ISO 8601	yyyy'-'MM'-'dd'T'HH':'mm':'ss
"u"	Ordenable porque usa ceros iniciales para año, mes, día, hora, minuto y segundo	yyyy'-'MM'-'dd HH':'mm':'ss'Z'

Las cadenas de formato estándar también se pueden usar en operaciones de análisis, que requieren que una cadena de entrada se ajuste exactamente a un patrón determinado para que la operación de análisis se realice correctamente. Muchas cadenas de formato estándar se asignan a varias cadenas de formato personalizado, por lo que un valor de fecha y hora se puede representar en una variedad de formatos y la operación de análisis todavía se realiza correctamente.

En las secciones siguientes se describen los especificadores de formato estándar para [los valores Date](#), [DateTime](#), [DateTimeZone](#) y [Time](#).

Formatos de fecha

Este grupo incluye los formatos siguientes:

- [El especificador de formato de fecha corta \("d"\)](#)
- [El especificador de formato de fecha larga \("D"\)](#)

El especificador de formato de fecha corta ("d")

El especificador de formato estándar "d" representa un formato de fecha personalizado definido por una referencia cultural específica. Por ejemplo, el texto de formato personalizado devuelto por la referencia cultural invariable es "MM/dd/yyyy".

En el ejemplo siguiente se usa el especificador de formato "d" para mostrar un valor de fecha.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(2024, 4, 10), [Format = "d", Culture = ""]),
        // Displays 04/10/2024

        Date.ToText(#date(2024, 4, 10), [Format = "d", Culture = "en-US"]),
        // Displays 4/10/2024

        Date.ToText(#date(2024, 4, 10), [Format = "d", Culture = "en-NZ"]),
        // Displays 10/4/2024

        Date.ToText(#date(2024, 4, 10), [Format = "d", Culture = "de-DE"])
        // Displays 10.4.2024
    }
in
    Source
```

[Volver a la tabla](#)

El especificador de formato de fecha larga ("D")

El especificador de formato estándar "D" representa un formato de fecha personalizado definido por una referencia cultural específica. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "dddd, dd MMMM yyyy".

En el ejemplo siguiente se usa el especificador de formato "D" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(2024, 4, 10), [Format = "D", Culture = ""]),
        // Displays Wednesday, 10 April, 2024
    }
```

```

        Date.ToText(#date(2024, 4, 10), [Format = "D", Culture = "en-US"]),
        // Displays Wednesday, April 10, 2024

        Date.ToText(#date(2024, 4, 10), [Format = "D", Culture = "pt-BR"]),
        // Displays quarta-feira, 10 de abril de 2024

        Date.ToText(#date(2024, 4, 10), [Format = "D", Culture = "es-MX"])
        // Displays miércoles, 10 de abril de 2024
    }
in
    Source

```

[Volver a la tabla](#)

Formatos de fecha y hora

Este grupo incluye los formatos siguientes:

- El especificador de formato de fecha completa y hora corta ("f").
- El especificador de formato de fecha completa y hora larga ("F").
- El especificador de formato de fecha general y hora corta ("g").
- El especificador de formato de fecha general y hora larga ("G").
- El especificador de formato de operación de ida y vuelta ("O", "o").
- El especificador de formato RFC1123 ("R", "r").
- El especificador de formato que se puede ordenar ("s").
- El especificador de formato universal que se puede ordenar ("u").

El especificador de formato de fecha completa y hora corta ("f")

El especificador de formato estándar "f" representa una combinación de los patrones de fecha larga ("D") y hora corta ("t"), separados por un espacio.

La cadena de resultado se ve afectada por la información de formato de una referencia cultural específica.

En el ejemplo siguiente se usa el especificador de formato "f" para mostrar un valor de fecha y hora.

Power Query M

```

let
    Source =
    {

```

```

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "f",
Culture = "en-US"]),
        // Displays Wednesday, April 10, 2024 6:30 AM

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "f",
Culture = "fr-FR"])
        // Displays mercredi 10 avril 2024 06:30
    }
in
    Source

```

[Volver a la tabla](#)

El especificador de formato de fecha completa y hora larga ("F")

El especificador de formato estándar "F" representa un formato de fecha y hora personalizado definido por una referencia cultural específica. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "dddd, dd MMMM yyyy HH:mm:ss".

La cadena de resultado se ve afectada por la información de formato de una referencia cultural específica.

En el ejemplo siguiente se usa el especificador de formato "F" para mostrar un valor de fecha y hora.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "F",
Culture = ""]),
        // Displays Wednesday, 10 April, 2024 06:30:00

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "F",
Culture = "en-US"]),
        // Displays Wednesday, April 10, 2024 6:30:00 AM

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "F",
Culture = "fr-FR"])
        // Displays mercredi 10 avril 2024 06:30:00
    }
in
    Source

```

[Volver a la tabla](#)

El especificador de formato de fecha general y hora corta ("g")

El especificador de formato estándar "g" representa una combinación de los patrones de fecha corta ("d") y hora corta ("t"), separados por un espacio. El texto resultante se ve afectado por la información de formato de una referencia cultural específica.

La cadena de resultado se ve afectada por la información de formato de una referencia cultural específica.

En el ejemplo siguiente se usa el especificador de formato "g" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "g",
        Culture = ""]),
        // Displays 04/10/2024 06:30

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "g",
        Culture = "en-US"]),
        // Displays 4/10/2024 6:30 AM

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "g",
        Culture = "fr-BE"])
        // Displays 10-04-24 06:30
    }
in
    Source
```

[Volver a la tabla](#)

El especificador de formato de fecha general y hora larga ("G")

El especificador de formato estándar "G" representa una combinación de los patrones de fecha corta ("d") y hora larga ("T"), separados por un espacio. El texto resultante se ve afectado por la información de formato de una referencia cultural específica.

La cadena de resultado se ve afectada por la información de formato de una referencia cultural específica.

En el ejemplo siguiente se usa el especificador de formato "G" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "G",
        Culture = ""]),
        // Displays 04/10/2024 06:30:00

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "G",
        Culture = "en-US"]),
        // Displays 4/10/2024 6:30:00 AM

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "G",
        Culture = "nl-BE"])
        // Displays 10/04/2024 6:30:00
    }
in
    Source
```

[Volver a la tabla](#)

El especificador de formato de operación de ida y vuelta ("O", "o")

El especificador de formato estándar "O" u "o" representa una cadena de formato de fecha y hora personalizado mediante un patrón que conserva la información de la zona horaria y emite una cadena de resultado que cumple con la norma ISO 8601. En el caso de los valores **dateTimeZone**, este especificador de formato está diseñado para conservar los valores de fecha, hora y zona horaria en texto.

El especificador de formato estándar "O" o "o" corresponde al especificador de formato "aaaa"-'MM'-'dd'T'HH':mm':ss'.' cadena de formato personalizado ffffffffxxx para los valores **dateTimeZone**. En este texto, los pares de comillas simples que delimitan caracteres individuales, como los guiones, los dos puntos y la letra "T", indican que el carácter individual es un literal que no se puede cambiar. Los apóstrofos no aparecen en la cadena de salida.

Especificador de formato estándar "O" o "o" (y "aaaa"-'MM'-'dd'T'HH':mm':ss'.' ffffffffxxx" formato personalizado) aprovecha las formas en que ISO 8601 representa información de zona horaria para conservar los valores de **DateTimeZone**:

- El componente de zona horaria de [DateTimeZone.ToLocal](#) valores de fecha y hora es un desplazamiento de UTC (por ejemplo, +01:00, -07:00).
- El componente de zona horaria de [DateTimeZone.ToUtc](#) valores de fecha y hora usa +00.00 para representar UTC.

Dado que el especificador de formato estándar "O" u "o" se ajusta a un estándar internacional, la operación de formato o análisis que utiliza el especificador siempre usa la referencia cultural de todos los idiomas y el calendario gregoriano.

En el ejemplo siguiente se usa el especificador de formato "o" para mostrar una serie de **valores DateTimeZone** en un sistema de la zona horaria del Pacífico de EE. UU.

Power Query M

```
let
    date1 = #datetimezone(2024, 6, 15, 13, 45, 30, 0, 0),
    Source =
    {
        Text.Format("#{0} (Unspecified) --> #{1}", {date1,
        DateTimeZone.ToText(date1, [Format = "O"])}),
        Text.Format("#{0} (Utc) --> #{1}", {date1,
        DateTimeZone.ToText(DateTimeZone.ToUtc(date1), [Format = "O"])}),
        Text.Format("#{0} (Local) --> #{1}", {date1,
        DateTimeZone.ToText(DateTimeZone.ToLocal(date1), [Format = "O"])}))
    }
in
    Source

// The example displays the following output:
//      6/15/2024 1:45:30 PM +00:00 (Unspecified) --> 2024-06-
15T13:45:30.0000000+00:00
//      6/15/2024 1:45:30 PM +00:00 (Utc) --> 2024-06-
15T13:45:30.0000000+00:00
//      6/15/2024 1:45:30 PM +00:00 (Local) --> 2024-06-15T08:45:30.0000000-
07:00
```

ⓘ Nota

El valor devuelto por [DateTimeZone.ToLocal](#) depende de si ejecuta Power Query en una máquina local o en línea. Por ejemplo, en el ejemplo anterior en un sistema de la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve **-07:00** la **hora local** porque lee la hora establecida en el equipo local. Sin embargo, Power Query Online devuelve **+00:00** porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

En el ejemplo siguiente se usa el especificador de formato "o" para crear una cadena con formato y, a continuación, se restaura el valor de fecha y hora original llamando a una rutina de análisis de fecha y hora.

Power Query M

```
let
    // Round-trip a local time
    #"Origin Local Date" = DateTimeZone.ToLocal(
        #datetimezone(2024, 4, 10, 6, 30, 0, 0, 0)
    ),
    #"Local Date Text" = DateTimeZone.ToText(
        #"Origin Local Date", [Format = "o"]
    ),
    #"New Local Date" = DateTimeZone.FromText("#Local Date Text"),
    #"Local Round Trip" = Text.Format(
        "Round-tripped #{0} Local to #{1} Local.",
        {
            DateTimeZone.ToText("#Origin Local Date"),
            DateTimeZone.ToText("#New Local Date")
        }
    ),

    // Round-trip a UTC time
    #"Origin UTC Date" = DateTimeZone.ToUtc(
        #datetimezone(2024, 4, 12, 9, 30, 0, 0, 0)
    ),
    #"UTC Date Text" = DateTimeZone.ToText(
        #"Origin UTC Date", [Format = "o"]
    ),
    #"New UTC Date" = DateTimeZone.FromText("#UTC Date Text"),
    #"UTC Round Trip" = Text.Format(
        "Round-tripped #{0} UTC to #{1} UTC.",
        {
            DateTimeZone.ToText("#Origin UTC Date"),
            DateTimeZone.ToText("#New UTC Date")
        }
    ),

    // Round-trip an 18 hour offset time
    #"Origin Offset Date" = DateTimeZone.ToLocal(
        #datetimezone(2024, 4, 10, 6, 30, 0, 0, 0) + #duration(0, 18, 0, 0)
    ),
    #"Offset Date Text" = DateTimeZone.ToText(
        #"Origin Offset Date", [Format = "o"]
    ),
    #"New Offset Date" = DateTimeZone.FromText("#Offset Date Text"),
    #"Offset Round Trip" = Text.Format(
        "Round-tripped #{0} to #{1}.",
        {
            DateTimeZone.ToText("#Origin Offset Date"),
            DateTimeZone.ToText("#New Offset Date")
        }
    )
```

```

    ),

    #"Round Trip Results" =
        {#"Local Round Trip", #"UTC Round Trip", #"Offset Round Trip"}
in
    #"Round Trip Results"

// The example displays the following output in Power Query Desktop:
//    Round-tripped 4/9/2024 11:30:00 PM -07:00 Local to 4/9/2024 11:30:00
//    PM -07:00 Local.
//    Round-tripped 4/12/2024 9:30:00 AM +00:00 UTC to 4/12/2024 9:30:00 AM
//    +00:00 UTC.
//    Round-tripped 4/10/2024 5:30:00 PM -07:00 to 4/10/2024 5:30:00 PM
//    -07:00.

// The example displays the following output in Power Query Online:
//    Round-tripped 4/10/2024 6:30:00 AM +00:00 Local to 4/10/2024 6:30:00
//    AM +00:00 Local.
//    Round-tripped 4/12/2024 9:30:00 AM +00:00 UTC to 4/12/2024 9:30:00 AM
//    +00:00 UTC.
//    Round-tripped 4/11/2024 12:30:00 AM +00:00 to 4/11/2024 12:30:00 AM
//    +00:00.

```

[Volver a la tabla](#)

El especificador de formato RFC1123 ("R", "r")

El especificador de formato estándar "R" o "r" representa una cadena de formato de fecha y hora personalizada que no está definida por una referencia cultural específica. Siempre es lo mismo, independientemente de la referencia cultural utilizada o del proveedor de formato proporcionado. La cadena de formato personalizado es "ddd, dd MMM yyyy HH':'mm':'ss 'GMT'". Cuando se utiliza este especificador de formato estándar, la operación de formato o análisis utiliza siempre la referencia cultural de todos los idiomas.

Aunque el estándar RFC 1123 expresa una hora como hora universal coordinada (UTC), la operación de formato no modifica el valor de la fecha y hora con formato. Por lo tanto, debe convertir el valor DateTime a UTC llamando al método de función [DateTimeZone.ToUtc](#) antes de realizar la operación de formato.

En el ejemplo siguiente se usa el especificador de formato "r" para mostrar un valor de hora y fecha en un sistema en la zona horaria del Pacífico de EE. UU. (siete horas detrás de utc).

Power Query M

```

let
    date1 = #datetimezone(2024, 4, 10, 6, 30, 0, -7, 0),

```

```

dateOffset = DateTimeZone.ToUtc(date1),
Source =
{
    DateTimeZone.ToText(date1, [Format = "r"]),
    // Displays Wed, 10 Apr 2024 13:30:00 GMT

    DateTimeZone.ToText(dateOffset, [Format = "r"])
    // Displays Wed, 10 Apr 2024 13:30:00 GMT
}
in
    Source

```

[Volver a la tabla](#)

El especificador de formato que se puede ordenar ("s")

El especificador de formato estándar "s" representa una cadena de formato de fecha y hora personalizada que refleja un estándar definido (ISO 8601) y es de solo lectura. Por consiguiente, siempre es el mismo, sea cual fuere la referencia cultural utilizada o el proveedor de formato proporcionado. La cadena de formato personalizado es "yyyy'-MM'-'dd'T'HH':mm':ss". La finalidad del especificador de formato "s" es generar cadenas de resultado de forma coherente en orden ascendente o descendente según los valores de fecha y hora.

Cuando se utiliza este especificador de formato estándar, la operación de formato o análisis utiliza siempre la referencia cultural de todos los idiomas.

En el ejemplo siguiente se usa el especificador de formato "s" para mostrar un valor de fecha y hora en un sistema en la zona horaria del Pacífico de EE. UU.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "s",
        Culture = "en-US"])
        // Displays 2024-04-10T06:30:00
    }
in
    Source

```

[Volver a la tabla](#)

El especificador de formato universal que se puede ordenar ("u")

El especificador de formato estándar "u" representa una cadena de formato de fecha y hora personalizada que siempre es la misma, independientemente de la referencia cultural usada o del proveedor de formato proporcionado. La cadena de formato personalizado es "yyyy'-'MM'-'dd HH':'mm':'ss'Z'". El patrón refleja una norma definida y la propiedad es de solo lectura. Cuando se utiliza este especificador de formato estándar, la operación de formato o análisis utiliza siempre la referencia cultural de todos los idiomas.

Aunque la cadena de resultado debe expresar una hora como hora universal coordinada (UTC), no se realiza ninguna conversión del valor `dateTimeZone` **original** durante la operación de formato. Por lo tanto, debe convertir un valor de **DateTimeZone** a UTC llamando a la función [DateTimeZone.ToUtc](#) antes de aplicarle formato.

En el ejemplo siguiente se usa el especificador de formato "u" para mostrar un valor de fecha y hora.

Power Query M

```
let
    date1 = #datetimezone(2024, 4, 10, 6, 30, 0, -7, 0),
    dateOffset = DateTimeZone.ToUtc(date1),
    Source =
    {
        DateTimeZone.ToText(dateOffset, [Format = "u"]),
        // Displays 2024-04-10 13:30:00Z
    }
in
    Source
```

[Volver a la tabla](#)

Formatos de hora

Este grupo incluye los formatos siguientes:

- [El especificador de formato de hora corta \("t"\).](#)
- [El especificador de formato de hora larga \("T"\).](#)

El especificador de formato de hora corta ("t")

El especificador de formato estándar "t" representa una cadena de formato de fecha y hora personalizada definida por la referencia cultural especificada. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "HH:mm".

La cadena de resultado se ve afectada por la información de formato de una referencia cultural específica.

En el ejemplo siguiente se usa el especificador de formato "t" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "t",
Culture = ""]),
        // Displays 06:30

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "t",
Culture = "en-US"]),
        // Displays 6:30 AM

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "t",
Culture = "es-ES"])
        // Displays 6:30
    }
in
    Source
```

[Volver a la tabla](#)

El especificador de formato de hora larga ("T")

El especificador de formato estándar "T" representa una cadena de formato de fecha y hora personalizada definida por la referencia cultural específica. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "HH:mm:ss".

En el ejemplo siguiente se usa el especificador de formato "T" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "T",
Culture = ""]),
        // Displays 06:30:00

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "T",
```

```

Culture = "en-US"]],
    // Displays 6:30:00 AM

    DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "T",
Culture = "es-ES"])
    // Displays 6:30:00
}
in
Source

```

[Volver a la tabla](#)

Formatos de fecha parcial

Este grupo incluye los formatos siguientes:

- [El especificador de formato de mes \("M", "m"\).](#)
- [El especificador de formato de mes y año \("Y", "y"\)](#)

El especificador de formato de mes ("M", "m")

El especificador de formato estándar "M" o "m" representa una cadena de formato de fecha y hora personalizada definida por la referencia cultural específica. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "MMMM dd".

En el ejemplo siguiente se usa el especificador de formato "m" para mostrar un valor de fecha y hora.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "m",
Culture = ""]),
        // Displays April 10

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "m",
Culture = "en-US"]),
        // Displays April 10

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "m",
Culture = "ms-MY"])
        // Displays 10 April
    }

```

in
Source

[Volver a la tabla](#)

El especificador de formato de mes y año ("Y", "y")

El especificador de formato estándar "Y" o "y" representa una cadena de formato de fecha personalizada definida por una referencia cultural específica. Por ejemplo, la cadena de formato personalizado para la referencia cultural de todos los idiomas es "yyyy MMMM".

En el ejemplo siguiente se usa el especificador de formato "y" para mostrar un valor de fecha y hora.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "Y",
Culture = ""]),
        // Displays 2024 April

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "Y",
Culture = "en-US"]),
        // Displays April 2024

        DateTime.ToText(#datetime(2024, 4, 10, 6, 30, 0), [Format = "y",
Culture = "af-ZA"])
        // Displays April 2024
    }
in
    Source
```

[Volver a la tabla](#)

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Conversión de tipos Date, Time, DateTime y DateTimeZone](#)
- [Funciones de fecha](#)
- [Funciones DateTime](#)
- [Funciones DateTimeZone](#)
- [Funciones de hora](#)

- Cadenas con formato de fecha y hora personalizado

Comentarios

¿Le ha resultado útil esta página?



Sí



No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Cadenas con formato de fecha y hora personalizado

Artículo • 20/11/2024

Una cadena de formato de fecha y hora define la representación de texto de un [valor Date](#), [DateTime](#), [DateTimeZone](#) o [Time](#) resultante de una operación de formato.

También puede definir la representación de un valor de fecha y hora que se necesite en una operación de análisis para convertir correctamente la cadena en una fecha y hora.

Una cadena de formato personalizado consta de uno o varios especificadores de formato de fecha y hora personalizado. Cualquier cadena que no sea una [cadena](#) de formato de fecha y hora estándar se interpreta como una cadena de formato de fecha y hora personalizada.

En las operaciones de formato, se pueden usar cadenas de formato de fecha y hora personalizadas con el `ToText` método de una instancia de fecha y hora y zona horaria.

En el ejemplo siguiente se ilustran sus usos.

Power Query M

```
let
    Source =
    {
        Text.From("Today is " & Date.ToText(#date(2011, 6, 10), [Format =
"MMMM dd yyyy"] & "."),
        Text.Format("The current date and time: #{0}", {DateTimeZone.ToText(
#datetimezone(2011, 6, 10, 15, 24, 16, 0, 0), [Format =
"MM/dd/yy H:mm:ss zzz"])}
    )
    }
in
    Source

// The example displays the following output:
//     Today is June 10, 2011.
//     The current date and time: 06/10/11 15:24:16 +00:00
```

En las operaciones de análisis, se pueden usar cadenas de formato de fecha y hora personalizadas con las **funciones Date**, **DateTime**, **Time** y **DateTimeZone** . Estas funciones requieren que una cadena de entrada se ajuste exactamente a un patrón determinado para que la operación de análisis se realice correctamente. En el ejemplo siguiente se muestra una llamada a la función [DateTime.FromText](#) para analizar una fecha que debe incluir un mes, un día y un año de dos dígitos.

Power Query M

```

let
    dateValues = { "30-12-2011", "12-30-2011", "30-12-11", "12-30-11"},
    pattern = "MM-dd-yy",
    convertedDates = List.Transform(dateValues, (dateValue) =>
        try Text.Format("Converted '#{0}' to #{1}.", {dateValue,
DateTime.FromText(dateValue, [Format=pattern])})
        otherwise Text.Format("Unable to convert '#{0}' to a date and
time.", {dateValue}))
    in
        convertedDates

// The example displays the following output:
//     Unable to convert '30-12-2011' to a date and time.
//     Unable to convert '12-30-2011' to a date and time.
//     Unable to convert '30-12-11' to a date and time.
//     Converted '12-30-11' to 12/30/2011.

```

En la tabla siguiente se describen los especificadores de formato de fecha y hora personalizados, y se muestra la cadena de resultado producida por cada especificador de formato. De forma predeterminada, las cadenas de resultado reflejan las convenciones de formato de la referencia cultural en-us. Si un especificador de formato determinado genera una cadena de resultado localizada, el ejemplo también indica la referencia cultural a la que se aplica dicha cadena. Para obtener más información sobre el uso de cadenas de formato de fecha y hora personalizadas, vaya a la [sección Notas](#).

 Expandir tabla

Especificador de formato	Descripción	Ejemplos
"d"	El día del mes, de 1 a 31.	2009-06-01T13:45:30 -> 1
	Más información: Especificador de formato personalizado "d" .	2009-06-15T13:45:30 -> 15
"dd"	El día del mes, de 01 a 31.	2009-06-01T13:45:30 -> 01
	Más información: Especificador de formato personalizado "dd" .	2009-06-15T13:45:30 -> 15
"ddd"	El nombre abreviado del día de la semana.	2009-06-15T13:45:30 -> Lun (en-US)
		2009-06-15T13:45:30 -> Пн (ru-RU)
		2009-06-15T13:45:30 -> lun. (fr-FR)
"dddd"	El nombre completo del día de la semana.	2009-06-15T13:45:30 -> Lun (en-US)

Especificador de formato	Descripción	Ejemplos
	Más información: Especificador de formato personalizado "dddd" .	2009-06-15T13:45:30 -> понедельник (ru-RU) 2009-06-15T13:45:30 -> lundi (fr-FR)
"f"	Las décimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "f" .	2009-06-15T13:45:30.6170000 -> 6 2009-06-15T13:45:30.05 -> 0
"ff"	Las centésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "ff" .	2009-06-15T13:45:30.6170000 -> 61 2009-06-15T13:45:30.0050000 -> 00
"fff"	Los milisegundos de un valor de fecha y hora. Más información: Especificador de formato personalizado "fff" .	6/15/2009 13:45:30.617 -> 617 6/15/2009 13:45:30.0005 -> 000
"ffff"	Las diezmilésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "ffff" .	2009-06-15T13:45:30.6175000 -> 6175 2009-06-15T13:45:30.0000500 -> 0000
"fffff"	Las cienmilésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "fffff" .	2009-06-15T13:45:30.6175400 -> 61754 6/15/2009 13:45:30.0000005 -> 00000
"ffffff"	Las millonésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "ffffff" .	2009-06-15T13:45:30.6175420 -> 617542 2009-06-15T13:45:30.00000005 -> 000000
"fffffff"	Las diezmillonésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "fffffff" .	2009-06-15T13:45:30.6175425 -> 6175425 2009-06-15T13:45:30.0001150 -> 0001150

Especificador de formato	Descripción	Ejemplos
"F"	Si es distinto de cero, las décimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "F" .	2009-06-15T13:45:30.6170000 -> 6 2009-06-15T13:45:30.0500000 -> (sin resultado)
"FF"	Si es distinto de cero, las centésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "FF" .	2009-06-15T13:45:30.6170000 -> 61 2009-06-15T13:45:30.0050000 -> (sin resultado)
"FFF"	Si es distinto de cero, los milisegundos de un valor de fecha y hora. Más información: Especificador de formato personalizado "FFF" .	2009-06-15T13:45:30.6170000 -> 617 2009-06-15T13:45:30.0005000 -> (sin resultado)
"FFFF"	Si es distinto de cero, las diezmilésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "FFFF" .	2009-06-15T13:45:30.5275000 -> 5275 2009-06-15T13:45:30.0000500 -> (sin resultado)
"FFFFF"	Si es distinto de cero, las cienmilésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "FFFFF" .	2009-06-15T13:45:30.6175400 -> 61754 2009-06-15T13:45:30.0000050 -> (sin resultado)
"FFFFFF"	Si es distinto de cero, las millonésimas de segundo de un valor de fecha y hora. Más información: Especificador de formato personalizado "FFFFFF" .	2009-06-15T13:45:30.6175420 -> 617542 2009-06-15T13:45:30.0000005 -> (sin resultado)
"FFFFFFF"	Si es distinto de cero, las diezmillonésimas de segundo de un valor de fecha y hora.	2009-06-15T13:45:30.6175425 -> 6175425 2009-06-15T13:45:30.0001150 -> 000115

Especificador de formato	Descripción	Ejemplos
	Más información: Especificador de formato personalizado "FFFFFFF" .	
"g", "gg"	El período o la era. Más información: Especificador de formato personalizado "g" o "gg" .	2009-06-15T13:45:30.6170000 -> A.D.
"h"	La hora, usando un reloj de 12 horas de 1 a 12. Más información: Especificador de formato personalizado "h" .	2009-06-15T01:45:30 -> 1 2009-06-15T13:45:30 -> 1
"hh"	La hora, usando un reloj de 12 horas de 01 a 12. Más información: Especificador de formato personalizado "hh" .	2009-06-15T01:45:30 -> 01 2009-06-15T13:45:30 -> 01
"H"	La hora, usando un reloj de 24 horas de 0 a 23. Más información: Especificador de formato personalizado "H" .	2009-06-15T01:45:30 -> 1 2009-06-15T13:45:30 -> 13
"HH"	La hora, usando un reloj de 24 horas de 00 a 23. Más información: Especificador de formato personalizado "HH" .	2009-06-15T01:45:30 -> 01 2009-06-15T13:45:30 -> 13
"K"	Información de la zona horaria. Más información: Especificador de formato personalizado "K" .	2009-06-15T13:45:30, Sin especificar :> 2009-06-15T13:45:30, Utc -> +00:00 2009-06-15T13:45:30, Local -> -07:00 (depende de la configuración del equipo local o en la nube)
"m"	Minutos, de 0 a 59. Más información: Especificador de formato personalizado "m" .	2009-06-15T01:09:30 -> 9 2009-06-15T13:29:30 -> 29
"mm"	Minutos, de 00 a 59.	2009-06-15T01:09:30 -> 09 2009-06-15T01:45:30 -> 45

Especificador de formato	Descripción	Ejemplos
	Más información: Especificador de formato personalizado "mm" .	
"M"	Meses, de 1 a 12. Más información: Especificador de formato personalizado "M" .	2009-06-15T13:45:30 -> 6
"MM"	Meses, de 01 a 12. Más información: Especificador de formato personalizado "MM" .	2009-06-15T13:45:30 -> 06
"MMM"	El nombre abreviado del mes. Más información: Especificador de formato personalizado "MMM" .	2009-06-15T13:45:30 -> Jun (en-US) 2009-06-15T13:45:30 -> juin (fr-FR) 2009-06-15T13:45:30 -> Jun (zu-ZA)
"MMMM"	El nombre completo del mes. Más información: Especificador de formato personalizado "MMMM" .	2009-06-15T13:45:30 -> June (en-US) 2009-06-15T13:45:30 -> juni (da-DK) 2009-06-15T13:45:30 -> Juni (zu-ZA)
"s"	Segundos, de 0 a 59. Más información: Especificador de formato personalizado "s" .	2009-06-15T13:45:09 -> 9
"ss"	Segundos, de 00 a 59. Más información: Especificador de formato personalizado "ss" .	2009-06-15T13:45:09 -> 09
"t"	El primer carácter del designador AM/PM. Más información: Especificador de formato personalizado "t" .	2009-06-15T13:45:30 -> P (en-US) 2009-06-15T13:45:30 -> 午 (ja-JP) 2009-06-15T13:45:30 -> (fr-FR)
"tt"	El designador AM/PM. Más información: Especificador de formato personalizado "tt" .	2009-06-15T13:45:30 -> PM (en-US) 2009-06-15T13:45:30 -> 午後 (ja-JP) 2009-06-15T13:45:30 -> (fr-FR)

Especificador de formato	Descripción	Ejemplos
"y"	El año, de 0 a 99. Más información: Especificador de formato personalizado "y" .	0001-01-01T00:00:00 -> 1 0900-01-01T00:00:00 -> 0 1900-01-01T00:00:00 -> 0 2009-06-15T13:45:30 -> 9 2019-06-15T13:45:30 -> 19
"yy"	El año, de 00 a 99. Más información: Especificador de formato personalizado "aaaa" .	0001-01-01T00:00:00 -> 01 0900-01-01T00:00:00 -> 00 1900-01-01T00:00:00 -> 00 2019-06-15T13:45:30 -> 19
"yyy"	El año, con un mínimo de tres dígitos. Más información: Especificador de formato personalizado "aaaa" .	0001-01-01T00:00:00 -> 001 0900-01-01T00:00:00 -> 900 1900-01-01T00:00:00 -> 1900 2009-06-15T13:45:30 -> 2009
"yyyy"	El año como un número de cuatro dígitos. Más información: Especificador de formato personalizado "aaaa" .	0001-01-01T00:00:00 -> 0001 0900-01-01T00:00:00 -> 0900 1900-01-01T00:00:00 -> 1900 2009-06-15T13:45:30 -> 2009
"yyyyy"	El año como un número de cinco dígitos. Más información: Especificador de formato personalizado "aaaa" .	0001-01-01T00:00:00 -> 00001 2009-06-15T13:45:30 -> 02009
"z"	Desfase de horas con respecto a la hora UTC, sin ceros iniciales. Más información: Especificador de formato personalizado "z" .	2009-06-15T13:45:30-07:00 -> -7

Especificador de formato	Descripción	Ejemplos
"zz"	Desfase de horas con respecto a la hora UTC, con un cero inicial para un valor de un solo dígito. Más información: Especificador de formato personalizado "zz" .	2009-06-15T13:45:30-07:00 -> -07
"zzz"	Desfase de horas y minutos con respecto a la hora UTC. Más información: Especificador de formato personalizado "zzz" .	2009-06-15T13:45:30-07:00 -> -07:00
":"	El separador de hora. Más información: Especificador de formato personalizado ":" .	2009-06-15T13:45:30 -> : (en-US) 2009-06-15T13:45:30 -> . (it-IT) 2009-06-15T13:45:30 -> : (ja-JP)
"/"	El separador de fecha. Más información: Especificador de formato personalizado "/" .	2009-06-15T13:45:30 -> / (en-US) 2009-06-15T13:45:30 -> - (ar-DZ) 2009-06-15T13:45:30 -> . (tr-TR)
"cadena" 'cadena'	Delimitador de cadena literal. Más información: Literales de carácter .	2009-06-15T13:45:30 ("arr:" h:m t) -> arr: 1:45 P 2009-06-15T13:45:30 ('arr:' h:m t) -> arr: 1:45 P
%	Define el siguiente carácter como un especificador de formato personalizado. Más información: Uso de especificadores de formato personalizado único .	2009-06-15T13:45:30 (%h) -> 1
\, "", '	Secuencias de escape. Más información: Literales de caracteres y Uso de las secuencias de escape .	2009-06-15T13:45:30 (h \h) -> 1 h 2009-06-15T13:45:30 (h "h") -> 1 h 2009-06-15T13:45:30 (h 'h') -> 1 h
Cualquier otro carácter	El carácter se copia en la cadena de resultado sin modificar.	2009-06-15T01:45:30 (arr hh:mm t) -> arr 01:45 A

Especificador de formato	Descripción	Ejemplos
	Más información: Literales de carácter .	

En las secciones siguientes se proporciona información adicional sobre cada especificador de formato de fecha y hora personalizado. A menos que se indique lo contrario, cada especificador genera una representación de cadena idéntica independientemente de si se usa con un valor `Date`, `DateTime`, `DateTimeZone` o `Time` .

Especificador de formato de día "d"

Especificador de formato personalizado "d"

El especificador de formato personalizado "d" representa el día del mes como un número de 1 a 31. Un día con un solo dígito tiene un formato sin un cero inicial.

Si el especificador de formato "d" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "d". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "d" en varias cadenas de formato.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "d,
M", Culture = ""]),
        // Displays 29, 8

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "d,
MMMM", Culture = "en-US"]),
        // Displays 29, August

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "d,
MMMM", Culture = "es-MX"])
        // Displays 29, agosto
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "dd"

La cadena de formato personalizado "dd" representa el día del mes como un número de 01 a 31. Un día con un solo dígito tiene un formato con un cero inicial.

En el ejemplo siguiente se incluye el especificador de formato personalizado "dd" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 2, 6, 30, 15), [Format = "dd,
MM", Culture = ""])
        // Displays 02, 01
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "ddd"

El especificador de formato personalizado "ddd" representa el nombre abreviado del día de la semana. El nombre abreviado localizado del día de la semana se recupera de la referencia cultural actual o especificada.

En el ejemplo siguiente se incluye el especificador de formato personalizado "ddd" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "ddd d
MMM", Culture = "en-US"]),
        // Displays Thu 29 Aug

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "ddd d
MMM", Culture = "fr-FR"])
        // Displays jeu. 29 août
    }
in
    Source
```

in
Source

[Volver a la tabla](#)

Especificador de formato personalizado "dddd"

El especificador de formato personalizado "dddd" (más cualquier número de especificadores "d" adicionales) representa el nombre completo del día de la semana. El nombre localizado del día de la semana se recupera de la referencia cultural actual o especificada.

En el ejemplo siguiente se incluye el especificador de formato personalizado "dddd" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "dddd
dd MMMM", Culture = "en-US"]),
        // Displays Thursday 29 August

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "dddd
dd MMMM", Culture = "it-IT"])
        // Displays giovedì 29 agosto
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de fracción de segundos en minúsculas "f"

Especificador de formato personalizado "f"

El especificador de formato personalizado "f" representa el dígito más significativo de la fracción de segundos; es decir, representa las décimas de segundo de un valor de fecha y hora.

Si el especificador de formato "f" se usa sin otros especificadores de formato, se interpreta como el especificador de formato de fecha y hora estándar "f". Para obtener

más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

Cuando se usan especificadores de formato "f" como parte de una cadena de formato proporcionada para analizar el número de fracciones de segundos, el número de especificadores de formato "f" indica el número de dígitos más significativos de la fracción de segundos que debe estar presente para analizar correctamente la cadena.

En el ejemplo siguiente se incluye el especificador de formato personalizado "f" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
"hh:mm:ss:f", Culture = ""]),
        // Displays 07:27:15:0

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
"hh:mm:ss:F", Culture = ""])
        // Displays 07:27:15:
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "ff"

El especificador de formato personalizado "ff" representa los dos dígitos más significativos de la fracción de segundos; es decir, representa las centésimas de segundo de un valor de fecha y hora.

En el ejemplo siguiente se incluye el especificador de formato personalizado "ff" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
"hh:mm:ss:ff", Culture = ""]),
        // Displays 07:27:15:01

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
```

```
"hh:mm:ss:FF", Culture = "")])  
    // Displays 07:27:15:01  
}  
in  
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "fff"

El especificador de formato personalizado "fff" representa los tres dígitos más significativos de la fracción de segundos; es decir, representa los milisegundos de un valor de fecha y hora.

En el ejemplo siguiente se incluye el especificador de formato personalizado "fff" en una cadena de formato personalizado.

Power Query M

```
let  
    Source =  
    {  
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =  
            "hh:mm:ss:fff", Culture = ""]),  
        // Displays 07:27:15:018  
  
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =  
            "hh:mm:ss:FFF", Culture = ""]),  
        // Displays 07:27:15:018  
    }  
in  
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "ffff"

El especificador de formato personalizado "ffff" representa los cuatro dígitos más significativos de la fracción de segundos; es decir, representa las diezmilésimas de segundo de un valor de fecha y hora.

Aunque es posible mostrar las diez milésimas de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "fffff"

El especificador de formato personalizado "fffff" representa los cinco dígitos más significativos de la fracción de segundo; es decir, representa las cienmilésimas de segundo de un valor de fecha y hora.

Aunque es posible mostrar las centésimas milésimas de segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "ffffff"

El especificador de formato personalizado "ffffff" representa los seis dígitos más significativos de la fracción de segundos; es decir, representa las millonésimas de segundo de un valor de fecha y hora.

Aunque es posible mostrar la millonésima parte de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "fffffff"

El especificador de formato personalizado "fffffff" representa los siete dígitos más significativos de la fracción de segundos; es decir, representa las diezmillonésimas de segundo de un valor de fecha y hora.

Aunque es posible mostrar las diez millonésimas de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de fracción de segundos en mayúsculas "F"

Especificador de formato personalizado "F"

El especificador de formato personalizado "F" representa el dígito más significativo de la fracción de segundos; es decir, representa las décimas de segundo de un valor de fecha y hora. No se muestra nada si el dígito es cero y tampoco se muestra el separador decimal que sigue al número de segundos.

Si el especificador de formato "F" se usa sin otros especificadores de formato, se interpreta como el especificador de formato de fecha y hora estándar "F". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

El número de especificadores de formato "F" usados al analizar indica el número máximo de dígitos más significativos de la fracción de segundos que puede estar presente para analizar correctamente la cadena.

En el ejemplo siguiente se incluye el especificador de formato personalizado "F" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
"hh:mm:ss:f", Culture = ""]),
        // Displays 07:27:15:0

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
"hh:mm:ss:F", Culture = ""])
        // Displays 07:27:15:
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "FF"

El especificador de formato personalizado "FF" representa los dos dígitos más significativos de la fracción de segundos; es decir, representa las centésimas de segundo

de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los dos dígitos significativos son cero y, en ese caso, no se muestra el separador decimal que sigue al número de segundos.

En el ejemplo siguiente se incluye el especificador de formato personalizado "FF" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
        "hh:mm:ss:ff", Culture = ""]),
        // Displays 07:27:15:01

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
        "hh:mm:ss:FF", Culture = ""])
        // Displays 07:27:15:01
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "FFF"

El especificador de formato personalizado "FFF" representa los tres dígitos más significativos de la fracción de segundos; es decir, representa los milisegundos de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los tres dígitos significativos son cero y, en ese caso, el separador decimal que sigue al número de segundos tampoco se muestra.

En el ejemplo siguiente se incluye el especificador de formato personalizado "FFF" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
        "hh:mm:ss:fff", Culture = ""]),
        // Displays 07:27:15:018

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15.018), [Format =
        "hh:mm:ss:FFF", Culture = ""])
        // Displays 07:27:15:018
    }
in
    Source
```

```
}  
in  
Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "FFFF"

El especificador de formato personalizado "FFFF" representa los cuatro dígitos más significativos de la fracción de segundos; es decir, representa las diezmilésimas de segundo de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los cuatro dígitos significativos son cero y, en ese caso, no se muestra el separador decimal que sigue al número de segundos.

Aunque es posible mostrar las diez milésimas de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "FFFFFF"

El especificador de formato personalizado "FFFFFF" representa los cinco dígitos más significativos de la fracción de segundos; es decir, representa las cienmilésimas de segundo de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los cinco dígitos significativos son cero y, en ese caso, no se muestra el separador decimal que sigue al número de segundos.

Aunque es posible mostrar las centésimas milésimas de segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "FFFFFFF"

El especificador de formato personalizado "FFFFFFF" representa los seis dígitos más significativos de la fracción de segundos; es decir, representa las millonésimas de

segundo de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los seis dígitos significativos son cero y, en ese caso, no se muestra el separador decimal que sigue al número de segundos.

Aunque es posible mostrar la millonésima parte de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato personalizado "FFFFFFF"

El especificador de formato personalizado "FFFFFFF" representa los siete dígitos más significativos de la fracción de segundos; es decir, representa las diezmillonésimas de segundo de un valor de fecha y hora. No se muestran ceros finales. No se muestra nada si los siete dígitos significativos son cero y, en ese caso, el separador decimal que sigue al número de segundos tampoco se muestra.

Aunque es posible mostrar las diez millonésimas de un segundo componente de un valor de hora, es posible que ese valor no sea significativo. La precisión de los valores de fecha y hora depende de la resolución del reloj del sistema. En los sistemas operativos Windows Server 2019 y Windows 11, la resolución del reloj es de aproximadamente 1 milisegundos.

[Volver a la tabla](#)

Especificador de formato de era "g"

Especificador de formato personalizado "g" o "gg"

Los especificadores de formato personalizado «g» o «gg» (más cualquier número de especificadores «g» adicionales) representan el período o la era, como A.D. La operación de formato omite este especificador si la fecha a la que se va a dar formato no tiene una cadena de período o era asociada.

Si el especificador de formato "g" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "g". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "g" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(70, 08, 04), [Format = "MM/dd/yyyy g", Culture =
""]),
        // Displays 08/04/0070 A.D.

        Date.ToText(#date(70, 08, 04), [Format = "MM/dd/yyyy g", Culture =
"fr-FR"])
        // Displays 08/04/0070 ap. J.-C.
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de hora en minúsculas "h"

Especificador de formato personalizado "h"

El especificador de formato personalizado "h" representa la hora como un número del 1 al 12; es decir, la hora se representa como en un reloj de 12 horas que cuenta las horas enteras desde medianoche o mediodía. Una hora determinada después de la medianoche no se distingue de la misma hora después del mediodía. La hora no se redondea y se da formato a una hora de un solo dígito sin un cero inicial. Por ejemplo, dada una hora de 5:43 de la mañana o de la tarde, este especificador de formato personalizado muestra "5".

Si el especificador de formato "h" se usa sin otros especificadores de formato personalizado, se interpreta como un especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "h" en una cadena de formato personalizado.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = ""]),
        // Displays 6:9:1 P

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = "el-GR"]),
        // Displays 6:9:1 μ

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = ""]),
        // Displays 9:18:1.5 A

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = "el-GR"]),
        // Displays 9:18:1.5 π
    }
in
    Source

```

[Volver a la tabla](#)

Especificador de formato personalizado "hh"

El especificador de formato personalizado "hh" (más cualquier número de especificadores "h" adicionales) representa la hora como un número del 01 al 12; es decir, la hora se representa como en un reloj de 12 horas que cuenta las horas enteras desde medianoche o mediodía. Una hora determinada después de la medianoche no se distingue de la misma hora después del mediodía. La hora no se redondea y se da formato a una hora de un solo dígito con un cero inicial. Por ejemplo, dada una hora de 5:43 de la mañana o de la tarde, este especificador de formato muestra "05".

En el ejemplo siguiente se incluye el especificador de formato personalizado "hh" en una cadena de formato personalizado.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss tt", Culture = ""]),
        // Displays 06:09:01 PM

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss tt", Culture = "hu-HU"]),
    }

```

```
// Displays 06:09:01 du.

DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = ""]),
// Displays 09:18:01.50 AM

DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = "hu-HU"])
// Displays 09:18:01.50 de.
}
in
Source
```

[Volver a la tabla](#)

Especificador de formato de hora en mayúsculas "H"

Especificador de formato personalizado "H"

El especificador de formato personalizado "H" representa la hora como un número del 0 al 23; es decir, la hora se representa como en un reloj de 24 horas de base cero que cuenta las horas desde medianoche. Una hora con un solo dígito tiene un formato sin un cero inicial.

Si el especificador de formato "H" se usa sin otros especificadores de formato personalizado, se interpreta como especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "H" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 6, 9, 1), [Format = "H:mm:ss",
Culture = ""])
        // Displays 6:09:01
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "HH"

El especificador de formato personalizado "HH" (más cualquier número de especificadores "H" adicionales) representa la hora como un número del 00 al 23; es decir, la hora se representa como en un reloj de 24 horas de base cero que cuenta las horas desde medianoche. Una hora con un solo dígito tiene un formato con un cero inicial.

En el ejemplo siguiente se incluye el especificador de formato personalizado "HH" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 6, 9, 1), [Format =
"HH:mm:ss", Culture = ""])
        // Displays 06:09:01
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de zona horaria "K"

Especificador de formato personalizado "K"

El especificador de formato personalizado "K" representa la información de zona horaria de un valor de fecha y hora. Cuando este especificador de formato se usa con **valores DateTimeZone**, la cadena de resultado se define como:

- Para la zona horaria local, este especificador genera una cadena de resultado que contiene el desplazamiento local de la hora universal coordinada (UTC), por ejemplo, "-07:00", si la consulta se ejecuta en Power Query Desktop. Si la consulta se ejecuta en Power Query Online, la cadena de resultado no genera ningún desplazamiento desde la hora UTC, es decir, "+00:00".
- Para una hora UTC, la cadena de resultado no genera ningún desplazamiento a partir de la hora UTC; es decir, "+00:00 para representar una fecha UTC.

- Para una hora de una zona horaria no especificada, el resultado está vacío.

Si el especificador de formato "K" se usa sin otros especificadores de formato personalizado, se interpreta como especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se muestra la cadena resultante del uso del especificador de formato personalizado "K" con varios valores en un sistema de la zona horaria del Pacífico de EE. UU.

Power Query M

```
let
    Source =
    {
        DateTimeZone.ToText(DateTimeZone.LocalNow(), [Format="%K"]),
        // Displays -07:00 (Desktop) or +00:00 (Online)

        DateTimeZone.ToText(DateTimeZone.UtcNow(), [Format="%K"]),
        // Displays +00:00

        Text.Format("'#{0}'", {DateTime.ToText(DateTime.LocalNow(),
[Format="%K"])}),
        // Displays ''
    }
in
    Source
```

⚠ Nota

El valor devuelto por [DateTimeZone.LocalNow](#) depende de si ejecuta Power Query en una máquina local o en línea. Por ejemplo, en el ejemplo anterior en un sistema de la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve `-07:00` porque lee la hora establecida en el equipo local. Sin embargo, Power Query Online devuelve `+00:00` porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

[Volver a la tabla](#)

Especificador de formato de minuto "m"

Especificador de formato personalizado "m"

El especificador de formato personalizado "m" representa el minuto como un número de 0 a 59. El minuto representa los minutos enteros que han transcurrido desde la última hora. Un minuto con un solo dígito tiene un formato sin un cero inicial.

Si el especificador de formato "m" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "m". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "m" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = ""]),
        // Displays 6:9:1 P

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = "el-GR"]),
        // Displays 6:9:1 μ

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = ""]),
        // Displays 9:18:1.5 A

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = "el-GR"])
        // Displays 9:18:1.5 π
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "mm"

El especificador de formato personalizado "mm" (más cualquier número de especificadores "m" adicionales) representa el minuto como un número de 00 a 59. El minuto representa los minutos enteros que han transcurrido desde la última hora. Un minuto con un solo dígito tiene un formato con un cero inicial.

En el ejemplo siguiente se incluye el especificador de formato personalizado "mm" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss
tt", Culture = ""]),
        // Displays 06:09:01 PM

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss
tt", Culture = "hu-HU"]),
        // Displays 06:09:01 du.

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = ""]),
        // Displays 09:18:01.50 AM

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = "hu-HU"])
        // Displays 09:18:01.50 de.
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de mes "M"

Especificador de formato personalizado "M"

El especificador de formato personalizado "M" representa el mes como un número del 1 al 12 (o del 1 al 13 para los calendarios con 13 meses). Un mes con un solo dígito tiene un formato sin un cero inicial.

Si el especificador de formato "M" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "M". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "M" en una cadena de formato personalizado.

```
let
    Source =
    {
        Date.ToText(#date(2024, 8, 18), [Format = "(M) MMM, MMMM", Culture =
"en-US"]),
        // Displays (8) Aug, August

        Date.ToText(#date(2024, 8, 18), [Format = "(M) MMM, MMMM", Culture =
"nl-NL"]),
        // Displays (8) aug, augustus

        Date.ToText(#date(2024, 8, 18), [Format = "(M) MMM, MMMM", Culture =
"lv-LV"])
        // Displays (8) aug., augusts
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "MM"

El especificador de formato personalizado "MM" representa el mes como un número del 01 al 12 (o del 1 al 13 para los calendarios con 13 meses). Un mes con un solo dígito tiene un formato con un cero inicial.

En el ejemplo siguiente se incluye el especificador de formato personalizado "MM" en una cadena de formato personalizado.

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 2, 6, 30, 15), [Format = "dd,
MM", Culture = ""])
        // Displays 02, 01
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "MMM"

El especificador de formato personalizado "MMM" representa el nombre abreviado del mes. El nombre abreviado localizado del mes se recupera de los nombres de mes abreviados de la referencia cultural actual o especificada. Si hay un especificador de formato personalizado "d" o "dd" en la cadena de formato personalizado, el nombre del mes se recupera de los nombres genitivos abreviados en su lugar.

En el ejemplo siguiente se incluye el especificador de formato personalizado "MMM" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "ddd d MMM", Culture = "en-US"]),
        // Displays Thu 29 Aug

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "ddd d MMM", Culture = "fr-FR"])
        // Displays jeu. 29 août
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "MMMM"

El especificador de formato personalizado "MMMM" representa el nombre completo del mes. El nombre localizado del mes se recupera de la referencia cultural actual o especificada. Si hay un especificador de formato personalizado "d" o "dd" en la cadena de formato personalizado, el nombre del mes se recupera de los nombres genitivos abreviados en su lugar.

En el ejemplo siguiente se incluye el especificador de formato personalizado "MMMM" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "dddd dd MMMM", Culture = "en-US"]),
        // Displays Thursday 29 August
    }
in
    Source
```

```

        DateTime.ToText(#datetime(2024, 8, 29, 19, 27, 15), [Format = "dddd
dd MMMM", Culture = "it-IT"])
        // Displays giovedì 29 agosto
    }
in
Source

```

[Volver a la tabla](#)

Especificador de formato de segundos "s"

Especificador de formato personalizado "s"

El especificador de formato personalizado "s" representa los segundos como un número de 0 a 59. El resultado representa los segundos enteros que han transcurrido desde el último minuto. Un segundo con un solo dígito tiene un formato sin un cero inicial.

Si el especificador de formato "s" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "s". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "s" en una cadena de formato personalizado.

Power Query M

```

let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F
t", Culture = ""]),
        // Displays 6:9:1 P

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F
t", Culture = "el-GR"]),
        // Displays 6:9:1 μ

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"h:m:s.F t", Culture = ""]),
        // Displays 9:18:1.5 A

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"h:m:s.F t", Culture = "el-GR"])
        // Displays 9:18:1.5 π
    }

```

in
Source

[Volver a la tabla](#)

Especificador de formato personalizado "ss"

El especificador de formato personalizado "ss" (más cualquier número de especificadores "s" adicionales) representa los segundos como un número de 00 a 59. El resultado representa los segundos enteros que han transcurrido desde el último minuto. Un segundo con un solo dígito tiene un formato con un cero inicial.

En el ejemplo siguiente se incluye el especificador de formato personalizado "ss" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss tt", Culture = ""]),
        // Displays 06:09:01 PM

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss tt", Culture = "hu-HU"]),
        // Displays 06:09:01 du.

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "hh:mm:ss.ff tt", Culture = ""]),
        // Displays 09:18:01.50 AM

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "hh:mm:ss.ff tt", Culture = "hu-HU"])
        // Displays 09:18:01.50 de.
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de merídiem "t"

Especificador de formato personalizado "t"

El especificador de formato personalizado "t" representa el primer carácter del designador AM/PM. El designador localizado adecuado se recupera de la referencia cultural actual o específica. El designador AM se usa para todas las horas de 0:00:00 (medianoche) a 11:59:59.999. El designador PM se usa para todas las horas de 12:00:00 (mediodía) a 23:59:59.999.

Si el especificador de formato "t" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "t". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "t" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = ""]),
            // Displays 6:9:1 P

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "h:m:s.F t", Culture = "el-GR"]),
            // Displays 6:9:1 μ

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = ""]),
            // Displays 9:18:1.5 A

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format = "h:m:s.F t", Culture = "el-GR"])
            // Displays 9:18:1.5 π
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "tt"

El especificador de formato personalizado "tt" (más cualquier número de especificadores "t" adicionales) representa designador AM/PM completo. El designador localizado adecuado se recupera de la referencia cultural actual o específica. El

designador AM se usa para todas las horas de 0:00:00 (medianoche) a 11:59:59.999. El designador PM se usa para todas las horas de 12:00:00 (mediodía) a 23:59:59.999.

Asegúrese de usar el especificador "tt" para aquellos idiomas en los que sea necesario mantener la distinción entre a. m. y p. m. Un ejemplo es el japonés, en el que los designadores de a.m. y p.m. se diferencian en el segundo carácter en vez de en el primero.

En el ejemplo siguiente se incluye el especificador de formato personalizado "tt" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss
tt", Culture = ""]),
        // Displays 06:09:01 PM

        DateTime.ToText(#datetime(2024, 1, 1, 18, 9, 1), [Format = "hh:mm:ss
tt", Culture = "hu-HU"]),
        // Displays 06:09:01 du.

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = ""]),
        // Displays 09:18:01.50 AM

        DateTime.ToText(#datetime(2024, 1, 1, 9, 18, 1.500), [Format =
"hh:mm:ss.ff tt", Culture = "hu-HU"])
        // Displays 09:18:01.50 de.
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de año "y"

Especificador de formato personalizado "y"

El especificador de formato personalizado "y" representa el año como un número de uno o dos dígitos. Si el año tiene más de dos dígitos, en el resultado sólo aparecen los dos dígitos de orden inferior. Si el primer dígito de un año de dos dígitos comienza con un cero (por ejemplo, 2008), se aplica formato al número sin el cero inicial.

Si el especificador de formato "y" se usa sin otros especificadores de formato personalizado, se interpretará como el especificador de formato de fecha y hora estándar "y". Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "y" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(1, 12, 1), [Format = "%y"]),
        // Displays 1

        Date.ToText(#date(2024, 1, 1), [Format = "%y"])
        // Displays 24
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "yy"

El especificador de formato personalizado "yy" representa el año como un número de dos dígitos. Si el año tiene más de dos dígitos, en el resultado sólo aparecen los dos dígitos de orden inferior. Si el año de dos dígitos tiene menos de dos dígitos significativos, el número se rellenará con ceros iniciales hasta obtener dos dígitos.

En una operación de análisis, se interpreta un año de dos dígitos que se analiza mediante el especificador de formato personalizado "yy" en función del calendario actual del proveedor de formato. En el ejemplo siguiente se analiza la representación en forma de cadena de una fecha con un año de dos dígitos utilizando el calendario gregoriano predeterminado de la referencia cultural actual (en este caso, en-US). Los valores devueltos para la fecha de cuatro dígitos dependen del intervalo de 100 años establecido por el sistema operativo.

Power Query M

```
let
    // Define the date format and value
    fmt = "dd-MMM-yy",
```

```
// Convert year 49 to a 4-digit year
firstDate = Text.Format("#{0}", { Date.FromText("24-Jan-49", [Format =
fmt]) })),

// Convert year 50 to a 4-digit year
finalDate = Text.Format("#{0}", { Date.FromText("24-Jan-50", [Format =
fmt]) })),
Heading = "Default Two Digit Year Range: 1950 - 2049",
result = {Heading, firstDate, finalDate}
in
result

// The example displays the following output:
//      Default Two Digit Year Range: 1950 - 2049
//      1/24/2049
//      1/24/1950
```

En el ejemplo siguiente se incluye el especificador de formato personalizado "yy" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(1, 12, 1), [Format = "yy"]),
        // Displays 01

        Date.ToText(#date(2024, 1, 1), [Format = "yy"])
        // Displays 24
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "yyy"

El especificador de formato personalizado "yyy" representa el año con un mínimo de tres dígitos. Si el año tiene más de tres dígitos significativos, se incluyen en la cadena de resultado. Si el año tiene menos de tres dígitos, el número se rellenará con ceros iniciales hasta obtener tres dígitos.

ⓘ Nota

Para el calendario budista tailandés, que puede tener años de cinco dígitos, este especificador de formato muestra todos los dígitos significativos.

En el ejemplo siguiente se incluye el especificador de formato personalizado "yyy" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(1, 12, 1), [Format = "yyy"]),
        // Displays 001

        Date.ToText(#date(2024, 1, 1), [Format = "yyy"])
        // Displays 2024
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato personalizado "yyyy"

El especificador de formato personalizado "Yyyy" representa el año con un mínimo de cuatro dígitos. Si el año tiene más de cuatro dígitos significativos, se incluyen en la cadena resultante. Si el año tiene menos de cuatro dígitos, el número se completa con ceros iniciales hasta obtener cuatro dígitos.

ⓘ Nota

En el calendario budista tailandés, que puede tener años de cinco dígitos, este especificador de formato muestra un mínimo de cuatro dígitos.

En el ejemplo siguiente se incluye el especificador de formato personalizado "yyyy" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(1, 12, 1), [Format = "yyyy"]),
        // Displays 0001

        Date.ToText(#date(2024, 1, 1), [Format = "yyyy"])
        // Displays 2024
    }
in
    Source
```

in
Source

[Volver a la tabla](#)

Especificador de formato personalizado "yyyyy"

El especificador de formato personalizado "yyyyy" (más cualquier número de especificadores "y" adicionales) representa el año con un mínimo de cinco dígitos. Si el año tiene más de cinco dígitos significativos, se incluyen en la cadena resultante. Si el año tiene menos de cinco dígitos, el número se rellenará con ceros iniciales hasta obtener cinco dígitos.

Si hay especificadores "y" adicionales, el número se rellenará con tantos ceros iniciales como sean necesarios para obtener el número de especificadores "y".

En el ejemplo siguiente se incluye el especificador de formato personalizado "yyyyy" en una cadena de formato personalizado.

Power Query M

```
let
    Source =
    {
        Date.ToText(#date(1, 12, 1), [Format = "yyyyy"]),
        // Displays 00001

        Date.ToText(#date(2024, 1, 1), [Format = "yyyyy"])
        // Displays 02024
    }
in
    Source
```

[Volver a la tabla](#)

Especificador de formato de desfase "z"

Especificador de formato personalizado "z"

Con los valores `dateTimeZone`, el especificador de formato personalizado "z" representa el desplazamiento firmado de la zona horaria especificada de la hora universal coordinada (UTC), medida en horas. La diferencia horaria se muestra siempre con un signo inicial. Un signo más (+) indica las horas de adelanto y un signo menos (-)

indica las horas de retraso con respecto a la hora UTC. Un desfase con un solo dígito tiene un formato *sin* un cero inicial.

En la tabla siguiente se muestra cómo cambia el valor de desplazamiento en función de la función `DateTimeZone` .

 Expandir tabla

Valor de <code>DateTimeZone</code>	Valor de desplazamiento
<code>DateTimeZone.LocalNow</code>	En Power Query Desktop, el desplazamiento firmado de la zona horaria del sistema operativo local desde UTC. En Power Query Online, devuelve <code>+00</code> .
<code>DateTimeZone.UtcNow</code>	Devuelve <code>+0</code> .

Si el especificador de formato "z" se usa sin otros especificadores de formato personalizado, se interpreta como un especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

En el ejemplo siguiente se incluye el especificador de formato personalizado "z" en una cadena de formato personalizado en un sistema de la zona horaria del Pacífico de EE. UU.

Power Query M

```
let
    Source =
    {
        DateTimeZone.ToText(DateTimeZone.LocalNow(), [Format="{0:%z}"]),
        // Displays {0:-7} on Power Query Desktop
        // Displays {0:+0} on Power Query Online

        DateTimeZone.ToText(DateTimeZone.UtcNow,[Format="{0:%z}"]),
        // Displays {0:+0}

        DateTimeZone.ToText(DateTimeZone.SwitchZone(
            #datetimezone(2024, 8, 1, 0, 0, 0, 0, 0), 6),
            [Format = "{0:%z}"])
        // Displays {0:+6}
    }
in
    Source
```

ⓘ Nota

El valor devuelto por `DateTimeZone.LocalNow` depende de si ejecuta Power Query en una máquina local o en línea. Por ejemplo, en el ejemplo anterior en un sistema de la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve `{0:-7}` porque lee la hora establecida en el equipo local. Sin embargo, Power Query Online devuelve `{0:+0}` porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

[Volver a la tabla](#)

Especificador de formato personalizado "zz"

Con los valores `DateTimeZone`, el especificador de formato personalizado "zz" representa el desplazamiento firmado de la zona horaria especificada de UTC, medida en horas. La diferencia horaria se muestra siempre con un signo inicial. Un signo más (+) indica las horas de adelanto y un signo menos (-) indica las horas de retraso con respecto a la hora UTC. Un desfase con un solo dígito tiene un formato *con* un cero inicial.

En la tabla siguiente se muestra cómo cambia el valor de desplazamiento en función de la función `DateTimeZone`.

 Expandir tabla

Valor de <code>DateTimeZone</code>	Valor de desplazamiento
<code>DateTimeZone.LocalNow</code>	En Power Query Desktop, el desplazamiento firmado de la zona horaria del sistema operativo local desde UTC. En Power Query Online, devuelve <code>+00</code> .
<code>DateTimeZone.UtcNow</code>	Devuelve <code>+00</code> .

En el ejemplo siguiente se incluye el especificador de formato personalizado "zz" en una cadena de formato personalizado en un sistema de la zona horaria del Pacífico de EE. UU.

Power Query M

```
let
    Source =
    {
```

```

DateTimeZone.ToText(DateTimeZone.LocalNow(), [Format="{0:zz}"]),
// Displays {0:-07} on Power Query Desktop
// Displays {0:+00} on Power Query Online

DateTimeZone.ToText(DateTimeZone.UtcNow(),[Format="{0:zz}"]),
// Displays {0:+00}

DateTimeZone.ToText(DateTimeZone.SwitchZone(
    #datetimezone(2024, 8, 1, 0, 0, 0, 0, 0), 6),
    [Format = "{0:zz}"])
)
// Displays {0:+06}
}
in
Source

```

ⓘ Nota

El valor devuelto por **DateTimeZone.LocalNow** depende de si ejecuta Power Query en una máquina local o en línea. Por ejemplo, en el ejemplo anterior en un sistema de la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve **{0:-07}** porque lee la hora establecida en el equipo local. Sin embargo, Power Query Online devuelve **{0:+00}** porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

[Volver a la tabla](#)

Especificador de formato personalizado "zzz"

Con los valores **DateTimeZone** , el especificador de formato personalizado "zzz" representa el desplazamiento firmado de la zona horaria especificada de UTC, medida en horas y minutos. La diferencia horaria se muestra siempre con un signo inicial. Un signo más (+) indica las horas de adelanto y un signo menos (-) indica las horas de retraso con respecto a la hora UTC. Un desfase con un solo dígito tiene un formato con un cero inicial.

En la tabla siguiente se muestra cómo cambia el valor de desplazamiento en función de la función **DateTimeZone** .

[Expandir tabla](#)

Valor <code>dateTimeZoneValue</code>	Valor de desplazamiento
<code>DateTimeZone.LocalNow</code>	En Power Query Desktop, el desplazamiento firmado de la zona horaria del sistema operativo local desde UTC. En Power Query Online, devuelve <code>+00</code> .
<code>DateTimeZone.UtcNow</code>	Devuelve <code>+00:00</code> .

En el ejemplo siguiente se incluye el especificador de formato personalizado "zzz" en una cadena de formato personalizado en un sistema de la zona horaria del Pacífico de EE. UU.

```
Power Query M

let
    Source =
    {
        DateTimeZone.ToText(DateTimeZone.LocalNow(), [Format="{0:zzz}"]),
        // Displays {0:-07:00} on Power Query Desktop
        // Displays {0:+00:00} on Power Query Online

        DateTimeZone.ToText(DateTimeZone.UtcNow(), [Format="{0:zzz}"]),
        // Displays {0:+00:00}

        DateTimeZone.ToText(DateTimeZone.SwitchZone(
            #datetimezone(2024, 8, 1, 0, 0, 0, 0, 6),
            [Format = "{0:zzz}"]
        ),
        // Displays {0:+06:00}
    }
in
    Source
```

ⓘ Nota

El valor devuelto por [`DateTimeZone.LocalNow`](#) depende de si ejecuta Power Query en una máquina local o en línea. Por ejemplo, en el ejemplo anterior en un sistema de la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve `{0:-07:00}` porque lee la hora establecida en el equipo local. Sin embargo, Power Query Online devuelve `{0:+00:00}` porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

[Volver a la tabla](#)

Especificadores de separador de fecha y hora

Especificador de formato personalizado ":"

El especificador de formato personalizado ":" representa el separador de hora, que se usa para diferenciar horas, minutos y segundos. El separador de hora localizado adecuado se recupera de la referencia cultural actual o especificada.

ⓘ Nota

Para cambiar el separador de hora en una determinada cadena de fecha y hora, especifique el carácter separador en un delimitador de cadena literal. Por ejemplo, la cadena de formato personalizado `hh_dd_ss` genera una cadena en que "_" (guión bajo) siempre se utiliza como separador de hora.

Si el especificador de formato ":" se usa sin otros especificadores de formato personalizado, se interpreta como especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un especificador de formato único, vaya a [Usar especificadores de formato personalizado único](#) más adelante en este artículo.

[Volver a la tabla](#)

Especificador de formato personalizado "/"

El especificador de formato personalizado "/" representa el separador de fecha, que se usa para diferenciar años, meses y días. El separador de fecha localizado adecuado se recupera de la referencia cultural actual o especificada.

ⓘ Nota

Para cambiar el separador de fecha en una determinada cadena de fecha y hora, especifique el carácter separador en un delimitador de cadena literal. Por ejemplo, la cadena de formato personalizado `mm/dd/yyyy` genera una cadena en que "/" siempre se utiliza como separador de fecha.

Si el especificador de formato "/" se usa sin otros especificadores de formato personalizado, se interpreta como especificador de formato de fecha y hora estándar y produce un error de expresión. Para obtener más información sobre el uso de un

especificador de formato único, vaya a [Usar especificadores](#) de formato personalizado único más adelante en este artículo.

[Volver a la tabla](#)

Literales de carácter

Los siguientes caracteres de una cadena de formato de fecha y hora personalizada están reservados y siempre se interpretan como caracteres de formato o, en el caso de `"`, `'`, `/` y `\`, como caracteres especiales.

- `F`
- `H`
- `K`
- `M`
- `d`
- `f`
- `g`
- `h`
- `m`
- `s`
- `t`
- `y`
- `z`
- `%`
- `:`
- `/`
- `"`
- `'`
- `\`

Todos los demás caracteres se interpretan siempre como literales de carácter y, en una operación de formato, se incluyen en la cadena de resultado sin modificar. En una operación de análisis, deben coincidir exactamente con los caracteres de la cadena de entrada; la comparación distingue entre mayúsculas y minúsculas.

En el ejemplo siguiente se incluyen los caracteres literales "PST" (para hora estándar del Pacífico) y "PDT" (para horario de verano del Pacífico) para representar la zona horaria local en una cadena de formato. Tenga en cuenta que la cadena se incluye en la cadena

de resultado y que una cadena que incluye la cadena de zona horaria local también se analiza correctamente.

Power Query M

```
let
    #"Date Formats" = {"dd MMM yyyy hh:mm tt PST", "dd MMM yyyy hh:mm tt PDT"},
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 18, 16, 50, 0), [Format = #"Date Formats"{1}]),
        try DateTime.ToText(DateTime.FromText(
            "25 Dec 2023 12:00 pm PST", [Format = #"Date Formats"{0}]))
            otherwise "Unable to parse '" & "25 Dec 2023 12:00 pm PST" & "'"
    }
in
    Source

// The example displays the following output text:
//      18 Aug 2024 04:50 PM PDT
//      12/25/2023 12:00:00 PM
```

Hay dos formas de indicar que los caracteres se han de interpretar como caracteres literales y no como caracteres de reserva, para que se puedan incluir en una cadena de resultado o analizarse correctamente en una cadena de entrada:

- Al incluir un escape con cada carácter reservado. Para obtener más información, vaya a [Uso de las secuencias de escape](#).

En el ejemplo siguiente se incluyen los caracteres literales "pst" (para hora estándar del Pacífico) para representar la zona horaria local en una cadena de formato. Como "s" y "t" son cadenas de formato personalizado, ambos caracteres deben incluir un escape para interpretarse como literales de carácter.

Power Query M

```
let
    #"Date Format" = "dd MMM yyyy hh:mm tt p's't'",
    Source =
    {
        DateTime.ToText(#datetime(2024, 8, 18, 16, 50, 0), [Format = #"Date Format"]),
        try DateTime.ToText(DateTime.FromText(
            "25 Dec 2023 12:00 pm pst", [Format = #"Date Format"]))
            otherwise "Unable to parse '" & "25 Dec 2023 12:00 pm
pst" & "'"
    }
in
    Source
```

```
// The example displays the following output text:  
//      18 Aug 2024 04:50 PM pst  
//      12/25/2016 12:00:00 PM
```

- Al incluir toda la cadena literal en apóstrofes. El ejemplo siguiente es similar al anterior, salvo que "pst" está entre apóstrofes para indicar que toda la cadena delimitada debe interpretarse como literales de caracteres.

Power Query M

```
let  
    #"Date Format" = "dd MMM yyyy hh:mm tt 'pst'",  
    Source =  
    {  
        DateTime.ToText(#datetime(2024, 8, 18, 16, 50, 0), [Format =  
        #"Date Format"]),  
        try DateTime.ToText(DateTime.FromText(  
            "25 Dec 2023 12:00 pm pst", [Format = #"Date Format"]))  
        otherwise "Unable to parse '" & "25 Dec 2023 12:00 pm  
    pst" & "'"  
    }  
in  
    Source  
  
// The example displays the following output text:  
//      18 Aug 2024 04:50 PM pst  
//      12/25/2016 12:00:00 PM
```

Notas

Usar especificadores de formato personalizado únicos

Una cadena con formato de fecha y hora personalizado se compone de dos o más caracteres. Los métodos de formato de fecha y hora interpretan cualquier cadena de un único carácter como una cadena de formato de fecha y hora estándar. Si no reconocen el carácter como especificador de formato válido, producen un error de expresión. Por ejemplo, una cadena de formato que solo se compone del especificador "h" se interpreta como una cadena de formato de fecha y hora estándar. Pero en este caso concreto se inicia una excepción porque no existe ningún especificador de formato de fecha y hora estándar "h".

Para usar cualquiera de los especificadores de formato de fecha y hora personalizado como el único especificador en una cadena de formato (es decir, usar el especificador de formato personalizado "d", "f", "F", "g", "h", "H", "K", "m", "M", "s", "t", "y", "z", ":" o

"/"), incluya un espacio delante o detrás del especificador, o incluya un especificador de formato de porcentaje ("%") delante del único especificador de fecha y hora personalizado.

Por ejemplo, "%h" se interpreta como una cadena de formato de fecha y hora personalizado que muestra la hora representada por el valor de fecha y hora actual. También puede usar la cadena de formato "h" o "h ", aunque esto incluye un espacio en la cadena de resultado junto con la hora. En el ejemplo siguiente se muestran estas tres cadenas de formato.

Power Query M

```
let
    date = #datetime(2024, 6, 15, 13, 45, 0),
    Source =
    {
        Text.Format("'#{0}'", {DateTime.ToText(date, [Format = "%h"])}),
        Text.Format("'#{0}'", {DateTime.ToText(date, [Format = " h"])}),
        Text.Format("'#{0}'", {DateTime.ToText(date, [Format = "h "])})
    }
in
    Source

// The example displays a list with the following output text,
// with <sp> representing a space:
//      '1'
//      ' 1'
//      '1 '
```

Uso de las secuencias de escape

Los caracteres "d", "f", "F", "g", "h", "H", "K", "m", "M", "s", "t", "y", "z", ":" o "/" en una cadena de formato se interpretan como especificadores de formato personalizado en lugar de como caracteres literales.

Para evitar que un carácter se interprete como especificador de formato, puede hacer lo siguiente:

- Precedítelo con una barra diagonal inversa.
- Envolverlo con una sola comilla.
- Envolverlo con dos comillas dobles.

Cada uno de estos caracteres actúa como una secuencia de escape. La secuencia de escape indica que el siguiente carácter o carácter rodeado es un literal de texto que debe incluirse en la cadena de resultado sin cambios.

Para incluir una comilla doble en una cadena de resultado, debe escaparla con otra comilla doble ("").

En el ejemplo siguiente se usan diferentes secuencias de escape para evitar que la operación de formato interprete los caracteres "h" y "m" como especificadores de formato.

Power Query M

```
let
    date = #datetime(2024, 6, 15, 13, 45, 30.90),
    format1 = "h \h m \m",
    format2 = "h ""h"" m ""m""",
    format3 = "h 'h' m 'm'",
    Source =
    {
        Text.Format("#{0} ({1}) -> #{2}", {DateTime.ToText(date), format1,
        DateTime.ToText(date, format1)}),
        Text.Format("#{0} ({1}) -> #{2}", {DateTime.ToText(date), format2,
        DateTime.ToText(date, format2)}),
        Text.Format("#{0} ({1}) -> #{2}", {DateTime.ToText(date), format3,
        DateTime.ToText(date, format3)})
    }
in
    Source

// The example displays the following output text:
//      6/15/2024 1:45:30 PM (h \h m \m) -> 1 h 45 m
//      6/15/2024 1:45:30 PM (h "h" m "m") -> 1 h 45 m
//      6/15/2024 1:45:30 PM (h 'h' m 'm') -> 1 h 45 m
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Conversión de tipos Date, Time, DateTime y DateTimeZone](#)
- [Funciones de fecha](#)
- [Funciones DateTime](#)
- [Funciones DateTimeZone](#)
- [Funciones de hora](#)
- [Cadenas con formato de fecha y hora estándar](#)

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

Cómo afecta la referencia cultural al formato de texto

Los distintos países y grupos de idiomas tienen convenciones diferentes para dar formato a diferentes tipos de texto, como números, fechas y hora. En Power Query, la *referencia cultural* hace referencia a la configuración regional cuyas convenciones se usan para dar formato a este tipo de texto en Power Query M. Esta configuración regional se compone generalmente del idioma y del país donde se habla el idioma, por ejemplo "en-US" para el idioma inglés hablado en los Estados Unidos de América.

Referencia cultural predeterminada

La referencia cultural predeterminada se establece en la configuración regional del sistema (Windows, MacOS) de un documento determinado donde las consultas se crean por primera vez. Por ejemplo, si crea las consultas en Power Query Desktop, la referencia cultural predeterminada se define mediante la configuración regional establecida en el equipo local. Sin embargo, si crea las consultas en Power Query Online, la referencia cultural predeterminada se define mediante la configuración regional establecida en el servicio en línea. Independientemente de dónde cree la consulta, si mueve la consulta a una ubicación diferente que usa una referencia cultural predeterminada diferente, la consulta sigue usando la referencia cultural de la ubicación original.

Para usar la referencia cultural predeterminada actual, no se requiere ninguna configuración de referencia cultural en el código M de Power Query.

Sin embargo, puede cambiar la referencia cultural predeterminada en el cuadro de diálogo Configuración de Power Query donde cree la consulta. Por ejemplo, si ejecuta Power Query desde Excel:

1. En Power Query, seleccione > >
2. En **Libro actual**, seleccione **Configuración** regional.
3. Seleccione la configuración regional que desea usar.

Otras versiones de Power Query funcionan de forma similar. En general, en Power Query, seleccione **Opciones**, que abre el **cuadro de diálogo Opciones**. A continuación, seleccione **Configuración** regional y seleccione la configuración regional que desea usar.

Referencia cultural invariable

La referencia cultural invariable no distingue la referencia cultural; está asociado al idioma inglés, pero no a ningún país o región. Especifique la referencia cultural invariable por nombre

mediante una cadena vacía ("") en funciones que incluyan el parámetro de referencia cultural.

A diferencia de los datos confidenciales de la referencia cultural, que están sujetos a cambios por personalización de usuario o por actualizaciones del sistema operativo, los datos de referencia cultural invariable son estables a lo largo del tiempo y entre las referencias culturales instaladas y los usuarios no pueden personalizarlos. Esto hace que la referencia cultural invariable sea especialmente útil para las operaciones que requieren resultados independientes de la referencia cultural, como las operaciones de formato y análisis que conservan los datos con formato, o las operaciones de ordenación y ordenación que requieren que los datos se muestren en un orden fijo, independientemente de la referencia cultural.

Para usar la referencia cultural invariable en Power Query M, use el valor de texto en blanco ("") en las funciones numéricas que admiten referencia cultural o (Culture = "") en funciones de fecha y hora que admiten la referencia cultural.

Contenido relacionado

- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas con formato de fecha y hora personalizado](#)
- [Cadenas con formato numérico estándar](#)
- [cadenas de formato numérico personalizado](#)

Last updated on 22/11/2025

Capitalización en Power Query M

Artículo • 31/05/2025

Trabajar con datos de texto a veces puede ser desordenado. Por ejemplo, el nombre de la ciudad Redmond podría representarse en una base de datos usando diferentes capitalizaciones ("Redmond", "redmond", "REDMOND"). Esto podría provocar un problema al transformar los datos en Power Query porque el lenguaje de fórmulas M de Power Query distingue mayúsculas de minúsculas.

Afortunadamente, Power Query M proporciona funciones para limpiar y normalizar el caso de datos de texto. Hay funciones para convertir texto en minúsculas (`abc`), mayúsculas (`ABC`) o mayúsculas iniciales (`Abc`). Además, Power Query M también proporciona varias maneras de omitir mayúsculas y minúsculas por completo.

En este artículo se muestra cómo cambiar el uso de mayúsculas y minúsculas de palabras en texto, listas y tablas. También describe varias maneras de omitir mayúsculas y minúsculas al manipular datos en texto, listas y tablas. Además, en este artículo se describe cómo ordenar según el caso.

Cambio de mayúsculas y minúsculas del texto

Hay tres funciones que convierten texto en minúsculas, mayúsculas y capitalización de títulos. Las funciones son `Text.Lower`, `Text.Uppery` `Text.Proper`. En los siguientes ejemplos sencillos se muestra cómo se pueden usar estas funciones en texto.

Convertir todos los caracteres del texto en minúsculas

En el ejemplo siguiente se muestra cómo convertir todos los caracteres de una cadena en minúsculas.

Power Query M

```
let
    Source = Text.Lower("The quick brown fox jumps over the lazy dog.")
in
    Source
```

Este código genera la siguiente salida:

```
the quick brown fox jumps over the lazy dog.
```

Convertir todos los caracteres del texto a mayúsculas

En el ejemplo siguiente se muestra cómo convertir todos los caracteres de una cadena de texto en mayúsculas.

Power Query M

```
let
    Source = Text.Upper("The quick brown fox jumps over the lazy dog.")
in
    Source
```

Este código genera la siguiente salida:

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG.

Convertir todas las palabras en mayúsculas iniciales

En el ejemplo siguiente se muestra cómo convertir todas las palabras de la oración en mayúsculas iniciales.

Power Query M

```
let
    Source = Text.Proper("The quick brown fox jumps over the lazy dog.")
in
    Source
```

Este código genera la siguiente salida:

The Quick Brown Fox Jumps Over The Lazy Dog.

Cambio de mayúsculas y minúsculas en listas

Al modificar mayúsculas y minúsculas en listas, la función más común a utilizar es [List.Transform](#). En los ejemplos sencillos siguientes se muestra cómo se puede usar esta función en listas.

Convertir todos los elementos en minúsculas

En el ejemplo siguiente se muestra cómo cambiar todos los elementos de una lista a minúsculas.

Power Query M

```

let
    Source = {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},
    #"Lower Case" = List.Transform(Source, Text.Lower)
in
    #"Lower Case"

```

Este código genera la siguiente salida:

	List
1	squash
2	pumpkin
3	apple
4	pear
5	orange
6	apple
7	pear
8	pear

Convertir todos los elementos en mayúsculas

En el ejemplo siguiente se muestra cómo cambiar todos los elementos de una lista a mayúsculas.

Power Query M

```

let
    Source = {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},
    #"Upper Case" = List.Transform(Source, Text.Upper)
in
    #"Upper Case"

```

Este código genera la siguiente salida:

	List
1	SQUASH
2	PUMPKIN
3	APPLE
4	PEAR
5	ORANGE
6	APPLE
7	PEAR
8	PEAR

Convertir todos los elementos al formato adecuado

En el ejemplo siguiente se muestra cómo cambiar todos los elementos de una lista a mayúsculas y minúsculas adecuadas.

Power Query M

```
let
    Source = {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},
    #"Proper Case" = List.Transform(Source, Text.Proper)
in
    #"Proper Case"
```

Este código genera la siguiente salida:

	List
1	Squash
2	Pumpkin
3	Apple
4	Pear
5	Orange
6	Apple
7	Pear
8	Pear

Cambiar mayúsculas y minúsculas en tablas

Al cambiar el caso en las tablas, la función más común que se utiliza es [Table.TransformColumns](#). También hay una función que puede usar para cambiar las mayúsculas y minúsculas del texto contenido en una fila, denominada [Table.TransformRows](#). Sin embargo, esta función no se usa con tanta frecuencia.

En los siguientes ejemplos sencillos se muestra cómo se puede usar la `Table.TransformColumns` función para cambiar el caso en las tablas.

Convertir todos los elementos de una columna de tabla en minúsculas

En el ejemplo siguiente se muestra cómo cambiar todos los elementos de una columna de tabla a minúsculas, en este caso, los nombres de cliente.

Power Query M

```
let
    Source = #table(type table [CUSTOMER = text, FRUIT = text],
        {
            {"Tulga", "Squash"},
            {"suSanna", "Pumpkin"},
            {"LESLIE", "ApPlE"},
            {"Willis", "pear"},
            {"Dilbar", "orange"},
            {"ClaudiA", "APPLE"},
            {"afonso", "Pear"},
            {"SErgio", "pear"}
        }
    ),
    #"Lower Case" = Table.TransformColumns(Source, {"CUSTOMER", Text.Lower})
in
    #"Lower Case"
```

Este código genera la siguiente salida:

	 CUSTOMER	 FRUIT
1	tulga	Squash
2	susanna	Pumpkin
3	leslie	ApPIE
4	willis	pear
5	dilbar	orange
6	claudia	APPLE
7	afonso	Pear
8	sergio	pear

Convertir todos los elementos de una columna de la tabla en mayúsculas

En el ejemplo siguiente se muestra cómo convertir todos los elementos de una columna de tabla a mayúsculas, en este caso, los nombres de las frutas.

Power Query M

```
let
    Source = #table(type table [CUSTOMER = text, FRUIT = text],
        {
            {"Tulga", "Squash"},
            {"suSanna", "Pumpkin"},
            {"LESLIE", "ApPIE"},
            {"Willis", "pear"},
            {"Dilbar", "orange"},
            {"ClaudiA", "APPLE"},
            {"afonso", "Pear"},
            {"SErgio", "pear"}
        }
    ),
    #"Upper Case" = Table.TransformColumns(Source, {"FRUIT", Text.Upper})
in
    #"Upper Case"
```

Este código genera la siguiente salida:

	 CUSTOMER	 FRUIT
1	Tulga	SQUASH
2	suSanna	PUMPKIN
3	LESLIE	APPLE
4	Willis	PEAR
5	Dilbar	ORANGE
6	ClaudiA	APPLE
7	afonso	PEAR
8	SErgio	PEAR

Convertir todos los elementos de una tabla a mayúsculas iniciales

En el ejemplo siguiente se muestra cómo cambiar todos los elementos de ambas columnas de tabla a mayúsculas y minúsculas adecuadas.

Power Query M

```
let
    Source = #table(type table [CUSTOMER = text, FRUIT = text],
        {
            {"Tulga", "Squash"},
            {"suSanna", "Pumpkin"},
            {"LESLIE", "ApPLE"},
            {"Willis", "pear"},
            {"Dilbar", "orange"},
            {"ClaudiA", "APPLE"},
            {"afonso", "Pear"},
            {"SErgio", "pear"}
        }
    ),
    #"Customer Case" = Table.TransformColumns(Source, {"CUSTOMER", Text.Proper}),
    #"Proper Case" = Table.TransformColumns(#"Customer Case", {"FRUIT",
Text.Proper})
in
    #"Proper Case"
```

Este código genera la siguiente salida:

	ABC CUSTOMER ▾	ABC FRUIT ▾
1	Tulga	Squash
2	Susanna	Pumpkin
3	Leslie	Apple
4	Willis	Pear
5	Dilbar	Orange
6	Claudia	Apple
7	Afonso	Pear
8	Sergio	Pear

Ignorar mayúsculas y minúsculas

En muchos casos al buscar o reemplazar elementos, es posible que tenga que omitir el caso del elemento que busca. Dado que el lenguaje de fórmulas M de Power Query distingue entre mayúsculas y minúsculas, las comparaciones entre elementos que son idénticos pero tienen mayúsculas y minúsculas diferentes resultan en la consideración de dichos elementos como diferentes, no idénticos. Un método de omitir mayúsculas y minúsculas implica el uso de la función [Comparar.OrdinalIgnoreCase](#) en funciones que incluyen un parámetro `equationCriteria` o un parámetro `comparer`. Otro método para ignorar la diferencia entre mayúsculas y minúsculas implica el uso de la opción `IgnoreCase` (si está disponible) en funciones que incluyen el parámetro `options`.

Ignorar las mayúsculas y minúsculas en el texto

Las búsquedas en texto a veces requieren que se ignore la distinción entre mayúsculas y minúsculas para poder encontrar todas las instancias del texto buscado. Las funciones de texto suelen usar la `Comparar.OrdinalIgnoreCase` función en el `comparer` parámetro para ignorar la distinción entre mayúsculas y minúsculas al comprobar la igualdad.

En el ejemplo siguiente se muestra cómo omitir mayúsculas y minúsculas al determinar si una frase contiene una palabra específica, independientemente del caso.

Power Query M

```
let
    Source = Text.Contains(
        "The rain in spain falls mainly on the plain.",
        "Spain",
        Comparer.OrdinalIgnoreCase
```

```
)  
in  
    Source
```

Este código genera la siguiente salida:

```
true
```

En el ejemplo siguiente se muestra cómo recuperar la posición inicial de la última aparición de la palabra "the" en la oración, independientemente del caso.

Power Query M

```
let  
    Source = Text.PositionOf(  
        "THE RAIN IN SPAIN FALLS MAINLY ON THE PLAIN.",  
        "the",  
        Occurrence.Last,  
        Comparer.OrdinalIgnoreCase  
    )  
in  
    Source
```

Este código genera la siguiente salida:

```
34
```

Omitir mayúsculas y minúsculas en listas

Cualquier función de lista que contenga un parámetro opcional `equationCriteria` puede usar la función [Comparer.OrdinalIgnoreCase](#) para ignorar mayúsculas y minúsculas en la lista.

En el ejemplo siguiente se comprueba si una lista contiene un elemento específico, ignorando las mayúsculas y minúsculas. En este ejemplo, [List.Contains](#) solo se puede comparar un elemento de la lista, no se puede comparar una lista con una lista. Para ello, debe usar [List.ContainsAny](#).

Power Query M

```
let  
    Source = List.Contains(  
        {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},  
        "apple",  
        Comparer.OrdinalIgnoreCase  
    )
```

```
in
Source
```

Este código genera la siguiente salida:

```
true
```

En los ejemplos siguientes se comprueba si una lista contiene todos los elementos especificados en el segundo parámetro (`value`), ignorando mayúsculas y minúsculas. Si algún elemento no está incluido en la lista, como `cucumber` en el segundo ejemplo, la función devuelve FALSE.

Power Query M

```
let
    Source = List.ContainsAll(
        {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},
        {"apple", "pear", "squash", "pumpkin"},
        Comparer.OrdinalIgnoreCase
    )
in
    Source
```

Este código genera la siguiente salida:

```
true
```

Power Query M

```
let
    Source = List.ContainsAll(
        {"Squash", "Pumpkin", "ApPlE", "pear", "orange", "APPLE", "Pear", "pear"},
        {"apple", "pear", "squash", "pumpkin", "cucumber"},
        Comparer.OrdinalIgnoreCase
    )
in
    Source
```

Este código genera la siguiente salida:

```
false
```

En el ejemplo siguiente se comprueba si alguno de los elementos de la lista son manzanas o peras, ignorando mayúsculas y minúsculas.

Power Query M

```

let
    Source = List.ContainsAny(
        {"Squash", "Pumpkin", "ApPle", "PEAR", "orange", "APPLE", "Pear", "peaR"},
        {"apple", "pear"},
        Comparer.OrdinalIgnoreCase
    )
in
    Source

```

Este código genera la siguiente salida:

```
true
```

En el ejemplo siguiente solo se mantienen elementos únicos, ignorando las mayúsculas y minúsculas.

Power Query M

```

let
    Source = List.Distinct(
        {"Squash", "Pumpkin", "ApPle", "PEAR", "orange", "APPLE", "Pear", "peaR"},
        Comparer.OrdinalIgnoreCase
    )
in
    Source

```

Este código genera la siguiente salida:

	List
1	Squash
2	Pumpkin
3	ApPIE
4	PEAR
5	orange

En el ejemplo anterior, la salida muestra el caso del primer elemento único que se encuentra en la lista. Por lo tanto, aunque hay dos manzanas (ApPLe y APPLE), solo se muestra el primer ejemplo encontrado.

En el siguiente ejemplo, solo se mantienen los elementos únicos ignorando las mayúsculas, pero todos los resultados se devuelven en minúsculas.

Power Query M

```

let
    Source = List.Distinct(
        {"Squash", "Pumpkin", "ApPlE", "PEAR", "orange", "APPLE", "Pear", "peaR"},
        Comparer.OrdinalIgnoreCase
    ),
    #"Lower Case" = List.Transform(Source, Text.Lower)
in
    #"Lower Case"

```

Este código genera la siguiente salida:

	List
1	squash
2	pumpkin
3	apple
4	pear
5	orange

Omitir mayúsculas y minúsculas en tablas

Las tablas tienen varias maneras de ignorar la diferencia entre mayúsculas y minúsculas. Las funciones de tabla como [Table.Contains](#), [Table.Distinct](#) y [Table.PositionOf](#) todas contienen `equationCriteria` parámetros. Estos parámetros pueden usar la función [Comparer.OrdinalIgnoreCase](#) para ignorar mayúsculas y minúsculas en las tablas, de la misma manera que las listas de las secciones anteriores. Las funciones de tabla, como [Table.MatchesAnyRows](#) que contienen un parámetro `condition`, también pueden usar [Comparer.OrdinalIgnoreCase](#) anidadas en otras funciones de tabla para ignorar las mayúsculas y minúsculas. Otras funciones de tabla, específicamente para coincidencia aproximada, pueden usar la `IgnoreCase` opción .

En el ejemplo siguiente se muestra cómo seleccionar filas específicas que contienen la palabra "pear" al ignorar mayúsculas y minúsculas. En este ejemplo, se usa el parámetro `condition` de [Table.SelectRows](#) con [Text.Contains](#) como condicional para realizar las comparaciones, ignorando las mayúsculas y minúsculas.

Power Query M

```

let
    Source = #table(type table[CUSTOMER = text, FRUIT = text],
        {
            {"Tulga", "Squash"},

```

```

        {"suSanna", "Pumpkin"},
        {"LESLIE", "ApPlE"},
        {"Willis", "pear"},
        {"Dilbar", "orange"},
        {"ClaudiA", "APPLE"},
        {"afonso", "Pear"},
        {"SErgio", "pear"}
    }),
    #"Select Rows" = Table.SelectRows(
        Source, each Text.Contains([FRUIT], "pear", Comparer.OrdinalIgnoreCase))
in
    #"Select Rows"

```

Este código genera la siguiente salida:

	A ^B _C CUSTOMER ▾	A ^B _C FRUIT ▾
1	Willis	pear
2	afonso	Pear
3	SErgio	pear

En el ejemplo siguiente se muestra cómo determinar si alguna de las filas de una tabla contiene un `pear` elemento en la `FRUIT` columna . En este ejemplo se usa [Comparer.OrdinalIgnoreCase](#) en una función [Text.Contains](#) usando el parámetro `condition` de la función [Table.MatchesAnyRows](#).

Power Query M

```

let
    Source = #table(type table [CUSTOMER = text, FRUIT = text],
        {
            {"Tulga", "Squash"},
            {"suSanna", "Pumpkin"},
            {"LESLIE", "ApPlE"},
            {"Willis", "PEAR"},
            {"Dilbar", "orange"},
            {"ClaudiA", "APPLE"},
            {"afonso", "Pear"},
            {"SErgio", "peAR"}
        }
    ),
    #"Select Rows" = Table.MatchesAnyRows(Source,
        each Text.Contains([FRUIT], "pear", Comparer.OrdinalIgnoreCase))
in
    #"Select Rows"

```

Este código genera la siguiente salida:

true

En el ejemplo siguiente se muestra cómo tomar una tabla con valores especificados por los usuarios que contienen una columna que enumera sus frutos favoritos, sin ningún formato establecido. Esta columna se utiliza inicialmente para coincidencia aproximada con el fin de extraer los nombres de sus frutas favoritas, la cual se muestra posteriormente en una columna propia, llamada **Cluster**. A continuación, se examina la columna **Clúster** para determinar las distintas frutas que se encuentran en la columna. Una vez que se determinan las frutas únicas, un paso final es cambiar todos los nombres de las frutas a minúsculas.

Power Query M

```
let
    // Load a table of user's favorite fruits into Source
    Source = #table(type table [Fruit = text], {"blueberries"},
        {"Blue berries are simply the best"}, {"strawberries"}, {"Strawberries =
<3"},
        {"Apples"}, {"'sples"}, {"4ppl3s"}, {"Bananas"}, {"fav fruit is bananas"},
        {"Banas"}, {"My favorite fruit, by far, is Apples. I simply love them!"}}
    ),
    // Create a Cluster column and fuzzy match the fruits into that column
    #"Cluster fuzzy match" = Table.AddFuzzyClusterColumn(
        Source, "Fruit", "Cluster",
        [IgnoreCase = true, IgnoreSpace = true, Threshold = 0.5]
    ),
    // Find the distinct fruits from the Cluster column
    #"Ignore cluster case" = Table.Distinct(
        Table.SelectColumns(#"Cluster fuzzy match", "Cluster"),
        Comparer.OrdinalIgnoreCase
    ),
    // Set all of the distinct fruit names to lower case
    #"Set lower case" = Table.TransformColumns(#"Ignore cluster case",
        {"Cluster", Text.Lower}
    )
in
    #"Set lower case"
```

Este código genera la siguiente salida:

	Cluster
1	blueberries
2	strawberries
3	apples
4	bananas

Ordenación y manejo de mayúsculas y minúsculas

Las listas y las tablas se pueden ordenar mediante [List.Sort](#) o [Table.Sort](#), respectivamente. Sin embargo, la ordenación del texto depende del caso de los elementos asociados en la lista o tabla para determinar el criterio de ordenación real (ascendente o descendente).

La forma más común de ordenación usa texto que está todo en minúsculas, todo en mayúsculas o en capitalización de título. Si hay una combinación de estos casos, el criterio de ordenación ascendente es el siguiente:

1. Cualquier texto de la columna de lista o tabla que comience por una letra mayúscula es primero.
2. Si hay texto coincidente, pero uno usa mayúsculas y minúsculas y otro está en todas mayúsculas, la versión en mayúsculas se coloca primero.
3. A continuación, se ordenan las minúsculas.

Para el orden descendente, los pasos enumerados anteriormente se procesan a la inversa.

Por ejemplo, el siguiente ejemplo tiene una combinación de todo en minúsculas, todo en mayúsculas, y con mayúsculas iniciales que se van a ordenar en orden ascendente.

Power Query M

```
let
    Source = { "Alpha", "Beta", "Zulu", "ALPHA", "gamma", "alpha",
               "beta", "Gamma", "Sierra", "zulu", "GAMMA", "ZULU" },
    SortedList = List.Sort(Source, Order.Ascending)
in
    SortedList
```

Este código genera la siguiente salida:

	List
1	ALPHA
2	Alpha
3	Beta
4	GAMMA
5	Gamma
6	Sierra
7	ZULU
8	Zulu
9	alpha
10	beta
11	gamma
12	zulu

Aunque no es habitual, es posible que tenga una combinación de letras mayúsculas y minúsculas en texto para ordenar. El criterio de ordenación ascendente en este caso es:

1. Cualquier texto de la columna de lista o tabla que comience por una letra mayúscula es primero.
2. Si hay texto coincidente, a continuación se selecciona el texto con el mayor número de letras mayúsculas a la izquierda.
3. A continuación, se ordenan las minúsculas, priorizando primero la disposición del número máximo de letras mayúsculas a la derecha.

En cualquier caso, puede ser más conveniente convertir el texto en un caso coherente antes de ordenar.

Normalización de Power BI Desktop

Power Query M es sensible a las mayúsculas y diferencia entre distintas capitalizaciones del mismo texto. Por ejemplo, "Foo", "foo" y "FOO" se tratan como diferentes. Sin embargo, cuando los datos se cargan en Power BI Desktop, los valores de texto se normalizan, lo que significa que Power BI Desktop los trata como el mismo valor independientemente de su mayúscula. Por lo tanto, si necesita transformar los datos al tiempo que mantiene la distinción entre mayúsculas y minúsculas en los datos, debe controlar la transformación de datos en Power Query antes de cargar los datos en Power BI Desktop.

Por ejemplo, en la tabla siguiente de Power Query se muestran casos diferentes en cada fila de la tabla.

	Column1
1	Foo
2	TOO
3	foo
4	tOo
5	FOO
6	Too
7	fOo
8	too

Cuando esta tabla se carga en Power BI Desktop, los valores de texto se normalizan, lo que da lugar a la tabla siguiente.

Clipboard	
	Column1
	Foo
	TOO
	Foo
	TOO
	Foo
	TOO
	Foo
	TOO

La primera aparición de "foo" y la primera aparición de "too" determinan cómo se tratarán "foo" y "too" en el resto de las filas de la tabla de Power BI Desktop. En este ejemplo, todas las instancias de "foo" se normalizan con el valor "Foo" y todas las instancias de "too" se normalizan con el valor "TOO".

Referencia de la función M de Power Query

La referencia de la función M de Power Query incluye artículos para cada una de las más de 700 funciones. Estos artículos de referencia se generan automáticamente a partir de la ayuda en el producto. Para más información sobre las funciones y cómo funcionan en una expresión, vaya a Descripción de las [funciones de Power Query M](#).

Funciones por categoría

- [Acceso a funciones de datos](#)
- [Funciones binarias](#)
- [Funciones del combinador](#)
- [Funciones del comparador](#)
- [Funciones de fecha](#)
- [Funciones DateTime](#)
- [Funciones DateTimeZone](#)
- [Funciones de duración](#)
- [Control de errores](#)
- [Funciones de expresiones](#)
- [Valores de función](#)
- [Funciones de lista](#)
- [Funciones de líneas](#)
- [funciones lógicas](#)
- [Funciones numéricas](#)
- [Funciones de registro](#)
- [Funciones de sustituto](#)
- [Funciones divisoras](#)
- [Funciones de tabla](#)
- [Funciones de texto](#)
- [Funciones de hora](#)
- [Funciones de tipo](#)
- [Funciones de URI](#)
- [Funciones de valor](#)

Descripción de las funciones de M de Power Query

En el lenguaje de fórmulas M de Power Query, una *función* es una asignación de un conjunto de valores de entrada a un único valor de salida. Para escribir una función, primero se indican los parámetros de esa función y, después, se proporciona una expresión para calcular el resultado de la función. El cuerpo de la función va detrás del signo igual (= >). Opcionalmente, se puede incluir información de tipo en los parámetros y en el valor devuelto de la función. Una función se define y se invoca en el cuerpo de una instrucción **let**. Los parámetros y/o el valor devuelto pueden ser implícitos o explícitos. Los parámetros y/o valor devuelto implícitos son de tipo **any**. Escribir **any** es similar a un tipo de objeto en otros lenguajes. Todos los tipos de M se derivan del tipo **any**.

Una función es un valor, como un número o un valor de texto, y se puede incluir en línea, como cualquier otra expresión. En el ejemplo siguiente se muestra una función que es el valor de una **variable Add**, que luego se invoca o se ejecuta, desde otras variables. Cuando se invoca una función, se especifica un conjunto de valores que sustituye lógicamente por el conjunto necesario de valores de entrada dentro de la expresión del cuerpo de la función.

Ejemplo: parámetros explícitos y valor devuelto

Power Query M

```
let
    AddOne = (x as number) as number => x + 1,
    //additional expression steps
    CalcAddOne = AddOne(5)
in
    CalcAddOne
```

Ejemplo: parámetros implícitos y valor devuelto

Power Query M

```
let
    Add = (x, y) => x + y,
    AddResults =
        [
            OnePlusOne = Add(1, 1),      // equals 2
            OnePlusTwo = Add(1, 2)      // equals 3
        ]
in
    AddResults
```

Hallar el primer elemento de una lista mayor que 5 o, en caso contrario, NULL

Power Query M

```
let
    FirstGreaterThan5 = (list) =>
        let
            GreaterThan5 = List.Select(list, (n) => n > 5),
            First = List.First(GreaterThan5)
        in
            First,
    Results =
    [
        Found      = FirstGreaterThan5({3,7,9}), // equals 7
        NotFound   = FirstGreaterThan5({1,3,4})  // equals null
    ]
in
    Results
```

Las funciones se pueden usar de forma recursiva. Para hacer referencia de forma recursiva a una función, anteponga @ al identificador.

Power Query M

```
let
    fact = (num) => if num = 0 then 1 else num * @fact (num-1)
in
    fact(5) // equals 120
```

Palabra clave each

La palabra clave **each** se usa para crear funciones simples fácilmente. `each ...` es azúcar sintáctica para una firma de función que toma el `_` parámetro `(_) => ...`.

La **palabra clave each** es útil cuando se combina con el operador de búsqueda, que se aplica de forma predeterminada a `_`. Por ejemplo, `each [CustomerID]` es igual `each _[CustomerID]` que , que es igual `(_) => _[CustomerID]` que .

Ejemplo: Uso de cada uno de los filtros de fila de tabla

Power Query M

```
Table.SelectRows(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"] ,
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"] ,
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    each [CustomerID] = 2
)[Name]
```

```
// equals "Jim"
```

Last updated on 14/02/2025

Acceso a las funciones de datos

Estas funciones acceden a los datos y devuelven valores de tabla. La mayoría de estas funciones devuelven un valor de tabla denominado **tabla de navegación**. La interfaz de usuario de Power Query emplea estas tablas de navegación principalmente para proporcionar una experiencia de navegación mediante los conjuntos de datos jerárquicos potencialmente voluminosos que se devuelven.

 Expandir tabla

Nombre	Descripción
AccessControlEntry.ConditionTolIdentities	Devuelve una lista de las identidades que la condición aceptará.
Access.Database	Devuelve una representación estructural de una base de datos de Microsoft Access.
ActiveDirectory.Domains	Devuelve una lista de dominios de Active Directory en el mismo bosque que el dominio especificado o del dominio del equipo, si no se especifica ninguno.
AdobeAnalytics.Cubes	Devuelve los conjuntos de informes en Adobe Analytics.
AdoDotNet.DataSource	Devuelve la colección de esquemas de un origen de datos de ADO.NET.
AdoDotNet.Query	Devuelve el resultado de ejecutar una consulta nativa en un origen de datos ADO.NET.
AnalysisServices.Database	Devuelve una tabla de cubos multidimensionales o de modelos tabulares de la base de datos de Analysis Services.
AnalysisServices.Databases	Devuelve las bases de datos de Analysis Services de un host concreto.
AzureStorage.BlobContents	Devuelve el contenido del blob especificado de un almacén de Azure Storage.
AzureStorage.Blobs	Devuelve una tabla de navegación que contiene los contenedores que se encuentran en la cuenta especificada desde un almacén de Azure Storage. Cada fila refleja el nombre del contenedor y un vínculo a los blobs del contenedor.
AzureStorage.DataLake	Devuelve una tabla de navegación que contiene los documentos que se han encontrado en el contenedor especificado y en sus subcarpetas desde Azure Data Lake Storage.

Nombre	Descripción
AzureStorage.DataLakeContents	Devuelve el contenido del archivo especificado desde un sistema de archivos de Azure Data Lake Storage.
AzureStorage.Tables	Devuelve una tabla de navegación que contiene las tablas que se encuentran en la cuenta especificada desde un almacén de Azure Storage. Cada fila contiene un vínculo a la tabla de Azure.
Cdm.Contents	Esta función no está disponible porque requiere .NET 4.5.
Csv.Document	Devuelve el contenido del documento CSV como una tabla mediante la codificación especificada.
Cube.AddAndExpandDimensionColumn	Combina la tabla de dimensiones especificada en el contexto de filtro del cubo y cambia la granularidad dimensional del contexto de filtro expandiendo el conjunto especificado de atributos de dimensión.
Cube.AddMeasureColumn	AAdds una columna al cubo que contiene los resultados de la medida aplicada en el contexto de fila de cada fila.
Cube.ApplyParameter	Devuelve un cubo después de aplicar un parámetro a él.
Cube.AttributeMemberId	Devuelve el identificador de miembro único del valor de la propiedad members.
Cube.AttributeMemberProperty	Devuelve una propiedad de un atributo de dimensión.
Cube.CollapseAndRemoveColumns	Cambia la granularidad dimensional del contexto de filtro del cubo al contraer los atributos asignados a las columnas especificadas.
Cube.Dimensions	Devuelve una tabla que contiene el conjunto de dimensiones disponibles.
Cube.DisplayFolders	Devuelve un árbol anidado de tablas que representan la jerarquía de carpetas para mostrar de los objetos (por ejemplo, dimensiones y medidas).
Cube.MeasureProperties	Devuelve una tabla que contiene el conjunto de propiedades de medida disponibles que se expanden en el cubo.
Cube.MeasureProperty	Devuelve una propiedad de una medida (propiedad cell).
Cube.Measures	Devuelve una tabla que contiene el conjunto de medidas disponibles.
Cube.Parameters	Devuelve una tabla que contiene el conjunto de parámetros que se pueden aplicar al cubo.

Nombre	Descripción
Cube.Properties	Devuelve una tabla que contiene el conjunto de propiedades disponibles para las dimensiones que se expanden en el cubo.
Cube.PropertyKey	Devuelve la clave de una propiedad .
Cube.ReplaceDimensions	Reemplaza el conjunto de dimensiones devueltas por Cube.Dimensions .
Cube.Transform	Aplica una lista de funciones de cubo.
DB2.Database	Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos DB2.
DeltaLake.Metadata	Dada una tabla de Delta Lake, devuelve las entradas de registro de esa tabla.
DeltaLake.Table	Devuelve el contenido de la tabla Delta Lake.
Essbase.Cubes	Devuelve los cubos de una instancia de Essbase agrupados por servidor de Essbase.
Excel.CurrentWorkbook	Devuelve el contenido del libro de Excel actual.
Excel.Workbook	Devuelve el contenido del libro de Excel.
Exchange.Contents	Devuelve una tabla de contenido de una cuenta de Microsoft Exchange.
FabricAI.Prompt	Devuelve el resultado de pasar la entrada especificada a un modelo de IA.
File.Contents	Devuelve el contenido del archivo especificado como binario.
Folder.Contents	Devuelve una tabla con las propiedades y el contenido de los archivos y carpetas que se encuentran en la carpeta especificada.
Folder.Files	Devuelve una tabla que contiene las propiedades y el contenido de los archivos encontrados en la carpeta y subcarpeta especificadas. Cada fila contiene las propiedades de la carpeta o el archivo, y un vínculo a su contenido.
GoogleAnalytics.Accounts	Devuelve las cuentas de Google Analytics de la credencial actual.
Hdfs.Contents	Devuelve una tabla que contiene las propiedades y el contenido de los archivos y carpetas que se encuentran en la carpeta especificada desde un sistema de archivos de Hadoop. Cada fila contiene las propiedades de la carpeta o el archivo, y un vínculo a su contenido.

Nombre	Descripción
Hdfs.Files	Devuelve una tabla que contiene las propiedades y el contenido de los archivos que se encuentran en la carpeta y subcarpeta especificadas de un sistema de archivos de Hadoop. Cada fila contiene las propiedades del archivo y un vínculo a su contenido.
HdInsight.Containers	Devuelve una tabla de navegación que contiene los contenedores que se encuentran en la cuenta especificada desde un almacén de Azure Storage. Cada fila tiene el nombre de contenedor y la tabla que contiene sus archivos.
HdInsight.Contents	Devuelve una tabla de navegación que contiene los contenedores que se encuentran en la cuenta especificada desde un almacén de Azure Storage. Cada fila tiene el nombre de contenedor y la tabla que contiene sus archivos.
HdInsight.Files	Devuelve una tabla que contiene las propiedades y el contenido de los blobs que se encuentran en el contenedor especificado desde un almacén de Azure Storage. Cada fila contiene las propiedades de archivo/carpeta y un vínculo a su contenido.
Html.Table	Devuelve una tabla que contiene los resultados de ejecutar los selectores CSS especificados en el HTML proporcionado.
Identity.From	Crea una identidad.
Identity.IsMemberOf	Determina si una identidad es miembro de una colección de identidades.
IdentityProvider.Default	El proveedor de identidades predeterminado del host actual.
Informix.Database	Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos de Informix.
Json.Document	Devuelve el contenido del documento JSON. El contenido se puede pasar directamente a la función como texto o puede ser el valor binario devuelto por una función como File.Contents .
Json.FromValue	Genera una representación JSON de un valor, value, determinado con la codificación de texto especificada mediante codificación.
MySQL.Database	Devuelve una tabla de tablas, vistas y funciones escalares almacenadas de SQL disponibles en una base de datos MySQL.
OData.Feed	Devuelve una tabla de fuentes de OData ofrecidas por un

Nombre	Descripción
	servicio OData.
Odbc.DataSource	Devuelve una tabla de tablas y vistas SQL del origen de datos ODBC.
Odbc.InferOptions	Devuelve el resultado de intentar inferir las capacidades de SQL de un controlador ODBC.
Odbc.Query	Devuelve el resultado de ejecutar una consulta nativa en un origen de datos ODBC.
OleDb.DataSource	Devuelve una tabla de tablas y vistas SQL del origen de datos OLE DB.
OleDb.Query	Devuelve el resultado de ejecutar una consulta nativa en un origen de datos de OLE DB.
Oracle.Database	Devuelve una tabla de tablas y vistas SQL de la base de datos de Oracle.
Pdf.Tables	Devuelve cualquier tabla encontrada en un archivo PDF.
PostgreSQL.Database	Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos PostgreSQL.
RData.FromBinary	Devuelve un registro de tramas de datos del archivo RData.
Salesforce.Data	Devuelve los objetos de la cuenta de Salesforce.
Salesforce.Reports	Devuelve los informes de la cuenta de Salesforce.
SapBusinessWarehouse.Cubes	Devuelve los elementos InfoCube y las consultas en un sistema SAP Business Warehouse agrupados por InfoArea.
SapHana.Database	Muestra los paquetes de una base de datos SAP HANA.
SharePoint.Contents	Devuelve una tabla que contiene contenido de un sitio de SharePoint. Cada fila contiene las propiedades de la carpeta o el archivo, y un vínculo a su contenido.
SharePoint.Files	Devuelve una tabla que contiene documentos de un sitio de SharePoint. Cada fila contiene las propiedades de la carpeta o el archivo, y un vínculo a su contenido.
SharePoint.Tables	Devuelve una tabla que contiene contenido de una lista de SharePoint.
Soda.Feed	Devuelve una tabla a partir del contenido de la dirección URL especificada cuyo formato se corresponde con la API SODA 2.0. La dirección URL debe apuntar a un origen válido compatible con SODA que termine en una extensión .csv.

Nombre	Descripción
Sql.Database	Devuelve una tabla de tablas sql, vistas y funciones almacenadas de la base de datos de SQL Server.
Sql.Databases	Devuelve una tabla de bases de datos en un servidor SQL Server.
Sybase.Database	Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos de Sybase.
Teradata.Database	Devuelve una tabla de tablas y vistas SQL de la base de datos de Teradata.
WebAction.Request	Crea una acción que, al ejecutarse, devolverá los resultados de realizar una solicitud de método en la dirección URL usando HTTP como valor binario.
Web.BrowserContents	Devuelve el código HTML de la dirección URL especificada, tal y como lo ve un explorador web.
Web.Contents	Devuelve el contenido descargado de la dirección URL como binario.
Web.Headers	Devuelve los encabezados HTTP descargados de la dirección URL como un valor de registro.
Web.Page	Devuelve el contenido del documento HTML dividido en sus estructuras constitutivas, así como una representación de todo el documento y su texto después de quitar las etiquetas.
Xml.Document	Devuelve el contenido del documento XML como una tabla jerárquica (lista de registros).
Xml.Tables	Devuelve el contenido de un documento XML como una colección anidada de tablas sin formato.

Last updated on 16/07/2025

AccessControlEntry.ConditionToIdentities

16/09/2025

Sintaxis

```
AccessControlEntry.ConditionToIdentities(identityProvider as function, condition  
as function) as list
```

Acerca de

Con el `identityProvider` especificado, convierte el `condition` en la lista de identidades para las que `condition` devolvería `true` en todos los contextos de autorización con `identityProvider` como proveedor de identidades. Se produce un error si no es posible convertir `condition` en una lista de identidades, por ejemplo, si `condition` consulta atributos distintos de las identidades de usuario o grupo para tomar una decisión.

Tenga en cuenta que la lista de identidades representa las identidades tal como aparecen en `condition` y no se realiza ninguna normalización (como la expansión de grupos) en ellas.

Access.Database

16/09/2025

Sintaxis

```
Access.Database(database as binary, optional options as nullable record) as table
```

Acerca de

Devuelve una representación estructural de una base de datos de Access, `database`. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es false).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.

El parámetro record se especifica como [option1 = value1, option2 = value2...], por ejemplo.

ActiveDirectory.Domains

16/09/2025

Sintaxis

```
ActiveDirectory.Domains(optional forestRootDomainName as nullable text) as table
```

Acerca de

Devuelve una lista de dominios de Active Directory en el mismo bosque que el dominio especificado o el dominio del equipo actual si no se especifica ninguno.

AdobeAnalytics.Cubes

16/09/2025

Sintaxis

```
AdobeAnalytics.Cubes(optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de paquetes multidimensionales de Adobe Analytics. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).
- `MaxRetryCount`: número de reintentos a realizar al sondear para obtener el resultado de la consulta. El valor predeterminado es 120.
- `RetryInterval`: la duración del tiempo entre los reintentos. El valor predeterminado es 1 segundo.
- `Implementation`: especifica la versión de la API de Adobe Analytics. Los valores válidos son: "2.0". El valor predeterminado usa la versión 1.4 de la API.

AdoDotNet.DataSource

Syntax

```
AdoDotNet.DataSource(  
    providerName as text,  
    connectionString as any,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve la colección de esquemas del origen de datos ADO.NET con el nombre `providerName` del proveedor y la cadena `connectionString` de conexión. `connectionString` puede ser texto o un registro de pares de valor de propiedad. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de que se cancele. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.
- `TypeMap`

Last updated on 22/11/2025

AdoDotNet.Query

Syntax

```
AdoDotNet.Query(  
    providerName as text,  
    connectionString as any,  
    query as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve el resultado de ejecutarse `query` con la cadena `connectionString` de conexión mediante el proveedor `providerName` de ADO.NET . `connectionString` puede ser texto o un registro de pares de valor de propiedad. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.

Last updated on 22/11/2025

AnalysisServices.Database

Syntax

```
AnalysisServices.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de cubos multidimensionales o modelos tabulares de la base de datos `database` de Analysis Services en el servidor `server`. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `Query`: una consulta MDX nativa que se usa para recuperar datos.
- `TypedMeasureColumns`: valor lógico que indica si los tipos especificados en el modelo tabular o multidimensional se usarán para los tipos de las columnas de medida agregadas. Cuando se establece en false, el tipo "number" se usará para todas las columnas de medida. El valor predeterminado de esta opción es false.
- `Culture`: nombre de referencia cultural que especifica la referencia cultural de los datos. Esto corresponde a la propiedad de cadena de conexión "Identificador de configuración regional".
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de que se cancele. El valor predeterminado depende del controlador.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `SubQueries`: número (0, 1 o 2) que establece el valor de la propiedad "SubQueries" en la cadena de conexión. Esto controla el comportamiento de los miembros calculados en subselecciones o subcubos. (El valor predeterminado es 2).
- `Implementation`

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
-

AnalysisServices.Databases

Syntax

```
AnalysisServices.Databases(server as text, optional options as nullable record) as table
```

Acerca de

Devuelve bases de datos en una instancia de Analysis Services, `server`. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `TypedMeasureColumns`: valor lógico que indica si los tipos especificados en el modelo tabular o multidimensional se usarán para los tipos de las columnas de medida agregadas. Cuando se establece en false, el tipo "number" se usará para todas las columnas de medida. El valor predeterminado de esta opción es false.
- `Culture`: nombre de referencia cultural que especifica la referencia cultural de los datos. Esto corresponde a la propiedad de cadena de conexión "Identificador de configuración regional".
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de que se cancele. El valor predeterminado depende del controlador.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `SubQueries`: número (0, 1 o 2) que establece el valor de la propiedad "SubQueries" en la cadena de conexión. Esto controla el comportamiento de los miembros calculados en subselecciones o subcubos. (El valor predeterminado es 2).
- `Implementation`

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

AzureStorage.BlobContents

Syntax

```
AzureStorage.BlobContents(url as text, optional options as nullable record) as binary
```

Acerca de

Devuelve el contenido del blob en la dirección URL, **url**, desde un almacén de Azure Storage. **options** puede especificarse para controlar las siguientes opciones:

- **BlockSize**: número de bytes que se van a leer antes de esperar al consumidor de datos. El valor predeterminado es 4 MB.
- **RequestSize**: número de bytes que se van a intentar leer en una única solicitud HTTP al servidor. El valor predeterminado es 4 MB.
- **ConcurrentRequests**: la opción ConcurrentRequests admite la descarga más rápida de datos especificando el número de solicitudes que se van a realizar en paralelo, a costa del uso de memoria. La memoria necesaria es (ConcurrentRequest * RequestSize). El valor predeterminado es 16.

Last updated on 22/11/2025

AzureStorage.Blobs

Syntax

```
AzureStorage.Blobs(account as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de navegación que contiene una fila para cada contenedor que se encuentra en la dirección URL de la cuenta, `account`, desde un almacén de Azure Storage. Cada fila contiene un vínculo a los blobs de contenedor. `options` puede especificarse para controlar las siguientes opciones:

- `BlockSize`: número de bytes que se van a leer antes de esperar al consumidor de datos. El valor predeterminado es 4 MB.
- `RequestSize`: número de bytes que se van a intentar leer en una única solicitud HTTP al servidor. El valor predeterminado es 4 MB.
- `ConcurrentRequests`: la opción `ConcurrentRequests` admite la descarga más rápida de datos especificando el número de solicitudes que se van a realizar en paralelo, a costa del uso de memoria. La memoria necesaria es (`ConcurrentRequest * RequestSize`). El valor predeterminado es 16.

Last updated on 22/11/2025

AzureStorage.DataLake

16/09/2025

Sintaxis

```
AzureStorage.DataLake(endpoint as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de navegación que contiene los documentos encontrados en el contenedor especificado y sus subcarpetas en la dirección URL de la cuenta, `endpoint`, desde un sistema de archivos de Azure Data Lake Storage. Se puede especificar `options` para controlar las siguientes opciones:

- `BlockSize`: número de bytes que se van a leer antes de esperar en el consumidor de datos. El valor predeterminado es 4 MB.
- `RequestSize`: el número de bytes que se van a intentar leer en una única solicitud HTTP al servidor. El valor predeterminado es 4 MB.
- `ConcurrentRequests`: la opción `ConcurrentRequests` admite la descarga más rápida de datos especificando el número de solicitudes que se van a realizar en paralelo, a costa del uso de memoria. La memoria necesaria es $(\text{ConcurrentRequest} * \text{RequestSize})$. El valor predeterminado es 16.
- `HierarchicalNavigation`: un valor lógico (true/false) que controla si los archivos se devuelven en una vista de directorio similar a un árbol o en una lista plana. El valor predeterminado es false.

AzureStorage.DataLakeContents

Syntax

```
AzureStorage.DataLakeContents(url as text, optional options as nullable record) as binary
```

Acerca de

Devuelve el contenido del archivo en la dirección URL, `url`, de un sistema de archivos de Azure Data Lake Storage. `options` puede especificarse para controlar las siguientes opciones:

- `BlockSize`: número de bytes que se van a leer antes de esperar al consumidor de datos. El valor predeterminado es 4 MB.
- `RequestSize`: número de bytes que se van a intentar leer en una única solicitud HTTP al servidor. El valor predeterminado es 4 MB.
- `ConcurrentRequests`: la opción `ConcurrentRequests` admite la descarga más rápida de datos especificando el número de solicitudes que se van a realizar en paralelo, a costa del uso de memoria. La memoria necesaria es (`ConcurrentRequest * RequestSize`). El valor predeterminado es 16.

Last updated on 22/11/2025

AzureStorage.Tables

16/09/2025

Sintaxis

```
AzureStorage.Tables(account as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de navegación que contiene una fila para cada tabla que se encuentra en la URL de la cuenta, `account`, desde una bóveda de almacenamiento de Azure. Cada fila contiene un vínculo a la tabla de Azure. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `Timeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar la solicitud al servidor. El valor predeterminado es específico del origen.

Cdm.Contents

16/09/2025

Sintaxis

```
Cdm.Contents(table as table) as table
```

Acerca de

Esta función no está disponible en el contexto actual.

Csv.Document

Sintaxis

```
Csv.Document(  
    source as any,  
    optional columns as any,  
    optional delimiter as any,  
    optional extraValues as nullable number,  
    optional encoding as nullable number  
) as table
```

Acercas de

Devuelve el contenido del documento CSV como una tabla.

- `columns` puede ser null, el número de columnas, una lista de nombres de columna, un tipo de tabla o un registro de opciones
- `delimiter` puede ser un solo carácter, una lista de caracteres o el valor `" "`, que indica que las filas deben dividirse con caracteres de espacio en blanco consecutivos.
Predeterminado: `","`.
- Para los valores admitidos de [ExtraValues.Type](#), refiérase a `extraValues`.
- `encoding` especifica el [tipo de codificación de texto](#).

Si se especifica un registro para `columns` (y `delimiter`, `extraValues` y `encoding` son null), se pueden proporcionar los siguientes campos de registro:

- `Delimiter`: un delimitador de columna de caracteres único. Predeterminado: `","`.
- `Columns`: puede ser null, el número de columnas, una lista de nombres de columna o un tipo de tabla. Si el número de columnas es menor que el número de la entrada, las columnas extra se omitirán. Si el número de columnas es mayor que el número de la entrada, las columnas extra serán null. Si no se especifica, el número de columnas se determinará en función de lo que refleje la entrada.
- `Encoding`: codificación de texto del archivo. Valor predeterminado: 65001 (UTF-8).
- `CsvStyle`: especifica cómo se tratan las comillas.
 - [CsvStyle QuoteAfterDelimiter](#) (valor predeterminado): las comillas de un campo solo son significativas inmediatamente después del delimitador.

- [CsvStyle QuoteAlways](#): las comillas de un campo siempre son significativas, independientemente de dónde aparezcan.
- `QuoteStyle`: especifica cómo se tratan los saltos de línea entre comillas.
 - [QuoteStyle.Csv](#) (valor predeterminado): los saltos de línea entre comillas se tratan como parte de los datos, no como el final de la fila actual.
 - [QuoteStyle.None](#): todos los saltos de línea se tratan como el final de la fila actual, incluso cuando se producen dentro de un valor entre comillas.
- `IncludeByteOrderMark`: valor lógico que indica si se debe incluir una marca de orden de bytes (BOM) al principio de la salida CSV. Cuando se establece en `true`, se escribe la BOM (por ejemplo, UTF-8 BOM: `0xEF 0xBB 0xBF`); cuando se establece en `false`, no se incluye ninguna boM. Esta opción solo es aplicable en escenarios de salida. El valor predeterminado es `false`.
- `ExtraValues`: consulte [ExtraValues.Type](#) los valores admitidos de `ExtraValues`.

Ejemplo 1

Procesar texto CSV con encabezados de columna.

Uso

Power Query M

```
let
    csv = Text.Combine({"OrderID,Item", "1,Fishing rod", "2,1 lb. worms"}, "#(cr)#(lf)")
in
    Table.PromoteHeaders(Csv.Document(csv))
```

Salida

Power Query M

```
Table.FromRecords({
    [OrderID = "1", Item = "Fishing rod"],
    [OrderID = "2", Item = "1 lb. worms"]
})
```

Ejemplo 2

Procesar texto CSV con varios caracteres delimitadores. En este ejemplo, el tercer parámetro especifica el patrón `#|#` delimitador que se va a usar en lugar del valor predeterminado.

Uso

Power Query M

```
let
    csv = Text.Combine({"OrderID#|#Color", "1#|#Red", "2#|#Blue"}, "#(cr)#(lf)")
in
    Table.PromoteHeaders(Csv.Document(csv, null, "#|#"))
```

Salida

Power Query M

```
Table.FromRecords({
    [OrderID = "1", Color = "Red"],
    [OrderID = "2", Color = "Blue"]
})
```

Last updated on 04/04/2026

Cube.AddAndExpandDimensionColumn

Sintaxis

```
Cube.AddAndExpandDimensionColumn(  
    cube as table,  
    dimensionSelector as any,  
    attributeNames as list,  
    optional newColumnNames as any  
) as table
```

Acerca de

Combina la tabla de dimensiones especificada, `dimensionSelector`, en el contexto de filtro de `cube` y cambia la granularidad dimensional expandiendo el conjunto especificado, `attributeNames`, de atributos de dimensión. Los atributos de dimensión se agregan a la vista tabular con columnas denominadas `newColumnNames` o `attributeNames` si no se especifica.

Last updated on 04/04/2026

Cube.AddMeasureColumn

16/09/2025

Sintaxis

```
Cube.AddMeasureColumn(  
    cube as table,  
    column as text,  
    measureSelector as any  
) as table
```

Acerca de

Agrega una columna con el nombre `column` al `cube` que contiene los resultados de la medida `measureSelector` aplicados en el contexto de fila de cada fila. La aplicación de la medición se ve afectada por los cambios en la granularidad y segmentación de las dimensiones. Los valores de medida se ajustarán después de realizar determinadas operaciones de cubo.

Cube.ApplyParameter

16/09/2025

Sintaxis

```
Cube.ApplyParameter(  
    cube as table,  
    parameter as any,  
    optional arguments as nullable list  
) as table
```

Acerca de

Devuelve un cubo después de aplicar `parameter` con `arguments` a `cube`.

Cube.AttributeMemberId

Syntax

```
Cube.AttributeMemberId(attribute as any) as any
```

Acerca de

Devuelve el identificador de miembro único de un valor de propiedad miembro. `attribute`.

Devuelve null para cualquier otro valor.

Last updated on 22/11/2025

Cube.AttributeMemberProperty

16/09/2025

Sintaxis

```
Cube.AttributeMemberProperty(attribute as any, propertyName as text) as any
```

Acercas de

Devuelve la propiedad `propertyName` del atributo de dimensión `attribute`.

Cube.CollapseAndRemoveColumns

16/09/2025

Sintaxis

```
Cube.CollapseAndRemoveColumns(cube as table, columnNames as list) as table
```

Acerca de

Cambia la granularidad dimensional del contexto de filtro para el `cube`, colapsando los atributos asignados a las columnas especificadas `columnNames`. Las columnas también se quitan de la vista tabular del cubo.

Cube.Dimensions

16/09/2025

Sintaxis

```
Cube.Dimensions(cube as table) as table
```

Acerca de

Devuelve una tabla que contiene el conjunto de dimensiones disponibles dentro de la `cube`. Cada dimensión es una tabla que contiene un conjunto de atributos de dimensión y cada atributo de dimensión se representa como una columna de la tabla de dimensiones. Las dimensiones se pueden expandir en el cubo mediante [Cube.AddAndExpandDimensionColumn](#).

Cube.DisplayFolders

16/09/2025

Sintaxis

```
Cube.DisplayFolders(cube as table) as table
```

Acerca de

Devuelve un árbol anidado de tablas que representa la jerarquía de carpetas de visualización de los objetos (por ejemplo, dimensiones y medidas) disponibles para su uso en la `cube`.

Cube.MeasureProperties

16/09/2025

Sintaxis

```
Cube.MeasureProperties(cube as table) as table
```

Acerca de

Devuelve una tabla que contiene el conjunto de propiedades disponibles para las medidas que se expanden en el cubo.

Cube.MeasureProperty

Syntax

```
Cube.MeasureProperty(measure as any, propertyName as text) as any
```

Acerca de

Devuelve la propiedad `propertyName` de medida `measure`.

Last updated on 22/11/2025

Cube.Measures

16/09/2025

Sintaxis

```
Cube.Measures(cube as any) as table
```

Acerca de

Devuelve una tabla que contiene el conjunto de medidas disponibles en el `cube`. Cada medida se representa como una función. Las medidas se pueden aplicar al cubo mediante `Cube.AddMeasureColumn`.

Cube.Parameters

16/09/2025

Sintaxis

```
Cube.Parameters(cube as table) as table
```

Acerca de

Devuelve una tabla que contiene el conjunto de parámetros que se pueden aplicar a `cube`. Cada parámetro es una función que se puede invocar para obtener `cube` con el parámetro y sus argumentos aplicados.

Cube.Properties

16/09/2025

Sintaxis

```
Cube.Properties(cube as table) as table
```

Acerca de

Devuelve una tabla que contiene el conjunto de propiedades disponibles para las dimensiones que se expanden en el cubo.

Cube.PropertyKey

Syntax

```
Cube.PropertyKey(property as any) as any
```

Acerca de

Devuelve la clave de la propiedad `property`.

Last updated on 22/11/2025

Cube.ReplaceDimensions

16/09/2025

Sintaxis

```
Cube.ReplaceDimensions(cube as table, dimensions as table) as table
```

Acercas de

Reemplaza el conjunto de dimensiones devueltas por `Cube.Dimensions`. Por ejemplo, esta función se puede usar para agregar una columna de ID a un atributo de dimensión, de modo que el origen de datos pueda agrupar por el ID en lugar del valor mostrado.

Cube.Transform

16/09/2025

Sintaxis

```
Cube.Transform(cube as table, transforms as list) as table
```

Acerca de

Aplica las funciones de cubo de la lista, `transforms`, en `cube`.

DB2.Database

Syntax

```
DB2.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos Db2 en el servidor `server` de la instancia de base de datos denominada `database`. El puerto puede especificarse opcionalmente con el servidor, separados por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de que se cancele. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).
- `Implementation`: especifica la implementación interna del proveedor de base de datos que se va a usar. Los valores válidos son: "IBM" y "Microsoft".
- `BinaryCodePage`: número del CCSID (identificador de juego de caracteres codificado) para decodificar datos binarios db2 FOR BIT en cadenas de caracteres. Se aplica a Implementación = "Microsoft". Establezca 0 para deshabilitar la conversión (valor predeterminado). Establezca 1 para convertir en función de la codificación de base de datos. Establezca otro número CCSID para convertir en codificación de aplicación.
- `PackageCollection`: especifica un valor de cadena para la colección de paquetes (el valor predeterminado es "NULLID") para permitir el uso de paquetes compartidos necesarios

para procesar instrucciones SQL. Se aplica a Implementación = "Microsoft".

- `UseDb2ConnectGateway`: especifica si la conexión se realiza a través de una puerta de enlace de Db2 Connect. Se aplica a Implementación = "Microsoft".

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Last updated on 22/11/2025

DeltaLake.Metadata

16/09/2025

Sintaxis

```
DeltaLake.Metadata(table as table) as table
```

Acerca de

Dada una tabla de Delta Lake, devuelve las entradas de registro de esa tabla.

DeltaLake.Table

16/09/2025

Sintaxis

```
DeltaLake.Table(directory as table, optional options as nullable record) as any
```

Acerca de

Devuelve el contenido de la tabla Delta Lake.

Essbase.Cubes

16/09/2025

Sintaxis

```
Essbase.Cubes(url as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de cubos agrupados por el servidor de Essbase desde una instancia de Essbase en el servidor APS **url**. Se puede especificar un parámetro de registro opcional, **options**, para controlar las siguientes opciones:

- **CommandTimeout**: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.

Excel.CurrentWorkbook

Syntax

```
Excel.CurrentWorkbook() as table
```

Acerca de

Devuelve el contenido del libro de Excel actual. Devuelve tablas, rangos con nombre y matrices dinámicas. A diferencia de [Excel.Workbook](#), no devuelve hojas.

Last updated on 22/11/2025

Excel.Workbook

Syntax

```
Excel.Workbook(  
    workbook as binary,  
    optional useHeaders as any,  
    optional delayTypes as nullable logical  
) as table
```

Acerca de

Devuelve el contenido del libro de Excel.

- `useHeaders` puede ser NULL, un valor lógico (true/false) que indica si la primera fila de cada tabla devuelta debe tratarse como un encabezado o un registro de opciones. Valor predeterminado: false.
- `delayTypes` puede ser null o un valor lógico (true/false) que indica si las columnas de cada tabla devuelta deben dejarse sin tipo. Valor predeterminado: false.

Si se especifica un registro para `useHeaders` (y `delayTypes` es null), se pueden proporcionar los siguientes campos de registro:

- `UseHeaders`: puede ser null o un valor lógico (true/false) que indica si la primera fila de cada tabla devuelta debe tratarse como un encabezado. Valor predeterminado: false.
- `DelayTypes`: puede ser null o un valor lógico (true/false) que indica si las columnas de cada tabla devuelta deben dejarse sin tipo. Valor predeterminado: false.
- `InferSheetDimensions`: puede ser null o un valor lógico (true/false) que indica si el área de una hoja de cálculo que contiene datos debe deducirse leyendo la propia hoja de cálculo, en lugar de leer los metadatos de dimensiones del archivo. Esto puede ser útil en los casos en los que los metadatos de dimensiones son incorrectos. Tenga en cuenta que esta opción solo se admite para archivos de Excel open XML, no para archivos de Excel heredados. Valor predeterminado: false.

ⓘ Nota:

El `useHeaders` parámetro o el `UseHeaders` campo de registro convierte números y fechas en texto mediante la referencia cultural actual y, por tanto, se comporta de forma diferente cuando se ejecuta en entornos con diferentes referencias culturales del sistema operativo establecidas. Se recomienda usar [Table.PromoteHeaders](#) en su lugar. Por

ejemplo, en lugar de usar `Excel.Workbook(File.Contents("C:\myfile.xlsx", true, true))` o `Excel.Workbook(File.Contents("C:\myfile.xlsx", [UseHeaders = true], null))`, use `Table.PromoteHeaders(Excel.Workbook(File.Contents("C:\myfile.xlsx", null, true), [PromoteAllScalars = true]))` en su lugar.

Ejemplo 1

Devuelve el contenido de Sheet1 de un libro de Excel.

Uso

Power Query M

```
Excel.Workbook(File.Contents("C:\Book1.xlsx"), null, true){[Item="Sheet1"]}[Data]
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = "ID", Column2 = "Name", Column3 = "Phone"],  
    [Column1 = 1, Column2 = "Bob", Column3 = "123-4567"],  
    [Column1 = 3, Column2 = "Pam", Column3 = "543-7890"],  
    [Column1 = 2, Column2 = "Jim", Column3 = "987-6543"]  
})
```

Last updated on 22/11/2025

Exchange.Contents

16/09/2025

Sintaxis

```
Exchange.Contents (optional mailboxAddress as nullable text) as table
```

Acerca de

Devuelve una tabla de contenido de la cuenta de Microsoft Exchange `mailboxAddress`. Si no se especifica `mailboxAddress`, se usará la cuenta predeterminada para la credencial.

FabricAI.Prompt

Syntax

```
FabricAI.Prompt(input as text, optional context as any) as text
```

Acerca de

Devuelve el resultado de pasar el especificado `input` a un modelo de IA. Se puede proporcionar un valor opcional `context` para especificar datos adicionales relevantes para la solicitud.

Ejemplo 1

Use un modelo de IA para clasificar una revisión del producto.

Uso

Power Query M

```
FabricAI.Prompt(  
    "Categorize the review as positive, negative, or neutral, and extract a single  
    key word in parentheses.",  
    [Review = "This is a great product. It only broke four times before we had to  
    return it!"]  
)
```

Salida

```
"Negative (broke)"
```

Last updated on 19/11/2025

File.Contents

16/09/2025

Sintaxis

```
File.Contents(path as text, optional options as nullable record) as binary
```

Acerca de

Devuelve el contenido del archivo, `path`, como binario. El parámetro `options` está pensado actualmente solo para uso interno.

Folder.Contents

16/09/2025

Sintaxis

```
Folder.Contents(path as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada carpeta y archivo que se encuentra en la carpeta `path`. Cada fila contiene propiedades de la carpeta o archivo y un vínculo a su contenido. El parámetro `options` está pensado actualmente solo para uso interno.

Folder.Files

Sintaxis

```
Folder.Files(path as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada archivo que se encuentra en la carpeta especificada y todas sus subcarpetas.

- **path**: la ruta de acceso a la carpeta desde la que desea recuperar los archivos. La ruta de acceso de carpeta proporcionada debe ser una ruta de acceso absoluta válida.
- **options**: este parámetro está pensado actualmente solo para uso interno.

Cada fila de la tabla devuelta contiene propiedades del archivo y un vínculo a su contenido.

Ejemplo

Devuelve una tabla que contiene todos los archivos que se encuentran en C:\test-examples\example-folder y todas sus subcarpetas.

Uso

Power Query M

```
Folder.Files("C:\test-examples\example-folder")
```

Salida

A table containing the files, their properties, and a link to their content.

Last updated on 26/03/2026

GoogleAnalytics.Accounts

16/09/2025

Sintaxis

```
GoogleAnalytics.Accounts(optional options as nullable record) as table
```

Acerca de

Devuelve las cuentas de Google Analytics a las que se puede acceder desde la credencial actual.

Hdfs.Contents

16/09/2025

Sintaxis

```
Hdfs.Contents(url as text) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada carpeta y archivo que se encuentra en la dirección URL de la carpeta, `url`, desde un sistema de archivos de Hadoop. Cada fila contiene propiedades de la carpeta o archivo y un vínculo a su contenido.

Hdfs.Files

16/09/2025

Sintaxis

```
Hdfs.Files(url as text) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada archivo que se encuentra en la dirección URL de la carpeta, `url` y subcarpetas de un sistema de archivos de Hadoop. Cada fila contiene propiedades del archivo y un vínculo a su contenido.

HdInsight.Containers

Syntax

```
HdInsight.Containers(account as text) as table
```

Acerca de

Devuelve una tabla de navegación que contiene una fila para cada contenedor que se encuentra en la dirección URL de la cuenta, `account`, desde un almacén de Azure Storage. Cada fila contiene un vínculo a los blobs de contenedor.

Last updated on 22/11/2025

HdInsight.Contents

Syntax

```
HdInsight.Contents(account as text) as table
```

Acerca de

Devuelve una tabla de navegación que contiene una fila para cada contenedor que se encuentra en la dirección URL de la cuenta, `account`, desde un almacén de Azure Storage. Cada fila contiene un vínculo a los blobs de contenedor.

Last updated on 22/11/2025

HdInsight.Files

16/09/2025

Sintaxis

```
HdInsight.Files(account as text, containerName as text) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada archivo de blob que se encuentra en la dirección URL del contenedor, `account`, desde un almacén de Azure Storage. Cada fila contiene propiedades del archivo y un vínculo a su contenido.

Html.Table

Syntax

```
Html.Table(  
    html as any,  
    columnNameSelectorPairs as list,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla que contiene los resultados de ejecutar los selectores CSS especificados en el proporcionado `html`. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `RowSelector`

Ejemplo 1

Devuelve una tabla de un valor de texto html de ejemplo.

Uso

Power Query M

```
Html.Table("<div class='\"name\"'>Jo</div><span>Manager</span>", {{"Name", ".name"}, {"Title", "span"}}, [RowSelector=".name"])\
```

Salida

```
#table({"Name", "Title"}, {{"Jo", "Manager"}})
```

Ejemplo 2

Extrae todas las referencias href de un valor de texto html de ejemplo.

Uso

Power Query M

```
Html.Table("<a href='\"'/test.html\"'>Test</a>", {"Link", "a", each [Attributes] [href]})
```

Salida

```
#table({"Link"}, {""/test.html"})
```

Last updated on 22/11/2025

Identity.From

16/09/2025

Sintaxis

```
Identity.From(identityProvider as function, value as any) as record
```

Acerca de

Crea una identidad.

Identity.IsMemberOf

16/09/2025

Sintaxis

```
Identity.IsMemberOf(identity as record, collection as record) as logical
```

Acerca de

Determina si una identidad es miembro de una colección de identidades.

IdentityProvider.Default

16/09/2025

Sintaxis

```
IdentityProvider.Default() as any
```

Acerca de

Proveedor de identidades predeterminado para el host actual.

Informix.Database

16/09/2025

Sintaxis

```
Informix.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos de Informix en el servidor `server` en la instancia de base de datos denominada `database`. El puerto puede especificarse opcionalmente junto al servidor, separado por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Json.Document

Sintaxis

```
Json.Document(jsonText as any, optional encoding as nullable number) as any
```

Acerca de

Devuelve el contenido del documento JSON.

- `jsonText`: el contenido del documento JSON. El valor de este parámetro puede ser texto o un valor binario devuelto por una función como [File.Contents](#).
- `encoding`: Un [TextEncoding.Type](#) que especifica la codificación utilizada en el documento JSON. Si `encoding` se omite, se usa UTF8.

Ejemplo 1

Devuelve el contenido del texto JSON especificado como un registro.

Uso

Power Query M

```
let
    Source = "{
        \"project\": \"\"Contosoware\"\",
        \"description\": \"\"A comprehensive initiative aimed at enhancing digital
presence.\"\",
        \"components\": [
            \"\"Website Development\"\",
            \"\"CRM Implementation\"\",
            \"\"Mobile Application\"\"
        ]
    }",
    jsonDocument = Json.Document(Source)
in
    jsonDocument
```

Salida

Power Query M


```
[
  project = "Contosoware",
  description = "A comprehensive initiative aimed at enhancing digital presence."
  components =
  {
    "Website Development",
    "CRM Implementation",
    "Mobile Application"
  }
]
```

Ejemplo 2

Devuelve el contenido de un archivo JSON local.

Uso

Power Query M

```
let
    Source = Json.Document(
        File.Contents("C:\test-examples\JSON\Contosoware.json")
    )
in
    Source
```

Salida

A record, list, or primitive value representing the JSON data contained in the file

Ejemplo 3

Devuelve el contenido de un archivo JSON con codificación UTF16 en línea.

Uso

Power Query M

```
let
    Source = Json.Document(
        Web.Contents("https://contoso.com/products/Contosoware.json"),
        TextEncoding.Utf16
    )
in
    Source
```

Salida

A record, list, or primitive value representing the JSON UTF16 data contained in the file

Last updated on 04/04/2026

Json.FromValue

Syntax

```
Json.FromValue(value as any, optional encoding as nullable number) as binary
```

Acerca de

Genera una representación JSON de un valor `value` determinado con una codificación de texto especificada por `encoding`. Si `encoding` se omite, se usa UTF8. Los valores se representan de la manera siguiente:

- Los valores NULL, text y lógicos se representan como los tipos JSON correspondientes.
- Los números se representan como números en JSON, excepto que `#infinity`, `-#infinity` y `#nan` se convierten en NULL.
- Las listas se representan como matrices JSON
- Los registros se representan como objetos JSON
- Las tablas se representan como una matriz de objetos
- Las fechas, las horas, las horas, las datetimezones y las duraciones se representan como texto ISO-8601
- Los valores binarios se representan como texto codificado en base 64
- Los tipos y funciones producen un error

Ejemplo

Convierta un valor complejo en JSON.

Uso

Power Query M

```
Text.FromBinary(Json.FromValue([A = {1, true, "3"}, B = #date(2012, 3, 25)]))
```

Salida

```
"{"A": [1, true, "3"], "B": "2012-03-25"}"
```

MySQL.Database

Syntax

```
MySQL.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas, vistas y funciones escalares almacenadas de SQL disponibles en una base de datos MySQL en el servidor `server` de la instancia de base de datos denominada `database`. El puerto puede especificarse opcionalmente con el servidor, separados por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `Encoding`: valor `textEncoding` que especifica el juego de caracteres usado para codificar todas las consultas enviadas al servidor (el valor predeterminado es NULL).
- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de que se cancele. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `TreatTinyAsBoolean`: un valor lógico (true/false) que determina si se fuerzan las columnas `tinyint` en el servidor como valores lógicos. El valor predeterminado es true.
- `OldGuids`: una columna lógica (true/false) que establece si las columnas `char(36)` (si son falsas) o `binary(16)` (si son true) se tratarán como GUID. El valor predeterminado es false.
- `ReturnSingleDatabase`: un valor lógico (true/false) que establece si se devuelven todas las tablas de todas las bases de datos (si es false) o para devolver tablas y vistas de la base de datos especificada (si es true). El valor predeterminado es false.

- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Last updated on 22/11/2025

OData.Feed

Syntax

```
OData.Feed(  
    serviceUri as text,  
    optional headers as nullable record,  
    optional options as any  
) as any
```

Acercas de

Devuelve una tabla de fuentes de OData ofrecidas por un servicio OData desde un URI `serviceUri`, encabezados `headers`. Valor booleano que especifica si se deben usar conexiones simultáneas o un parámetro de registro opcional, `options`, se puede especificar para controlar las siguientes opciones:

- `Query`: agregue mediante programación parámetros de consulta a la dirección URL sin tener que preocuparse por el escape.
- `Headers`: si se especifica este valor como registro, se proporcionarán encabezados adicionales a una solicitud HTTP.
- `ExcludedFromCacheKey`: al especificar este valor como una lista se excluirán estas claves de encabezado HTTP de formar parte del cálculo de los datos de almacenamiento en caché.
- `ApiKeyName`: si el sitio de destino tiene una noción de una clave de API, este parámetro se puede usar para especificar el nombre (no el valor) del parámetro de clave que se debe usar en la dirección URL. El valor de clave real se proporciona en la credencial.
- `Timeout`: si se especifica este valor como una duración, se cambiará el tiempo de espera de una solicitud HTTP. El valor predeterminado es 600 segundos.
- `EnableBatch`: un valor lógico (true/false) que establece si se permite la generación de una solicitud de OData \$batch si se supera MaxUriLength (el valor predeterminado es false).
- 'MaxUriLength: número que indica la longitud máxima de un URI permitido enviado a un servicio OData. Si se supera y EnableBatch es true, la solicitud se realizará en un punto de conexión de OData \$batch; de lo contrario, se producirá un error (el valor predeterminado es 2048).
- `Concurrent`: un valor lógico (true/false) cuando se establece en true, las solicitudes al servicio se realizarán simultáneamente. Cuando se establece en false, las solicitudes se realizarán secuencialmente. Cuando no se especifica, el valor se determinará mediante la anotación AsincrónicaRequestsSupported del servicio. Si el servicio no especifica si se admite AsynchronousRequestsSupported, las solicitudes se realizarán secuencialmente.

- **ODataVersion**: número (3 o 4) que especifica la versión del protocolo OData que se va a usar para este servicio OData. Cuando no se especifique, se solicitarán todas las versiones admitidas. El encabezado OData-Version devuelto por el servicio determinará la versión del servicio.
- **FunctionOverloads**: un valor lógico (true/false) cuando se establece en true, las sobrecargas de importación de funciones se mostrarán en el navegador como entradas independientes, cuando se establezca en false, las sobrecargas de importación de funciones se mostrarán como una función de unión en el navegador. Valor predeterminado para V3: false. Valor predeterminado para V4: true.
- **MoreColumns**: un valor lógico (true/false) cuando se establece en true, agrega una columna "Más columnas" a cada fuente de entidades que contiene tipos abiertos y tipos polimórficos. Esto contendrá los campos no declarados en el tipo base. Cuando es false, este campo no está presente. El valor predeterminado es False.
- **IncludeAnnotations**: una lista separada por comas de los nombres o patrones de términos calificados del espacio de nombres que se van a incluir con "*" como carácter comodín. De forma predeterminada, no se incluye ninguna de las anotaciones.
- **IncludeMetadataAnnotations**: una lista separada por comas de los nombres o patrones de términos calificados del espacio de nombres que se van a incluir en las solicitudes de documentos de metadatos, con "*" como carácter comodín. De forma predeterminada, incluye las mismas anotaciones que IncludeAnnotations.
- **OmitValues**: permite que el servicio OData evite escribir determinados valores en las respuestas. Si el servicio lo reconoce, deduciremos esos valores de los campos omitidos. Entre las opciones se incluyen:
 - **ODataOmitValues.Nulls**: permite que el servicio OData omita valores NULL.
- **Implementation**: especifica la implementación del conector de OData que se va a usar. Los valores válidos son "2.0" o null.

Ejemplo 1

Conéctese al servicio TripPin OData.

Uso

Power Query M

```
OData.Feed("https://services.odata.org/V4/TripPinService")
```

Salida

table

Odbc.DataSource

Sintaxis

```
Odbc.DataSource(connectionString as any, optional options as nullable record) as table
```

Acerca de

Devuelve una lista de tablas y vistas SQL del origen de datos ODBC especificado mediante la cadena de conexión `connectionString`. `connectionString` puede ser texto o un registro de pares de valores y propiedades. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado es de 15 segundos.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.

Ejemplo

Devuelve las tablas y vistas SQL de la cadena de conexión proporcionada.

Uso

Power Query M

```
Odbc.DataSource("dsn=your_dsn")
```

Salida

Power Query M

table

Last updated on 25/03/2026

Odbc.InferOptions

Sintaxis

```
Odbc.InferOptions(connectionString as any) as record
```

Acerca de

Devuelve el resultado de intentar inferir las funcionalidades de SQL con la cadena `connectionString` de conexión mediante ODBC. `connectionString` puede ser texto o un registro de pares de valores y propiedades. Los valores de propiedad pueden ser texto o número.

Ejemplo

Devuelve las funcionalidades de SQL inferidas para una cadena de conexión.

Uso

Power Query M

```
Odbc.InferOptions("dsn=your_dsn")
```

Salida

```
record
```

Last updated on 03/04/2026

Odbc.Query

Syntax

```
Odbc.Query(  
    connectionString as any,  
    query as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve el resultado de la ejecución `query` con la cadena `connectionString` de conexión mediante ODBC. `connectionString` puede ser texto o un registro de pares de valor de propiedad. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado es de 15 segundos.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.

Ejemplo

Devuelve el resultado de ejecutar una consulta simple en la cadena de conexión proporcionada.

Uso

Power Query M

```
Odbc.Query("dsn=your_dsn", "select * from Customers")
```

Salida

table

Last updated on 03/04/2026

OleDb.DataSource

Syntax

```
OleDb.DataSource(connectionString as any, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL del origen de datos OLE DB especificado por la cadena `connectionString` de conexión. `connectionString` puede ser texto o un registro de pares de valor de propiedad. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es true).
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

OleDb.Query

16/09/2025

Sintaxis

```
OleDb.Query(  
    connectionString as any,  
    query as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve el resultado de ejecutar `query` con la cadena de conexión `connectionString` mediante OLE DB. `connectionString` puede ser texto o un registro de pares de valores y propiedades. Los valores de propiedad pueden ser texto o número. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `SqlCompatibleWindowsAuth`: un valor lógico (true/false) que determina si se deben generar opciones de cadena de conexión compatibles con SQL Server para la autenticación de Windows. El valor predeterminado es true.

Oracle.Database

16/09/2025

Sintaxis

```
Oracle.Database(server as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL de la base de datos de Oracle en el servidor `server`. El puerto puede especificarse opcionalmente junto con el servidor, separado por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Pdf.Tables

Sintaxis

```
Pdf.Tables(pdf as binary, optional options as nullable record) as table
```

Acerca de

Devuelve las tablas que se encuentran en `pdf`. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `Implementation`: versión del algoritmo que se va a usar al identificar tablas. Las versiones anteriores solo están disponibles para la compatibilidad con versiones anteriores, para evitar que las consultas antiguas se interrumpan mediante actualizaciones de algoritmos. La versión más reciente siempre debe proporcionar los mejores resultados. Los valores válidos son "1.3", "1.2", "1.1" o null.
- `StartPage`: especifica la primera página del intervalo de páginas que se van a examinar. Valor predeterminado: 1.
- `EndPage`: especifica la última página del intervalo de páginas que se va a examinar. Valor predeterminado: la última página del documento.
- `MultiPageTables`: controla si las tablas similares de las páginas consecutivas se combinarán automáticamente en una sola tabla. Valor predeterminado: true.
- `EnforceBorderLines`: controla si las líneas de borde siempre se aplican como límites de celda (cuando es true) o simplemente se usan como una sugerencia entre muchas para determinar los límites de celda (cuando es false). Valor predeterminado: false.

Ejemplo

Devuelve las tablas contenidas en sample.pdf.

Uso

Power Query M

```
Pdf.Tables(File.Contents("c:\sample.pdf"))
```

Salida

```
#table({"Name", "Kind", "Data"}, ...)
```

Last updated on 03/04/2026

PostgreSQL.Database

16/09/2025

Sintaxis

```
PostgreSQL.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos PostgreSQL en el servidor `server` en la instancia de base de datos denominada `database`. El puerto puede especificarse opcionalmente junto al servidor, separado por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

RData.FromBinary

16/09/2025

Sintaxis

```
RData.FromBinary(stream as binary) as any
```

Acerca de

Devuelve un conjunto de marcos de datos del archivo RData.

Salesforce.Data

16/09/2025

Sintaxis

```
Salesforce.Data(optional loginUrl as any, optional options as nullable record) as table
```

Acerca de

Devuelve los objetos de la cuenta de Salesforce proporcionadas en las credenciales. La cuenta se conectará a través del entorno proporcionado `loginUrl`. Si no se proporciona ningún entorno, la cuenta se conectará al entorno de producción (<https://login.salesforce.com>). Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es false).
- `ApiVersion`: versión de la API de Salesforce que se va a usar para esta consulta. Cuando no se especifica, se usa la versión de API 29.0.
- `Timeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar la solicitud al servidor. El valor predeterminado es específico del origen.

Salesforce.Reports

16/09/2025

Sintaxis

```
Salesforce.Reports(optional loginUrl as nullable text, optional options as nullable record) as table
```

Acerca de

Devuelve los informes de la cuenta de Salesforce proporcionada en las credenciales. La cuenta se conectará a través del entorno proporcionado `loginUrl`. Si no se proporciona ningún entorno, la cuenta se conectará a la producción (<https://login.salesforce.com> [↗](#)). Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `ApiVersion`: versión de la API de Salesforce que se va a usar para esta consulta. Cuando no se especifica, se usa la versión de API 29.0.
- `Timeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar la solicitud al servidor. El valor predeterminado es específico del origen.

SapBusinessWarehouse.Cubes

Sintaxis

```
SapBusinessWarehouse.Cubes(  
    server as text,  
    systemNumberOrSystemId as text,  
    clientId as text,  
    optional optionsOrLogonGroup as any,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de InfoCubes y consultas agrupadas por InfoArea desde una instancia de SAP Business Warehouse en el servidor `server` con el número de sistema `systemNumberOrSystemId` y el identificador de cliente `clientId`. Se puede especificar un parámetro de registro opcional, `optionsOrLogonGroup`, para controlar las siguientes opciones:

Last updated on 04/04/2026

SapHana.Database

Syntax

```
SapHana.Database(server as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de paquetes multidimensionales de la base de datos `server` de SAP HANA . Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `Distribution`: que [SapHanaDistribution](#) establece el valor de la propiedad "Distribution" en la cadena de conexión. El enrutamiento de instrucciones es el método para evaluar el nodo de servidor correcto de un sistema distribuido antes de la ejecución de instrucciones. El valor predeterminado es [SapHanaDistribution.All](#).
- `Implementation`: especifica la implementación del conector de SAP HANA que se va a usar.
- `EnableColumnBinding`: enlaza variables a las columnas de un conjunto de resultados de SAP HANA al capturar datos. Puede mejorar potencialmente el rendimiento a costa de un uso de memoria ligeramente mayor. El valor predeterminado es false.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado es de 15 segundos.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.

SharePoint.Contents

Sintaxis

```
SharePoint.Contents(url as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada carpeta y documento encontrados en el sitio de SharePoint especificado, `url`. Cada fila contiene las propiedades de la carpeta o el archivo, y un vínculo a su contenido. Se puede especificar `options` para controlar las siguientes opciones:

- `ApiVersion`: un número (14 o 15), o bien el texto Auto que especifica la versión de la API de SharePoint que se va a usar para este sitio. Cuando no se especifica, se usa la versión 14 de la API. Cuando se especifica Auto, la versión del servidor se detectará de forma automática si es posible; de lo contrario, el valor predeterminado de la versión es 14. Los sitios de SharePoint que no están en inglés requieren al menos la versión 15.
- `Implementation`: opcional. Especifica la versión del conector de SharePoint que se va a usar. Los valores aceptados son "2.0" o null. Si el valor es "2.0", se usa la implementación 2.0 del conector de SharePoint. Si el valor es NULL, se usa la implementación original del conector de SharePoint.

Last updated on 04/04/2026

SharePoint.Files

16/09/2025

Sintaxis

```
SharePoint.Files(url as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla que contiene una fila para cada documento encontrado en el sitio de SharePoint especificado, `url`, así como en sus subcarpetas. Cada fila contiene propiedades de la carpeta o archivo y un vínculo a su contenido. `options` se puede especificar para controlar las siguientes opciones:

- `ApiVersion`: un número (14 o 15) o el texto "Automático" que especifica la versión de la API de SharePoint que se va a usar para este sitio. Cuando no se especifica, se usa la versión 14 de la API. Cuando se especifica Auto, la versión del servidor se detectará automáticamente si es posible; de lo contrario, la versión tiene como valor predeterminado 14. Los sitios de SharePoint que no son en inglés requieren al menos la versión 15.

SharePoint.Tables

16/09/2025

Sintaxis

```
SharePoint.Tables(url as text, optional options as nullable record) as table
```

Acercas de

Devuelve una tabla que contiene una fila para cada elemento de lista que se encuentra en la lista de SharePoint especificada, **url**. Cada fila contiene propiedades de la lista. **options** se puede especificar para controlar las siguientes opciones:

- **ApiVersion**: un número (14 o 15) o el texto "Automático" que especifica la versión de la API de SharePoint que se va a usar para este sitio. Cuando no se especifica, se usa la versión 14 de la API. Cuando se especifica Auto, la versión del servidor se detectará automáticamente si es posible; de lo contrario, la versión tiene como valor predeterminado 14. Los sitios de SharePoint que no son en inglés requieren al menos la versión 15.
- **Implementation**: opcional. Especifica la versión del conector de SharePoint que se va a usar. Los valores aceptados son "2.0" o null. Si el valor es "2.0", se usa la implementación 2.0 del conector de SharePoint. Si el valor es NULL, se usa la implementación original del conector de SharePoint.
- **ViewMode**: opcional. Esta opción solo es válida para la implementación 2.0. Los valores aceptados son "All" y "Default". Si no se especifica ningún valor, el valor se establece en "All". Cuando "Todo"; se especifica, la vista incluye todas las columnas creadas por el usuario y definidas por el sistema. Cuando se especifica "Default", la vista coincidirá con lo que ve el usuario al mirar la lista en línea en la vista que el usuario establezca como Predeterminado en su configuración. Si el usuario edita su vista predeterminada para agregar o quitar columnas creadas por el usuario o definidas por el sistema, o creando una nueva vista y estabándola como predeterminada, estos cambios se propagarán a través del conector.
- **DisableAppendNoteColumns**: impide que el conector use un punto de conexión independiente para las columnas de notas.

Soda.Feed

16/09/2025

Sintaxis

```
Soda.Feed(url as text) as table
```

Acercas de

Devuelve una tabla del contenido en la URL especificada **url** formateada según la API SODA 2.0. La dirección URL debe apuntar a un origen compatible con SODA válido que termine en una extensión .csv.

Sql.Database

Syntax

```
Sql.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas SQL, vistas y funciones almacenadas de la base de datos `database` de SQL Server en el servidor `server`. El puerto puede especificarse opcionalmente con el servidor, separados por dos puntos o una coma. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `MaxDegreeOfParallelism`: número que establece el valor de la cláusula de consulta "maxdop" en la consulta SQL generada.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).
- `MultiSubnetFailover`: un valor lógico (true/false) que establece el valor de la propiedad "MultiSubnetFailover" en la cadena de conexión (el valor predeterminado es false).
- `UnsafeTypeConversions`: un valor lógico (true/false) que, si es true, intenta plegar las conversiones de tipos que podrían producir errores y provocar que se produzca un error en toda la consulta. No se recomienda para uso general.
- `ContextInfo`: valor binario que se usa para establecer el CONTEXT_INFO antes de ejecutar cada comando.

- `OmitSRID`: un valor lógico (true/false) que, si es true, omite el SRID al generar Well-Known Texto a partir de tipos geometry y geography.
- `EnableCrossDatabaseFolding`: valor lógico (true/false) que, si es true, permite el plegado de consultas entre bases de datos del mismo servidor. El valor predeterminado es false.

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Last updated on 22/11/2025

Sql.Databases

16/09/2025

Sintaxis

```
Sql.Databases(server as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de bases de datos en el servidor SQL Server especificado, `server`. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `MaxDegreeOfParallelism`: número que establece el valor de la cláusula de consulta "maxdop" en la consulta SQL generada.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).
- `MultiSubnetFailover`: un valor lógico (true/false) que establece el valor de la propiedad "MultiSubnetFailover" en la cadena de conexión (el valor predeterminado es false).
- `UnsafeTypeConversions`: valor lógico ("true" o "false") que, cuando es "true", intenta plegar las conversiones de tipo que podrían fallar y provocar un fallo en toda la consulta. No se recomienda para uso general.
- `ContextInfo`: valor binario que se usa para establecer el CONTEXT_INFO antes de ejecutar cada comando.
- `OmitSRID`: valor lógico ("true"/"false") que, cuando es "true", omite el SRID al producir Well-Known Text a partir de tipos geométricos y geográficos.
- `EnableCrossDatabaseFolding`: valor lógico (true/false) que, si es true, permite el plegado de consultas entre bases de datos en el mismo servidor. El valor predeterminado es false.

El parámetro record se especifica como [option1 = value1, option2 = value2...] por ejemplo.

No admite la configuración de una consulta SQL para que se ejecute en el servidor.

[Sql.Database](#) debe usarse en su lugar para ejecutar una consulta SQL.

Sybase.Database

16/09/2025

Sintaxis

```
Sybase.Database(  
    server as text,  
    database as text,  
    optional options as nullable record  
) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL disponibles en una base de datos de Sybase en el servidor `server` en la instancia de base de datos denominada `database`. El puerto puede especificarse opcionalmente junto con el servidor, separado por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

Teradata.Database

16/09/2025

Sintaxis

```
Teradata.Database(server as text, optional options as nullable record) as table
```

Acerca de

Devuelve una tabla de tablas y vistas SQL de la base de datos de Teradata en el servidor `server`. Opcionalmente, se puede especificar el puerto junto con el servidor, separado por dos puntos. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `CreateNavigationProperties`: un valor lógico (true/false) que establece si se van a generar propiedades de navegación en los valores devueltos (el valor predeterminado es true).
- `NavigationPropertyNameGenerator`: función que se usa para la creación de nombres para las propiedades de navegación.
- `Query`: una consulta SQL nativa que se usa para recuperar datos. Si la consulta genera varios conjuntos de resultados, solo se devolverá la primera.
- `CommandTimeout`: duración que controla cuánto tiempo se permite ejecutar la consulta del lado servidor antes de cancelarla. El valor predeterminado es diez minutos.
- `ConnectionTimeout`: duración que controla cuánto tiempo se debe esperar antes de abandonar un intento de realizar una conexión con el servidor. El valor predeterminado depende del controlador.
- `HierarchicalNavigation`: un valor lógico (true/false) que establece si se van a ver las tablas agrupadas por sus nombres de esquema (el valor predeterminado es false).

El parámetro record se especifica como [option1 = value1, option2 = value2...] o [Query = "select ..."] por ejemplo.

WebAction.Request

Syntax

```
WebAction.Request(  
    method as text,  
    url as text,  
    optional options as nullable record  
) as action
```

Acerca de

Crea una acción que, cuando se ejecuta, devolverá los resultados de realizar una `method` solicitud en el `url` uso de HTTP como un valor binario. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `Query`: agregue mediante programación parámetros de consulta a la dirección URL sin tener que preocuparse por el escape.
- `ApiKeyName`: si el sitio de destino tiene una noción de una clave de API, este parámetro se puede usar para especificar el nombre (no el valor) del parámetro de clave que se debe usar en la dirección URL. El valor de clave real se proporciona en la credencial.
- `Headers`: si se especifica este valor como registro, se proporcionarán encabezados adicionales a una solicitud HTTP.
- `Timeout`: si se especifica este valor como una duración, se cambiará el tiempo de espera de una solicitud HTTP. El valor predeterminado es 100 segundos.
- `ExcludedFromCacheKey`: al especificar este valor como una lista se excluirán estas claves de encabezado HTTP de formar parte del cálculo de los datos de almacenamiento en caché.
- `IsRetry`: si se especifica este valor lógico como true, se omitirá cualquier respuesta existente en la memoria caché al capturar datos.
- `ManualStatusHandling`: si se especifica este valor como una lista, se impedirá cualquier control integrado de solicitudes HTTP cuya respuesta tenga uno de estos códigos de estado.
- `RelativePath`: al especificar este valor como texto se anexa a la dirección URL base antes de realizar la solicitud.
- `Content`: especificar este valor hará que su contenido se convierta en el cuerpo de la solicitud HTTP.

Tenga en cuenta que esta función está deshabilitada en la mayoría de los contextos. Considere la posibilidad de usar [Web.Contents](#) o [Web.Headers](#) en su lugar.

Ejemplo 1

Realice una solicitud GET en Bing.

Uso

Power Query M

```
WebAction.Request(WebMethod.Get, "https://bing.com")
```

Salida

Action

Last updated on 22/11/2025

Web.BrowserContents

16/09/2025

Sintaxis

```
Web.BrowserContents(url as text, optional options as nullable record) as text
```

Acerca de

Devuelve el código HTML del `url` especificado, tal como lo ve un explorador web. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `ApiKeyName`: si el sitio de destino tiene una noción de una clave de API, este parámetro se puede usar para especificar el nombre (no el valor) del parámetro de clave que se debe usar en la dirección URL. El valor real de la clave se proporciona en la credencial.
- `WaitFor`: especifica una condición que se debe esperar antes de descargar el CÓDIGO HTML, además de esperar a que se cargue la página (que siempre se realiza). Puede ser un registro que contenga campos Timeout y/o Selector. Si solo se especifica un tiempo de espera, la función esperará la cantidad de tiempo especificada antes de descargar el código HTML. Si se especifica un selector y un tiempo de espera, y el tiempo de espera transcurre antes de que el selector exista en la página, se producirá un error. Si se especifica un selector sin tiempo de espera, se aplica un tiempo de espera predeterminado de 30 segundos.

Ejemplo 1

Devuelve el código HTML de <https://microsoft.com> .

Uso

Power Query M

```
Web.BrowserContents("https://microsoft.com")
```

Salida

```
"<!DOCTYPE html><html xmlns=..."
```

Ejemplo 2

Devuelve el código HTML de <https://microsoft.com> después de esperar a que exista un selector CSS.

Uso

Power Query M

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Selector = "div.ready"]])
```

Salida

```
"<!DOCTYPE html><html xmlns=..."
```

Ejemplo 3

Devuelve el código HTML de <https://microsoft.com> después de esperar diez segundos.

Uso

Power Query M

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Timeout =  
#duration(0,0,0,10)]])
```

Salida

```
"<!DOCTYPE html><html xmlns=..."
```

Ejemplo 4

Devuelve el código HTML de <https://microsoft.com> después de esperar hasta diez segundos para que exista un selector CSS.

Uso

Power Query M

```
Web.BrowserContents("https://microsoft.com", [WaitFor = [Selector = "div.ready",  
Timeout = #duration(0,0,0,10)]])
```

Salida

```
"<!DOCTYPE html><html xmlns=..."
```

Web.Contents

Syntax

```
Web.Contents(url as text, optional options as nullable record) as binary
```

Acerca de

Devuelve el contenido descargado de `url` como binario. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `Query`: agregue mediante programación parámetros de consulta a la dirección URL sin tener que preocuparse por el escape.
- `ApiKeyName`: si el sitio de destino tiene una noción de una clave de API, este parámetro se puede usar para especificar el nombre (no el valor) del parámetro de clave que se debe usar en la dirección URL. El valor de clave real se proporciona en la credencial.
- `Headers`: si se especifica este valor como registro, se proporcionarán encabezados adicionales a una solicitud HTTP.
- `Timeout`: si se especifica este valor como una duración, se cambiará el tiempo de espera de una solicitud HTTP. El valor predeterminado es 100 segundos.
- `ExcludedFromCacheKey`: al especificar este valor como una lista se excluirán estas claves de encabezado HTTP de formar parte del cálculo de los datos de almacenamiento en caché.
- `IsRetry`: si se especifica este valor lógico como true, se omitirá cualquier respuesta existente en la memoria caché al capturar datos.
- `ManualStatusHandling`: si se especifica este valor como una lista, se impedirá cualquier control integrado de solicitudes HTTP cuya respuesta tenga uno de estos códigos de estado.
- `RelativePath`: al especificar este valor como texto se anexa a la dirección URL base antes de realizar la solicitud.
- `Content`: al especificar este valor se cambia la solicitud web de get a POST, utilizando el valor de la opción como el contenido de POST.

La solicitud HTTP se realiza como GET (cuando no se especifica ningún contenido) o post (cuando hay contenido). Las solicitudes POST solo se pueden realizar de forma anónima.

Los encabezados de la respuesta HTTP están disponibles como metadatos en el resultado binario. Fuera de un contexto de conector de datos personalizado, solo hay disponible un

subconjunto de encabezados de respuesta (por motivos de seguridad).

Ejemplo 1

Recupere el contenido del uso de `"https://bing.com/search?q=Power+Query"` las `RelativePath` opciones y `Query` . Estas opciones se pueden usar para consultar dinámicamente una dirección URL base estática.

Uso

Power Query M

```
let
    searchText = "Power Query"
in
    Web.Contents(
        "https://www.bing.com",
        [
            RelativePath = "search",
            Query = [q = searchText]
        ]
    )
```

Salida

binary

Ejemplo 2

Realice una operación POST en una dirección URL, pasando una carga JSON binaria y analizando la respuesta como JSON.

Uso

Power Query M

```
let
    url = ...,
    headers = [{"Content-Type" = "application/json"}],
    postData = Json.FromValue([x = 235.7, y = 41.53]),
    response = Web.Contents(
        url,
        [
            Headers = headers,
            Content = postData
        ]
    ),
```

```
jsonResponse = Json.Document(response)
in
jsonResponse
```

Salida

table

Ejemplo 3

Conéctese a una dirección URL segura que acepte una clave de autenticación como parte de su cadena de consulta. En lugar de codificar de forma rígida la clave secreta en M (lo que podría suponer un riesgo de seguridad), la clave se puede proporcionar de forma segura especificando su nombre (no su valor) en M, eligiendo autenticación de API web y escribiendo el valor de clave como parte de la credencial de api web. Cuando se usa de esta manera, el ejemplo siguiente generará una solicitud a `"https://contoso.com/api/customers/get?api_key=*****"`.

Uso

Power Query M

```
Web.Contents("https://contoso.com/api/customers/get", [ApiKeyName="api_key"])
```

Salida

binary

Información adicional

[Control de código de estado con Web.Contents en conectores personalizados](#)

Last updated on 22/11/2025

Web.Headers

16/09/2025

Sintaxis

```
Web.Headers(url as text, optional options as nullable record) as record
```

Acerca de

Devuelve los encabezados descargados de `url` como registro. Se puede proporcionar un parámetro de registro opcional, `options`, para especificar propiedades adicionales. El registro puede contener los siguientes campos:

- `Query`: agregue mediante programación parámetros de consulta a la dirección URL sin tener que preocuparse por el escape.
- `ApiKeyName`: si el sitio de destino tiene una noción de una clave de API, este parámetro se puede usar para especificar el nombre (no el valor) del parámetro de clave que se debe usar en la dirección URL. El valor real de la clave se proporciona en la credencial.
- `Headers`: si se especifica este valor como registro, se proporcionarán encabezados adicionales a una solicitud HTTP.
- `Timeout`: si se especifica este valor como una duración, se cambiará el tiempo de espera de una solicitud HTTP. El valor predeterminado es 100 segundos.
- `ExcludedFromCacheKey`: Especificar este valor como una lista excluirá estas claves de encabezado HTTP de formar parte del cálculo para los datos de caché.
- `IsRetry`: si se especifica este valor lógico como true, se omitirá cualquier respuesta existente en la memoria caché al capturar datos.
- `ManualStatusHandling`: si se especifica este valor como una lista, se impedirá el control integrado de las solicitudes HTTP cuya respuesta tenga uno de estos códigos de estado.
- `RelativePath`: Especificar este valor como texto lo anexa a la URL de base antes de realizar la solicitud.

La solicitud HTTP se realiza con el método HEAD. Fuera de un contexto de conector de datos personalizado, solo hay disponible un subconjunto de encabezados de respuesta (por motivos de seguridad).

Ejemplo 1

Recupere los encabezados HTTP para "https://bing.com/search?q=Power+Query" mediante las opciones RelativePath y Query.

Uso

Power Query M

```
let
    searchText = "Power Query"
in
    Web.Headers(
        "https://www.bing.com",
        [
            RelativePath = "search",
            Query = [q = searchText]
        ]
    )
```

Salida

Power Query M

```
([
    #"Cache-Control" = "private, max-age=0",
    #"Content-Encoding" = "gzip",
    #"Content-Length" = "0",
    #"Content-Type" = "text/html; charset=utf-8",
    Date = "Tue, 14 Dec 2021 16:57:25 GMT",
    Expires = "Tue, 14 Dec 2021 16:56:25 GMT",
    Vary = "Accept-Encoding"
]
meta [
    Response.Status = 200
])
```

Web.Page

16/09/2025

Sintaxis

```
Web.Page(html as any) as table
```

Acerca de

Devuelve el contenido del documento HTML dividido en sus estructuras constituyentes, así como una representación del documento completo y su texto después de quitar etiquetas.

Xml.Document

16/09/2025

Sintaxis

```
Xml.Document(contents as any, optional encoding as nullable number) as table
```

Acerca de

Devuelve el contenido del documento XML como una tabla jerárquica.

Xml.Tables

16/09/2025

Sintaxis

```
Xml.Tables(  
    contents as any,  
    optional options as nullable record,  
    optional encoding as nullable number  
) as table
```

Acerca de

Devuelve el contenido del documento XML como una colección de tablas planas organizadas jerárquicamente.

Ejemplo 1

Recupere el contenido de un archivo XML local.

Uso

Power Query M

```
Xml.Tables(File.Contents("C:\invoices.xml"))
```

Salida

table

Funciones binarias

Estas funciones crean y manipulan datos binarios.

Formatos binarios

Lectura de números

 Expandir tabla

Nombre	Description
BinaryFormat.7BitEncodedSignedInteger	Formato binario que lee un entero de 64 bits con signo que se codificó mediante una codificación de longitud variable de 7 bits.
BinaryFormat.7BitEncodedUnsignedInteger	Formato binario que lee un entero de 64 bits sin signo codificado mediante una codificación de longitud variable de 7 bits.
BinaryFormat.Binary	Devuelve un formato binario que lee un valor binario.
BinaryFormat.Byte	Formato binario que lee un entero de 8 bits sin signo.
BinaryFormat.Choice	Devuelve un formato binario que elige el siguiente formato binario basado en un valor que ya se ha leído.
BinaryFormat.Decimal	Formato binario que lee un valor decimal de 16 bytes de .NET.
BinaryFormat.Double	Formato binario que lee un valor de punto flotante ieee de precisión doble de 8 bytes.
BinaryFormat.Group	Devuelve un formato binario que lee un grupo de elementos. Cada valor de elemento va precedido por un valor de clave único. El resultado es una lista de valores de elemento.
BinaryFormat.Length	Devuelve un formato binario que limita la cantidad de datos que se pueden leer. Tanto como BinaryFormat.ListBinaryFormat.Binary se pueden usar para leer hasta el final de los datos. BinaryFormat.Length se puede usar para limitar el número de bytes leídos.
BinaryFormat.List	Devuelve un formato binario que lee una secuencia de elementos y devuelve una lista.
BinaryFormat.Null	Formato binario que lee cero bytes y devuelve null.

Nombre	Description
BinaryFormat.Record	Devuelve un formato binario que lee un registro. Cada campo del registro puede tener un formato binario diferente.
BinaryFormat.SignedInteger16	Formato binario que lee un entero de 16 bits con signo.
BinaryFormat.SignedInteger32	Formato binario que lee un entero de 32 bits con signo.
BinaryFormat.SignedInteger64	Formato binario que lee un entero de 64 bits con signo.
BinaryFormat.Single	Formato binario que lee un valor de punto flotante de precisión sencilla IEEE de 4 bytes.
BinaryFormat.Text	Devuelve un formato binario que lee un valor de texto. El valor de codificación opcional especifica la codificación del texto.
BinaryFormat.Transform	Devuelve un formato binario que transformará los valores leídos por otro formato binario.
BinaryFormat.UnsignedInteger16	Formato binario que lee un entero de 16 bits sin signo.
BinaryFormat.UnsignedInteger32	Formato binario que lee un entero de 32 bits sin signo.
BinaryFormat.UnsignedInteger64	Formato binario que lee un entero de 64 bits sin signo.

Control del orden de bytes

 Expandir tabla

Nombre	Description
BinaryFormat.ByteOrder	Devuelve un formato binario con el orden de bytes especificado por una función.
Table.PartitionValues	Devuelve información sobre cómo se particiona una tabla.

Datos binarios

 Expandir tabla

Nombre	Description
Binary.ApproximateLength	Devuelve la longitud aproximada del binario.

Nombre	Description
Binary.Buffer	Almacena en búfer el valor binario en la memoria. El resultado de esta llamada es un valor binario estable, lo que significa que tendrá una longitud determinista y un orden de bytes.
Binary.Combine	Combina una lista de archivos binarios en un solo binario.
Binary.Compress	Comprime un valor binario mediante el tipo de compresión especificado.
Binary.Decompress	Descomprime un valor binario mediante el tipo de compresión especificado.
Binary.From	Devuelve un valor binario del valor especificado.
Binary.FromList	Convierte una lista de números en un valor binario.
Binary.FromText	Descodifica los datos de un formulario de texto en binario.
Binary.InferContentType	Devuelve un registro con el campo Content.Type que contiene el tipo MIME inferido.
Binary.Length	Devuelve la longitud de los valores binarios.
Binary.Range	Devuelve un subconjunto del valor binario que comienza en un desplazamiento.
Binary.Split	Divide el binario especificado en una lista de archivos binarios mediante el tamaño de página especificado.
Binary.ToList	Convierte un valor binario en una lista de números
Binary.ToText	Codifica los datos binarios en un formulario de texto.
Binary.View	Crea o extiende un binario con controladores definidos por el usuario para las operaciones de consulta y acción.
Binary.ViewError	Crea un registro de error modificado que no desencadenará una reserva cuando lo produzca un controlador definido en una vista (a través de Binary.View).
Binary.ViewFunction	Crea una función que puede ser interceptada por un manejador definido en una vista (a través de <code>Binary.View</code>).
#binary	Crea un valor binario a partir de números o texto.

Binary.ApproximateLength

Syntax

```
Binary.ApproximateLength(binary as nullable binary) as nullable number
```

Acerca de

Devuelve la longitud aproximada de **binary** o un error si el origen de datos no admite una longitud aproximada.

Ejemplo 1

Obtenga la longitud aproximada del valor binario.

Uso

Power Query M

```
Binary.ApproximateLength(Binary.FromText("i45WMlSKjQUA", BinaryEncoding.Base64))
```

Salida

9

Last updated on 22/11/2025

Binary.Buffer

16/09/2025

Sintaxis

```
Binary.Buffer(binary as nullable binary) as nullable binary
```

Acerca de

Almacena en búfer el valor binario en memoria. El resultado de esta llamada es un valor binario estable, lo que significa que tendrá una longitud determinista y un orden de bytes.

Ejemplo 1

Cree una versión estable del valor binario.

Uso

Power Query M

```
Binary.Buffer(Binary.FromList({0..10}))
```

Salida

```
#binary({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10})
```

Binary.Combine

16/09/2025

Sintaxis

```
Binary.Combine(binaries as list) as binary
```

Acerca de

Combina una lista de archivos binarios en un solo binario.

Binary.Compress

Syntax

```
Binary.Compress(binary as nullable binary, compressionType as number) as nullable binary
```

Acerca de

Comprime un valor binario mediante el tipo de compresión especificado. El resultado de esta llamada es una copia comprimida de la entrada. Los tipos de compresión incluyen:

- [Compression.GZip](#)
- [Compression.Deflate](#)

Ejemplo 1

Comprima el valor binario.

Uso

Power Query M

```
Binary.Compress(Binary.FromList(List.Repeat({10}, 1000)), Compression.Deflate)
```

Salida

```
#binary({227, 226, 26, 5, 163, 96, 20, 12, 119, 0, 0})
```

Last updated on 22/11/2025

Binary.Decompress

Syntax

```
Binary.Decompress(binary as nullable binary, compressionType as number) as nullable binary
```

Acerca de

Descomprime un valor binario mediante el tipo de compresión especificado. El resultado de esta llamada es una copia descomprimida de la entrada. Los tipos de compresión incluyen:

- [Compression.GZip](#)
- [Compression.Deflate](#)

Ejemplo 1

Descomprima el valor binario.

Uso

Power Query M

```
Binary.Decompress(#binary({115, 103, 200, 7, 194, 20, 134, 36, 134, 74, 134, 84, 6, 0}), Compression.Deflate)
```

Salida

```
#binary({71, 0, 111, 0, 111, 0, 100, 0, 98, 0, 121, 0, 101, 0})
```

Last updated on 22/11/2025

Binary.From

Syntax

```
Binary.From(value as any, optional encoding as nullable number) as nullable binary
```

Acerca de

Devuelve un `binary` valor de la clase especificada `value`. Si el `value` especificado es `null`, **Binary.From** devuelve `null`. Si el objeto especificado `value` es `binary`, `value` se devuelve. Los valores de los siguientes tipos se pueden convertir en un `binary` valor:

- `text`: valor `binary` de la representación de texto. Consulte [Binary.FromText](#) para obtener más información.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo 1

Obtenga el `binary` valor de "1011".

Uso

Power Query M

```
Binary.From("1011")
```

Salida

```
Binary.FromText("1011", BinaryEncoding.Base64)
```

Last updated on 22/11/2025

Binary.FromList

16/09/2025

Sintaxis

```
Binary.FromList(list as list) as binary
```

Acerca de

Convierte una lista de números en un valor binario.

Binary.FromText

Syntax

```
Binary.FromText(text as nullable text, optional encoding as nullable number) as nullable binary
```

Acerca de

Devuelve el resultado de convertir el valor `text` de texto en un binario (lista de `number`). `encoding` se puede especificar para indicar la codificación utilizada en el valor de texto. Los valores siguientes `BinaryEncoding` se pueden usar para `encoding`.

- [BinaryEncoding.Base64](#): codificación base 64
- [BinaryEncoding.Hex](#): codificación hexadecimal

Ejemplo 1

Descodifique "1011" en binario.

Uso

Power Query M

```
Binary.FromText("1011")
```

Salida

```
Binary.FromText("1011", BinaryEncoding.Base64)
```

Ejemplo 2

Descodifique "1011" en binario con codificación Hexadecimal.

Uso

Power Query M

```
Binary.FromText("1011", BinaryEncoding.Hex)
```

Salida

```
Binary.FromText("EBE=", BinaryEncoding.Base64)
```

Last updated on 03/04/2026

Binary.InferContentType

16/09/2025

Sintaxis

```
Binary.InferContentType(source as binary) as record
```

Acerca de

Devuelve un registro con el campo Content.Type que contiene el tipo MIME inferido. Si el tipo de contenido inferido es text/*y se detecta una página de códigos de codificación, devuelve además el campo Content.Encoding que contiene la codificación de la secuencia. Si el tipo de contenido inferido es text/csv y el formato se delimita, además devuelve el campo Csv.PotentialDelimiter que contiene una tabla para el análisis de posibles delimitadores. Si el tipo de contenido inferido es text/csv y el formato es de ancho fijo, además devuelve el campo Csv.PotentialPositions que contiene una lista para el análisis de posibles posiciones de columna de ancho fijo.

Binary.Length

16/09/2025

Sintaxis

```
Binary.Length(binary as nullable binary) as nullable number
```

Acerca de

Devuelve el número de caracteres.

Binary.Range

Syntax

```
Binary.Range(  
    binary as binary,  
    offset as number,  
    optional count as nullable number  
) as binary
```

Acerca de

Devuelve un subconjunto del valor binario que comienza en el desplazamiento `binary`. Un parámetro opcional, `offset`, establece la longitud máxima del subconjunto.

Ejemplo 1

Devuelve un subconjunto del valor binario a partir del desplazamiento 6.

Uso

Power Query M

```
Binary.Range(#binary({0..10}), 6)
```

Salida

```
#binary({6, 7, 8, 9, 10})
```

Ejemplo 2

Devuelve un subconjunto de longitud 2 del desplazamiento 6 del valor binario.

Uso

Power Query M

```
Binary.Range(#binary({0..10}), 6, 2)
```

Salida

```
#binary({6, 7})
```

Last updated on 22/11/2025

Binary.Split

Sintaxis

```
Binary.Split(binary as binary, pageSize as number) as list
```

Acerca de

Divide `binary` en una lista de archivos binarios donde el primer elemento de la lista de salida es un binario que contiene los primeros `pageSize` bytes del binario de origen, el siguiente elemento de la lista de salida es un binario que contiene los siguientes `pageSize` bytes del binario de origen, etc.

Last updated on 16/09/2025

Binary.ToList

16/09/2025

Sintaxis

```
Binary.ToList(binary as binary) as list
```

Acerca de

Convierte un valor binario en una lista de números.

Binary.ToText

16/09/2025

Sintaxis

```
Binary.ToText(binary as nullable binary, optional encoding as nullable number) as nullable text
```

Acerca de

Devuelve el resultado de convertir una lista binaria de números `binary` en un valor de texto. Opcionalmente, se puede especificar `encoding` para indicar la codificación que se usará en el valor de texto generado. Los siguientes valores de `BinaryEncoding` se pueden usar para `encoding`.

- `BinaryEncoding.Base64`: codificación base 64
- `BinaryEncoding.Hex`: codificación hexadecimal

Binary.View

Syntax

```
Binary.View(binary as nullable binary, handlers as record) as binary
```

Acerca de

Devuelve una vista de `binary` dónde se usan las funciones especificadas en `handlers` en lugar del comportamiento predeterminado de una operación cuando se aplica la operación a la vista.

Si `binary` se proporciona, todas las funciones del controlador son opcionales. Si `binary` no se proporciona, se requiere la `GetStream` función de controlador. Si no se especifica una función de controlador para una operación, el comportamiento predeterminado de la operación se aplica en `binary` su lugar (excepto en el caso de `GetExpression`).

Las funciones de controlador deben devolver un valor que es semánticamente equivalente al resultado de aplicar la operación en `binary` (o la vista resultante en el caso de `GetExpression`).

Si una función de controlador genera un error, el comportamiento predeterminado de la operación se aplica a la vista.

`Binary.View` se puede usar para implementar el plegado en un origen de datos: la traducción de consultas M en operaciones específicas del origen (por ejemplo, para descargar una sección de un archivo).

Consulte la documentación publicada del conector personalizado de Power Query para obtener una descripción más completa de `Binary.View`.

Ejemplo

Cree una vista básica que no requiera acceso a los datos para determinar la longitud.

Uso

Power Query M

```
Binary.View(  
    null,  
    [  
        GetLength = () => 12,  
        GetStream = () => Text.ToBinary("hello world!")  
    ]  
)
```

Salida

```
Text.ToBinary("hello world!")
```

Last updated on 03/04/2026

Binary.ViewError

Syntax

```
Binary.ViewError(errorRecord as record) as record
```

Acerca de

Crea un registro de error modificado desde `errorRecord` el que no desencadenará una reserva cuando se genere mediante un controlador definido en una vista (a través de [Binary.View](#)).

Last updated on 11/02/2026

Binary.ViewFunction

16/09/2025

Sintaxis

```
Binary.ViewFunction(function as function) as function
```

Acerca de

Crea una función de vista basada en `function` que puede ser manejada en una vista creada por [Binary.View](#).

El controlador `OnInvoke` de **Binary.View** se puede usar para definir un controlador para la función de vista.

Al igual que con los controladores para las operaciones incorporadas, si no se especifica un controlador para `OnInvoke`, o si no gestiona la función de vista, o si el controlador genera un error, `function` se aplica sobre la vista.

Consulte la documentación publicada del conector personalizado de Power Query para obtener una descripción más completa de **Binary.View** y funciones de vista personalizadas.

BinaryFormat.7BitEncodedSignedInteger

16/09/2025

Sintaxis

```
BinaryFormat.7BitEncodedSignedInteger(binary as binary) as any
```

Acerca de

Un formato binario que lee un entero con signo de 64 bits que se codificó mediante una codificación de longitud variable de 7 bits.

BinaryFormat.7BitEncodedUnsignedInteger

16/09/2025

Sintaxis

```
BinaryFormat.7BitEncodedUnsignedInteger(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 64 bits sin signo codificado mediante una codificación de longitud variable de 7 bits.

BinaryFormat.Binary

16/09/2025

Sintaxis

```
BinaryFormat.Binary(optional length as any) as function
```

Acerca de

Devuelve un formato binario que lee un valor binario. Si se especifica `length`, el valor binario contendrá muchos bytes. Si no se especifica `length`, el valor binario contendrá los bytes restantes. El `length` se puede especificar como un número o como formato binario de la longitud que precede a los datos binarios.

BinaryFormat.Byte

16/09/2025

Sintaxis

```
BinaryFormat.Byte(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 8 bits sin signo.

BinaryFormat.ByteOrder

16/09/2025

Sintaxis

```
BinaryFormat.ByteOrder(binaryFormat as function, byteOrder as number) as function
```

Acerca de

Devuelve un formato binario con el orden de bytes especificado por `binaryFormat`. El orden de bytes predeterminado es `ByteOrder.BigEndian`.

BinaryFormat.Choice

Syntax

```
BinaryFormat.Choice(  
    binaryFormat as function,  
    chooseFunction as function,  
    optional type as nullable type,  
    optional combineFunction as nullable function  
) as function
```

Acerca de

Devuelve un formato binario que elige el siguiente formato binario basado en un valor que ya se ha leído. El valor de formato binario generado por esta función funciona en fases:

- El formato binario especificado por el `binaryFormat` parámetro se usa para leer un valor.
- El valor se pasa a la función de elección especificada por el `chooseFunction` parámetro .
- La función choice inspecciona el valor y devuelve un segundo formato binario.
- El segundo formato binario se usa para leer un segundo valor.
- Si se especifica la función combine, los valores primero y segundo se pasan a la función combine y se devuelve el valor resultante.
- Si no se especifica la función combine, se devuelve el segundo valor.
- Se devuelve el segundo valor.

El parámetro opcional `type` indica el tipo de formato binario que devolverá la función choice.

Se `type any` puede especificar , `type list` o `type binary` . Si no se especifica el `type` parámetro , `type any` se usa. Si `type list` se usa o `type binary` , el sistema puede devolver un streaming `binary` o `list` un valor en lugar de uno almacenado en búfer, lo que puede reducir la cantidad de memoria necesaria para leer el formato.

Ejemplo 1

Lea una lista de bytes donde el primer byte determina el número de elementos.

Uso

```
let
    binaryData = #binary({2, 3, 4, 5}),
    listFormat = BinaryFormat.Choice(
        BinaryFormat.Byte,
        (length) => BinaryFormat.List(BinaryFormat.Byte, length)
    )
in
    listFormat(binaryData)
```

Salida

```
{3, 4}
```

Ejemplo 2

Lea una lista de bytes donde el número de elementos viene determinado por el primer byte y conserve la primera lectura de bytes.

Uso

Power Query M

```
let
    binaryData = #binary({2, 3, 4, 5}),
    listFormat = BinaryFormat.Choice(
        BinaryFormat.Byte,
        (length) => BinaryFormat.Record([
            length = length,
            list = BinaryFormat.List(BinaryFormat.Byte, length)
        ])
    )
in
    listFormat(binaryData)
```

Salida

```
[length = 2, list = {3, 4}]
```

Ejemplo 3

Lea una lista de bytes donde el número de elementos viene determinado por el primer byte mediante una lista de streaming.

Uso

Power Query M

```
let
  binaryData = #binary({2, 3, 4, 5}),
  listFormat = BinaryFormat.Choice(
    BinaryFormat.Byte,
    (length) => BinaryFormat.List(BinaryFormat.Byte, length),
    type list
  )
in
  listFormat(binaryData)
```

Salida

```
{3, 4}
```

Last updated on 04/04/2026

BinaryFormat.Decimal

16/09/2025

Sintaxis

```
BinaryFormat.Decimal(binary as binary) as any
```

Acerca de

Formato binario que lee un valor decimal de 16 bytes de .NET.

BinaryFormat.Double

16/09/2025

Sintaxis

```
BinaryFormat.Double(binary as binary) as any
```

Acerca de

Un formato binario que lee un valor de punto flotante de doble precisión IEEE de 8 bytes.

BinaryFormat.Group

Syntax

```
BinaryFormat.Group(  
    binaryFormat as function,  
    group as list,  
    optional extra as nullable function,  
    optional lastKey as any  
) as function
```

Acerca de

Los parámetros son los siguientes:

- El `binaryFormat` parámetro especifica el formato binario del valor de clave.
- El `group` parámetro proporciona información sobre el grupo de elementos conocidos.
- El parámetro opcional `extra` se puede usar para especificar una función que devolverá un valor de formato binario para el valor después de cualquier clave inesperada. Si no se especifica el `extra` parámetro, se producirá un error si hay valores de clave inesperados.

El `group` parámetro especifica una lista de definiciones de elementos. Cada definición de elemento es una lista que contiene 3-5 valores, como se indica a continuación:

- Valor de clave. Valor de la clave que corresponde al elemento. Debe ser único dentro del conjunto de elementos.
- Formato de elemento. Formato binario correspondiente al valor del elemento. Esto permite que cada elemento tenga un formato diferente.
- Repetición del elemento. Valor [BinaryOccurrence.Type](#) de cuántas veces se espera que el elemento aparezca en el grupo. Los elementos necesarios que no están presentes provocan un error. Los elementos duplicados obligatorios o opcionales se controlan como valores de clave inesperados.
- Valor de elemento predeterminado (opcional). Si el valor de elemento predeterminado aparece en la lista de definiciones de elemento y no es NULL, se usará en lugar del valor predeterminado. El valor predeterminado para los elementos repetidos o opcionales es NULL y el valor predeterminado para los valores repetidos es una lista vacía { }.
- Transformación de valor de elemento (opcional). Si la función de transformación de valor de elemento está presente en la lista de definiciones de elementos y no es null, se llamará a para transformar el valor del elemento antes de que se devuelva. Solo se llama a la función de transformación si el elemento aparece en la entrada (nunca se llamará con el valor predeterminado).

Ejemplo 1

A continuación se supone un valor de clave que es un solo byte, con 4 elementos esperados en el grupo, todos los cuales tienen un byte de datos después de la clave. Los elementos aparecen en la entrada de la siguiente manera:

- Se requiere la clave 1 y aparece con el valor 11.
- La clave 2 se repite y aparece dos veces con el valor 22 y da como resultado un valor de { 22, 22 }.
- La clave 3 es opcional y no aparece y da como resultado un valor null.
- La clave 4 se repite, pero no aparece, y da como resultado un valor de { }.
- La clave 5 no forma parte del grupo, pero aparece una vez con el valor 55. Se llama a la función adicional con el valor de clave 5 y devuelve el formato correspondiente a ese valor ([BinaryFormat.Byte](#)). El valor 55 se lee y descarta.

Uso

Power Query M

```
let
    b = #binary({
        1, 11,
        2, 22,
        2, 22,
        5, 55,
        1, 11
    }),
    f = BinaryFormat.Group(
        BinaryFormat.Byte,
        {
            {1, BinaryFormat.Byte, BinaryOccurrence.Required},
            {2, BinaryFormat.Byte, BinaryOccurrence.Repeating},
            {3, BinaryFormat.Byte, BinaryOccurrence.Optional},
            {4, BinaryFormat.Byte, BinaryOccurrence.Repeating}
        },
        (extra) => BinaryFormat.Byte
    )
in
    f(b)
```

Salida

```
{11, {22, 22}, null, {}}
```

Ejemplo 2

En el ejemplo siguiente se muestra la transformación de valor de elemento y el valor de elemento predeterminado. El elemento de repetición con la clave 1 suma la lista de valores leídos mediante [List.Sum](#). El elemento opcional con la clave 2 tiene un valor predeterminado de 123 en lugar de null.

Uso

Power Query M

```
let
    b = #binary({
        1, 101,
        1, 102
    }),
    f = BinaryFormat.Group(
        BinaryFormat.Byte,
        {
            {1, BinaryFormat.Byte, BinaryOccurrence.Repeating,
              0, (list) => List.Sum(list)},
            {2, BinaryFormat.Byte, BinaryOccurrence.Optional, 123}
        }
    )
in
    f(b)
```

Salida

```
{203, 123}
```

Last updated on 22/11/2025

BinaryFormat.Length

16/09/2025

Sintaxis

```
BinaryFormat.Length(binaryFormat as function, length as any) as function
```

Acerca de

Devuelve un formato binario que limita la cantidad de datos que se pueden leer. Tanto [BinaryFormat.List](#) como [BinaryFormat.Binary](#) se pueden usar para leer hasta el final de los datos. **BinaryFormat.Length** se puede usar para limitar el número de bytes leídos. El parámetro `binaryFormat` especifica el formato binario que se va a limitar. El parámetro `length` especifica el número de bytes que se van a leer. El parámetro `length` puede ser un valor numérico o un valor de formato binario que especifica el formato del valor de longitud que aparece que precede al valor que se lee.

Ejemplo 1

Limite el número de bytes leídos a 2 al leer una lista de bytes.

Uso

Power Query M

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.Length(
        BinaryFormat.List(BinaryFormat.Byte),
        2
    )
in
    listFormat(binaryData)
```

Salida

```
{1, 2}
```

Ejemplo 2

Limite el número de bytes leídos al leer una lista de bytes al valor del byte que precede a la lista.

Uso

Power Query M

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.Length(
        BinaryFormat.List(BinaryFormat.Byte),
        BinaryFormat.Byte
    )
in
    listFormat(binaryData)
```

Salida

```
{2}
```

BinaryFormat.List

Syntax

```
BinaryFormat.List(binaryFormat as function, optional countOrCondition as any) as function
```

Acerca de

Devuelve un formato binario que lee una secuencia de elementos y devuelve un `list`. El `binaryFormat` parámetro especifica el formato binario de cada elemento. Hay tres maneras de determinar el número de elementos leídos:

- `countOrCondition` Si no se especifica , el formato binario leerá hasta que no haya más elementos.
- Si es `countOrCondition` un número, el formato binario leerá muchos elementos.
- Si es `countOrCondition` una función, esa función se invocará para cada lectura de elementos. La función devuelve true para continuar y false para dejar de leer elementos. El elemento final se incluye en la lista.
- `countOrCondition` Si es un formato binario, se espera que el recuento de elementos precede a la lista y el formato especificado se use para leer el recuento.

Ejemplo 1

Lee bytes hasta el final de los datos.

Uso

Power Query M

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.List(BinaryFormat.Byte)
in
    listFormat(binaryData)
```

Salida

```
{1, 2, 3}
```

Ejemplo 2

Lee dos bytes.

Uso

Power Query M

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.List(BinaryFormat.Byte, 2)
in
    listFormat(binaryData)
```

Salida

```
{1, 2}
```

Ejemplo 3

Lee bytes hasta que el valor de bytes es mayor o igual que dos.

Uso

Power Query M

```
let
    binaryData = #binary({1, 2, 3}),
    listFormat = BinaryFormat.List(BinaryFormat.Byte, (x) => x < 2)
in
    listFormat(binaryData)
```

Salida

```
{1, 2}
```

BinaryFormat.Null

16/09/2025

Sintaxis

```
BinaryFormat.Null(binary as binary) as any
```

Acerca de

Formato binario que lee cero bytes y devuelve null.

BinaryFormat.Record

Syntax

```
BinaryFormat.Record(record as record) as function
```

Acerca de

Devuelve un formato binario que lee un registro. El `record` parámetro especifica el formato del registro. Cada campo del registro puede tener un formato binario diferente. Si un campo contiene un valor que no es un valor de formato binario, no se lee ningún dato para ese campo y se devuelve el valor del campo al resultado.

Ejemplo 1

Lee un registro que contiene un entero de 16 bits y un entero de 32 bits.

Uso

Power Query M

```
let
    binaryData = #binary({
        0x00, 0x01,
        0x00, 0x00, 0x00, 0x02
    }),
    recordFormat = BinaryFormat.Record([
        A = BinaryFormat.UnsignedInteger16,
        B = BinaryFormat.UnsignedInteger32
    ])
in
    recordFormat(binaryData)
```

Salida

```
[A = 1, B = 2]
```

BinaryFormat.SignedInteger16

16/09/2025

Sintaxis

```
BinaryFormat.SignedInteger16(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 16 bits con signo.

BinaryFormat.SignedInteger32

16/09/2025

Sintaxis

```
BinaryFormat.SignedInteger32(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 32 bits con signo.

BinaryFormat.SignedInteger64

16/09/2025

Sintaxis

```
BinaryFormat.SignedInteger64(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 64 bits con signo.

BinaryFormat.Single

16/09/2025

Sintaxis

```
BinaryFormat.Single(binary as binary) as any
```

Acerca de

Formato binario que lee un valor de punto flotante de precisión sencilla IEEE de 4 bytes.

BinaryFormat.Text

Syntax

```
BinaryFormat.Text(length as any, optional encoding as nullable number) as function
```

Acerca de

Devuelve un formato binario que lee un valor de texto. `length` especifica el número de bytes que se van a decodificar o el formato binario de la longitud que precede al texto. El valor opcional `encoding` especifica la codificación del texto. `encoding` Si no se especifica , la codificación se determina a partir de las marcas de orden de bytes Unicode. Si no hay marcas de orden de bytes presentes, `TextEncoding.Utf8` se usa.

Ejemplo 1

Descodifique dos bytes como texto ASCII.

Uso

Power Query M

```
let
    binaryData = #binary({65, 66, 67}),
    textFormat = BinaryFormat.Text(2, TextEncoding.Ascii)
in
    textFormat(binaryData)
```

Salida

```
"AB"
```

Ejemplo 2

Descodificar texto ASCII donde la longitud del texto en bytes aparece antes del texto como un byte.

Uso

Power Query M

```
let
  binaryData = #binary({2, 65, 66}),
  textFormat = BinaryFormat.Text(
    BinaryFormat.Byte,
    TextEncoding.Ascii
  )
in
  textFormat(binaryData)
```

Salida

"AB"

Last updated on 22/11/2025

BinaryFormat.Transform

Syntax

```
BinaryFormat.Transform(binaryFormat as function, function as function) as function
```

Acerca de

Devuelve un formato binario que transformará los valores leídos por otro formato binario. El `binaryFormat` parámetro especifica el formato binario que se usará para leer el valor. `function` se invoca con el valor leído y devuelve el valor transformado.

Ejemplo 1

Lea un byte y agregue uno a él.

Uso

Power Query M

```
let
    binaryData = #binary({1}),
    transformFormat = BinaryFormat.Transform(
        BinaryFormat.Byte,
        (x) => x + 1
    )
in
    transformFormat(binaryData)
```

Salida

2

Last updated on 22/11/2025

BinaryFormat.UnsignedInteger16

16/09/2025

Sintaxis

```
BinaryFormat.UnsignedInteger16(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 16 bits sin signo.

BinaryFormat.UnsignedInteger32

16/09/2025

Sintaxis

```
BinaryFormat.UnsignedInteger32(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 32 bits sin signo.

BinaryFormat.UnsignedInteger64

16/09/2025

Sintaxis

```
BinaryFormat.UnsignedInteger64(binary as binary) as any
```

Acerca de

Formato binario que lee un entero de 64 bits sin signo.

#binary

16/09/2025

Sintaxis

```
#binary(value as any) as any
```

Acerca de

Crea un valor binario a partir de una lista de números o un valor de texto codificado en base 64.

Ejemplo 1

Cree un valor binario a partir de una lista de números.

Uso

Power Query M

```
#binary({0x30, 0x31, 0x32})
```

Salida

```
Text.ToBinary("012")
```

Ejemplo 2

Cree un valor binario a partir de un valor de texto codificado en base 64.

Uso

Power Query M

```
#binary("1011")
```

Salida

```
Binary.FromText("1011", BinaryEncoding.Base64)
```


Funciones del combinador

Artículo • 17/12/2024

Estas funciones las usan otras funciones de biblioteca que combinan valores. Por ejemplo, [Table.ToList](#) y [Table.CombineColumns](#) aplicar una función de combinador a cada fila de una tabla para generar un único valor para cada fila.

 Expandir tabla

Nombre	Descripción
Combiner.CombineTextByDelimiter	Devuelve una función que combina una lista de texto mediante el delimitador especificado.
Combiner.CombineTextByEachDelimiter	Devuelve una función que combina una lista de texto mediante una secuencia de delimitadores.
Combiner.CombineTextByLengths	Devuelve una función que combina una lista de texto mediante las longitudes especificadas.
Combiner.CombineTextByPositions	Devuelve una función que combina una lista de texto mediante las posiciones de salida especificadas.
Combiner.CombineTextByRanges	Devuelve una función que combina una lista de texto mediante las posiciones y longitudes especificadas.

Comentarios

¿Le ha resultado útil esta página?

 **Sí**

 **No**

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Combiner.CombineTextByDelimiter

Syntax

```
Combiner.CombineTextByDelimiter(delimiter as text, optional quoteStyle as nullable number) as function
```

Acerca de

Devuelve una función que combina una lista de valores de texto en un solo valor de texto mediante el delimitador especificado.

Ejemplo 1

Combine una lista de valores de texto mediante un delimitador de punto y coma.

Uso

Power Query M

```
Combiner.CombineTextByDelimiter(";")({"a", "b", "c"})
```

Salida

```
"a;b;c"
```

Ejemplo 2

Combine el texto de dos columnas mediante un delimitador de comas y comillas de estilo CSV.

Uso

Power Query M

```
let
    Source = #table(
        type table [Column1 = text, Column2 = text],
        {"a", "b"}, {"c", "d,e,f"}
    ),
    Merged = Table.CombineColumns(
        Source,
        {"Column1", "Column2"},
        Combiner.CombineTextByDelimiter(",", QuoteStyle.Csv),
    )
```

```
        "Merged"  
    )  
in  
    Merged
```

Salida

Power Query M

```
#table(  
    type table [Merged = text],  
    {"a,b"}, {"c","d,e,f"}  
)
```

Last updated on 22/11/2025

Combiner.CombineTextByEachDelimiter

16/09/2025

Sintaxis

```
Combiner.CombineTextByEachDelimiter(delimiters as list, optional quoteStyle as nullable number) as function
```

Acerca de

Devuelve una función que combina una lista de valores de texto en un único valor de texto mediante una secuencia de delimitadores.

Ejemplo 1

Combine una lista de valores de texto mediante una secuencia de delimitadores.

Uso

Power Query M

```
Combiner.CombineTextByEachDelimiter({"=", "+"})({"a", "b", "c"})
```

Salida

```
"a=b+c"
```

Combiner.CombineTextByLengths

Syntax

```
Combiner.CombineTextByLengths(lengths as list, optional template as nullable text)  
as function
```

Acerca de

Devuelve una función que combina una lista de valores de texto en un solo valor de texto mediante las longitudes especificadas.

Ejemplo 1

Combine una lista de valores de texto extrayendo los números de caracteres especificados de cada valor de entrada.

Uso

Power Query M

```
Combiner.CombineTextByLengths({1, 2, 3})({"aaa", "bbb", "ccc"})
```

Salida

```
"abbccc"
```

Ejemplo 2

Combine una lista de valores de texto extrayendo los números de caracteres especificados, después de rellenar previamente el resultado con el texto de la plantilla.

Uso

Power Query M

```
Combiner.CombineTextByLengths({1, 2, 3}, "*****")({"aaa", "bbb", "ccc"})
```

Salida

```
"abbccc***"
```

Last updated on 16/09/2025

Combiner.CombineTextByPositions

Syntax

```
Combiner.CombineTextByPositions(positions as list, optional template as nullable text) as function
```

Acerca de

Devuelve una función que combina una lista de valores de texto en un valor de texto único mediante las posiciones de salida especificadas.

Ejemplo

Combine una lista de valores de texto colocándolos en la salida en las posiciones especificadas.

Uso

Power Query M

```
Combiner.CombineTextByPositions({0, 5, 10})({"abc", "def", "ghi"})
```

Salida

```
"abc def ghi"
```

Last updated on 03/04/2026

Combiner.CombineTextByRanges

Sintaxis

```
Combiner.CombineTextByRanges(ranges as list, optional template as nullable text) as function
```

Acerca de

Devuelve una función que combina una lista de valores de texto en un solo valor de texto mediante las posiciones y longitudes de salida especificadas. Una longitud nula indica que se debe incluir todo el valor de texto.

Ejemplo

Combine una lista de valores de texto mediante las longitudes y posiciones de salida especificadas.

Uso

Power Query M

```
Combiner.CombineTextByRanges({{0, 1}, {3, 2}, {6, null}})({"abc", "def", "ghijkl"})
```

Salida

```
"a de ghijkl"
```

Last updated on 03/04/2026

Funciones de comparador

Artículo • 23/11/2024

Estas funciones prueban la igualdad y determinan la ordenación.

 Expandir tabla

Nombre	Descripción
Comparer.Equals	Devuelve un valor lógico basado en la comparación de igualdad de dos valores determinados.
Comparer.FromCulture	Devuelve una función de comparador basada en la referencia cultural y la distinción entre mayúsculas y minúsculas especificadas.
Comparer.Ordinal	Devuelve una función de comparador que usa reglas de ordinales para comparar valores.
Comparer.OrdinalIgnoreCase	Devuelve una función de comparador que no distingue mayúsculas de minúsculas que usa reglas ordinales para comparar los valores proporcionados.

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Comparer.Equals

Syntax

```
Comparer.Equals(  
    comparer as function,  
    x as any,  
    y as any  
) as logical
```

Acerca de

Devuelve un `logical` valor basado en la comprobación de igualdad sobre los dos valores especificados y `x y`, mediante el proporcionado `comparer`.

`comparer` es un `Comparer` objeto que se usa para controlar la comparación. Un comparador es una función que acepta dos argumentos y devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo. Los comparadores se pueden usar para proporcionar comparaciones que no distinguen mayúsculas de minúsculas o de referencia cultural y de configuración regional.

Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- [Comparer.Ordinal](#): se usa para realizar una comparación ordinal exacta.
- [Comparer.OrdinalIgnoreCase](#): se usa para realizar una comparación exacta sin distinción entre mayúsculas y minúsculas
- [Comparer.FromCulture](#): se usa para realizar una comparación que tenga en cuenta la referencia cultural.

Ejemplo

Compare "1" y "A" con la configuración regional "en-US" para determinar si los valores son iguales.

Uso

Power Query M

```
Comparer.Equals(Comparer.FromCulture("en-US"), "1", "A")
```

Salida

false

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 25/03/2026

Comparer.FromCulture

Syntax

```
Comparer.FromCulture(culture as text, optional ignoreCase as nullable logical) as function
```

Acerca de

Devuelve una función de comparador que usa y **culture** la distinción de mayúsculas y minúsculas especificada por **ignoreCase** para realizar comparaciones.

Una función del comparador acepta dos argumentos y devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo.

El valor predeterminado de **ignoreCase** es false. **culture** debe ser una de las configuraciones regionales admitidas por .NET Framework (por ejemplo, "en-US").

Ejemplo 1

Compare "a" y "A" mediante "en-US" configuración regional para determinar si los valores son iguales.

Uso

Power Query M

```
Comparer.FromCulture("en-US")("a", "A")
```

Salida

-1

Ejemplo 2

Compare "a" y "A" con "en-US" configuración regional omite el caso para determinar si los valores son iguales.

Uso

Power Query M

```
Comparer.FromCulture("en-US", true)("a", "A")
```

Salida

0

Last updated on 04/04/2026

Comparer.Ordinal

Syntax

```
Comparer.Ordinal(x as any, y as any) as number
```

Acerca de

Devuelve una función de comparador que usa reglas ordinales para comparar los valores proporcionados `x` y `y`.

Ejemplo 1

Con las reglas ordinales, compare si "encyclo^{pæ}dia" y "encyclo^{pæ}dia" son equivalentes. Tenga en cuenta que son equivalentes mediante `Comparer.FromCulture("en-US")`.

Una función del comparador acepta dos argumentos y devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo.

Uso

Power Query M

```
Comparer.Equals(Comparer.Ordinal, "encyclopædia", "encyclopædia")
```

Salida

```
false
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Comparer.OrdinalIgnoreCase

Syntax

```
Comparer.OrdinalIgnoreCase(x as any, y as any) as number
```

Acerca de

Devuelve una función de comparador que no distingue mayúsculas de minúsculas que usa reglas ordinales para comparar los valores `x` proporcionados y `y`.

Una función del comparador acepta dos argumentos y devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo.

Ejemplo

Con las reglas ordinales que no distinguen mayúsculas de minúsculas, compare "Abc" con "abc". Nota "Abc" es menor que "abc" con `Comparer.Ordinal`.

Uso

Power Query M

```
Comparer.OrdinalIgnoreCase("Abc", "abc")
```

Salida

0

Last updated on 25/03/2026

Funciones de fecha

Artículo • 29/05/2025

Estas funciones crean y manipulan el componente de fecha de los valores date, datetime y datetimezone.

[Expandir tabla](#)

Nombre	Descripción
Date.AddDays	Devuelve un valor Date/DateTime/DateTimeZone con la parte del día incrementada por el número de días proporcionados. También se encarga de incrementar las partes correspondientes al mes y al año del valor cuando sea necesario.
Date.AddMonths	Devuelve un valor DateTime con la parte del mes incrementada en n meses.
Date.AddQuarters	Devuelve un valor Date/DateTime/DateTimeZone incrementado por el número de trimestres proporcionados. Cada trimestre se define como una duración de tres meses. También gestiona el incremento de la parte del año del valor según corresponda.
Date.AddWeeks	Devuelve un valor Date/DateTime/DateTimeZone incrementado por el número de semanas proporcionadas. Cada semana se define como una duración de siete días. También gestiona el incremento de las partes de mes y año del valor según corresponda.
Date.AddYears	Devuelve un valor DateTime con la parte del año incrementada en n años.
Date.Day	Devuelve el día de un valor de fecha y hora.
Date.DayOfWeek	Devuelve un número (de 0 a 6) que indica el día de la semana del valor proporcionado.
Date.DayOfWeekName	Devuelve el nombre del día de la semana.
Date.DayOfYear	Devuelve un número que representa el día del año a partir de un valor DateTime.
Date.DaysInMonth	Devuelve el número de días del mes a partir de un valor DateTime.
Date.EndOfDay	Devuelve el final del día.
Date.EndOfMonth	Devuelve el final del mes.
Date.EndOfQuarter	Devuelve el fin del trimestre.
Date.EndOfWeek	Devuelve el final de la semana.
Date.EndOfYear	Devuelve el final del año.

Nombre	Descripción
Date.From	Devuelve un valor de fecha a partir de un valor.
Date.FromText	Crea una fecha a partir de formatos de fecha locales, universales y personalizados.
Date.IsInCurrentDay	Indica si el valor <code>dateTime</code> de fecha y hora especificado se produce durante el día actual, determinado por la fecha y hora actuales del sistema.
Date.IsInCurrentMonth	Devuelve un valor lógico que indica si la fecha, fecha y hora, o zona horaria especificadas ocurrieron durante el mes actual, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInCurrentQuarter	Devuelve un valor lógico que indica si la Fecha/FechaHora/ZonaHoraria especificada ocurrió durante el trimestre actual, según la fecha y hora actuales en el sistema.
Date.IsInCurrentWeek	Devuelve un valor lógico que indica si la Fecha/Fecha y Hora/Zona Horaria especificadas ocurrieron durante la semana actual, según lo determine la fecha y hora actuales en el sistema.
Date.IsInCurrentYear	Devuelve un valor lógico que indica si las Fecha/DateTime/DateTimeZone dadas se produjeron durante el año actual, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInNextDay	Indica si el valor datetime especificado <code>dateTime</code> se produce durante el día siguiente, determinado por la fecha y hora actuales del sistema.
Date.IsInNextMonth	Devuelve un valor lógico que indica si la fecha y hora especificadas se produjeron durante el mes siguiente, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInNextNDays	Indica si el valor datetime especificado <code>dateTime</code> se produce durante el siguiente número de días, determinado por la fecha y hora actuales del sistema.
Date.IsInNextNMonths	Indica si el valor datetime especificado se produce durante el siguiente número de meses, determinado por la fecha y hora actuales del sistema.
Date.IsInNextNQuarters	Indica si el valor datetime especificado <code>dateTime</code> se produce durante el siguiente número de trimestres, determinado por la fecha y hora actuales del sistema.
Date.IsInNextNWeeks	Indica si el valor datetime especificado <code>dateTime</code> se produce durante el siguiente número de semanas, determinado por la fecha y hora actuales del sistema.
Date.IsInNextNYears	Indica si el valor de fecha y hora especificado, <code>dateTime</code> , se produce durante el siguiente número de años, determinado por la fecha y la hora actuales del sistema.

Nombre	Descripción
Date.IsInNextQuarter	Devuelve un valor lógico que indica si la Fecha, Fecha y Hora, o Zona Horaria especificadas ocurrieron durante el próximo trimestre, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInNextWeek	Devuelve un valor lógico que indica si la Fecha, FechaHora o ZonaHoraria especificadas se produjeron durante la semana siguiente, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInNextYear	Devuelve un valor lógico que indica si la fecha, fecha y hora, o la zona horaria especificada ocurre durante el año siguiente, tal como lo determina la fecha y hora actuales del sistema.
Date.IsInPreviousDay	Indica si el valor datetime especificado tiene lugar durante el día anterior, determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousMonth	Devuelve un valor lógico que indica si la Fecha/FechaHora/ZonaHoraria especificada ocurrió durante el mes anterior, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousNDays	Indica si el valor de fecha y hora (dateTime) dado tiene lugar durante el número de días anteriores, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousNMonths	Indica si el valor de fecha y hora especificado ocurre durante los meses anteriores, según lo determina la fecha y hora actuales del sistema.
Date.IsInPreviousNQuarters	Indica si el valor de fecha y hora especificado, dateTime, ocurre durante el número anterior de trimestres, tal como se determina por la fecha y hora actuales del sistema.
Date.IsInPreviousNWeeks	Indica si el valor de fecha y hora dado se produce durante las semanas anteriores, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousNYears	Indica si el valor de fecha y hora dado, dateTime, ocurre durante los años previos, determinados por la fecha y hora actuales del sistema.
Date.IsInPreviousQuarter	Devuelve un valor lógico que indica si la Fecha/FechaHora/ZonaHoraria especificada ocurrió durante el trimestre anterior, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousWeek	Devuelve un valor lógico que indica si la Fecha/Fecha y Hora/Zona Horaria especificada ocurrió durante la semana anterior, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInPreviousYear	Devuelve un valor lógico que indica si la Fecha, Fecha y hora, o Zona horaria especificada ocurrió durante el año anterior, según lo determinado por la fecha y hora actuales del sistema.
Date.IsInYearToDate	Devuelve un valor lógico que indica si la fecha y hora especificadas se produjeron en el período que comienza el 1 de enero del año actual y

Nombre	Descripción
	finaliza en el día actual, según lo determinado por la fecha y hora actuales en el sistema.
Date.IsLeapYear	Devuelve un valor lógico que indica si la parte del año de un valor DateTime es un año bisiesto.
Date.Month	Devuelve el mes de un valor DateTime.
Date.MonthName	Devuelve el nombre del componente mes.
Date.QuarterOfYear	Devuelve un número comprendido entre 1 y 4 para el trimestre del año a partir de un valor DateTime.
Date.StartOfDay	Devuelve el inicio del día.
Date.StartOfMonth	Devuelve el inicio del mes.
Date.StartOfQuarter	Devuelve el inicio del trimestre.
Date.StartOfWeek	Devuelve el inicio de la semana.
Date.StartOfYear	Devuelve el inicio del año.
Date.ToRecord	Devuelve un registro que contiene partes de un valor de Fecha.
Date.ToText	Devuelve un valor de texto a partir de un valor de fecha.
Date.WeekOfMonth	Devuelve un número para el recuento de semanas del mes actual.
Date.WeekOfYear	Devuelve un número para el recuento de semanas del año actual.
Date.Year	Devuelve el año de un valor DateTime.
#date	Crea un valor de fecha a partir de año, mes y día.

Date.AddDays

Sintaxis

```
Date.AddDays(dateTime as any, numberOfDays as number) as any
```

Acerca de

Devuelve el resultado de `date`, `datetime` o `datetimezone` de sumar `numberOfDays` días al valor `datetime dateTime`.

- `dateTime`: valor de `date`, `datetime` o `datetimezone` al que se agregan días.
- `numberOfDays`: número de días que se van a agregar.

Ejemplo

Agregue 5 días al `date`, `datetime` o `datetimezone` valor que representa la fecha 14/5/2011.

Uso

Power Query M

```
Date.AddDays(#date(2011, 5, 14), 5)
```

Salida

```
#date(2011, 5, 19)
```

Contenido relacionado

- [#date](#)
- [#datetime](#)
- [#datetimezone](#)

Date.AddMonths

16/09/2025

Sintaxis

```
Date.AddMonths(dateTime as any, numberOfMonths as number) as any
```

Acerca de

Devuelve el resultado de `date`, `datetime` o `datetimezone` de sumar `numberOfMonths` meses al valor `datetime` `dateTime`.

- `dateTime`: valor de `date`, `datetime` o `datetimezone` al que se agregan meses.
- `numberOfMonths`: El número de meses a agregar.

Ejemplo 1

Agregue 5 meses al `date`, `datetime` o `datetimezone` valor que representa la fecha 14/5/2011.

Uso

Power Query M

```
Date.AddMonths(#date(2011, 5, 14), 5)
```

Salida

```
#date(2011, 10, 14)
```

Ejemplo 2

Agregue 18 meses al valor `date`, `datetime` o `datetimezone` que representa la fecha y hora del 14/5/2011 08:15:22 a.m.

Uso

Power Query M

```
Date.AddMonths(#datetime(2011, 5, 14, 8, 15, 22), 18)
```

Salida

```
#datetime(2012, 11, 14, 8, 15, 22)
```

Contenido relacionado

- [#date](#)
- [#datetime](#)
- [#datetimezone](#)

Date.AddQuarters

16/09/2025

Sintaxis

```
Date.AddQuarters(dateTime as any, numberOfQuarters as number) as any
```

Acerca de

Devuelve el resultado de `date`, `datetime` o `datetimezone` de sumar `numberOfQuarters` trimestres al valor `dateTime`.

- `dateTime`: valor `date`, `datetime` o `datetimezone` al que se van a sumar trimestres.
- `numberOfQuarters`: número de trimestres que se van a agregar.

Ejemplo 1

Agregue 1 trimestre al `date`, `datetime` o `datetimezone` valor que representa la fecha 14/5/2011.

Uso

Power Query M

```
Date.AddQuarters(#date(2011, 5, 14), 1)
```

Salida

```
#date(2011, 8, 14)
```

Contenido relacionado

- [#date](#)
- [#datetime](#)
- [#datetimezone](#)

Date.AddWeeks

16/09/2025

Sintaxis

```
Date.AddWeeks(dateTime as any, numberOfWeeks as number) as any
```

Acerca de

Devuelve el resultado de `date`, `datetime` o `datetimezone` de sumar `numberOfWeeks` semanas al valor `datetime dateTime`.

- `dateTime`: valor de `date`, `datetime` o `datetimezone` al que se agregan semanas.
- `numberOfWeeks`: número de semanas a agregar.

Ejemplo 1

Agregue 2 semanas al `date`, `datetime` o `datetimezone` valor que representa la fecha 14/5/2011.

Uso

Power Query M

```
Date.AddWeeks(#date(2011, 5, 14), 2)
```

Salida

```
#date(2011, 5, 28)
```

Contenido relacionado

- [#date](#)
- [#datetime](#)
- [#datetimezone](#)

Date.AddYears

16/09/2025

Sintaxis

```
Date.AddYears(dateTime as any, numberOfYears as number) as any
```

Acerca de

Devuelve el valor `date`, `datetime` o `datetimezone`, que es el resultado de sumar `numberOfYears` a un valor `datetime` `dateTime`.

- `dateTime`: valor de `date`, `datetime` o `datetimezone` al que se agregan años.
- `numberOfYears`: la cantidad de años que se van a añadir.

Ejemplo 1

Agregue 4 años al `date`, `datetime` o `datetimezone` valor que representa la fecha 14/5/2011.

Uso

Power Query M

```
Date.AddYears(#date(2011, 5, 14), 4)
```

Salida

```
#date(2015, 5, 14)
```

Ejemplo 2

Agregue 10 años al `date`, `datetime` o `datetimezone` valor que representa la fecha y hora del 14/5/2011 08:15:22 a. m.

Uso

Power Query M

```
Date.AddYears(#datetime(2011, 5, 14, 8, 15, 22), 10)
```

Salida

```
#datetime(2021, 5, 14, 8, 15, 22)
```

Contenido relacionado

- [#date](#)
- [#datetime](#)
- [#datetimezone](#)

Date.Day

Syntax

```
Date.Day(dateTime as any) as nullable number
```

Acerca de

Devuelve el componente day de un `date` valor , `datetime` o `datetimezone` .

- `dateTime`: valor `date`, `datetime` o `datetimezone` del que se extrae el componente de día.

Ejemplo 1

Obtenga el componente de día de un `date` valor , `datetime` o `datetimezone` que representa la fecha y hora del 14/5/2011 05:00:00 p. m.

Uso

Power Query M

```
Date.Day(#datetime(2011, 5, 14, 17, 0, 0))
```

Salida

14

Last updated on 22/11/2025

Date.DayOfWeek

Syntax

```
Date.DayOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Acerca de

Devuelve un número (de 0 a 6) que indica el día de la semana del proporcionado `dateTime`.

- `dateTime`: un `date` valor, `datetime` o `datetimezone`.
- `firstDayOfWeek` `Day`: valor que indica qué día se debe considerar el primer día de la semana. Los valores permitidos son [Day.Sunday](#), [Day.Monday](#), [Day.Tuesday](#), [Day.Wednesday](#), [Day.Thursday](#), [Day.Friday](#) o [Day.Saturday](#). Si no se especifica, se usa un valor predeterminado dependiente de la referencia cultural.

Ejemplo 1

Obtenga el día de la semana representado por el lunes, 21 de febrero de 2011, tratando el domingo como primer día de la semana.

Uso

Power Query M

```
Date.DayOfWeek(#date(2011, 02, 21), Day.Sunday)
```

Salida

1

Ejemplo 2

Obtenga el día de la semana representado por el lunes, 21 de febrero de 2011, tratando el lunes como primer día de la semana.

Uso

Power Query M

```
Date.DayOfWeek(#date(2011, 02, 21), Day.Monday)
```

Salida

0

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 02/04/2026

Date.DayOfWeekName

Syntax

```
Date.DayOfWeekName(date as any, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve el día del nombre de la semana para el proporcionado `date`. También se puede proporcionar una opción `culture` (por ejemplo, "en-US").

Ejemplo

Obtenga el día del nombre de la semana.

Uso

Power Query M

```
Date.DayOfWeekName(#date(2011, 12, 31), "en-US")
```

Salida

```
"Saturday"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 25/03/2026

Date.DayOfYear

Syntax

```
Date.DayOfYear(dateTime as any) as nullable number
```

Acerca de

Devuelve un número que representa el día del año en el `date` valor proporcionado , `datetime` o `datetimezone` , `dateTime`.

Ejemplo

El día del año para el 1 de marzo de 2011.

Uso

Power Query M

```
Date.DayOfYear(#date(2011, 03, 01))
```

Salida

```
60
```

Last updated on 25/03/2026

Date.DaysInMonth

Syntax

```
Date.DaysInMonth(dateTime as any) as nullable number
```

Acerca de

Devuelve el número de días del mes en el `date` valor `datetime`, `datetimezone` o `dateTime` .

- `dateTime`: valor `date`, `datetime` o `datetimezone` para el que se devuelve el número de días del mes.

Ejemplo

Número de días del mes de diciembre representado por `#date(2011, 12, 01)` .

Uso

Power Query M

```
Date.DaysInMonth(#date(2011, 12, 01))
```

Salida

```
31
```

Last updated on 25/03/2026

Date.EndOfDay

```
Date.EndOfDay(dateTime as any) as any
```

Acerca de

Devuelve el final del día representado por `dateTime`. Se conserva la información de zona horaria.

- `dateTime`: un `date` valor, `datetime` o `datetimezone` desde el que se calcula el final del día.

Ejemplo 1

Obtenga el final del día para las 14/5/2011 05:00:00 p. m.

Uso

Power Query M

```
Date.EndOfDay(#datetime(2011, 5, 14, 17, 0, 0))
```

Salida

```
#datetime(2011, 5, 14, 23, 59, 59.9999999)
```

Ejemplo 2

Obtenga el final del día para el 17/5/2011 05:00:00 -7:00.

Uso

Power Query M

```
Date.EndOfDay(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

Salida

```
#datetimezone(2011, 5, 17, 23, 59, 59.9999999, -7, 0)
```

Date.EndOfMonth

Syntax

```
Date.EndOfMonth(dateTime as any) as any
```

Acerca de

Devuelve el final del mes que contiene `dateTime`.

- `dateTime`: un `date` valor, `datetime` o `datetimezone` desde el que se calcula el final del mes.

Ejemplo 1

Obtenga el final del mes para el 14/5/2011.

Uso

Power Query M

```
Date.EndOfMonth(#date(2011, 5, 14))
```

Salida

```
#date(2011, 5, 31)
```

Ejemplo 2

Obtenga el final del mes para el 17/5/2011 05:00:00 -7:00.

Uso

Power Query M

```
Date.EndOfMonth(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

Salida

```
#datetimezone(2011, 5, 31, 23, 59, 59.9999999, -7, 0)
```

Date.EndOfQuarter

Syntax

```
Date.EndOfQuarter(dateTime as any) as any
```

Acerca de

Devuelve el final del trimestre que contiene `dateTime`. Se conserva la información de zona horaria.

- `dateTime`: un `date` valor, `datetime` o `datetimezone` desde el que se calcula el final del trimestre.

Ejemplo

Busque el final del trimestre para el 10 de octubre de 2011, 8:00 a. m.

Uso

Power Query M

```
Date.EndOfQuarter(#datetime(2011, 10, 10, 8, 0, 0))
```

Salida

```
#datetime(2011, 12, 31, 23, 59, 59.9999999)
```

Last updated on 25/03/2026

Date.EndOfWeek

Syntax

```
Date.EndOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as any
```

Acerca de

Devuelve el final de la semana que contiene `dateTime`. Esta función toma un valor opcional `Day`, `firstDayOfWeek`, para establecer como el primer día de la semana para este cálculo relativo. El valor predeterminado es [Day.Sunday](#).

- `dateTime`: un `date` valor, `datetime` o `datetimezone` desde el que se calcula el último día de la semana.
- `firstDayOfWeek`: *[Opcional]* Valor [Day.Type](#) que representa el primer día de la semana. Los valores posibles son `Day.Sunday`, `Day.Monday`, `Day.Tuesday`, `Day.Wednesday`, `Day.Thursday` y `Day.Saturday` `Day.Friday`. El valor predeterminado es `Day.Sunday`.

Ejemplo 1

Obtenga el final de la semana para el 14/5/2011.

Uso

Power Query M

```
Date.EndOfWeek(#date(2011, 5, 14))
```

Salida

```
#date(2011, 5, 14)
```

Ejemplo 2

Obtenga el final de la semana para el 17/5/2011 05:00:00 -7:00, con domingo como primer día de la semana.

Uso

Power Query M

```
Date.EndOfWeek(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0), Day.Sunday)
```

Salida

```
#datetimezone(2011, 5, 21, 23, 59, 59.9999999, -7, 0)
```

Last updated on 22/11/2025

Date.EndOfYear

Syntax

```
Date.EndOfYear(dateTime as any) as any
```

Acerca de

Devuelve el final del año que contiene `dateTime`, incluidas las fracciones de segundo. Se conserva la información de zona horaria.

- `dateTime`: un `date` valor, `datetime` o `datetimezone` desde el que se calcula el final del año.

Ejemplo 1

Obtener el final del año para 14/5/2011 05:00:00 p. m.

Uso

Power Query M

```
Date.EndOfYear(#datetime(2011, 5, 14, 17, 0, 0))
```

Salida

```
#datetime(2011, 12, 31, 23, 59, 59.9999999)
```

Ejemplo 2

Obtenga el final de la hora del 17/5/2011 05:00:00 -7:00.

Uso

Power Query M

```
Date.EndOfYear(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

Salida

```
#datetimezone(2011, 12, 31, 23, 59, 59.9999999, -7, 0)
```

Date.From

Sintaxis

```
Date.From(value as any, optional culture as nullable text) as nullable date
```

Acerca de

Devuelve un valor de fecha del valor especificado.

- **value**: valor que se va a cubrir en una fecha. Si el valor especificado es **null**, esta función devuelve **null**. Si el valor especificado es **date**, **value** se devuelve. Los valores de los siguientes tipos se pueden convertir en un **date** valor:
 - **text**: un valor **date** de la representación textual. Consulte [Date.FromText](#) para obtener más información.
 - **datetime**: el componente de fecha de **value**.
 - **datetimezone**: el componente de fecha del valor de fecha y hora locales equivalente de **value**.
 - **number**: componente de fecha del equivalente datetime de un número de punto flotante cuyo componente entero es el número de días antes o después de la medianoche, 30 de diciembre de 1899, y cuyo componente fraccionario representa la hora en ese día dividida por 24. Por ejemplo, la medianoche, el 31 de diciembre de 1899 está representada por 1,0; 6 A.M., 1 de enero de 1900 está representado por 2.25; medianoche, 29 de diciembre de 1899 está representada por -1.0; y 6 A.M., 29 de diciembre de 1899 están representados por -1.25. El valor base es medianoche, 30 de diciembre de 1899. El valor mínimo es medianoche, 1 de enero de 0100. El valor máximo es el último momento del 31 de diciembre de 9999.
 - Si **value** es de cualquier otro tipo, se devuelve un error.
- **culture**: la referencia cultural del valor especificado (por ejemplo, "en-US").

Ejemplo 1

Convierta la fecha y hora especificadas en un valor de fecha.

Uso

Power Query M

```
Date.From(#datetime(1899, 12, 30, 06, 45, 12))
```

Salida

```
#date(1899, 12, 30)
```

Ejemplo 2

Convierta el número especificado en un valor de fecha.

Uso

Power Query M

```
Date.From(43910)
```

Salida

```
#date(2020, 3, 20)
```

Ejemplo 3

Convierta las fechas de texto alemán en la columna Fecha publicada a valores de fecha.

Uso

Power Query M

```
let
    Source = #table(type table [Account Code = text, Posted Date = text, Sales = number],
        {
            {"US-2004", "20 Januar 2023", 580},
            {"CA-8843", "18 Juli, 2023", 280},
            {"PA-1274", "12 Januar, 2022", 90},
            {"PA-4323", "14 April 2023", 187},
            {"US-1200", "14 Dezember, 2022", 350},
            {"PTY-507", "4 Juni, 2023", 110}
        }
    ),
    #"Filtered rows" = Table.TransformColumns(
        Source,
        {"Posted Date", each Date.From(_, "de-DE"), type date}
    )
```

```
in
#"Filtered rows"
```

Salida

Power Query M

```
#table(type table [Account Code = text, Posted Date = date, Sales = number],
{
    {"US-2004", #date(2023, 1, 20), 580},
    {"CA-8843", #date(2023, 7, 18), 280},
    {"PA-1274", #date(2022, 1, 12), 90},
    {"PA-4323", #date(2023, 4, 14), 187},
    {"US-1200", #date(2022, 12, 14), 350},
    {"PTY-507", #date(2023, 6, 4), 110}
})
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas con formato de fecha y hora personalizado](#)

Last updated on 04/04/2026

Date.FromText

16/09/2025

Sintaxis

```
Date.FromText(text as nullable text, optional options as any) as nullable date
```

Acerca de

Crea un valor de fecha a partir de una representación textual.

- `text`: valor de texto que se va a cubrir en una fecha.
- `options`: opcional `record` que se puede proporcionar para especificar propiedades adicionales. `record` puede contener los campos siguientes:
 - `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, vaya a [Cadenas de formato](#) de fecha y hora estándar y [Cadenas de formato de fecha y hora personalizados](#). Omitir este campo o proporcionar `null` resultados al analizar la fecha con un mejor esfuerzo.
 - `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en `"en-US" "MMM"` es `"Jan", "Feb", "Mar", ...`, mientras que en `"ru-RU" "MMM"` es `"январь", "февраль", "март", ...`. Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` es `null` o se omite, se usa [Culture.Current](#).

Para admitir flujos de trabajo heredados, `options` también puede ser un valor de texto. Esto tiene el mismo comportamiento que `options = [Format = null, Culture = options]`.

Ejemplo 1

Convertir `"2010-12-31"` en un valor `date`.

Uso

Power Query M

```
Date.FromText("2010-12-31")
```

Salida

```
#date(2010, 12, 31)
```

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
Date.FromText("30 Dez 2010", [Format="dd MMM yyyy", Culture="de-DE"])
```

Salida

```
#date(2010, 12, 30)
```

Ejemplo 3

Busque la fecha en el calendario gregoriano que corresponde al principio de 1400 en el calendario Hijri.

Uso

Power Query M

```
Date.FromText("1400", [Format="yyyy", Culture="ar-SA"])
```

Salida

```
#date(1979, 11, 20)
```

Ejemplo 4

Convierta las fechas de texto italiano con meses abreviados en la columna Fecha publicada a valores de fecha.

Uso

Power Query M

```

let
    Source = #table(type table [Account Code = text, Posted Date = text, Sales =
number],
        {
            {"US-2004", "20 gen. 2023", 580},
            {"CA-8843", "18 lug. 2024", 280},
            {"PA-1274", "12 gen. 2023", 90},
            {"PA-4323", "14 apr. 2023", 187},
            {"US-1200", "14 dic. 2023", 350},
            {"PTY-507", "4 giu. 2024", 110}
        }
    ),
    #"Converted Date" = Table.TransformColumns(
        Source,
        {"Posted Date", each Date.FromText(_, [Culture = "it-IT"]), type date}
    )
in
    #"Converted Date"

```

Salida

Power Query M

```

#table(type table [Account Code = text, Posted Date = date, Sales = number],
{
    {"US-2004", #date(2023, 1, 20), 580},
    {"CA-8843", #date(2024, 7, 18), 280},
    {"PA-1274", #date(2023, 1, 12), 90},
    {"PA-4323", #date(2023, 4, 14), 187},
    {"US-1200", #date(2023, 12, 14), 350},
    {"PTY-507", #date(2024, 6, 4), 110}
})

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas con formato de fecha y hora personalizado](#)

Date.IsInCurrentDay

Syntax

```
Date.IsInCurrentDay(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el día actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Example

Determine si la hora actual del sistema está en el día actual.

Uso

Power Query M

```
Date.IsInCurrentDay(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInCurrentMonth

Syntax

```
Date.IsInCurrentMonth(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el mes actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en el mes actual.

Uso

Power Query M

```
Date.IsInCurrentMonth(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInCurrentQuarter

Syntax

```
Date.IsInCurrentQuarter(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el trimestre actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo

Determine si la hora actual del sistema está en el trimestre actual.

Uso

Power Query M

```
Date.IsInCurrentQuarter(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInCurrentWeek

Syntax

```
Date.IsInCurrentWeek(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la semana actual, según lo determinado por la fecha y hora actuales del sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en la semana actual.

Uso

Power Query M

```
Date.IsInCurrentWeek(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInCurrentYear

Syntax

```
Date.IsInCurrentYear(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el año actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en el año actual.

Uso

Power Query M

```
Date.IsInCurrentYear(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInNextDay

Syntax

```
Date.IsInNextDay(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el día siguiente, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el día actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo

Determine si el día después de la hora actual del sistema está en el día siguiente.

Uso

Power Query M

```
Date.IsInNextDay(Date.AddDays(DateTime.FixedLocalNow(), 1))
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInNextMonth

Syntax

```
Date.IsInNextMonth(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el mes siguiente, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el mes actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el mes después de la hora actual del sistema está en el mes siguiente.

Uso

Power Query M

```
Date.IsInNextMonth(Date.AddMonths(DateTime.FixedLocalNow(), 1))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInNextNDays

Syntax

```
Date.IsInNextNDays(dateTime as any, days as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de días, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el día actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `days`: número de días.

Ejemplo 1

Determine si el día después de la hora actual del sistema se encuentra en los dos días siguientes.

Uso

Power Query M

```
Date.IsInNextNDays(Date.AddDays(DateTime.FixedLocalNow(), 1), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInNextNMonths

Sintaxis

```
Date.IsInNextNMonths(dateTime as any, months as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de meses, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el mes actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `months`: número de meses.

Ejemplo

Determine si el mes después de la hora actual del sistema está en los próximos dos meses.

Uso

Power Query M

```
Date.IsInNextNMonths(Date.AddMonths(DateTime.FixedLocalNow(), 1), 2)
```

Salida

```
true
```

Last updated on 24/03/2026

Date.IsInNextNQuarters

Syntax

```
Date.IsInNextNQuarters(dateTime as any, quarters as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de trimestres, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el trimestre actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `quarters`: número de trimestres.

Ejemplo 1

Determine si el trimestre después de la hora actual del sistema está en los dos trimestres siguientes.

Uso

Power Query M

```
Date.IsInNextNQuarters(Date.AddQuarters(DateTime.FixedLocalNow(), 1), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInNextNWeeks

Syntax

```
Date.IsInNextNWeeks(dateTime as any, weeks as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de semanas, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produce dentro de la semana actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `weeks`: número de semanas.

Ejemplo

Determine si la semana después de la hora actual del sistema se encuentra en las dos semanas siguientes.

Uso

Power Query M

```
Date.IsInNextNWeeks(Date.AddDays(DateTime.FixedLocalNow(), 7), 2)
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInNextNYears

Syntax

```
Date.IsInNextNYears(dateTime as any, years as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el siguiente número de años, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca dentro del año actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `years`: número de años.

Ejemplo

Determine si el año después de la hora actual del sistema está en los próximos dos años.

Uso

Power Query M

```
Date.IsInNextNYears(Date.AddYears(DateTime.FixedLocalNow(), 1), 2)
```

Salida

true

Last updated on 25/03/2026

Date.IsInNextQuarter

Syntax

```
Date.IsInNextQuarter(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el trimestre siguiente, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el trimestre actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el trimestre después de la hora actual del sistema se encuentra en el trimestre siguiente.

Uso

Power Query M

```
Date.IsInNextQuarter(Date.AddQuarters(DateTime.FixedLocalNow(), 1))
```

Salida

true

Last updated on 22/11/2025

Date.IsInNextWeek

Syntax

```
Date.IsInNextWeek(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la próxima semana, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produce dentro de la semana actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la semana después de la hora actual del sistema está en la semana siguiente.

Uso

Power Query M

```
Date.IsInNextWeek(Date.AddDays(DateTime.FixedLocalNow(), 7))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInNextYear

Syntax

```
Date.IsInNextYear(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el próximo año, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca dentro del año actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo

Determine si el año después de la hora actual del sistema está en el próximo año.

Uso

Power Query M

```
Date.IsInNextYear(Date.AddYears(DateTime.FixedLocalNow(), 1))
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInPreviousDay

Syntax

```
Date.IsInPreviousDay(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el día anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el día actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el día anterior a la hora actual del sistema está en el día anterior.

Uso

Power Query M

```
Date.IsInPreviousDay(Date.AddDays(DateTime.FixedLocalNow(), -1))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInPreviousMonth

Syntax

```
Date.IsInPreviousMonth(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el mes anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el mes actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el mes anterior a la hora actual del sistema está en el mes anterior.

Uso

Power Query M

```
Date.IsInPreviousMonth(Date.AddMonths(DateTime.FixedLocalNow(), -1))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInPreviousNDays

Syntax

```
Date.IsInPreviousNDays(dateTime as any, days as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el número anterior de días, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el día actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `days`: número de días.

Ejemplo

Determine si el día anterior a la hora actual del sistema está en los dos días anteriores.

Uso

Power Query M

```
Date.IsInPreviousNDays(Date.AddDays(DateTime.FixedLocalNow(), -1), 2)
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInPreviousNMonths

Sintaxis

```
Date.IsInPreviousNMonths(dateTime as any, months as number) as nullable logical
```

Acerca de

Indica si el valor de fecha y hora especificado `dateTime` ocurre durante los meses anteriores, según la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá `false` cuando se pasa un valor que se produce en el mes actual.

- `dateTime`: un valor de `date`, `datetime` o `datetimezone` que se va a evaluar.
- `months`: número de meses.

Ejemplo

Determine si el mes anterior al tiempo actual del sistema está dentro de los dos meses previos.

Uso

Power Query M

```
Date.IsInPreviousNMonths(Date.AddMonths(DateTime.FixedLocalNow(), -1), 2)
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsInPreviousNQuarters

16/09/2025

Sintaxis

```
Date.IsInPreviousNQuarters(dateTime as any, quarters as number) as nullable logical
```

Acerca de

Indica si el valor de fecha y hora especificado `dateTime` se produce durante el número de trimestres anteriores, según lo determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el trimestre actual.

- `dateTime`: un valor de `date`, `datetime` o `datetimezone` que se va a evaluar.
- `quarters`: número de trimestres.

Ejemplo 1

Determine si el trimestre que precede a la hora actual del sistema está dentro de los dos trimestres anteriores.

Uso

Power Query M

```
Date.IsInPreviousNQuarters(Date.AddQuarters(DateTime.FixedLocalNow(), -1), 2)
```

Salida

```
true
```

Date.IsInPreviousNWeeks

Syntax

```
Date.IsInPreviousNWeeks(dateTime as any, weeks as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el número anterior de semanas, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produce dentro de la semana actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `weeks`: número de semanas.

Ejemplo 1

Determine si la semana anterior a la hora actual del sistema está en las dos semanas anteriores.

Uso

Power Query M

```
Date.IsInPreviousNWeeks(Date.AddDays(DateTime.FixedLocalNow(), -7), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInPreviousNYears

Syntax

```
Date.IsInPreviousNYears(dateTime as any, years as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el número anterior de años, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca dentro del año actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.
- `years`: número de años.

Ejemplo 1

Determine si el año anterior a la hora actual del sistema está en los dos años anteriores.

Uso

Power Query M

```
Date.IsInPreviousNYears(Date.AddYears(DateTime.FixedLocalNow(), -1), 2)
```

Salida

true

Last updated on 22/11/2025

Date.IsInPreviousQuarter

Syntax

```
Date.IsInPreviousQuarter(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el trimestre anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce en el trimestre actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el trimestre anterior a la hora actual del sistema se encuentra en el trimestre anterior.

Uso

Power Query M

```
Date.IsInPreviousQuarter(Date.AddQuarters(DateTime.FixedLocalNow(), -1))
```

Salida

true

Last updated on 22/11/2025

Date.IsInPreviousWeek

Syntax

```
Date.IsInPreviousWeek(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la semana anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produce dentro de la semana actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la semana anterior a la hora actual del sistema está en la semana anterior.

Uso

Power Query M

```
Date.IsInPreviousWeek(Date.AddDays(DateTime.FixedLocalNow(), -7))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInPreviousYear

Syntax

```
Date.IsInPreviousYear(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el año anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca dentro del año actual.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el año anterior a la hora actual del sistema está en el año anterior.

Uso

Power Query M

```
Date.IsInPreviousYear(Date.AddYears(DateTime.FixedLocalNow(), -1))
```

Salida

```
true
```

Last updated on 22/11/2025

Date.IsInYearToDate

Syntax

```
Date.IsInYearToDate(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el año actual y está activado o antes del día actual, según lo determinado por la fecha y hora actuales en el sistema.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo

Determine si la hora actual del sistema está en el año hasta la fecha.

Uso

Power Query M

```
Date.IsInYearToDate(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 25/03/2026

Date.IsLeapYear

Syntax

```
Date.IsLeapYear(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado cae en es un año bisiesto.

- `dateTime`: valor `date`, `datetime` o `datetimezone` que se va a evaluar.

Ejemplo

Determine si el año 2012, representado por `#date(2012, 01, 01)` es un año bisiesto.

Uso

Power Query M

```
Date.IsLeapYear(#date(2012, 01, 01))
```

Salida

```
true
```

Last updated on 25/03/2026

Date.Month

Syntax

```
Date.Month(dateTime as any) as nullable number
```

Acerca de

Devuelve el componente month del valor proporcionado `datetime` , `dateTime`.

Ejemplo 1

Busque el mes en `#datetime(2011, 12, 31, 9, 15, 36)`.

Uso

Power Query M

```
Date.Month(#datetime(2011, 12, 31, 9, 15, 36))
```

Salida

```
12
```

Last updated on 22/11/2025

Date.MonthName

Syntax

```
Date.MonthName(date as any, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve el nombre del componente de mes para el proporcionado `date`. También se puede proporcionar una opción `culture` (por ejemplo, "en-US").

Ejemplo 1

Obtenga el nombre del mes.

Uso

Power Query M

```
Date.MonthName(#datetime(2011, 12, 31, 5, 0, 0), "en-US")
```

Salida

```
"December"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Date.QuarterOfYear

Syntax

```
Date.QuarterOfYear(dateTime as any) as nullable number
```

Acerca de

Devuelve un número comprendido entre 1 y 4 que indica en qué trimestre del año cae la fecha `dateTime` . `dateTime` puede ser un `date` valor , `datetime` o `datetimezone` .

Ejemplo 1

Busque en qué trimestre del año cae la fecha `#date(2011, 12, 31)`.

Uso

Power Query M

```
Date.QuarterOfYear(#date(2011, 12, 31))
```

Salida

4

Last updated on 22/11/2025

Date.StartOfDay

Syntax

```
Date.StartOfDay(dateTime as any) as any
```

Acerca de

Devuelve el inicio del día representado por `dateTime`. `dateTime` debe ser un `date` valor , `datetime` o `datetimezone` .

Ejemplo 1

Busque el inicio del día del 10 de octubre de 2011, 8:00 a. m.

Uso

Power Query M

```
Date.StartOfDay(#datetime(2011, 10, 10, 8, 0, 0))
```

Salida

```
#datetime(2011, 10, 10, 0, 0, 0)
```

Last updated on 22/11/2025

Date.StartOfMonth

Syntax

```
Date.StartOfMonth(dateTime as any) as any
```

Acerca de

Devuelve el inicio del mes que contiene `dateTime`. `dateTime` debe ser un `date` valor o `datetime`.

Ejemplo 1

Busque el inicio del mes para el 10 de octubre de 2011, 8:10:32AM.

Uso

Power Query M

```
Date.StartOfMonth(#datetime(2011, 10, 10, 8, 10, 32))
```

Salida

```
#datetime(2011, 10, 1, 0, 0, 0)
```

Last updated on 22/11/2025

Date.StartOfQuarter

Syntax

```
Date.StartOfQuarter(dateTime as any) as any
```

Acerca de

Devuelve el inicio del trimestre que contiene `dateTime`. `dateTime` debe ser un `date` valor , `datetime` o `datetimezone` .

Ejemplo 1

Busque el inicio del trimestre para el 10 de octubre de 2011, 8:00 a. m.

Uso

Power Query M

```
Date.StartOfQuarter(#datetime(2011, 10, 10, 8, 0, 0))
```

Salida

```
#datetime(2011, 10, 1, 0, 0, 0)
```

Last updated on 22/11/2025

Date.StartOfWeek

Syntax

```
Date.StartOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as any
```

Acerca de

Devuelve el inicio de la semana que contiene `dateTime`. `dateTime` debe ser un `date` valor, `datetime` o `datetimezone`.

Ejemplo 1

Busque el inicio de la semana del martes 11 de octubre de 2011.

Uso

Power Query M

```
Date.StartOfWeek(#datetime(2011, 10, 11, 8, 10, 32))
```

Salida

Power Query M

```
// Sunday, October 9th, 2011  
#datetime(2011, 10, 9, 0, 0, 0)
```

Ejemplo 2

Busque el inicio de la semana para martes, 11 de octubre de 2011, usando lunes como inicio de la semana.

Uso

Power Query M

```
Date.StartOfWeek(#datetime(2011, 10, 11, 8, 10, 32), Day.Monday)
```

Salida

Power Query M

```
// Monday, October 10th, 2011  
#datetime(2011, 10, 10, 0, 0, 0)
```

Last updated on 22/11/2025

Date.StartOfYear

Syntax

```
Date.StartOfYear(dateTime as any) as any
```

Acerca de

Devuelve el inicio del año que contiene `dateTime`. `dateTime` debe ser un `date` valor , `datetime` o `datetimezone` .

Ejemplo

Busque el inicio del año para el 10 de octubre de 2011, 8:10:32 a. m.

Uso

Power Query M

```
Date.StartOfYear(#datetime(2011, 10, 10, 8, 10, 32))
```

Salida

```
#datetime(2011, 1, 1, 0, 0, 0)
```

Last updated on 25/03/2026

Date.ToRecord

Syntax

```
Date.ToRecord(date as date) as record
```

Acerca de

Devuelve un registro que contiene las partes del valor de fecha especificado, **date**.

- date** **date**: valor para el que se va a calcular el registro de sus partes.

Ejemplo

Convierta el **#date(2011, 12, 31)** valor en un registro que contenga partes del valor de fecha.

Uso

Power Query M

```
Date.ToRecord(#date(2011, 12, 31))
```

Salida

Power Query M

```
[  
    Year = 2011,  
    Month = 12,  
    Day = 31  
]
```

Last updated on 25/03/2026

Date.ToText

Sintaxis

```
Date.ToText(  
    date as nullable date,  
    optional options as any,  
    optional culture as nullable text  
) as nullable text
```

Acerca de

Devuelve una representación textual de `date`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `culture` solo se usa para flujos de trabajo heredados. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, ve a [Cadenas con formato de fecha y hora estándar](#) y [Cadenas con formato de fecha y hora personalizado](#). Si se omite este campo o se proporciona `null`, se dará formato a la fecha con el valor predeterminado definido por `Culture`.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "MMM" es "Jan", "Feb", "Mar", ..., mientras que en "ru-RU" "MMM" es "янв", "фев", "мар", Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` es `null` o se omite, se usa [Culture.Current](#).

Para admitir flujos de trabajo heredados, `options` y `culture` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = options, Culture = culture]`.

Ejemplo 1

Convertir `#date(2010, 12, 31)` en un valor `text`. *La salida del resultado puede variar en función de la referencia cultural actual.*

Uso

```
Date.ToText(#date(2010, 12, 31))
```

Salida

```
"12/31/2010"
```

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
Date.ToText(#date(2010, 12, 31), [Format="dd MMM yyyy", Culture="de-DE"])
```

Salida

```
"31 Dez 2010"
```

Ejemplo 3

Busque el año en el calendario Hijri correspondiente al 1 de enero de 2000 en el calendario gregoriano.

Uso

Power Query M

```
Date.ToText(#date(2000, 1, 1), [Format="yyyy", Culture="ar-SA"])
```

Salida

```
"1420"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
 - [cadenas de formato de fecha y hora estándar](#)
 - [Cadenas con formato de fecha y hora personalizado](#)
-

Date.WeekOfMonth

Syntax

```
Date.WeekOfMonth(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Acerca de

Devuelve un número comprendido entre 1 y 6 que indica en qué semana del mes entra la fecha `dateTime` .

- `dateTime` `datetime`: valor para el que se determina la semana del mes.

Ejemplo

Determine la semana del 15 de marzo en 2011.

Uso

Power Query M

```
Date.WeekOfMonth(#date(2011, 03, 15))
```

Salida

3

Last updated on 25/03/2026

Date.WeekOfYear

Syntax

```
Date.WeekOfYear(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number
```

Acerca de

Devuelve un número de 1 a 54 que indica la semana del año en que se encuentra, `dateTime`, .

- `dateTime` `datetime`: valor para el que se determina la semana del año.
- `firstDayOfWeek`: valor opcional [Day.Type](#) que indica qué día se considera el inicio de una nueva semana (por ejemplo, `Day.Sunday`). Si no se especifica, se usa un valor predeterminado dependiente de la referencia cultural.

Ejemplo 1

Determine la semana del año que contiene el 27 de marzo de 2011.

Uso

Power Query M

```
Date.WeekOfYear(#date(2011, 03, 27))
```

Salida

14

Ejemplo 2

Determine qué semana del año contiene el 27 de marzo de 2011, usando lunes como inicio de la semana.

Uso

Power Query M

```
Date.WeekOfYear(#date(2011, 03, 27), Day.Monday)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Date.Year

Sintaxis

```
Date.Year(dateTime as any) as nullable number
```

Acerca de

Devuelve el componente year del valor proporcionado `datetime` , `dateTime` .

Ejemplo

Busque el año en `#datetime(2011, 12, 31, 9, 15, 36)`.

Uso

Power Query M

```
Date.Year(#datetime(2011, 12, 31, 9, 15, 36))
```

Salida

```
2011
```

Last updated on 25/03/2026

#date

16/09/2025

Sintaxis

```
#date(  
    year as number,  
    month as number,  
    day as number  
) as date
```

Acerca de

Crea un valor de fecha a partir de números enteros que representan el año, mes y día. Genera un error si no se cumplen estas condiciones:

- $1 \leq \text{año} \leq 9999$
- $1 \leq \text{mes} \leq 12$
- $1 \leq \text{día} \leq 31$

Ejemplo 1

Cree una fecha que represente el 26 de diciembre de 2023.

Uso

Power Query M

```
#date(2023, 12, 26)
```

Salida

```
#date(2023, 12, 26)
```

Ejemplo 2

Convertir una fecha en texto con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
Date.ToText(#date(2023, 12, 26), [Format="dd MMM yyyy", Culture="de-DE"])
```

Salida

"26 Dez 2023"

Ejemplo 3

Obtenga las filas de una tabla que contenga una fecha en 2023.

Uso

Power Query M

```
let
Source = #table(type table [Account Code = text, Posted Date = date, Sales =
number],
    {
        {"US-2004", #date(2023,1,20), 580},
        {"CA-8843", #date(2023,7,18), 280},
        {"PA-1274", #date(2022,1,12), 90},
        {"PA-4323", #date(2023,4,14), 187},
        {"US-1200", #date(2022,12,14), 350},
        {"PTY-507", #date(2023,6,4), 110}
    }),
#"Filtered rows" = Table.SelectRows(
    Source,
    each Date.Year([Posted Date]) = 2023
)
in
#"Filtered rows"
```

Salida

Power Query M

```
#table (type table [Account Code = text, Posted Date = date, Sales = number],
{
    {"US-2004", #date(2023, 1, 20), 580},
    {"CA-8843", #date(2023, 7, 18), 280},
    {"PA-4323", #date(2023, 4, 14), 187},
    {"PTY-507", #date(2023, 6, 4), 110}
})
```

Funciones DateTime

Artículo • 23/11/2024

Estas funciones crean y manipulan valores datetime y datetimezone.

 Expandir tabla

Nombre	Descripción
DateTime.AddZone	Agrega la información de zona horaria al valor de fecha y hora.
DateTime.Date	Devuelve el componente de fecha del valor date, datetime o datetimezone especificado.
DateTime.FixedLocalNow	Devuelve la fecha y hora actuales en la zona horaria local. Este valor es fijo y no cambia con llamadas sucesivas.
DateTime.From	Crea una fecha y hora a partir del valor especificado.
DateTime.FromFileTime	Crea una fecha y hora a partir de un número largo de 64 bits.
DateTime.FromText	Crea un valor datetime a partir de formatos de datetime local y universal.
DateTime.IsInCurrentHour	Indica si esta fecha y hora se produce durante la hora actual, determinada por la fecha y hora actuales del sistema.
DateTime.IsInCurrentMinute	Indica si esta fecha y hora se produce durante el minuto actual, según lo determinado por la fecha y hora actuales en el sistema.
DateTime.IsInCurrentSecond	Indica si esta fecha y hora se produce durante el segundo actual, según lo determinado por la fecha y hora actuales en el sistema.
DateTime.IsInNextHour	Indica si esta fecha y hora se produce durante la hora siguiente, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro de la hora actual.
DateTime.IsInNextMinute	Indica si esta fecha y hora se produce durante el minuto siguiente, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce en el minuto actual.
DateTime.IsInNextNHours	Indica si esta fecha y hora se produce durante el siguiente número de horas, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro de la hora actual.

Nombre	Descripción
DateTime.IsInNextNMinutes	Indica si esta fecha y hora se produce durante el siguiente número de minutos, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce en el minuto actual.
DateTime.IsInNextNSeconds	Indica si esta fecha y hora se produce durante el siguiente número de segundos, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro del segundo actual.
DateTime.IsInNextSecond	Indica si esta fecha y hora se produce durante el segundo siguiente, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro del segundo actual.
DateTime.IsInPreviousHour	Indica si esta fecha y hora se produce durante la hora anterior, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro de la hora actual.
DateTime.IsInPreviousMinute	Indica si esta fecha y hora se produce durante el minuto anterior, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce en el minuto actual.
DateTime.IsInPreviousNHours	Indica si esta fecha y hora se produce durante el número anterior de horas, según lo determinado por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro de la hora actual.
DateTime.IsInPreviousNMinutes	Indica si esta fecha y hora se produce durante el número anterior de minutos, según lo determinado por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce en el minuto actual.
DateTime.IsInPreviousNSeconds	Indica si esta fecha y hora se produce durante el número de segundos anterior, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro del segundo actual.
DateTime.IsInPreviousSecond	Indica si esta fecha y hora se produce durante el segundo anterior, determinada por la fecha y hora actuales del sistema. Esta función devuelve <code>false</code> cuando se pasa un valor que se produce dentro del segundo actual.
DateTime.LocalNow	Devuelve la fecha y hora actuales en la zona horaria local.
DateTime.Time	Devuelve la parte de hora del valor datetime especificado.

Nombre	Descripción
DateTime.ToRecord	Devuelve un registro que contiene las partes del valor datetime.
DateTime.ToText	Devuelve una representación textual del valor datetime.
#datetime	Crea un valor DateTime de año, mes, día, hora, minuto y segundo.

Comentarios

¿Le ha resultado útil esta página?



Sí



No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

DateTime.AddZone

Syntax

```
DateTime.AddZone(  
    dateTime as nullable datetime,  
    timezoneHours as number,  
    optional timezoneMinutes as nullable number  
) as nullable datetimetype
```

Acerca de

Agrega información de zona horaria al `dateTime` valor. La información de zona horaria incluye `timezoneHours` y, opcionalmente `timezoneMinutes`, que especifica el desplazamiento deseado a partir de la hora UTC.

Ejemplo 1

Establezca la zona horaria en UTC+7:30 (7 horas y 30 minutos después de la HORA UTC).

Uso

Power Query M

```
DateTime.AddZone(#datetime(2010, 12, 31, 11, 56, 02), 7, 30)
```

Salida

```
#datetimezone(2010, 12, 31, 11, 56, 2, 7, 30)
```

Last updated on 22/11/2025

DateTime.Date

16/09/2025

Sintaxis

```
DateTime.Date(dateTime as any) as nullable date
```

Acerca de

Devuelve el componente de fecha del parámetro `dateTime` si el parámetro es un `date`, `datetime` o `datetimezone` valor, o `null` si el parámetro es `null`.

Ejemplo 1

Busque el valor de fecha de `#datetime(2010, 12, 31, 11, 56, 02)`.

Uso

Power Query M

```
DateTime.Date(#datetime(2010, 12, 31, 11, 56, 02))
```

Salida

```
#date(2010, 12, 31)
```

DateTime.FixedLocalNow

16/09/2025

Sintaxis

```
DateTime.FixedLocalNow() as datetime
```

Acerca de

Devuelve un valor `datetime` establecido en la fecha y hora actuales del sistema. Este valor es fijo y no cambiará con llamadas sucesivas, a diferencia de [DateTime.LocalNow](#), que puede devolver valores diferentes durante el transcurso de la ejecución de una expresión.

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

DateTime.From

Syntax

```
DateTime.From(value as any, optional culture as nullable text) as nullable datetime
```

Acerca de

Devuelve un `datetime` valor de la clase especificada `value`. También se puede proporcionar una opción `culture` (por ejemplo, "en-US"). Si el objeto especificado `value` es `null`, **DateTime.From** devuelve `null`. Si el objeto especificado `value` es `datetime`, `value` se devuelve. Los valores de los siguientes tipos se pueden convertir en un `datetime` valor:

- `text`: un `datetime` valor de representación textual. Consulte [DateTime.FromText](#) para obtener más información.
- `date`: con `datetime value` como componente de fecha y `12:00:00 AM` como componente de hora.
- `datetimezone`: equivalente local `datetime` de `value`.
- `time datetime`: con la fecha equivalente de la fecha de automatización OLE de `0` como componente de fecha y `value` como componente de hora.
- `number`: equivalente `datetime` de la fecha de automatización OLE expresada por `value`.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo 1

Convertir `#time(06, 45, 12)` en un `datetime` valor.

Uso

Power Query M

```
DateTime.From(#time(06, 45, 12))
```

Salida

```
#datetime(1899, 12, 30, 06, 45, 12)
```

Ejemplo 2

Convertir `#date(1975, 4, 4)` en un `datetime` valor.

Uso

Power Query M

```
DateTime.From(#date(1975, 4, 4))
```

Salida

```
#datetime(1975, 4, 4, 0, 0, 0)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)

Last updated on 22/11/2025

DateTime.FromFileTime

Syntax

```
DateTime.FromFileTime(fileTime as nullable number) as nullable datetime
```

Acerca de

Crea un `datetime` valor a partir del `fileTime` valor y lo convierte en la zona horaria local.

Filetime es un valor de hora de archivo de Windows que representa el número de intervalos de 100 nanosegundos transcurridos desde la medianoche 12:00, 1 de enero de 1601 A.D. (C.E.)

Hora universal coordinada (UTC).

Ejemplo

Convierta `129876402529842245` en un valor `datetime`.

Uso

Power Query M

```
DateTime.FromFileTime(129876402529842245)
```

Salida

```
#datetime(2012, 7, 24, 14, 50, 52.9842245)
```

Last updated on 25/03/2026

DateTime.FromText

Sintaxis

```
DateTime.FromText(text as nullable text, optional options as any) as nullable datetime
```

Acerca de

Crea un valor `datetime` a partir de una representación textual, `text`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, vaya a [Cadenas de formato de fecha y hora estándar](#) y [Cadenas de formato de fecha y hora personalizados](#). Si se omite este campo o se proporciona `null`, se analizará la fecha con el mejor esfuerzo.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "MMM" es "Jan", "Feb", "Mar", ..., mientras que en "ru-RU" "MMM" es "янв", "фев", "мар", Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` se `null` omite, [Culture.Current](#) se usa .

Para admitir flujos de trabajo heredados, `options` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = null, Culture = options]`.

Ejemplo 1

Convertir "2010-12-31T01:30:00" en un valor datetime.

Uso

Power Query M

```
DateTime.FromText("2010-12-31T01:30:25")
```

Salida

```
#datetime(2010, 12, 31, 1, 30, 25)
```

Ejemplo 2

Convertir "2010-12-31T01:30:00.121212" en un valor datetime.

Uso

Power Query M

```
DateTime.FromText("30 Dez 2010 02:04:50.369730", [Format="dd MMM yyyy  
HH:mm:ss.ffffff", Culture="de-DE"])
```

Salida

```
#datetime(2010, 12, 30, 2, 4, 50.36973)
```

Ejemplo 3

Convertir "2010-12-31T01:30:00" en un valor datetime.

Uso

Power Query M

```
DateTime.FromText("2000-02-08T03:45:12Z", [Format="yyyy-MM-dd'T'HH:mm:ss'Z'",  
Culture="en-US"])
```

Salida

```
#datetime(2000, 2, 8, 3, 45, 12)
```

Ejemplo 4

Convertir "20101231T013000" en un valor datetime.

Uso

Power Query M

```
DateTime.FromText("20101231T013000", [Format="yyyyMMdd'T'HHmmss", Culture="en-US"])
```

Salida

```
#datetime(2010, 12, 31, 1, 30, 0)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
 - [Cadenas de formato de fecha y hora estándar](#)
 - [Cadenas de formato de fecha y hora personalizadas](#)
-

Last updated on 22/11/2025

DateTime.IsInCurrentHour

Syntax

```
DateTime.IsInCurrentHour(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la hora actual, según lo determinado por la fecha y hora actuales del sistema.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en la hora actual.

Uso

Power Query M

```
DateTime.IsInCurrentHour(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInCurrentMinute

Syntax

```
DateTime.IsInCurrentMinute(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el minuto actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en el minuto actual.

Uso

Power Query M

```
DateTime.IsInCurrentMinute(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInCurrentSecond

Syntax

```
DateTime.IsInCurrentSecond(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el segundo actual, determinado por la fecha y hora actuales del sistema.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora actual del sistema está en el segundo actual.

Uso

Power Query M

```
DateTime.IsInCurrentSecond(DateTime.FixedLocalNow())
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInNextHour

Syntax

```
DateTime.IsInNextHour(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la próxima hora, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro de la hora actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo

Determine si la hora después de la hora actual del sistema se encuentra en la hora siguiente.

Uso

Power Query M

```
DateTime.IsInNextHour(DateTime.FixedLocalNow() + #duration(0, 1, 0, 0))
```

Salida

```
true
```

Last updated on 25/03/2026

DateTime.IsInNextMinute

Syntax

```
DateTime.IsInNextMinute(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el minuto siguiente, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca en el minuto actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo

Determine si el minuto después de la hora actual del sistema está en el minuto siguiente.

Uso

Power Query M

```
DateTime.IsInNextMinute(DateTime.FixedLocalNow() + #duration(0, 0, 1, 0))
```

Salida

```
true
```

Last updated on 25/03/2026

DateTime.IsInNextNHours

Syntax

```
DateTime.IsInNextNHours(dateTime as any, hours as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de horas, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro de la hora actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.
- `hours`: número de horas.

Ejemplo 1

Determine si la hora después de la hora actual del sistema se encuentra en las dos horas siguientes.

Uso

Power Query M

```
DateTime.IsInNextNHours(DateTime.FixedLocalNow() + #duration(0, 2, 0, 0), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInNextNMinutes

Syntax

```
DateTime.IsInNextNMinutes(dateTime as any, minutes as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de minutos, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca en el minuto actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.
- `minutes`: número de minutos.

Ejemplo 1

Determine si el minuto después de la hora actual del sistema está en los próximos dos minutos.

Uso

Power Query M

```
DateTime.IsInNextNMinutes(DateTime.FixedLocalNow() + #duration(0, 0, 2, 0), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInNextNSeconds

Syntax

```
DateTime.IsInNextNSeconds(dateTime as any, seconds as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el siguiente número de segundos, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro del segundo actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.
- `seconds`: número de segundos.

Ejemplo 1

Determine si el segundo después de la hora del sistema actual se encuentra en los dos segundos siguientes.

Uso

Power Query M

```
DateTime.IsInNextNSeconds(DateTime.FixedLocalNow() + #duration(0, 0, 0, 2), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInNextSecond

Syntax

```
DateTime.IsInNextSecond(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el segundo siguiente, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro del segundo actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el segundo después de la hora actual del sistema está en el segundo siguiente.

Uso

Power Query M

```
DateTime.IsInNextSecond(DateTime.FixedLocalNow() + #duration(0, 0, 0, 1))
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInPreviousHour

Syntax

```
DateTime.IsInPreviousHour(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante la hora anterior, según lo determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro de la hora actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si la hora anterior a la hora actual del sistema está en la hora anterior.

Uso

Power Query M

```
DateTime.IsInPreviousHour(DateTime.FixedLocalNow() - #duration(0, 1, 0, 0))
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInPreviousMinute

Syntax

```
DateTime.IsInPreviousMinute(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` de fecha y hora especificado se produce durante el minuto anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca en el minuto actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el minuto anterior a la hora actual del sistema está en el minuto anterior.

Uso

Power Query M

```
DateTime.IsInPreviousMinute(DateTime.FixedLocalNow() - #duration(0, 0, 1, 0))
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInPreviousNHours

Syntax

```
DateTime.IsInPreviousNHours(dateTime as any, hours as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el número anterior de horas, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro de la hora actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.
- `hours`: número de horas.

Ejemplo

Determine si la hora anterior a la hora actual del sistema está en las dos horas anteriores.

Uso

Power Query M

```
DateTime.IsInPreviousNHours(DateTime.FixedLocalNow() - #duration(0, 2, 0, 0), 2)
```

Salida

```
true
```

Last updated on 25/03/2026

DateTime.IsInPreviousNMinutes

Syntax

```
DateTime.IsInPreviousNMinutes(dateTime as any, minutes as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el número anterior de minutos, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pase un valor que se produzca en el minuto actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.
- `minutes`: número de minutos.

Ejemplo

Determine si el minuto anterior a la hora actual del sistema está en los dos minutos anteriores.

Uso

Power Query M

```
DateTime.IsInPreviousNMinutes(DateTime.FixedLocalNow() - #duration(0, 0, 2, 0), 2)
```

Salida

true

Last updated on 25/03/2026

DateTime.IsInPreviousNSeconds

Syntax

```
DateTime.IsInPreviousNSeconds(dateTime as any, seconds as number) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el número de segundos anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro del segundo actual.

- `dateTime`: un `datetime` >valor o `datetimezone` que se va a evaluar.
- `seconds`: número de segundos.

Ejemplo 1

Determine si la segunda antes de la hora actual del sistema se encuentra en los dos segundos anteriores.

Uso

Power Query M

```
DateTime.IsInPreviousNSeconds(DateTime.FixedLocalNow() - #duration(0, 0, 0, 2), 2)
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.IsInPreviousSecond

Syntax

```
DateTime.IsInPreviousSecond(dateTime as any) as nullable logical
```

Acerca de

Indica si el valor `dateTime` datetime especificado se produce durante el segundo anterior, determinado por la fecha y hora actuales del sistema. Tenga en cuenta que esta función devolverá false cuando se pasa un valor que se produce dentro del segundo actual.

- `dateTime`: un `datetime` valor o `datetimezone` que se va a evaluar.

Ejemplo 1

Determine si el segundo antes de la hora actual del sistema está en el segundo anterior.

Uso

Power Query M

```
DateTime.IsInPreviousSecond(DateTime.FixedLocalNow() - #duration(0, 0, 0, 1))
```

Salida

```
true
```

Last updated on 22/11/2025

DateTime.LocalNow

Sintaxis

```
DateTime.LocalNow() as datetime
```

Acerca de

Devuelve un valor `datetime` establecido en la fecha y hora actuales del sistema.

El valor devuelto por esta función depende de si está ejecutando la consulta en una máquina local o en línea. Por ejemplo, si ejecuta la consulta en un sistema ubicado en la zona horaria del Pacífico de EE. UU., Power Query Desktop devuelve la fecha y hora establecidas en el equipo local. Sin embargo, si ejecuta la consulta en la nube, Power Query Online devuelve la hora UTC porque lee la hora establecida en las máquinas virtuales en la nube, que se establecen en UTC.

Ejemplo 1

invoque esta función en una máquina local que ejecute Power Query Desktop.

Uso

Power Query M

```
DateTime.LocalNow()
```

Salida

The current local date and time.

Ejemplo 2

invoque esta función en la nube que ejecuta Power Query Online.

Uso

Power Query M

```
DateTime.LocalNow()
```

Salida

The current online (UTC) date and time.

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

Last updated on 26/03/2026

DateTime.Time

Syntax

```
DateTime.Time(dateTime as any) as nullable time
```

Acerca de

Devuelve la parte de hora del valor datetime especificado, **dateTime**.

Ejemplo 1

Busque el valor de hora de #datetime(2010, 12, 31, 11, 56, 02).

Uso

Power Query M

```
DateTime.Time(#datetime(2010, 12, 31, 11, 56, 02))
```

Salida

```
#time(11, 56, 2)
```

Last updated on 22/11/2025

DateTime.ToRecord

Syntax

```
DateTime.ToRecord(dateTime as datetime) as record
```

Acerca de

Devuelve un registro que contiene las partes del valor datetime especificado, `dateTime`.

- `dateTime` `datetime`: valor para el que se va a calcular el registro de sus partes.

Ejemplo 1

Convierta el `#datetime(2011, 12, 31, 11, 56, 2)` valor en un registro que contenga valores de fecha y hora.

Uso

Power Query M

```
DateTime.ToRecord(#datetime(2011, 12, 31, 11, 56, 2))
```

Salida

Power Query M

```
[
    Year = 2011,
    Month = 12,
    Day = 31,
    Hour = 11,
    Minute = 56,
    Second = 2
]
```


DateTime.ToText

Sintaxis

```
DateTime.ToText(  
    dateTime as nullable datetime,  
    optional options as any,  
    optional culture as nullable text  
) as nullable text
```

Acercas de

Devuelve una representación textual de `dateTime`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `culture` solo se usa para flujos de trabajo heredados. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, vaya a [Cadenas de formato de fecha y hora estándar](#) y [Cadenas de formato de fecha y hora personalizados](#). Si se omite este campo o se proporciona `null`, se dará formato a la fecha con el valor predeterminado definido por `Culture`.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "MMM" es "Jan", "Feb", "Mar", ..., mientras que en "ru-RU" "MMM" es "янв", "фев", "мар", Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` se `null` omite, [Culture.Current](#) se usa.

Para admitir flujos de trabajo heredados, `options` y `culture` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = options, Culture = culture]`.

Ejemplo 1

Convertir `#datetime(2010, 12, 31, 01, 30, 25)` en un valor `text`. *La salida del resultado puede variar en función de la referencia cultural actual.*

Uso

```
DateTime.ToText(#datetime(2010, 12, 31, 01, 30, 25))
```

Salida

```
"12/31/2010 1:30:25 AM"
```

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
DateTime.ToText(#datetime(2010, 12, 30, 2, 4, 50.36973), [Format="dd MMM yyyy  
HH:mm:ss.ffffff", Culture="de-DE"])
```

Salida

```
"30 Dez 2010 02:04:50.369730"
```

Ejemplo 3

Convertir con el patrón ISO 8601.

Uso

Power Query M

```
DateTime.ToText(#datetime(2000, 2, 8, 3, 45, 12), [Format="yyyy-MM-dd'T'HH:mm:ss'Z'",  
Culture="en-US"])
```

Salida

```
"2000-02-08T03:45:12Z"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
 - [Cadenas de formato de fecha y hora estándar](#)
 - [Cadenas de formato de fecha y hora personalizadas](#)
-

#datetime

Syntax

```
#datetime(  
    year as number,  
    month as number,  
    day as number,  
    hour as number,  
    minute as number,  
    second as number  
) as datetime
```

Acerca de

Crea un valor datetime a partir de números que representan el segundo año, mes, día, hora, minuto y (fracciones). Genera un error si estas condiciones no son verdaderas:

- $1 \leq \text{año} \leq 9999$
- $1 \leq \text{mes} \leq 12$
- $1 \leq \text{día} \leq 31$
- $0 \leq \text{hora} \leq 23$
- $0 \leq \text{minuto} \leq 59$
- $0 \leq \text{segundo} < 60$

Last updated on 22/11/2025

Funciones DateTimeZone

Artículo • 20/11/2024

Estas funciones crean y manipulan valores datetimezone.

 Expandir tabla

Nombre	Descripción
DateTimeZone.FixedLocalNow	Devuelve un valor DateTimeZone establecido en la fecha, hora y desplazamiento de zona horaria actuales del sistema.
DateTimeZone.FixedUtcNow	Devuelve la fecha y la hora actuales en UTC (la zona horaria GMT).
DateTimeZone.From	Devuelve un valor DateTimeZone a partir de un valor.
DateTimeZone.FromFileTime	Devuelve un valor DateTimeZone a partir de un valor numérico.
DateTimeZone.FromText	Crea un valor de datetimezone a partir de formatos de datetimezone locales, universales y personalizados.
DateTimeZone.LocalNow	Devuelve un valor DateTime establecido en la fecha y hora del sistema actuales.
DateTimeZone.RemoveZone	Devuelve un DateTime con la información de zona quitada del valor DateTimeZone de entrada.
DateTimeZone.SwitchZone	Cambia la información de zona horaria del valor DateTimeZone de entrada.
DateTimeZone.ToLocal	Devuelve un valor DateTime a partir de la zona horaria local.
DateTimeZone.ToRecord	Devuelve un registro que contiene elementos de un valor DateTime.
DateTimeZone.ToText	Devuelve un valor de texto a partir de un valor DateTime.
DateTimeZone.ToUtc	Devuelve un valor DateTime a la zona horaria UTC.
DateTimeZone.UtcNow	Devuelve un valor DateTime establecido en la fecha y hora del sistema actuales en la zona horaria UTC.
DateTimeZone.ZoneHours	Obtiene la hora de zona horaria del valor.
DateTimeZone.ZoneMinutes	Obtiene el minuto de zona horaria del valor.
#datetimezone	Crea un valor de fecha y zona horaria de números enteros de año, mes, día, hora, minuto, segundo (fraccional), horas de desplazamiento y minutos de desplazamiento.

Comentarios

¿Le ha resultado útil esta página?



Sí



No

[Proporcionar comentarios sobre el producto](#)

| [Preguntar a la comunidad](#) ↗

DateTimeZone.FixedLocalNow

16/09/2025

Sintaxis

```
DateTimeZone.FixedLocalNow() as datetimezone
```

Acerca de

Devuelve un valor `datetime` establecido en la fecha y hora actuales del sistema. El valor devuelto contiene información de zona horaria que representa la zona horaria local. Este valor es fijo y no cambiará con llamadas sucesivas, a diferencia de [DateTimeZone.LocalNow](#), que puede devolver valores diferentes durante el transcurso de la ejecución de una expresión.

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

DateTimeZone.FixedUtcNow

16/09/2025

Sintaxis

```
DateTimeZone.FixedUtcNow() as datetimetype
```

Acerca de

Devuelve la fecha y hora actuales en UTC (la zona horaria GMT). Este valor es fijo y no cambiará con llamadas sucesivas.

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

DateTimeZone.From

Syntax

```
DateTimeZone.From(value as any, optional culture as nullable text) as nullable datetimetypezone
```

Acerca de

Crea un `datetimetypezone` a partir del valor especificado.

- `value`: valor utilizado para crear un `datetimetypezone`.
- `culture`: (Opcional) Referencia cultural que se va a usar al transformar el valor (por ejemplo, "en-US").

Los valores de los siguientes tipos se pueden convertir en un `datetimetypezone` valor:

- `text`: devuelve un valor desde una representación `datetimetypezone` textual. Consulte [DateTimeZone.FromText](#) para obtener más información.
- `date`: devuelve un `datetimetypezone` con `value` como componente de fecha, `12:00:00 AM` como componente de hora y el desplazamiento correspondiente a la zona horaria local.
- `datetime`: devuelve un `datetimetypezone` con `value` como fecha y hora y el desplazamiento correspondiente a la zona horaria local.
- `datetimetypezone`: devuelve `value`.
- `time`: devuelve un `datetimetypezone` objeto con la fecha equivalente de la fecha de automatización OLE de `0` como componente de fecha, `value` como componente de hora y el desplazamiento correspondiente a la zona horaria local. La fecha de automatización OLE consta de un número de punto flotante cuyo componente entero es el número de días antes o después de la medianoche, 30 de diciembre de 1899, y cuyo componente fraccionario representa la hora en ese día dividida por 24. Por ejemplo, la medianoche del 31 de diciembre de 1899 está representada por 1.0; las 6 a.m., 1 de enero de 1900, está representado por 2.25; la medianoche del 29 de diciembre de 1899 está representada por -1.0; y las 6 a.m., 29 de diciembre de 1899, está representado por -1.25. El valor base es medianoche, 30 de diciembre de 1899. El valor mínimo es medianoche, 1 de enero de 0100. El valor máximo es el último momento del 31 de diciembre de 9999.

- `number`: devuelve un `datetimezone` objeto con el equivalente datetime de la fecha de automatización OLE expresada por `value` y el desplazamiento correspondiente a la zona horaria local.
- `null`: devuelve `null`.

Si `value` es de cualquier otro tipo, se devuelve un error.

El valor del desplazamiento correspondiente a la zona horaria local es diferente al ejecutar esta función localmente en lugar de ejecutarla en línea. Cuando se ejecuta localmente, se devuelve la zona horaria local. Cuando se ejecuta en línea, se devuelve la zona horaria UTC (+00:00).

Ejemplo 1

Convierta la representación textual de una fecha, hora y zona horaria en un `datetimezone` valor.

Usage

Power Query M

```
DateTimeZone.From("2020-10-30T01:30:00-08:00")
```

Salida

```
#datetimezone(2020, 10, 30, 01, 30, 00, -8, 00)
```

Ejemplo 2

Convierta la representación textual de la fecha, hora y zona horaria en portugués brasileño a un valor `datetimezone`.

Usage

Power Query M

```
DateTimeZone.From("13 de agosto de 2025 15:43:00 -03:00", "pt-BR")
```

Salida

```
#datetimezone(2025, 08, 13, 15, 43, 00, -3, 00)
```

Ejemplo 3

Convierta un número que represente el 1 de enero de 2025 a las 12 p. m. en un `datetimezone` valor. La zona horaria del resultado depende de si el ejemplo se ejecuta localmente o en línea.

Usage

Power Query M

```
DateTimeZone.From(45658.5)
```

Salida

```
#datetimezone(2025, 01, 01, 12, 00, 00, 0, 00)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Variantes locales, fijas y UTC de las funciones de hora actual](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)
- [Number.From](#)

Last updated on 04/04/2026

DateTimeZone.FromFileTime

Syntax

```
DateTimeZone.FromFileTime(fileTime as nullable number) as nullable datetimezone
```

Acerca de

Crea un `datetimezone` valor a partir del `fileTime` valor y lo convierte en la zona horaria local.

Filetime es un valor de hora de archivo de Windows que representa el número de intervalos de 100 nanosegundos transcurridos desde la medianoche 12:00, 1 de enero de 1601 A.D. (C.E.)

Hora universal coordinada (UTC).

Ejemplo 1

Convierta `129876402529842245` en un valor `datetimezone`.

Uso

Power Query M

```
DateTimeZone.FromFileTime(129876402529842245)
```

Salida

```
#datetimezone(2012, 7, 24, 14, 50, 52.9842245, -7, 0)
```

Last updated on 22/11/2025

DateTimeZone.FromText

Sintaxis

```
DateTimeZone.FromText(text as nullable text, optional options as any) as nullable datetimezone
```

Acercas de

Crea un valor `datetimezone` a partir de una representación textual, `text`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, vaya a [Cadenas de formato de fecha y hora estándar](#) y [Cadenas de formato de fecha y hora personalizados](#). Si se omite este campo o se proporciona `null`, se analizará la fecha con el mejor esfuerzo.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "MMM" es "Jan", "Feb", "Mar", ..., mientras que en "ru-RU" "MMM" es "янв", "фев", "мар", Cuando `Format` es null, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` se `null` omite, [Culture.Current](#) se usa .

Para admitir flujos de trabajo heredados, `options` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = null, Culture = options]`.

Ejemplo 1

Convertir "2010-12-31T01:30:00-08:00" en un valor `datetimezone`.

Uso

Power Query M

```
DateTimeZone.FromText("2010-12-31T01:30:00-08:00")
```

Salida

```
#datetimezone(2010, 12, 31, 1, 30, 0, -8, 0)
```

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
DateTimeZone.FromText("30 Dez 2010 02:04:50.369730 +02:00", [Format="dd MMM yyyy HH:mm:ss.ffffff zzz", Culture="de-DE"])
```

Salida

```
#datetimezone(2010, 12, 30, 2, 4, 50.36973, 2, 0)
```

Ejemplo 3

Convertir con ISO 8601.

Uso

Power Query M

```
DateTimeZone.FromText("2009-06-15T13:45:30.0000000-07:00", [Format="O", Culture="en-US"])
```

Salida

```
#datetimezone(2009, 6, 15, 13, 45, 30, -7, 0)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)

DateTimeZone.LocalNow

16/09/2025

Sintaxis

```
DateTimeZone.LocalNow() as datetimezone
```

Acerca de

Devuelve un valor `datetimezone` establecido en la fecha y hora actuales del sistema. El valor devuelto contiene información de zona horaria que representa la zona horaria local.

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

DateTimeZone.RemoveZone

Syntax

```
DateTimeZone.RemoveZone(dateTimeZone as nullable datetimetimezone) as nullable datetime
```

Acerca de

Devuelve un #datetime valor de con información de **dateTimeZone** zona horaria quitada.

Ejemplo 1

Quite la información de zona horaria del valor #datetimetimezone(2011, 12, 31, 9, 15, 36, -7, 0).

Uso

Power Query M

```
DateTimeZone.RemoveZone(#datetimetimezone(2011, 12, 31, 9, 15, 36, -7, 0))
```

Salida

```
#datetime(2011, 12, 31, 9, 15, 36)
```

Last updated on 22/11/2025

DateTimeZone.SwitchZone

Syntax

```
DateTimeZone.SwitchZone(  
    dateTimeZone as nullable datetimezone,  
    timeZoneHours as number,  
    optional timeZoneMinutes as nullable number  
) as nullable datetimezone
```

Acerca de

Cambia la información de zona horaria a en el valor `dateTimeZone` `datetimezone` a la nueva información de zona horaria proporcionada por `timeZoneHours` y opcionalmente `timeZoneMinutes`. Si `dateTimeZone` no tiene un componente de zona horaria, se genera un error.

Ejemplo 1

Cambie la información de zona horaria de `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` a 8 horas.

Uso

Power Query M

```
DateTimeZone.SwitchZone(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30), 8)
```

Salida

```
#datetimezone(2010, 12, 31, 12, 26, 2, 8, 0)
```

Ejemplo 2

Cambie la información de zona horaria de `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` a -30 minutos.

Uso

Power Query M

```
DateTimeZone.SwitchZone(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30), 0, -30)
```

Salida

```
#datetimezone(2010, 12, 31, 3, 56, 2, 0, -30)
```

Last updated on 23/01/2026

DateTimeZone.ToLocal

Syntax

```
DateTimeZone.ToLocal(dateTimeZone as nullable datetimetype) as nullable datetimetype
```

Acerca de

Cambia la información de zona horaria del valor `dateTimeZone` `datetimetype` a la información de zona horaria local. Si `dateTimeZone` no tiene un componente de zona horaria, se agrega la información de zona horaria local.

Ejemplo 1

Cambie la información de zona horaria de `#datetimetype(2010, 12, 31, 11, 56, 02, 7, 30)` a la zona horaria local (suponiendo PST).

Uso

Power Query M

```
DateTimeZone.ToLocal(#datetimetype(2010, 12, 31, 11, 56, 02, 7, 30))
```

Salida

```
#datetimetype(2010, 12, 31, 12, 26, 2, -8, 0)
```

Last updated on 16/09/2025

DateTimeZone.ToRecord

Syntax

```
DateTimeZone.ToRecord(dateTimeZone as datetimestamp) as record
```

Acerca de

Devuelve un registro que contiene las partes del valor datetimestamp especificado, `dateTimeZone`.

- `dateTimeZone` `datetimestamp`: valor para el que se va a calcular el registro de sus partes.

Ejemplo 1

Convierta el `#datetimestamp(2011, 12, 31, 11, 56, 2, 8, 0)` valor en un registro que contenga valores de fecha, hora y zona.

Uso

Power Query M

```
DateTimeZone.ToRecord(#datetimestamp(2011, 12, 31, 11, 56, 2, 8, 0))
```

Salida

Power Query M

```
[
    Year = 2011,
    Month = 12,
    Day = 31,
    Hour = 11,
    Minute = 56,
    Second = 2,
    ZoneHours = 8,
    ZoneMinutes = 0
]
```

DateTimeZone.ToText

Sintaxis

```
DateTimeZone.ToText(  
    dateTimeZone as nullable datetimetypezone,  
    optional options as any,  
    optional culture as nullable text  
) as nullable text
```

Acercas de

Devuelve una representación textual de `dateTimeZone`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `culture` solo se usa para flujos de trabajo heredados. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, ve a [Cadenas con formato de fecha y hora estándar](#) y [Cadenas con formato de fecha y hora personalizado](#). Si se omite este campo o se proporciona `null`, se dará formato a la fecha con el valor predeterminado definido por `Culture`.
- `Culture`: cuando `Format` no es `null`, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "MMM" es "Jan", "Feb", "Mar", ..., mientras que en "ru-RU" "MMM" es "янв", "фев", "мар", Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` se `null` omite, [Culture.Current](#) se usa .

Para admitir flujos de trabajo heredados, `options` y `culture` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = options, Culture = culture]`.

Ejemplo 1

Convertir `#datetimetypezone(2010, 12, 31, 01, 30, 25, 2, 0)` en un valor `text`. *La salida del resultado puede variar en función de la referencia cultural actual.*

Uso

```
DateTimeZone.ToText(#datetimezone(2010, 12, 31, 01, 30, 25, 2, 0))
```

Salida

```
"12/31/2010 1:30:25 AM +02:00"
```

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
DateTimeZone.ToText(#datetimezone(2010, 12, 30, 2, 4, 50.36973, -8,0), [Format="dd  
MMM yyyy HH:mm:ss.ffffff zzz", Culture="de-DE"])
```

Salida

```
"30 Dez 2010 02:04:50.369730 -08:00"
```

Ejemplo 3

Convertir con el patrón ISO 8601.

Uso

Power Query M

```
DateTimeZone.ToText(#datetimezone(2000, 2, 8, 3, 45, 12, 2, 0),[Format="O",  
Culture="en-US"])
```

Salida

```
"2000-02-08T03:45:12.0000000+02:00"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
 - [Cadenas de formato de fecha y hora estándar](#)
 - [Cadenas de formato de fecha y hora personalizadas](#)
-

DateTimeZone.ToUtc

Syntax

```
DateTimeZone.ToUtc(dateTimeZone as nullable datetimetype) as nullable datetimetype
```

Acerca de

Cambia la información de zona horaria del valor `dateTimeZone` `datetime` a la información de zona horaria UTC o universal. Si `dateTimeZone` no tiene un componente de zona horaria, se agrega la información de zona horaria UTC.

Ejemplo 1

Cambie la información de zona horaria de `#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30)` a la zona horaria UTC.

Uso

Power Query M

```
DateTimeZone.ToUtc(#datetimezone(2010, 12, 31, 11, 56, 02, 7, 30))
```

Salida

```
#datetimezone(2010, 12, 31, 4, 26, 2, 0, 0)
```

Last updated on 22/11/2025

DateTimeZone.UtcNow

16/09/2025

Sintaxis

```
DateTimeZone.UtcNow() as datetimezone
```

Acerca de

Devuelve la fecha y hora actuales en UTC (la zona horaria GMT).

Ejemplo 1

Obtenga la fecha y hora actuales en UTC.

Uso

Power Query M

```
DateTimeZone.UtcNow()
```

Salida

```
#datetimezone(2011, 8, 16, 23, 34, 37.745, 0, 0)
```

Contenido relacionado

[Variantes locales, fijas y UTC de las funciones de hora actual](#)

DateTimeZone.ZoneHours

Sintaxis

```
DateTimeZone.ZoneHours(dateTimeZone as nullable datetimezone) as nullable number
```

Acerca de

Devuelve el componente hora de zona horaria de un `datetimezone` valor.

- `dateTimeZone` `datetimezone`: valor del que se extrae el componente hora de zona horaria. Si `dateTimeZone` es `null`, la función devuelve `null`.

Ejemplo

Obtenga el componente de horas de zona horaria del valor especificado `datetimezone` .

Uso

Power Query M

```
DateTimeZone.ZoneHours(#datetimezone(2024, 4, 28, 13, 24, 22, 7, 30))
```

Salida

7

Last updated on 25/03/2026

DateTimeZone.ZoneMinutes

Sintaxis

```
DateTimeZone.ZoneMinutes(dateTimeZone as nullable datetimezone) as nullable number
```

Acerca de

Devuelve el componente de minutos de zona horaria de un `datetimezone` valor.

- `dateTimeZone datetimezone`: valor del que se extrae el componente de minutos de zona horaria. Si `dateTimeZone` es `null`, la función devuelve `null`.

Ejemplo

Obtenga el componente de minutos de zona horaria del valor especificado `datetimezone` .

Uso

Power Query M

```
DateTimeZone.ZoneMinutes(#datetimezone(2024, 4, 28, 13, 24, 22, 7, 30))
```

Salida

```
30
```

Last updated on 04/04/2026

#datetimezone

Syntax

```
#datetimezone(  
  year as number,  
  month as number,  
  day as number,  
  hour as number,  
  minute as number,  
  second as number,  
  offsetHours as number,  
  offsetMinutes as number  
) as datetimezone
```

Acerca de

Crea un valor `datetimezone` a partir de números que representan el año, el mes, el día, la hora, el minuto, el segundo (fraccionario), las horas de desplazamiento (fraccionaria) y los minutos de desplazamiento. Genera un error si estas condiciones no son verdaderas:

- $1 \leq \text{año} \leq 9999$
- $1 \leq \text{mes} \leq 12$
- $1 \leq \text{día} \leq 31$
- $0 \leq \text{hora} \leq 23$
- $0 \leq \text{minuto} \leq 59$
- $0 \leq \text{segundo} < 60$
- $-14 \leq \text{offset-hours} + \text{offset-minutes} / 60 \leq 14$

Last updated on 22/11/2025

Funciones de duración

Artículo • 05/02/2025

Estas funciones crean y manipulan valores de duración.

 Expandir tabla

Nombre	Descripción
Duration.Days	Devuelve la parte de los días de una duración.
Duration.From	Crea una duración a partir del valor especificado.
Duration.FromText	Devuelve un valor de duración de un valor de texto.
Duration.Hours	Devuelve la parte de las horas de una duración.
Duration.Minutes	Devuelve la parte de los minutos de una duración.
Duration.Seconds	Devuelve la parte de los segundos de una duración.
Duration.ToRecord	Devuelve un registro que contiene las partes de la duración.
Duration.TotalDays	Devuelve los días totales que abarca esta duración.
Duration.TotalHours	Devuelve el total de horas que abarca esta duración.
Duration.TotalMinutes	Devuelve el total de minutos que abarca esta duración.
Duration.TotalSeconds	Devuelve el total de segundos que abarca esta duración.
Duration.ToText	Devuelve el texto del formato "d.h:m:s".
#duration	Crea un valor de duración a partir de días, horas, minutos y segundos.

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Duration.Days

Syntax

```
Duration.Days(duration as nullable duration) as nullable number
```

Acerca de

Devuelve la parte de días de `duration`.

Ejemplo

Extraiga el número de días entre dos fechas.

Uso

Power Query M

```
Duration.Days(#date(2022, 3, 4) - #date(2022, 2, 25))
```

Salida

7

Last updated on 25/03/2026

Duration.From

Syntax

```
Duration.From(value as any) as nullable duration
```

Acerca de

Devuelve un valor de duración del valor especificado.

- `value`: valor del que se deriva la duración. Si el objeto especificado `value` es `null`, esta función devuelve `null`. Si el objeto especificado `value` es , `duration value` se devuelve . Los valores de los siguientes tipos se pueden convertir en un `duration` valor:
 - `text`: valor `duration` de formularios de tiempo transcurridos textuales (d.h:m:s). Consulte [Duration.FromText](#) para obtener más información.
 - `number duration`: equivalente al número de días enteros y fraccionarios expresados por `value`.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo 1

Convertir `2.525` en un `duration` valor.

Uso

Power Query M

```
Duration.From(2.525)
```

Salida

```
#duration(2, 12, 36, 0)
```

Ejemplo 2

Convierta el valor `"2.05:55:20.34567"` de texto en un `duration` valor.

Uso

Power Query M

```
Duration.From("2.05:55:20.34567")
```

Salida

```
#duration(2, 5, 55, 20.3456700)
```

Last updated on 04/04/2026

Duration.FromText

Syntax

```
Duration.FromText(text as nullable text) as nullable duration
```

Acerca de

Devuelve un valor de duración del texto especificado, `text`. Esta función puede analizar los siguientes formatos:

- (-)hh:mm(:ss(.ff))
- (-)ddd(.hh:mm(:ss(.ff)))

(Todos los intervalos son inclusivos)

- ddd: número de días.
- hh: Número de horas, entre 0 y 23.
- mm: número de minutos, entre 0 y 59.
- ss: número de segundos, entre 0 y 59.
- ff: Fracción de segundos, entre 0 y 9999999.

Ejemplo 1

Convertir `"2.05:55:20"` en un `duration` valor.

Uso

Power Query M

```
Duration.FromText("2.05:55:20")
```

Salida

```
#duration(2, 5, 55, 20)
```

Last updated on 22/11/2025

Duration.Hours

Syntax

```
Duration.Hours(duration as nullable duration) as nullable number
```

Acerca de

Devuelve la parte de horas de `duration`.

Ejemplo 1

Extraiga las horas de un valor de duración.

Uso

Power Query M

```
Duration.Hours(#duration(5, 4, 3, 2))
```

Salida

4

Last updated on 22/11/2025

Duration.Minutes

Syntax

```
Duration.Minutes(duration as nullable duration) as nullable number
```

Acerca de

Devuelve la parte de minutos de `duration`.

Ejemplo

Extraiga los minutos de un valor de duración.

Uso

Power Query M

```
Duration.Minutes(#duration(5, 4, 3, 2))
```

Salida

3

Last updated on 25/03/2026

Duration.Seconds

Sintaxis

```
Duration.Seconds(duration as nullable duration) as nullable number
```

Acerca de

Devuelve el valor de los segundos de `duration`.

Ejemplo

Extrae los segundos de un valor de duración.

Uso

Power Query M

```
Duration.Seconds(#duration(5, 4, 3, 2))
```

Salida

2

Last updated on 25/03/2026

Duration.ToRecord

Syntax

```
Duration.ToRecord(duration as duration) as record
```

Acerca de

Devuelve un registro que contiene las partes del valor de duración, `duration`.

- `duration`: a `duration` partir del cual se crea el registro.

Ejemplo 1

Convierta `#duration(2, 5, 55, 20)` en un registro de sus partes, incluidos días, horas, minutos y segundos, si procede.

Uso

Power Query M

```
Duration.ToRecord(#duration(2, 5, 55, 20))
```

Salida

Power Query M

```
[  
    Days = 2,  
    Hours = 5,  
    Minutes = 55,  
    Seconds = 20  
]
```

Duration.TotalDays

Syntax

```
Duration.TotalDays(duration as nullable duration) as nullable number
```

Acerca de

Devuelve el total de días distribuidos por `duration`.

Ejemplo

Busque los días totales distribuidos por un valor de duración.

Uso

Power Query M

```
Duration.TotalDays(#duration(5, 4, 3, 2))
```

Salida

```
5.1687731481481478
```

Last updated on 25/03/2026

Duration.TotalHours

Syntax

```
Duration.TotalHours(duration as nullable duration) as nullable number
```

Acerca de

Devuelve el total de horas distribuidas por `duration`.

Ejemplo 1

Busque las horas totales distribuidas por un valor de duración.

Uso

Power Query M

```
Duration.TotalHours(#duration(5, 4, 3, 2))
```

Salida

```
124.05055555555555
```

Last updated on 22/11/2025

Duration.TotalMinutes

Syntax

```
Duration.TotalMinutes(duration as nullable duration) as nullable number
```

Acerca de

Devuelve el total de minutos distribuidos por `duration`.

Ejemplo

Busque el total de minutos distribuidos por un valor de duración.

Uso

Power Query M

```
Duration.TotalMinutes(#duration(5, 4, 3, 2))
```

Salida

```
7443.0333333333338
```

Last updated on 25/03/2026

Duration.TotalSeconds

Syntax

```
Duration.TotalSeconds(duration as nullable duration) as nullable number
```

Acerca de

Devuelve el total de segundos distribuidos por `duration`.

Ejemplo 1

Busque los segundos totales distribuidos por un valor de duración.

Uso

Power Query M

```
Duration.TotalSeconds(#duration(5, 4, 3, 2))
```

Salida

```
446582
```

Last updated on 22/11/2025

Duration.ToText

Syntax

```
Duration.ToText(duration as nullable duration, optional format as nullable text) as nullable text
```

Acerca de

Devuelve una representación textual con el formato "day.hour:mins:sec" del valor de duración especificado, `duration`.

- `duration`: a `duration` partir del cual se calcula la representación textual.
- `format`: (*Opcional*) En desuso, se producirá un error si no es null.

Ejemplo

Convertir `#duration(2, 5, 55, 20)` en un valor de texto.

Uso

Power Query M

```
Duration.ToText(#duration(2, 5, 55, 20))
```

Salida

```
"2.05:55:20"
```

Last updated on 04/04/2026

#duration

16/09/2025

Sintaxis

```
#duration(  
  days as number,  
  hours as number,  
  minutes as number,  
  seconds as number  
) as duration
```

Acerca de

Crea un valor de duración a partir de números que representan días, horas, minutos y (fracciones) segundos.

Funciones de control de errores

Artículo • 16/07/2024

Estas funciones se pueden usar para realizar un seguimiento de errores o construirlos.

 Expandir tabla

Nombre	Descripción
Diagnostics.ActivityId	Devuelve un identificador opaco para la evaluación que se está ejecutando actualmente.
Diagnostics.CorrelationId	Devuelve un identificador opaco para correlacionar las solicitudes entrantes con las salientes.
Diagnostics.Trace	Escribe una entrada de seguimiento, si está habilitado el seguimiento, y devuelve el valor.
Error.Record	Devuelve un registro de error de los valores de texto proporcionados por motivos, mensajes, detalles y código de error.

Comentarios

¿Le ha resultado útil esta página?

 [Sí](#)

 [No](#)

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Diagnostics.ActivityId

Syntax

```
Diagnostics.ActivityId() as nullable text
```

Acerca de

Devuelve un identificador opaco para la evaluación que se está ejecutando actualmente.

Last updated on 22/11/2025

Diagnostics.CorrelationId

16/09/2025

Sintaxis

```
Diagnostics.CorrelationId() as nullable text
```

Acerca de

Devuelve un identificador opaco para correlacionar las solicitudes entrantes con las salientes.

Diagnostics.Trace

Syntax

```
Diagnostics.Trace(  
    traceLevel as number,  
    message as anynonnull,  
    value as any,  
    optional delayed as nullable logical  
) as any
```

Acerca de

Escribe un seguimiento `message`, si el seguimiento está habilitado y devuelve `value`. Un parámetro `delayed` opcional especifica si se va a retrasar la evaluación de `value` hasta que se realiza el seguimiento del mensaje. `traceLevel` puede tomar uno de los siguientes valores:

- [TraceLevel.Critical](#)
- [TraceLevel.Error](#)
- [TraceLevel.Warning](#)
- [TraceLevel.Information](#)
- [TraceLevel.Verbose](#)

Ejemplo 1

Haga un seguimiento del mensaje antes de invocar `Text.From` la función y devuelva el resultado.

Uso

Power Query M

```
Diagnostics.Trace(TraceLevel.Information, "TextValueFromNumber", () =>  
Text.From(123), true)
```

Salida

"123"

Error.Record

Sintaxis

```
Error.Record(  
    reason as text,  
    optional message as nullable text,  
    optional detail as any,  
    optional parameters as nullable list,  
    optional errorCode as nullable text  
) as record
```

Acercas de

Devuelve un registro de error a partir de los valores de texto proporcionados para motivo, mensaje, detalle y código de error.

- **reason**: causa de alto nivel del error.
- **message**: (Opcional) Descripción del error.
- **detail**: (Opcional) Información detallada adicional sobre el error.
- **parameters**: (Opcional) Lista de valores que proporcionan contexto adicional para el error, que normalmente se usa para el diagnóstico o el control mediante programación.
- **errorCode**: (Opcional) Identificador del error.

Ejemplo 1

Controle un error de división por cero.

Uso

Power Query M

```
let  
    input = 100,  
    divisor = 0,  
    result = try if divisor = 0 then  
        error Error.Record(  
            "DivideByZero",  
            "You attempted to divide by zero."  
        )  
    else  
        input / divisor
```



```
in
    result
```

Salida

Power Query M

```
[
    HasError = true,
    Error =
    [
        Reason = "DivideByZero",
        Message = "You attempted to divide by zero.",
        Detail = null,
        Message.Format = null,
        Message.Parameters = null,
        ErrorCode = null
    ]
]
```

Ejemplo 2

Controle una entrada con un error de identificador de cliente inexistente. Si no se produce ningún error, indique una entrada correcta.

Uso

Power Query M

```
let
    CustomerId = 12345,
    result = try if CustomerId > 9999 then
        error Error.Record(
            "CustomerNotFound",
            Text.Format("Customer ID #{0} wasn't found.", {CustomerId}),
            "Customer doesn't exist.",
            {
                Text.Format("Invalid ID = #{0}", {CustomerId}),
                "Valid IDs: https://api.contoso.com/customers"
            },
            "ERR404"
        )
    else CustomerId
in
    result
```

Salida

Power Query M

```
[
  HasError = true,
  Error = [
    Reason = "CustomerNotFound",
    Message = "Customer ID 12345 wasn't found.",
    Detail = "Customer doesn't exist.",
    Message.Format = "Customer ID 12345 wasn't found.",
    Message.Parameters = {
      "Invalid ID = 12345",
      "Valid IDs: https://api.contoso.com/customers"
    },
    ErrorCode = "ERR404"
  ]
]
```

Last updated on 03/04/2026

Funciones de expresión

11/08/2025

Estas funciones permiten la construcción y evaluación del código M.

 Expandir tabla

Nombre	Descripción
Expression.Constant	Devuelve la representación de código fuente M de un valor constante.
Expression.Evaluate	Devuelve el resultado de evaluar una expresión M.
Expression.Identifier	Devuelve la representación de código fuente M de un identificador.

Expression.Constant

Syntax

```
Expression.Constant(value as any) as text
```

Acerca de

Devuelve la representación de código fuente M de un valor constante.

Ejemplo 1

Obtenga la representación de código fuente M de un valor numérico.

Uso

Power Query M

```
Expression.Constant(123)
```

Salida

```
"123"
```

Ejemplo 2

Obtiene la representación de código fuente M de un valor de fecha.

Uso

Power Query M

```
Expression.Constant(#date(2035, 01, 02))
```

Salida

```
"#date(2035, 1, 2)"
```

Ejemplo 3

Obtiene la representación de código fuente M de un valor de texto.

Uso

Power Query M

```
Expression.Constant("abc")
```

Salida

```
""abc""
```

Last updated on 22/11/2025

Expression.Evaluate

Syntax

```
Expression.Evaluate(document as text, optional environment as nullable record) as any
```

Acerca de

Devuelve el resultado de evaluar una expresión `document` M con los identificadores disponibles a los que se puede hacer referencia mediante `environment`.

Ejemplo 1

Evalúe una suma simple.

Uso

Power Query M

```
Expression.Evaluate("1 + 1")
```

Salida

2

Ejemplo 2

Evalúe una suma más compleja.

Uso

Power Query M

```
Expression.Evaluate("List.Sum({1, 2, 3})", [List.Sum = List.Sum])
```

Salida

6

Ejemplo 3

Evalúe la concatenación de un valor de texto con un identificador.

Uso

Power Query M

```
Expression.Evaluate(Expression.Constant("""abc") & " & " &  
Expression.Identifier("x"), [x = "def"""])
```

Salida

```
""""abcdef""""
```

Last updated on 22/11/2025

Expression.Identifier

Syntax

```
Expression.Identifier(name as text) as text
```

Acerca de

Devuelve la representación de código fuente M de un identificador `name`.

Ejemplo 1

Obtenga la representación de código fuente M de un identificador.

Uso

Power Query M

```
Expression.Identifier("MyIdentifier")
```

Salida

```
"MyIdentifier"
```

Ejemplo 2

Obtenga la representación de código fuente M de un identificador que contiene un espacio.

Uso

Power Query M

```
Expression.Identifier("My Identifier")
```

Salida

```
"#"My Identifier"'"
```


Valores de función

Artículo • 18/12/2024

Estas funciones crean e invocan otras funciones M.

 Expandir tabla

Nombre	Descripción
Function.From	Toma una función unaria <code>function</code> y crea una nueva función con el tipo <code>functionType</code> que construye una lista de sus argumentos y la pasa a <code>function</code> .
Function.Invoke	Invoca la función especificada usando el argumento proporcionado y devuelve el resultado.
Function.InvokeAfter	Devuelve el resultado de invocar la función después de que se haya superado el retraso de duración.
Function.InvokeWithErrorContext	Esta función está pensada solo para uso interno.
Function.IsDataSource	Devuelve si la función se considera un origen de datos o no.
Function.ScalarVector	Devuelve una función escalar de tipo <code>scalarFunctionType</code> que invoca <code>vectorFunction</code> con una sola fila de argumentos y devuelve su salida única.

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Function.From

Syntax

```
Function.From(functionType as type, function as function) as function
```

Acerca de

Toma una función `function` unaria y crea una nueva función con el tipo `functionType` que construye una lista de sus argumentos y la pasa a `function`.

Ejemplo 1

List.Sum Convierte en una función de dos argumentos cuyos argumentos se agregan juntos.

Uso

Power Query M

```
Function.From(type function (a as number, b as number) as number, List.Sum)(2, 1)
```

Salida

3

Ejemplo 2

Convierte una función que toma una lista en una función de dos argumentos.

Uso

Power Query M

```
Function.From(type function (a as text, b as text) as text, (list) => list{0} & list{1})("2", "1")
```

Salida

"21"

Function.Invoke

Sintaxis

```
Function.Invoke(function as function, args as list) as any
```

Acerca de

Invoca la función especificada mediante la lista especificada de argumentos y devuelve el resultado.

Ejemplo

[Record.FieldNames](#) Invoca con un argumento [A=1,B=2].

Uso

Power Query M

```
Function.Invoke(Record.FieldNames, {[A = 1, B = 2]})
```

Salida

```
{"A", "B"}
```

Last updated on 04/04/2026

Function.InvokeAfter

16/09/2025

Sintaxis

```
Function.InvokeAfter(function as function, delay as duration) as any
```

Acerca de

Devuelve el resultado de invocar `function` después de que haya transcurrido la duración `delay`.

Function.InvokeWithErrorContext

16/09/2025

Sintaxis

```
Function.InvokeWithErrorContext(function as function, context as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Function.IsDataSource

16/09/2025

Sintaxis

```
Function.IsDataSource(function as function) as logical
```

Acerca de

Devuelve si `function` se considera un origen de datos o no.

Function.ScalarVector

Sintaxis

```
Function.ScalarVector(scalarFunctionType as type, vectorFunction as function) as function
```

Acerca de

Devuelve una función escalar de tipo `scalarFunctionType` que invoca `vectorFunction` con una sola fila de argumentos y devuelve su salida única. Además, cuando la función escalar se aplica repetidamente para cada fila de una tabla de entradas, como en `Table.AddColumn`, en lugar de `vectorFunction` se aplicará una vez para todas las entradas.

`vectorFunction` se le pasará una tabla cuyas columnas coincidan en nombre y posición con los parámetros de `scalarFunctionType`. Cada fila de esta tabla contiene los argumentos de una llamada a la función escalar, con las columnas correspondientes a los parámetros de `scalarFunctionType`.

`vectorFunction` debe devolver una lista de la misma longitud que la tabla de entrada, donde el elemento en cada posición debe ser el mismo resultado de evaluar la función escalar en la misma posición de la fila de entrada.

Se espera que la tabla de datos de entrada se procese en flujo, por lo que se espera que `vectorFunction` transmita su salida a medida que ingresan los datos, procesando solo un fragmento de datos a la vez. En concreto, `vectorFunction` no deben enumerar su tabla de entrada más de una vez.

Ejemplo 1

Multiplique dos columnas de la tabla de entrada mediante el procesamiento de las entradas en lotes de 100.

Uso

Power Query M

```
let
    Compute.ScoreScalar = (left, right) => left * right,
    // When Function.ScalarVector batching kicks in, we'll receive all
    // of the inputs for the entire table here at once.
```



```

Compute.ScoreVector = (input) => let
    chunks = Table.Split(input, 100),
    scoreChunk = (chunk) => Table.TransformRows(chunk, each
Compute.ScoreScalar([left], [right]))
in
    List.Combine(List.Transform(chunks, scoreChunk)),
Compute.Score = Function.ScalarVector(
    type function (left as number, right as number) as number,
    Compute.ScoreVector
),
Final = Table.AddColumn(
    Table.FromRecords({
        [a = 1, b = 2],
        [a = 3, b = 4]
    }),
    "Result",
    each Compute.Score([a], [b])
)
in
    Final

```

Salida

Power Query M

```

Table.FromRecords({
    [a = 1, b = 2, Result = 2],
    [a = 3, b = 4, Result = 12]
})

```

Ejemplo 2

Calcule las puntuaciones de las pruebas en lotes de dos y rellene un campo de identificador de lote que se puede usar para comprobar que el procesamiento por lotes funciona según lo previsto.

Uso

Power Query M

```

let
    _GradeTest = (right, total) => Number.Round(right / total, 2),
    _GradeTests = (inputs as table) as list => let
        batches = Table.Split(inputs, 2),
        gradeBatch = (batch as table) as list =>
            let
                batchId = Text.NewGuid()
            in
                Table.TransformRows(batch, each [Grade = _GradeTest([right], [total]),

```

```

BatchId = batchId])
in
    List.Combine(List.Transform(batches, gradeBatch)),
    GradeTest = Function.ScalarVector(type function (right as number, total as number)
as number, _GradeTests),
    Tests = #table(type table [Test Name = text, Right = number, Total = number],
    {
        {"Quiz 1", 3, 4},
        {"Test 1", 17, 22},
        {"Quiz 2", 10, 10}
    }),
    // To break batching, replace [Right] with {[Right]}{0}.
    TestsWithGrades = Table.AddColumn(Tests, "Grade Info", each GradeTest([Right],
[Total])), type record),
    // To verify batching, also expand BatchId.
    Final = Table.ExpandRecordColumn(TestsWithGrades, "Grade Info", {"Grade"})
in
    Final

```

Salida

Power Query M

```

#table(
    type table [Test Name = text, Right = number, Total = number, Grade = number],
    {
        {"Quiz 1", 3, 4, 0.75},
        {"Test 1", 17, 22, 0.77},
        {"Quiz 2", 10, 10, 1}
    }
)

```

Last updated on 06/11/2025

Funciones de líneas

Estas funciones convierten listas de texto a y desde valores binarios y de texto único.

 Expandir tabla

Nombre	Description
Lines.FromBinary	Convierte un valor binario en una lista de valores de texto divididos en saltos de línea.
Lines.FromText	Convierte un valor de texto en una lista de valores de texto divididos en saltos de línea.
Lines.ToBinary	Convierte una lista de texto en un valor binario mediante la codificación especificada y lineSeparator.El lineSeparator especificado se anexa a cada línea. Si no se especifica, se usan los caracteres de retorno de carro y avance de línea.
Lines.ToText	Convierte una lista de texto en un solo texto. El lineSeparator especificado se anexa a cada línea. Si no se especifica, se usan los caracteres de retorno de carro y avance de línea.

Last updated on 22/11/2025

Lines.FromBinary

Sintaxis

```
Lines.FromBinary(  
    binary as binary,  
    optional quoteStyle as any,  
    optional includeLineSeparators as nullable logical,  
    optional encoding as nullable number  
) as list
```

Acerca de

Convierte un valor binario en una lista de valores de texto divididos en saltos de línea.

- `binary`: valor binario que se va a convertir en la lista.
- `quoteStyle`: especifica cómo se controlan los saltos de línea. El valor de `quoteStyle` puede ser `null`. El valor predeterminado es [QuoteStyle.None](#).
- `includeLineSeparators`: especifica si se deben incluir los caracteres de salto de línea en el texto. El valor de `includeLineSeparators` puede ser `null`. El valor predeterminado es `false`.
- `encoding`: especifica la codificación de texto del valor binario. El valor de `encoding` puede ser `null`. El valor predeterminado es `65001` (UTF-8).

Si se especifica un registro para `quoteStyle` (y `includeLineSeparators` son `encoding null`), se pueden proporcionar los siguientes campos de registro:

- `QuoteStyle`: especifica cómo se tratan los saltos de línea entre comillas.
 - [QuoteStyle.Csv](#): los saltos de línea entre comillas se tratan como parte de los datos, no como el final de la fila actual.
 - [QuoteStyle.None](#): todos los saltos de línea se tratan como el final de la fila actual, incluso cuando se producen dentro de un valor entre comillas. Este valor es el valor predeterminado si no se especifica la `CsvStyle` opción.
- `CsvStyle`: especifica cómo se tratan las comillas. No se debe usar con [QuoteStyle.None](#).
 - [CsvStyle.QuoteAfterDelimiter](#): las comillas de un campo solo son significativas inmediatamente después de `Delimiter`.
 - [CsvStyle.QuoteAlways](#): las comillas de un campo siempre son significativas, independientemente de dónde aparezcan.

- `Delimiter`: un delimitador de caracteres único. Solo se debe usar con [CsvStyle QuoteAfterDelimiter](#).
 - `IncludeLineSeparators`: especifica si se deben incluir los caracteres de salto de línea en el texto. El valor predeterminado es `false`.
 - `Encoding`: codificación de texto del valor binario. El valor predeterminado es `65001` (UTF-8).
-

Last updated on 02/04/2026

Lines.FromText

Sintaxis

```
Lines.FromText(  
    text as text,  
    optional quoteStyle as any,  
    optional includeLineSeparators as nullable logical  
) as list
```

Acerca de

Convierte un valor de texto en una lista de valores de texto divididos en saltos de línea.

- `text`: valor de texto que se va a convertir en la lista de valores de texto.
- `quoteStyle`: especifica cómo se controlan los saltos de línea. El valor de `quoteStyle` puede ser `null`. El valor predeterminado es [QuoteStyle.None](#).
- `includeLineSeparators`: especifica si se deben incluir los caracteres de salto de línea en el texto. El valor de `includeLineSeparators` puede ser `null`. El valor predeterminado es `false`.

Si se especifica un registro para `quoteStyle` (y `includeLineSeparators` es `null`), se pueden proporcionar los siguientes campos de registro:

- `QuoteStyle`: especifica cómo se tratan los saltos de línea entre comillas.
 - [QuoteStyle.Csv](#): los saltos de línea entre comillas se tratan como parte de los datos, no como el final de la fila actual.
 - [QuoteStyle.None](#): todos los saltos de línea se tratan como el final de la fila actual, incluso cuando se producen dentro de un valor entre comillas. Este valor es el valor predeterminado si no se especifica la `CsvStyle` opción.
- `CsvStyle`: especifica cómo se tratan las comillas. No se debe usar con [QuoteStyle.None](#).
 - [CsvStyle.QuoteAfterDelimiter](#): las comillas de un campo solo son significativas inmediatamente después de `Delimiter`.
 - [CsvStyle.QuoteAlways](#): las comillas de un campo siempre son significativas, independientemente de dónde aparezcan.
- `Delimiter`: un delimitador de caracteres único. Solo se debe usar con [CsvStyle.QuoteAfterDelimiter](#).

- `IncludeLineSeparators`: especifica si se deben incluir los caracteres de salto de línea en el texto. El valor predeterminado es `false`.
-

Last updated on 02/04/2026

Lines.ToBinary

Syntax

```
Lines.ToBinary(  
    lines as list,  
    optional lineSeparator as nullable text,  
    optional encoding as nullable number,  
    optional includeByteOrderMark as nullable logical  
) as binary
```

Acerca de

Convierte una lista de texto en un valor binario mediante la codificación especificada y lineSeparator. El lineSeparator especificado se anexa a cada línea. Si no se especifica, se usan los caracteres de retorno de carro y avance de línea.

Last updated on 22/11/2025

Lines.ToText

Syntax

```
Lines.ToText(lines as list, optional lineSeparator as nullable text) as text
```

Acerca de

Convierte una lista de texto en un solo texto. El lineSeparator especificado se anexa a cada línea. Si no se especifica, se usan los caracteres de retorno de carro y avance de línea.

Last updated on 22/11/2025

Enumerar funciones

14/08/2025

Estas funciones crean y manipulan valores de lista.

Información

 Expandir tabla

Nombre	Descripción
List.Count	Devuelve el número de elementos de una lista.
List.IsEmpty	Devuelve <code>true</code> si la lista está vacía.
List.NonNullCount	Devuelve el número de elementos que no son NULL en la lista.

Selección

 Expandir tabla

Nombre	Descripción
List.Alternate	Devuelve una lista formada por todos los elementos de desplazamiento con numeración impar de una lista.
List.Buffer	Almacena en búfer una lista.
List.Distinct	Devuelve una lista de valores con duplicados quitados.
List.FindText	Devuelve una lista de valores (incluidos los campos de registro) que contienen el texto especificado.
List.First	Devuelve el primer valor de la lista o el valor predeterminado especificado si está vacío.
List.FirstN	Devuelve el primer conjunto de elementos de la lista especificando la cantidad de elementos a devolver o una condición de calificación.
List.InsertRange	Inserta valores en una lista en el índice especificado.
List.IsDistinct	Indica si hay duplicados en la lista.
List.Last	Devuelve el último valor de la lista o el valor predeterminado especificado si está vacío.

Nombre	Descripción
List.LastN	Devuelve el último valor de la lista. Opcionalmente, puede especificar cuántos valores se van a devolver o una condición apta.
List.MatchesAll	Devuelve <code>true</code> si todos los valores de la lista cumplen la función de condición.
List.MatchesAny	Devuelve <code>true</code> si cualquier valor cumple la función condition.
List.Positions	Devuelve una lista de desplazamientos para la entrada.
List.Range	Devuelve un subconjunto de la lista comenzando en un desplazamiento.
List.Select	Devuelve una lista de valores que coinciden con la condición.
List.Single	Devuelve el elemento de lista uno para una lista de longitud uno; de lo contrario, se inicia una excepción.
List.SingleOrDefault	Devuelve el único elemento de una lista de longitud uno y el valor predeterminado de una lista vacía.
List.Skip	Devuelve una lista que omite el número especificado de elementos al principio de la lista.

Funciones de transformación

 Expandir tabla

Nombre	Descripción
List.Accumulate	AAccumula un valor de resumen de los elementos de la lista.
List.Combine	Devuelve una sola lista combinando varias listas.
List.ConformToPageReader	Esta función está pensada solo para uso interno.
List.RemoveFirstN	Devuelve una lista que omite el número especificado de elementos al principio de la lista.
List.RemoveItems	Quita los elementos de la primera lista que están presentes en la segunda lista.
List.RemoveLastN	Devuelve una lista que quita el número especificado de elementos del final de la lista.
List.RemoveMatchingItems	Quita todas las coincidencias de los valores de entrada.
List.RemoveNulls	Quita todos los valores de <code>null</code> de la lista especificada.
List.RemoveRange	Quita el número de recuento de valores a partir de la posición especificada.

Nombre	Descripción
List.Repeat	Devuelve una lista que es <code>count</code> repeticiones de la lista original.
List.ReplaceMatchingItems	Reemplaza las apariciones de los valores existentes en la lista por los nuevos valores que coinciden con la condición.
List.ReplaceRange	Reemplaza el número de valores <code>count</code> a partir de <code>position</code> por los valores de reemplazo.
List.ReplaceValue	Busca en una lista el valor especificado y lo reemplaza.
List.Reverse	Invierte el orden de los valores de la lista.
List.Split	Divide la lista especificada en una lista de listas con el tamaño de página especificado.
List.Transform	Devuelve una nueva lista de valores calculados a partir de esta lista.
List.TransformMany	Devuelve una lista cuyos elementos se transforman de la lista de entrada mediante funciones especificadas.
List.Zip	Devuelve una lista de listas combinando elementos en la misma posición en varias listas.

Funciones de pertenencia

Dado que todos los valores se pueden probar para obtener igualdad, estas funciones pueden funcionar en listas heterogéneas.

[Expandir tabla](#)

Nombre	Descripción
List.AllTrue	Devuelve <code>true</code> si todas las expresiones son verdaderas.
List.AnyTrue	Devuelve true si al menos una expresión es verdadera.
List.Contains	Indica si la lista contiene el valor.
List.ContainsAll	Indica dónde una lista incluye todos los valores de otra lista.
List.ContainsAny	Indica dónde una lista incluye cualquiera de los valores de otra lista.
List.PositionOf	Devuelve los desplazamientos de un valor en una lista.
List.PositionOfAny	Devuelve el primer desplazamiento de un valor en una lista.

Operaciones de conjunto

 Expandir tabla

Nombre	Descripción
List.Difference	Devuelve la diferencia de las dos listas dadas.
List.Intersect	Devuelve la intersección de los valores de lista encontrados en la entrada.
List.Union	Devuelve la unión de los valores de lista encontrados en la entrada.

Pedido

Las funciones de ordenación realizan comparaciones. Todos los valores que se comparan deben ser comparables entre sí. Esto significa que todos deben provenir del mismo tipo de datos (o incluir null, que siempre compara los más pequeños). De lo contrario, se produce una excepción `Expression.Error`.

Entre los tipos de datos comparables se incluyen:

- Número
- Duración
- FechaHora
- Mensaje de texto
- Lógico
- Nulo

 Expandir tabla

Nombre	Descripción
List.Max	Devuelve el valor máximo o el valor predeterminado de una lista vacía.
List.MaxN	Devuelve los valores máximos de la lista. Se debe especificar el número de valores que se van a devolver o una condición de filtrado.
List.Median	Devuelve el valor medio de la lista.
List.Min	Devuelve el valor mínimo o el valor predeterminado de una lista vacía.
List.MinN	Devuelve los valores mínimos de la lista. Se puede especificar el número de valores que se van a devolver o una condición de filtrado.
List.Sort	Ordena una lista de datos según los criterios especificados.

Nombre	Descripción
List.Percentile	Devuelve uno o varios percentiles de muestra correspondientes a las probabilidades dadas.

Promedios

Estas funciones funcionan en listas homogéneas de Números, DateTimes y Duraciones.

[Expandir tabla](#)

Nombre	Descripción
List.Average	Devuelve el promedio de los valores. Funciona con valores number, date, datetime, datetimetimezone y duration.
List.Mode	Devuelve el valor más frecuente de la lista.
List.Modes	Devuelve una lista de los valores más frecuentes de la lista.
List.StandardDeviation	Devuelve una estimación basada en muestras de la desviación estándar. Esta función realiza una estimación basada en ejemplo. El resultado es un número para los valores numéricos y una duración para los valores de fecha y hora y las duraciones.

Adición

Estas funciones funcionan sobre listas homogéneas de números o duraciones.

[Expandir tabla](#)

Nombre	Descripción
List.Sum	Devuelve la suma de los elementos de la lista.

Valores numéricos

Estas funciones solo funcionan con números.

[Expandir tabla](#)

Nombre	Descripción
List.Covariance	Devuelve la covarianza entre las dos listas de números.
List.Product	Devuelve el producto de los números de la lista.

Generadores

Estas funciones generan una lista de valores.

[Expandir tabla](#)

Nombre	Descripción
List.Dates	Genera una lista de valores <code>date</code> dada un valor inicial, una cantidad y un valor de duración incremental.
List.DateTimeTimes	Genera una lista de valores <code>datetime</code> dada un valor inicial, una cantidad y un valor de duración incremental.
List.DateTimeZones	Genera una lista de valores <code>datetimezone</code> dada un valor inicial, una cantidad y un valor de duración incremental.
List.Durations	Genera una lista de valores <code>duration</code> dada un valor inicial, una cantidad y un valor de duración incremental.
List.Generate	Genera una lista de valores.
List.Numbers	Devuelve una lista de números dado un valor inicial, un recuento y un valor de incremento opcional.
List.Random	Devuelve una lista de números aleatorios.
List.Times	Genera una lista de valores <code>time</code> dada un valor inicial, una cantidad y un valor de duración incremental.

Valores de parámetro

Especificación de ocurrencia

- [Occurrence.First](#) = 0;
- [Occurrence.Last](#) = 1;
- [Occurrence.All](#) = 2;

Criterio de ordenación

- [Order.Ascending](#) = 0;
- [Order.Descending](#) = 1;

Criterios de ecuación

En muchas de las funciones de lista, el `equationCriteria` parámetro indica cómo se determina la igualdad entre los elementos de lista. Los criterios de ecuación para los valores de lista se pueden especificar como:

- Un valor de función que es:
 - Selector principal que determina el valor en la lista para aplicar los criterios de igualdad.
 - Función del comparador que se usa para especificar el tipo de comparación que se va a aplicar. Se pueden especificar funciones de comparador integradas, vaya a [Funciones de comparador](#).
- Como un valor de lista que tiene:
 - Exactamente dos elementos.
 - El primer elemento es el selector de claves tal y como se especificó anteriormente.
 - El segundo elemento es un comparador como se especificó anteriormente.

En la mayoría de las funciones de lista, la función de comparador usada en el `equationCriteria` parámetro debe ser una de las [funciones de comparador](#) integradas. En esas funciones de lista, el uso de un comparador personalizado produce un error. Sin embargo, las siguientes funciones de lista permiten usar un comparador personalizado:

- [List.Contains](#)
- [List.ContainsAll](#)
- [List.ContainsAny](#)
- [List.PositionOf](#)
- [List.PositionOfAny](#)

Para obtener más información y ejemplos, vaya a [List.Distinct](#) y [List.PositionOf](#).

Criterios de comparación

El criterio de comparación se puede proporcionar como cualquiera de los siguientes valores:

- Valor numérico para especificar un criterio de ordenación. Para obtener más información, ve a [Criterio de ordenación](#).
- Para calcular una clave que se va a usar para ordenar, se puede usar una función de un argumento.

- Para seleccionar una clave y un orden de control, el criterio de comparación puede ser una lista que contenga la clave y el orden.
- Para controlar completamente la comparación, se puede usar una función de dos argumentos que devuelve -1, 0 o 1 dada la relación entre las entradas izquierda y derecha. [Value.Compare](#) es un método que se puede usar para delegar esta lógica.

Para obtener más información y ejemplos, vaya a [List.Sort](#).

Operaciones de reemplazo

Las operaciones de reemplazo se especifican mediante un valor de lista. Cada elemento de esta lista debe ser:

- Un valor de lista de exactamente dos elementos.
- El primer elemento es el valor anterior de la lista, que se va a reemplazar.
- El segundo elemento es el nuevo valor, que debe reemplazar todas las apariciones del valor anterior en la lista.

List.Accumulate

16/10/2025

Sintaxis

```
List.Accumulate(  
    list as list,  
    seed as any,  
    accumulator as function  
) as any
```

Acerca de

Acumula un valor de resumen de los elementos de la lista especificada mediante el acumulador.

- `list`: La lista a iterar.
- `seed`: valor acumulado inicial.
- `accumulator`: función que toma el estado actual y el elemento actual y devuelve el nuevo estado.

Ejemplo 1

Acumula el valor de resumen de los elementos de la lista.

Uso

Power Query M

```
let  
    Source = List.Accumulate(  
        {1, 2, 3, 4, 5},  
        0,  
        (runningSum, nextNumber) => runningSum + nextNumber  
    )  
in  
    Source
```

Salida

Ejemplo 2

Concatene cada palabra de la lista con un espacio entre, pero no incluya un espacio al principio.

Uso

Power Query M

```
let
    Source = List.Accumulate(
        {"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog."},
        null,
        (fullTextSoFar, nextPart) =>
            Text.Combine({fullTextSoFar, nextPart}, " ")
    )
in
    Source
```

Salida "The quick brown fox jumps over the lazy dog."

Ejemplo 3

Cree una lista de horas de finalización de procesos a partir de una fecha de inicio y una lista de tiempos de ejecución de procesos.

Uso

Power Query M

```
let
    #"Process Duration" =
        {
            #duration(0,1,0,0),
            #duration(0,2,0,0),
            #duration(0,3,0,0)
        },
    #"Start Time" = #datetime(2025, 9, 8, 19, 0, 0),
    #"Process Timeline" = List.Accumulate(
        #"Process Duration",
        {#"Start Time"},
        (accumulatedTimes, nextDuration) =>
            accumulatedTimes & {List.Last(accumulatedTimes) + nextDuration}
    )
in
    #"Process Timeline"
```

Salida

Power Query M

```
{  
    #datetime(2025, 9, 8, 19, 0, 0),  
    #datetime(2025, 9, 8, 20, 0, 0),  
    #datetime(2025, 9, 8, 22, 0, 0),  
    #datetime(2025, 9, 9, 1, 0, 0)  
}
```

List.AllTrue

16/09/2025

Sintaxis

```
List.AllTrue(list as list) as logical
```

Acerca de

Devuelve true si todas las expresiones de la lista `list` son true.

Ejemplo 1

Determine si todas las expresiones de la lista {true, true, 2 > 0} son true.

Uso

Power Query M

```
List.AllTrue({true, true, 2 > 0})
```

Salida

true

Ejemplo 2

Determine si todas las expresiones de la lista {true, true, 2 < 0} son true.

Uso

Power Query M

```
List.AllTrue({true, false, 2 < 0})
```

Salida

false

List.Alternate

Syntax

```
List.Alternate(  
    list as list,  
    count as number,  
    optional repeatInterval as nullable number,  
    optional offset as nullable number  
) as list
```

Acerca de

Devuelve una lista formada por todos los elementos de desplazamiento con numeración impar de una lista. Alterna entre tomar y omitir valores de la lista `list` en función de los parámetros.

- `count`: especifica el número de valores que se omiten cada vez.
- `repeatInterval`: intervalo de repetición opcional para indicar cuántos valores se agregan entre los valores omitidos.
- `offset`: parámetro de desplazamiento de opción para empezar a omitir los valores en el desplazamiento inicial.

Ejemplo 1

Cree una lista a partir de {1..10} que omita el primer número.

Uso

Power Query M

```
List.Alternate({1..10}, 1)
```

Salida

```
{2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Ejemplo 2

Cree una lista a partir de {1..10} que omita todos los demás números.

Uso

Power Query M

```
List.Alternate({1..10}, 1, 1)
```

Salida

```
{2, 4, 6, 8, 10}
```

Ejemplo 3

Cree una lista a partir de {1..10} que comience en 1 y omita todos los demás números.

Uso

Power Query M

```
List.Alternate({1..10}, 1, 1, 1)
```

Salida

```
{1, 3, 5, 7, 9}
```

Ejemplo 4

Cree una lista a partir de {1..10} que comience en 1, omita un valor, mantenga dos valores, etc.

Uso

Power Query M

```
List.Alternate({1..10}, 1, 2, 1)
```

Salida

```
{1, 3, 4, 6, 7, 9, 10}
```

Last updated on 22/11/2025

List.AnyTrue

16/09/2025

Sintaxis

```
List.AnyTrue(list as list) as logical
```

Acerca de

Devuelve true si alguna expresión de la lista `list` es true.

Ejemplo 1

Determine si alguna de las expresiones de la lista {true, false, 2 > 0} es true.

Uso

Power Query M

```
List.AnyTrue({true, false, 2>0})
```

Salida

true

Ejemplo 2

Determine si alguna de las expresiones de la lista {2 = 0, false, 2 < 0} es true.

Uso

Power Query M

```
List.AnyTrue({2 = 0, false, 2 < 0})
```

Salida

false

List.Average

Syntax

```
List.Average(list as list, optional precision as nullable number) as any
```

Acerca de

Devuelve el valor medio de los elementos de la lista, `list`. El resultado se da en el mismo tipo de datos que los valores de la lista. Solo funciona con los valores number, date, time, datetime, datetimezone y duration. Si la lista está vacía, se devuelve null.

Ejemplo 1

Busque el promedio de la lista de números, `{3, 4, 6}`.

Uso

Power Query M

```
List.Average({3, 4, 6})
```

Salida

```
4.33333333333333
```

Ejemplo 2

Busque el promedio de los valores de fecha 1 de enero de 2011, 2 de enero de 2011 y 3 de enero de 2011.

Uso

Power Query M

```
List.Average({#date(2011, 1, 1), #date(2011, 1, 2), #date(2011, 1, 3)})
```

Salida

```
#date(2011, 1, 2)
```

List.Buffer

Syntax

```
List.Buffer(list as list) as list
```

Acerca de

Almacena en búfer la lista `list` en memoria. El resultado de esta llamada es una lista estable.

Ejemplo 1

Cree una copia estable de la lista {1..10}.

Uso

Power Query M

```
List.Buffer({1..10})
```

Salida

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Last updated on 22/11/2025

List.Combine

Syntax

```
List.Combine(lists as list) as list
```

Acerca de

Toma una lista de listas, `lists` y las combina en una sola lista nueva.

Ejemplo 1

Combine las dos listas simples {1, 2} y {3, 4}.

Uso

Power Query M

```
List.Combine({{1, 2}, {3, 4}})
```

Salida

Power Query M

```
{  
    1,  
    2,  
    3,  
    4  
}
```

Ejemplo 2

Combine las dos listas, {1, 2} y {3, {4, 5}}, una de las cuales contiene una lista anidada.

Uso

Power Query M

```
List.Combine({{1, 2}, {3, {4, 5}}})
```

Salida

Power Query M

```
{  
    1,  
    2,  
    3,  
    {4, 5}  
}
```

Last updated on 22/11/2025

List.ConformToPageReader

16/09/2025

Sintaxis

```
List.ConformToPageReader(list as list, optional options as nullable record) as table
```

Acerca de

Esta función está pensada solo para uso interno.

List.Contains

16/09/2025

Sintaxis

```
List.Contains(  
    list as list,  
    value as any,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si la lista contiene el valor especificado. Devuelve `true` si el valor se encuentra en la lista; de lo contrario, `false` devuelve .

- `list`: La lista en la que buscar.
- `value`: valor que se va a buscar en la lista.
- `equationCriteria`: (Opcional) Comparador usado para determinar si los dos valores son iguales.

Ejemplo 1

Determine si la lista {1, 2, 3, 4, 5} contiene 3.

Uso

Power Query M

```
List.Contains({1, 2, 3, 4, 5}, 3)
```

Salida

```
true
```

Ejemplo 2

Determine si la lista {1, 2, 3, 4, 5} contiene 6.

Uso

Power Query M

```
List.Contains({1, 2, 3, 4, 5}, 6)
```

Salida

```
false
```

Ejemplo 3

Si se omite el caso, determine si la lista contiene "rhubarb".

Uso

Power Query M

```
List.Contains({"Pears", "Bananas", "Rhubarb", "Peaches"},  
             "rhubarb",  
             Comparer.OrdinalIgnoreCase  
            )
```

Salida

```
true
```

Ejemplo 4

Determine si la lista contiene la fecha 8 de abril de 2022.

Uso

Power Query M

```
let  
    Source = {#date(2024, 2, 23), #date(2023, 12, 2), #date(2022, 4, 8),  
              #date(2021, 7, 6)},  
    ContainsDate = List.Contains(Source, Date.From("4/8/2022"))  
in  
    ContainsDate
```

Salida

```
true
```


Contenido relacionado

[Criterios de ecuación](#)

List.ContainsAll

Sintaxis

```
List.ContainsAll(  
    list as list,  
    values as list,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si la lista incluye todos los valores de otra lista. Devuelve `true` si todos los valores se encuentran en la lista; de lo contrario, `false` devuelve .

- `list`: La lista en la que buscar.
- `values`: la lista de valores que se van a buscar en la primera lista.
- `equationCriteria`: (Opcional) Comparador usado para determinar si dos valores son iguales.

Ejemplo 1

Determine si la lista {1, 2, 3, 4, 5} contiene 3 y 4.

Uso

Power Query M

```
List.ContainsAll({1, 2, 3, 4, 5}, {3, 4})
```

Salida

`true`

Ejemplo 2

Determine si la lista {1, 2, 3, 4, 5} contiene 5 y 6.

Uso

Power Query M

```
List.ContainsAll({1, 2, 3, 4, 5}, {5, 6})
```

Salida

```
false
```

Ejemplo 3

Determine si la lista contiene un perro y un caballo, mientras ignora mayúsculas.

Uso

Power Query M

```
List.ContainsAll({"dog", "cat", "racoon", "horse", "rabbit"}, {"DOG", "Horse"},  
Comparer.OrdinalIgnoreCase)
```

Salida

```
true
```

Ejemplo 4

Determine si la lista contiene las fechas del 8 de abril de 2022 y del 6 de julio de 2021.

Uso

Power Query M

```
let  
    Source = {#date(2024, 2, 23), #date(2023, 12, 2), #date(2022, 4, 8),  
#date(2021, 7, 6)},  
    ContainsDates = List.ContainsAll(Source, {#date(2022, 4, 8), #date(2021, 7,  
6)})  
in  
    ContainsDates
```

Salida

```
true
```

Contenido relacionado

Last updated on 04/04/2026

List.ContainsAny

Sintaxis

```
List.ContainsAny(  
    list as list,  
    values as list,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si la lista contiene cualquiera de los valores de otra lista. Devuelve `true` si los valores se encuentran en la lista; de lo contrario, `false` devuelve .

- `list`: La lista en la que buscar.
- `values`: la lista de valores que se van a buscar en la primera lista.
- `equationCriteria`: (Opcional) Comparador usado para determinar si dos valores son iguales.

Ejemplo 1

Determine si la lista {1, 2, 3, 4, 5} contiene 3 o 9.

Uso

Power Query M

```
List.ContainsAny({1, 2, 3, 4, 5}, {3, 9})
```

Salida

`true`

Ejemplo 2

Determine si la lista {1, 2, 3, 4, 5} contiene 6 o 7.

Uso

Power Query M

```
List.ContainsAny({1, 2, 3, 4, 5}, {6, 7})
```

Salida

```
false
```

Ejemplo 3

Comprueba si la lista contiene un caballo o un búho, sin tener en cuenta las mayúsculas.

Uso

Power Query M

```
List.ContainsAny({"dog", "cat", "racoon", "horse", "rabbit"}, {"Horse", "OWL"},  
Comparer.OrdinalIgnoreCase)
```

Salida

```
true
```

Ejemplo 4

Determine si la lista contiene una fecha del 8 de abril de 2022 o del 12 de enero de 2021.

Uso

Power Query M

```
let  
    Source = {#date(2024, 2, 23), #date(2023, 12, 2), #date(2022, 4, 8),  
    #date(2021, 7, 6)},  
    ContainsDates = List.ContainsAny(Source, {Date.From("Apr 8, 2022"),  
    Date.From("Jan 11, 2021")})  
in  
    ContainsDates
```

Salida

```
true
```

Contenido relacionado

Last updated on 04/04/2026

List.Count

24/10/2025

Sintaxis

```
List.Count(list as list) as number
```

Acerca de

Devuelve el número de elementos de la lista especificada.

- **list**: La lista que debe examinarse.

Ejemplo 1

Busque el número de valores de la lista {1, 2, 3}.

Uso

Power Query M

```
List.Count({1, 2, 3})
```

Salida

3

Ejemplo 2

Encuentra el número de veces que el rojo aparece en una lista de colores, ignorando mayúsculas y minúsculas.

Uso

Power Query M

```
let
    colors = {"Red", "Blue", "GREEN", "red", "Yellow", "RED"},
    redCount = List.Count(
        List.Select(colors, each Text.Lower(_) = "red")
    )
```



```
)  
in  
    redCount
```

Salida

3

Ejemplo 3

Busque el número de clientes de una lista que compró más de \$100.

Uso

Power Query M

```
let  
    customers = {  
        [CustomerID = 101, CustomerName = "Alice", AmountPurchased = 150],  
        [CustomerID = 102, CustomerName = "Bob", AmountPurchased = 75],  
        [CustomerID = 103, CustomerName = "Carol", AmountPurchased = 50],  
        [CustomerID = 104, CustomerName = "Dave", AmountPurchased = 90],  
        [CustomerID = 105, CustomerName = "Eve", AmountPurchased = 120]  
    },  
    highValueCustomerCount = List.Count(  
        List.Select(customers, each [AmountPurchased] > 100)  
    )  
in  
    highValueCustomerCount
```

Salida

2

List.Covariance

Sintaxis

```
List.Covariance(numberList1 as list, numberList2 as list) as nullable number
```

Acerca de

Devuelve la covarianza entre dos listas, `numberList1` y `numberList2`. `numberList1` y `numberList2` deben contener el mismo número de `number` valores.

Ejemplo

Calcule la covarianza entre dos listas.

Uso

Power Query M

```
List.Covariance({1, 2, 3}, {1, 2, 3})
```

Salida

```
0.666666666666666667
```

Last updated on 25/03/2026

List.Dates

16/09/2025

Sintaxis

```
List.Dates(  
    start as date,  
    count as number,  
    step as duration  
) as list
```

Acerca de

Devuelve una lista de `date` valores de tamaño `count`, empezando por `start`. El incremento especificado, `step`, es un `duration` valor que se agrega a cada valor.

Ejemplo 1

Cree una lista de 5 valores a partir de la Nochevieja (`#date(2011, 12, 31)`) incrementando en 1 día (`#duration(1, 0, 0, 0)`).

Uso

Power Query M

```
List.Dates(#date(2011, 12, 31), 5, #duration(1, 0, 0, 0))
```

Salida

Power Query M

```
{  
    #date(2011, 12, 31),  
    #date(2012, 1, 1),  
    #date(2012, 1, 2),  
    #date(2012, 1, 3),  
    #date(2012, 1, 4)  
}
```

Contenido relacionado

#duration

List.DateTimes

16/09/2025

Sintaxis

```
List.DateTimes(  
    start as datetime,  
    count as number,  
    step as duration  
) as list
```

Acerca de

Devuelve una lista de `datetime` valores de tamaño `count`, empezando por `start`. El incremento especificado, `step`, es un `duration` valor que se agrega a cada valor.

Ejemplo

Cree una lista de 10 valores a partir de 5 minutos antes del Día del Año Nuevo (`#datetime(2011, 12, 31, 23, 55, 0)`) incrementando en 1 minuto (`#duration(0, 0, 1, 0)`).

Uso

Power Query M

```
List.DateTimes(#datetime(2011, 12, 31, 23, 55, 0), 10, #duration(0, 0, 1, 0))
```

Salida

Power Query M

```
{  
    #datetime(2011, 12, 31, 23, 55, 0),  
    #datetime(2011, 12, 31, 23, 56, 0),  
    #datetime(2011, 12, 31, 23, 57, 0),  
    #datetime(2011, 12, 31, 23, 58, 0),  
    #datetime(2011, 12, 31, 23, 59, 0),  
    #datetime(2012, 1, 1, 0, 0, 0),  
    #datetime(2012, 1, 1, 0, 1, 0),  
    #datetime(2012, 1, 1, 0, 2, 0),  
    #datetime(2012, 1, 1, 0, 3, 0),  
}
```

```
#datetime(2012, 1, 1, 0, 4, 0)
```

```
}
```

List.DateTimeZones

16/09/2025

Sintaxis

```
List.DateTimeZones(  
    start as datetimezone,  
    count as number,  
    step as duration  
) as list
```

Acerca de

Devuelve una lista de `datetimezone` valores de tamaño `count`, empezando por `start`. El incremento especificado, `step`, es un `duration` valor que se agrega a cada valor.

Ejemplo 1

Cree una lista de 10 valores a partir de 5 minutos antes del Día del Año Nuevo (`#datetimezone(2011, 12, 31, 23, 55, 0, -8, 0)`) incrementando en 1 minuto (`#duration(0, 0, 1, 0, 0)`).

Uso

Power Query M

```
List.DateTimeZones(#datetimezone(2011, 12, 31, 23, 55, 0, -8, 0), 10, #duration(0, 0, 1, 0, 0))
```

Salida

Power Query M

```
{  
    #datetimezone(2011, 12, 31, 23, 55, 0, -8, 0),  
    #datetimezone(2011, 12, 31, 23, 56, 0, -8, 0),  
    #datetimezone(2011, 12, 31, 23, 57, 0, -8, 0),  
    #datetimezone(2011, 12, 31, 23, 58, 0, -8, 0),  
    #datetimezone(2011, 12, 31, 23, 59, 0, -8, 0),  
    #datetimezone(2012, 1, 1, 0, 0, 0, -8, 0),  
    #datetimezone(2012, 1, 1, 0, 1, 0, -8, 0),  
    #datetimezone(2012, 1, 1, 0, 2, 0, -8, 0),  
}
```

```
#datetimezone(2012, 1, 1, 0, 3, 0, -8, 0),  
#datetimezone(2012, 1, 1, 0, 4, 0, -8, 0)  
}
```


List.Difference

16/09/2025

```
List.Difference(  
    list1 as list,  
    list2 as list,  
    optional equationCriteria as any  
) as list
```

Acerca de

Devuelve los elementos de la lista `list1` que no están en la lista `list2`. Se admiten valores duplicados. Se puede especificar un valor opcional de criterio de ecuación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo 1

Buscar los elementos de la lista {1, 2, 3, 4, 5} que no están en la lista {4, 5, 3}.

Uso

Power Query M

```
List.Difference({1, 2, 3, 4, 5}, {4, 5, 3})
```

Salida

```
{1, 2}
```

Ejemplo 2

Buscar los elementos de la lista {1, 2} que no están en la lista {1, 2, 3}.

Uso

Power Query M

```
List.Difference({1, 2}, {1, 2, 3})
```

Salida



Contenido relacionado

[Criterios de ecuación](#)

List.Distinct

16/09/2025

Sintaxis

```
List.Distinct(list as list, optional equationCriteria as any) as list
```

Acerca de

Devuelve una lista que contiene todos los valores de la lista especificada con duplicados quitados. Si la lista especificada está vacía, el resultado es una lista vacía.

- **list**: lista a partir de la cual se extraen valores distintos.
- **equationCriteria**: (Opcional) Especifica cómo se determina la igualdad al comparar valores. Este parámetro puede ser una función selectora de claves, una función de comparador o una lista que contenga un selector de claves y un comparador.

Ejemplo 1

Quitar los duplicados de la lista {1, 1, 2, 3, 3, 3}.

Uso

Power Query M

```
List.Distinct({1, 1, 2, 3, 3, 3})
```

Salida

```
{1, 2, 3}
```

Ejemplo 2

A partir del final de la lista, seleccione las frutas que tienen una longitud de texto única.

Uso

Power Query M

```
let
    Source = {"Apple", "Banana", "Cherry", "Date", "Fig"},
    Result = List.Distinct(List.Reverse(Source), each Text.Length(_))
in
    Result
```

Salida

```
{"Fig", "Date", "Cherry", "Apple"}
```

Ejemplo 3

A partir del principio de la lista, seleccione las frutas únicas al ignorar mayúsculas y minúsculas.

Uso

Power Query M

```
let
    Source = {"apple", "Pear", "aPPlE", "banana", "ORANGE", "pear", "Banana",
    "Cherry"},
    Result = List.Distinct(Source, Comparer.OrdinalIgnoreCase)
in
    Result
```

Salida

```
{"apple", "Pear", "banana", "ORANGE", "Cherry"}
```

Ejemplo 4

Extraiga de una lista de listas las primeras listas con nombres de país únicos al omitir mayúsculas y minúsculas. Coloque las listas extraídas en las filas de una nueva tabla.

Uso

Power Query M

```
let
    Source = {
        {"USA", #date(2023, 8, 1), 567},
        {"canada", #date(2023, 8, 1), 254},
        {"Usa", #date(2023, 7, 1), 450},
        {"CANADA", #date(2023, 6, 1), 357},
        {"Panama", #date(2023, 6, 2), 20},
        {"panama", #date(2023, 7, 1), 40}
    }
```

```

    },
    DistinctByCountry = List.Distinct(
        Source,
        {each _{0}, Comparer.OrdinalIgnoreCase}
    ),
    ToTable = Table.FromRows(DistinctByCountry, {"Country", "Date", "Value"}),
    ChangeTypes = Table.TransformColumnTypes(
        ToTable, {"Country", type text}, {"Date", type date}, {"Value",
Int64.Type}}
    )
in
    ChangeTypes

```

Salida

Power Query M

```

#table(type table[Country = text, Date = date, Value = Int64.Type],
{
    {"USA", #date(2023, 8, 1), 567},
    {"canada", #date(2023, 8, 1), 254},
    {"Panama", #date(2023, 6, 2), 20}
}
)

```

Contenido relacionado

[Criterios de ecuación](#)

List.Durations

Sintaxis

```
List.Durations(  
    start as duration,  
    count as number,  
    step as duration  
) as list
```

Acerca de

Devuelve una lista de `count duration` valores, comenzando por `start` y incrementado por el especificado `duration step`.

Ejemplo

Cree una lista de 5 valores a partir de 1 hora e incremente en una hora.

Uso

Power Query M

```
List.Durations(#duration(0, 1, 0, 0), 5, #duration(0, 1, 0, 0))
```

Salida

Power Query M

```
{  
    #duration(0, 1, 0, 0),  
    #duration(0, 2, 0, 0),  
    #duration(0, 3, 0, 0),  
    #duration(0, 4, 0, 0),  
    #duration(0, 5, 0, 0)  
}
```

List.FindText

Sintaxis

```
List.FindText(list as list, text as text) as list
```

Acerca de

Devuelve una lista de los valores de la lista `list` que contenía el valor `text`.

Ejemplo

Busque los valores de texto de la lista {"a", "b", "ab"} que coincidan con "a".

Uso

Power Query M

```
List.FindText({"a", "b", "ab"}, "a")
```

Salida

```
{"a", "ab"}
```

Last updated on 25/03/2026

List.First

16/09/2025

Sintaxis

```
List.First(list as list, optional defaultValue as any) as any
```

Acerca de

Devuelve el primer elemento de la lista `list` o el valor predeterminado opcional, `defaultValue`, si la lista está vacía. Si la lista está vacía y no se especifica un valor predeterminado, la función devuelve `null`.

Ejemplo 1

Busque el primer valor de la lista {1, 2, 3}.

Uso

Power Query M

```
List.First({1, 2, 3})
```

Salida

1

Ejemplo 2

Busque el primer valor de la lista {}. Si la lista está vacía, devuelva -1.

Uso

Power Query M

```
List.First({}, -1)
```

Salida

List.FirstN

16/09/2025

Sintaxis

```
List.FirstN(list as list, countOrCondition as any) as any
```

Acerca de

- Si se especifica un número, se devuelven hasta ese número de elementos.
- Si se especifica una condición, se devuelven todos los elementos que cumplen inicialmente la condición. Si un elemento no cumple la condición, no se tienen en cuenta más elementos.

Ejemplo 1

Busque los valores iniciales de la lista {3, 4, 5, -1, 7, 8, 2} que son mayores que 0.

Uso

Power Query M

```
List.FirstN({3, 4, 5, -1, 7, 8, 2}, each _ > 0)
```

Salida

```
{3, 4, 5}
```

List.Generate

16/09/2025

Sintaxis

```
List.Generate(  
    initial as function,  
    condition as function,  
    next as function,  
    optional selector as nullable function  
) as list
```

Acerca de

Genera una lista de valores mediante las funciones proporcionadas. La función `initial` genera un valor de candidato inicial, que luego se prueba con `condition`. Si se aprueba el valor candidato, se devuelve como parte de la lista resultante y se genera el siguiente valor candidato pasando el valor recién aprobado a `next`. Una vez que un valor candidato no coincide con `condition`, el proceso de generación de listas se detiene. También se puede proporcionar un parámetro opcional, `selector`, para transformar los elementos de la lista resultante.

Ejemplo 1

Cree una lista empezando por diez, disminuyendo repetidamente por uno y asegurándose de que cada elemento sea mayor que cero.

Uso

Power Query M

```
List.Generate(() => 10, each _ > 0, each _ - 1)
```

Salida

```
{10, 9, 8, 7, 6, 5, 4, 3, 2, 1}
```

Ejemplo 2

Genere una lista de registros que contengan x e y, donde x es un valor e y es una lista. x debe permanecer menor que 10 y representar el número de elementos de la lista y. Una vez generada la lista, devuelva solo los valores x.

Uso

Power Query M

```
List.Generate(  
    () => [x = 1, y = {}],  
    each [x] < 10,  
    each [x = List.Count([y]), y = [y] & {x}],  
    each [x]  
)
```

Salida

```
{1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

List.InsertRange

16/09/2025

Sintaxis

```
List.InsertRange(  
    list as list,  
    index as number,  
    values as list  
) as list
```

Acerca de

Devuelve una nueva lista generada al insertar los valores de `values` en `list` en `index`. La primera posición de la lista está en el índice 0.

- `list`: la lista de destino donde se van a insertar los valores.
- `index`: índice de la lista de destino(`list`) donde se van a insertar los valores. La primera posición de la lista está en el índice 0.
- `values`: la lista de valores que se van a insertar en `list`.

Ejemplo 1

Inserte la lista ({3, 4}) en la lista de destino ({1, 2, 5}) en el índice 2.

Uso

Power Query M

```
List.InsertRange({1, 2, 5}, 2, {3, 4})
```

Salida

Power Query M

```
{  
    1,  
    2,  
    3,  
    4,
```

```
    5  
}
```

Ejemplo 2

Inserte una lista con una lista anidada ({1, {1.1, 1.2}}) en una lista de destino ({2, 3, 4}) en el índice 0.

Uso

Power Query M

```
List.InsertRange({2, 3, 4}, 0, {1, {1.1, 1.2}})
```

Salida

Power Query M

```
{  
    1,  
    {  
        1.1,  
        1.2  
    },  
    2,  
    3,  
    4  
}
```

List.Intersect

16/09/2025

Sintaxis

```
List.Intersect(lists as list, optional equationCriteria as any) as list
```

Acerca de

Devuelve la intersección de los valores de lista encontrados en la lista de entrada **lists**. Se puede especificar un parámetro opcional, **equationCriteria**.

Ejemplo 1

Hallar la intersección de las listas {1..5}, {2..6}, {3..7}.

Uso

Power Query M

```
List.Intersect({{1..5}, {2..6}, {3..7}})
```

Salida

```
{3, 4, 5}
```

Contenido relacionado

[Criterios de ecuación](#)

List.IsDistinct

16/09/2025

Sintaxis

```
List.IsDistinct(list as list, optional equationCriteria as any) as logical
```

Acerca de

Devuelve un valor lógico si hay duplicados en la lista `list`; `true` si la lista es distinta, `false` si hay valores duplicados.

Ejemplo 1

Se busca si la lista {1, 2, 3} es distinta (es decir, no hay duplicados).

Uso

Power Query M

```
List.IsDistinct({1, 2, 3})
```

Salida

`true`

Ejemplo 2

Se busca si la lista {1, 2, 3, 3} es distinta (es decir, no hay duplicados).

Uso

Power Query M

```
List.IsDistinct({1, 2, 3, 3})
```

Salida

`false`

Contenido relacionado

[Criterios de ecuación](#)

List.IsEmpty

16/09/2025

Sintaxis

```
List.IsEmpty(list as list) as logical
```

Acerca de

Devuelve `true` si la lista, `list`, no contiene valores (longitud 0). Si la lista contiene valores (longitud > 0), devuelve `false`.

Ejemplo 1

Busque si la lista {} está vacía.

Uso

```
Power Query M
```

```
List.IsEmpty({})
```

Salida

```
true
```

Ejemplo 2

Busque si la lista {1, 2} está vacía.

Uso

```
Power Query M
```

```
List.IsEmpty({1, 2})
```

Salida

```
false
```


List.Last

Sintaxis

```
List.Last(list as list, optional defaultValue as any) as any
```

Acerca de

Devuelve el último elemento de la lista especificada o el valor predeterminado opcional si la lista está vacía.

- `list`: La lista que debe examinarse.
- `defaultValue`: (Opcional) Valor predeterminado que se va a devolver si la lista está vacía.
Si la lista está vacía y no se especifica un valor predeterminado, la función devuelve `null`.

Ejemplo 1

Busque el último valor de la lista {1, 2, 3}.

Uso

Power Query M

```
List.Last({1, 2, 3})
```

Salida

3

Ejemplo 2

Busque el último valor de la lista {} o -1 si está vacío.

Uso

Power Query M

```
List.Last({}, -1)
```

Salida

Last updated on 01/11/2025

List.LastN

Sintaxis

```
List.LastN(list as list, optional countOrCondition as any) as any
```

Acerca de

Devuelve una lista del último elemento o elementos de la lista especificada.

- **list**: La lista que debe examinarse. Si la lista está vacía, se devuelve una lista vacía.
- **countOrCondition**: (Opcional) Admite la recopilación de varios elementos o elementos de filtrado. Aunque este parámetro aparece como opcional, se produce un error si este valor no se proporciona o es **null**. Este parámetro se puede especificar de dos maneras:
 - Si se especifica un número, se devuelven hasta esa cantidad de elementos.
 - Si se especifica una condición, se devuelven todos los elementos que cumplen la condición, comenzando al final de la lista. Si un elemento no cumple la condición, no se tienen en cuenta más elementos.

Ejemplo 1

Busque el último valor de la lista {3, 4, 5, -1, 7, 8, 2}.

Uso

Power Query M

```
List.LastN({3, 4, 5, -1, 7, 8, 2}, 1)
```

Salida

```
{2}
```

Ejemplo 2

Busque los últimos valores de la lista {3, 4, 5, -1, 7, 8, 2} que son mayores que 0.

Uso

Power Query M

```
List.LastN({3, 4, 5, -1, 7, 8, 2}, each _ > 0)
```

Salida

```
{7, 8, 2}
```

Last updated on 01/11/2025

List.MatchesAll

Sintaxis

```
List.MatchesAll(list as list, condition as function) as logical
```

Acerca de

Devuelve `true` si todos los valores de la lista cumplen la función de condición; de lo contrario, devuelve `false`.

- `list`: la lista que contiene los valores que se van a comprobar.
- `condition`: condición para comprobar los valores de la lista.

Ejemplo 1

Determine si todos los valores de la lista {11, 12, 13} son mayores que 10.

Uso

Power Query M

```
List.MatchesAll({11, 12, 13}, each _ > 10)
```

Salida

```
true
```

Ejemplo 2

Determine si todos los valores de la lista {1, 2, 3} son mayores que 10.

Uso

Power Query M

```
List.MatchesAll({1, 2, 3}, each _ > 10)
```

Salida

false

Ejemplo 3

Determine si todos los valores de texto de la lista contienen "anna" ignorando mayúsculas y minúsculas.

Uso

Power Query M

```
let
    Source = {"Savannah", "Annabelle", "Annals", "wannabe", "MANNA"},
    Result = List.MatchesAll(Source, each Text.Contains(_, "anna",
        Comparer.OrdinalIgnoreCase))
in
    Result
```

Salida

true

Ejemplo 4

Determine si todas las fechas contienen el año 2021.

Uso

Power Query M

```
let
    Source = {#date(2021, 11, 28), #date(2021, 1, 14), #date(2021, 12, 31),
        #date(2021, 7, 6)},
    Result = List.MatchesAll(Source, each Date.Year(_) = 2021)
in
    Result
```

Salida

true

List.MatchesAny

16/09/2025

Sintaxis

```
List.MatchesAny(list as list, condition as function) as logical
```

Acerca de

Devuelve `true` si cualquiera de los valores de la lista cumple la función de condición; de lo contrario, devuelve `false`.

- `list`: la lista que contiene los valores que se van a comprobar.
- `condition`: condición para comprobar los valores de la lista.

Ejemplo 1

Determine si alguno de los valores de la lista {9, 10, 11} es mayor que 10.

Uso

Power Query M

```
List.MatchesAny({9, 10, 11}, each _ > 10)
```

Salida

```
true
```

Ejemplo 2

Determine si alguno de los valores de la lista {1, 2, 3} es mayor que 10.

Uso

Power Query M

```
List.MatchesAny({1, 2, 3}, each _ > 10)
```

Salida

false

Ejemplo 3

Determine si alguno de los valores de texto de la lista contiene "cat" sin distinguir entre mayúsculas y minúsculas.

Uso

Power Query M

```
let
    Source = {"A Brown Fox", "A Loyal Dog", "A Curious Cat", "A Wild Horse", "A Rascally Rabbit"},
    Result = List.MatchesAny(Source, each Text.Contains(_, "cat", Comparer.OrdinalIgnoreCase))
in
    Result
```

Salida

true

Ejemplo 4

Determine si alguna de las fechas contiene el año 2021.

Uso

Power Query M

```
let
    Source = {#date(2024, 11, 28), #date(2023, 1, 14), #date(2021, 12, 31), #date(2025, 7, 6)},
    Result = List.MatchesAny(Source, each Date.Year(_) = 2021)
in
    Result
```

Salida

true

List.Max

18/09/2025

Sintaxis

```
List.Max(  
    list as list,  
    optional default as any,  
    optional comparisonCriteria as any,  
    optional includeNulls as nullable logical  
) as any
```

Acerca de

Devuelve el elemento máximo de la lista o el valor predeterminado opcional si la lista está vacía.

- **list**: lista de valores.
- **default**: (Opcional) Valor que se va a devolver si la lista está vacía.
- **comparisonCriteria**: (Opcional) Función que se usa para transformar los valores antes de compararlos. Si este parámetro es **null**, los valores se comparan sin ninguna transformación.
- **includeNulls**: (Opcional) Indica si **null** los valores de la lista deben incluirse para determinar el elemento máximo. El valor predeterminado es **true**.

Ejemplo 1

Busque el valor máximo en la lista especificada.

Uso

Power Query M

```
List.Max({1, 4, 7, 3, -2, 5}, 1)
```

Salida

Ejemplo 2

Busque el valor máximo en la lista especificada o devuelva -1 si está vacío.

Uso

Power Query M

```
List.Max({}, -1)
```

Salida

-1

Ejemplo 3

Busque el elemento en una lista de valores de texto que aparece último en orden alfabético. Si la lista está vacía, devuelva "none".

Uso

Power Query M

```
let
    Source = {"boy", "dog", "girl", "zebra", "cat", "mouse", "rabbit"},
    MaxText = List.Max(Source, "none")
in
    MaxText
```

Salida

"zebra"

Ejemplo 4

Encuentra la fecha más reciente de una lista de fechas alemanas. Si la lista está vacía, devuelva el 1 de enero de 2000.

Uso

Power Query M

```
let
    Source = {"12.02.2024", "15.05.2025", "10.10.2021", "16.01.2025",
"30.12.2022"},
```

```
MaxDate = List.Max(Source, #date(2000, 1, 1), each Date.FromText(_, [Culture =  
"de-DE"])))  
in  
MaxDate
```

Salida

```
"15.05.2025"
```

Contenido relacionado

[Criterios de comparación](#)

List.MaxN

16/10/2025

Sintaxis

```
List.MaxN(  
    list as list,  
    countOrCondition as any,  
    optional comparisonCriteria as any,  
    optional includeNulls as nullable logical  
) as list
```

Acerca de

Devuelve los valores máximos de la lista especificada. Una vez ordenadas las filas, se pueden especificar parámetros opcionales para filtrar aún más el resultado.

- **list**: la lista de valores.
- **countOrCondition**: especifica el número de valores que se van a devolver o una condición de filtro. Si se especifica un número, se devuelve una lista de hasta **countOrCondition** elementos en orden descendente. Si se especifica una condición, la lista devuelta incluye todos los elementos que cumplen la condición.
- **comparisonCriteria**: (Opcional) Función que se usa para transformar los valores antes de compararlos. Si este parámetro es **null**, los valores se comparan sin ninguna transformación.
- **includeNulls**: (Opcional) Indica si **null** los valores de la lista deben incluirse para determinar el elemento máximo. El valor predeterminado es **true**.

Ejemplo 1

Busque los cinco valores principales de la lista especificada.

Uso

Power Query M

```
List.MaxN({3, 4, 5, -1, 7, 8, 2}, 5)
```

Salida

```
{8, 7, 5, 4, 3}
```

Ejemplo 2

Busque las palabras con más de 3 caracteres.

Uso

Power Query M

```
List.MaxN(  
    {"boy", "dog", "pony", "cat", "rabbit", "bat"},  
    each Text.Length(_) > 3  
)
```

Salida

```
{"rabbit", "pony"}
```

Ejemplo 3

Encuentra las tres fechas más recientes de una lista de fechas alemanas.

Uso

Power Query M

```
let  
    Source = {"12.02.2024", "15.05.2025", "10.10.2021", "16.01.2025",  
    "30.12.2022"},  
    MaxDate = List.MaxN(Source, 3, each Date.FromText(_, [Culture = "de-DE"]))  
in  
    MaxDate
```

Salida

Power Query M

```
{  
    "15.05.2025",  
    "16.01.2025",  
    "12.02.2024"  
}
```


Contenido relacionado

[Criterios de comparación](#)

List.Median

Sintaxis

```
List.Median(list as list, optional comparisonCriteria as any) as any
```

Acerca de

Devuelve el elemento de mediana de la lista `list`. Esta función devuelve `null` si la lista no contiene ningún valor que no sea `null`. Si el número de elementos es par, la función elige el menor de los dos elementos medianos, a menos que la lista esté formada en su totalidad por valores de fecha y hora, duración, números u horas, en cuyo caso devuelve la media de los dos elementos.

Ejemplo

Hallar la mediana de la lista `{5, 3, 1, 7, 9}`.

Uso

Power Query M

```
powerquery-mList.Median({5, 3, 1, 7, 9})
```

Salida

5

Contenido relacionado

[Criterios de comparación](#)

Last updated on 25/03/2026

List.Min

16/09/2025

Sintaxis

```
List.Min(  
    list as list,  
    optional default as any,  
    optional comparisonCriteria as any,  
    optional includeNulls as nullable logical  
) as any
```

Acerca de

Devuelve el elemento mínimo de la lista `list` o el valor predeterminado `default` opcional si la lista está vacía. Se puede especificar un valor de criterio de comparación `comparisonCriteria` opcional para determinar cómo se van a comparar los elementos de la lista. Si este parámetro es NULL, se usa el comparador predeterminado.

Ejemplo 1

Hallar el valor mínimo de la lista {1, 4, 7, 3, -2, 5}.

Uso

Power Query M

```
List.Min({1, 4, 7, 3, -2, 5})
```

Salida

-2

Ejemplo 2

Hallar el valor mínimo de la lista {} o devolver -1 si está vacía.

Uso

Power Query M

```
List.Min({}, -1)
```

Salida

-1

Contenido relacionado

[Criterios de comparación](#)

List.MinN

Sintaxis

```
List.MinN(  
    list as list,  
    countOrCondition as any,  
    optional comparisonCriteria as any,  
    optional includeNulls as nullable logical  
) as list
```

Acerca de

Devuelve los valores mínimos de la lista, `list`. El parámetro, `countOrCondition`, especifica el número de valores que se va a devolver o una condición de filtrado. El parámetro opcional, `comparisonCriteria`, especifica cómo se comparan los valores de la lista.

- `list`: la lista de valores.
- `countOrCondition`: si se especifica un número, se devuelve una lista de hasta `countOrCondition` elementos en orden ascendente. Si se especifica una condición, se devuelve una lista de elementos que cumplen inicialmente la condición. Si un elemento no cumple la condición, no se tienen en cuenta más elementos. Si este parámetro es NULL, se devuelve el valor más pequeño de la lista.
- `comparisonCriteria`: (*Opcional*) Se puede especificar un valor opcional `comparisonCriteria` para determinar cómo comparar los elementos de la lista. Si este parámetro es NULL, se usa el comparador predeterminado.

Ejemplo

Hallar los 5 valores menores de la lista {3, 4, 5, -1, 7, 8, 2}.

Uso

Power Query M

```
List.MinN({3, 4, 5, -1, 7, 8, 2}, 5)
```

Salida

`{-1, 2, 3, 4, 5}`

Contenido relacionado

[Criterios de comparación](#)

Last updated on 04/04/2026

List.Mode

16/09/2025

Sintaxis

```
List.Mode(list as list, optional equationCriteria as any) as any
```

Acerca de

Devuelve el elemento que aparece con más frecuencia en `list`. Si la lista está vacía, se produce una excepción. Si aparecen varios elementos con la misma frecuencia máxima, se elige el último. Se puede especificar un valor opcional de criterio de comparación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo 1

Hallar el elemento que aparece con más frecuencia en la lista `{"A", 1, 2, 3, 3, 4, 5}`.

Uso

Power Query M

```
List.Mode({"A", 1, 2, 3, 3, 4, 5})
```

Salida

3

Ejemplo 2

Hallar el elemento que aparece con más frecuencia en la lista `{"A", 1, 2, 3, 3, 4, 5, 5}`.

Uso

Power Query M

```
List.Mode({"A", 1, 2, 3, 3, 4, 5, 5})
```

Salida

5

Contenido relacionado

[Criterios de ecuación](#)

List.Modes

Sintaxis

```
List.Modes(list as list, optional equationCriteria as any) as list
```

Acerca de

Devuelve los elementos que aparecen con más frecuencia en `list`. Si la lista está vacía, se produce un error. Si aparecen varios elementos con la misma frecuencia máxima, se devuelven todos. Se puede especificar un valor opcional de criterio de comparación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo

Hallar los elementos que aparecen con más frecuencia en la lista `{"A", 1, 2, 3, 3, 4, 5, 5}`.

Uso

Power Query M

```
List.Modes({"A", 1, 2, 3, 3, 4, 5, 5})
```

Salida

```
{3, 5}
```

Contenido relacionado

[Criterios de ecuación](#)

Last updated on 25/03/2026

List.NonNullCount

16/09/2025

Sintaxis

```
List.NonNullCount(list as list) as number
```

Acerca de

Devuelve el número de elementos que no son NULL en la lista `list`.

List.Numbers

16/09/2025

Sintaxis

```
List.Numbers(  
    start as number,  
    count as number,  
    optional increment as nullable number  
) as list
```

Acerca de

Devuelve una lista de números dado un valor inicial, un recuento y un valor de incremento opcional. El valor de incremento predeterminado es 1.

- **start**: valor inicial de la lista.
- **count**: número de valores que se va a crear.
- **increment**: *[Opcional]* Valor por el que se va a incrementar. Si se omite, los valores se incrementan en 1.

Ejemplo 1

Generar una lista de 10 números consecutivos que empieza en 1.

Uso

Power Query M

```
List.Numbers(1, 10)
```

Salida

Power Query M

```
{  
    1,  
    2,  
    3,  
    4,  
    5,  
    6,
```

```
7,  
8,  
9,  
10  
}
```

Ejemplo 2

Generar una lista de 10 números que empieza en 1, con un incremento de 2 en cada número posterior.

Uso

Power Query M

```
List.Numbers(1, 10, 2)
```

Salida

Power Query M

```
{  
  1,  
  3,  
  5,  
  7,  
  9,  
  11,  
  13,  
  15,  
  17,  
  19  
}
```

List.Percentile

16/09/2025

Sintaxis

```
List.Percentile(  
    list as list,  
    percentiles as any,  
    optional options as nullable record  
) as any
```

Acerca de

Devuelve uno o más percentiles de ejemplo de la lista `list`. Si el valor `percentiles` es un número comprendido entre 0,0 y 1,0, se tratará como percentil y el resultado será un valor único correspondiente a esa probabilidad. Si el valor `percentiles` es una lista de números con valores comprendidos entre 0,0 y 1,0, el resultado será una lista de percentiles correspondientes a la probabilidad de entrada.

Los usuarios avanzados pueden usar la opción `PercentileMode` en `options` para elegir un método de interpolación más específico, pero no se recomienda para la mayoría de los usos. Los símbolos predefinidos `PercentileMode.ExcelInc` y `PercentileMode.ExcelExc` coinciden con los métodos de interpolación usados por las funciones `PERCENTILE.INC` de Excel y `PERCENTILE.EXC`. El comportamiento predeterminado coincide con `PercentileMode.ExcelInc`. Los símbolos `PercentileMode.SqlCont` y `PercentileMode.SqlDisc` coinciden con el comportamiento de SQL Server para `PERCENTILE_CONT` y `PERCENTILE_DISC`, respectivamente.

Ejemplo 1

Busque el primer cuartil de la lista `{5, 3, 1, 7, 9}`.

Uso

Power Query M

```
List.Percentile({5, 3, 1, 7, 9}, 0.25)
```

Salida

Ejemplo 2

Busque los cuartiles de la lista {5, 3, 1, 7, 9} mediante un método de interpolación que coincida con el de Excel `PERCENTILE.EXC`.

Uso

Power Query M

```
List.Percentile({5, 3, 1, 7, 9}, {0.25, 0.5, 0.75},  
[PercentileMode=PercentileMode.ExcelExc])
```

Salida

```
{2, 5, 8}
```

List.PositionOf

Sintaxis

```
List.PositionOf(  
    list as list,  
    value as any,  
    optional occurrence as nullable number,  
    optional equationCriteria as any  
) as any
```

Acerca de

Devuelve el desplazamiento en el que aparece el valor especificado en una lista. Devuelve -1 si el valor no aparece.

- `list`: La lista en la que buscar.
- `value`: valor que se va a buscar en la lista.
- `occurrence`: (Opcional) Repetición específica que se va a notificar. Este valor puede ser [Occurrence.First](#), [Occurrence.Last](#) o [Occurrence.All](#). Si no se especifica `occurrence`, se usa `Occurrence.First`.
- `equationCriteria`: (Opcional) Especifica cómo se determina la igualdad al comparar valores. Este parámetro puede ser una función selectora de claves, una función de comparador o una lista que contenga un selector de claves y un comparador.

Ejemplo 1

Hallar la posición en la lista {1, 2, 3} en la que aparece el valor 3.

Uso

Power Query M

```
List.PositionOf({1, 2, 3}, 3)
```

Salida

2

Ejemplo 2

Busque la posición en la lista de todas las instancias de fechas de 2022.

Uso

Power Query M

```
let
    Source = {
        #date(2021, 5, 10),
        #date(2022, 6, 28),
        #date(2023, 7, 15),
        #date(2022, 12, 31),
        #date(2022, 4, 8),
        #date(2024, 3, 20)
    },
    YearList = List.Transform(Source, each Date.Year(_)),
    TargetYear = 2022,
    FindPositions = List.PositionOf(YearList, TargetYear, Occurrence.All)
in
    FindPositions
```

Salida

```
{1, 3, 4}
```

Ejemplo 3

Busque la posición en la lista de la última aparición de la palabra perro, ignorando mayúsculas y minúsculas.

Uso

Power Query M

```
let
    Source = List.PositionOf(
        {"dog", "cat", "DOG", "pony", "bat", "rabbit", "dOG"},
        "dog",
        Occurrence.Last,
        Comparer.OrdinalIgnoreCase
    )
in
    Source
```

Salida

Ejemplo 4

Busque la posición en la lista que está dentro de dos unidades del número 28.

Uso

Power Query M

```
let
    Source = { 10, 15, 20, 25, 30 },
    Position = List.PositionOf(
        Source,
        28,
        Occurrence.First,
        (x, y) => Number.Abs(x - y) <= 2
    )
in
    Position
```

Salida

4

Contenido relacionado

[Criterios de ecuación](#)

Last updated on 22/11/2025

List.PositionOfAny

16/09/2025

Sintaxis

```
List.PositionOfAny(  
    list as list,  
    values as list,  
    optional occurrence as nullable number,  
    optional equationCriteria as any  
) as any
```

Acerca de

Devuelve el desplazamiento en el que aparece un elemento de la lista de valores especificada en una lista. Devuelve -1 si no se encuentra ninguna aparición.

- **list**: La lista para buscar.
- **values**: la lista de valores que se van a buscar en la lista original.
- **occurrence**: (Opcional) La ocurrencia específica para informar. Este valor puede ser [Occurrence.First](#), [Occurrence.Last](#) o [Occurrence.All](#). Si no se especifica **occurrence**, se usa [Occurrence.First](#).
- **equationCriteria**: (Opcional) Especifica cómo se determina la igualdad al comparar valores. Este parámetro puede ser una función selectora de claves, una función de comparador o una lista que contenga un selector de claves y un comparador.

Ejemplo 1

Hallar la primera posición en la lista {1, 2, 3} en la que el valor 2 o 3 aparece.

Uso

Power Query M

```
List.PositionOfAny({1, 2, 3}, {2, 3})
```

Salida

Ejemplo 2

Busque la posición en la lista de todas las fechas de 2022 o 2023.

Uso

Power Query M

```
let
    Source = {
        #date(2021, 5, 10),
        #date(2022, 6, 28),
        #date(2023, 7, 15),
        #date(2025, 12, 31),
        #date(2022, 4, 8),
        #date(2024, 3, 20)
    },
    YearList = List.Transform(Source, each Date.Year(_)),
    TargetYear = {2022, 2023},
    FindPositions = List.PositionOfAny(YearList, TargetYear, Occurrence.All)
in
    FindPositions
```

Salida

```
{1, 2, 4}
```

Ejemplo 3

Busque la posición en la lista de la última aparición de la palabra perro o gato, ignorando mayúsculas y minúsculas.

Uso

Power Query M

```
let
    Source = List.PositionOfAny(
        {"dog", "cat", "DOG", "pony", "bat", "rabbit", "dOG"},
        {"dog", "cat"},
        Occurrence.Last,
        Comparer.OrdinalIgnoreCase
    )
in
    Source
```

Salida

Ejemplo 4

Busque cualquier posición en la lista que esté dentro de dos unidades del número 17 o 28.

Uso

Power Query M

```
let
    Source = { 10, 15, 20, 25, 30 },
    Position = List.PositionOfAny(
        Source,
        {17, 28},
        Occurrence.All,
        (x, y) => Number.Abs(x - y) <= 2
    )
in
    Position
```

Salida

```
{1, 4}
```

Contenido relacionado

[Criterios de ecuación](#)

List.Positions

15/10/2025

Syntax

```
List.Positions(list as list) as list
```

Acerca de

Devuelve una lista de desplazamientos para la lista de entrada especificada.

- `list`: lista de entrada.

Cuando se usa [List.Transform](#) para cambiar una lista, se puede usar la lista de posiciones para dar acceso a la transformación a la posición.

Ejemplo 1

Busque los desplazamientos de valores de la lista {1, 2, 3, 4, null, 5}.

Uso

Power Query M

```
List.Positions({1, 2, 3, 4, null, 5})
```

Salida

```
{0, 1, 2, 3, 4, 5}
```

Ejemplo 2

Cree una tabla que asigne un identificador a cada cliente en función de la posición del cliente en la lista.

Uso

Power Query M

```

let
    customers = {"Alice", "Bob", "Charlie", "Diana"},
    resultTable =
        Table.FromRecords(
            List.Transform(
                List.Positions(customers),
                each [
                    IDNumber = _ + 1,    // Make it 1-based
                    CustomerName = customers[_]
                ]
            ),
            type table [IDNumber = Int64.Type, CustomerName = text]
        )
in
    resultTable

```

Salida

Power Query M

```

#table (type table[IDNumber = Int64.Type, CustomerName = text],
{
    {1, "Alice"},
    {2, "Bob"},
    {3, "Charlie"},
    {4, "Diana"}
})

```

List.Product

16/09/2025

Sintaxis

```
List.Product(numbersList as list, optional precision as nullable number) as nullable number
```

Acerca de

Devuelve el producto de los números que no son NULL de la lista, `numbersList`. Devuelve null si no hay valores no NULL en la lista.

Ejemplo 1

Calcula el producto de los números de la lista `{1, 2, 3, 3, 4, 5, 5}`.

Uso

Power Query M

```
List.Product({1, 2, 3, 3, 4, 5, 5})
```

Salida

1800

List.Random

Syntax

```
List.Random(count as number, optional seed as nullable number) as list
```

Acerca de

Devuelve una lista de números aleatorios entre 0 y 1, dado el número de valores que se van a generar y un valor de inicialización opcional.

- **count**: número de valores aleatorios que se van a generar.
- **seed**: *[Opcional]* Valor numérico usado para inicializar el generador de números aleatorios. Si se omite una lista única de números aleatorios se genera cada vez que se llama a la función. Si especifica el valor de inicialización con un número cada llamada a la función genera la misma lista de números aleatorios.

Ejemplo 1

Cree una lista de 3 números aleatorios.

Uso

Power Query M

```
List.Random(3)
```

Salida

```
{0.992332, 0.132334, 0.023592}
```

Ejemplo 2

Cree una lista de 3 números aleatorios, especificando el valor de inicialización.

Uso

Power Query M

```
List.Random(3, 2)
```


Salida

{0.883002, 0.245344, 0.723212}

Last updated on 22/11/2025

List.Range

16/09/2025

Sintaxis

```
List.Range(  
    list as list,  
    offset as number,  
    optional count as nullable number  
) as list
```

Acerca de

Devuelve un subconjunto de `list` que comienza en `offset`. Un parámetro opcional, `count`, establece el número máximo de elementos del subconjunto.

Ejemplo 1

Encontrar el subconjunto que empieza en el desplazamiento 6 de la lista de los números 1 a 10.

Uso

Power Query M

```
List.Range({1..10}, 6)
```

Salida

```
{7, 8, 9, 10}
```

Ejemplo 2

Encontrar el subconjunto de longitud 2 a partir del desplazamiento 6 de la lista de los números 1 a 10.

Uso

Power Query M

```
List.Range({1..10}, 6, 2)
```

Salida

```
{7, 8}
```

List.RemoveFirstN

16/09/2025

Sintaxis

```
List.RemoveFirstN(list as list, optional countOrCondition as any) as list
```

Acerca de

Devuelve una lista que quita el primer elemento de la lista `list`. Si `list` es una lista vacía, se devuelve una lista vacía. Esta función toma un parámetro opcional, `countOrCondition`, para admitir la eliminación de varios valores como se muestra a continuación.

- Si se especifica un número, se eliminan hasta esa cantidad de elementos.
- Si se especifica una condición, se quitan los elementos coincidentes consecutivos al principio de `list`.
- Si este parámetro es NULL, se observa el comportamiento predeterminado.

Ejemplo 1

Cree una lista a partir de {1, 2, 3, 4, 5} sin los primeros 3 números.

Uso

Power Query M

```
List.RemoveFirstN({1, 2, 3, 4, 5}, 3)
```

Salida

```
{4, 5}
```

Ejemplo 2

Cree una lista a partir de {5, 4, 2, 6, 1} que comience con un número menor que 3.

Uso

Power Query M

```
List.RemoveFirstN({5, 4, 2, 6, 1}, each _ > 3)
```

Salida

```
{2, 6, 1}
```

List.RemoveItems

Syntax

```
List.RemoveItems(list1 as list, list2 as list) as list
```

Acerca de

Quita todas las apariciones de los valores especificados de **list2** de **list1**. Si los valores de **list2** no existen en **list1**, se devuelve la lista original.

Ejemplo 1

Quite los elementos de la lista {2, 4, 6} de la lista {1, 2, 3, 4, 2, 5, 5}.

Uso

Power Query M

```
List.RemoveItems({1, 2, 3, 4, 2, 5, 5}, {2, 4, 6})
```

Salida

```
{1, 3, 5, 5}
```

Last updated on 22/11/2025

List.RemoveLastN

16/09/2025

Sintaxis

```
List.RemoveLastN(list as list, optional countOrCondition as any) as list
```

Acerca de

Devuelve una lista que quita los últimos elementos `countOrCondition` del final de la lista `list`. Si `list` tiene menos de `countOrCondition` elementos, se devuelve una lista vacía.

- Si se especifica un número, se pueden quitar hasta esa cantidad de elementos.
- Si se especifica una condición, se quitan los elementos coincidentes consecutivos al final de `list`.
- Si este parámetro es NULL, solo se quita un elemento.

Ejemplo 1

Cree una lista a partir de {1, 2, 3, 4, 5} sin los últimos 3 números.

Uso

Power Query M

```
List.RemoveLastN({1, 2, 3, 4, 5}, 3)
```

Salida

```
{1, 2}
```

Ejemplo 2

Cree una lista a partir de {5, 4, 2, 6, 4} que termine con un número menor que 3.

Uso

Power Query M

```
List.RemoveLastN({5, 4, 2, 6, 4}, each _ > 3)
```

Salida

```
{5, 4, 2}
```


List.RemoveMatchingItems

16/09/2025

Sintaxis

```
List.RemoveMatchingItems(  
    list1 as list,  
    list2 as list,  
    optional equationCriteria as any  
) as list
```

Acerca de

Quita de la lista `list2` todas las repeticiones de los valores especificados en `list1`. Si los valores de `list2` no existen en `list1`, se devuelve la lista original. Se puede especificar un valor opcional de criterio de ecuación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo 1

Crear una lista a partir de {1, 2, 3, 4, 5, 5} sin {1, 5}.

Uso

Power Query M

```
List.RemoveMatchingItems({1, 2, 3, 4, 5, 5}, {1, 5})
```

Salida

```
{2, 3, 4}
```

Contenido relacionado

[Criterios de ecuación](#)

List.RemoveNulls

16/09/2025

Sintaxis

```
List.RemoveNulls(list as list) as list
```

Acerca de

Elimina todas las ocurrencias de valores "null" en el `list`. Si no hay valores 'null' en la lista, se devuelve la lista original.

Ejemplo 1

Quite los valores "null" de la lista {1, 2, 3, null, 4, 5, null, 6}.

Uso

Power Query M

```
List.RemoveNulls({1, 2, 3, null, 4, 5, null, 6})
```

Salida

```
{1, 2, 3, 4, 5, 6}
```

List.RemoveRange

16/09/2025

Sintaxis

```
List.RemoveRange(  
    list as list,  
    index as number,  
    optional count as nullable number  
) as list
```

Acerca de

Quita los `count` valores en `list` a partir de la posición especificada, `index`.

Ejemplo 1

Quite 3 valores de la lista {1, 2, 3, 4, -6, -2, -1, 5} a partir del índice 4.

Uso

Power Query M

```
List.RemoveRange({1, 2, 3, 4, -6, -2, -1, 5}, 4, 3)
```

Salida

```
{1, 2, 3, 4, 5}
```

List.Repeat

Syntax

```
List.Repeat(list as list, count as number) as list
```

Acerca de

Devuelve una lista que es **count** repeticiones de la lista original, **list**.

Ejemplo 1

Cree una lista que tenga {1, 2} repetida 3 veces.

Uso

Power Query M

```
List.Repeat({1, 2}, 3)
```

Salida

```
{1, 2, 1, 2, 1, 2}
```

Last updated on 22/11/2025

List.ReplaceMatchingItems

16/09/2025

Sintaxis

```
List.ReplaceMatchingItems(  
    list as list,  
    replacements as list,  
    optional equationCriteria as any  
) as list
```

Acerca de

Realiza los reemplazos especificados en la lista `list`. Una operación de reemplazo `replacements` consta de una lista de dos valores (el valor anterior y el nuevo) proporcionados en una lista. Se puede especificar un valor opcional de criterio de ecuación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo 1

Crear una lista de {1, 2, 3, 4, 5} reemplazando el valor 5 por -5 y el valor 1 por -1.

Uso

Power Query M

```
List.ReplaceMatchingItems({1, 2, 3, 4, 5}, {{5, -5}, {1, -1}})
```

Salida

```
{-1, 2, 3, 4, -5}
```

Contenido relacionado

[Criterios de ecuación](#)

List.ReplaceRange

16/09/2025

Sintaxis

```
List.ReplaceRange(  
    list as list,  
    index as number,  
    count as number,  
    replaceWith as list  
) as list
```

Acerca de

Reemplaza los valores `count` en el `list` con la lista `replaceWith`, empezando en la posición especificada, que es `index`.

Ejemplo 1

Reemplace {7, 8, 9} en la lista {1, 2, 7, 8, 9, 5} por {3, 4}.

Uso

Power Query M

```
List.ReplaceRange({1, 2, 7, 8, 9, 5}, 2, 3, {3, 4})
```

Salida

```
{1, 2, 3, 4, 5}
```

List.ReplaceValue

Syntax

```
List.ReplaceValue(  
    list as list,  
    oldValue as any,  
    newValue as any,  
    replacer as function  
) as list
```

Acerca de

Busca en una lista de valores, `list`, para el valor `oldValue` y reemplaza cada aparición por el valor `newValue` de reemplazo .

Ejemplo 1

Reemplace todos los valores "a" de la lista {"a", "B", "a", "a"} por "A".

Uso

Power Query M

```
List.ReplaceValue({"a", "B", "a", "a"}, "a", "A", Replacer.ReplaceText)
```

Salida

```
{"A", "B", "A", "A"}
```

Last updated on 22/11/2025

List.Reverse

16/09/2025

Sintaxis

```
List.Reverse(list as list) as list
```

Acerca de

Devuelve una lista con los valores de la lista `list` en orden invertido.

Ejemplo 1

Cree una lista a partir de {1..10} en orden inverso.

Uso

Power Query M

```
List.Reverse({1..10})
```

Salida

```
{10, 9, 8, 7, 6, 5, 4, 3, 2, 1}
```


List.Select

Sintaxis

```
List.Select(list as list, selection as function) as list
```

Acerca de

Devuelve los valores de la lista especificada que coinciden con la condición de selección.

- `list`: La lista que debe examinarse.
- `selection`: función que determina los valores que se van a seleccionar.

Ejemplo 1

Busque los valores de la lista {1, -3, 4, 9, -2} mayores que 0.

Uso

Power Query M

```
List.Select({1, -3, 4, 9, -2}, each _ > 0)
```

Salida

```
{1, 4, 9}
```

Ejemplo 2

Selecione las fechas de la lista que caigan en sábado o domingo.

Uso

Power Query M

```
let
    dates = {
        #date(2025, 10, 20), // Monday
        #date(2025, 10, 21), // Tuesday
        #date(2025, 10, 25), // Saturday
        #date(2025, 10, 26), // Sunday
        #date(2025, 10, 27)  // Monday
    }
```

```

    },
    weekendDates = List.Select(
        dates,
        each Date.DayOfWeek(_, Day.Monday) >= 5
    )
in
    weekendDates

```

Salida

Power Query M

```

{
    #date(2025, 10, 25),
    #date(2025, 10, 26)
}

```

Ejemplo 3

Muestra una tabla de clientes activos con totales de compra superiores a 100 USD.

Uso

Power Query M

```

let
    customers = {
        [Name = "Alice", Status = "Active", Purchases = 150],
        [Name = "Bob", Status = "Inactive", Purchases = 200],
        [Name = "Carol", Status = "Active", Purchases = 90],
        [Name = "Dave", Status = "Active", Purchases = 120]
    },
    highValueActiveCustomers = List.Select(
        customers,
        each [Status] = "Active" and [Purchases] > 100
    ),
    resultTable = Table.FromRecords(
        highValueActiveCustomers,
        type table [Name = text, Status = text, Purchases = number]
    )
in
    resultTable

```

Salida

Power Query M

```

#table(type table[Name = text, Status = text, Purchases = number],
{

```

```
    {"Alice", "Active", 150},  
    {"Dave", "Active", 120}  
  })
```

Last updated on 31/10/2025

List.Single

Sintaxis

```
List.Single(list as list) as any
```

Acerca de

Si solo hay un elemento en la lista `list`, devuelve ese elemento. Si hay más de un elemento o la lista está vacío, la función genera un error.

Ejemplo 1

Busque el valor único en la lista {1}.

Uso

Power Query M

```
List.Single({1})
```

Salida

1

Ejemplo 2

Busque el valor único en la lista {1, 2, 3}.

Uso

Power Query M

```
List.Single({1, 2, 3})
```

Salida

[Expression.Error] There were too many elements in the enumeration to complete the operation.

List.SingleOrDefault

16/09/2025

Sintaxis

```
List.SingleOrDefault(list as list, optional default as any) as any
```

Acerca de

Si solo hay un elemento en la lista `list`, devuelve ese elemento. Si la lista está vacía, la función devuelve null a menos que se especifique un `default` opcional. Si hay más de un elemento en la lista, la función devuelve un error.

Ejemplo 1

Busque el valor único en la lista {1}.

Uso

Power Query M

```
List.SingleOrDefault({1})
```

Salida

1

Ejemplo 2

Busque el valor único en la lista {}.

Uso

Power Query M

```
List.SingleOrDefault({})
```

Salida

null

Ejemplo 3

Busque el valor único en la lista {}. Si está vacío, devuelva -1.

Uso

Power Query M

```
List.SingleOrDefault({}, -1)
```

Salida

-1

List.Skip

16/09/2025

Sintaxis

```
List.Skip(list as list, optional countOrCondition as any) as list
```

Acerca de

Devuelve una lista que omita el primer elemento de la lista `list`. Si `list` es una lista vacía, se devuelve una lista vacía. Esta función toma un parámetro opcional, `countOrCondition`, para admitir la omisión de varios valores como se muestra a continuación.

- Si se especifica un número, se omiten hasta esa cantidad de elementos.
- Si se especifica una condición, se omiten los elementos coincidentes consecutivos al principio de `list`.
- Si este parámetro es NULL, se observa el comportamiento predeterminado.

Ejemplo 1

Cree una lista a partir de {1, 2, 3, 4, 5} sin los primeros 3 números.

Uso

Power Query M

```
List.Skip({1, 2, 3, 4, 5}, 3)
```

Salida

```
{4, 5}
```

Ejemplo 2

Cree una lista a partir de {5, 4, 2, 6, 1} que comience con un número menor que 3.

Uso

Power Query M


```
List.Skip({5, 4, 2, 6, 1}, each _ > 3)
```

Salida

```
{2, 6, 1}
```

List.Sort

Syntax

```
List.Sort(list as list, optional comparisonCriteria as any) as list
```

Acerca de

Ordena una lista de datos, `list` según los criterios opcionales especificados. Un parámetro opcional, `comparisonCriteria`, se puede especificar como criterio de comparación. Esto puede tomar los siguientes valores:

- Para controlar el orden, el criterio de comparación puede ser un valor de enumeración `Order`. ([Order.Descending](#), [Order.Ascending](#)).
- Para calcular una clave que se va a usar para ordenar, se puede usar una función de 1 argumento.
- Para seleccionar una clave y un orden de control, el criterio de comparación puede ser una lista que contenga la clave y el orden (`{each 1 / _, Order.Descending}`).
- Para controlar completamente la comparación, se puede usar una función de 2 argumentos. Esta función se pasará a dos elementos de la lista (cualquier dos elementos, en cualquier orden). La función debe devolver uno de los siguientes valores:
 - `-1`: el primer elemento es menor que el segundo elemento.
 - `0`: los elementos son iguales.
 - `1`: el primer elemento es mayor que el segundo elemento.

[Value.Compare](#) es un método que se puede usar para delegar esta lógica.

Ejemplo 1

Ordene la lista {2, 3, 1}.

Uso

Power Query M

```
List.Sort({2, 3, 1})
```

Salida

{1, 2, 3}

Ejemplo 2

Ordene la lista {2, 3, 1} en orden descendente.

Uso

Power Query M

```
List.Sort({2, 3, 1}, Order.Descending)
```

Salida

{3, 2, 1}

Ejemplo 3

Ordene la lista {2, 3, 1} en orden descendente mediante el **Value.Compare** método .

Uso

Power Query M

```
List.Sort({2, 3, 1}, (x, y) => Value.Compare(1/x, 1/y))
```

Salida

{3, 2, 1}

Last updated on 22/11/2025

List.Split

16/09/2025

Sintaxis

```
List.Split(list as list, pageSize as number) as list
```

Acerca de

Divide `list` en una lista de listas donde el primer elemento de la lista de salida es una lista que contiene los primeros elementos `pageSize` de la lista de origen, el siguiente elemento de la lista de salida es una lista que contiene los siguientes elementos `pageSize` de la lista de origen, etc.

List.StandardDeviation

Sintaxis

```
List.StandardDeviation(numbersList as list) as nullable number
```

Acerca de

Devuelve una estimación basada en muestras de la desviación estándar de los valores de la lista, `numbersList`. Si `numbersList` es una lista de números, se devuelve un número. Se genera un error en una lista vacía o una lista de elementos que no son de tipo `number`.

Ejemplo

Busque la desviación estándar de los números de 1 a 5.

Uso

Power Query M

```
List.StandardDeviation({1..5})
```

Salida

```
1.5811388300841898
```

Last updated on 26/03/2026

List.Sum

Sintaxis

```
List.Sum(list as list, optional precision as nullable number) as any
```

Acerca de

Devuelve la suma de los valores no NULL de la lista, `list`. Devuelve null si no hay valores no NULL en la lista.

Ejemplo

Busque la suma de los números de la lista `{1, 2, 3}`.

Uso

Power Query M

```
List.Sum({1, 2, 3})
```

Salida

6

Last updated on 25/03/2026

List.Times

Syntax

```
List.Times(  
    start as time,  
    count as number,  
    step as duration  
) as list
```

Acerca de

Devuelve una lista de `time` valores de tamaño `count`, empezando por `start`. El incremento especificado, `step`, es un `duration` valor que se agrega a cada valor.

Ejemplo 1

Cree una lista de 4 valores a partir del mediodía (`#time(12, 0, 0)`) incrementando en una hora (`#duration(0, 1, 0, 0, 0)`).

Uso

Power Query M

```
List.Times(#time(12, 0, 0), 4, #duration(0, 1, 0, 0))
```

Salida

Power Query M

```
{  
    #time(12, 0, 0),  
    #time(13, 0, 0),  
    #time(14, 0, 0),  
    #time(15, 0, 0)  
}
```

List.Transform

Sintaxis

```
List.Transform(list as list, transform as function) as list
```

Acerca de

Devuelve una nueva lista de valores aplicando la función `transform` de transformación a la lista, `list`.

Ejemplo

Agregue 1 a cada valor de la lista {1, 2}.

Uso

Power Query M

```
List.Transform({1, 2}, each _ + 1)
```

Salida

```
{2, 3}
```

Last updated on 25/03/2026

List.TransformMany

Syntax

```
List.TransformMany(  
    list as list,  
    collectionTransform as function,  
    resultTransform as function  
) as list
```

Acerca de

Devuelve una lista cuyos elementos se proyectan desde la lista de entrada.

La `collectionTransform` función transforma cada elemento en una lista intermedia y la `resultTransform` función recibe el elemento original, así como un elemento de la lista intermedia para construir el resultado final.

La `collectionTransform` función tiene la firma `(x as any) as list => ...`, donde `x` es un elemento de `list`. La `resultTransform` función proyecta la forma del resultado y tiene la firma `(x as any, y as any) as any => ...`, donde `x` es un elemento de `list` y `y` es un elemento de la lista generada pasando `x` a `collectionTransform`.

Ejemplo 1

Aplane una lista de personas y sus mascotas.

Uso

Power Query M

```
List.TransformMany(  
    {  
        [Name = "Alice", Pets = {"Scruffy", "Sam"}],  
        [Name = "Bob", Pets = {"Walker"}]  
    },  
    each [Pets],  
    (person, pet) => [Name = person[Name], Pet = pet]  
)
```

Salida

Power Query M

```
{  
    [Name = "Alice", Pet = "Scruffy"],  
    [Name = "Alice", Pet = "Sam"],  
    [Name = "Bob", Pet = "Walker"]  
}
```

Last updated on 22/11/2025

List.Union

16/09/2025

Sintaxis

```
List.Union(lists as list, optional equationCriteria as any) as list
```

Acerca de

Toma una lista de listas `lists`, combina los elementos de las listas individuales y los devuelve en la lista de salida. Como resultado, la lista devuelta contiene todos los elementos de cualquiera de las listas de entrada. Esta operación mantiene la semántica multiconjunto tradicional, por lo que los valores duplicados se buscan como parte de la unión. Se puede especificar un valor opcional de criterio de ecuación (`equationCriteria`) para controlar la igualdad de las pruebas.

Ejemplo 1

Crear una combinación de la lista {1..5}, {2..6}, {3..7}.

Uso

Power Query M

```
List.Union({{1..5}, {2..6}, {3..7}})
```

Salida

```
{1, 2, 3, 4, 5, 6, 7}
```

Contenido relacionado

[Criterios de ecuación](#)

List.Zip

Syntax

```
List.Zip(lists as list) as list
```

Acerca de

Toma una lista de listas, `lists` y devuelve una lista de listas que combinan elementos en la misma posición.

Ejemplo 1

Comprime las dos listas simples {1, 2} y {3, 4}.

Uso

Power Query M

```
List.Zip({{1, 2}, {3, 4}})
```

Salida

Power Query M

```
{  
    {1, 3},  
    {2, 4}  
}
```

Ejemplo 2

Comprime las dos listas simples de diferentes longitudes {1, 2} y {3}.

Uso

Power Query M

```
List.Zip({{1, 2}, {3}})
```

Salida

Power Query M

```
{  
    {1, 3},  
    {2, null}  
}
```

Last updated on 22/11/2025

Funciones lógicas

Artículo • 23/11/2024

Estas funciones crean y manipulan valores lógicos (es decir, `true` o `false`).

 Expandir tabla

Nombre	Descripción
Logical.From	Crea un valor lógico a partir del valor especificado.
Logical.FromText	Crea un valor lógico a partir de los valores de texto "true" y "false".
Logical.ToText	Devuelve el texto "true" o "false" dado un valor lógico.

Comentarios

¿Le ha resultado útil esta página?

 **Sí**

 **No**

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Logical.From

Sintaxis

```
Logical.From(value as any) as nullable logical
```

Acerca de

Devuelve un valor de `logical` del `value` especificado. Si el `value` especificado es `null`, `Logical.From` devuelve `null`. Si el `value` especificado es `logical`, se devuelve `value`. Los valores de los siguientes tipos se pueden convertir en un valor de `logical`:

- `text`: un valor `logical` del valor de texto, ya sea `"true"` o `"false"`. Consulte [Logical.FromText](#) para obtener más información.
- `number`: `false` si `value` es igual a `0`, `true` de lo contrario.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo

Convierta `2` en un valor de `logical`.

Uso

Power Query M

```
Logical.From(2)
```

Salida

```
true
```

Last updated on 03/04/2026

Logical.FromText

Syntax

```
Logical.FromText(text as nullable text) as nullable logical
```

Acerca de

Crea un valor lógico a partir del valor `text` de texto , "true" o "false". Si `text` contiene una cadena diferente, se genera un error. El valor `text` de texto no distingue mayúsculas de minúsculas.

Ejemplo 1

Cree un valor lógico a partir de la cadena de texto "true".

Uso

Power Query M

```
Logical.FromText("true")
```

Salida

```
true
```

Ejemplo 2

Cree un valor lógico a partir de la cadena de texto "a".

Uso

Power Query M

```
Logical.FromText("a")
```

Salida

```
[Expression.Error] Could not convert to a logical.
```

Logical.ToText

Sintaxis

```
Logical.ToText(logicalValue as nullable logical) as nullable text
```

Acerca de

Crea un valor de texto a partir del valor `logicalValue` lógico , o `true` `false`. Si `logicalValue` no es un valor lógico, se genera un error.

Ejemplo

Cree un valor de texto a partir del elemento lógico `true`.

Uso

Power Query M

```
Logical.ToText(true)
```

Salida

```
"true"
```

Last updated on 25/03/2026

Funciones Number

15/08/2025

Estas funciones crean y manipulan valores numéricos.

Información

 Expandir tabla

Nombre	Descripción
Number.IsEven	Devuelve <code>true</code> si un valor es un número par.
Number.IsNaN	Devuelve <code>true</code> si un valor es Number.NaN .
Number.IsOdd	Devuelve <code>true</code> si un valor es un número impar.

Conversión y formato

 Expandir tabla

Nombre	Descripción
Byte.From	Devuelve un valor numérico entero de 8 bits del valor especificado.
Currency.From	Devuelve un valor de moneda a partir del valor dado.
Decimal.From	Devuelve un valor de número decimal del valor especificado.
Double.From	Devuelve un valor de número doble del valor especificado.
Int8.From	Devuelve un número entero de 8 bits con signo del valor especificado.
Int16.From	Devuelve un valor numérico entero de 16 bits del valor especificado.
Int32.From	Devuelve un valor numérico entero de 32 bits del valor especificado.
Int64.From	Devuelve un valor numérico entero de 64 bits del valor especificado.
Number.From	Devuelve un valor numérico a partir de un dato.
Number.FromText	Devuelve un valor numérico de un valor de texto.
Number.ToText	Convierte el número especificado en texto.
Percentage.From	Devuelve un valor de porcentaje del valor especificado.

Nombre	Descripción
Single.From	Devuelve un valor de número único del valor especificado.

Redondeo

 Expandir tabla

Nombre	Descripción
Number.Round	Si el valor es un entero, devuelve un número anulable.
Number.RoundAwayFromZero	Devuelve Number.RoundUp(valor) cuando value ≥ 0 y Number.RoundDown(valor) cuando el valor < 0 .
Number.RoundDown	Devuelve el entero más grande menor o igual que un valor numérico.
Number.RoundTowardZero	Devuelve Number.RoundDown(x) cuando $x \geq 0$ y Number.RoundUp(x) cuando $x < 0$.
Number.RoundUp	Devuelve el entero mayor o igual que un valor numérico.

Operaciones

 Expandir tabla

Nombre	Descripción
Number.Abs	Devuelve el valor absoluto de un número.
Number.Combinations	Devuelve el número de combinaciones de un número determinado de elementos para el tamaño de combinación opcional.
Number.Exp	Devuelve un número que representa e elevado a una potencia.
Number.Factorial	Devuelve el factorial de un número.
Number.IntegerDivide	Divide dos números y devuelve la parte completa del número resultante.
Number.Ln	Devuelve el logaritmo natural (neperiano) de un número.
Number.Log	Devuelve el logaritmo de un número a la base.
Number.Log10	Devuelve el logaritmo en base 10 de un número.
Number.Mod	Divide dos números y devuelve el resto del número resultante.

Nombre	Descripción
Number.Permutations	Devuelve el número de permutaciones totales de un número determinado de elementos para el tamaño de permutación opcional.
Number.Power	Devuelve un número elevado por una potencia.
Number.Sign	Devuelve 1 para números positivos, -1 para números negativos o 0 para cero.
Number.Sqrt	Devuelve la raíz cuadrada de un número.

Aleatorio

 Expandir tabla

Nombre	Descripción
Number.Random	Devuelve un número fraccionario aleatorio entre 0 y 1.
Number.RandomBetween	Devuelve un número aleatorio entre los dos valores numéricos especificados.

Trigonometría

 Expandir tabla

Nombre	Descripción
Number.Acos	Devuelve el arcocoseno de un número.
Number.Asin	Devuelve el arcoseno de un número.
Number.Atan	Devuelve la arcotangente de un número.
Number.Atan2	Devuelve la arcotangente de la división de dos números.
Number.Cos	Devuelve el coseno de un número.
Number.Cosh	Devuelve el coseno hiperbólico de un número.
Number.Sin	Devuelve el seno de un número.
Number.Sinh	Devuelve el seno hiperbólico de un número.
Number.Tan	Devuelve la tangente de un número.
Number.Tanh	Devuelve la tangente hiperbólica de un número.

Bytes (unidades de información digital)

 Expandir tabla

Nombre	Descripción
Number.BitwiseAnd	Devuelve el resultado de una operación AND bit a bit en los operandos proporcionados.
Number.BitwiseNot	Devuelve el resultado de una operación NOT a nivel de bits en los operandos proporcionados.
Number.BitwiseOr	Devuelve el resultado de una operación OR bit a bit en los operandos proporcionados.
Number.BitwiseShiftLeft	Devuelve el resultado de una operación de desplazamiento a la izquierda bit a bit en los operandos.
Number.BitwiseShiftRight	Devuelve el resultado de una operación de desplazamiento bit a bit a la derecha en los operandos.
Number.BitwiseXor	Devuelve el resultado de una operación XOR bit a bit en los operandos proporcionados.

Byte.From

Syntax

```
Byte.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor entero `number` de 8 bits del especificado `value`. Si el `value` especificado es `null`, `Byte.From` devuelve `null`. Si el elemento especificado `value` está `number` dentro del intervalo de un entero de 8 bits sin una parte fraccionaria, `value` se devuelve. Si tiene parte fraccionaria, el número se redondea con el modo de redondeo especificado. El modo de redondeo predeterminado es [RoundingMode.ToEven](#). Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). Consulte para [Number.Round](#) conocer los modos de redondeo disponibles. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor entero `number` de 8 bits de `"4"`.

Uso

Power Query M

```
Byte.From("4")
```

Salida

4

Ejemplo 2

Obtenga el valor entero `number` de 8 bits de `"4.5"` mediante [RoundingMode.AwayFromZero](#).

Uso

Power Query M

```
Byte.From("4.5", null, RoundingMode.AwayFromZero)
```

Salida

5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 02/04/2026

Currency.From

16/09/2025

Sintaxis

```
Currency.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor de `currency` del `value` especificado. Si el `value` especificado es `null`, **Currency.From** devuelve `null`. Si el `value` especificado se encuentra `number` dentro del rango monetario, la parte fraccionaria del `value` se redondea a 4 dígitos decimales y se devuelve. Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). El rango válido para la moneda es de `-922,337,203,685,477.5808` a `922,337,203,685,477.5807`. Consulte [Number.Round](#) para conocer los modos de redondeo disponibles. El valor predeterminado es [RoundingMode.ToEven](#). También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor `currency` de `"1.23455"`.

Uso

Power Query M

```
Currency.From("1.23455")
```

Salida

1.2346

Ejemplo 2

Obtenga el valor `currency` de `"1.23455"` mediante `RoundingMode.Down`.

Uso

Power Query M

```
Currency.From("1.23455", "en-US", RoundingMode.Down)
```

Salida

1.2345

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Decimal.From

16/09/2025

Sintaxis

```
Decimal.From(value as any, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un valor de `number` decimal del `value` especificado. Si el `value` especificado es `null`, **Decimal.From** devuelve `null`. Si el `value` dado es `number` dentro del rango de Decimal, se devuelve `value`; de lo contrario, se devuelve un error. Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor decimal `number` de "4.5".

Uso

Power Query M

```
Decimal.From("4.5")
```

Salida

4.5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Double.From

Syntax

```
Double.From(value as any, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un valor Double `number` del especificado `value`. Si el objeto especificado `value` es `null`, **Double.From** devuelve `null`. Si el valor especificado `value` está `number` dentro del intervalo de Double, `value` se devuelve; de lo contrario, se devuelve un error. Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). También se puede proporcionar una opción `culture` (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor Double `number` de "4".

Uso

Power Query M

```
Double.From("4.5")
```

Salida

4.5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Int8.From

Syntax

```
Int8.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor entero `number` de 8 bits con signo del especificado `value`. Si el `value` especificado es `null`, **Int8.From** devuelve `null`. Si el elemento especificado `value` está `number` dentro del intervalo de enteros de 8 bits con signo sin una parte fraccionaria, `value` se devuelve. Si tiene parte fraccionaria, el número se redondea con el modo de redondeo especificado. El modo de redondeo predeterminado es [RoundingMode.ToEven](#). Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). Consulte para [Number.Round](#) conocer los modos de redondeo disponibles. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor entero `number` de 8 bits con signo de `"4"`.

Uso

Power Query M

```
Int8.From("4")
```

Salida

4

Ejemplo 2

Obtenga el valor entero `number` de 8 bits con signo de uso de `"4.5"` [RoundingMode.AwayFromZero](#).

Uso

Power Query M

```
Int8.From("4.5", null, RoundingMode.AwayFromZero)
```

Salida

5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Int16.From

Syntax

```
Int16.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor entero `number` de 16 bits del especificado `value`. Si el objeto especificado `value` es `null`, **Int16.From** devuelve `null`. Si el valor especificado `value` está `number` dentro del intervalo de enteros de 16 bits sin una parte fraccionaria, `value` se devuelve. Si tiene parte fraccionaria, el número se redondea con el modo de redondeo especificado. El modo de redondeo predeterminado es [RoundingMode.ToEven](#). Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). Consulte para [Number.Round](#) conocer los modos de redondeo disponibles. También se puede proporcionar una opción `culture` (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor entero `number` de 16 bits de `"4"`.

Uso

Power Query M

```
Int64.From("4")
```

Salida

4

Ejemplo 2

Obtenga el valor entero `number` de 16 bits de `"4.5"` mediante `RoundingMode.AwayFromZero`.

Uso

Power Query M

```
Int16.From("4.5", null, RoundingMode.AwayFromZero)
```

Salida

5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Int32.From

Syntax

```
Int32.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor entero `number` de 32 bits del especificado `value`. Si el objeto especificado `value` es `null`, **Int32.From** devuelve `null`. Si el elemento especificado `value` está `number` dentro del intervalo de enteros de 32 bits sin una parte fraccionaria, `value` se devuelve. Si tiene parte fraccionaria, el número se redondea con el modo de redondeo especificado. El modo de redondeo predeterminado es [RoundingMode.ToEven](#). Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). Consulte para [Number.Round](#) conocer los modos de redondeo disponibles. También se puede proporcionar una opción `culture` (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor entero `number` de 32 bits de `"4"`.

Uso

Power Query M

```
Int32.From("4")
```

Salida

4

Ejemplo 2

Obtenga el valor entero `number` de 32 bits de `"4.5"` mediante `RoundingMode.AwayFromZero`.

Uso

Power Query M

```
Int32.From("4.5", null, RoundingMode.AwayFromZero)
```

Salida

5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Int64.From

16/09/2025

Sintaxis

```
Int64.From(  
    value as any,  
    optional culture as nullable text,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve un valor entero `number` de 64 bits del especificado `value`. Si el `value` especificado es `null`, **Int64.From** devuelve `null`. Si el valor especificado `value` está `number` dentro del intervalo de enteros de 64 bits sin una parte fraccionaria, `value` se devuelve. Si tiene parte fraccionaria, el número se redondea con el modo de redondeo especificado. El modo de redondeo predeterminado es [RoundingMode.ToEven](#). Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). Consulte [Number.Round](#) para conocer los modos de redondeo disponibles. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor entero `number` de 64 bits de `"4"`.

Uso

Power Query M

```
Int64.From("4")
```

Salida

4

Ejemplo 2

Obtenga el valor entero `number` de 64 bits de `"4.5"` mediante `RoundingMode.AwayFromZero`.

Uso

Power Query M

```
Int64.From("4.5", null, RoundingMode.AwayFromZero)
```

Salida

5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Number.Abs

Sintaxis

```
Number.Abs(number as nullable number) as nullable number
```

Acerca de

Devuelve el valor absoluto de `number`. Si `number` es null, **Number.Abs** devuelve null.

- `number`: un `number` para el que se va a calcular el valor absoluto.

Ejemplo

Valor absoluto de -3.

Uso

Power Query M

```
Number.Abs(-3)
```

Salida

3

Last updated on 25/03/2026

Number.Acos

Syntax

```
Number.Acos(number as nullable number) as nullable number
```

Acerca de

Devuelve el arcocoseno de `number`.

Last updated on 22/11/2025

Number.Asin

16/09/2025

Sintaxis

```
Number.Asin(number as nullable number) as nullable number
```

Acerca de

Devuelve el arco seno de `number`.

Number.Atan

16/09/2025

Sintaxis

```
Number.Atan(number as nullable number) as nullable number
```

Acerca de

Devuelve la arcotangente de **number**.

Number.Atan2

16/09/2025

Sintaxis

```
Number.Atan2(y as nullable number, x as nullable number) as nullable number
```

Acerca de

Devuelve el ángulo, en radianes, cuya tangente es el cociente y/x de los dos números y y x .

Number.BitwiseAnd

Syntax

```
Number.BitwiseAnd(number1 as nullable number, number2 as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de realizar una operación "And" bit a bit entre `number1` y `number2`.

Last updated on 22/11/2025

Number.BitwiseNot

Syntax

```
Number.BitwiseNot(number as any) as any
```

Acerca de

Devuelve el resultado de realizar una operación "Not" bit a bit en `number`.

Last updated on 22/11/2025

Number.BitwiseOr

Syntax

```
Number.BitwiseOr(number1 as nullable number, number2 as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de realizar un "Or" bit a bit entre `number1` y `number2`.

Last updated on 22/11/2025

Number.BitwiseShiftLeft

16/09/2025

Sintaxis

```
Number.BitwiseShiftLeft(number1 as nullable number, number2 as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de realizar un desplazamiento bit a bit a la izquierda en `number1`, mediante el número especificado de bits `number2`.

Number.BitwiseShiftRight

16/09/2025

Sintaxis

```
Number.BitwiseShiftRight(number1 as nullable number, number2 as nullable number)  
as nullable number
```

Acerca de

Devuelve el resultado de realizar un desplazamiento bit a bit a la derecha en `number1`, mediante el número especificado de bits `number2`.

Number.BitwiseXor

16/09/2025

Sintaxis

```
Number.BitwiseXor(number1 as nullable number, number2 as nullable number) as  
nullable number
```

Acerca de

Devuelve el resultado de la operación "XOR" a nivel de bits (Exclusive-OR) entre **number1** y **number2**.

Number.Combinations

16/09/2025

Sintaxis

```
Number.Combinations(setSize as nullable number, combinationSize as nullable number) as nullable number
```

Acerca de

Devuelve el número de combinaciones únicas de una lista de elementos, **setSize** con el tamaño de combinación especificado, **combinationSize**.

- **setSize**: número de elementos de la lista.
- **combinationSize**: número de elementos de cada combinación.

Ejemplo 1

Busque el número de combinaciones de un total de 5 elementos cuando cada combinación sea un grupo de 3.

Uso

Power Query M

```
Number.Combinations(5, 3)
```

Salida

10

Number.Cos

24/10/2025

Sintaxis

```
Number.Cos(number as nullable number) as nullable number
```

Acerca de

Devuelve el coseno del ángulo especificado.

- **number**: ángulo medido en radianes.

Ejemplo 1

Busque el coseno del ángulo 0.

Uso

```
Power Query M
```

```
Number.Cos(0)
```

Salida

1

Ejemplo 2

Encuentra el coseno de π radianes.

Uso

```
Power Query M
```

```
Number.Cos(Number.PI)
```

Salida

Number.Cosh

16/09/2025

Sintaxis

```
Number.Cosh(number as nullable number) as nullable number
```

Acerca de

Devuelve el coseno hiperbólico de **number**.

Number.Exp

16/09/2025

Sintaxis

```
Number.Exp(number as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de elevar e a la potencia de **number** (función exponencial).

- **number**: Un **number** para el cual se debe calcular la función exponencial. Si **number** es null, **Number.Exp** devuelve null.

Ejemplo 1

Eleva e a la potencia de 3.

Uso

```
Power Query M
```

```
Number.Exp(3)
```

Salida

```
20.085536923187668
```

Number.Factorial

Sintaxis

```
Number.Factorial(number as nullable number) as nullable number
```

Acerca de

Devuelve el factorial del número `number`.

Ejemplo

Busque el factorial de 10.

Uso

Power Query M

```
Number.Factorial(10)
```

Salida

```
3628800
```

Last updated on 25/03/2026

Number.From

Syntax

```
Number.From(value as any, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un valor de `number` del `value` especificado. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US"). Si el `value` especificado es `null`, **Number.From** devuelve `null`. Si el `value` especificado es `number`, se devuelve `value`. Los valores de los siguientes tipos se pueden convertir en un `number` valor:

- `text`: un `number` valor de representación textual. Los formatos de texto comunes se controlan ("15", "3,423.10", "5.0E-10"). Consulte [Number.FromText](#) para obtener más información.
- `logical`: 1 para `true`, 0 para `false`.
- `datetime`: número de punto flotante de precisión doble que contiene un equivalente de fecha de automatización OLE.
- `datetimezone`: número de punto flotante de precisión doble que contiene un equivalente de fecha de automatización OLE de la fecha y hora locales de `value`.
- `date`: número de punto flotante de precisión doble que contiene un equivalente de fecha de automatización OLE.
- `time`: expresado en días fraccionarios.
- `duration`: expresado en días enteros y fraccionarios.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo 1

Obtenga el `number` valor de `"4"`.

Uso

```
Power Query M
```

```
Number.From("4")
```

Salida

Ejemplo 2

Obtenga el `number` valor de `#datetime(2020, 3, 20, 6, 0, 0)`.

Uso

Power Query M

```
Number.From(#datetime(2020, 3, 20, 6, 0, 0))
```

Salida

43910.25

Ejemplo 3

Obtenga el `number` valor de `"12.3%"`.

Uso

Power Query M

```
Number.From("12.3%")
```

Salida

0.123

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas con formato numérico estándar](#)
- [cadenas de formato numérico personalizado](#)

Number.FromText

Syntax

```
Number.FromText(text as nullable text, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un `number` valor del valor de texto especificado, `text`.

- `text`: representación textual de un valor numérico. La representación debe tener un formato de número común, como "15", "3,423.10" o "5.0E-10".
- `culture`: una referencia cultural opcional que controla cómo `text` se interpreta (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor numérico de `"4"`.

Uso

Power Query M

```
Number.FromText("4")
```

Salida

4

Ejemplo 2

Obtenga el valor numérico de `"5.0e-10"`.

Uso

Power Query M

```
Number.FromText("5.0e-10")
```

Salida

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas con formato numérico estándar](#)
- [cadenas de formato numérico personalizado](#)

Last updated on 22/11/2025

Number.IntegerDivide

Syntax

```
Number.IntegerDivide(  
    number1 as nullable number,  
    number2 as nullable number,  
    optional precision as nullable number  
) as nullable number
```

Acerca de

Devuelve la parte entera del resultado de dividir un número, `number1`, por otro número, `number2`. Si `number1` o `number2` son NULL, **Number.IntegerDivide** devuelve null.

- `number1`: dividendo.
- `number2`: El divisor.

Ejemplo 1

Divida entre 6 y 4.

Uso

Power Query M

```
Number.IntegerDivide(6, 4)
```

Salida

1

Ejemplo 2

Divida 8.3 por 3.

Uso

Power Query M

```
Number.IntegerDivide(8.3, 3)
```


Number.IsEven

16/09/2025

Sintaxis

```
Number.IsEven(number as number) as logical
```

Acerca de

Indica si el valor, `number`, es incluso devolviendo `true` si es par, `false` de lo contrario.

Ejemplo 1

Compruebe si 625 es un número par.

Uso

```
Power Query M
```

```
Number.IsEven(625)
```

Salida

```
false
```

Ejemplo 2

Compruebe si 82 es un número par.

Uso

```
Power Query M
```

```
Number.IsEven(82)
```

Salida

```
true
```

Number.IsNaN

16/09/2025

Sintaxis

```
Number.IsNaN(number as number) as logical
```

Acerca de

Indica si el valor es NaN (no un número). Devuelve `true` si `number` es equivalente a `Number.IsNaN`, `false` de lo contrario.

Ejemplo 1

Compruebe si 0 dividido por 0 es NaN.

Uso

```
Power Query M
```

```
Number.IsNaN(0/0)
```

Salida

```
true
```

Ejemplo 2

Compruebe si 1 dividido por 0 es NaN.

Uso

```
Power Query M
```

```
Number.IsNaN(1/0)
```

Salida

```
false
```


Number.IsOdd

16/09/2025

Sintaxis

```
Number.IsOdd(number as number) as logical
```

Acerca de

Indica si el valor es impar. Devuelve `true` si `number` es un número impar, `false` de lo contrario.

Ejemplo 1

Compruebe si 625 es un número impar.

Uso

Power Query M

```
Number.IsOdd(625)
```

Salida

```
true
```

Ejemplo 2

Compruebe si 82 es un número impar.

Uso

Power Query M

```
Number.IsOdd(82)
```

Salida

```
false
```

Number.Ln

Sintaxis

```
Number.Ln(number as nullable number) as nullable number
```

Acerca de

Devuelve el logaritmo natural de un número, `number`. Si `number` es null `Number.Ln` devuelve null.

Ejemplo

Obtenga el logaritmo natural de 15.

Uso

Power Query M

```
Number.Ln(15)
```

Salida

```
2.70805020110221
```

Last updated on 25/03/2026

Number.Log

16/09/2025

Sintaxis

```
Number.Log(number as nullable number, optional base as nullable number) as nullable number
```

Acerca de

Devuelve el logaritmo de un número, **number**, a la base de **base** especificada. Si no se especifica **base**, el valor predeterminado es Number.E. Si **number** es null **Number.Log** devuelve null.

Ejemplo 1

Obtenga el logaritmo base 10 de 2.

Uso

```
Power Query M
```

```
Number.Log(2, 10)
```

Salida

```
0.3010299956639812
```

Ejemplo 2

Obtenga el logaritmo base e de 2.

Uso

```
Power Query M
```

```
Number.Log(2)
```

Salida

0.69314718055994529

Number.Log10

16/09/2025

Sintaxis

```
Number.Log10(number as nullable number) as nullable number
```

Acerca de

Devuelve el logaritmo base 10 de un número, `number`. Si `number` es null **Number.Log10** devuelve null.

Ejemplo 1

Obtenga el logaritmo base 10 de 2.

Uso

```
Power Query M
```

```
Number.Log10(2)
```

Salida

```
0.3010299956639812
```

Number.Mod

16/10/2025

Syntax

```
Number.Mod(  
    number as nullable number,  
    divisor as nullable number,  
    optional precision as nullable number  
) as nullable number
```

Acerca de

Devuelve el resto resultante de la división de enteros de `number` por `divisor`. Si `number` o `divisor` son `null`, esta función devuelve `null`.

- `number`: dividendo.
- `divisor`: El divisor.
- `precision`: (Opcional) Precisión de la división de enteros. Este parámetro puede ser [Precision.Double](#) para `Double` precisión o [Precision.Decimal](#) para `Decimal` precisión. El valor predeterminado es `Precision.Double`.

Ejemplo 1

Busque el resto cuando divida entre 5 y 3.

Uso

Power Query M

```
Number.Mod(5, 3)
```

Salida

2

Ejemplo 2

Encuentre el resto cuando divida 10,5 entre 0,2, con `Double` precisión y `Decimal` precisión.

Uso

Power Query M

```
let
    Dividend = 10.5,
    Divisor = 0.2,

    #"Use Double Precision" = Number.Mod(Dividend, Divisor, Precision.Double),
    #"Use Decimal Precision" = Number.Mod(Dividend, Divisor, Precision.Decimal),

    // Convert to text to inspect precision
    #"Double To Text" = Number.ToText(#"Use Double Precision", "G"),
    #"Decimal To Text" = Number.ToText(#"Use Decimal Precision", "G"),

    #"Display Result" = [
        DoublePrecision = #"Double To Text",
        DecimalPrecision = #"Decimal To Text"
    ]

in
    #"Display Result"
```

Salida

Power Query M

```
[
    DoublePrecision = "0.09999999999999994",
    DecimalPrecision = "0.1"
]
```

Number.Permutations

Syntax

```
Number.Permutations(setSize as nullable number, permutationSize as nullable number)  
as nullable number
```

Acerca de

Devuelve el número de permutaciones que se pueden generar a partir de una serie de elementos, , **setSize** con un tamaño de permutación especificado, **permutationSize**.

Ejemplo 1

Busque el número de permutaciones de un total de 5 elementos en grupos de 3.

Uso

Power Query M

```
Number.Permutations(5, 3)
```

Salida

60

Last updated on 22/11/2025

Number.Power

Syntax

```
Number.Power(number as nullable number, power as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de elevar `number` al poder de `power`. Si `number` o `power` son NULL, **Number.Power** devuelve null.

- `number`: la base.
- `power`: exponente.

Ejemplo 1

Busque el valor de 5 elevado a la potencia de 3 (5 cubos).

Uso

Power Query M

```
Number.Power(5, 3)
```

Salida

125

Last updated on 22/11/2025

Number.Random

Sintaxis

```
Number.Random() as number
```

Acerca de

Devuelve un número aleatorio entre 0 y 1.

Ejemplo

Obtiene un número aleatorio.

Uso

Power Query M

```
Number.Random()
```

Salida

```
0.919303
```

Last updated on 25/03/2026

Number.RandomBetween

16/09/2025

Sintaxis

```
Number.RandomBetween(bottom as number, top as number) as number
```

Acerca de

Devuelve un número aleatorio entre `bottom` y `top`.

Ejemplo 1

Obtiene un número aleatorio entre 1 y 5.

Uso

Power Query M

```
Number.RandomBetween(1, 5)
```

Salida

2.546797

Number.Round

Syntax

```
Number.Round(  
    number as nullable number,  
    optional digits as nullable number,  
    optional roundingMode as nullable number  
) as nullable number
```

Acerca de

Devuelve el resultado del redondeo `number` al número más cercano. Si `number` es null, **Number.Round** devuelve null.

De forma predeterminada, `number` se redondea al entero más cercano y los empates se rompen redondeando al número par más cercano (usando [RoundingMode.ToEven](#), también conocido como "redondeo del bancario").

Sin embargo, estos valores predeterminados se pueden invalidar a través de los siguientes parámetros opcionales.

- `digits`: hace `number` que se redondee al número especificado de dígitos decimales.
- `roundingMode`: invalida el comportamiento predeterminado de desempate cuando `number` está en el punto medio entre dos valores redondeados potenciales (consulte [RoundingMode.Type](#) para conocer los valores posibles).

Ejemplo 1

Redondeo 1.234 al entero más cercano.

Uso

Power Query M

```
Number.Round(1.234)
```

Salida

Ejemplo 2

Redondeo de 1,56 al entero más cercano.

Uso

Power Query M

```
Number.Round(1.56)
```

Salida

2

Ejemplo 3

Redondeo de 1,2345 a dos posiciones decimales.

Uso

Power Query M

```
Number.Round(1.2345, 2)
```

Salida

1.23

Ejemplo 4

Redondeo de 1,2345 a tres posiciones decimales (Redondeo hacia arriba).

Uso

Power Query M

```
Number.Round(1.2345, 3, RoundingMode.Up)
```

Salida

1.235

Ejemplo 5

Redondeo de 1,2345 a tres posiciones decimales (Redondeo hacia abajo).

Uso

Power Query M

```
Number.Round(1.2345, 3, RoundingMode.Down)
```

Salida

1.234

Last updated on 22/11/2025

Number.RoundAwayFromZero

Syntax

```
Number.RoundAwayFromZero(number as nullable number, optional digits as nullable number) as nullable number
```

Acerca de

Devuelve el resultado del redondeo **number** basado en el signo del número. Esta función redondea los números positivos y los números negativos hacia abajo. Si **digits** se especifica, **number** se redondea al **digits** número de dígitos decimales.

Ejemplo 1

Redondea el número -1,2 lejos de cero.

Uso

Power Query M

```
Number.RoundAwayFromZero(-1.2)
```

Salida

-2

Ejemplo 2

Redondea el número 1,2 lejos de cero.

Uso

Power Query M

```
Number.RoundAwayFromZero(1.2)
```

Salida

2

Ejemplo 3

Redondea el número -1,234 a dos posiciones decimales lejos de cero.

Uso

Power Query M

```
Number.RoundAwayFromZero(-1.234, 2)
```

Salida

-1.24

Last updated on 22/11/2025

Number.RoundDown

16/09/2025

Sintaxis

```
Number.RoundDown(number as nullable number, optional digits as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de redondear **number** a la baja al entero anterior más alto. Si **number** es NULL, esta función devuelve null. Si se proporciona **digits**, **number** se redondea al número especificado de dígitos decimales.

Ejemplo 1

Redondear 1,234 a la baja a un entero.

Uso

Power Query M

```
Number.RoundDown(1.234)
```

Salida

1

Ejemplo 2

Redondear 1,999 a la baja a un entero.

Uso

Power Query M

```
Number.RoundDown(1.999)
```

Salida

Ejemplo 3

Redondear 1,999 a la baja a dos decimales.

Uso

Power Query M

```
Number.RoundDown(1.999, 2)
```

Salida

1.99

Number.RoundTowardZero

16/09/2025

Sintaxis

```
Number.RoundTowardZero(number as nullable number, optional digits as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de redondear `number` según el signo del número. Esta función redondea los números positivos hacia abajo y los números negativos hacia arriba. Si se especifica `digits`, `number` se redondea al número `digits` de dígitos decimales.

Ejemplo 1

Redondea el número -1.2 hacia cero.

Uso

```
Power Query M
```

```
Number.RoundTowardZero(-1.2)
```

Salida

```
-1
```

Ejemplo 2

Redondea el número 1,2 hacia cero.

Uso

```
Power Query M
```

```
Number.RoundTowardZero(1.2)
```

Salida

Ejemplo 3

Redondea el número -1,234 a dos cifras decimales con dirección a cero.

Uso

Power Query M

```
Number.RoundTowardZero(-1.234, 2)
```

Salida

-1.23

Number.RoundUp

Syntax

```
Number.RoundUp(number as nullable number, optional digits as nullable number) as nullable number
```

Acerca de

Devuelve el resultado de redondear `number` hasta el siguiente entero más alto. Si `number` es null, esta función devuelve null. Si `digits` se proporciona, `number` se redondea al número especificado de dígitos decimales.

Ejemplo 1

Redondea de 1,234 a entero.

Uso

Power Query M

```
Number.RoundUp(1.234)
```

Salida

2

Ejemplo 2

Redondea de 1,999 a entero.

Uso

Power Query M

```
Number.RoundUp(1.999)
```

Salida

2

Ejemplo 3

Redondea de 1,234 a dos posiciones decimales.

Uso

Power Query M

```
Number.RoundUp(1.234, 2)
```

Salida

1.24

Last updated on 22/11/2025

Number.Sign

Syntax

```
Number.Sign(number as nullable number) as nullable number
```

Acerca de

Devuelve 1 para si `number` es un número positivo, -1 si es un número negativo y 0 si es cero. Si `number` es null, `Number.Sign` devuelve null.

Ejemplo 1

Determine el signo de 182.

Uso

```
Power Query M  
Number.Sign(182)
```

Salida

```
1
```

Ejemplo 2

Determine el signo de -182.

Uso

```
Power Query M  
Number.Sign(-182)
```

Salida

```
-1
```

Ejemplo 3

Determine el signo de 0.

Uso

Power Query M

```
Number.Sign(0)
```

Salida

0

Last updated on 22/11/2025

Number.Sin

Syntax

```
Number.Sin(number as nullable number) as nullable number
```

Acerca de

Devuelve el seno de `number`.

Ejemplo

Busque el seno del ángulo 0.

Uso

Power Query M
Number.Sin(0)

Salida

0

Last updated on 25/03/2026

Number.Sinh

Syntax

```
Number.Sinh(number as nullable number) as nullable number
```

Acerca de

Devuelve el seno hiperbólico de `number`.

Last updated on 22/11/2025

Number.Sqrt

Syntax

```
Number.Sqrt(number as nullable number) as nullable number
```

Acerca de

Devuelve la raíz cuadrada de `number`. Si `number` es null, **Number.Sqrt** devuelve null. Si es un valor negativo, **Number.NaN** se devuelve (no un número).

Ejemplo 1

Busque la raíz cuadrada de 625.

Uso

Power Query M

```
Number.Sqrt(625)
```

Salida

25

Ejemplo 2

Busque la raíz cuadrada de 85.

Uso

Power Query M

```
Number.Sqrt(85)
```

Salida

9.2195444572928871

Number.Tan

Syntax

```
Number.Tan(number as nullable number) as nullable number
```

Acerca de

Devuelve la tangente de `number`.

Ejemplo 1

Busque la tangente del ángulo 1.

Uso

Power Query M

```
Number.Tan(1)
```

Salida

```
1.5574077246549023
```

Last updated on 22/11/2025

Number.Tanh

16/09/2025

Sintaxis

```
Number.Tanh(number as nullable number) as nullable number
```

Acerca de

Devuelve la tangente hiperbólica de `number`.

Number.ToText

16/09/2025

Sintaxis

```
Number.ToText(  
    number as nullable number,  
    optional format as nullable text,  
    optional culture as nullable text  
) as nullable text
```

Acerca de

Convierte el valor numérico en un valor `number` de texto según el formato especificado por `format`.

El formato es un valor de texto que indica cómo se debe convertir el número. Para obtener más información sobre los valores de formato admitidos, vaya a [Cadenas de formato numérico estándar](#) y [Cadenas de formato numérico personalizado](#).

También se puede proporcionar una opción `culture` (por ejemplo, "en-US") para controlar el comportamiento dependiente de la cultura de `format`.

Ejemplo 1

Convierta un número en texto sin especificar un formato.

Uso

Power Query M

```
Number.ToText(4)
```

Salida

```
"4"
```

Ejemplo 2

Convierta un número en formato exponencial.

Uso

Power Query M

```
Number.ToText(4, "e")
```

Salida

```
"4.000000e+000"
```

Ejemplo 3

Convierta un número en formato de porcentaje con solo una posición decimal.

Uso

Power Query M

```
Number.ToText(-0.1234, "P1")
```

Salida

```
"-12.3 %"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [cadenas de formato numérico estándar](#)
- [cadenas de formato numérico personalizado](#)

Percentage.From

Syntax

```
Percentage.From(value as any, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un valor de `percentage` del `value` especificado. Si el `value` especificado es `null`, **Percentage.From** devuelve `null`. Si el elemento especificado `value` tiene `text` un símbolo de porcentaje final, se devolverá el número decimal convertido. De lo contrario, el valor se convertirá en un `number` mediante [Number.From](#). También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor `percentage` de "12.3%".

Uso

Power Query M

```
Percentage.From("12.3%")
```

Salida

0.123

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Single.From

Syntax

```
Single.From(value as any, optional culture as nullable text) as nullable number
```

Acerca de

Devuelve un valor Single `number` del especificado `value`. Si el `value` especificado es `null`, **Single.From** devuelve `null`. Si el valor especificado `value` está `number` dentro del intervalo de Single, `value` se devuelve; de lo contrario, se devuelve un error. Si `value` es de cualquier otro tipo, primero se convertirá en un `number` mediante [Number.FromText](#). También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga el valor Single `number` de "1.5".

Uso

Power Query M

```
Single.From("1.5")
```

Salida

1.5

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Funciones de registro

19/06/2025

Estas funciones crean y manipulan valores de registro.

Información

 Expandir tabla

Nombre	Descripción
Record.FieldCount	Devuelve el número de campos de un registro.
Record.HasFields	Devuelve true si el nombre de campo o los nombres de campo están presentes en un registro.

Transformaciones

 Expandir tabla

Nombre	Descripción
Geography.FromWellKnownText	Traduce texto que representa un valor geográfico en formato de texto de Well-Known (WKT) en un registro estructurado.
Geography.ToWellKnownText	Convierte un valor de punto geográfico estructurado en su representación de texto de Well-Known (WKT).
GeographyPoint.From	Crea un registro que representa un punto geográfico a partir de partes.
Geometry.FromWellKnownText	Traduce texto que representa un valor geométrico en formato de texto de Well-Known (WKT) en un registro estructurado.
Geometry.ToWellKnownText	Convierte un valor de punto geométrico estructurado en su representación de texto de Well-Known (WKT).
GeometryPoint.From	Crea un registro que representa un punto geométrico a partir de partes.
Record.AddField	Agrega un campo a partir de un nombre y valor de campo.
Record.Combine	Combina los registros de una lista.
Record.RemoveFields	Quita los campos especificados del registro de entrada.

Nombre	Descripción
Record.RenameFields	Devuelve un nuevo registro que cambia el nombre de los campos especificados. Los campos resultantes conservarán su orden original. Esta función admite el intercambio y encadenamiento de nombres de campo. Sin embargo, todos los nombres de destino más los nombres de campo restantes deben constituir un conjunto único o se producirá un error.
Record.ReorderFields	Reordena los campos de registro para que coincidan con el orden de una lista de nombres de campo.
Record.TransformFields	Transforma campos aplicando transformOperations. Para obtener más información sobre los valores admitidos por transformOperations, vaya a Valores de parámetro .

Selección

 Expandir tabla

Nombre	Descripción
Record.Field	Devuelve el valor del campo especificado. Esta función se puede usar para crear dinámicamente la sintaxis de búsqueda de campos para un registro determinado. De ese modo, es una versión dinámica de la sintaxis record[field].
Record.FieldNames	Devuelve una lista de nombres de campo en orden de los campos del registro.
Record.FieldOrDefault	Devuelve el valor de un campo de un registro o el valor predeterminado si el campo no existe.
Record.FieldValues	Devuelve una lista de valores de campo en orden de los campos del registro.
Record.SelectFields	Devuelve un nuevo registro que contiene los campos seleccionados del registro de entrada. Se mantiene el orden original de los campos.

Serialización

 Expandir tabla

Nombre	Descripción
Record.FromList	Devuelve un registro si se proporciona una lista de valores de campo y un conjunto de campos.
Record.FromTable	Devuelve un registro de una tabla de registros que contiene valores y nombres de

Nombre	Descripción
	campo.
Record.ToList	Devuelve una lista de valores que contienen los valores de campo del registro de entrada.
Record.ToTable	Devuelve una tabla de registros que contiene nombres de campo y valores de un registro de entrada.

Valores de parámetros

Las siguientes definiciones de tipo se usan para describir los valores de parámetro a los que se hace referencia en las funciones Record anteriores.

[Expandir tabla](#)

Definición de tipo	Descripción
Opción MissingField	Más información: MissingField.Type
Operaciones de transformación	Las operaciones de transformación se pueden especificar mediante cualquiera de los valores siguientes: • Valor de lista de dos elementos, primer elemento que es el nombre del campo y el segundo elemento que es la función de transformación aplicada a ese campo para generar un nuevo valor. • Se puede proporcionar una lista de transformaciones proporcionando un valor de lista, y cada elemento es el valor de lista de 2 elementos como se describe anteriormente. Para obtener ejemplos, vaya a la descripción de Record.TransformFields
Cambiar el nombre de las operaciones	Las operaciones de cambio de nombre de un registro se pueden especificar como cualquiera de: Una sola operación de cambio de nombre, representada por una lista de dos nombres de campo, antiguos y nuevos. Para obtener ejemplos, vaya a la descripción de Record.RenameFields.

Geography.FromWellKnownText

16/09/2025

Sintaxis

```
Geography.FromWellKnownText(input as nullable text) as nullable record
```

Acerca de

Traduce texto que representa un valor geográfico en formato de texto de Well-Known (WKT) en un registro estructurado. WKT es un formato estándar definido por Open Geospatial Consortium (OGC) y es el formato de serialización típico que usan las bases de datos, incluido SQL Server.

Geography.ToWellKnownText

Syntax

```
Geography.ToWellKnownText(input as nullable record, optional omitSRID as nullable logical) as nullable text
```

Acerca de

Convierte un valor de punto geográfico estructurado en su representación de texto de Well-Known (WKT), tal como se define en Open Geospatial Consortium (OGC), también el formato de serialización usado por muchas bases de datos, incluido SQL Server.

Last updated on 22/11/2025

GeographyPoint.From

16/09/2025

Sintaxis

```
GeographyPoint.From(  
    longitude as number,  
    latitude as number,  
    optional z as nullable number,  
    optional m as nullable number,  
    optional srid as nullable number  
) as record
```

Acercas de

Crea un registro que representa un punto geográfico a partir de sus partes constituyentes, como la longitud, la latitud y, si está presente, la elevación (Z) y la medida (M). Se puede proporcionar un identificador de referencia espacial opcional (SRID) si es diferente del valor predeterminado (4326).

Geometry.FromWellKnownText

16/09/2025

Sintaxis

```
Geometry.FromWellKnownText(input as nullable text) as nullable record
```

Acerca de

Traduce texto que representa un valor geométrico en formato de texto de Well-Known (WKT) en un registro estructurado. WKT es un formato estándar definido por Open Geospatial Consortium (OGC) y es el formato de serialización típico que usan las bases de datos, incluido SQL Server.

Geometry.ToWellKnownText

Syntax

```
Geometry.ToWellKnownText(input as nullable record, optional omitSRID as nullable logical) as nullable text
```

Acerca de

Convierte un valor de punto geométrico estructurado en su representación de texto de Well-Known (WKT), tal como se define en Open Geospatial Consortium (OGC), también el formato de serialización usado por muchas bases de datos, incluido SQL Server.

Last updated on 22/11/2025

GeometryPoint.From

16/09/2025

Sintaxis

```
GeometryPoint.From(  
    x as number,  
    y as number,  
    optional z as nullable number,  
    optional m as nullable number,  
    optional srid as nullable number  
) as record
```

Acerca de

Crea un registro que representa un punto geométrico a partir de sus partes constituyentes, como coordenada X, coordenada Y y, si está presente, coordenada Z y medida (M). Se puede proporcionar un identificador de referencia espacial opcional (SRID) si es diferente del valor predeterminado (0).

Record.AddField

16/09/2025

Sintaxis

```
Record.AddField(  
    record as record,  
    fieldName as text,  
    value as any,  
    optional delayed as nullable logical  
) as record
```

Acerca de

Agrega un campo a un registro `record`, dado el nombre del campo `fieldName` y el valor `value`.

Ejemplo 1

Agregue el campo Address al registro.

Uso

Power Query M

```
Record.AddField([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "Address",  
"123 Main St.")
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567", Address = "123 Main St."]
```

Record.Combine

Syntax

```
Record.Combine(records as list) as record
```

Acerca de

Combina los registros de la especificada `records`. `records` Si contiene valores que no son de registro, se devuelve un error.

Ejemplo

Cree un registro combinado a partir de los registros.

Uso

Power Query M

```
Record.Combine({  
    [CustomerID = 1, Name = "Bob"],  
    [Phone = "123-4567"]  
})
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Last updated on 25/03/2026

Record.Field

Syntax

```
Record.Field(record as record, field as text) as any
```

Acerca de

Devuelve el valor del especificado `field` en `.record`. Si no se encuentra el campo, se genera un error.

Ejemplo

Busque el valor del campo "CustomerID" en el registro.

Uso

Power Query M

```
Record.Field([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "CustomerID")
```

Salida

1

Last updated on 25/03/2026

Record.FieldCount

16/09/2025

Sintaxis

```
Record.FieldCount(record as record) as number
```

Acerca de

Devuelve el número de campos del registro `record`.

Ejemplo 1

Busque el número de campos del registro.

Uso

Power Query M

```
Record.FieldCount([CustomerID = 1, Name = "Bob"])
```

Salida

2

Record.FieldNames

16/09/2025

Sintaxis

```
Record.FieldNames(record as record) as list
```

Acerca de

Devuelve los nombres de los campos del registro `record` como texto.

Ejemplo 1

Busque los nombres de los campos del registro.

Uso

Power Query M

```
Record.FieldNames([OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0])
```

Salida

```
{"OrderID", "CustomerID", "Item", "Price"}
```

Record.FieldOrDefault

16/09/2025

Sintaxis

```
Record.FieldOrDefault(  
    record as nullable record,  
    field as text,  
    optional defaultValue as any  
) as any
```

Acerca de

Devuelve el valor del campo especificado `field` en el registro `record`. Si no se encuentra el campo, se devuelve el `defaultValue` opcional.

Ejemplo 1

Busque el valor del campo "Phone" en el registro o devuelva null si no existe.

Uso

Power Query M

```
Record.FieldOrDefault([CustomerID = 1, Name = "Bob"], "Phone")
```

Salida

null

Ejemplo 2

Busque el valor del campo "Teléfono" en el registro o devuelva el valor predeterminado si no existe.

Uso

Power Query M

```
Record.FieldOrDefault([CustomerID = 1, Name = "Bob"], "Phone", "123-4567")
```

Salida

```
"123-4567"
```

Record.FieldValues

16/09/2025

Sintaxis

```
Record.FieldValues(record as record) as list
```

Acerca de

Devuelve una lista de los valores de campo del registro `record`.

Ejemplo 1

Busque los valores de campo en el registro.

Uso

Power Query M

```
Record.FieldValues([CustomerID = 1, Name = "Bob", Phone = "123-4567"])
```

Salida

```
{1, "Bob", "123-4567"}
```


Record.FromList

Syntax

```
Record.FromList(list as list, fields as any) as record
```

Acerca de

Devuelve un registro dado un `list` de valores de campo y un conjunto de campos. Se `fields` puede especificar mediante una lista de valores de texto o un tipo de registro. Se produce un error si los campos no son únicos.

Ejemplo 1

Cree un registro a partir de una lista de valores de campo y una lista de nombres de campo.

Uso

Power Query M

```
Record.FromList({1, "Bob", "123-4567"}, {"CustomerID", "Name", "Phone"})
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Ejemplo 2

Cree un registro a partir de una lista de valores de campo y un tipo de registro.

Uso

Power Query M

```
Record.FromList({1, "Bob", "123-4567"}, type [CustomerID = number, Name = text,  
Phone = number])
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Record.FromTable

Sintaxis

```
Record.FromTable(table as table) as record
```

Acerca de

Devuelve un registro de una tabla de registros `table` que contiene nombres de campo y nombres `{[Name = name, Value = value]}` de valor . Se produce un error si los nombres de campo no son únicos.

Ejemplo 1

Cree un registro a partir de la tabla del formulario `Table.FromRecords({[Name = "CustomerID", Value = 1], [Name = "Name", Value = "Bob"], [Name = "Phone", Value = "123-4567"]})`.

Uso

Power Query M

```
Record.FromTable(  
    Table.FromRecords({  
        [Name = "CustomerID", Value = 1],  
        [Name = "Name", Value = "Bob"],  
        [Name = "Phone", Value = "123-4567"]  
    })  
)
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Last updated on 23/01/2026

Record.HasFields

16/09/2025

Sintaxis

```
Record.HasFields(record as record, fields as any) as logical
```

Acerca de

Indica si el registro `record` tiene los campos especificados en `fields`, devolviendo un valor lógico (true o false). Se pueden especificar varios valores de campo mediante una lista.

Ejemplo 1

Compruebe si el registro tiene el campo "CustomerID".

Uso

Power Query M

```
Record.HasFields([CustomerID = 1, Name = "Bob", Phone = "123-4567"], "CustomerID")
```

Salida

```
true
```

Ejemplo 2

Compruebe si el registro tiene el campo "CustomerID" y "Address".

Uso

Power Query M

```
Record.HasFields([CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
{ "CustomerID", "Address" })
```

Salida

false

Record.RemoveFields

Syntax

```
Record.RemoveFields(  
    record as record,  
    fields as any,  
    optional missingField as nullable number  
) as record
```

Acerca de

Devuelve un registro que quita todos los campos especificados en la lista `fields` de la entrada `record`. Si el campo especificado no existe, se produce un error.

Ejemplo 1

Quite el campo "Price" del registro.

Uso

Power Query M

```
Record.RemoveFields([CustomerID = 1, Item = "Fishing rod", Price = 18.00], "Price")
```

Salida

```
[CustomerID = 1, Item = "Fishing rod"]
```

Ejemplo 2

Quite los campos "Price" y "Item" del registro.

Uso

Power Query M

```
Record.RemoveFields([CustomerID = 1, Item = "Fishing rod", Price = 18.00], {"Price",  
"Item"})
```

Salida

[CustomerID = 1]

Last updated on 23/01/2026

Record.RenameFields

16/09/2025

Sintaxis

```
Record.RenameFields(  
    record as record,  
    renames as list,  
    optional missingField as nullable number  
) as record
```

Acerca de

Devuelve un registro después de cambiar el nombre de los campos de la entrada `record` a los nuevos nombres de campo especificados en la lista `renames`. Para múltiples cambios de nombre, se puede usar una lista anidada ({ {old1, new1}, {old2, new2} }).

Ejemplo 1

Cambie el nombre del campo "UnitPrice" a "Price" del registro.

Uso

Power Query M

```
Record.RenameFields(  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", UnitPrice = 100.0],  
    {"UnitPrice", "Price"}  
)
```

Salida

```
[OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0]
```

Ejemplo 2

Cambie el nombre de los campos "UnitPrice" a "Price" y "OrderNum" a "OrderID" del registro.

Uso

Power Query M

```
Record.RenameFields(  
    [OrderNum = 1, CustomerID = 1, Item = "Fishing rod", UnitPrice = 100.0],  
    {  
        {"UnitPrice", "Price"},  
        {"OrderNum", "OrderID"}  
    }  
)
```

Salida

```
[OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0]
```

Record.ReorderFields

16/09/2025

Sintaxis

```
Record.ReorderFields(  
    record as record,  
    fieldOrder as list,  
    optional missingField as nullable number  
) as record
```

Acerca de

Reordena los campos de un registro para que coincidan con el orden de una lista de nombres de campo.

- **record**: registro que contiene los campos que se van a reordenar.
- **fieldOrder**: lista que contiene el nuevo orden de los campos que se van a aplicar al registro. Los valores de campo se mantienen y los campos que no aparecen en este parámetro se dejan en sus posiciones originales.
- **missingField**: especifica la acción esperada para los valores que faltan en una fila que contiene menos campos de los esperados. Los valores siguientes son válidos:
 - **MissingField.Error**: (Valor predeterminado) Indica que los campos que faltan deben producir un error. Si no se especifica ningún valor para el **missingField** parámetro, se usa este valor.
 - **MissingField.Ignore**: indica que se deben omitir los campos que faltan.
 - **MissingField.UseNull**: indica que los campos que faltan deben incluirse como **null** valores.

Ejemplo 1

Reordene algunos de los campos del registro.

Uso

Power Query M

```
Record.ReorderFields(  
    [CustomerID = 1, OrderID = 1, Item = "Fishing rod", Price = 100.0],
```

```
{ "OrderID", "CustomerID" }  
)
```

Salida

```
[OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0]
```

Ejemplo 2

Reordene algunos de los campos del registro e incluya `null` los campos que faltan.

Uso

Power Query M

```
let  
    Source = [CustomerID = 3, First Name = "Paul", Phone = "543-7890", Purchase =  
"Fishing Rod"],  
    reorderedRecord = Record.ReorderFields(  
        Source,  
        {"Purchase", "Last Name", "First Name"},  
        MissingField.UseNull  
    )  
in  
    reorderedRecord
```

Salida

```
[CustomerID = 3, Purchase = "Fishing Rod", Phone = "543-7890", Last Name = null, First  
Name = "Paul"]
```

Record.SelectFields

Sintaxis

```
Record.SelectFields(  
    record as record,  
    fields as any,  
    optional missingField as nullable number  
) as record
```

Acerca de

Devuelve un registro que incluye solo los campos especificados en la lista `fields` de la entrada `record`.

Ejemplo

Seleccione los campos "Item" y "Price" en el registro.

Uso

Power Query M

```
Record.SelectFields(  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],  
    {"Item", "Price"}  
)
```

Salida

```
[Item = "Fishing rod", Price = 100]
```

Last updated on 25/03/2026

Record.ToList

16/09/2025

Sintaxis

```
Record.ToList(record as record) as list
```

Acerca de

Devuelve una lista de valores que contienen los valores de campo de la entrada `record`.

Ejemplo 1

Extraiga los valores de campo de un registro.

Uso

Power Query M

```
Record.ToList([A = 1, B = 2, C = 3])
```

Salida

```
{1, 2, 3}
```

Record.ToTable

Syntax

```
Record.ToTable(record as record) as table
```

Acerca de

Devuelve una tabla que contiene las columnas `Name` y `Value` con una fila para cada campo de `record`.

Ejemplo

Devuelve la tabla del registro.

Uso

Power Query M

```
Record.ToTable([OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0])
```

Salida

Power Query M

```
Table.FromRecords({
    [Name = "OrderID", Value = 1],
    [Name = "CustomerID", Value = 1],
    [Name = "Item", Value = "Fishing rod"],
    [Name = "Price", Value = 100]
})
```

Record.TransformFields

16/09/2025

Sintaxis

```
Record.TransformFields(  
    record as record,  
    transformOperations as list,  
    optional missingField as nullable number  
) as record
```

Acerca de

Devuelve un registro después de aplicar las transformaciones especificadas en la lista `transformOperations` a `record`. Uno o varios campos se pueden transformar en un momento dado.

En el caso de que se transforme un solo campo, se espera que `transformOperations` sea una lista con dos elementos. El primer elemento de `transformOperations` especifica un nombre de campo y el segundo elemento de `transformOperations` especifica la función que se va a usar para la transformación. Por ejemplo: `{"Quantity", Number.FromText}`

En el caso de que se transformen varios campos, se espera que `transformOperations` sea una lista de listas, donde cada lista interna es un par de nombre de campo y operación de transformación. Por ejemplo: `{{"Quantity",Number.FromText},{"UnitPrice", Number.FromText}}`

Ejemplo 1

Convierta el campo "Precio" en número.

Uso

Power Query M

```
Record.TransformFields(  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = "100.0"],  
    {"Price", Number.FromText}  
)
```

Salida

```
[OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100]
```

Ejemplo 2

Convierta los campos "OrderID" y "Price" en números.

Uso

Power Query M

```
Record.TransformFields(  
    [OrderID = "1", CustomerID = 1, Item = "Fishing rod", Price = "100.0"],  
    {"OrderID", Number.FromText}, {"Price", Number.FromText}  
)
```

Salida

```
[OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100]
```


Funciones de reemplazo

Artículo • 17/12/2024

Otras funciones de la biblioteca usan estas funciones para reemplazar un valor determinado.

 Expandir tabla

Nombre	Descripción
Replacer.ReplaceText	Esta función se puede proporcionar a List.ReplaceValue o Table.ReplaceValue para reemplazar los valores de texto en listas y tablas, respectivamente.
Replacer.ReplaceValue	Esta función se proporciona para List.ReplaceValue o Table.ReplaceValue para reemplazar valores en los valores de lista y tabla, respectivamente.

Comentarios

¿Le ha resultado útil esta página?

 Sí

 No

[Proporcionar comentarios sobre el producto](#) | [Preguntar a la comunidad](#) 

Replacer.ReplaceText

Syntax

```
Replacer.ReplaceText(  
    text as nullable text,  
    old as text,  
    new as text  
) as nullable text
```

Acerca de

Reemplaza el `old` texto del original `text` por el `new` texto. Esta función de reemplazo se puede usar en `List.ReplaceValue` y `Table.ReplaceValue`.

Ejemplo 1

Reemplace el texto "hE" por "He" en la cadena "hEllo world".

Uso

Power Query M

```
Replacer.ReplaceText("hEllo world", "hE", "He")
```

Salida

```
"Hello world"
```

Last updated on 22/11/2025

Replacer.ReplaceValue

Sintaxis

```
Replacer.ReplaceValue(  
    value as any,  
    old as any,  
    new as any  
) as any
```

Acerca de

Reemplaza el valor de `old` en el `value` original por el valor de `new`. Esta función de reemplazo se puede usar en [List.ReplaceValue](#) y [Table.ReplaceValue](#).

Ejemplo

Reemplace el valor 11 por el valor 10.

Uso

Power Query M

```
Replacer.ReplaceValue(11, 11, 10)
```

Salida

10

Last updated on 04/04/2026

Funciones de separación

Estas funciones dividen texto.

 Expandir tabla

Nombre	Description
Splitter.SplitByNothing	Devuelve una función que no divide, devolviendo su argumento como una lista de elementos único.
Splitter.SplitTextByCharacterTransition	Devuelve una función que divide el texto en una lista de texto según una transición de un tipo de carácter a otro.
Splitter.SplitTextByAnyDelimiter	Devuelve una función que divide el texto por cualquier delimitador admitido.
Splitter.SplitTextByDelimiter	Devuelve una función que dividirá el texto según un delimitador.
Splitter.SplitTextByEachDelimiter	Devuelve una función que divide el texto por cada delimitador a su vez.
Splitter.SplitTextByLengths	Devuelve una función que divide el texto según las longitudes especificadas.
Splitter.SplitTextByPositions	Devuelve una función que divide el texto según las posiciones especificadas.
Splitter.SplitTextByRanges	Devuelve una función que divide el texto según los intervalos especificados.
Splitter.SplitTextByRepeatedLengths	Devuelve una función que divide el texto en una lista de texto después de la longitud especificada repetidamente.
Splitter.SplitTextByWhitespace	Devuelve una función que divide el texto según el espacio en blanco.

Splitter.SplitByNothing

16/09/2025

Sintaxis

```
Splitter.SplitByNothing() as function
```

Acerca de

Devuelve una función que no divide, devolviendo su argumento como una lista con un único elemento.

Splitter.SplitTextByAnyDelimiter

Syntax

```
Splitter.SplitTextByAnyDelimiter(  
    delimiters as list,  
    optional quoteStyle as nullable number,  
    optional startAtEnd as nullable logical  
) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto en cualquiera de los delimitadores especificados.

Ejemplo 1

Divida la entrada por coma o punto y coma, omita las comillas y los delimitadores entre comillas y comillas y comience desde el principio de la entrada.

Uso

Power Query M

```
Splitter.SplitTextByAnyDelimiter({"", ",", ";"}, QuoteStyle.Csv)("a,b;""c,d;e""",f")
```

Salida

```
{"a", "b", "c,d;e", "f"}
```

Ejemplo 2

Divida la entrada por coma o punto y coma, omita las comillas y los delimitadores entre comillas y comillas y comience desde el final de la entrada.

Uso

Power Query M

```
let  
    startAtEnd = true  
in
```

```
Splitter.SplitTextByAnyDelimiter({"", " ";"}, QuoteStyle.Csv, startAtEnd)
("a,""b;c,d")
```

Salida

```
{"a,b", "c", "d"}
```

Last updated on 22/11/2025

Splitter.SplitTextByCharacterTransition

Sintaxis

```
Splitter.SplitTextByCharacterTransition(before as anynonnull, after as anynonnull)  
as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto según una transición de un tipo de carácter a otro. Los parámetros `before` y `after` pueden ser una lista de caracteres o una función que toma un carácter y devuelve true/false.

Ejemplo

Divida la entrada cada vez que una letra mayúscula o minúscula vaya seguida de un dígito.

Uso

Power Query M

```
Splitter.SplitTextByCharacterTransition({"A".."Z", "a".."z"}, {"0".."9"})("Abc123")
```

Salida

```
{"Abc", "123"}
```

Last updated on 25/03/2026

Splitter.SplitTextByDelimiter

Syntax

```
Splitter.SplitTextByDelimiter(  
    delimiter as text,  
    optional quoteStyle as nullable number,  
    optional csvStyle as nullable number  
) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto según el delimitador especificado.

Ejemplo

Divida la entrada por coma y omita comas entre comillas.

Uso

Power Query M

```
Splitter.SplitTextByDelimiter(",", QuoteStyle.Csv)("a, \"b,c\", d")
```

Salida

```
{"a", "b,c", "d"}
```

Last updated on 25/03/2026

Splitter.SplitTextByEachDelimiter

Syntax

```
Splitter.SplitTextByEachDelimiter(  
    delimiters as list,  
    optional quoteStyle as nullable number,  
    optional startAtEnd as nullable logical  
) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto en cada delimitador especificado en secuencia.

Ejemplo 1

Divida la entrada por coma y, a continuación, punto y coma, comenzando desde el principio de la entrada.

Uso

Power Query M

```
Splitter.SplitTextByEachDelimiter({"", ",", ";"})("a,b;c,d")
```

Salida

```
{"a", "b", "c,d"}
```

Ejemplo 2

Divida la entrada por coma y, a continuación, punto y coma, tratando comillas como cualquier otro carácter y comenzando desde el final de la entrada.

Uso

Power Query M

```
let  
    startAtEnd = true  
in
```

```
Splitter.SplitTextByEachDelimiter({"", ";", "\""}, QuoteStyle.None, startAtEnd)
("a,\"\"b;c\"\",d")
```

Salida

```
{"a","\"b", "c\"\"", "d"}
```

Last updated on 22/11/2025

Splitter.SplitTextByLengths

16/09/2025

Sintaxis

```
Splitter.SplitTextByLengths(lengths as list, optional startAtEnd as nullable logical) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto por cada longitud especificada.

Ejemplo 1

Divida la entrada en los dos primeros caracteres seguidos de los tres siguientes, comenzando desde el principio de la entrada.

Uso

Power Query M

```
Splitter.SplitTextByLengths({2, 3})("AB123")
```

Salida

```
{"AB", "123"}
```

Ejemplo 2

Divida la entrada en los tres primeros caracteres seguidos de los dos siguientes, comenzando desde el final de la entrada.

Uso

Power Query M

```
let  
    startAtEnd = true
```

in

```
Splitter.SplitTextByLengths({5, 2}, startAtEnd)("RedmondWA98052")
```

```
{"WA", "98052"}
```

Splitter.SplitTextByPositions

16/09/2025

Sintaxis

```
Splitter.SplitTextByPositions(positions as list, optional startAtEnd as nullable logical) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto en cada posición especificada.

Ejemplo 1

Divida la entrada en las posiciones especificadas, comenzando desde el principio de la entrada.

Uso

Power Query M

```
Splitter.SplitTextByPositions({0, 3, 4})("ABC|12345")
```

Salida

```
{"ABC", "|", "12345"}
```

Ejemplo 2

Divida la entrada en las posiciones especificadas, comenzando desde el final de la entrada.

Uso

Power Query M

```
let
    startAtEnd = true
in
    Splitter.SplitTextByPositions({0, 5}, startAtEnd)("Redmond98052")
```

Salida

```
{"Redmond", "98052"}
```

Splitter.SplitTextByRanges

Syntax

```
Splitter.SplitTextByRanges(ranges as list, optional startAtEnd as nullable logical)  
as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto según los desplazamientos y longitudes especificados. Una longitud nula indica que se debe incluir toda la entrada restante.

Ejemplo 1

Divida la entrada por los pares de posición y longitud especificados, comenzando desde el principio de la entrada. Tenga en cuenta que los intervalos de este ejemplo se superponen.

Uso

Power Query M

```
Splitter.SplitTextByRanges({{0, 4}, {2, 10}})("codelimiter")
```

Salida

```
{"code", "delimiter"}
```

Ejemplo 2

Divida la entrada por los pares de posición y longitud especificados, comenzando desde el final de la entrada.

Uso

Power Query M

```
let  
    startAtEnd = true
```



```
in  
Splitter.SplitTextByRanges({{0, 5}, {6, 2}}, startAtEnd)("RedmondWA?98052")
```

Salida

```
{"WA", "98052"}
```

Ejemplo 3

Divida la entrada en un código postal de longitud fija seguido de un nombre de ciudad de longitud variable.

Uso

Power Query M

```
Splitter.SplitTextByRanges({{0, 5}, {5, null}})("98052Redmond")
```

Salida

```
{"98052", "Redmond"}
```

Last updated on 04/04/2026

Splitter.SplitTextByRepeatedLengths

16/09/2025

Sintaxis

```
Splitter.SplitTextByRepeatedLengths(length as number, optional startAtEnd as nullable logical) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto después de la longitud especificada repetidamente.

Ejemplo 1

Divida repetidamente la entrada en fragmentos de tres caracteres, empezando desde el principio de la entrada.

Uso

Power Query M

```
Splitter.SplitTextByRepeatedLengths(3)("12345678")
```

Salida

```
{"123", "456", "78"}
```

Ejemplo 2

Divida repetidamente la entrada en fragmentos de tres caracteres, comenzando desde el final de la entrada.

Uso

Power Query M

```
let  
    startAtEnd = true
```

in

```
Splitter.SplitTextByRepeatedLengths(3, startAtEnd)("87654321")
```

Salida

```
{"87", "654", "321"}
```

Splitter.SplitTextByWhitespace

16/09/2025

Sintaxis

```
Splitter.SplitTextByWhitespace(optional quoteStyle as nullable number) as function
```

Acerca de

Devuelve una función que divide el texto en una lista de texto en un espacio en blanco.

Ejemplo 1

Divida la entrada por caracteres de espacio en blanco, tratando comillas como cualquier otro carácter.

Uso

Power Query M

```
Splitter.SplitTextByWhitespace(QuoteStyle.None)("a b#(tab)c")
```

Salida

```
{"a", "b", "c"}
```

Funciones de tabla

29/08/2025

Estas funciones crean y manipulan valores de tabla.

Construcción de tabla

 Expandir tabla

Nombre	Descripción
#table	Crea un valor de tabla a partir de columnas y filas.
ItemExpression.From	Devuelve el árbol de sintaxis abstracta (AST) para el cuerpo de una función.
RowExpression.Column	Devuelve un árbol de sintaxis abstracta (AST) que representa el acceso a una columna dentro de una expresión de fila.
RowExpression.From	Devuelve el árbol de sintaxis abstracta (AST) para el cuerpo de una función.
Table.FromColumns	Crea una tabla a partir de una lista de columnas y valores especificados.
Table.FromList	Convierte una lista en una tabla aplicando la función de división especificada a cada elemento de la lista.
Table.FromRecords	Convierte una lista de registros en una tabla.
Table.FromRows	Crea una tabla a partir de una lista de valores de fila y columnas opcionales.
Table.FromValue	Crea una tabla con una columna a partir del valor o valores proporcionados.
Table.WithErrorContext	Esta función está pensada solo para uso interno.
Table.View	Crea o extiende una tabla con controladores definidos por el usuario para las operaciones de consulta y acción.
Table.ViewError	Crea un registro de error modificado que no desencadenará una falla cuando sea lanzado por un controlador definido en una vista a través de Table.View .
Table.ViewFunction	Crea una función que puede ser interceptada por un manejador definido en una vista (a través de Table.View).

Conversiones

 Expandir tabla

Nombre	Descripción
Table.ToColumns	Crea una lista de listas anidadas de valores de columna a partir de una tabla.
Table.ToList	Convierte una tabla en una lista aplicando la función de combinación especificada a cada fila de valores de la tabla.
Table.ToRecords	Convierte una tabla en una lista de registros.
Table.ToRows	Crea una lista de listas anidadas de valores de fila a partir de una tabla.

Información

 Expandir tabla

Nombre	Descripción
Table.ApproximateRowCount	Devuelve el número aproximado de filas de la tabla.
Table.ColumnCount	Devuelve el número de columnas de la tabla.
Table.IsEmpty	Indica si la tabla contiene filas.
Table.PartitionValues	Devuelve información sobre cómo se particiona una tabla.
Table.Profile	Devuelve un perfil de las columnas de una tabla.
Table.RowCount	Devuelve el número de filas de la tabla.
Table.Schema	Devuelve una tabla que contiene una descripción de las columnas (es decir, el esquema) de la tabla especificada.
Tables.GetRelationships	Obtiene las relaciones entre un conjunto de tablas.

Operaciones de fila

 Expandir tabla

Nombre	Descripción
Table.AlternateRows	Mantiene el desplazamiento inicial y luego alterna entre tomar y omitir las filas siguientes.
Table.Combine	Devuelve una tabla que es el resultado de combinar una lista de tablas.
Table.FindText	Devuelve todas las filas que contienen el texto especificado en la tabla.

Nombre	Descripción
Table.First	Devuelve la primera fila o un valor predeterminado especificado.
Table.FirstN	Devuelve las primeras filas de recuento especificadas.
Table.FirstValue	Devuelve la primera columna de la primera fila de la tabla o un valor predeterminado especificado.
Table.FromPartitions	Devuelve una tabla que es el resultado de combinar un conjunto de tablas con particiones.
Table.InsertRows	Inserta una lista de filas en la tabla en la posición especificada.
Table.Last	Devuelve la última fila o un valor predeterminado especificado.
Table.LastN	Devuelve el último número especificado de filas.
Table.MatchesAllRows	Indica si todas las filas de la tabla cumplen la condición especificada.
Table.MatchesAnyRows	Indica si alguna de las filas de la tabla cumple la condición especificada.
Table.Partition	Divide la tabla en una lista de tablas en función del número de grupos y columnas especificados.
Table.Range	Devuelve las filas que comienzan en el desplazamiento especificado.
Table.RemoveFirstN	Devuelve una tabla con el número especificado de filas quitadas de la tabla a partir de la primera fila.
Table.RemoveLastN	Devuelve una tabla con el número especificado de filas quitadas de la tabla a partir de la última fila.
Table.RemoveRows	Quita el número especificado de filas.
Table.RemoveRowsWithErrors	Devuelve una tabla con las filas quitadas de la tabla de entrada que contienen un error en al menos una de las celdas. Si se especifica una lista de columnas, solo se inspeccionan las celdas de las columnas especificadas para detectar errores.
Table.Repeat	Repite las filas de las tablas un número especificado de veces.
Table.ReplaceRows	Reemplaza el intervalo especificado de filas por la fila o filas proporcionadas.
Table.ReverseRows	Devuelve una tabla con las filas en orden inverso.
Table.SelectRows	Selecciona las filas que cumplen la función condicional.
Table.SelectRowsWithErrors	Devuelve una tabla con solo las filas de la tabla de entrada que contienen un error en al menos una de las celdas. Si se especifica una lista de

Nombre	Descripción
	columnas, solo se inspeccionan las celdas de las columnas especificadas para detectar errores.
Table.SingleRow	Devuelve una única fila de la tabla.
Table.Skip	Devuelve una tabla con el primer número especificado de filas omitidas.
Table.SplitAt	Devuelve una lista que contiene las primeras filas de recuento especificadas y las filas restantes.

Operaciones de columna

 Expandir tabla

Nombre	Descripción
Table.Column	Devuelve una columna especificada de datos de la tabla como una lista.
Table.ColumnNames	Devuelve los nombres de columna como una lista.
Table.ColumnsOfType	Devuelve una lista con los nombres de las columnas que coinciden con los tipos especificados.
Table.DemoteHeaders	Degrada los encabezados de columna a la primera fila de valores.
Table.DuplicateColumn	Duplica una columna con el nombre especificado. Los valores y el tipo se copian de la columna de origen.
Table.HasColumns	Indica si la tabla contiene la columna o columnas especificadas.
Table.Pivot	Dado un par de columnas que representan pares de atributo-valor, convierte los datos de la columna de atributos a encabezados de columna.
Table.PrefixColumns	Devuelve una tabla donde todas las columnas se han prefijado con el texto especificado.
Table.PromoteHeaders	Promueve la primera fila de valores como encabezados de columna nuevos (es decir, como nombres de columna).
Table.RemoveColumns	Quita las columnas especificadas.
Table.ReorderColumns	Devuelve una tabla con las columnas en el orden especificado.
Table.RenameColumns	Devuelve una tabla con las columnas cuyo nombre se ha cambiado según se especifica.
Table.SelectColumns	Devuelve una tabla con solo las columnas especificadas.

Nombre	Descripción
Table.TransformColumnNames	Transforma los nombres de columna mediante la función especificada.
Table.Unpivot	Convierte un conjunto de columnas de una tabla en pares de valor de atributo.
Table.UnpivotOtherColumns	Convierte todas las columnas distintas de un conjunto especificado en pares de atributo-valor.

Transformación

 Expandir tabla

Nombre	Descripción
Table.AddColumn	Agrega una columna con el nombre especificado. El valor se calcula mediante la función de selección especificada con cada fila tomada como entrada.
Table.AddFuzzyClusterColumn	Agrega una nueva columna con valores representativos obtenidos por valores de agrupación aproximadas de la columna especificada en la tabla.
Table.AddIndexColumn	Anexa una columna con valores de posición explícitos.
Table.AddJoinColumn	Realiza una combinación entre tablas en las columnas proporcionadas y genera el resultado de combinación en una nueva columna.
Table.AddKey	Agrega una clave a una tabla.
Table.AggregateTableColumn	Agrega una columna de tablas a varias columnas de la tabla contenedora.
Table.CombineColumns	Combina las columnas especificadas en una nueva columna mediante la función de combinador especificada.
Table.CombineColumnsToRecord	Combina las columnas especificadas en una nueva columna con valores de registro donde cada registro tiene nombres de campo y valores correspondientes a los nombres de columna y los valores de las columnas que se combinaron.
Table.ConformToPageReader	Esta función está pensada solo para uso interno.
Table.ExpandListColumn	Dada una columna de listas de una tabla, cree una copia de una fila para cada valor de su lista.
Table.ExpandRecordColumn	Expande una columna de registros en columnas con cada uno de los valores.

Nombre	Descripción
Table.ExpandTableColumn	Expande una columna de registros o una columna que contiene tablas en varias columnas dentro de la tabla contenedora.
Table.FillDown	Propaga el valor de una celda anterior a las celdas con valores NULL siguientes en la columna.
Table.FillUp	Propaga el valor de una celda a las celdas con valores NULL anteriores en la columna.
Table.FilterWithDataTable	Esta función está pensada solo para uso interno.
Table.FuzzyGroup	Agrupar las filas de la tabla en función de la coincidencia aproximada de claves.
Table.FuzzyJoin	Combina las filas de las dos tablas que tienen una coincidencia aproximada en función de las claves especificadas.
Table.FuzzyNestedJoin	Realiza una combinación aproximada entre tablas en columnas proporcionadas y genera el resultado de combinación en una nueva columna.
Table.Group	Agrupar las filas de la tabla que tienen la misma clave.
Table.Join	Combina las filas de las dos tablas que coinciden en función de las claves especificadas.
Table.Keys	Devuelve las claves de la tabla especificada.
Table.NestedJoin	Realiza una combinación entre tablas en las columnas proporcionadas y genera el resultado de combinación en una nueva columna.
Table.PartitionClave	Devuelve la clave de partición de la tabla especificada.
Table.ReplaceErrorValues	Reemplaza los valores de error de las columnas especificadas por el valor especificado correspondiente.
Table.ReplaceKeys	Reemplaza las claves de la tabla especificada.
Table.ReplacePartitionKey	Reemplaza la clave de partición de la tabla especificada.
Table.ReplaceRelationshipIdentity	Esta función está pensada solo para uso interno.
Table.ReplaceValue	Reemplaza un valor por otro en las columnas especificadas.
Table.Split	Divide la tabla especificada en una lista de tablas con el tamaño de página especificado.
Table.SplitColumn	Divide la columna especificada en un conjunto de columnas adicionales mediante la función divisora especificada.

Nombre	Descripción
Table.TransformColumns	Transforma los valores de una o varias columnas.
Table.TransformColumnTypes	Aplica transformaciones de tipo del formulario { columna, tipo } mediante una referencia cultural específica.
Table.TransformRows	Transforma las filas de la tabla mediante la función de transformación especificada.
Table.Transpose	Convierte las columnas en filas y filas en columnas.

Membresía

 Expandir tabla

Nombre	Descripción
Table.Contains	Indica si el registro especificado aparece como una fila en la tabla.
Table.ContainsAll	Indica si todos los registros especificados aparecen como filas en la tabla.
Table.ContainsAny	Indica si alguno de los registros especificados aparece como filas en la tabla.
Table.Distinct	Quita las filas duplicadas de la tabla.
Table.IsDistinct	Indica si la tabla solo contiene filas distintas (sin duplicados).
Table.PositionOf	Devuelve la posición o las posiciones de la fila dentro de la tabla.
Table.PositionOfAny	Devuelve la posición o las posiciones de cualquiera de las filas especificadas dentro de la tabla.
Table.RemoveMatchingRows	Quita todas las apariciones de las filas especificadas de la tabla.
Table.ReplaceMatchingRows	Reemplaza todas las filas especificadas por la fila o filas proporcionadas.

Pedido

 Expandir tabla

Nombre	Descripción
Table.AddRankColumn	Anexa una columna con la clasificación de una o más columnas.
Table.Max	Devuelve la fila más grande o el valor predeterminado mediante los criterios

Nombre	Descripción
	especificados.
Table.MaxN	Devuelve la fila o las filas más grandes mediante los criterios especificados.
Table.Min	Devuelve la fila más pequeña o un valor predeterminado mediante los criterios especificados.
Table.MinN	Devuelve la fila o filas más pequeñas mediante los criterios especificados.
Table.Sort	Ordena la tabla con uno o varios nombres de columna y criterios de comparación.

Otros

 Expandir tabla

Nombre	Descripción
Table.Buffer	Almacena en búfer una tabla en memoria, aislándola de los cambios externos durante la evaluación.
Table.StopFolding	Impide que se ejecuten operaciones posteriores en la fuente original de los datos.

Valores de parámetros

Asignar nombres a columnas de salida

Este parámetro es una lista de valores de texto que especifican los nombres de columna de la tabla resultante. Este parámetro se usa generalmente en las [funciones de construcción Table](#), como [Table.FromRows](#) y [Table.FromList](#).

Criterios de comparación

El criterio de comparación se puede proporcionar como cualquiera de los siguientes valores:

- Valor numérico para especificar un criterio de ordenación. Más información: [Criterio de ordenación](#)
- Para calcular una clave que se va a usar para ordenar, se puede usar una función de un argumento.

- Para seleccionar una clave y un orden de control, el criterio de comparación puede ser una lista que contenga la clave y el orden.
- Para controlar completamente la comparación, se puede usar una función de dos argumentos que devuelve -1, 0 o 1 dada la relación entre las entradas izquierda y derecha. [Value.Compare](#) se puede usar para delegar esta lógica.

Para obtener ejemplos, vaya a la descripción de [Table.Sort](#).

Criterios de recuento o de condicionalidad

Este criterio se usa generalmente en operaciones de ordenación o fila. Determina el número de filas devueltas en la tabla y puede tomar dos formas, un número o una condición.

- Un número indica cuántos valores se van a devolver en línea con la función adecuada.
- Si se especifica una condición, se devuelven las filas que contienen valores que cumplen inicialmente la condición. Una vez que se produce un error en un valor de la condición, no se tienen en cuenta otros valores.

Más información: [Table.FirstN](#), [Table.MaxN](#)

Manejo de valores adicionales

Los valores adicionales se usan para indicar cómo la función debe controlar valores adicionales en una fila. Este parámetro se especifica como un número, que se asigna a las siguientes opciones:

```
ExtraValues.List = 0
```

```
ExtraValues.Error = 1
```

```
ExtraValues.Ignore = 2
```

Más información: [Table.FromList](#), [ExtraValues.Type](#)

Falta el control de columnas

Este parámetro se usa para indicar cómo la función debe controlar las columnas que faltan. Este parámetro se especifica como un número, que se asigna a las siguientes opciones:

```
MissingField.Error = 0
```

```
MissingField.Ignore = 1
```

```
MissingField.UseNull = 2;
```

Este parámetro se usa en operaciones de columna o transformación, por ejemplo, en [Table.TransformColumns](#). Más información: [MissingField.Type](#)

Criterio de ordenación

La ordenación se usa para indicar cómo se deben ordenar los resultados. Este parámetro se especifica como un número, que se asigna a las siguientes opciones:

```
Order.Ascending = 0
```

```
Order.Descending = 1
```

Más información: [Order.Type](#)

Criterios de ecuación

Los criterios de ecuación para las tablas se pueden especificar como:

- Un valor de función que es uno de los siguientes:
 - Selector de claves que determina la columna de la tabla para aplicar los criterios de igualdad.
 - Función del comparador que se usa para especificar el tipo de comparación que se va a aplicar. Se pueden especificar funciones de comparador integradas. Más información: [Funciones de comparador](#)
- Lista de las columnas de la tabla para aplicar los criterios de igualdad.

Para obtener ejemplos, vaya a la descripción de [Table.Distinct](#).

ItemExpression.From

Syntax

```
ItemExpression.From(function as function) as record
```

Acerca de

Devuelve el árbol de sintaxis abstracta (AST) para el cuerpo de , normalizado en una `function` elemento:

- La función debe ser una lambda de 1 argumento.
- Todas las referencias al parámetro de función se reemplazan por [ItemExpression.Item](#).
- El AST se simplificará para contener solo nodos de los tipos:
 - `Constant`
 - `Invocation`
 - `Unary`
 - `Binary`
 - `If`
 - `FieldAccess`

Se produce un error si no se puede devolver una expresión de elemento AST para el cuerpo de `function`.

Esta función es idéntica a [RowExpression.From](#).

Ejemplo

Devuelve el AST para el cuerpo de la función `each _ <> null`.

Uso

Power Query M

```
ItemExpression.From(each _ <> null)
```

Salida

Power Query M

```
[
  Kind = "Binary",
  Operator = "NotEquals",
  Left = ItemExpression.Item,
  Right =
  [
    Kind = "Constant",
    Value = null
  ]
]
```

Last updated on 25/03/2026

RowExpression.Column

Sintaxis

```
RowExpression.Column(columnName as text) as record
```

Acerca de

Devuelve un árbol de sintaxis abstracta (AST) que representa el acceso a la columna `columnName` de la fila dentro de una expresión de fila.

Ejemplo

Crea un AST que representa el acceso a la columna "CustomerName".

Uso

Power Query M

```
RowExpression.Column("CustomerName")
```

Salida

Power Query M

```
[
  Kind = "FieldAccess",
  Expression = RowExpression.Row,
  MemberName = "CustomerName"
]
```

Last updated on 25/03/2026

RowExpression.From

Syntax

```
RowExpression.From(function as function) as record
```

Acerca de

Devuelve el árbol de sintaxis abstracta (AST) para el cuerpo de `function`, normalizado en una *expresión de fila*:

- La función debe ser una lambda de 1 argumento.
- Todas las referencias al parámetro de función se reemplazan por [RowExpression.Row](#).
- Todas las referencias a columnas se reemplazan por [RowExpression.Column\(columnName\)](#).
- El AST se simplificará para contener solo nodos de los tipos:
 - `Constant`
 - `Invocation`
 - `Unary`
 - `Binary`
 - `If`
 - `FieldAccess`

Se produce un error si no se puede devolver una expresión de fila AST para el cuerpo de `function`.

Esta función es idéntica a [ItemExpression.From](#).

Ejemplo 1

Devuelve el AST para el cuerpo de la función `each [CustomerID] = "ALFKI"`.

Uso

Power Query M

```
RowExpression.From(each [CustomerName] = "ALFKI")
```

Salida

Power Query M

```
[
    Kind = "Binary",
    Operator = "Equals",
    Left = RowExpression.Column("CustomerName"),
    Right =
    [
        Kind = "Constant",
        Value = "ALFKI"
    ]
]
```

Last updated on 22/11/2025

Table.AddColumn

Syntax

```
Table.AddColumn(  
    table as table,  
    newColumnName as text,  
    columnGenerator as function,  
    optional columnType as nullable type  
) as table
```

Acerca de

Agrega una columna denominada `newColumnName` a la tabla `table`. Los valores de la columna se calculan mediante la función `columnGenerator` de selección especificada con cada fila tomada como entrada.

Ejemplo

Agregue una columna de número denominada "TotalPrice" a la tabla, con cada valor siendo la suma de las columnas [Price] y [Shipping].

Uso

Power Query M

```
Table.AddColumn(  
    Table.FromRecords({  
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0, Shipping  
= 10.00],  
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0, Shipping =  
15.00],  
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0, Shipping  
= 10.00]  
    }),  
    "TotalPrice",  
    each [Price] + [Shipping],  
    type number  
)
```

Salida

Power Query M

```
Table.FromRecords({
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100, Shipping = 10,
    TotalPrice = 110],
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5, Shipping = 15,
    TotalPrice = 20],
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25, Shipping = 10,
    TotalPrice = 35]
})
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 25/03/2026

Table.AddFuzzyClusterColumn

Syntax

```
Table.AddFuzzyClusterColumn(  
    table as table,  
    columnName as text,  
    newColumnName as text,  
    optional options as nullable record  
) as table
```

Acerca de

Agrega una nueva columna `newColumnName` a `table` con valores representativos de `columnName`. Los representantes se obtienen mediante valores coincidentes fuzzily en `columnName`, para cada fila.

Se puede incluir un conjunto opcional de para especificar cómo comparar las columnas de `options` clave. Entre las opciones se incluyen:

- `Culture`: permite agrupar registros en función de reglas específicas de la referencia cultural. Puede ser cualquier nombre de referencia cultural válido. Por ejemplo, una opción Culture de "ja-JP" agrupa registros basados en la referencia cultural japonesa. El valor predeterminado es "", que se agrupa en función de la referencia cultural de inglés invariable.
- `IgnoreCase`: valor lógico (true/false) que permite la agrupación de claves sin distinción entre mayúsculas y minúsculas. Por ejemplo, cuando es true, "Uvas" se agrupa con "uvas". El valor predeterminado es true.
- `IgnoreSpace`: valor lógico (true/false) que permite combinar elementos de texto para buscar grupos. Por ejemplo, cuando es true, "Gra pes" se agrupa con "Uvas". El valor predeterminado es true.
- `SimilarityColumnName`: un nombre para la columna que muestra la similitud entre un valor de entrada y el valor representativo de esa entrada. El valor predeterminado es NULL, en cuyo caso no se agregará una nueva columna para similitudes.
- `Threshold`: un número entre 0,00 y 1,00 que especifica la puntuación de similitud en la que se agruparán dos valores. Por ejemplo, "Uvas" y "Graes" (falta la "p") solo se agrupan si esta opción está establecida en menos de 0,90. Un umbral de 1,00 solo permite coincidencias exactas. (Tenga en cuenta que una "coincidencia exacta" aproximada podría omitir diferencias como mayúsculas y minúsculas, orden de palabras y puntuación). El valor predeterminado es 0,80.

- **TransformationTable**: tabla que permite agrupar registros en función de asignaciones de valores personalizadas. Debe contener columnas "From" y "To". Por ejemplo, "Uvas" se agrupa con "Raisins" si se proporciona una tabla de transformación con la columna "From" que contiene "Uvas" y la columna "To" que contiene "Raisins". Tenga en cuenta que la transformación se aplicará a todas las apariciones del texto de la tabla de transformación. Con la tabla de transformación anterior, "Las uvas son dulces" también se agruparán con "Las pasas son dulces".

Ejemplo 1

Busque los valores representativos de la ubicación de los empleados.

Uso

Power Query M

```
Table.AddFuzzyClusterColumn(  
    Table.FromRecords(  
        {  
            [EmployeeID = 1, Location = "Seattle"],  
            [EmployeeID = 2, Location = "seattl"],  
            [EmployeeID = 3, Location = "Vancouver"],  
            [EmployeeID = 4, Location = "Seattle"],  
            [EmployeeID = 5, Location = "vancouver"],  
            [EmployeeID = 6, Location = "Seattle"],  
            [EmployeeID = 7, Location = "Vancouver"]  
        },  
        type table [EmployeeID = nullable number, Location = nullable text]  
    ),  
    "Location",  
    "Location_Cleaned",  
    [IgnoreCase = true, IgnoreSpace = true]  
)
```

Salida

Power Query M

```
Table.FromRecords(  
    {  
        [EmployeeID = 1, Location = "Seattle", Location_Cleaned = "Seattle"],  
        [EmployeeID = 2, Location = "seattl", Location_Cleaned = "Seattle"],  
        [EmployeeID = 3, Location = "Vancouver", Location_Cleaned = "Vancouver"],  
        [EmployeeID = 4, Location = "Seattle", Location_Cleaned = "Seattle"],  
        [EmployeeID = 5, Location = "vancouver", Location_Cleaned = "Vancouver"],  
        [EmployeeID = 6, Location = "Seattle", Location_Cleaned = "Seattle"],  
        [EmployeeID = 7, Location = "Vancouver", Location_Cleaned = "Vancouver"]  
    },  
    type table [EmployeeID = nullable number, Location = nullable text,
```

```
Location_Cleaned = nullable text]
)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Table.AddIndexColumn

Syntax

```
Table.AddIndexColumn(  
    table as table,  
    newColumnName as text,  
    optional initialValue as nullable number,  
    optional increment as nullable number,  
    optional columnType as nullable type  
) as table
```

Acerca de

Anexa una columna denominada `newColumnName` a con `table` valores de posición explícitos. Valor opcional, `initialValue`, el valor de índice inicial. Un valor opcional, `increment`, especifica cuánto se debe incrementar cada valor de índice.

Ejemplo 1

Agregue una columna de índice denominada "Index" a la tabla.

Uso

Power Query M

```
Table.AddIndexColumn(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Index"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567", Index = 0],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543", Index = 1],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890", Index = 2],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550", Index = 3]
```

```
[CustomerID = 4, Name = "Ringo", Phone = "232-1550", Index = 3]
})
```

Ejemplo 2

Agregue una columna de índice denominada "index", empezando por el valor 10 e incrementándose en 5, a la tabla.

Uso

Power Query M

```
Table.AddIndexColumn(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    "Index",
    10,
    5
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567", Index = 10],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543", Index = 15],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890", Index = 20],
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550", Index = 25]
})
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Table.AddJoinColumn

Syntax

```
Table.AddJoinColumn(  
    table1 as table,  
    key1 as any,  
    table2 as any,  
    key2 as any,  
    newColumnName as text  
) as table
```

Acerca de

Combina las filas de `table1` con las filas de `table2` en función de la igualdad de los valores de las columnas de clave seleccionadas por `key1` (para `table1`) y `key2` (para `table2`). Los resultados se escriben en la columna denominada `newColumnName`. Esta función se comporta de forma similar a [Table.Join](#) con `joinKind` de `LeftOuter`, salvo que los resultados de la combinación se presentan de forma anidada en lugar de aplanada.

Ejemplo

Agregue una columna de combinación a (`[saleID = 1, item = "Shirt"]`, `[saleID = 2, item = "Hat"]`) denominado "price/stock" de la tabla (`[saleID = 1, price = 20]`, `[saleID = 2, price = 10]`) unido en `[saleID]`.

Uso

Power Query M

```
Table.AddJoinColumn(  
    Table.FromRecords(  
        [saleID = 1, item = "Shirt"],  
        [saleID = 2, item = "Hat"]  
    ),  
    "saleID",  
    () => Table.FromRecords(  
        [saleID = 1, price = 20, stock = 1234],  
        [saleID = 2, price = 10, stock = 5643]  
    ),  
    "saleID",
```

```
"price"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [  
        saleID = 1,  
        item = "Shirt",  
        price = Table.FromRecords({[saleID = 1, price = 20, stock = 1234]})  
    ],  
    [  
        saleID = 2,  
        item = "Hat",  
        price = Table.FromRecords({[saleID = 2, price = 10, stock = 5643]})  
    ]  
})
```

Last updated on 04/04/2026

Table.AddKey

Syntax

```
Table.AddKey(  
    table as table,  
    columns as list,  
    isPrimary as logical  
) as table
```

Acerca de

Agrega una clave a `table`, donde `columns` es la lista de nombres de columna que definen la clave y `isPrimary` especifica si la clave es principal.

Ejemplo 1

Agregue una clave principal de una sola columna a una tabla.

Uso

Power Query M

```
let  
    table = Table.FromRecords({  
        [Id = 1, Name = "Hello There"],  
        [Id = 2, Name = "Good Bye"]  
    }),  
    resultTable = Table.AddKey(table, {"Id"}, true)  
in  
    resultTable
```

Salida

Power Query M

```
Table.FromRecords({  
    [Id = 1, Name = "Hello There"],  
    [Id = 2, Name = "Good Bye"]  
})
```

Table.AddRankColumn

Sintaxis

```
Table.AddRankColumn(  
    table as table,  
    newColumnName as text,  
    comparisonCriteria as any,  
    optional options as nullable record  
) as table
```

Acerca de

Anexa una columna denominada `newColumnName` a `table` con la clasificación de una o varias columnas descritas por `comparisonCriteria`. Los usuarios avanzados pueden usar la opción `RankKind` en `options` para elegir un método de clasificación más específico.

Ejemplo

Agregue una columna denominada "RevenueRank" a la tabla que clasifica la columna "Ingresos" de mayor a menor.

Uso

Power Query M

```
Table.AddRankColumn(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Revenue = 200],  
        [CustomerID = 2, Name = "Jim", Revenue = 100],  
        [CustomerID = 3, Name = "Paul", Revenue = 200],  
        [CustomerID = 4, Name = "Ringo", Revenue = 50]  
    }  
),  
    "RevenueRank",  
    {"Revenue", Order.Descending},  
    [RankKind = RankKind.Competition]  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Revenue = 200, RevenueRank = 1],  
    [CustomerID = 3, Name = "Paul", Revenue = 200, RevenueRank = 1],  
    [CustomerID = 2, Name = "Jim", Revenue = 100, RevenueRank = 3],  
    [CustomerID = 4, Name = "Ringo", Revenue = 50, RevenueRank = 4]  
})
```

Contenido relacionado

[Criterios de comparación](#)

Last updated on 04/04/2026

Table.AggregateTableColumn

Sintaxis

```
Table.AggregateTableColumn(  
    table as table,  
    column as text,  
    aggregations as list  
) as table
```

Acerca de

Agrega tablas de `table` [`column`] en varias columnas que contienen valores agregados para las tablas. `aggregations` se usa para especificar las columnas que contienen las tablas que se van a agregar, las funciones de agregación que se aplicarán a las tablas para generar sus valores y los nombres de las columnas de agregado que se van a crear.

Ejemplo

Agregue columnas de tabla de `[t]` en la tabla `{{t = {[a=1, b=2, c=3], [a=2,b=4,c=6]}, b = 2}}` en la suma de `[t.a]`, el mínimo y máximo de `[t.b]` y el recuento de valores de `[t.a]`.

Uso

Power Query M

```
Table.AggregateTableColumn(  
    Table.FromRecords(  
        {  
            [  
                t = Table.FromRecords({  
                    [a = 1, b = 2, c = 3],  
                    [a = 2, b = 4, c = 6]  
                }),  
                b = 2  
            ]  
        },  
        type table [t = table [a = number, b = number, c = number], b = number]  
    ),  
    "t",  
    {  
        {"a", List.Sum, "sum of t.a"},  
        {"b", List.Min, "min of t.b"},  
        {"b", List.Max, "max of t.b"},  
    }  
)
```



```
        {"a", List.Count, "count of t.a"}  
    }  
)
```

Salida

```
Table.FromRecords({[#"sum of t.a" = 3, #"min of t.b" = 2, #"max of t.b" = 4, #"count of  
t.a" = 2, b = 2]})
```

Last updated on 25/03/2026

Table.AlternateRows

Syntax

```
Table.AlternateRows(  
    table as table,  
    offset as number,  
    skip as number,  
    take as number  
) as table
```

Acerca de

Mantiene el desplazamiento inicial y luego alterna entre tomar y omitir las filas siguientes.

- **table**: la tabla de entrada.
- **offset**: número de filas que se deben mantener antes de iniciar iteraciones.
- **skip**: número de filas que se van a quitar en cada iteración.
- **take**: número de filas que se van a mantener en cada iteración.

Ejemplo 1

Devuelve una tabla de la tabla que, a partir de la primera fila, omita 1 valor y, a continuación, mantenga 1 valor.

Uso

Power Query M

```
Table.AlternateRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    }),  
    1,  
    1,  
    1  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
})
```

Last updated on 22/11/2025

Table.ApproximateRowCount

Syntax

```
Table.ApproximateRowCount(table as table) as number
```

Acerca de

Devuelve el número aproximado de filas de `table` o un error si el origen de datos no admite la aproximación.

Ejemplo 1

Calcule el número de combinaciones distintas de ciudad y estado en una tabla grande, que se puede usar como una estimación de cardinalidad para las columnas. Las estimaciones de cardinalidad son lo suficientemente importantes como para que varios orígenes de datos (como SQL Server) admitan esta aproximación concreta, a menudo mediante un algoritmo denominado HyperLogLog.

Uso

Power Query M

```
Table.ApproximateRowCount(Table.Distinct(Table.SelectColumns(sqlTable, {"city", "state"})))
```

Salida

number

Last updated on 22/11/2025

Table.Buffer

Syntax

```
Table.Buffer(table as table, optional options as nullable record) as table
```

Acerca de

Almacena en búfer una tabla en memoria, aislándola de los cambios externos durante la evaluación. El almacenamiento en búfer es superficial. Fuerza la evaluación de cualquier valor de celda escalar, pero deja valores no escalares (registros, listas, tablas, etc.) as-is.

- **table**: tabla que se va a almacenar en búfer en memoria.
- **options**: [Opcional] Se pueden usar los siguientes valores de registro de opciones:
 - **BufferMode**: modo de búfer que describe el tipo de almacenamiento en búfer que se va a realizar. Esta opción puede ser [BufferMode.Eager](#) o [BufferMode.Delayed](#).

El uso de esta función podría o no hacer que las consultas se ejecuten más rápido. En algunos casos, puede hacer que las consultas se ejecuten más lentamente debido al costo adicional de leer todos los datos y almacenarlos en memoria, así como el hecho de que el almacenamiento en búfer impide el plegado de bajada. Si no es necesario almacenar en búfer los datos, pero solo quiere evitar el plegado de bajada, use [Table.StopFolding](#) en su lugar.

Ejemplo

Cargue todas las filas de una tabla SQL en memoria para que las operaciones de nivel inferior ya no puedan consultar el servidor SQL Server.

Uso

Power Query M

```
let
    Source = Sql.Database("SomeSQLServer", "MyDb"),
    MyTable = Source{[Item="MyTable"]}[Data],
    BufferMyTable = Table.Buffer(MyTable)
in
    BufferMyTable
```

Salida

Power Query M

table

Last updated on 25/03/2026

Table.Column

16/09/2025

Sintaxis

```
Table.Column(table as table, column as text) as list
```

Acerca de

Devuelve la columna de datos especificados por `column` de la tabla `table` como una lista.

Ejemplo 1

Devuelve los valores de la columna [Name] de la tabla.

Uso

Power Query M

```
Table.Column(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Name"  
)
```

Salida

```
{"Bob", "Jim", "Paul", "Ringo"}
```

Table.ColumnCount

Sintaxis

```
Table.ColumnCount(table as table) as number
```

Acerca de

Devuelve el número de columnas de la tabla `table`.

Ejemplo

Busque el número de columnas de la tabla.

Uso

Power Query M

```
Table.ColumnCount(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Salida

3

Last updated on 25/03/2026

Table.ColumnNames

Sintaxis

```
Table.ColumnNames(table as table) as list
```

Acerca de

Devuelve los nombres de columna de la tabla `table` como una lista de texto.

Ejemplo

Busque los nombres de columna de la tabla.

Uso

Power Query M

```
Table.ColumnNames(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)
```

Salida

```
{"CustomerID", "Name", "Phone"}
```

Last updated on 25/03/2026

Table.ColumnsOfType

Syntax

```
Table.ColumnsOfType(table as table, listOfTypes as list) as list
```

Acerca de

Devuelve una lista con los nombres de las columnas de la tabla `table` que coinciden con los tipos especificados en `listOfTypes`.

Ejemplo 1

Devuelve los nombres de las columnas de tipo `Number.Type` de la tabla.

Uso

Power Query M

```
Table.ColumnsOfType(  
    Table.FromRecords(  
        {[a = 1, b = "hello"]},  
        type table[a = Number.Type, b = Text.Type]  
    ),  
    {type number}  
)
```

Salida

```
{"a"}
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Table.Combine

16/09/2025

Sintaxis

```
Table.Combine(tables as list, optional columns as any) as table
```

Acercas de

Devuelve una tabla que es el resultado de combinar una lista de tablas, `tables`. La tabla resultante tendrá una estructura de tipo de fila definida por `columns` o por una unión de los tipos de entrada si no se especifica `columns`.

Ejemplo 1

Se combinan las tres tablas juntas.

Uso

Power Query M

```
Table.Combine({  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    Table.FromRecords({[CustomerID = 2, Name = "Jim", Phone = "987-6543"]}),  
    Table.FromRecords({[CustomerID = 3, Name = "Paul", Phone = "543-7890"]})  
})
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
})
```

Ejemplo 2

Combine tres tablas con estructuras diferentes.

Uso

Power Query M

```
Table.Combine({
    Table.FromRecords({[Name = "Bob", Phone = "123-4567"]}),
    Table.FromRecords({[Fax = "987-6543", Phone = "838-7171"]}),
    Table.FromRecords({[Cell = "543-7890"]})
})
```

Salida

Power Query M

```
Table.FromRecords({
    [Name = "Bob", Phone = "123-4567", Fax = null, Cell = null],
    [Name = null, Phone = "838-7171", Fax = "987-6543", Cell = null],
    [Name = null, Phone = null, Fax = null, Cell = "543-7890"]
})
```

Ejemplo 3

Se combinan dos tablas y el proyecto en el tipo especificado.

Uso

Power Query M

```
Table.Combine(
    {
        Table.FromRecords({[Name = "Bob", Phone = "123-4567"]}),
        Table.FromRecords({[Fax = "987-6543", Phone = "838-7171"]}),
        Table.FromRecords({[Cell = "543-7890"]})
    },
    {"CustomerID", "Name"}
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = null, Name = "Bob"],
    [CustomerID = null, Name = null],
    [CustomerID = null, Name = null]
})
```


Table.CombineColumns

Sintaxis

```
Table.CombineColumns(  
    table as table,  
    sourceColumns as list,  
    combiner as function,  
    column as text  
) as table
```

Acerca de

Combina las columnas especificadas en una nueva columna mediante la función de combinador especificada.

Ejemplo

Combine los apellidos y los nombres en una nueva columna, separadas por una coma.

Uso

Power Query M

```
Table.CombineColumns(  
    Table.FromRecords({[FirstName = "Bob", LastName = "Smith"]}),  
    {"LastName", "FirstName"},  
    Combiner.CombineTextByDelimiter(",", QuoteStyle.None),  
    "FullName"  
)
```

Salida

```
Table.FromRecords({[FullName = "Smith,Bob"]})
```

Table.CombineColumnsToRecord

Syntax

```
Table.CombineColumnsToRecord(  
    table as table,  
    newColumnName as text,  
    sourceColumns as list,  
    optional options as nullable record  
) as table
```

Acerca de

Combina las columnas especificadas de `table` en una nueva columna con valores de registro denominada `newColumnName` donde cada registro tiene nombres de campo y valores correspondientes a los nombres de columna y los valores de las columnas que se combinaron. Si se especifica un registro para `options`, se pueden proporcionar las siguientes opciones:

- `DisplayNameColumn`: cuando se especifica como texto, indica que el nombre de columna especificado debe tratarse como el nombre para mostrar del registro. No es necesario que sea una de las columnas del propio registro.
- `TypeName`: cuando se especifica como texto, proporciona un nombre de tipo lógico para el registro resultante que se puede usar durante la carga de datos para controlar el comportamiento del entorno de carga.

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Last updated on 22/11/2025

Table.ConformToPageReader

16/09/2025

Sintaxis

```
Table.ConformToPageReader(table as table, shapingFunction as function) as table
```

Acerca de

Esta función está pensada solo para uso interno.

Table.Contains

16/09/2025

Sintaxis

```
Table.Contains(  
    table as table,  
    row as record,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si el registro especificado, `row`, aparece como una fila de `table`. Se puede especificar un parámetro `equationCriteria` opcional para controlar la comparación entre las filas de la tabla.

Ejemplo 1

Determinar si la tabla contiene la fila.

Uso

Power Query M

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [Name = "Bob"]  
)
```

Salida

true

Ejemplo 2

Determinar si la tabla contiene la fila.

Uso

Power Query M

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [Name = "Ted"]  
)
```

Salida

false

Ejemplo 3

Determinar si la tabla contiene la fila comparando solo la columna [Name].

Uso

Power Query M

```
Table.Contains(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    [CustomerID = 4, Name = "Bob"],  
    "Name"  
)
```

Salida

true

Contenido relacionado

[Criterios de ecuación](#)

Table.ContainsAll

16/09/2025

Sintaxis

```
Table.ContainsAll(  
    table as table,  
    rows as list,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si todos los registros especificados en la lista de registros `rows` aparecen como filas en `table`. Se puede especificar un parámetro `equationCriteria` opcional para controlar la comparación entre las filas de la tabla.

Ejemplo 1

Determinar si la tabla contiene todas las filas comparando solo la columna [CustomerID].

Uso

Power Query M

```
Table.ContainsAll(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    {  
        [CustomerID = 1, Name = "Bill"],  
        [CustomerID = 2, Name = "Fred"]  
    },  
    "CustomerID"  
)
```

Salida

true

Ejemplo 2

Determinar si la tabla contiene todas las filas.

Uso

Power Query M

```
Table.ContainsAll(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    {  
        [CustomerID = 1, Name = "Bill"],  
        [CustomerID = 2, Name = "Fred"]  
    }  
)
```

Salida

false

Contenido relacionado

[Criterios de ecuación](#)

Table.ContainsAny

16/09/2025

Sintaxis

```
Table.ContainsAny(  
    table as table,  
    rows as list,  
    optional equationCriteria as any  
) as logical
```

Acerca de

Indica si alguno de los registros especificados en la lista de registros `rows` aparece como una fila en `table`. Se puede especificar un parámetro `equationCriteria` opcional para controlar la comparación entre las filas de la tabla.

Ejemplo 1

Determinar si la tabla `{{[a = 1, b = 2], [a = 3, b = 4]}}` contiene las filas `[a = 1, b = 2]` o `[a = 3, b = 5]`.

Uso

Power Query M

```
Table.ContainsAny(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }),  
    {  
        [a = 1, b = 2],  
        [a = 3, b = 5]  
    }  
)
```

Salida

true

Ejemplo 2

Determinar si la tabla `{{[a = 1, b = 2], [a = 3, b = 4]}}` contiene las filas `[a = 1, b = 3]` o `[a = 3, b = 5]`.

Uso

Power Query M

```
Table.ContainsAny(  
    Table.FromRecords(  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    )),  
    {  
        [a = 1, b = 3],  
        [a = 3, b = 5]  
    }  
)
```

Salida

false

Ejemplo 3

Determinar si la tabla `(Table.FromRecords({[a = 1, b = 2], [a = 3, b = 4]}))` contiene las filas `[a = 1, b = 3]` o `[a = 3, b = 5]` comparando solo la columna `[a]`.

Uso

Power Query M

```
Table.ContainsAny(  
    Table.FromRecords(  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    )),  
    {  
        [a = 1, b = 3],  
        [a = 3, b = 5]  
    },  
    "a"  
)
```

Salida

true

Contenido relacionado

[Criterios de ecuación](#)

Table.DemoteHeaders

Syntax

```
Table.DemoteHeaders(table as table) as table
```

Acerca de

Degrada los encabezados de columna (es decir, nombres de columna) a la primera fila de valores. Los nombres de columna predeterminados son "Column1", "Column2" y así sucesivamente.

Ejemplo 1

Degradar la primera fila de valores de la tabla.

Uso

Power Query M

```
Table.DemoteHeaders(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    })  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = "CustomerID", Column2 = "Name", Column3 = "Phone"],  
    [Column1 = 1, Column2 = "Bob", Column3 = "123-4567"],  
    [Column1 = 2, Column2 = "Jim", Column3 = "987-6543"]  
})
```


Table.Distinct

Sintaxis

```
Table.Distinct(table as table, optional equationCriteria as any) as table
```

Acerca de

Quita las filas duplicadas de la tabla. Un parámetro opcional, `equationCriteria`, especifica en qué columnas de la tabla se comprueba si hay duplicación. Si no se especifica `equationCriteria`, se comprueban todas las columnas.

Dado que Power Query a veces descarga ciertas operaciones en orígenes de datos back-end (conocidos como "plegado") y, a veces, optimiza las consultas omitiendo las operaciones que no son estrictamente necesarias, en general no hay ninguna garantía de qué duplicado específico se conservará. Por ejemplo, no se puede presuponer que se va a conservar la primera fila con un conjunto único de valores de columna y se van a quitar las filas situadas más abajo en la tabla. Si desea que la eliminación duplicada se comporte de forma predecible, primero almacene en búfer la tabla mediante [Table.Buffer](#).

Ejemplo 1

Se quitan las filas duplicadas de la tabla.

Uso

Power Query M

```
Table.Distinct(  
    Table.FromRecords({  
        [a = "A", b = "a"],  
        [a = "B", b = "b"],  
        [a = "A", b = "a"]  
    })  
)
```

Salida

Power Query M

```
Table.FromRecords({
    [a = "A", b = "a"],
    [a = "B", b = "b"]
})
```

Ejemplo 2

Se quitan las filas duplicadas de la columna [b] de la tabla `{{[a = "A", b = "a"], [a = "B", b = "a"], [a = "A", b = "b"]}}`.

Uso

Power Query M

```
Table.Distinct(
    Table.FromRecords({
        [a = "A", b = "a"],
        [a = "B", b = "a"],
        [a = "A", b = "b"]
    }),
    "b"
)
```

Salida

Power Query M

```
Table.FromRecords({
    [a = "A", b = "a"],
    [a = "A", b = "b"]
})
```

Contenido relacionado

[Criterios de ecuación](#)

Table.DuplicateColumn

Syntax

```
Table.DuplicateColumn(  
    table as table,  
    columnName as text,  
    newColumnName as text,  
    optional columnType as nullable type  
) as table
```

Acerca de

Duplique la columna denominada `columnName` a la tabla `table`. Los valores y el tipo de la columna `newColumnName` se copian de la columna `columnName`.

Example

Duplique la columna "a" a una columna denominada "columna copiada" en la tabla `([a = 1, b = 2], [a = 3, b = 4])`.

Uso

Power Query M

```
Table.DuplicateColumn(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4]  
    }),  
    "a",  
    "copied column"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [a = 1, b = 2, #"copied column" = 1],  
    [a = 3, b = 4, #"copied column" = 3]  
})
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 22/11/2025

Table.ExpandListColumn

16/09/2025

Sintaxis

```
Table.ExpandListColumn(table as table, column as text) as table
```

Acerca de

Dado un `table` donde `column` contiene una lista de valores, divide la lista en una fila para cada valor. Los valores de las otras columnas se duplican en cada nueva fila creada. Esta función también puede expandir tablas anidadas si las trata como listas de registros.

Ejemplo 1

Divida la columna de lista [Nombre].

Uso

Power Query M

```
Table.ExpandListColumn(  
    Table.FromRecords({[Name = {"Bob", "Jim", "Paul"}, Discount = .15]}),  
    "Name"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Name = "Bob", Discount = 0.15],  
    [Name = "Jim", Discount = 0.15],  
    [Name = "Paul", Discount = 0.15]  
})
```

Ejemplo 2

Divida la columna anidada de la tabla [Componentes].

Uso

Power Query M

```
Table.ExpandListColumn(  
    #table(  
        {"Part", "Components"},  
        {  
            {"Tool", #table({"Name", "Quantity"}, {"Thingamajig", 2}, {"Widget",  
3}})}  
        }  
    ),  
    "Components"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Part = "Tool", Components = [Name = "Thingamajig", Quantity = 2]],  
    [Part = "Tool", Components = [Name = "Widget", Quantity = 3]]  
})
```

Table.ExpandRecordColumn

Sintaxis

```
Table.ExpandRecordColumn(  
    table as table,  
    column as text,  
    fieldNames as list,  
    optional newColumnNames as nullable list  
) as table
```

Acercas de

Dado el `column` de registros de la entrada `table`, crea una tabla con una columna para cada campo del registro. Opcionalmente, `newColumnNames` se puede especificar para garantizar nombres únicos para las columnas de la nueva tabla.

- `table`: La tabla original con la columna de registro que se va a expandir.
- `column`: columna que se va a expandir.
- `fieldNames`: la lista de campos que se van a expandir en columnas de la tabla.
- `newColumnNames`: la lista de nombres de columna que se van a asignar a las nuevas columnas. Los nuevos nombres de columna no pueden duplicar ninguna columna de la nueva tabla.

Ejemplo

Expanda la columna `[a]` de la tabla `{{[a = [aa = 1, bb = 2, cc = 3], b = 2]}}` en 3 columnas `"aa"`, `"bb"` y `"cc"`.

Uso

Power Query M

```
Table.ExpandRecordColumn(  
    Table.FromRecords({  
        [  
            a = [aa = 1, bb = 2, cc = 3],  
            b = 2  
        ]  
    })),  
    "a",
```

```
    {"aa", "bb", "cc"}  
  )
```

Salida

```
Table.FromRecords({[aa = 1, bb = 2, cc = 3, b = 2]})
```

Last updated on 25/03/2026

Table.ExpandTableColumn

16/09/2025

Sintaxis

```
Table.ExpandTableColumn(  
    table as table,  
    column as text,  
    columnNames as list,  
    optional newColumnNames as nullable list  
) as table
```

Acerca de

Expande las tablas de `table[column]` en varias filas y columnas. `columnNames` se usa para seleccionar las columnas que se van a expandir desde la tabla interna. Especifique `newColumnNames` para evitar conflictos entre las columnas existentes y las columnas nuevas.

Ejemplo 1

Expanda columnas en `[a]` en la tabla `{{[t = {[a=1, b=2, c=3], [a=2,b=4,c=6]}, b = 2]}}` a 3 columnas `[t.a]`, `[t.b]` y `[t.c]`.

Uso

Power Query M

```
Table.ExpandTableColumn(  
    Table.FromRecords(  
        [  
            t = Table.FromRecords(  
                [a = 1, b = 2, c = 3],  
                [a = 2, b = 4, c = 6]  
            )),  
            b = 2  
        ]  
    )),  
    "t",  
    {"a", "b", "c"},  
    {"t.a", "t.b", "t.c"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [t.a = 1, t.b = 2, t.c = 3, b = 2],  
    [t.a = 2, t.b = 4, t.c = 6, b = 2]  
})
```

Table.FillDown

16/09/2025

Sintaxis

```
Table.FillDown(table as table, columns as list) as table
```

Acerca de

Devuelve una tabla de la `table` especificada donde el valor de una celda anterior se propaga a las celdas con valores NULL siguientes en el `columns` especificado.

Ejemplo 1

Devuelve una tabla con los valores nulos en la columna [Place] rellenados con el valor inmediatamente superior en la tabla.

Uso

Power Query M

```
Table.FillDown(  
    Table.FromRecords({  
        [Place = 1, Name = "Bob"],  
        [Place = null, Name = "John"],  
        [Place = 2, Name = "Brad"],  
        [Place = 3, Name = "Mark"],  
        [Place = null, Name = "Tom"],  
        [Place = null, Name = "Adam"]  
    }),  
    {"Place"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Place = 1, Name = "Bob"],  
    [Place = 1, Name = "John"],  
    [Place = 2, Name = "Brad"],  
    [Place = 3, Name = "Mark"],  
    [Place = 3, Name = "Tom"],
```

```
[Place = 3, Name = "Adam"]  
})
```

Table.FillUp

16/09/2025

Sintaxis

```
Table.FillUp(table as table, columns as list) as table
```

Acercas de

Devuelve una tabla del `table` especificado donde el valor de la celda siguiente se propaga a las celdas con valores NULL anteriores en el `columns` especificado.

Ejemplo 1

Devuelve una tabla con los valores NULL de la columna [Column2] rellenados con el valor debajo de ellos de la tabla.

Uso

Power Query M

```
Table.FillUp(  
    Table.FromRecords({  
        [Column1 = 1, Column2 = 2],  
        [Column1 = 3, Column2 = null],  
        [Column1 = 5, Column2 = 3]  
    }),  
    {"Column2"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = 1, Column2 = 2],  
    [Column1 = 3, Column2 = 3],  
    [Column1 = 5, Column2 = 3]  
})
```

Table.FilterWithDataTable

16/09/2025

Sintaxis

```
Table.FilterWithDataTable(table as table, dataTableIdentifier as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Table.FindText

16/09/2025

Sintaxis

```
Table.FindText(table as table, text as text) as table
```

Acerca de

Devuelve las filas de la tabla `table` que contienen el texto `text`. Si no se encuentra el texto, se devuelve una tabla vacía.

Ejemplo 1

Busque las filas de la tabla que contienen "Bob".

Uso

Power Query M

```
Table.FindText(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    ),  
    "Bob"  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Table.First

16/09/2025

Sintaxis

```
Table.First(table as table, optional default as any) as any
```

Acerca de

Devuelve la primera fila de **table** o un valor predeterminado opcional, **default**, si la tabla está vacía.

Ejemplo 1

Busque la primera fila de la tabla.

Uso

Power Query M

```
Table.First(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Ejemplo 2

Busque la primera fila de la tabla **{{}}** o devuelva [a = 0, b = 0] si está vacía.

Uso

Power Query M


```
Table.First(Table.FromRecords({}), [a = 0, b = 0])
```

Salida

```
[a = 0, b = 0]
```

Table.FirstN

16/09/2025

Sintaxis

```
Table.FirstN(table as table, countOrCondition as any) as table
```

Acerca de

Devuelve la(s) primera(s) fila(s) de la tabla `table`, en función del valor de `countOrCondition`.

- Si `countOrCondition` es un número, se devolverá esa cantidad de filas (empezando por la parte superior).
- Si `countOrCondition` es una condición, las filas que cumplan la condición se devolverán hasta que una fila no cumpla la condición.

Ejemplo 1

Buscar las primeras dos filas de la tabla.

Uso

Power Query M

```
Table.FirstN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    }),  
    2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
})
```

Ejemplo 2

Buscar las primeras filas en las que [a] > 0 en la tabla.

Uso

Power Query M

```
Table.FirstN(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4],  
        [a = -5, b = -6]  
    }),  
    each [a] > 0  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [a = 1, b = 2],  
    [a = 3, b = 4]  
})
```

Table.FirstValue

16/09/2025

Sintaxis

```
Table.FirstValue(table as table, optional default as any) as any
```

Acerca de

Devuelve la primera columna de la primera fila de la tabla `table` o un valor predeterminado especificado.

Table.FromColumns

Syntax

```
Table.FromColumns(lists as list, optional columns as any) as table
```

Acerca de

Crea una tabla de tipo `columns` a partir de una lista `lists` que contiene listas anidadas con los nombres y valores de columna. Si algunas columnas tienen más valores, otros, los valores que faltan se rellenarán con el valor predeterminado , "null", si las columnas son nullable.

Ejemplo 1

Devuelve una tabla de una lista de nombres de cliente en una lista. Cada valor del elemento de lista de clientes se convierte en un valor de fila y cada lista se convierte en una columna.

Uso

Power Query M

```
Table.FromColumns({  
    {1, "Bob", "123-4567"},  
    {2, "Jim", "987-6543"},  
    {3, "Paul", "543-7890"}  
})
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = 1, Column2 = 2, Column3 = 3],  
    [Column1 = "Bob", Column2 = "Jim", Column3 = "Paul"],  
    [Column1 = "123-4567", Column2 = "987-6543", Column3 = "543-7890"]  
})
```

Ejemplo 2

Cree una tabla a partir de una lista determinada de columnas y una lista de nombres de columna.

Uso

Power Query M

```
Table.FromColumns(  
    {  
        {1, "Bob", "123-4567"},  
        {2, "Jim", "987-6543"},  
        {3, "Paul", "543-7890"}  
    },  
    {"CustomerID", "Name", "Phone"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = 2, Phone = 3],  
    [CustomerID = "Bob", Name = "Jim", Phone = "Paul"],  
    [CustomerID = "123-4567", Name = "987-6543", Phone = "543-7890"]  
})
```

Ejemplo 3

Cree una tabla con un número diferente de columnas por fila. El valor de fila que falta es NULL.

Uso

Power Query M

```
Table.FromColumns(  
    {  
        {1, 2, 3},  
        {4, 5},  
        {6, 7, 8, 9}  
    },  
    {"column1", "column2", "column3"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [column1 = 1, column2 = 4, column3 = 6],  
    [column1 = 2, column2 = 5, column3 = 7],  
    [column1 = 3, column2 = null, column3 = 8],  
})
```

```
[column1 = null, column2 = null, column3 = 9]
})
```

Last updated on 22/11/2025

Table.FromList

Syntax

```
Table.FromList(  
    list as list,  
    optional splitter as nullable function,  
    optional columns as any,  
    optional default as any,  
    optional extraValues as nullable number  
) as table
```

Acerca de

Convierte una lista en `list` una tabla aplicando la [función de división](#) opcional, `splitter`, a cada elemento de la lista. De forma predeterminada, se supone que la lista es una lista de valores de texto divididos por comas. Opcional `columns` puede ser el número de columnas, una lista de columnas o `tableType`. Opcional `default` y `extraValues` también se puede especificar.

Ejemplo 1

Cree una tabla a partir de una lista con el divisor predeterminado.

Uso

Power Query M

```
Table.FromList(  
    {"a,apple", "b,ball", "c,cookie", "d,door"},  
    null,  
    {"Letter", "Example Word"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Letter = "a", #"Example Word" = "apple"],  
    [Letter = "b", #"Example Word" = "ball"],  
    [Letter = "c", #"Example Word" = "cookie"],  
    [Letter = "d", #"Example Word" = "door"]  
})
```


Ejemplo 2

Cree una tabla a partir de una lista mediante un divisor personalizado.

Uso

Power Query M

```
Table.FromList(  
    {"a,apple", "b,ball", "c,cookie", "d,door"},  
    Splitter.SplitByNothing(),  
    {"Letter and Example Word"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [#"Letter and Example Word" = "a,apple"],  
    [#"Letter and Example Word" = "b,ball"],  
    [#"Letter and Example Word" = "c,cookie"],  
    [#"Letter and Example Word" = "d,door"]  
})
```

Ejemplo 3

Cree una tabla a partir de la lista mediante el [Record.FieldValues](#) divisor.

Uso

Power Query M

```
Table.FromList(  
    {  
        [CustomerID = 1, Name = "Bob"],  
        [CustomerID = 2, Name = "Jim"]  
    },  
    Record.FieldValues,  
    {"CustomerID", "Name"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob"],
```

```
[CustomerID = 2, Name = "Jim"]  
})
```

Last updated on 22/11/2025


```

        Table.FromPartitions(
            "Day",
            {
                {3, #table({"Foo"}, {"Bar"})},
                {4, #table({"Foo"}, {"Bar"})}
            }
        )
    }
}
)

```

Salida

Power Query M

```

Table.FromRecords({
    [
        Foo = "Bar",
        Day = 1,
        Month = "Jan",
        Year = 1994
    ],
    [
        Foo = "Bar",
        Day = 2,
        Month = "Jan",
        Year = 1994
    ],
    [
        Foo = "Bar",
        Day = 3,
        Month = "Feb",
        Year = 1994
    ],
    [
        Foo = "Bar",
        Day = 4,
        Month = "Feb",
        Year = 1994
    ]
})

```

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Table.FromRecords

20/09/2025

Sintaxis

```
Table.FromRecords(  
    records as list,  
    optional columns as any,  
    optional missingField as nullable number  
) as table
```

Acerca de

Convierte una lista especificada de registros en una tabla.

- **records**: lista de registros que se van a convertir en una tabla.
- **columns**: (Opcional) Una lista de los nombres de columna de la tabla o el tipo de la tabla.
- **missingField**: (Opcional) Especifica cómo controlar los campos que faltan en una fila. Use uno de los siguientes valores:
 - **MissingField.Error**: los campos que faltan producen un error (valor predeterminado).
 - **MissingField.UseNull**: los campos que faltan se incluyen como **null** valores.

El uso **MissingField.Ignore** de en este parámetro produce un error.

Ejemplo 1

Crear una tabla a partir de registros con los nombres de los campos del registro como nombres de columna.

Uso

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
})
```

Salida

Power Query M

```
#table(type table[CustomerID = any, Name = any, Phone = any],
{
    {1, "Bob", "123-4567"},
    {2, "Jim", "987-6543"},
    {3, "Paul", "543-7890"}
})
```

Ejemplo 2

Crear una tabla a partir de registros que tengan columnas tipadas y seleccionar las columnas numéricas.

Uso

Power Query M

```
Table.ColumnsOfType(
    Table.FromRecords(
        {[CustomerID = 1, Name = "Bob"]},
        type table[CustomerID = Number.Type, Name = Text.Type]
    ),
    {type number}
)
```

Salida

```
{"CustomerID"}
```

Ejemplo 3

Cree una tabla que contenga el nombre, el segundo nombre, y el apellido de los clientes a partir de los registros especificados. Si falta alguno de los valores, reemplace el valor por `null`.

Uso

Power Query M

```
Table.FromRecords({
    [CustomerID = 1, FirstName = "Bob", MiddleInitial = "C", LastName = "Smith"],
    [CustomerID = 2, FirstName = "Sarah", LastName = "Jones"],
    [CustomerID = 3, FirstName = "Harry", MiddleInitial = "H"]
})
```

```
    },  
    type table [FirstName = nullable text, MiddleInitial = nullable text, LastName  
= nullable text],  
    MissingField.UseNull)
```

Salida

Power Query M

```
#table(type table[FirstName = text, MiddleInitial = text, LastName = text],  
{  
    {"Bob", "C", "Smith"},  
    {"Sarah", null, "Jones"},  
    {"Harry", "H", null}  
})
```

Contenido relacionado

[Campo que falta](#)

Table.FromRows

16/09/2025

Sintaxis

```
Table.FromRows(rows as list, optional columns as any) as table
```

Acerca de

Crea una tabla a partir de la lista `rows` donde cada elemento de la lista es una lista interna que contiene los valores de columna de una sola fila. Se podría proporcionar una lista opcional de nombres de columna, un tipo de tabla o una serie de columnas para `columns`.

Ejemplo 1

Devuelve una tabla con la columna [CustomerID] con valores {1, 2}, columna [Name] con valores {"Bob", "Jim"} y columna [Phone] con valores {"123-4567", "987-6543"}.

Uso

Power Query M

```
Table.FromRows(  
    {  
        {1, "Bob", "123-4567"},  
        {2, "Jim", "987-6543"}  
    },  
    {"CustomerID", "Name", "Phone"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
})
```

Ejemplo 2

Devuelve una tabla con columna [CustomerID] con valores {1, 2}, columna [Name] con valores {"Bob", "Jim"} y columna [Phone] con valores {"123-4567", "987-6543"}, donde [CustomerID] es tipo de número y [Nombre] y [Teléfono] son tipos de texto.

Uso

Power Query M

```
Table.FromRows(  
    {  
        {1, "Bob", "123-4567"},  
        {2, "Jim", "987-6543"}  
    },  
    type table [CustomerID = number, Name = text, Phone = text]  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
})
```

Table.FromValue

16/09/2025

Sintaxis

```
Table.FromValue(value as any, optional options as nullable record) as table
```

Acerca de

Crea una tabla con una columna que contiene el valor proporcionado o la lista de valores, `value`. Se puede especificar un parámetro de registro opcional, `options`, para controlar las siguientes opciones:

- `DefaultColumnName`: nombre de columna utilizado al construir una tabla a partir de una lista o un valor escalar.

Ejemplo 1

Cree una tabla a partir del valor 1.

Uso

Power Query M

```
Table.FromValue(1)
```

Salida

```
Table.FromRecords({[Value = 1]})
```

Ejemplo 2

Cree una tabla a partir de la lista.

Uso

Power Query M

```
Table.FromValue({1, "Bob", "123-4567"})
```

Salida

Power Query M

```
Table.FromRecords({  
    [Value = 1],  
    [Value = "Bob"],  
    [Value = "123-4567"]  
})
```

Ejemplo 3

Cree una tabla a partir del valor 1, con un nombre de columna personalizado.

Uso

Power Query M

```
Table.FromValue(1, [DefaultColumnName = "MyValue"])
```

Salida

```
Table.FromRecords({[MyValue = 1]})
```

Table.FuzzyGroup

Syntax

```
Table.FuzzyGroup(  
    table as table,  
    key as any,  
    aggregatedColumns as list,  
    optional options as nullable record  
) as table
```

Acerca de

Agrupar las filas de `table` por valores coincidentes fuzzily en la columna especificada, `key`, para cada fila. Para cada grupo, se construye un registro que contiene las columnas de clave (y sus valores) junto con las columnas agregadas especificadas por `aggregatedColumns`. Esta función no puede garantizar que devuelva un orden fijo de filas.

Se puede incluir un conjunto opcional de para especificar cómo comparar las columnas de `options` clave. Entre las opciones se incluyen:

- `Culture`: permite agrupar registros en función de reglas específicas de la referencia cultural. Puede ser cualquier nombre de referencia cultural válido. Por ejemplo, una opción `Culture` de "ja-JP" agrupa registros basados en la referencia cultural japonesa. El valor predeterminado es "", que se agrupa en función de la referencia cultural de inglés invariable.
- `IgnoreCase`: valor lógico (true/false) que permite la agrupación de claves sin distinción entre mayúsculas y minúsculas. Por ejemplo, cuando es true, "Uvas" se agrupa con "uvas". El valor predeterminado es true.
- `IgnoreSpace`: valor lógico (true/false) que permite combinar elementos de texto para buscar grupos. Por ejemplo, cuando es true, "Gra pes" se agrupa con "Uvas". El valor predeterminado es true.
- `SimilarityColumnName`: un nombre para la columna que muestra la similitud entre un valor de entrada y el valor representativo de esa entrada. El valor predeterminado es NULL, en cuyo caso no se agregará una nueva columna para similitudes.
- `Threshold`: un número entre 0,00 y 1,00 que especifica la puntuación de similitud en la que se agruparán dos valores. Por ejemplo, "Uvas" y "Graes" (falta la "p") solo se agrupan si esta opción está establecida en menos de 0,90. Un umbral de 1,00 solo permite coincidencias exactas. (Tenga en cuenta que una "coincidencia exacta" aproximada podría

omitir diferencias como mayúsculas y minúsculas, orden de palabras y puntuación). El valor predeterminado es 0,80.

- **TransformationTable**: tabla que permite agrupar registros en función de asignaciones de valores personalizadas. Debe contener columnas "From" y "To". Por ejemplo, "Uvas" se agrupa con "Raisins" si se proporciona una tabla de transformación con la columna "From" que contiene "Uvas" y la columna "To" que contiene "Raisins". Tenga en cuenta que la transformación se aplicará a todas las apariciones del texto de la tabla de transformación. Con la tabla de transformación anterior, "Las uvas son dulces" también se agruparán con "Las pasas son dulces".

Ejemplo 1

Agrupe la tabla agregando una columna de agregado [Count] que contiene el número de empleados en cada ubicación (`each Table.RowCount(_)`).

Uso

Power Query M

```
Table.FuzzyGroup(  
    Table.FromRecords(  
        {  
            [EmployeeID = 1, Location = "Seattle"],  
            [EmployeeID = 2, Location = "seattl"],  
            [EmployeeID = 3, Location = "Vancouver"],  
            [EmployeeID = 4, Location = "Seattle"],  
            [EmployeeID = 5, Location = "vancouver"],  
            [EmployeeID = 6, Location = "Seattle"],  
            [EmployeeID = 7, Location = "Vancouver"]  
        },  
        type table [EmployeeID = nullable number, Location = nullable text]  
    ),  
    "Location",  
    {"Count", each Table.RowCount(_)},  
    [IgnoreCase = true, IgnoreSpace = true]  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Location = "Seattle", Count = 4],  
    [Location = "Vancouver", Count = 3]  
})
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Table.FuzzyJoin

Syntax

```
Table.FuzzyJoin(  
    table1 as table,  
    key1 as any,  
    table2 as table,  
    key2 as any,  
    optional joinKind as nullable number,  
    optional joinOptions as nullable record  
) as table
```

Acercas de

Combina las filas de `table1` con las filas de `table2` basadas en una coincidencia aproximada de los valores de las columnas de clave seleccionadas por `key1` (para `table1`) y `key2` (para `table2`).

La coincidencia aproximada es una comparación basada en la similitud del texto en lugar de la igualdad de texto.

De forma predeterminada, se realiza una combinación interna, pero se puede incluir un opcional `joinKind` para especificar el tipo de combinación. Entre las opciones se incluyen:

- [JoinKind.Inner](#)
- [JoinKind.LeftOuter](#)
- [JoinKind.RightOuter](#)
- [JoinKind.FullOuter](#)
- [JoinKind.LeftAnti](#)
- [JoinKind.RightAnti](#)
- [JoinKind.LeftSemi](#)
- [JoinKind.RightSemi](#)

Se puede incluir un conjunto opcional de para especificar cómo comparar las columnas de `joinOptions` clave. Entre las opciones se incluyen:

- `ConcurrentRequests`: número comprendido entre 1 y 8 que especifica el número de subprocesos paralelos que se van a usar para la coincidencia aproximada. El valor predeterminado es 1.
- `Culture`: permite la coincidencia de registros en función de las reglas específicas de la referencia cultural. Puede ser cualquier nombre de referencia cultural válido. Por ejemplo,

una opción Culture de "ja-JP" coincide con los registros basados en la referencia cultural japonesa. El valor predeterminado es "", que coincide en función de la referencia cultural de inglés invariable.

- **IgnoreCase**: valor lógico (true/false) que permite la coincidencia de claves sin distinción entre mayúsculas y minúsculas. Por ejemplo, cuando es true, "Uvas" coincide con "uvas". El valor predeterminado es true.
- **IgnoreSpace**: valor lógico (true/false) que permite combinar elementos de texto con el fin de buscar coincidencias. Por ejemplo, cuando es true, "Gra pes" coincide con "Uvas". El valor predeterminado es true.
- **NumberOfMatches**: número entero que especifica el número máximo de filas coincidentes que se pueden devolver para cada fila de entrada. Por ejemplo, un valor de 1 devolverá como máximo una fila coincidente para cada fila de entrada. Si no se proporciona esta opción, se devuelven todas las filas coincidentes.
- **SimilarityColumnName**: un nombre para la columna que muestra la similitud entre un valor de entrada y el valor representativo de esa entrada. El valor predeterminado es NULL, en cuyo caso no se agregará una nueva columna para similitudes.
- **Threshold**: un número entre 0,00 y 1,00 que especifica la puntuación de similitud con la que se coincidirán dos valores. Por ejemplo, "Uvas" y "Graes" (falta la "p") solo se coinciden si esta opción está establecida en menos de 0,90. Un umbral de 1,00 solo permite coincidencias exactas. (Tenga en cuenta que una "coincidencia exacta" aproximada podría omitir diferencias como mayúsculas y minúsculas, orden de palabras y puntuación). El valor predeterminado es 0,80.
- **TransformationTable**: tabla que permite la coincidencia de registros en función de asignaciones de valores personalizadas. Debe contener columnas "From" y "To". Por ejemplo, "Uvas" coincide con "Raisins" si se proporciona una tabla de transformación con la columna "From" que contiene "Uvas" y la columna "To" que contiene "Raisins". Tenga en cuenta que la transformación se aplicará a todas las apariciones del texto de la tabla de transformación. Con la tabla de transformación anterior, "Las uvas son dulces" también coincidirán con "Las pasas son dulces".

Ejemplo 1

Combinación aproximada izquierda de dos tablas basadas en [FirstName]

Uso

Power Query M

```
Table.FuzzyJoin(  
    Table.FromRecords(  
        {  
            [CustomerID = 1, FirstName1 = "Bob", Phone = "555-1234"],
```

```

        [CustomerID = 2, FirstName1 = "Robert", Phone = "555-4567"]
    },
    type table [CustomerID = nullable number, FirstName1 = nullable text, Phone
= nullable text]
    ),
    {"FirstName1"},
    Table.FromRecords(
        {
            [CustomerStateID = 1, FirstName2 = "Bob", State = "TX"],
            [CustomerStateID = 2, FirstName2 = "bOB", State = "CA"]
        },
        type table [CustomerStateID = nullable number, FirstName2 = nullable text,
State = nullable text]
    ),
    {"FirstName2"},
    JoinKind.LeftOuter,
    [IgnoreCase = true, IgnoreSpace = false]
)

```

Salida

Power Query M

```

Table.FromRecords({
    [
        CustomerID = 1,
        FirstName1 = "Bob",
        Phone = "555-1234",
        CustomerStateID = 1,
        FirstName2 = "Bob",
        State = "TX"
    ],
    [
        CustomerID = 1,
        FirstName1 = "Bob",
        Phone = "555-1234",
        CustomerStateID = 2,
        FirstName2 = "bOB",
        State = "CA"
    ],
    [
        CustomerID = 2,
        FirstName1 = "Robert",
        Phone = "555-4567",
        CustomerStateID = null,
        FirstName2 = null,
        State = null
    ]
})

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
-

Last updated on 22/11/2025

Table.FuzzyNestedJoin

Syntax

```
Table.FuzzyNestedJoin(  
    table1 as table,  
    key1 as any,  
    table2 as table,  
    key2 as any,  
    newColumnName as text,  
    optional joinKind as nullable number,  
    optional joinOptions as nullable record  
) as table
```

Acerca de

Combina las filas de `table1` con las filas de `table2` basadas en una coincidencia aproximada de los valores de las columnas de clave seleccionadas por `key1` (para `table1`) y `key2` (para `table2`). Los resultados se devuelven en una nueva columna denominada `newColumnName`.

La coincidencia aproximada es una comparación basada en la similitud del texto en lugar de la igualdad de texto.

`joinKind` Opcional especifica el tipo de combinación que se va a realizar. De forma predeterminada, se realiza una combinación externa izquierda si no se especifica `joinKind`.

Entre las opciones se incluyen:

- [JoinKind.Inner](#)
- [JoinKind.LeftOuter](#)
- [JoinKind.RightOuter](#)
- [JoinKind.FullOuter](#)
- [JoinKind.LeftAnti](#)
- [JoinKind.RightAnti](#)
- [JoinKind.LeftSemi](#)
- [JoinKind.RightSemi](#)

Se puede incluir un conjunto opcional de para especificar cómo comparar las columnas de `joinOptions` clave. Entre las opciones se incluyen:

- `ConcurrentRequests`: número comprendido entre 1 y 8 que especifica el número de subprocesos paralelos que se van a usar para la coincidencia aproximada. El valor predeterminado es 1.

- **Culture**: permite la coincidencia de registros en función de las reglas específicas de la referencia cultural. Puede ser cualquier nombre de referencia cultural válido. Por ejemplo, una opción Culture de "ja-JP" coincide con los registros basados en la referencia cultural japonesa. El valor predeterminado es "", que coincide en función de la referencia cultural de inglés invariable.
- **IgnoreCase**: valor lógico (true/false) que permite la coincidencia de claves sin distinción entre mayúsculas y minúsculas. Por ejemplo, cuando es true, "Uvas" coincide con "uvas". El valor predeterminado es true.
- **IgnoreSpace**: valor lógico (true/false) que permite combinar elementos de texto con el fin de buscar coincidencias. Por ejemplo, cuando es true, "Gra pes" coincide con "Uvas". El valor predeterminado es true.
- **NumberOfMatches**: número entero que especifica el número máximo de filas coincidentes que se pueden devolver para cada fila de entrada. Por ejemplo, un valor de 1 devolverá como máximo una fila coincidente para cada fila de entrada. Si no se proporciona esta opción, se devuelven todas las filas coincidentes.
- **SimilarityColumnName**: un nombre para la columna que muestra la similitud entre un valor de entrada y el valor representativo de esa entrada. El valor predeterminado es NULL, en cuyo caso no se agregará una nueva columna para similitudes.
- **Threshold**: un número entre 0,00 y 1,00 que especifica la puntuación de similitud con la que se coincidirán dos valores. Por ejemplo, "Uvas" y "Graes" (falta la "p") solo se coinciden si esta opción está establecida en menos de 0,90. Un umbral de 1,00 solo permite coincidencias exactas. (Tenga en cuenta que una "coincidencia exacta" aproximada podría omitir diferencias como mayúsculas y minúsculas, orden de palabras y puntuación). El valor predeterminado es 0,80.
- **TransformationTable**: tabla que permite la coincidencia de registros en función de asignaciones de valores personalizadas. Debe contener columnas "From" y "To". Por ejemplo, "Uvas" coincide con "Raisins" si se proporciona una tabla de transformación con la columna "From" que contiene "Uvas" y la columna "To" que contiene "Raisins". Tenga en cuenta que la transformación se aplicará a todas las apariciones del texto de la tabla de transformación. Con la tabla de transformación anterior, "Las uvas son dulces" también coincidirán con "Las pasas son dulces".

Ejemplo 1

Combinación aproximada izquierda de dos tablas basadas en [FirstName]

Uso

```

Table.FuzzyNestedJoin(
    Table.FromRecords(
        {
            [CustomerID = 1, FirstName1 = "Bob", Phone = "555-1234"],
            [CustomerID = 2, FirstName1 = "Robert", Phone = "555-4567"]
        },
        type table [CustomerID = nullable number, FirstName1 = nullable text, Phone
= nullable text]
    ),
    {"FirstName1"},
    Table.FromRecords(
        {
            [CustomerStateID = 1, FirstName2 = "Bob", State = "TX"],
            [CustomerStateID = 2, FirstName2 = "bOB", State = "CA"]
        },
        type table [CustomerStateID = nullable number, FirstName2 = nullable text,
State = nullable text]
    ),
    {"FirstName2"},
    "NestedTable",
    JoinKind.LeftOuter,
    [IgnoreCase = true, IgnoreSpace = false]
)

```

Salida

Power Query M

```

Table.FromRecords({
    [
        CustomerID = 1,
        FirstName1 = "Bob",
        Phone = "555-1234",
        NestedTable = Table.FromRecords({
            [
                CustomerStateID = 1,
                FirstName2 = "Bob",
                State = "TX"
            ],
            [
                CustomerStateID = 2,
                FirstName2 = "bOB",
                State = "CA"
            ]
        })
    ],
    [
        CustomerID = 2,
        FirstName1 = "Robert",
        Phone = "555-4567",
        NestedTable = Table.FromRecords({})
    ]
})

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Table.Group

Sintaxis

```
Table.Group(  
    table as table,  
    key as any,  
    aggregatedColumns as list,  
    optional groupKind as nullable number,  
    optional comparer as nullable function  
) as table
```

Acercas de

Agrupar las filas de `table` por las columnas de clave definidas por `key`. `key` puede ser un nombre de columna único o una lista de nombres de columna. Para cada grupo, se crea un registro que contiene las columnas de clave (y sus valores) y cualquier columna agregada especificada mediante `aggregatedColumns`. Opcionalmente, también se pueden especificar `groupKind` y `comparer`.

Si los datos ya están ordenados por las columnas de clave, se puede proporcionar un `groupKind` de `GroupKind.Local`. Esto puede mejorar el rendimiento de la agrupación en determinados casos, ya que se supone que todas las filas con un conjunto determinado de valores de clave son contiguas.

Al pasar un `comparer`, tenga en cuenta que si trata las claves diferentes como iguales, se puede colocar una fila en un grupo cuyas claves difieren de las suyas propias.

Esta función no garantiza el orden de las filas que devuelve.

Ejemplo

Agrupe la tabla agregando una columna de agregado [total] que contiene la suma de precios ("cada List.Sum([precio])").

Uso


```
Table.Group(
    Table.FromRecords({
        [CustomerID = 1, price = 20],
        [CustomerID = 2, price = 10],
        [CustomerID = 2, price = 20],
        [CustomerID = 1, price = 10],
        [CustomerID = 3, price = 20],
        [CustomerID = 3, price = 5]
    }),
    "CustomerID",
    {"total", each List.Sum([price])}
)
```

Salida

Power Query M

```
Table.FromRecords(
    {
        [CustomerID = 1, total = 30],
        [CustomerID = 2, total = 30],
        [CustomerID = 3, total = 25]
    },
    {"CustomerID", "total"}
)
```

Contenido relacionado

[Funciones de comparador](#)

Last updated on 02/04/2026

Table.HasColumns

16/09/2025

Sintaxis

```
Table.HasColumns(table as table, columns as any) as logical
```

Acercas de

Indica si el `table` contiene las columnas especificadas `columns`. Devuelve `true` si la tabla contiene la(s) columna(s); `false`, de lo contrario.

Ejemplo 1

Determine si la tabla tiene la columna [Name].

Uso

Power Query M

```
Table.HasColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Name"  
)
```

Salida

```
true
```

Ejemplo 2

Busque si la tabla tiene la columna [Name] y [PhoneNumber].

Uso

Power Query M

```
Table.HasColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    {"Name", "PhoneNumber"}  
)
```

Salida

false

Table.InsertRows

16/09/2025

Sintaxis

```
Table.InsertRows(  
    table as table,  
    offset as number,  
    rows as list  
) as table
```

Acerca de

Devuelve una tabla con la lista de filas, `rows`, insertada en el `table` en la posición especificada, `offset`. Cada columna de la fila que se va a insertar debe coincidir con los tipos de columna de la tabla.

Ejemplo 1

Inserte la fila en la tabla en la posición 1.

Uso

Power Query M

```
Table.InsertRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    }),  
    1,  
    {[CustomerID = 3, Name = "Paul", Phone = "543-7890"]}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
})
```

```
[CustomerID = 2, Name = "Jim", Phone = "987-6543"]
})
```

Ejemplo 2

Inserte dos filas en la tabla en la posición 1.

Uso

Power Query M

```
Table.InsertRows(
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),
    1,
    {
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]
    }
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]
})
```

Table.IsDistinct

Sintaxis

```
Table.IsDistinct(table as table, optional comparisonCriteria as any) as logical
```

Acercas de

Indica si `table` contiene solo filas distintas (sin duplicados). Devuelve `true` si las filas son diferentes; de lo contrario, devuelve `false`. Un parámetro opcional, `comparisonCriteria`, especifica en qué columnas de la tabla se comprueba si hay duplicación. Si no se especifica `comparisonCriteria`, se comprueban todas las columnas.

Ejemplo 1

Determinar si la tabla es distinta.

Uso

Power Query M

```
Table.IsDistinct(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    )  
)
```

Salida

```
true
```

Ejemplo 2

Determinar si la tabla es distinta en la columna.

Uso

Power Query M

```
Table.IsDistinct(  
  Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 5, Name = "Bob", Phone = "232-1550"]  
  }),  
  "Name"  
)
```

Salida

false

Contenido relacionado

[Criterios de comparación](#)

Last updated on 16/09/2025

Table.IsEmpty

16/09/2025

Sintaxis

```
Table.IsEmpty(table as table) as logical
```

Acerca de

Indica si el `table` contiene filas. Devuelve `true` si no hay filas (es decir, la tabla está vacía), `false` de lo contrario.

Ejemplo 1

Determine si la tabla está vacía.

Uso

Power Query M

```
Table.IsEmpty(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    )  
)
```

Salida

`false`

Ejemplo 2

Determine si la tabla `({})` está vacía.

Uso

Power Query M


```
Table.IsEmpty(Table.FromRecords({}))
```

Salida

```
true
```

Table.Join

Syntax

```
Table.Join(  
    table1 as table,  
    key1 as any,  
    table2 as table,  
    key2 as any,  
    optional joinKind as nullable number,  
    optional joinAlgorithm as nullable number,  
    optional keyEqualityComparers as nullable list  
) as table
```

Acerca de

Combina las filas de `table1` con las de `table2` basándose en la igualdad de los valores de las columnas de clave que selecciona `key1` (para `table1`) y `key2` (para `table2`).

De forma predeterminada, se realiza una combinación interna, pero se puede incluir un opcional `joinKind` para especificar el tipo de combinación. Entre las opciones se incluyen:

- [JoinKind.Inner](#)
- [JoinKind.LeftOuter](#)
- [JoinKind.RightOuter](#)
- [JoinKind.FullOuter](#)
- [JoinKind.LeftAnti](#)
- [JoinKind.RightAnti](#)
- [JoinKind.LeftSemi](#)
- [JoinKind.RightSemi](#)

Se puede incluir un conjunto opcional de para especificar cómo comparar las columnas de `keyEqualityComparers` clave. Este parámetro está pensado actualmente solo para uso interno.

Ejemplo 1

Una dos tablas con una única columna de clave.

Uso

```

Table.Join(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    "CustomerID",
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25]
    }),
    "CustomerID"
)

```

Salida

Power Query M

```

Table.FromRecords({
    [CustomerID = 1, Name = "Bob", Phone = "123-4567", OrderID = 1, Item = "Fishing rod", Price = 100],
    [CustomerID = 1, Name = "Bob", Phone = "123-4567", OrderID = 2, Item = "1 lb. worms", Price = 5],
    [CustomerID = 2, Name = "Jim", Phone = "987-6543", OrderID = 3, Item = "Fishing net", Price = 25],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890", OrderID = 4, Item = "Fish tazer", Price = 200],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890", OrderID = 5, Item = "Band-aids", Price = 2],
    [CustomerID = 1, Name = "Bob", Phone = "123-4567", OrderID = 6, Item = "Tackle box", Price = 20]
})

```

Ejemplo 2

Combine dos tablas que tengan nombres de columna en conflicto mediante varias columnas de clave.

Uso

Power Query M

```

let
    customers = Table.FromRecords({

```

```

        [TenantID = 1, CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [TenantID = 1, CustomerID = 2, Name = "Jim", Phone = "987-6543"]
    )),
    orders = Table.FromRecords({
        [TenantID = 1, OrderID = 1, CustomerID = 1, Name = "Fishing rod", Price =
100.0],
        [TenantID = 1, OrderID = 2, CustomerID = 1, Name = "1 lb. worms", Price =
5.0],
        [TenantID = 1, OrderID = 3, CustomerID = 2, Name = "Fishing net", Price =
25.0]
    })
in
    Table.Join(
        customers,
        {"TenantID", "CustomerID"},
        Table.PrefixColumns(orders, "Order"),
        {"Order.TenantID", "Order.CustomerID"}
    )

```

Power Query M

```

Table.FromRecords({
    [TenantID = 1, CustomerID = 1, Name = "Bob", Phone = "123-4567", Order.TenantID
= 1, Order.OrderID = 1, Order.CustomerID = 1, Order.Name = "Fishing rod",
Order.Price = 100],
    [TenantID = 1, CustomerID = 1, Name = "Bob", Phone = "123-4567", Order.TenantID
= 1, Order.OrderID = 2, Order.CustomerID = 1, Order.Name = "1 lb. worms",
Order.Price = 5],
    [TenantID = 1, CustomerID = 2, Name = "Jim", Phone = "987-6543", Order.TenantID
= 1, Order.OrderID = 3, Order.CustomerID = 2, Order.Name = "Fishing net",
Order.Price = 25]
})

```

Table.Keys

Sintaxis

```
Table.Keys(table as table) as list
```

Acerca de

Devuelve las claves de la tabla especificada.

Ejemplo

Obtenga la lista de claves de una tabla.

Uso

Power Query M

```
let
    table = Table.FromRecords({
        [Id = 1, Name = "Hello There"],
        [Id = 2, Name = "Good Bye"]
    }),
    tableWithKeys = Table.AddKey(table, {"Id"}, true),
    keys = Table.Keys(tableWithKeys)
in
    keys
```

Salida

```
{[Columns = {"Id"}, Primary = true]}
```

Last updated on 25/03/2026

Table.Last

16/09/2025

Sintaxis

```
Table.Last(table as table, optional default as any) as any
```

Acerca de

Devuelve la última fila de `table` o un valor predeterminado opcional, `default`, si la tabla está vacía.

Ejemplo 1

Busque la última fila de la tabla.

Uso

Power Query M

```
Table.Last(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Salida

```
[CustomerID = 3, Name = "Paul", Phone = "543-7890"]
```

Ejemplo 2

Busque la última fila de la tabla `{}` o devuelva `[a = 0, b = 0]` si está vacía.

Uso

Power Query M

```
Table.Last(Table.FromRecords({})), [a = 0, b = 0])
```

Salida

```
[a = 0, b = 0]
```

Table.LastN

Syntax

```
Table.LastN(table as table, countOrCondition as any) as table
```

Acerca de

Devuelve las últimas filas de la tabla, **table** en función del valor de **countOrCondition**:

- Si **countOrCondition** es un número, se devolverán muchas filas a partir de la posición (final - **countOrCondition**).
- Si **countOrCondition** es una condición, las filas que cumplen la condición se devolverán en posición ascendente hasta que una fila no cumpla la condición.

Ejemplo 1

Busque las dos últimas filas de la tabla.

Uso

Power Query M

```
Table.LastN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    }),  
    2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
})
```

Ejemplo 2

Busque las últimas filas donde $[a] > 0$ de la tabla.

Uso

Power Query M

```
Table.LastN(  
    Table.FromRecords({  
        [a = -1, b = -2],  
        [a = 3, b = 4],  
        [a = 5, b = 6]  
    }),  
    each _ [a] > 0
```

Salida

Power Query M

```
Table.FromRecords({  
    [a = 3, b = 4],  
    [a = 5, b = 6]  
})
```

Last updated on 22/11/2025

Table.MatchesAllRows

16/09/2025

Sintaxis

```
Table.MatchesAllRows(table as table, condition as function) as logical
```

Acercas de

Indica si todas las filas de la `table` coinciden con el `condition` especificado. Devuelve `true` si todas las filas coinciden, `false` de lo contrario.

Ejemplo 1

Determine si todos los valores de fila de la columna [a] están incluso en la tabla.

Uso

Power Query M

```
Table.MatchesAllRows(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }),  
    each Number.Mod([a], 2) = 0  
)
```

Salida

true

Ejemplo 2

Busque si todos los valores de fila son [a = 1, b = 2], en la tabla `([{a = 1, b = 2}, [a = 3, b = 4]])`.

Uso

Power Query M

```
Table.MatchesAllRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = -3, b = 4]  
    }),  
    each _ = [a = 1, b = 2]  
)
```

Salida

false

Table.MatchesAnyRows

16/09/2025

Sintaxis

```
Table.MatchesAnyRows(table as table, condition as function) as logical
```

Acercas de

Indica si alguna de las filas en `table` coincide con el `condition` dado. Devuelve `true` si alguna de las filas coincide; de lo contrario, `false`.

Ejemplo 1

Determine si alguno de los valores de fila de la columna [a] está incluso en la tabla `{{[a = 2, b = 4], [a = 6, b = 8]}}`.

Uso

Power Query M

```
Table.MatchesAnyRows(  
    Table.FromRecords(  
        [a = 1, b = 4],  
        [a = 3, b = 8]  
    ),  
    each Number.Mod([a], 2) = 0  
)
```

Salida

`false`

Ejemplo 2

Determine si alguno de los valores de fila es [a = 1, b = 2], en la tabla `{{[a = 1, b = 2], [a = 3, b = 4]}}`.

Uso

Power Query M

```
Table.MatchesAnyRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = -3, b = 4]  
    }),  
    each _ = [a = 1, b = 2]  
)
```

Salida

true

Table.Max

16/09/2025

Sintaxis

```
Table.Max(  
    table as table,  
    comparisonCriteria as any,  
    optional default as any  
) as any
```

Acerca de

Devuelve la fila más grande en `table`, dado el elemento `comparisonCriteria`. Si la tabla está vacía, se devuelve el valor de `default` opcional.

Ejemplo 1

Buscar la fila con el valor mayor de la columna [a] en la tabla `([a = 2, b = 4], [a = 6, b = 8])`.

Uso

Power Query M

```
Table.Max(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }),  
    "a"  
)
```

Salida

```
[a = 6, b = 8]
```

Ejemplo 2

Buscar la fila con el valor mayor de la columna [a] en la tabla ({}). Devuelve -1 si está vacío.

Uso

Power Query M

```
Table.Max(#table({"a"}, {}), "a", -1)
```

Salida

-1

Contenido relacionado

[Criterios de comparación](#)

Table.MaxN

16/09/2025

Sintaxis

```
Table.MaxN(  
    table as table,  
    comparisonCriteria as any,  
    countOrCondition as any  
) as table
```

Acerca de

Devuelve las filas más grandes en `table`, dado el elemento `comparisonCriteria`. Una vez ordenadas las filas, se debe especificar el parámetro `countOrCondition` para filtrar aún más el resultado. Tenga en cuenta que el algoritmo de ordenación no puede garantizar un resultado ordenado fijo. El parámetro `countOrCondition` puede adoptar varias formas:

- Si se especifica un número, se devuelve una lista de hasta `countOrCondition` elementos en orden ascendente.
- Si se especifica una condición, se devuelve una lista de elementos que cumplen inicialmente la condición. Si un elemento no cumple la condición, no se tienen en cuenta más elementos.

Ejemplo 1

Buscar la fila con el valor más grande de la columna [a], con la condición [a] > 0, en la tabla. Las filas se ordenan antes de aplicarse el filtro.

Uso

Power Query M

```
Table.MaxN(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 0, b = 0],  
        [a = 6, b = 2]  
    }),  
    "a",
```



```
    each [a] > 0
)
```

Salida

Power Query M

```
Table.FromRecords({
    [a = 6, b = 2],
    [a = 2, b = 4]
})
```

Ejemplo 2

Buscar la fila con el valor más grande de la columna [a], con la condición [a] > 0, en la tabla. Las filas se ordenan antes de aplicarse el filtro.

Uso

Power Query M

```
Table.MaxN(
    Table.FromRecords({
        [a = 2, b = 4],
        [a = 8, b = 0],
        [a = 6, b = 2]
    }),
    "a",
    each [b] > 0
)
```

Salida

```
Table.FromRecords({})
```

Contenido relacionado

[Criterios de comparación](#)

Table.Min

16/09/2025

Sintaxis

```
Table.Min(  
    table as table,  
    comparisonCriteria as any,  
    optional default as any  
) as any
```

Acerca de

Devuelve la fila más pequeña en `table`, dado el elemento `comparisonCriteria`. Si la tabla está vacía, se devuelve el valor de `default` opcional.

Ejemplo 1

Buscar la fila con el valor menor de la columna [a] en la tabla.

Uso

Power Query M

```
Table.Min(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    }),  
    "a"  
)
```

Salida

```
[a = 2, b = 4]
```

Ejemplo 2

Buscar la fila con el valor menor de la columna [a] en la tabla. Devuelve -1 si está vacío.

Uso

Power Query M

```
Table.Min(#table({"a"}, {}), "a", -1)
```

Salida

-1

Contenido relacionado

[Criterios de comparación](#)

Table.MinN

16/09/2025

Sintaxis

```
Table.MinN(  
    table as table,  
    comparisonCriteria as any,  
    countOrCondition as any  
) as table
```

Acerca de

Devuelve las filas más pequeñas en `table`, dado el elemento `comparisonCriteria`. Una vez ordenadas las filas, se debe especificar el parámetro `countOrCondition` para filtrar aún más el resultado. Tenga en cuenta que el algoritmo de ordenación no puede garantizar un resultado ordenado fijo. El parámetro `countOrCondition` puede adoptar varias formas:

- Si se especifica un número, se devuelve una lista de hasta `countOrCondition` elementos en orden ascendente.
- Si se especifica una condición, se devuelve una lista de elementos que cumplen inicialmente la condición. Si un elemento no cumple la condición, no se tienen en cuenta más elementos.

Ejemplo 1

Buscar la fila con el valor más pequeño de la columna [a], con la condición [a] < 3, en la tabla. Las filas se ordenan antes de aplicarse el filtro.

Uso

Power Query M

```
Table.MinN(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 0, b = 0],  
        [a = 6, b = 4]  
    }),  
    "a",
```

```
    each [a] < 3
)
```

Salida

Power Query M

```
Table.FromRecords({
    [a = 0, b = 0],
    [a = 2, b = 4]
})
```

Ejemplo 2

Buscar la fila con el valor más pequeño de la columna [a], con la condición [b] < 0, en la tabla. Las filas se ordenan antes de aplicarse el filtro.

Uso

Power Query M

```
Table.MinN(
    Table.FromRecords({
        [a = 2, b = 4],
        [a = 8, b = 0],
        [a = 6, b = 2]
    }),
    "a",
    each [b] < 0
)
```

Salida

```
Table.FromRecords({})
```

Contenido relacionado

[Criterios de comparación](#)

Table.NestedJoin

Sintaxis

```
Table.NestedJoin(  
    table1 as table,  
    key1 as any,  
    table2 as any,  
    key2 as any,  
    newColumnName as text,  
    optional joinKind as nullable number,  
    optional keyEqualityComparers as nullable list  
) as table
```

Acerca de

Combina las filas de `table1` con las de `table2` basándose en la igualdad de los valores de las columnas de clave que selecciona `key1` (para `table1`) y `key2` (para `table2`). Los resultados se introducen en la columna denominada `newColumnName`.

El elemento `joinKind` opcional especifica el tipo de combinación que se va a realizar. De forma predeterminada, si no se especifica un elemento `joinKind`, se realiza una combinación externa izquierda.

Se puede incluir un conjunto opcional de `keyEqualityComparers` para especificar cómo comparar las columnas de clave. Esta característica solo está pensada para uso interno.

Ejemplo

Una dos tablas con una única columna de clave.

Uso

Power Query M

```
Table.NestedJoin(  
    Table.FromRecords(  
        [CustomerToCall = 1],  
        [CustomerToCall = 3]  
    )),  
    {"CustomerToCall"},  
    Table.FromRecords({
```

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
[CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
[CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
[CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
}),  
{"CustomerID"},  
"CustomerDetails"  
)
```

Salida

Power Query M

```
Table.FromRecords(  
    [CustomerToCall = 1, CustomerDetails = Table.FromRecords({[CustomerID = 1, Name  
= "Bob", Phone = "123-4567"]})],  
    [CustomerToCall = 3, CustomerDetails = Table.FromRecords({[CustomerID = 3, Name  
= "Paul", Phone = "543-7890"]})]  
)
```

Contenido relacionado

[Tipo de combinación](#)

Last updated on 04/04/2026

Table.Partition

Syntax

```
Table.Partition(  
    table as table,  
    column as text,  
    groups as number,  
    hash as function  
) as list
```

Acerca de

Divide en `table` una lista de número de `groups` tablas, en función del valor de `column` y una `hash` función. La `hash` función se aplica al valor de la `column` fila para obtener un valor hash para la fila. El módulo `groups` de valores hash determina en qué tablas devueltas se colocará la fila.

- `table`: la tabla que se va a crear particiones.
- `column`: columna en la que se va a aplicar un hash para determinar en qué tabla devuelta se encuentra la fila.
- `groups`: número de tablas en las que se dividirá la tabla de entrada.
- `hash`: función aplicada para obtener un valor hash.

Ejemplo

Particione la tabla `({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]})` en 2 tablas en la columna `[a]`, utilizando el valor de las columnas como función hash.

Uso

Power Query M

```
Table.Partition(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }  
),  
    "a",
```



```
2,  
each _  
)
```

Salida

Power Query M

```
{  
  Table.FromRecords({  
    [a = 2, b = 4],  
    [a = 2, b = 4]  
  }),  
  Table.FromRecords({  
    [a = 1, b = 4],  
    [a = 1, b = 4]  
  })  
}
```

Last updated on 25/03/2026

Table.PartitionLlave

16/09/2025

Sintaxis

```
Table.PartitionKey(table as table) as nullable list
```

Acerca de

Devuelve la clave de partición de la tabla especificada.

Table.PartitionValues

16/09/2025

Sintaxis

```
Table.PartitionValues(table as table) as table
```

Acerca de

Devuelve información sobre cómo se particiona una tabla. Se devuelve una tabla donde cada columna es una columna de partición de la tabla original y cada fila corresponde a una partición de la tabla original.

Table.Pivot

Syntax

```
Table.Pivot(  
    table as table,  
    pivotValues as list,  
    attributeColumn as text,  
    valueColumn as text,  
    optional aggregationFunction as nullable function  
) as table
```

Acerca de

Dado un par de columnas que representan pares de atributo-valor, convierte los datos de la columna de atributos a encabezados de columna.

Ejemplo 1

Tome los valores "a", "b" y "c" en la columna de atributo de la tabla ({ [key = "x", attribute = "a", value = 1], [key = "x", attribute = "c", value = 3], [key = "y", attribute = "a", value = 2], [key = "y", attribute = "b", value = 4] }) y pivote en su propia columna.

Uso

Power Query M

```
Table.Pivot(  
    Table.FromRecords(  
        [key = "x", attribute = "a", value = 1],  
        [key = "x", attribute = "c", value = 3],  
        [key = "y", attribute = "a", value = 2],  
        [key = "y", attribute = "b", value = 4]  
    )),  
    {"a", "b", "c"},  
    "attribute",  
    "value"  
)
```

Salida

Power Query M

```
Table.FromRecords({
    [key = "x", a = 1, b = null, c = 3],
    [key = "y", a = 2, b = 4, c = null]
})
```

Ejemplo 2

Tome los valores "a", "b" y "c" en la columna de atributo de la tabla (`{ [ key = "x", attribute = "a", value = 1 ], [ key = "x", attribute = "c", value = 3 ], [ key = "x", attribute = "c", value = 5 ], [ key = "y", attribute = "a", value = 2 ], [ key = "y", attribute = "b", value = 4 ] }`) y pivote en su propia columna. El atributo "c" para la clave "x" tiene varios valores asociados, por lo que use la función `List.Max` para resolver el conflicto.

Uso

Power Query M

```
Table.Pivot(
    Table.FromRecords({
        [key = "x", attribute = "a", value = 1],
        [key = "x", attribute = "c", value = 3],
        [key = "x", attribute = "c", value = 5],
        [key = "y", attribute = "a", value = 2],
        [key = "y", attribute = "b", value = 4]
    }),
    {"a", "b", "c"},
    "attribute",
    "value",
    List.Max
)
```

Salida

Power Query M

```
Table.FromRecords({
    [key = "x", a = 1, b = null, c = 5],
    [key = "y", a = 2, b = 4, c = null]
})
```

Table.PositionOf

27/09/2025

Sintaxis

```
Table.PositionOf(  
    table as table,  
    row as record,  
    optional occurrence as any,  
    optional equationCriteria as any  
) as any
```

Acerca de

Devuelve la posición de fila de la primera coincidencia de `row` en el elemento `table` especificado. Devuelve -1 si no se encuentra ninguna aparición.

- `table`: la tabla de entrada.
- `row`: fila de la tabla de la que se va a buscar la posición.
- `occurrence`: (Opcional) Especifica las ocurrencias de la fila que se van a devolver.
- `equationCriteria`: (Opcional) Controla la comparación entre las filas de la tabla.

Ejemplo 1

Buscar la posición de la primera coincidencia de `[a = 2, b = 4]` en la tabla `({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]})`.

Uso

Power Query M

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }),  
    [a = 2, b = 4]  
)
```

Salida

0

Ejemplo 2

Buscar la posición de la segunda coincidencia de [a = 2, b = 4] en la tabla ({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]}).

Uso

Power Query M

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }),  
    [a = 2, b = 4],  
    1  
)
```

Salida

2

Ejemplo 3

Buscar la posición de todas las coincidencias de [a = 2, b = 4] en la tabla ({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]}).

Uso

Power Query M

```
Table.PositionOf(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }),  
    [a = 2, b = 4],  
    null  
)
```

Occurrence.All

)

Salida

{0, 2}

Contenido relacionado

- [Repetición](#)
- [Criterios de ecuación](#)

Table.PositionOfAny

Sintaxis

```
Table.PositionOfAny(  
    table as table,  
    rows as list,  
    optional occurrence as nullable number,  
    optional equationCriteria as any  
) as any
```

Acerca de

Devuelve las posiciones del elemento `table` de la primera coincidencia de la lista de `rows`.
Devuelve -1 si no se encuentra ninguna aparición.

- `table`: la tabla de entrada.
- `rows`: la lista de filas de la tabla de la que se van a buscar las posiciones.
- `occurrence`: (*Opcional*) Especifica las apariciones de la fila que se van a devolver.
- `equationCriteria`: (*Opcional*) Controla la comparación entre las filas de la tabla.

Ejemplo 1

Buscar la posición de la primera coincidencia de `[a = 2, b = 4]` o `[a = 6, b = 8]` en la tabla `({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]})`.

Uso

Power Query M

```
Table.PositionOfAny(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 1, b = 4],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    })),  
    {  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    })
```

```
}  
)
```

Salida

0

Ejemplo 2

Buscar la posición de todas las coincidencias de [a = 2, b = 4] o [a = 6, b = 8] en la tabla ({[a = 2, b = 4], [a = 6, b = 8], [a = 2, b = 4], [a = 1, b = 4]}).

Uso

Power Query M

```
Table.PositionOfAny(  
    Table.FromRecords({  
        [a = 2, b = 4],  
        [a = 6, b = 8],  
        [a = 2, b = 4],  
        [a = 1, b = 4]  
    }  
),  
    {  
        [a = 2, b = 4],  
        [a = 6, b = 8]  
    },  
    Occurrence.All  
)
```

Salida

{0, 1, 2}

Contenido relacionado

- [Repetición](#)
- [Criterios de ecuación](#)

Table.PrefixColumns

16/09/2025

Sintaxis

```
Table.PrefixColumns(table as table, prefix as text) as table
```

Acercas de

Devuelve una tabla donde todos los nombres de columna del `table` proporcionado tienen como prefijo el texto especificado, `prefix`, además de un punto con el formato

```
prefix.ColumnName.
```

Ejemplo 1

Poner en las columnas el prefijo "MyTable" en la tabla.

Uso

Power Query M

```
Table.PrefixColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "MyTable"  
)
```

Salida

```
Table.FromRecords({[MyTable.CustomerID = 1, MyTable.Name = "Bob", MyTable.Phone = "123-4567"]})
```

Table.Profile

Syntax

```
Table.Profile(table as table, optional additionalAggregates as nullable list) as table
```

Acerca de

Devuelve un perfil para las columnas de `table`.

Se devuelve la siguiente información para cada columna (cuando procede):

- minimum
- maximum
- promedio
- desviación estándar
- count
- recuento nulo
- recuento distinto

Last updated on 22/11/2025

Table.PromoteHeaders

Syntax

```
Table.PromoteHeaders(table as table, optional options as nullable record) as table
```

Acerca de

Promueve la primera fila de valores como encabezados de columna nuevos (es decir, nombres de columna). De forma predeterminada, solo se promueven los valores de texto o número a los encabezados. Opciones válidas:

- **PromoteAllScalars**: si se establece **true** en , todos los valores escalares de la primera fila se promueven a encabezados mediante **Culture**, si se especifica (o la configuración regional del documento actual). En el caso de los valores que no se pueden convertir en texto, se usará un nombre de columna predeterminado.
- **Culture**: nombre de referencia cultural que especifica la referencia cultural de los datos.

Ejemplo 1

Promover la primera fila de valores de la tabla.

Uso

Power Query M

```
Table.PromoteHeaders(  
    Table.FromRecords({  
        [Column1 = "CustomerID", Column2 = "Name", Column3 = #date(1980, 1, 1)],  
        [Column1 = 1, Column2 = "Bob", Column3 = #date(1980, 1, 1)]  
    })  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Column3 = #date(1980, 1, 1)]})
```

Ejemplo 2

Promueva todos los escalares de la primera fila de la tabla a los encabezados.

Uso

Power Query M

```
Table.PromoteHeaders(  
    Table.FromRecords(  
        [Rank = 1, Name = "Name", Date = #date(1980, 1, 1)],  
        [Rank = 1, Name = "Bob", Date = #date(1980, 1, 1)]  
    ),  
    [PromoteAllScalars = true, Culture = "en-US"]  
)
```

Salida

```
Table.FromRecords({[1 = 1, Name = "Bob", #"1/1/1980" = #date(1980, 1, 1)]})
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Table.Range

Syntax

```
Table.Range(  
    table as table,  
    offset as number,  
    optional count as nullable number  
) as table
```

Acerca de

Devuelve las filas desde el `table` principio en el especificado `offset`. Un parámetro opcional, `count`, especifica cuántas filas se van a devolver. De forma predeterminada, se devuelven todas las filas después del desplazamiento.

Ejemplo 1

Devuelve todas las filas a partir del desplazamiento 1 de la tabla.

Uso

Power Query M

```
Table.Range(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    1  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Ejemplo 2

Devuelve una fila a partir del desplazamiento 1 de la tabla.

Uso

Power Query M

```
Table.Range(  
    Table.FromRecords(  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    ),  
    1,  
    1  
)
```

Salida

```
Table.FromRecords({[CustomerID = 2, Name = "Jim", Phone = "987-6543"]})
```

Last updated on 22/11/2025

Table.RemoveColumns

16/09/2025

Sintaxis

```
Table.RemoveColumns(  
    table as table,  
    columns as any,  
    optional missingField as nullable number  
) as table
```

Acerca de

Quita el elemento `columns` especificado del elemento `table` proporcionado. Si la columna especificada no existe, se produce un error a menos que el parámetro `missingField` opcional especifique un comportamiento alternativo (por ejemplo, [MissingField.UseNull](#) o [MissingField.Ignore](#)).

Ejemplo 1

Eliminar la columna [Phone] de la tabla.

Uso

Power Query M

```
Table.RemoveColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "Phone"  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob"]})
```

Ejemplo 2

Pruebe a quitar una columna inexistente de la tabla.

Uso

Power Query M

```
Table.RemoveColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "Address"  
)
```

Salida

```
[Expression.Error] The column 'Address' of the table wasn't found.
```

Table.RemoveFirstN

16/09/2025

Sintaxis

```
Table.RemoveFirstN(table as table, optional countOrCondition as any) as table
```

Acerca de

Devuelve una tabla que no contiene el primer número especificado de filas, `countOrCondition`, de la tabla `table`. El número de filas quitadas depende del parámetro `countOrCondition` opcional .

- Si `countOrCondition` se omite solo se quita la primera fila.
- Si `countOrCondition` es un número, se quitarán muchas filas (comenzando en la parte superior).
- Si `countOrCondition` es una condición, las filas que cumplen la condición se quitarán hasta que una fila no cumpla la condición.

Ejemplo 1

Quite la primera fila de la tabla.

Uso

Power Query M

```
Table.RemoveFirstN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    1  
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
})
```

Ejemplo 2

Quite las dos primeras filas de la tabla.

Uso

Power Query M

```
Table.RemoveFirstN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    2
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
})
```

Ejemplo 3

Quite las primeras filas donde [CustomerID] <=2 de la tabla.

Uso

Power Query M

```
Table.RemoveFirstN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"] ,
    })
)
```

```
[CustomerID = 3, Name = "Paul", Phone = "543-7890"] ,  
[CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
}),  
each [CustomerID] <= 2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Table.RemoveLastN

Syntax

```
Table.RemoveLastN(table as table, optional countOrCondition as any) as table
```

Acerca de

Devuelve una tabla que no contiene las últimas `countOrCondition` filas de la tabla `table`. El número de filas quitadas depende del parámetro `countOrCondition` opcional .

- Si `countOrCondition` se omite solo se quita la última fila.
- Si `countOrCondition` es un número, se quitarán muchas filas (comenzando en la parte inferior).
- Si `countOrCondition` es una condición, las filas que cumplen la condición se quitarán hasta que una fila no cumpla la condición.

Ejemplo 1

Quite la última fila de la tabla.

Uso

Power Query M

```
Table.RemoveLastN(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
1  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
})
```

```
[CustomerID = 3, Name = "Paul", Phone = "543-7890"]
})
```

Ejemplo 2

Quite las últimas filas donde [CustomerID] > 2 de la tabla.

Uso

Power Query M

```
Table.RemoveLastN(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    each [CustomerID] >= 2
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Last updated on 22/11/2025

Table.RemoveMatchingRows

02/10/2025

Sintaxis

```
Table.RemoveMatchingRows(  
    table as table,  
    rows as list,  
    optional equationCriteria as any  
) as table
```

Acerca de

Quita todas las apariciones de las filas especificadas de la tabla.

- **table**: la tabla que se va a buscar.
- **rows**: una lista que contiene información sobre las filas que se van a quitar.
- **equationCriteria**: (Opcional) Especifica cómo se determina la igualdad al comparar valores. Este parámetro puede ser una función de selector de claves, una función de comparador o una lista de las columnas de la tabla que se van a usar al comparar filas.

Ejemplo 1

Quite las filas donde [a = 1] de la tabla especificada.

Uso

Power Query M

```
Table.RemoveMatchingRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 3, b = 4],  
        [a = 1, b = 6]  
    }),  
    {[a = 1]},  
    "a"  
)
```

Salida

```
Table.FromRecords({[a = 3, b = 4]})
```


Ejemplo 2

Elimine los pedidos cancelados, ignorando las mayúsculas y minúsculas.

Uso

Power Query M

```
let
    CurrentOrders = #table(type table[OrderID = number, Product = text, Quantity = number],
        {
            {101, "Widget", 5},
            {102, "Gadget", 3},
            {103, "Widget", 5}
        }
    ),
    CanceledOrders = {
        [OrderID = 103, Product = "widget", Quantity = 5]
    },
    FilteredOrders = Table.RemoveMatchingRows(CurrentOrders, CanceledOrders,
        Comparer.OrdinalIgnoreCase)
in
    FilteredOrders
```

Salida

Power Query M

```
#table(type table[OrderID = number, Product = text, Quantity = number],
{
    {101, "Widget", 5},
    {102, "Gadget", 3}
})
```

Ejemplo 3

Elimine las tareas de mantenimiento que caen en festividades de EE. UU.

Uso

Power Query M

```
let
    MaintenanceSchedule = #table(type table [Task = text, Date = date],
    {
        {"HVAC Check", #date(2025, 7, 10)},           // Not a holiday
        {"Window Washing", #date(2025, 9, 1)},         // Labor Day
        {"Fire Drill", #date(2025, 9, 17)},            // Not a holiday
    })
```

```

        {"Light Bulb Replacement", #date(2025, 11, 27)} // Thanksgiving
    }},

    USHolidays = {
        [Date = #date(2025, 1, 1)],    // New Year's Day
        [Date = #date(2025, 7, 4)],    // Independence Day
        [Date = #date(2025, 9, 1)],    // Labor Day
        [Date = #date(2025, 11, 27)], // Thanksgiving
        [Date = #date(2025, 12, 25)]   // Christmas
    },

    FilteredSchedule = Table.RemoveMatchingRows(
        MaintenanceSchedule,
        USHolidays,
        {"Date"}
    )
in
    FilteredSchedule

```

Salida

Power Query M

```

#table(type table[Task = text, Date = date],
{
    {"HVAC Check", #date(2025, 7, 10)},
    {"Fire Drill", #date(2025, 9, 17)}
})

```

Contenido relacionado

[Criterios de ecuación](#)

Table.RemoveRows

Syntax

```
Table.RemoveRows(  
    table as table,  
    offset as number,  
    optional count as nullable number  
) as table
```

Acerca de

count Quita las filas del principio de , **table** comenzando en el **offset** especificado. Se usa un recuento predeterminado de 1 si no se proporciona el **count** parámetro .

Ejemplo 1

Quite la primera fila de la tabla.

Uso

Power Query M

```
Table.RemoveRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    0  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Ejemplo 2

Quite la fila en la posición 1 de la tabla.

Uso

Power Query M

```
Table.RemoveRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    1  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Ejemplo 3

Quite dos filas a partir de la posición 1 de la tabla.

Uso

Power Query M

```
Table.RemoveRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }  
),  
    1,  
    2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Last updated on 22/11/2025

Table.RemoveRowsWithErrors

Syntax

```
Table.RemoveRowsWithErrors(table as table, optional columns as nullable list) as table
```

Acerca de

Devuelve una tabla con las filas quitadas de la tabla de entrada que contienen un error en al menos una de las celdas. Si se especifica una lista de columnas, solo se inspeccionan las celdas de las columnas especificadas para detectar errores.

Ejemplo 1

Quite el valor de error de la primera fila.

Uso

Power Query M

```
Table.RemoveRowsWithErrors(  
    Table.FromRecords({  
        [Column1 = ...],  
        [Column1 = 2],  
        [Column1 = 3]  
    })  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = 2],  
    [Column1 = 3]  
})
```

Table.RenameColumns

16/09/2025

Sintaxis

```
Table.RenameColumns(  
    table as table,  
    renames as list,  
    optional missingField as nullable number  
) as table
```

Acerca de

Realiza los cambios de nombre especificados en las columnas de la tabla `table`. Una operación de reemplazo `renames` consta de una lista de dos valores, el nombre de columna anterior y el nombre de columna nuevo, proporcionados en una lista. Si la columna no existe, se produce una excepción a menos que el parámetro `missingField` opcional especifique una alternativa (por ejemplo, [MissingField.UseNull](#) o [MissingField.Ignore](#)).

Ejemplo 1

Reemplazar el nombre de columna "CustomerNum" por "CustomerID" en la tabla.

Uso

Power Query M

```
Table.RenameColumns(  
    Table.FromRecords({[CustomerNum = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"CustomerNum", "CustomerID"}  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Ejemplo 2

Reemplazar el nombre de columna "CustomerNum" por "CustomerID" y "PhoneNum" con "Phone" en la tabla.

Uso

Power Query M

```
Table.RenameColumns(  
    Table.FromRecords({[CustomerNum = 1, Name = "Bob", PhoneNum = "123-4567"]}),  
    {  
        {"CustomerNum", "CustomerID"},  
        {"PhoneNum", "Phone"}  
    }  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Ejemplo 3

Reemplazar el nombre de columna "NewCol" por "NewColumn" en la tabla y omitirlo si la columna no existe.

Uso

Power Query M

```
Table.RenameColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"NewCol", "NewColumn"},  
    MissingField.Ignore  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```


Table.ReorderColumns

Sintaxis

```
Table.ReorderColumns(  
    table as table,  
    columnOrder as list,  
    optional missingField as nullable number  
) as table
```

Acerca de

Devuelve una tabla del valor `table` de entrada, con las columnas en el orden especificado por `columnOrder`. Las columnas que no se especifican en la lista no se reordenan. Si la columna no existe, se produce un error a menos que el parámetro `missingField` opcional especifique una alternativa (por ejemplo [MissingField.UseNull](#) , o [MissingField.Ignore](#)).

Ejemplo 1

Cambiar el orden de las columnas [Phone] y [Name] en la tabla.

Uso

Power Query M

```
Table.ReorderColumns(  
    Table.FromRecords({[CustomerID = 1, Phone = "123-4567", Name = "Bob"]}),  
    {"Name", "Phone"}  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Ejemplo 2

Cambie el orden de las columnas [Teléfono] y [Dirección] o use "[MissingField.Ignore](#)" en la tabla. La tabla no cambia porque la columna [Address] no existe.

Uso

Power Query M

```
Table.ReorderColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"Phone", "Address"},  
    MissingField.Ignore  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Last updated on 02/04/2026

Table.Repeat

16/09/2025

Sintaxis

```
Table.Repeat(table as table, count as number) as table
```

Acercas de

Devuelve una tabla con las filas de la entrada `table` repetidas el número de veces especificado por `count`.

Ejemplo 1

Repita las filas de la tabla dos veces.

Uso

Power Query M

```
Table.Repeat(  
    Table.FromRecords({  
        [a = 1, b = "hello"],  
        [a = 3, b = "world"]  
    }),  
    2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [a = 1, b = "hello"],  
    [a = 3, b = "world"],  
    [a = 1, b = "hello"],  
    [a = 3, b = "world"]  
})
```

Table.ReplaceErrorValues

Syntax

```
Table.ReplaceErrorValues(table as table, errorReplacement as list) as table
```

Acerca de

Reemplaza los valores de error de las columnas especificadas de por **table** los nuevos valores de la **errorReplacement** lista. El formato de la lista es {{column1, value1}, ...}. Solo puede haber un valor de reemplazo por columna y especificar la columna más de una vez producirá un error.

Ejemplo 1

Reemplace el valor de error por el texto "world" de la tabla.

Uso

Power Query M

```
Table.ReplaceErrorValues(  
    Table.FromRows({{1, "hello"}, {3, ...}}, {"A", "B"}),  
    {"B", "world"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [A = 1, B = "hello"],  
    [A = 3, B = "world"]  
})
```

Ejemplo 2

Reemplace el valor de error de la columna A por el texto "hello" y en la columna B por el texto "world" de la tabla.

Uso

Power Query M

```
Table.ReplaceErrorValues(  
    Table.FromRows({{..., ...}, {1, 2}}, {"A", "B"}),  
    {"A", "hello"}, {"B", "world"}  
)
```

Salida

Power Query M

```
Table.FromRecords(  
    [A = "hello", B = "world"],  
    [A = 1, B = 2]  
)
```

Last updated on 22/11/2025

Table.ReplaceKeys

Syntax

```
Table.ReplaceKeys(table as table, keys as list) as table
```

Acerca de

Reemplaza las claves de la tabla especificada.

Ejemplo 1

Reemplace las claves existentes de una tabla.

Uso

Power Query M

```
let
    table = Table.FromRecords({
        [Id = 1, Name = "Hello There"],
        [Id = 2, Name = "Good Bye"]
    }),
    tableWithKeys = Table.AddKey(table, {"Id"}, true),
    resultTable = Table.ReplaceKeys(tableWithKeys, {[Columns = {"Id"}, Primary = false]})
in
    resultTable
```

Salida

Power Query M

```
Table.FromRecords({
    [Id = 1, Name = "Hello There"],
    [Id = 2, Name = "Good Bye"]
})
```

Table.ReplaceMatchingRows

16/09/2025

Sintaxis

```
Table.ReplaceMatchingRows(  
    table as table,  
    replacements as list,  
    optional equationCriteria as any  
) as table
```

Acerca de

Sustituye todas las filas especificadas en `table` por las proporcionadas. Las filas que se van a reemplazar y los reemplazos se especifican en `replacements` con el formato {old, new}. Se puede especificar un parámetro `equationCriteria` opcional para controlar la comparación entre las filas de la tabla.

Ejemplo 1

Reemplazar las filas [a = 1, b = 2] y [a = 2, b = 3] con [a = -1, b = -2],[a = -2, b = -3] en la tabla.

Uso

Power Query M

```
Table.ReplaceMatchingRows(  
    Table.FromRecords({  
        [a = 1, b = 2],  
        [a = 2, b = 3],  
        [a = 3, b = 4],  
        [a = 1, b = 2]  
    }),  
    {  
        {[a = 1, b = 2], [a = -1, b = -2]},  
        {[a = 2, b = 3], [a = -2, b = -3]}  
    }  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [a = -1, b = -2],  
    [a = -2, b = -3],  
    [a = 3, b = 4],  
    [a = -1, b = -2]  
})
```

Contenido relacionado

[Criterios de ecuación](#)

Table.ReplacePartitionKey

16/09/2025

Sintaxis

```
Table.ReplacePartitionKey(table as table, partitionKey as nullable list) as table
```

Acerca de

Reemplaza la clave de partición de la tabla especificada.

Table.ReplaceRelationshipIdentity

16/09/2025

Sintaxis

```
Table.ReplaceRelationshipIdentity(value as any, identity as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Table.ReplaceRows

Syntax

```
Table.ReplaceRows(  
    table as table,  
    offset as number,  
    count as number,  
    rows as list  
) as table
```

Acerca de

Reemplaza un número especificado de filas, , `count` en la entrada `table` por el especificado `rows`, comenzando después de `offset`. El `rows` parámetro es una lista de registros.

- `table`: la tabla donde se realiza el reemplazo.
- `offset`: número de filas que se van a omitir antes de realizar el reemplazo.
- `count`: número de filas que se van a reemplazar.
- `rows`: la lista de registros de fila que se van a insertar en en la `table` ubicación especificada por `.offset`

Ejemplo

A partir de la posición 1, reemplace 3 filas.

Uso

Power Query M

```
Table.ReplaceRows(  
    Table.FromRecords({  
        [Column1 = 1],  
        [Column1 = 2],  
        [Column1 = 3],  
        [Column1 = 4],  
        [Column1 = 5]  
    }),  
    1,  
    3,  
    {[Column1 = 6], [Column1 = 7]}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = 1],  
    [Column1 = 6],  
    [Column1 = 7],  
    [Column1 = 5]  
})
```

Last updated on 25/03/2026

Table.ReplaceValue

07/10/2025

Sintaxis

```
Table.ReplaceValue(  
    table as table,  
    oldValue as any,  
    newValue as any,  
    replacer as function,  
    columnsToSearch as list  
) as table
```

Acerca de

Reemplaza un valor por un nuevo valor en las columnas especificadas de una tabla.

- `table`: la tabla que se va a buscar.
- `oldValue`: valor que se va a reemplazar.
- `newValue`: valor de reemplazo.
- `replacer`: La función de reemplazo a utilizar. La función puede ser [Replacer.ReplaceText](#) para reemplazar el texto original por texto nuevo, [Replacer.ReplaceValue](#) para reemplazar el valor original por un nuevo valor o un reemplazador personalizado.
- `columnsToSearch`: lista que contiene la columna o columnas específicas de la tabla para buscar el valor que se va a reemplazar.

Ejemplo 1

Reemplace el texto "goodbye" por "world" en la columna B, para que haya una coincidencia solo con el valor completo.

Uso

Power Query M

```
Table.ReplaceValue(  
    Table.FromRecords({  
        [A = 1, B = "hello"],  
        [A = 2, B = "goodbye"],  
        [A = 3, B = "goodbyes"]  
    }  
)
```

```

    "goodbye",
    "world",
    Replacer.ReplaceValue,
    {"B"}
)

```

Salida

Power Query M

```

Table.FromRecords({
    [A = 1, B = "hello"],
    [A = 2, B = "world"],
    [A = 3, B = "goodbyes"]
})

```

Ejemplo 2

Reemplace el texto "ur" por "or" en la columna B, para que haya una coincidencia con cualquier parte del valor.

Uso

Power Query M

```

Table.ReplaceValue(
    Table.FromRecords({
        [A = 1, B = "hello"],
        [A = 2, B = "wurld"]
    }),
    "ur",
    "or",
    Replacer.ReplaceText,
    {"B"}
)

```

Salida

Power Query M

```

Table.FromRecords({
    [A = 1, B = "hello"],
    [A = 2, B = "world"]
})

```

Ejemplo 3

Anonimizar los nombres de los empleados de EE. UU.

Uso

Power Query M

```
Table.ReplaceValue(  
    Table.FromRecords({  
        [Name = "Cindy", Country = "US"],  
        [Name = "Bob", Country = "CA"]  
    }),  
    each if [Country] = "US" then [Name] else false,  
    each Text.Repeat("*", Text.Length([Name])),  
    Replacer.ReplaceValue,  
    {"Name"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Name = "*****", Country = "US"],  
    [Name = "Bob", Country = "CA"]  
})
```

Ejemplo 4

Anonimizar todas las columnas de empleados de EE. UU.

Uso

Power Query M

```
Table.ReplaceValue(  
    Table.FromRecords({  
        [Name = "Cindy", Country = "US"],  
        [Name = "Bob", Country = "CA"]  
    }),  
    each [Country] = "US",  
    "?",  
    (currentValue, isUS, replacementValue) =>  
        if isUS then  
            Text.Repeat(replacementValue, Text.Length(currentValue))  
        else  
            currentValue,
```

```
{ "Name", "Country" }  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Name = "?????", Country = "??"],  
    [Name = "Bob", Country = "CA"]  
})
```

Contenido relacionado

[Funciones de sustituto](#)

Table.ReverseRows

16/09/2025

Sintaxis

```
Table.ReverseRows(table as table) as table
```

Acerca de

Devuelve una tabla con las filas de la entrada `table` en orden inverso.

Ejemplo 1

Invertir las filas de la tabla.

Uso

Power Query M

```
Table.ReverseRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"]  
})
```

Table.RowCount

Syntax

```
Table.RowCount(table as table) as number
```

Acerca de

Devuelve el número de filas de `.table`

Ejemplo

Busque el número de filas de la tabla.

Uso

Power Query M

```
Table.RowCount(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Salida

3

Last updated on 25/03/2026

Table.Schema

Syntax

```
Table.Schema(table as table) as table
```

Acerca de

Devuelve una tabla que describe las columnas de `table`.

Cada fila de la tabla describe las propiedades de una columna de `table`:

[Expandir tabla](#)

Nombre de la columna	Descripción
<code>Name</code>	Nombre de la columna.
<code>Position</code>	Posición basada en 0 de la columna en <code>table</code> .
<code>TypeName</code>	Nombre del tipo de la columna.
<code>Kind</code>	Tipo del tipo de la columna.
<code>IsNullable</code>	Si la columna puede contener <code>null</code> valores.
<code>NumericPrecisionBase</code>	Base numérica (por ejemplo, base-2 o base-10) de los <code>NumericPrecision</code> campos y <code>NumericScale</code> .
<code>NumericPrecision</code>	Precisión de una columna numérica en la base especificada por <code>NumericPrecisionBase</code> . Este es el número máximo de dígitos que se pueden representar mediante un valor de este tipo (incluidos los dígitos fraccionarios).
<code>NumericScale</code>	Escala de una columna numérica en la base especificada por <code>NumericPrecisionBase</code> . Este es el número de dígitos de la parte fraccionaria de un valor de este tipo. Un valor de <code>0</code> indica una escala fija sin dígitos fraccionarios. Un valor de <code>null</code> indica que no se conoce la escala (ya sea porque está flotante o no está definida).
<code>DateTimePrecision</code>	Número máximo de dígitos fraccionarios admitidos en la parte de segundos de un valor de fecha o hora.

Nombre de la columna	Descripción
<code>MaxLength</code>	Número máximo de caracteres permitidos en una <code>text</code> columna o el número máximo de bytes permitidos en una <code>binary</code> columna.
<code>IsVariableLength</code>	Indica si esta columna puede variar en longitud (hasta <code>MaxLength</code>) o si es de tamaño fijo.
<code>NativeTypeName</code>	Nombre del tipo de la columna en el sistema de tipos nativos del origen (por ejemplo, <code>nvarchar</code> para SQL Server).
<code>NativeDefaultExpression</code>	Expresión predeterminada para un valor de esta columna en el lenguaje de expresión nativa del origen (por ejemplo, <code>42</code> o <code>newid()</code> para SQL Server).
<code>Description</code>	Descripción de la columna.

Last updated on 04/04/2026

Table.SelectColumns

16/09/2025

Sintaxis

```
Table.SelectColumns(  
    table as table,  
    columns as any,  
    optional missingField as nullable number  
) as table
```

Acerca de

Devuelve el elemento `table` con únicamente el valor `columns` especificado.

- `table`: la tabla proporcionada.
- `columns`: lista de columnas de la tabla `table` que se va a devolver. Las columnas de la tabla devuelta aparecen en el orden indicado en `columns`.
- `missingField`: *(Opcional)* Qué hacer si la columna no existe. Ejemplo: [MissingField.UseNull](#) o [MissingField.Ignore](#).

Ejemplo 1

Incluir solo la columna [Name].

Uso

Power Query M

```
Table.SelectColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    "Name"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Name = "Bob"],  
    [Name = "Jim"],  
    [Name = "Paul"],  
    [Name = "Ringo"]  
})
```

Ejemplo 2

Incluir solo las columnas [CustomerID] y [Name].

Uso

Power Query M

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"CustomerID", "Name"}  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob"]})
```

Ejemplo 3

Si la columna incluida no existe, el resultado predeterminado será un error.

Uso

Power Query M

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    "NewColumn"  
)
```

Salida

```
[Expression.Error] The field 'NewColumn' of the record wasn't found.
```

Ejemplo 4

Si la columna incluida no existe, la opción `MissingField.UseNull` crea una columna de valores NULL.

Uso

Power Query M

```
Table.SelectColumns(  
    Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}),  
    {"CustomerID", "NewColumn"},  
    MissingField.UseNull  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, NewColumn = null]})
```

Table.SelectRows

16/09/2025

Sintaxis

```
Table.SelectRows(table as table, condition as function) as table
```

Acerca de

Devuelve una tabla de filas de `table` que coincide con la selección `condition`.

Ejemplo 1

Seleccione las filas de la tabla donde los valores de la columna [CustomerID] son mayores que 2.

Uso

Power Query M

```
Table.SelectRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    each [CustomerID] > 2  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Ejemplo 2

Seleccione las filas de la tabla donde los nombres no contienen un "B".

Uso

Power Query M

```
Table.SelectRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    each not Text.Contains([Name], "B")  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

Table.SelectRowsWithErrors

Syntax

```
Table.SelectRowsWithErrors(table as table, optional columns as nullable list) as table
```

Acerca de

Devuelve una tabla con solo las filas de la tabla de entrada que contienen un error en al menos una de las celdas. Si se especifica una lista de columnas, solo se inspeccionan las celdas de las columnas especificadas para detectar errores.

Ejemplo 1

Seleccione los nombres de los clientes con errores en sus filas.

Uso

Power Query M

```
Table.SelectRowsWithErrors(  
    Table.FromRecords({  
        [CustomerID = ..., Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    })  
)[Name]
```

Salida

```
{"Bob"}
```

Last updated on 22/11/2025

Table.SingleRow

Syntax

```
Table.SingleRow(table as table) as record
```

Acerca de

Devuelve la fila única de la fila `table`. Si tiene `table` más de una fila, se genera un error.

Ejemplo

Devuelve la única fila de la tabla.

Uso

Power Query M

```
Table.SingleRow(Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]}))
```

Salida

```
[CustomerID = 1, Name = "Bob", Phone = "123-4567"]
```

Last updated on 04/04/2026

Table.Skip

Syntax

```
Table.Skip(table as table, optional countOrCondition as any) as table
```

Acerca de

Devuelve una tabla que no contiene el primer número especificado de filas, `countOrCondition`, de la tabla `table`. El número de filas omitidas depende del parámetro `countOrCondition` opcional .

- Si `countOrCondition` se omite solo se omite la primera fila.
- Si `countOrCondition` es un número, se omitirán muchas filas (comenzando en la parte superior).
- Si `countOrCondition` es una condición, las filas que cumplen la condición se omitirán hasta que una fila no cumpla la condición.

Ejemplo 1

Omita la primera fila de la tabla.

Uso

Power Query M

```
Table.Skip(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
    }),  
    1  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]  
})
```

```
[CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
})
```

Ejemplo 2

Omita las dos primeras filas de la tabla.

Uso

Power Query M

```
Table.Skip(
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
    }),
    2
)
```

Salida

Power Query M

```
Table.FromRecords({
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
    [CustomerID = 4, Name = "Ringo", Phone = "232-1550"]
})
```

Ejemplo 3

Omita las primeras filas donde [Precio] > 25 de la tabla.

Uso

Power Query M

```
Table.Skip(
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Band-aids", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],
    })
)
```

```
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
    }),
    each [Price] > 25
)
```

Salida

Power Query M

```
Table.FromRecords({
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5],
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25],
    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200],
    [OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2],
    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20],
    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100],
    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
})
```

Last updated on 22/11/2025

Table.Sort

16/09/2025

Sintaxis

```
Table.Sort(table as table, comparisonCriteria as any) as table
```

Acerca de

Ordena el elemento `table` usando la lista de uno o varios nombres de columna y el opcional `comparisonCriteria` en el formato { { col1, criterio de comparación }, {col2} }.

Ejemplo 1

Ordenar la tabla en la columna "OrderID".

Uso

Power Query M

```
Table.Sort(  
    Table.FromRecords({  
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],  
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],  
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],  
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],  
        [OrderID = 5, CustomerID = 3, Item = "Band aids", Price = 2.0],  
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],  
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],  
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],  
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]  
    }),  
    {"OrderID"}  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100],  
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5],  
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25],  
    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200],  
    [OrderID = 5, CustomerID = 3, Item = "Band aids", Price = 2],  
    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20],  
    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],  
    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100],  
    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]  
})
```

```

[OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200],
[OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2],
[OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20],
[OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
[OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100],
[OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
})

```

Ejemplo 2

Ordenar la tabla en la columna "OrderID" por orden descendente.

Uso

Power Query M

```

Table.Sort(
    Table.FromRecords({
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],
        [OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2.0],
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]
    }),
    {"OrderID", Order.Descending}
)

```

Salida

Power Query M

```

Table.FromRecords({
    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25],
    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100],
    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],
    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20],
    [OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2],
    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200],
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25],
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5],
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100]
})

```


Ejemplo 3

Ordenar la tabla en la columna "CustomerID" y luego "OrderID", estando "CustomerID" en orden ascendente.

Uso

Power Query M

```
Table.Sort(  
    Table.FromRecords({  
        [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100.0],  
        [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5.0],  
        [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25.0],  
        [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200.0],  
        [OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2.0],  
        [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20.0],  
        [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],  
        [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100.0],  
        [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]  
    }  
),  
    {  
        {"CustomerID", Order.Ascending},  
        "OrderID"  
    }  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [OrderID = 1, CustomerID = 1, Item = "Fishing rod", Price = 100],  
    [OrderID = 2, CustomerID = 1, Item = "1 lb. worms", Price = 5],  
    [OrderID = 6, CustomerID = 1, Item = "Tackle box", Price = 20],  
    [OrderID = 3, CustomerID = 2, Item = "Fishing net", Price = 25],  
    [OrderID = 4, CustomerID = 3, Item = "Fish tazer", Price = 200],  
    [OrderID = 5, CustomerID = 3, Item = "Bandaid", Price = 2],  
    [OrderID = 7, CustomerID = 5, Item = "Bait", Price = 3.25],  
    [OrderID = 8, CustomerID = 5, Item = "Fishing Rod", Price = 100],  
    [OrderID = 9, CustomerID = 6, Item = "Bait", Price = 3.25]  
})
```

Contenido relacionado

[Criterios de comparación](#)

Table.Split

16/09/2025

Sintaxis

```
Table.Split(table as table, pageSize as number) as list
```

Acerca de

table Divide en una lista de tablas donde el primer elemento de la lista es una tabla que contiene las primeras **pageSize** filas de la tabla de origen, el siguiente elemento de la lista es una tabla que contiene las filas siguientes **pageSize** de la tabla de origen, etc.

Ejemplo 1

Divida una tabla de cinco registros en tablas con dos registros cada uno.

Uso

Power Query M

```
let
    Customers = Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],
        [CustomerID = 4, Name = "Cristina", Phone = "232-1550"],
        [CustomerID = 5, Name = "Anita", Phone = "530-1459"]
    })
in
    Table.Split(Customers, 2)
```

Salida

Power Query M

```
{
    Table.FromRecords({
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]
    }),
    Table.FromRecords({
```

```
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"],  
        [CustomerID = 4, Name = "Cristina", Phone = "232-1550"]  
    }),  
    Table.FromRecords({  
        [CustomerID = 5, Name = "Anita", Phone = "530-1459"]  
    })  
}
```

Table.SplitAt

16/09/2025

```
Table.SplitAt(table as table, count as number) as list
```

Acerca de

Devuelve una lista que contiene dos tablas: una tabla con las primeras N filas de `table` (especificadas por `count`) y una tabla que contiene las filas restantes de `table`. Si las tablas de la lista resultante se enumeran exactamente una vez y en orden, la función solo enumerará `table` una vez.

Ejemplo 1

Devuelve las dos primeras filas de la tabla y las filas restantes de la tabla.

Uso

Power Query M

```
Table.SplitAt(#table({"a", "b", "c"}, {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}), 2)
```

Salida

Power Query M

```
{  
  #table({"a", "b", "c"}, {{1, 2, 3}, {4, 5, 6}}),  
  #table({"a", "b", "c"}, {{7, 8, 9}})  
}
```

Table.SplitColumn

16/09/2025

Sintaxis

```
Table.SplitColumn(  
    table as table,  
    sourceColumn as text,  
    splitter as function,  
    optional columnNamesOrNumber as any,  
    optional default as any,  
    optional extraColumns as any  
) as table
```

Acerca de

Divide la columna especificada en un conjunto de columnas adicionales mediante la función divisora especificada.

- **table**: tabla que contiene la columna que se va a dividir.
- **sourceColumn**: nombre de la columna que se va a dividir.
- **splitter**: la [función divisora](#) usada para dividir la columna (por ejemplo, [Splitter.SplitTextByDelimiter](#) o [Splitter.SplitTextByPositions](#)).
- **columnNamesOrNumber**: una lista de nombres de columna nuevos que se van a crear o el número de columnas nuevas.
- **default**: invalida el valor usado cuando no hay suficientes valores divididos para rellenar todas las columnas nuevas. El valor predeterminado de este parámetro es `null`.
- **extraColumns**: especifica qué hacer si puede haber más valores divididos que el número de columnas nuevas. Puede pasar un [ExtraValues.Type](#) valor de enumeración a este parámetro. El valor predeterminado es `ExtraValues.Ignore`.

Ejemplo 1

Divida la columna nombre en nombre y apellido.

Uso

```

let
    Source = #table(type table[CustomerID = number, Name = text, Phone = text],
        {
            {1, "Bob White", "123-4567"},
            {2, "Jim Smith", "987-6543"},
            {3, "Paul", "543-7890"},
            {4, "Cristina Best", "232-1550"}
        }),
    SplitColumns = Table.SplitColumn(
        Source,
        "Name",
        Splitter.SplitTextByDelimiter(" "))
in
    SplitColumns

```

Salida

Power Query M

```

#table(type table[CustomerID = number, Name.1 = text, Name.2 = text, Phone = text],
{
    {1, "Bob", "White", "123-4567"},
    {2, "Jim", "Smith", "987-6543"},
    {3, "Paul", null, "543-7890"},
    {4, "Cristina", "Best", "232-1550"}
})

```

Ejemplo 2

Divida la columna de nombre en nombre y apellido, y luego renombre las nuevas columnas.

Uso

Power Query M

```

let
    Source = #table(type table[CustomerID = number, Name = text, Phone = text],
        {
            {1, "Bob White", "123-4567"},
            {2, "Jim Smith", "987-6543"},
            {3, "Paul", "543-7890"},
            {4, "Cristina Best", "232-1550"}
        }),
    SplitColumns = Table.SplitColumn(
        Source,
        "Name",
        Splitter.SplitTextByDelimiter(" "),
        {"First Name", "Last Name"})

```

```
in
    SplitColumns
```

Salida

Power Query M

```
#table(type table[CustomerID = number, First Name = text, Last Name = text, Phone = text],
{
    {1, "Bob", "White", "123-4567"},
    {2, "Jim", "Smith", "987-6543"},
    {3, "Paul", null, "543-7890"},
    {4, "Cristina", "Best", "232-1550"}
})
```

Ejemplo 3

Divida la columna nombre en nombre de pila y apellido, renombre las nuevas columnas y rellene los espacios en blanco con "-Sin Registro-".

Uso

Power Query M

```
let
    Source = #table(type table[CustomerID = number, Name = text, Phone = text],
    {
        {1, "Bob White", "123-4567"},
        {2, "Jim Smith", "987-6543"},
        {3, "Paul", "543-7890"},
        {4, "Cristina Best", "232-1550"}
    }),
    SplitColumns = Table.SplitColumn(
        Source,
        "Name",
        Splitter.SplitTextByDelimiter(" "),
        {"First Name", "Last Name"},
        "-No Entry-")
in
    SplitColumns
```

Salida

Power Query M

```
#table(type table[CustomerID = number, First Name = text, Last Name = text, Phone = text],
```

```
{
    {1, "Bob", "White", "123-4567"},
    {2, "Jim", "Smith", "987-6543"},
    {3, "Paul", "-No Entry-", "543-7890"},
    {4, "Cristina", "Best", "232-1550"}
})
```

Ejemplo 4

Divida la columna de nombre en nombre y apellido, y luego renombre las nuevas columnas. Dado que puede haber más valores que el número de columnas disponibles, convierta la columna de apellidos en una lista que incluya todos los valores después del nombre.

Uso

Power Query M

```
let
    Source = #table(type table[CustomerID = number, Name = text, Phone = text],
        {
            {1, "Bob White", "123-4567"},
            {2, "Jim Smith", "987-6543"},
            {3, "Paul Green", "543-7890"},
            {4, "Cristina J. Best", "232-1550"}
        }
    ),
    SplitColumns = Table.SplitColumn(
        Source,
        "Name",
        Splitter.SplitTextByDelimiter(" "),
        {"First Name", "Last Name"},
        null,
        ExtraValues.List)
in
    SplitColumns
```

Salida

Power Query M

```
#table(type table[CustomerID = number, First Name = text, Last Name = text, Phone = text],
{
    {1, "Bob", {"White"}, "123-4567"},
    {2, "Jim", {"Smith"}, "987-6543"},
    {3, "Paul", {"Green"}, "543-7890"},
    {4, "Cristina", {"J.", "Best"}, "232-1550"}
})
```


Table.StopFolding

16/09/2025

Sintaxis

```
Table.StopFolding(table as table) as table
```

Acerca de

Impide que se ejecuten operaciones de nivel inferior en el origen de datos original de `table`.

Ejemplo 1

Captura datos de una tabla SQL de una manera que impide que las operaciones de bajada se ejecuten como una consulta en SQL Server.

Uso

Power Query M

```
let
    Source = Sql.Database("SomeSQLServer", "MyDb"),
    MyTable = Source{[Item="MyTable"]}[Data],
    MyLocalTable = Table.StopFolding(MyTable)
in
    MyLocalTable
```

Salida

`table`

Table.ToColumns

16/09/2025

Sintaxis

```
Table.ToColumns(table as table) as list
```

Acerca de

Crea una lista de listas anidadas a partir de la tabla , `table` . Cada elemento de lista es una lista interna que contiene los valores de columna.

Ejemplo 1

Cree una lista de los valores de columna de la tabla.

Uso

Power Query M

```
Table.ToColumns(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"]  
    })  
)
```

Salida

```
{{1, 2}, {"Bob", "Jim"}, {"123-4567", "987-6543"}}
```

Table.ToList

Sintaxis

```
Table.ToList(table as table, optional combiner as nullable function) as list
```

Acerca de

Convierte una tabla en una lista aplicando la función de combinación especificada a cada fila de valores de la tabla.

Ejemplo

Combinar el texto de cada fila con una coma.

Uso

Power Query M

```
Table.ToList(  
    Table.FromRows({  
        {Number.ToText(1), "Bob", "123-4567"},  
        {Number.ToText(2), "Jim", "987-6543"},  
        {Number.ToText(3), "Paul", "543-7890"}  
    }  
),  
    Combiner.CombineTextByDelimiter(",")  
)
```

Salida

```
{"1,Bob,123-4567", "2,Jim,987-6543", "3,Paul,543-7890"}
```

Contenido relacionado

[Funciones de combinador](#)

Table.ToRecords

16/09/2025

Sintaxis

```
Table.ToRecords(table as table) as list
```

Acerca de

Convierte una tabla, `table`, en una lista de registros.

Ejemplo 1

Convierta la tabla en una lista de registros.

Uso

Power Query M

```
Table.ToRecords(  
    Table.FromRows(  
        {  
            {1, "Bob", "123-4567"},  
            {2, "Jim", "987-6543"},  
            {3, "Paul", "543-7890"}  
        },  
        {"CustomerID", "Name", "Phone"}  
    )  
)
```

Salida

Power Query M

```
{  
    [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
    [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
    [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
}
```

Table.ToRows

Syntax

```
Table.ToRows(table as table) as list
```

Acerca de

Crea una lista de listas anidadas a partir de la tabla , `table`. Cada elemento de lista es una lista interna que contiene los valores de fila.

Ejemplo

Cree una lista de los valores de fila de la tabla.

Uso

Power Query M

```
Table.ToRows(  
    Table.FromRecords({  
        [CustomerID = 1, Name = "Bob", Phone = "123-4567"],  
        [CustomerID = 2, Name = "Jim", Phone = "987-6543"],  
        [CustomerID = 3, Name = "Paul", Phone = "543-7890"]  
    })  
)
```

Salida

Power Query M

```
{  
    {1, "Bob", "123-4567"},  
    {2, "Jim", "987-6543"},  
    {3, "Paul", "543-7890"}  
}
```

Table.TransformColumnNames

Syntax

```
Table.TransformColumnNames(  
    table as table,  
    nameGenerator as function,  
    optional options as nullable record  
) as table
```

Acerca de

Transforma los nombres de columna mediante la función especificada `nameGenerator`.

Opciones válidas:

`MaxLength` especifica la longitud máxima de los nombres de columna nuevos. Si la función dada da como resultado un nombre de columna más largo, se recortará el nombre largo.

`Comparer` se usa para controlar la comparación al generar nuevos nombres de columna. Los comparadores se pueden usar para proporcionar comparaciones que no distinguen mayúsculas de minúsculas o de referencia cultural y de configuración regional.

Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- `Comparer.Ordinal`: se usa para realizar una comparación ordinal exacta.
- `Comparer.OrdinalIgnoreCase`: se usa para realizar una comparación exacta sin distinción entre mayúsculas y minúsculas
- `Comparer.FromCulture`: se usa para realizar una comparación que tenga en cuenta la referencia cultural.

Ejemplo 1

Quitar el `#(tab)` carácter de los nombres de columna

Uso

Power Query M

```
Table.TransformColumnNames(Table.FromRecords({["Col#(tab)umn" = 1]}), Text.Clean)
```

Salida

```
Table.FromRecords({[Column = 1]})
```

Ejemplo 2

Transforme los nombres de columna para generar nombres que no distinguen mayúsculas de minúsculas de longitud 6.

Uso

Power Query M

```
Table.TransformColumnNames(  
    Table.FromRecords({[ColumnNum = 1, cOlumnum = 2, coLumnNUM = 3]}),  
    Text.Clean,  
    [MaxLength = 6, Comparer = Comparer.OrdinalIgnoreCase]  
)
```

Salida

```
Table.FromRecords({[Column = 1, cOlum1 = 2, coLum2 = 3]})
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Table.TransformColumns

Sintaxis

```
Table.TransformColumns(  
    table as table,  
    transformOperations as list,  
    optional defaultTransformation as nullable function,  
    optional missingField as nullable number  
) as table
```

Acerca de

Transforma la tabla especificada aplicando cada operación de columna en una lista.

- **table**: tabla que se va a transformar.
- **transformOperations**: las transformaciones que se van a realizar en la tabla. El formato de este parámetro es { nombre de columna, transformación } o { nombre de columna, transformación, nuevo tipo de columna }.
- **defaultTransformation**: (Opcional) Transformación predeterminada aplicada a todas las columnas que no aparecen en **transformOperations**.
- **missingField**: (Opcional) Especifica la acción esperada para los valores que faltan. Si no existe una columna enumerada en **transformOperations**, se genera un error (**MissingField.Error**) a menos que este parámetro especifique una alternativa. Use uno de los siguientes valores:
 - **MissingField.UseNull**: los campos que faltan se incluyen como **null** valores.
 - **MissingField.Ignore**: se omiten los campos que faltan.

Ejemplo 1

Convierta los valores de texto de la columna [A] en valores numéricos y los valores numéricos de la columna [B] en valores de texto.

Uso

Power Query M

```
Table.TransformColumns(  
    Table.FromRecords({  
        [A = "1", B = 2],  
        [A = "5", B = 10]  
    })
```



```

    }),
    {
        {"A", Number.FromText},
        {"B", Text.From}
    }
)

```

Salida

Power Query M

```

Table.FromRecords({
    [A = 1, B = "2"],
    [A = 5, B = "10"]
})

```

Ejemplo 2

Convierta los valores numéricos de la columna que falta [X] en valores de texto, usando `null` por defecto en las columnas que no existen.

Uso

Power Query M

```

Table.TransformColumns(
    Table.FromRecords({
        [A = "1", B = 2],
        [A = "5", B = 10]
    }),
    {"X", Number.FromText},
    null,
    MissingField.UseNull
)

```

Salida

Power Query M

```

Table.FromRecords({
    [A = "1", B = 2, X = null],
    [A = "5", B = 10, X = null]
})

```

Ejemplo 3

Incremente los valores numéricos de la columna [B] y conviértalos en valores de texto y convierta todas las demás columnas en números.

Uso

Power Query M

```
Table.TransformColumns(  
    Table.FromRecords({  
        [A = "1", B = 2],  
        [A = "5", B = 10]  
    }  
),  
{"B", each Text.From(_ + 1), type text},  
Number.FromText  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [A = 1, B = "3"],  
    [A = 5, B = "11"]  
})
```

Ejemplo 4

Mueva las tareas de mantenimiento programadas que se producen en un día festivo de EE. UU. al día siguiente o, si el día festivo se produce el viernes, al próximo lunes.

Uso

Power Query M

```
let  
    MaintenanceSchedule = #table(type table [Task = text, Date = date],  
    {  
        {"HVAC Check", #date(2025, 7, 10)},           // Not a holiday  
        {"Window Washing", #date(2025, 9, 1)},         // Labor Day  
        {"Fire Drill", #date(2025, 9, 17)},            // Not a holiday  
        {"Light Replacement", #date(2025, 11, 27)}     // Thanksgiving  
    }  
),  
    USHolidays = {  
        #date(2025, 1, 1),    // New Year's Day  
        #date(2025, 7, 4),    // Independence Day  
        #date(2025, 9, 1),    // Labor Day  
        #date(2025, 11, 27),  // Thanksgiving  
        #date(2025, 12, 25)   // Christmas  
    },
```

```
AdjustedSchedule = Table.TransformColumns(
    MaintenanceSchedule,
    {"Date", each if List.Contains(USHolidays, _) then
        if Date.DayOfWeek(_, Day.Sunday) = 5 then
            Date.AddDays(_, 3)      // Friday to Monday
        else
            Date.AddDays(_, 1)      // Other to next day
        else _, type date}}
)
in
    AdjustedSchedule
```

Salida

Power Query M

```
#table(type table[Task = text, Date = date],
{
    {"HVAC Check", #date(2025, 7, 10)},
    {"Window Washing", #date(2025, 9, 2)},
    {"Fire Drill", #date(2025, 9, 17)},
    {"Light Replacement", #date(2025, 11, 28)}
})
```

Last updated on 23/01/2026

Table.TransformColumnTypes

Sintaxis

```
Table.TransformColumnTypes(  
    table as table,  
    typeTransformations as list,  
    optional culture as nullable text  
) as table
```

Acercas de

Devuelve una tabla aplicando las operaciones de transformación a las columnas especificadas utilizando una cultura opcional.

- **table**: tabla de entrada que se va a transformar.
- **typeTransformations**: las transformaciones de tipo que se van a aplicar. El formato de una sola transformación es { nombre de columna, valor de tipo }. Se puede usar una lista de transformaciones para cambiar los tipos de más de una columna a la vez. Si no existe una columna, se genera un error.
- **culture**: (Opcional) La cultura que se usará al transformar los tipos de columna (por ejemplo, "en-US"). Si se especifica un registro para **culture**, puede contener los siguientes campos:
 - **Culture**: La cultura a utilizar al transformar los tipos de columna (por ejemplo, "en-US").
 - **MissingField**: si no existe una columna, se produce un error a menos que este campo proporcione un comportamiento alternativo (por ejemplo, [MissingField.UseNull](#) o [MissingField.Ignore](#)).

El valor de tipo del parámetro **typeTransformations** puede ser **any**, todos los tipos de **number**, **text**, todos los **date**, **time**, **datetime**, **datetimezone**, y tipos de **duration**, **logical** o **binary**. Los tipos **list**, **record**, **table** o **function** no son válidos para este parámetro.

Para cada columna enumerada en **typeTransformations**, el método ".From" correspondiente al valor de tipo especificado se utiliza normalmente para realizar la transformación. Por ejemplo, si se da un [Currency.Type](#) valor de tipo para una columna, la función [Currency.From](#) de transformación se aplica a cada valor de esa columna.

Ejemplo 1

Transforme los valores numéricos de la primera columna en valores de texto.

Uso

Power Query M

```
let
    Source = #table(type table [a = number, b = number],
        {
            {1, 2},
            {3, 4}
        }
    ),
    #"Transform Column" = Table.TransformColumnTypes(
        Source,
        {"a", type text}
    )
in
    #"Transform Column"
```

Salida

Power Query M

```
#table(type table [a = text, b = number],
{
    {"1", 2},
    {"3", 4}
})
```

Ejemplo 2

Transforme las fechas de la tabla en sus equivalentes de texto francés.

Uso

Power Query M

```
let
    Source = #table(type table [Company ID = text, Country = text, Date = date],
        {
            {"JS-464", "USA", #date(2024, 3, 24)},
            {"LT-331", "France", #date(2024, 10, 5)},
            {"XE-100", "USA", #date(2024, 5, 21)},
            {"RT-430", "Germany", #date(2024, 1, 18)},
            {"LS-005", "France", #date(2023, 12, 31)},
            {"UW-220", "Germany", #date(2024, 2, 25)}
        }
    )
```

```

    }),
    #"Transform Column" = Table.TransformColumnTypes(
        Source,
        {"Date", type text},
        "fr-FR"
    )
in
    #"Transform Column"

```

Salida

Power Query M

```

#table(type table [Company ID = text, Country = text, Date = text],
{
    {"JS-464", "USA", "24/03/2024"},
    {"LT-331", "France", "05/10/2024"},
    {"XE-100", "USA", "21/05/2024"},
    {"RT-430", "Germany", "18/01/2024"},
    {"LS-005", "France", "31/12/2023"},
    {"UW-220", "Germany", "25/02/2024"}
})

```

Ejemplo 3

Transforme las fechas de la tabla en sus equivalentes de texto alemán y los valores de la tabla en porcentajes.

Uso

Power Query M

```

let
    Source = #table(type table [Date = date, Customer ID = text, Value = number],
        {
            {#date(2024, 3, 12), "134282", .24368},
            {#date(2024, 5, 30), "44343", .03556},
            {#date(2023, 12, 14), "22", .3834}
        }
    ),
    #"Transform Columns" = Table.TransformColumnTypes(
        Source,
        {"Date", type text}, {"Value", Percentage.Type}},
        "de-DE")
in
    #"Transform Columns"

```

Salida

Power Query M

```
#table(type table [Date = text, Customer ID = text, Value = Percentage.Type],
{
    {"12.03.2024", "134282", .24368},
    {"30.05.2024", "44343", .03556},
    {"14.12.2023", "22", .3834}
})
```

Ejemplo 4

Aplique transformaciones con un valor de registro para `culture`.

Uso

Power Query M

```
let
    Source = #table(type table [Company ID = text, Country = text, Date = date],
    {
        {"JS-464", "USA", #date(2024, 3, 24)},
        {"LT-331", "France", #date(2024, 10, 5)},
        {"XE-100", "USA", #date(2024, 5, 21)},
        {"RT-430", "Germany", #date(2024, 1, 18)},
        {"LS-005", "France", #date(2023, 12, 31)},
        {"UW-220", "Germany", #date(2024, 2, 25)}
    }),
    #"Transform Column" = Table.TransformColumnTypes(
        Source,
        {"Date", type text}, {"NewColumn", type number}},
        [Culture="fr-FR", MissingField=MissingField.UseNull]
    )
in
    #"Transform Column"
```

Salida

Power Query M

```
#table(type table [Company ID = text, Country = text, Date = text, NewColumn =
number],
{
    {"JS-464", "USA", "24/03/2024", null},
    {"LT-331", "France", "05/10/2024", null},
    {"XE-100", "USA", "21/05/2024", null},
    {"RT-430", "Germany", "18/01/2024", null},
    {"LS-005", "France", "31/12/2023", null},
    {"UW-220", "Germany", "25/02/2024", null}
})
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)
- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 04/04/2026

Table.TransformRows

16/09/2025

Sintaxis

```
Table.TransformRows(table as table, transform as function) as list
```

Acerca de

Crea un `list` mediante la aplicación de la `transform` operación a cada fila de `table`.

Ejemplo 1

Transforme las filas de una tabla en una lista de números.

Uso

Power Query M

```
Table.TransformRows(  
    Table.FromRecords({  
        [a = 1],  
        [a = 2],  
        [a = 3],  
        [a = 4],  
        [a = 5]  
    }  
),  
    each [a]  
)
```

Salida

```
{1, 2, 3, 4, 5}
```

Ejemplo 2

Transforme las filas de una tabla numérica en registros textuales.

Uso

Power Query M

```
Table.TransformRows(  
    Table.FromRecords({  
        [a = 1],  
        [a = 2],  
        [a = 3],  
        [a = 4],  
        [a = 5]  
    }  
),  
(row) as record => [B = Number.ToText(row[a])]  
)
```

Salida

Power Query M

```
{  
    [B = "1"],  
    [B = "2"],  
    [B = "3"],  
    [B = "4"],  
    [B = "5"]  
}
```

Table.Transpose

Sintaxis

```
Table.Transpose(table as table, optional columns as any) as table
```

Acerca de

Convierte las columnas en filas y filas en columnas.

Ejemplo

Convierta las filas de la tabla de pares nombre-valor en columnas.

Uso

Power Query M

```
Table.Transpose(  
    Table.FromRecords({  
        [Name = "Full Name", Value = "Fred"],  
        [Name = "Age", Value = 42],  
        [Name = "Country", Value = "UK"]  
    })  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [Column1 = "Full Name", Column2 = "Age", Column3 = "Country"],  
    [Column1 = "Fred", Column2 = 42, Column3 = "UK"]  
})
```

Table.Unpivot

Syntax

```
Table.Unpivot(  
    table as table,  
    pivotColumns as list,  
    attributeColumn as text,  
    valueColumn as text  
) as table
```

Acerca de

Convierte un conjunto de columnas de una tabla en pares atributo-valor, combinados con el resto de los valores de cada fila.

Ejemplo 1

Tome las columnas "a", "b" y "c" de la tabla `{{[ key = "x", a = 1, b = null, c = 3 ], [ key = "y", a = 2, b = 4, c = null ]}}` y despivoté en pares de atributo-valor.

Uso

Power Query M

```
Table.Unpivot(  
    Table.FromRecords(  
        [key = "x", a = 1, b = null, c = 3],  
        [key = "y", a = 2, b = 4, c = null]  
    ),  
    {"a", "b", "c"},  
    "attribute",  
    "value"  
)
```

Salida

Power Query M

```
Table.FromRecords(  
    [key = "x", attribute = "a", value = 1],  
    [key = "x", attribute = "c", value = 3],  
    [key = "y", attribute = "a", value = 2],  
    ...  
)
```

```
[key = "y", attribute = "b", value = 4]
})
```

Last updated on 22/11/2025

Table.UnpivotOtherColumns

16/09/2025

Sintaxis

```
Table.UnpivotOtherColumns(  
    table as table,  
    pivotColumns as list,  
    attributeColumn as text,  
    valueColumn as text  
) as table
```

Acerca de

Convierte todas las columnas distintas de un conjunto especificado en pares de atributo-valor, combinadas con el resto de los valores de cada fila.

Ejemplo 1

Convierte todas las columnas distintas de un conjunto especificado en pares de atributo-valor, combinadas con el resto de los valores de cada fila.

Uso

Power Query M

```
Table.UnpivotOtherColumns(  
    Table.FromRecords({  
        [key = "key1", attribute1 = 1, attribute2 = 2, attribute3 = 3],  
        [key = "key2", attribute1 = 4, attribute2 = 5, attribute3 = 6]  
    }),  
    {"key"},  
    "column1",  
    "column2"  
)
```

Salida

Power Query M

```
Table.FromRecords({  
    [key = "key1", column1 = "attribute1", column2 = 1],  
    [key = "key1", column1 = "attribute2", column2 = 2],  
    [key = "key2", column1 = "attribute1", column2 = 4],  
    [key = "key2", column1 = "attribute2", column2 = 5],  
    [key = "key2", column1 = "attribute3", column2 = 6]  
})
```

```
[key = "key1", column1 = "attribute3", column2 = 3],  
[key = "key2", column1 = "attribute1", column2 = 4],  
[key = "key2", column1 = "attribute2", column2 = 5],  
[key = "key2", column1 = "attribute3", column2 = 6]  
})
```

Table.View

16/09/2025

Sintaxis

```
Table.View(table as nullable table, handlers as record) as table
```

Acerca de

Devuelve una vista de `table` dónde se usan las funciones especificadas en `handlers` en lugar del comportamiento predeterminado de una operación cuando se aplica la operación a la vista.

Si `table` se proporciona, todas las funciones del controlador son opcionales. Si no se proporciona `table`, se requieren las funciones de controlador `GetType` y `GetRows`. Si no se especifica una función de controlador para una operación, el comportamiento predeterminado de la operación se aplica en `table` su lugar (excepto en el caso de `GetExpression`).

Las funciones de controlador deben devolver un valor que es semánticamente equivalente al resultado de aplicar la operación en `table` (o la vista resultante en el caso de `GetExpression`).

Si una función de controlador genera un error, el comportamiento predeterminado de la operación se aplica a la vista.

Table.View se puede usar para implementar el plegado en un origen de datos: la traducción de consultas M a consultas específicas del origen (por ejemplo, para crear instrucciones T-SQL a partir de consultas M).

Consulte la documentación publicada del [conector personalizado de Power Query](#) para obtener una descripción más completa de **Table.View**.

Ejemplo 1

Cree una vista básica que no requiera acceso a las filas para determinar el tipo o el recuento de filas.

Uso


```
Table.View(  
    null,  
    [  
        GetType = () => type table [CustomerID = number, Name = text, Phone =  
nullable text],  
        GetRows = () => Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone =  
"123-4567"]}),  
        GetRowCount = () => 1  
    ]  
)
```

Salida

```
Table.FromRecords({[CustomerID = 1, Name = "Bob", Phone = "123-4567"]})
```

Table.ViewError

Syntax

```
Table.ViewError(errorRecord as record) as record
```

Acerca de

Crea un registro de error modificado desde `errorRecord` el que no desencadenará una reserva cuando se genere mediante un controlador definido en una vista (a través de [Table.View](#)).

Last updated on 11/02/2026

Table.ViewFunction

16/09/2025

Sintaxis

```
Table.ViewFunction(function as function) as function
```

Acerca de

Crea una función de vista basada en `function` que puede ser manejada en una vista creada por [Table.View](#).

El controlador `OnInvoke` de [Table.View](#) se puede usar para definir un controlador para la función de vista.

Al igual que sucede con los controladores de las operaciones integradas, si no se especifica ningún controlador `OnInvoke`, si no controla la función de vista o si el controlador genera un error, se aplica `function` en la vista.

Consulte la documentación publicada del [conector personalizado de Power Query](#) para obtener una descripción más completa de las funciones de vista personalizadas **Table.View**.

Table.WithErrorContext

16/09/2025

Sintaxis

```
Table.WithErrorContext(value as any, context as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Tables.GetRelationships

16/09/2025

Sintaxis

```
Tables.GetRelationships(tables as table, optional dataColumn as nullable text) as  
table
```

Acerca de

Obtiene las relaciones entre un conjunto de tablas. Se supone que el conjunto `tables` tiene una estructura similar a la de una tabla de navegación. La columna definida por `dataColumn` contiene las tablas de datos reales.

#table

Syntax

```
#table(columns as any, rows as any) as any
```

Acerca de

Crea un valor de tabla a partir de `columns` y `rows`. El `columns` valor puede ser una lista de nombres de columna, un tipo de tabla, un número de columnas o null. El `rows` valor es una lista de listas, donde cada elemento contiene los valores de columna de una sola fila.

Ejemplo 1

Cree una tabla vacía.

Uso

Power Query M

```
#table({}, {})
```

Salida

```
#table({}, {})
```

Ejemplo 2

Cree una tabla inferiendo el número de columnas de la primera fila.

Uso

Power Query M

```
#table(null, {"Betty", 90.3}, {"Carl", 89.5}))
```

Salida

```
#table({"Column1", "Column2"}, {"Betty", 90.3}, {"Carl", 89.5}))
```

Ejemplo 3

Cree una tabla especificando el número de columnas.

Uso

Power Query M

```
#table(2, {"Betty", 90.3}, {"Carl", 89.5})
```

Salida

```
#table({"Column1", "Column2"}, {"Betty", 90.3}, {"Carl", 89.5})
```

Ejemplo 4

Cree una tabla proporcionando una lista de nombres de columna.

Uso

Power Query M

```
#table({"Name", "Score"}, {"Betty", 90.3}, {"Carl", 89.5})
```

Salida

```
#table({"Name", "Score"}, {"Betty", 90.3}, {"Carl", 89.5})
```

Ejemplo 5

Cree una tabla con un tipo explícito.

Uso

Power Query M

```
#table(type table [Name = text, Score = number], {"Betty", 90.3}, {"Carl", 89.5})
```

Salida

```
#table(type table [Name = text, Score = number], {"Betty", 90.3}, {"Carl", 89.5})
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Last updated on 03/04/2026

Funciones de texto

09/06/2025

Estas funciones crean y manipulan valores de texto.

Información

 Expandir tabla

Nombre	Descripción
Text.InferNumberType	Deduca el tipo de número granular (<code>Int64.Type</code> , <code>Double.Type</code> , etc.) de un número codificado en texto.
Text.Length	Devuelve el número de caracteres de un valor de texto.

Comparaciones de texto

 Expandir tabla

Nombre	Descripción
Character.FromNumber	Convierte un número en un carácter de texto.
Character.ToNumber	Convierte un carácter en su valor numérico.
Guid.From	Devuelve un valor GUID a partir de un valor determinado.
Json.FromValue	Produce una representación JSON de un valor determinado.
Text.From	Crea un valor de texto a partir del valor dado.
Text.FromBinary	Descodifica los datos de un valor binario en un valor de texto mediante una codificación.
Text.NewGuid	Devuelve un valor GUID como un valor de texto.
Text.ToBinary	Codifica un valor de texto en un valor binario mediante una codificación.
Text.ToList	Devuelve la lista de caracteres de un valor de texto.
Value.FromText	Crea un valor con establecimiento inflexible de tipos a partir de una representación de texto.

Extracción

 Expandir tabla

Nombre	Descripción
Text.At	Devuelve un carácter que comienza en un desplazamiento de base cero.
Text.Middle	Devuelve una subcadena hasta una longitud concreta.
Text.Range	Devuelve un número de caracteres determinado de un valor de texto a partir de un desplazamiento de base cero.
Text.Start	Devuelve el recuento de caracteres desde el principio de un valor de texto.
Text.End	Devuelve el número de caracteres desde el final de un valor de texto.

Modificación

 Expandir tabla

Nombre	Descripción
Text.Insert	Inserta un valor de texto en otro en una posición especificada.
Text.Remove	Quita todas las coincidencias del carácter o de la lista de caracteres especificados del valor de texto de entrada.
Text.RemoveRange	Quita los caracteres de recuento en un desplazamiento de base cero de un valor de texto.
Text.Replace	Reemplaza todas las apariciones de una subcadena por un valor de texto nuevo.
Text.ReplaceRange	Reemplaza los caracteres de longitud de un valor de texto a partir de un desplazamiento de base cero por un nuevo valor de texto.
Text.Select	Selecciona todas las coincidencias del carácter o de la lista de caracteres especificados en el valor de texto de entrada.

Pertenencia

 Expandir tabla

Nombre	Descripción
Text.Contains	Devuelve <code>true</code> si se encontró una subcadena de valor de texto dentro de una cadena de valor de texto; de lo contrario, <code>false</code> .
Text.EndsWith	Devuelve un valor lógico que indica si se ha encontrado una subcadena de valor de texto al final de una cadena.
Text.PositionOf	Devuelve la primera posición del valor (- 1 si no se encuentra).
Text.PositionOfAny	Devuelve la primera posición en el valor de texto de cualquier carácter enumerado (- 1 si no se encuentra).
Text.StartsWith	Devuelve un valor lógico que indica si se ha encontrado una subcadena de valor de texto al principio de una cadena.

Transformaciones

[Expandir tabla](#)

Nombre	Descripción
Text.AfterDelimiter	Devuelve la parte de texto que hay después del delimitador especificado.
Text.BeforeDelimiter	Devuelve la parte de texto que hay antes del delimitador especificado.
Text.BetweenDelimiters	Devuelve la parte del texto entre el especificado <code>startDelimiter</code> y <code>endDelimiter</code> .
Text.Clean	Devuelve el valor de texto original con todos los caracteres no imprimibles quitados.
Text.Combine	Devuelve un valor de texto que es el resultado de combinar todos los valores de texto en uno y separarlos con un separador.
Text.Lower	Devuelve un valor de texto en minúscula.
Text.PadEnd	Devuelve texto de una longitud especificada rellenando el final del texto dado.
Text.PadStart	Devuelve texto de una longitud especificada rellenando el comienzo del texto dado.
Text.Proper	Devuelve un valor de texto con la primera letra de cada palabra en mayúscula.
Text.Repeat	Devuelve un valor de texto compuesto por el valor de texto de entrada repetido un determinado número de veces.
Text.Reverse	Invierte el texto proporcionado.

Nombre	Descripción
Text.Split	Devuelve una lista que contiene las partes de un valor de texto que están delimitadas por un separador concreto.
Text.SplitAny	Devuelve una lista que contiene las partes de un valor de texto que están delimitadas por cualquier separador.
Text.Trim	Quita todos los caracteres iniciales y finales especificados.
Text.TrimEnd	Quita todos los caracteres finales especificados.
Text.TrimStart	Quita todos los caracteres iniciales especificados.
Text.Upper	Devuelve un valor de texto en mayúscula.

Character.FromNumber

Syntax

```
Character.FromNumber(number as nullable number) as nullable text
```

Acerca de

Devuelve el carácter equivalente del número.

El proporcionado `number` debe ser un punto de código Unicode de 21 bits.

Ejemplo 1

Convierta un número en su valor de carácter equivalente.

Uso

Power Query M

```
Character.FromNumber(9)
```

Salida

```
"#(tab)"
```

Ejemplo 2

Convierta un carácter en un número y vuelva a volver.

Uso

Power Query M

```
Character.FromNumber(Character.ToNumber("A"))
```

Salida

```
"A"
```

Ejemplo 3

Convierta el punto de código hexadecimal para el emoticono "cara de grinning" en su par suplente UTF-16 equivalente.

Uso

Power Query M

```
Character.FromNumber(0x1F600)
```

Salida

```
"#(0001F600)"
```

Last updated on 22/11/2025

Character.ToNumber

Sintaxis

```
Character.ToNumber(character as nullable text) as nullable number
```

Acerca de

Devuelve el número equivalente de `.character`

El resultado será el punto de código Unicode de 21 bits representado por el carácter proporcionado o el par suplente.

Ejemplo 1

Convierta un carácter en su valor numérico equivalente.

Uso

Power Query M

```
Character.ToNumber("#(tab)")
```

Salida

9

Ejemplo 2

Convierta el par sustituto UTF-16 para el emoticono "cara sonriente" a su punto de código hexadecimal equivalente.

Uso

Power Query M

```
Number.ToText(Character.ToNumber("#(0001F600)"), "X")
```

Salida

"1F600"

Last updated on 04/04/2026

Guid.From

Syntax

```
Guid.From(value as nullable text) as nullable text
```

Acerca de

Devuelve un `Guid.Type` valor de la clase especificada `value`. Si el objeto especificado `value` es `null`, **Guid.From** devuelve `null`. Se realizará una comprobación para determinar si el especificado `value` tiene un formato aceptable. Formatos aceptables proporcionados en los ejemplos.

Ejemplo 1

El GUID se puede proporcionar como 32 dígitos hexadecimales contiguos.

Uso

Power Query M

```
Guid.From("05FE1DADC8C24F3BA4C2D194116B4967")
```

Salida

```
"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"
```

Ejemplo 2

El GUID se puede proporcionar como 32 dígitos hexadecimales separados por guiones en bloques de 8-4-4-4-4-12.

Uso

Power Query M

```
Guid.From("05FE1DAD-C8C2-4F3B-A4C2-D194116B4967")
```

Salida

"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"

Ejemplo 3

El GUID se puede proporcionar como 32 dígitos hexadecimales separados por guiones y entre llaves.

Uso

Power Query M

```
Guid.From("{05FE1DAD-C8C2-4F3B-A4C2-D194116B4967}")
```

Salida

"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"

Ejemplo 4

El GUID se puede proporcionar como 32 dígitos hexadecimales separados por guiones y entre paréntesis.

Uso

Power Query M

```
Guid.From("(05FE1DAD-C8C2-4F3B-A4C2-D194116B4967)")
```

Salida

"05fe1dad-c8c2-4f3b-a4c2-d194116b4967"

Last updated on 22/11/2025

Json.FromValue

Syntax

```
Json.FromValue(value as any, optional encoding as nullable number) as binary
```

Acerca de

Genera una representación JSON de un valor `value` determinado con una codificación de texto especificada por `encoding`. Si `encoding` se omite, se usa UTF8. Los valores se representan de la manera siguiente:

- Los valores NULL, text y lógicos se representan como los tipos JSON correspondientes.
- Los números se representan como números en JSON, excepto que `#infinity`, `-#infinity` y `#nan` se convierten en NULL.
- Las listas se representan como matrices JSON
- Los registros se representan como objetos JSON
- Las tablas se representan como una matriz de objetos
- Las fechas, las horas, las horas, las datetimezones y las duraciones se representan como texto ISO-8601
- Los valores binarios se representan como texto codificado en base 64
- Los tipos y funciones producen un error

Ejemplo

Convierta un valor complejo en JSON.

Uso

Power Query M

```
Text.FromBinary(Json.FromValue([A = {1, true, "3"}, B = #date(2012, 3, 25)]))
```

Salida

```
"{"A": [1, true, "3"], "B": "2012-03-25"}"
```

Text.AfterDelimiter

Syntax

```
Text.AfterDelimiter(  
    text as nullable text,  
    delimiter as text,  
    optional index as any  
) as any
```

Acerca de

Devuelve la parte de después del `text` especificado `delimiter`. Un valor numérico opcional `index` indica qué instancia de `delimiter` debe tenerse en cuenta. Una lista `index` opcional indica qué aparición `delimiter` debe tenerse en cuenta, así como si se debe realizar la indexación desde el inicio o el final de la entrada.

Ejemplo 1

Obtenga la parte de "111-222-333" después del guion (primero).

Uso

Power Query M

```
Text.AfterDelimiter("111-222-333", "-")
```

Salida

"222-333"

Ejemplo 2

Obtenga la parte de "111-222-333" después del segundo guión.

Uso

Power Query M

```
Text.AfterDelimiter("111-222-333", "-", 1)
```

Salida

"333"

Ejemplo 3

Obtenga la parte de "111-222-333" después del segundo guión del final.

Uso

Power Query M

```
Text.AfterDelimiter("111-222-333", "-", {1, RelativePosition.FromEnd})
```

Salida

"222-333"

Last updated on 22/11/2025

Text.At

Syntax

```
Text.At(text as nullable text, index as number) as nullable text
```

Acerca de

Devuelve el carácter del valor de texto, **text** en la posición **index**. El primer carácter del texto está en la posición 0.

Ejemplo 1

Busque el carácter en la posición 4 de la cadena "Hello, World".

Uso

Power Query M

```
Text.At("Hello, World", 4)
```

Salida

```
"o"
```

Last updated on 22/11/2025

Text.BeforeDelimiter

Syntax

```
Text.BeforeDelimiter(  
    text as nullable text,  
    delimiter as text,  
    optional index as any  
) as any
```

Acerca de

Devuelve la parte de antes del `text` especificado `delimiter`. Un valor numérico opcional `index` indica qué instancia de `delimiter` debe tenerse en cuenta. Una lista `index` opcional indica qué aparición `delimiter` debe tenerse en cuenta, así como si se debe realizar la indexación desde el inicio o el final de la entrada.

Ejemplo 1

Obtenga la parte de "111-222-333" antes del guion (primero).

Uso

Power Query M

```
Text.BeforeDelimiter("111-222-333", "-")
```

Salida

"111"

Ejemplo 2

Obtenga la parte de "111-222-333" antes del segundo guión.

Uso

Power Query M

```
Text.BeforeDelimiter("111-222-333", "-", 1)
```

Salida

"111-222"

Ejemplo 3

Obtenga la parte de "111-222-333" antes del segundo guion del final.

Uso

Power Query M

```
Text.BeforeDelimiter("111-222-333", "-", {1, RelativePosition.FromEnd})
```

Salida

"111"

Last updated on 22/11/2025

Text.BetweenDelimiters

16/09/2025

Sintaxis

```
Text.BetweenDelimiters(  
    text as nullable text,  
    startDelimiter as text,  
    endDelimiter as text,  
    optional startIndex as any,  
    optional endIndex as any  
) as any
```

Acercade

Devuelve la parte de `text` entre el especificado `startDelimiter` y `endDelimiter`. Un valor numérico opcional `startIndex` indica qué instancia de `startDelimiter` debe tenerse en cuenta. Una lista `startIndex` opcional indica qué aparición `startDelimiter` debe tenerse en cuenta, así como si se debe realizar la indexación desde el inicio o el final de la entrada. `endIndex` es similar, excepto que la indexación se realiza en relación con `startIndex`.

Ejemplo 1

Obtenga la parte de "111 (222) 333 (444)" entre el paréntesis abierto (primero) y el paréntesis cerrado (primero) que sigue.

Uso

Power Query M

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")")
```

Salida

"222"

Ejemplo 2

Obtenga la parte de "111 (222) 333 (444)" entre el segundo paréntesis abierto y el primer paréntesis cerrado que le sigue.

Uso

Power Query M

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")", 1, 0)
```

Salida

```
"444"
```

Ejemplo 3

Obtenga la parte de "111 (222) 333 (444)" entre el segundo paréntesis abierto desde el final y el segundo paréntesis cerrado que sigue.

Uso

Power Query M

```
Text.BetweenDelimiters("111 (222) 333 (444)", "(", ")", {1, RelativePosition.FromEnd}, {1, RelativePosition.FromStart})
```

Salida

```
"222) 333 (444"
```

Text.Clean

Syntax

```
Text.Clean(text as nullable text) as nullable text
```

Acerca de

Devuelve un valor de texto con todos los caracteres de control de `text` quitados.

Ejemplo

Quite las fuentes de línea y otros caracteres de control de un valor de texto.

Uso

Power Query M

```
Text.Clean("ABC#(lf)D")
```

Salida

```
"ABCD"
```

Last updated on 25/03/2026

Text.Combine

16/09/2025

Sintaxis

```
Text.Combine(texts as list, optional separator as nullable text) as text
```

Acerca de

Devuelve el resultado de combinar la lista de valores de texto, `texts`, en un solo valor de texto. Se omiten los valores de `null` presentes en `texts`. Se puede especificar un `separator` opcional usado en el texto combinado final.

Ejemplo 1

Combinar los valores de texto "Seattle" y "WA".

Uso

Power Query M

```
Text.Combine({"Seattle", "WA"})
```

Salida

```
"SeattleWA"
```

Ejemplo 2

Combine los valores de texto "Seattle" y "WA" separados por una coma y un espacio.

Uso

Power Query M

```
Text.Combine({"Seattle", "WA"}, ", ")
```

Salida

"Seattle, WA"

Ejemplo 3

Combine los valores "Seattle", `null` y "WA" separados por una coma y un espacio (tenga en cuenta que `null` se omite).

Uso

Power Query M

```
Text.Combine({"Seattle", null, "WA"}, ", ")
```

Salida

"Seattle, WA"

Ejemplo 4

Uso

Combine el nombre, el segundo inicial (si está presente) y el apellido en el nombre completo del individuo.

Power Query M

```
let
    Source = Table.FromRecords({
        [First Name = "Doug", Middle Initial = "J", Last Name = "Elis"],
        [First Name = "Anna", Middle Initial = "M", Last Name = "Jorayew"],
        [First Name = "Rada", Middle Initial = null, Last Name = "Mihaylova"]
    }),
    FullName = Table.AddColumn(Source, "Full Name", each Text.Combine({[First Name], [Middle Initial], [Last Name]}, " "))
in
    FullName
```

Salida

Power Query M

```
Table.FromRecords({
    [First Name = "Doug", Middle Initial = "J", Last Name = "Elis", Full Name = "Doug J Elis"],
    [First Name = "Anna", Middle Initial = "M", Last Name = "Jorayew", Full Name = "Anna M Jorayew"],
    [First Name = "Rada", Middle Initial = null, Last Name = "Mihaylova", Full Name = "Rada Mihaylova"]
})
```

```
[First Name = "Rada", Middle Initial = null, Last Name = "Mihaylova", Full  
Name = "Rada Mihaylova"]  
})
```

Text.Contains

Sintaxis

```
Text.Contains(  
    text as nullable text,  
    substring as text,  
    optional comparer as nullable function  
) as nullable logical
```

Acerca de

Detecta si `text` contiene el valor `substring`. Devuelve true si se encuentra el valor. Esta función no admite caracteres comodín ni expresiones regulares.

Se puede usar el argumento opcional `comparer` para especificar comparaciones que son compatibles con referencia cultural y configuración regional o que no distinguen mayúsculas de minúsculas. Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- [Comparer.Ordinal](#): se usa para realizar una comparación ordinal que distingue mayúsculas de minúsculas.
- [Comparer.OrdinalIgnoreCase](#): se usa para realizar una comparación ordinal sin distinción entre mayúsculas y minúsculas.
- [Comparer.FromCulture](#): se usa para realizar una comparación que tenga en cuenta la referencia cultural.

Si el primer argumento es null, esta función devuelve null.

Todos los caracteres se tratan literalmente. Por ejemplo, "DR", " DR", "DR " y " DR " no se consideran iguales entre sí.

Ejemplo 1

Buscar si el texto "Hola mundo" contiene "Hola".

Uso

Power Query M

```
Text.Contains("Hello World", "Hello")
```

Salida

```
true
```

Ejemplo 2

Buscar si el texto "Hola mundo" contiene "hola".

Uso

Power Query M

```
Text.Contains("Hello World", "hello")
```

Salida

```
false
```

Ejemplo 3

Permite buscar si el texto "Hola mundo" contiene "hola", usando un comparador que no distingue mayúsculas de minúsculas.

Uso

Power Query M

```
Text.Contains("Hello World", "hello", Comparer.OrdinalIgnoreCase)
```

Salida

```
true
```

Ejemplo 4

Busque las filas de una tabla que contengan "A-" o "7" en el código de cuenta.

Uso

Power Query M

```
let
    Source = #table(type table [Account Code = text, Posted Date = date, Sales = number],
```



```

{
    {"US-2004", #date(2023,1,20), 580},
    {"CA-8843", #date(2023,7,18), 280},
    {"PA-1274", #date(2022,1,12), 90},
    {"PA-4323", #date(2023,4,14), 187},
    {"US-1200", #date(2022,12,14), 350},
    {"PTY-507", #date(2023,6,4), 110}
}),
#"Filtered rows" = Table.SelectRows(
    Source,
    each Text.Contains([Account Code], "A-") or
        Text.Contains([Account Code], "7"))
in
    #"Filtered rows"

```

Salida

Power Query M

```

#table(type table [Account Code = text, Posted Date = date, Sales = number],
{
    {"CA-8843", #date(2023,7,18), 280},
    {"PA-1274", #date(2022,1,12), 90},
    {"PA-4323", #date(2023,4,14), 187},
    {"PTY-507", #date(2023,6,4), 110}
})

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Text.End

Syntax

```
Text.End(text as nullable text, count as number) as nullable text
```

Acerca de

Devuelve un `text` valor que es los últimos `count` caracteres del `text` valor `text`.

Ejemplo 1

Obtenga los últimos 5 caracteres del texto "Hello, World".

Uso

Power Query M

```
Text.End("Hello, World", 5)
```

Salida

```
"World"
```

Last updated on 22/11/2025

Text.EndsWith

Syntax

```
Text.EndsWith(  
    text as nullable text,  
    substring as text,  
    optional comparer as nullable function  
) as nullable logical
```

Acerca de

Indica si el texto especificado, `text`, finaliza con el valor especificado, `substring`. La indicación distingue mayúsculas de minúsculas.

`comparer` es un `Comparer` objeto que se usa para controlar la comparación. Los comparadores se pueden usar para proporcionar comparaciones que no distinguen mayúsculas de minúsculas o de referencia cultural y de configuración regional.

Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- [Comparer.Ordinal](#): se usa para realizar una comparación ordinal exacta.
- [Comparer.OrdinalIgnoreCase](#): se usa para realizar una comparación exacta sin distinción entre mayúsculas y minúsculas
- [Comparer.FromCulture](#): se usa para realizar una comparación que tenga en cuenta la referencia cultural.

Ejemplo 1

Compruebe si "Hello, World" termina con "world".

Uso

Power Query M

```
Text.EndsWith("Hello, World", "world")
```

Salida

false

Ejemplo 2

Compruebe si "Hello, World" termina con "World".

Uso

Power Query M

```
Text.EndsWith("Hello, World", "World")
```

Salida

true

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Text.Format

16/09/2025

Sintaxis

```
Text.Format(  
    formatString as text,  
    arguments as any,  
    optional culture as nullable text  
) as text
```

Acerca de

Devuelve texto con formato que se crea aplicando `arguments` de una lista o registro a una cadena de formato `formatString`. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Dar formato a una lista de números.

Uso

Power Query M

```
Text.Format("#{0}, #{1}, and #{2}.", {17, 7, 22})
```

Salida

```
"17, 7, and 22."
```

Ejemplo 2

Dar formato a diferentes tipos de datos de un registro según la referencia cultural inglesa de Estados Unidos.

Uso

Power Query M

```
Text.Format(
    "The time for the #[distance] km run held in #[city] on #[date] was #
    [duration].",
    [
        city = "Seattle",
        date = #date(2015, 3, 10),
        duration = #duration(0, 0, 54, 40),
        distance = 10
    ],
    "en-US"
)
```

Salida

```
"The time for the 10 km run held in Seattle on 3/10/2015 was 00:54:40."
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Text.From

16/09/2025

Sintaxis

```
Text.From(value as any, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve la representación de texto de un valor especificado.

- `value`: valor que se va a convertir en texto. El valor puede ser un `number`, `date`, `time`, `datetime`, `datetimezone`, `logical`, `duration` o `binary` valor. Si el valor especificado es `null`, esta función devuelve `null`.
- `culture`: (Opcional) Referencia cultural que se va a usar al convertir el valor en texto (por ejemplo, "en-US").

Ejemplo 1

Cree un valor de texto a partir del número 3.

Uso

```
Power Query M
```

```
Text.From(3)
```

Salida

```
"3"
```

Ejemplo 2

Obtiene el texto equivalente de la fecha y hora especificadas.

Uso

```
Power Query M
```

```
Text.From(#datetime(2024, 6, 24, 14, 32, 22))
```

Salida

```
"6/24/2024 2:32:22 PM"
```

Ejemplo 3

Obtiene el texto alemán equivalente a la fecha y hora especificadas.

Uso

Power Query M

```
Text.From(#datetime(2024, 6, 24, 14, 32, 22), "de-DE")
```

Salida

```
"24.06.2024 14:32:22"
```

Ejemplo 4

Obtenga un valor binario desde un texto codificado como hexadecimal y convierta el valor de nuevo a texto.

Uso

Power Query M

```
Text.From(Binary.FromText("10FF", BinaryEncoding.Hex))
```

Salida

```
"EP8="
```

Ejemplo 5

Obtenga las filas de la tabla que contienen datos para Francia y convierta las fechas en texto mediante la referencia cultural francesa.

Uso


```

let
    Source = #table(type table [Company ID = text, Country = text, Date = date],
        {
            {"JS-464", "USA", #date(2024, 3, 24)},
            {"LT-331", "France", #date(2024, 10, 5)},
            {"XE-100", "USA", #date(2024, 5, 21)},
            {"RT-430", "Germany", #date(2024, 1, 18)},
            {"LS-005", "France", #date(2023, 12, 31)},
            {"UW-220", "Germany", #date(2024, 2, 25)}
        }
    ),
    #"Convert Dates" = Table.TransformColumns(
        Table.SelectRows(Source, each [Country] = "France"),
        {"Date", each Text.From(_, "fr-FR")}
    )
in
    #"Convert Dates"

```

Salida

```

#table(type table [Company ID = text, Country = text, Date = text],
{
    {"LT-331", "France", "05/10/2024"},
    {"LS-005", "France", "31/12/2023"}
})

```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [cadenas de formato numérico estándar](#)
- [cadenas de formato numérico personalizado](#)
- [cadenas de formato de fecha y hora estándar](#)
- [Cadenas con formato de fecha y hora personalizado](#)

Text.FromBinary

Sintaxis

```
Text.FromBinary(binary as nullable binary, optional encoding as nullable number) as nullable text
```

Acerca de

Descodifica los datos de un valor binario a un valor de texto mediante el tipo de codificación especificado.

- **binary**: los datos binarios que se van a descodificar.
- **encoding**: (Opcional) Codificación usada para convertir el binario en texto. Use [BinaryEncoding.Type](#) para especificar el tipo de codificación. Si no se especifica este valor, el valor predeterminado es `BinaryEncoding.Utf8`.

Ejemplo 1

Codificar texto en binario, generar una cadena Base64 visible y, a continuación, descodificarla de nuevo en texto.

Uso

Power Query M

```
let
    originalText = "Testing 1-2-3",

    // Default UTF-8 binary
    binaryData = Text.ToBinary(originalText),

    // Convert binary to viewable Base64 string
    encodedText = Binary.ToText(binaryData, BinaryEncoding.Base64),

    // Decode back to text
    decodedText = Text.FromBinary(binaryData),

    result = [
        OriginalText = originalText,
        BinaryBase64 = encodedText,
        DecodedText = decodedText
    ]
```

```
in
    result
```

Salida

Power Query M

```
[
    OriginalText = "Testing 1-2-3",
    BinaryEncoded = "VGZzdGluZyAxLTItMw==",
    DecodedText = "Testing 1-2-3"
]
```

Ejemplo 2

Codificar texto en binario con una marca de orden de bytes (BOM), generar una cadena hexadecimal visible y, a continuación, decodificarla de nuevo en texto.

Uso

Power Query M

```
let
    originalText = "Testing 1-2-3",

    // Convert to binary with BOM
    binaryData = Text.ToBinary(originalText, TextEncoding.Utf16, true),

    // Show binary as hex to demonstrate presence of BOM (fffe)
    binaryAsHex = Binary.ToText(binaryData, BinaryEncoding.Hex),

    // Decode back to text
    decodedText = Text.FromBinary(binaryData, TextEncoding.Utf16),

    // Compare original text and decoded text
    isIdentical = originalText = decodedText,

    result = [
        OriginalText = originalText,
        BinaryHex = binaryAsHex,
        DecodedText = decodedText,
        IsIdentical = isIdentical
    ]
in
    result
```

Salida

Power Query M

```
[
  OriginalText = "Testing 1-2-3",
  DecodedText = "fffe540065007300740069006e006700200031002d0032002d003300",
  DecodedText = "Testing 1-2-3",
  IsIdentical = true
]
```

Last updated on 28/10/2025

Text.InferNumberType

Syntax

```
Text.InferNumberType(text as text, optional culture as nullable text) as type
```

Acerca de

Deduce el tipo de número granular (`Int64.Type`, `Double.Type`, etc.) de `text`. Se produce un error si `text` no es un número. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Text.Insert

Syntax

```
Text.Insert(  
    text as nullable text,  
    offset as number,  
    newText as text  
) as nullable text
```

Acerca de

Devuelve el resultado de insertar el valor `newText` de texto en el valor `text` de texto en la posición `offset`. Las posiciones comienzan en el número 0.

Ejemplo 1

Inserte "C" entre "B" y "D" en "ABD".

Uso

Power Query M

```
Text.Insert("ABD", 2, "C")
```

Salida

"ABCD"

Last updated on 22/11/2025

Text.Length

Sintaxis

```
Text.Length(text as nullable text) as nullable number
```

Acerca de

Devuelve el número de caracteres del texto `text`.

Ejemplo

Busque cuántos caracteres hay en el texto "Hola mundo".

Uso

Power Query M

```
Text.Length("Hello World")
```

Salida

11

Last updated on 25/03/2026

Text.Lower

16/09/2025

Sintaxis

```
Text.Lower(text as nullable text, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve el resultado de convertir todos los caracteres en `text` a minúsculas. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Obtenga la versión minúscula de "AbCd".

Uso

Power Query M

```
Text.Lower("AbCd")
```

Salida

```
"abcd"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Text.Middle

16/09/2025

Sintaxis

```
Text.Middle(  
    text as nullable text,  
    start as number,  
    optional count as nullable number  
) as nullable text
```

Acerca de

Devuelve `count` caracteres, o hasta el final de `text`, en el `start` del desplazamiento.

Ejemplo 1

Hallar la subcadena del texto "Hello World", comenzando en el índice 6 y abarcando 5 caracteres.

Uso

Power Query M

```
Text.Middle("Hello World", 6, 5)
```

Salida

```
"World"
```

Ejemplo 2

Hallar la subcadena del texto "Hello World", comenzando en el índice 6 y abarcando hasta el final.

Uso

Power Query M

```
Text.Middle("Hello World", 6, 20)
```

Salida

```
"World"
```

Ejemplo 3

Hallar la subcadena del texto "Hola mundo", comenzando en el índice 0 y abarcando 2 caracteres.

Uso

Power Query M

```
Text.Middle("Hello World", 0, 2)
```

Salida

```
"He"
```

Text.NewGuid

16/09/2025

Sintaxis

```
Text.NewGuid() as text
```

Acerca de

Devuelve un nuevo identificador único global aleatorio (GUID).

Text.PadEnd

Syntax

```
Text.PadEnd(  
    text as nullable text,  
    count as number,  
    optional character as nullable text  
) as nullable text
```

Acerca de

Devuelve un `text` valor relleno hasta la longitud `count` insertando espacios al final del valor `text` de texto . Se puede usar un carácter `character` opcional para especificar el carácter usado para el relleno. El carácter de relleno predeterminado es un espacio.

Ejemplo 1

Rellenar el final de un valor de texto para que tenga 10 caracteres.

Uso

Power Query M

```
Text.PadEnd("Name", 10)
```

Salida

```
"Name      "
```

Ejemplo 2

Rellenar el final de un valor de texto con "|", por lo que tiene 10 caracteres de longitud.

Uso

Power Query M

```
Text.PadEnd("Name", 10, "|")
```

Salida

"Name|||||"

Last updated on 22/11/2025

Text.PadStart

Syntax

```
Text.PadStart(  
    text as nullable text,  
    count as number,  
    optional character as nullable text  
) as nullable text
```

Acerca de

Devuelve un `text` valor relleno hasta la longitud `count` insertando espacios al principio del valor `text` de texto . Se puede usar un carácter `character` opcional para especificar el carácter usado para el relleno. El carácter de relleno predeterminado es un espacio.

Ejemplo 1

Rellenar el inicio de un valor de texto para que tenga 10 caracteres de longitud.

Uso

Power Query M

```
Text.PadStart("Name", 10)
```

Salida

```
"      Name"
```

Ejemplo 2

Rellenar el inicio de un valor de texto con "|", por lo que tiene 10 caracteres de longitud.

Uso

Power Query M

```
Text.PadStart("Name", 10, "|")
```

Salida

"|||||Name"

Last updated on 22/11/2025

Text.PositionOf

Syntax

```
Text.PositionOf(  
    text as text,  
    substring as text,  
    optional occurrence as nullable number,  
    optional comparer as nullable function  
) as any
```

Acerca de

Devuelve la posición de la aparición especificada del valor `substring` de texto que se encuentra en `text`. Se puede usar un parámetro `occurrence` opcional para especificar la posición de repetición que se va a devolver (primera aparición de forma predeterminada). Devuelve -1 si `substring` no se encontró.

`comparer` es un `Comparer` objeto que se usa para controlar la comparación. Los comparadores se pueden usar para proporcionar comparaciones que no distinguen mayúsculas de minúsculas o de referencia cultural y de configuración regional.

Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- `Comparer.Ordinal`: se usa para realizar una comparación ordinal exacta.
- `Comparer.OrdinalIgnoreCase`: se usa para realizar una comparación exacta sin distinción entre mayúsculas y minúsculas
- `Comparer.FromCulture`: se usa para realizar una comparación que tenga en cuenta la referencia cultural.

Ejemplo 1

Obtenga la posición de la primera aparición de "World" en el texto "Hello, World! Hola, mundo!".

Uso

Power Query M

```
Text.PositionOf("Hello, World! Hello, World!", "World")
```


Salida

7

Ejemplo 2

Obtenga la posición de la última aparición de "World" en "Hello, World! Hola, mundo!".

Uso

Power Query M

```
Text.PositionOf("Hello, World! Hello, World!", "World", Occurrence.Last)
```

Salida

21

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 22/11/2025

Text.PositionOfAny

16/09/2025

Sintaxis

```
Text.PositionOfAny(  
    text as text,  
    characters as list,  
    optional occurrence as nullable number  
) as any
```

Acerca de

Devuelve la primera posición de cualquier carácter de la lista `characters` que se encuentra en `text`. Se puede usar un parámetro opcional `occurrence` para especificar qué posición de ocurrencia devolver.

Ejemplo 1

Busque la primera posición de "W" o "H" en el texto "Hello, World!".

Uso

Power Query M

```
Text.PositionOfAny("Hello, World!", {"H", "W"})
```

Salida

0

Ejemplo 2

Busque todas las posiciones de "W" o "H" en el texto "Hello, World!".

Uso

Power Query M

```
Text.PositionOfAny("Hello, World!", {"H", "W"}, Occurrence.All)
```

Salida

{0, 7}

Text.Proper

16/09/2025

Sintaxis

```
Text.Proper(text as nullable text, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve el resultado de poner en mayúscula solo la primera letra de cada palabra en el texto `text`. Todas las demás letras se devuelven en minúsculas. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo 1

Use `Text.Proper` en una oración simple.

Uso

Power Query M

```
Text.Proper("the QUICK BrOWn fOx jUmPs oVER tHe LAzy DoG")
```

Salida

```
"The Quick Brown Fox Jumps Over The Lazy Dog"
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Text.Range

Syntax

```
Text.Range(  
    text as nullable text,  
    offset as number,  
    optional count as nullable number  
) as nullable text
```

Acerca de

Devuelve la subcadena del texto `text` encontrado en el desplazamiento `offset`. Se puede incluir un parámetro opcional, `count`, para especificar cuántos caracteres se van a devolver. Produce un error si no hay suficientes caracteres.

Ejemplo 1

Busque la subcadena del texto "Hola mundo" a partir del índice 6.

Uso

Power Query M

```
Text.Range("Hello World", 6)
```

Salida

"World"

Ejemplo 2

Busque la subcadena del texto "Hello World Hello" a partir del índice 6 que abarca 5 caracteres.

Uso

Power Query M

```
Text.Range("Hello World Hello", 6, 5)
```

Salida

"World"

Last updated on 22/11/2025

Text.Remove

Syntax

```
Text.Remove(text as nullable text, removeChars as any) as nullable text
```

Acerca de

Devuelve una copia del valor `text` de texto con todos los caracteres de `removeChars` quitados.

Ejemplo

Quitar caracteres y ; del valor de texto.

Uso

Power Query M

```
Text.Remove("a,b;c", {"", ",", ";"})
```

Salida

```
"abc"
```

Last updated on 25/03/2026

Text.RemoveRange

Syntax

```
Text.RemoveRange(  
    text as nullable text,  
    offset as number,  
    optional count as nullable number  
) as nullable text
```

Acerca de

Devuelve una copia del valor `text` de texto con todos los caracteres de la posición `offset` quitada. Un parámetro opcional, `count` se puede usar para especificar el número de caracteres que se van a quitar. El valor predeterminado de `count` es 1. Los valores de posición comienzan en 0.

Ejemplo 1

Quite 1 carácter del valor de texto "ABEFC" en la posición 2.

Uso

Power Query M

```
Text.RemoveRange("ABEFC", 2)
```

Salida

"ABFC"

Ejemplo 2

Quite dos caracteres del valor de texto "ABEFC" a partir de la posición 2.

Uso

Power Query M

```
Text.RemoveRange("ABEFC", 2, 2)
```


Salida

"ABC"

Last updated on 22/11/2025

Text.Repeat

16/09/2025

Sintaxis

```
Text.Repeat(text as nullable text, count as number) as nullable text
```

Acerca de

Devuelve un valor de texto compuesto por el texto de entrada `text` repetido `count` veces.

Ejemplo 1

Repita el texto "a" cinco veces.

Uso

Power Query M

```
Text.Repeat("a", 5)
```

Salida

```
"aaaaa"
```

Ejemplo 2

Repita el texto "helloworld" tres veces.

Uso

Power Query M

```
Text.Repeat("helloworld.", 3)
```

Salida

```
"helloworld.helloworld.helloworld."
```

Text.Replace

Syntax

```
Text.Replace(  
    text as nullable text,  
    old as text,  
    new as text  
) as nullable text
```

Acerca de

Devuelve el resultado de reemplazar todas las apariciones del valor `old` de texto en el valor `text` de texto por el valor `new` de texto . Esta función distingue mayúsculas de minúsculas.

Ejemplo 1

Reemplace todas las apariciones de "the" en una oración por "a".

Uso

Power Query M

```
Text.Replace("the quick brown fox jumps over the lazy dog", "the", "a")
```

Salida

```
"a quick brown fox jumps over a lazy dog"
```

Last updated on 22/11/2025

Text.ReplaceRange

16/09/2025

Sintaxis

```
Text.ReplaceRange(  
    text as nullable text,  
    offset as number,  
    count as number,  
    newText as text  
) as nullable text
```

Acerca de

Devuelve el resultado de quitar un número de caracteres, `count`, del valor `text` de texto a partir de la posición `offset` y, a continuación, insertar el valor `newText` de texto en la misma posición en `text`.

Ejemplo 1

Reemplace un carácter único en la posición 2 del valor de texto "ABGF" por el nuevo valor de texto "CDE".

Uso

Power Query M

```
Text.ReplaceRange("ABGF", 2, 1, "CDE")
```

Salida

"ABCDEF"

Text.Reverse

Syntax

```
Text.Reverse(text as nullable text) as nullable text
```

Acerca de

Invierte el objeto proporcionado `text`.

Ejemplo 1

Invertir el texto "123".

Uso

Power Query M

```
Text.Reverse("123")
```

Salida

```
"321"
```

Last updated on 22/11/2025

Text.Select

16/09/2025

Sintaxis

```
Text.Select(text as nullable text, selectChars as any) as nullable text
```

Acerca de

Devuelve una copia del valor de texto `text` con todos los caracteres que no están en `selectChars` quitados.

Ejemplo 1

Seleccione todos los caracteres del intervalo de "a" a "z" del valor de texto.

Uso

Power Query M

```
Text.Select("a,b;c", {"a".."z"})
```

Salida

```
"abc"
```

Text.Split

16/09/2025

Sintaxis

```
Text.Split(text as text, separator as text) as list
```

Acerca de

Devuelve una lista de valores de texto resultantes de la división de un valor de texto basado en el delimitador especificado.

- `text`: valor de texto que se va a dividir.
- `separator`: delimitador usado para dividir el texto. El delimitador puede ser un solo carácter o una secuencia de caracteres. Si se usa una secuencia de caracteres, el texto solo se divide en instancias en las que se produce la secuencia exacta.

Ejemplo 1

Cree una lista a partir del valor de texto delimitado por "|" "Nombre|Dirección|Número de Teléfono".

Uso

Power Query M

```
Text.Split("Name|Address|PhoneNumber", "|")
```

Salida

Power Query M

```
{
    "Name",
    "Address",
    "PhoneNumber"
}
```

Ejemplo 2

Cree una lista a partir del valor de texto mediante una secuencia de caracteres.

Uso

Power Query M

```
Text.Split("Name, the Customer, the Purchase Date", ", the ")
```

Salida

Power Query M

```
{  
    Name,  
    Customer,  
    Purchase Date  
}
```


Text.SplitAny

Syntax

```
Text.SplitAny(text as text, separators as text) as list
```

Acerca de

Devuelve una lista de valores de texto resultantes de la división de un valor de texto en función de cualquier carácter especificado en el delimitador.

- **text**: valor de texto que se va a dividir.
- **separator**: caracteres delimitadores usados para dividir el texto.

Ejemplo 1

Cree una lista a partir del texto especificado mediante los caracteres delimitadores especificados.

Uso

Power Query M

```
Text.SplitAny("Name|Customer ID|Purchase|Month-Day-Year", "|-")
```

Salida

Power Query M

```
{
    "Name",
    "Customer ID",
    "Purchase",
    "Month",
    "Day",
    "Year"
}
```

Text.Start

16/09/2025

Sintaxis

```
Text.Start(text as nullable text, count as number) as nullable text
```

Acerca de

Devuelve los primeros **count** caracteres de **text** como un valor de texto.

Ejemplo 1

Obtener los primeros cinco caracteres de "Hello, World".

Uso

Power Query M

```
Text.Start("Hello, World", 5)
```

Salida

"Hello"

Ejemplo 2

Use los cuatro primeros caracteres del nombre y los tres primeros caracteres del apellido para crear la dirección de correo electrónico de un individuo.

Uso

Power Query M

```
let
    Source = #table(type table [First Name = text, Last Name = text],
        {
            {"Douglas", "Elis"},
            {"Ana", "Jorayew"},
            {"Rada", "Mihaylova"}
        })
```

```

        EmailAddress = Table.AddColumn(
            Source,
            "Email Address",
            each Text.Combine({
                Text.Start([First Name], 4),
                Text.Start([Last Name], 3),
                "@contoso.com"
            })
        )
    in
        EmailAddress

```

Salida

Power Query M

```

#table(type table [First Name = text, Last Name = text, Email Address = text],
{
    {"Douglas", "Elis", "DougEli@contoso.com"},
    {"Ana", "Jorayew", "AnaJor@contoso.com"},
    {"Rada", "Mihaylova", "RadaMih@contoso.com"}
})

```

Text.StartsWith

Sintaxis

```
Text.StartsWith(  
    text as nullable text,  
    substring as text,  
    optional comparer as nullable function  
) as nullable logical
```

Acerca de

Devuelve true si el valor de texto `text` comienza por el valor de texto `substring`.

- `text`: valor `text` que se va a buscar.
- `substring`: valor `text` que es la subcadena que se va a buscar en `text`.
- `comparer`: (Opcional) Que `Comparer` se usa para controlar la comparación. Por ejemplo, [Comparer.OrdinalIgnoreCase](#) se pueden usar para realizar búsquedas que no distinguen mayúsculas de minúsculas.

`comparer` es un elemento `Comparer` que sirve para controlar la comparación. Los comparadores se pueden usar para proporcionar comparaciones que no distinguen mayúsculas de minúsculas o de referencia cultural y de configuración regional.

Los siguientes comparadores integrados están disponibles en el lenguaje de fórmulas:

- [Comparer.Ordinal](#): se usa para realizar una comparación ordinal exacta.
- [Comparer.OrdinalIgnoreCase](#): se usa para realizar una comparación exacta sin distinción entre mayúsculas y minúsculas.
- [Comparer.FromCulture](#): Se usa para realizar una comparación teniendo en cuenta aspectos culturales.

Ejemplo 1

Comprobar si el texto "Hello, World" empieza con el texto "hello".

Uso

```
Text.StartsWith("Hello, World", "hello")
```

Salida

```
false
```

Ejemplo 2

Comprobar si el texto "Hello, World" empieza con el texto "Hello".

Uso

Power Query M

```
Text.StartsWith("Hello, World", "Hello")
```

Salida

```
true
```

Ejemplo 3

Ignorando mayúsculas y minúsculas, compruebe si el texto "Hello, World" comienza con el texto "hello".

Uso

Power Query M

```
Text.StartsWith("Hello, World", "hello", Comparer.OrdinalIgnoreCase)
```

Salida

```
true
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Text.ToBinary

24/10/2025

Sintaxis

```
Text.ToBinary(  
    text as nullable text,  
    optional encoding as nullable number,  
    optional includeByteOrderMark as nullable logical  
) as nullable binary
```

Acerca de

Codifica un valor de texto en un valor binario mediante la codificación especificada.

- `text`: texto que se va a codificar.
- `encoding`: (Opcional) Codificación usada para convertir el texto en binario. Use [BinaryEncoding.Type](#) para especificar el tipo de codificación. Si no se especifica este valor, el valor predeterminado es `BinaryEncoding.Utf8`.
- `includeByteOrderMark`: (Opcional) Determina si se debe incluir una marca de orden de bytes (BOM) al principio de la salida binaria. Establecer `true` para incluir automáticamente el BOM; de lo contrario, `false`. Si no se especifica este valor, el valor predeterminado es `false`.

Ejemplo 1

Codificar texto en binario, generar una cadena Base64 visible y, a continuación, decodificarla de nuevo en texto.

Uso

Power Query M

```
let  
    originalText = "Testing 1-2-3",  
  
    // Default UTF-8 binary  
    binaryData = Text.ToBinary(originalText),  
  
    // Convert binary to viewable Base64 string  
    encodedText = Binary.ToText(binaryData, BinaryEncoding.Base64),
```

```

// Decode back to text
decodedText = Text.FromBinary(binaryData),

result = [
    OriginalText = originalText,
    BinaryBase64 = encodedText,
    DecodedText = decodedText
]
in
result

```

Salida

Power Query M

```

[
    OriginalText = "Testing 1-2-3",
    BinaryEncoded = "VGVzdGluZyAxLTItMw==",
    DecodedText = "Testing 1-2-3"
]

```

Ejemplo 2

Codificar texto en binario con una marca de orden de bytes (BOM), generar una cadena hexadecimal visible y, a continuación, descodificarla de nuevo en texto.

Uso

Power Query M

```

let
    originalText = "Testing 1-2-3",

    // Convert to binary with BOM
    binaryData = Text.ToBinary(originalText, TextEncoding.Utf16, true),

    // Show binary as hex to demonstrate presence of BOM (fffe)
    binaryAsHex = Binary.ToText(binaryData, BinaryEncoding.Hex),

    // Decode back to text
    decodedText = Text.FromBinary(binaryData, TextEncoding.Utf16),

    // Compare original text and decoded text
    isIdentical = originalText = decodedText,

    result = [
        OriginalText = originalText,
        BinaryHex = binaryAsHex,

```

```
        DecodedText = decodedText,  
        IsIdentical = isIdentical  
    ]  
in  
    result
```

Salida

Power Query M

```
[  
    OriginalText = "Testing 1-2-3",  
    DecodedText = "fffe540065007300740069006e006700200031002d0032002d003300",  
    DecodedText = "Testing 1-2-3",  
    IsIdentical = true  
]
```


Text.ToList

16/09/2025

Sintaxis

```
Text.ToList(text as text) as list
```

Acerca de

Devuelve una lista de valores de caracteres a partir del valor de texto especificado `text`.

Ejemplo 1

Cree una lista de valores de caracteres a partir del texto "Hola mundo".

Uso

Power Query M

```
Text.ToList("Hello World")
```

Salida

```
powerquery-m{
```

```
    "H",  
    "e",  
    "l",  
    "l",  
    "o",  
    " ",  
    "W",  
    "o",  
    "r",  
    "l",  
    "d"
```

```
}
```

Text.Trim

16/09/2025

Sintaxis

```
Text.Trim(text as nullable text, optional trim as any) as nullable text
```

Acerca de

Devuelve el resultado de quitar todos los caracteres iniciales y finales del especificado `text`. De forma predeterminada, se quitan todos los caracteres de espacio en blanco iniciales y finales.

- `text`: texto del que se van a quitar los caracteres iniciales y finales.
- `trim`: invalida los caracteres de espacio en blanco que se recortan de forma predeterminada. Este parámetro puede ser un carácter único o una lista de caracteres únicos. Cada operación de recorte inicial y final se detiene cuando se encuentra un carácter no recortado.

Ejemplo 1

Quitar los espacios en blanco iniciales y finales de " a b c d ".

Uso

Power Query M

```
Text.Trim(" a b c d ")
```

Salida

```
"a b c d"
```

Ejemplo 2

Quite los ceros iniciales y finales de la representación de texto de un número.

Uso

Power Query M

```
Text.Trim("0000056.4200", "0")
```

Salida

```
"56.42"
```

Ejemplo 3

Quite los corchetes iniciales y finales de una etiqueta HTML.

Uso

Power Query M

```
Text.Trim("<div/>", {"<", ">", "/"})
```

Salida

```
"div"
```

Ejemplo 4

Quite los caracteres especiales usados alrededor del estado de ventas pendiente.

Uso

Power Query M

```
let
    Source = #table(type table [Home Sale = text, Sales Date = date, Sales Status = text],
        {
            {"1620 Ferris Way", #date(2024, 8, 22), "###@@Pending@@###"},
            {"757 1st Ave. S.", #date(2024, 3, 15), "Sold"},
            {"22303 Fillmore", #date(2024, 10, 2), "###@@Pending@@###"}
        }
    ),
    #"Trimmed Status" = Table.TransformColumns(Source, {"Sales Status", each Text.Trim(_, {"#", "@"})})
in
    #"Trimmed Status"
```

Salida

Power Query M

```
#table(type table [Home Sale = text, Sales Date = date, Sales Status = text],
{
  {"1620 Ferris Way", #date(2024, 8, 22), "Pending"},
  {"757 1st Ave. S.", #date(2024, 3, 15), "Sold"},
  {"22303 Fillmore", #date(2024, 10, 2), "Pending"}
})
```

Text.TrimEnd

16/09/2025

Sintaxis

```
Text.TrimEnd(text as nullable text, optional trim as any) as nullable text
```

Acerca de

Devuelve el resultado de quitar todos los caracteres finales del especificado `text`. De forma predeterminada, se quitan todos los caracteres de espacio en blanco finales.

- `text`: texto del que se van a quitar los caracteres finales.
- `trim`: invalida los caracteres de espacio en blanco que se recortan de forma predeterminada. Este parámetro puede ser un carácter único o una lista de caracteres únicos. Cada operación de recorte final se detiene cuando se encuentra un carácter no recortado.

Ejemplo 1

Quitar los espacios en blanco finales de " a b c d ".

Uso

Power Query M

```
Text.TrimEnd(" a b c d ")
```

Salida

```
" a b c d"
```

Ejemplo 2

Quite los ceros finales de una representación de texto de un número de punto flotante acolchado.

Uso

Power Query M

```
Text.TrimEnd("03.487700000", "0")
```

Salida

```
"03.4877"
```

Ejemplo 3

Quite los caracteres de relleno final de un nombre de cuenta de ancho fijo.

Uso

Power Query M

```
let
    Source = #table(type table [Name = text, Account Name= text, Interest =
number],
        {
            {"Bob", "US-847263****@", 2.8410},
            {"Leslie", "FR-4648****@**", 3.8392},
            {"Ringo", "DE-2046790@****", 12.6600}
        }
    ),
    #"Trimmed Account" = Table.TransformColumns(Source, {"Account Name", each
Text.TrimEnd(_, {"*", "@"})})
in
    #"Trimmed Account"
```

Salida

Power Query M

```
#table(type table [Name = text, Account Name = text, Interest = number],
    {
        {"Bob", "US-847263", 2.841},
        {"Leslie", "FR-4648", 3.8392},
        {"Ringo", "DE-2046790", 12.66}
    }
),
```

Text.TrimStart

Sintaxis

```
Text.TrimStart(text as nullable text, optional trim as any) as nullable text
```

Acerca de

Devuelve el resultado de quitar todos los caracteres de plomo del especificado `text`. De forma predeterminada, se quitan todos los caracteres de espacio en blanco iniciales.

- `text`: texto del que se van a quitar los caracteres iniciales.
- `trim`: invalida los caracteres de espacio en blanco que se recortan de forma predeterminada. Este parámetro puede ser un carácter único o una lista de caracteres únicos. Cada operación de recorte inicial se detiene cuando se encuentra un carácter no recortado.

Ejemplo 1

Quitar los espacios en blanco iniciales en " a b c d ".

Uso

Power Query M

```
Text.TrimStart("  a b c d  ")
```

Salida

```
"a b c d "
```

Ejemplo 2

Quite los ceros iniciales de la representación de texto de un número.

Uso

Power Query M

```
Text.TrimStart("0000056.420", "0")
```

Salida

"56.420"

Ejemplo 3

Quite los caracteres de relleno iniciales de un nombre de cuenta de ancho fijo.

Uso

Power Query M

```
let
    Source = #table(type table [Name = text, Account Name= text, Interest =
number],
        {
            {"Bob", "@****847263-US", 2.8410},
            {"Leslie", "@*****4648-FR", 3.8392},
            {"Ringo", "@*****24679-DE", 12.6600}
        }),
    #"Trimmed Account" = Table.TransformColumns(Source, {"Account Name", each
Text.TrimStart(_, {"*", "@"})})
in
    #"Trimmed Account"
```

Salida

Power Query M

```
#table(type table [Name = text, Account Name = text, Interest = number],
    {
        {"Bob", "847263-US", 2.841},
        {"Leslie", "4648-FR", 3.8392},
        {"Ringo", "2046790-DE", 12.66}
    }),
```

Last updated on 26/03/2026

Text.Upper

Sintaxis

```
Text.Upper(text as nullable text, optional culture as nullable text) as nullable text
```

Acerca de

Devuelve el resultado de convertir todos los caracteres de `text` en mayúsculas. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US").

Ejemplo

Obtenga la versión en mayúsculas de "aBcD".

Uso

Power Query M

```
Text.Upper("aBcD")
```

Salida

"ABCD"

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Last updated on 25/03/2026

Funciones de hora

Estas funciones crean y manipulan valores de hora.

 Expandir tabla

Nombre	Description
Time.EndOfHour	Devuelve el final de la hora.
Time.From	Devuelve un valor de hora de un valor.
Time.FromText	Crea una hora a partir de formatos de hora locales, universales y personalizados.
Time.Hour	Devuelve un valor de hora de un valor DateTime.
Time.Minute	Devuelve un valor de minuto de un valor DateTime.
Time.Second	Devuelve un segundo valor de un valor DateTime.
Time.StartOfHour	Devuelve el inicio de la hora.
Time.ToRecord	Devuelve un registro que contiene partes de un valor Date.
Time.ToText	Devuelve un valor de texto de un valor de hora.
#time	Crea un valor de hora a partir de hora, minuto y segundo.

Last updated on 22/11/2025

Time.EndOfHour

Syntax

```
Time.EndOfHour(dateTime as any) as any
```

Acerca de

Devuelve el final de la hora representada por `dateTime`, incluidos los segundos fraccionarios. Se conserva la información de zona horaria.

- `dateTime`: un `time` valor, `datetime` o `datetimezone` desde el que se calcula el final de la hora.

Ejemplo 1

Obtenga el final de la hora del 14/5/2011 05:00:00 p. m.

Uso

Power Query M

```
Time.EndOfHour(#datetime(2011, 5, 14, 17, 0, 0))
```

Salida

```
#datetime(2011, 5, 14, 17, 59, 59.9999999)
```

Ejemplo 2

Obtenga el final de la hora del 17/5/2011 05:00:00 -7:00.

Uso

Power Query M

```
Time.EndOfHour(#datetimezone(2011, 5, 17, 5, 0, 0, -7, 0))
```

Salida

```
#datetimezone(2011, 5, 17, 5, 59, 59.9999999, -7, 0)
```

Last updated on 22/11/2025

Time.From

Syntax

```
Time.From(value as any, optional culture as nullable text) as nullable time
```

Acerca de

Devuelve un `time` valor de la clase especificada `value`. También se puede proporcionar un `culture` opcional (por ejemplo, "en-US"). Si el `value` especificado es `null`, **Time.From** devuelve `null`. Si el objeto especificado `value` es `time`, `value` se devuelve. Los valores de los siguientes tipos se pueden convertir en un `time` valor:

- `text`: un `time` valor de representación textual. Consulte [Time.FromText](#) para obtener más información.
- `datetime`: componente de hora de `value`.
- `datetimezone`: componente de hora del equivalente de fecha y hora local de `value`.
- `number`: equivalente `time` al número de días fraccionarios expresados por `value`. Si `value` es negativo o mayor o igual a 1, se devuelve un error.

Si `value` es de cualquier otro tipo, se devuelve un error.

Ejemplo 1

Convertir `0.7575` en un `time` valor.

Uso

```
Power Query M
```

```
Time.From(0.7575)
```

Salida

```
#time(18, 10, 48)
```

Ejemplo 2

Convertir `#datetime(1899, 12, 30, 06, 45, 12)` en un `time` valor.

Uso

Power Query M

```
Time.From(#datetime(1899, 12, 30, 06, 45, 12))
```

Salida

```
#time(06, 45, 12)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)

Last updated on 22/11/2025

Time.FromText

Sintaxis

```
Time.FromText(text as nullable text, optional options as any) as nullable time
```

Acerca de

Crea un valor `time` a partir de una representación textual, `text`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, ve a [Cadenas con formato de fecha y hora estándar](#) y [Cadenas con formato de fecha y hora personalizado](#). Si se omite este campo o se proporciona `null`, se analizará la hora con el mejor esfuerzo.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "tt" es "AM" or "PM", mientras que en "ar-EG" "tt" es "ص" or "م". Cuando `Format` es null, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` es null o se omite, se usa [Culture.Current](#).

Para admitir flujos de trabajo heredados, `options` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = null, Culture = options]`.

Ejemplo 1

Convertir "10:12:31am" en un valor de hora.

Uso

Power Query M

```
Time.FromText("10:12:31am")
```

Salida

```
#time(10, 12, 31)
```

Ejemplo 2

Convertir "1012" en un valor de hora.

Uso

Power Query M

```
Time.FromText("1012")
```

Salida

```
#time(10, 12, 00)
```

Ejemplo 3

Convertir "10" en un valor de hora.

Uso

Power Query M

```
Time.FromText("10")
```

Salida

```
#time(10, 00, 00)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)

Last updated on 22/11/2025

Time.Hour

16/09/2025

Sintaxis

```
Time.Hour(dateTime as any) as nullable number
```

Acerca de

Devuelve el componente hora del valor proporcionado `time`, `datetime` o `datetimezone`, `dateTime`.

Ejemplo 1

Busque la hora en `#datetime(2011, 12, 31, 9, 15, 36)`.

Uso

Power Query M

```
Time.Hour(#datetime(2011, 12, 31, 9, 15, 36))
```

Salida

9

Time.Minute

16/09/2025

Sintaxis

```
Time.Minute(dateTime as any) as nullable number
```

Acerca de

Devuelve el componente de minuto del valor `time`, `datetime` o `datetimezone` proporcionado, `dateTime`.

Ejemplo 1

Busque el minuto en `#datetime(2011, 12, 31, 9, 15, 36)`.

Uso

Power Query M

```
Time.Minute(#datetime(2011, 12, 31, 9, 15, 36))
```

Salida

15

Time.Second

Syntax

```
Time.Second(dateTime as any) as nullable number
```

Acerca de

Devuelve el segundo componente del valor proporcionado `time`, `datetime`, o `datetimezone` .
`dateTime`

Ejemplo 1

Busque el segundo valor de un valor datetime.

Uso

Power Query M

```
Time.Second(#datetime(2011, 12, 31, 9, 15, 36.5))
```

Salida

36.5

Last updated on 22/11/2025

Time.StartOfHour

Syntax

```
Time.StartOfHour(dateTime as any) as any
```

Acerca de

Devuelve el inicio de la hora representada por `dateTime`. `dateTime` debe ser un `time` valor, `datetime` o `datetimezone`.

Ejemplo

Busque el inicio de la hora del 10 de octubre de 2011, 8:10:32 a. m.

Uso

Power Query M

```
Time.StartOfHour(#datetime(2011, 10, 10, 8, 10, 32))
```

Salida

```
#datetime(2011, 10, 10, 8, 0, 0)
```

Last updated on 25/03/2026

Time.ToRecord

Syntax

```
Time.ToRecord(time as time) as record
```

Acerca de

Devuelve un registro que contiene las partes del valor de hora especificado, `time`.

- `time` `time`: valor para el que se va a calcular el registro de sus partes.

Ejemplo 1

Convierta el `#time(11, 56, 2)` valor en un registro que contenga valores de hora.

Uso

Power Query M

```
Time.ToRecord(#time(11, 56, 2))
```

Salida

Power Query M

```
[  
    Hour = 11,  
    Minute = 56,  
    Second = 2  
]
```

Last updated on 22/11/2025

Time.ToText

Sintaxis

```
Time.ToText(  
    time as nullable time,  
    optional options as any,  
    optional culture as nullable text  
) as nullable text
```

Acercas de

Devuelve una representación textual de `time`. Se puede proporcionar un parámetro de `record` opcional, `options`, para especificar más propiedades. `culture` solo se usa para flujos de trabajo heredados. `record` puede contener los campos siguientes:

- `Format`: un valor `text` que indica el formato que se usará. Para obtener más información, ve a [Cadenas con formato de fecha y hora estándar](#) y [Cadenas con formato de fecha y hora personalizado](#). Si se omite este campo o se proporciona `null`, se dará formato a la fecha con el valor predeterminado definido por `Culture`.
- `Culture`: cuando `Format` no es null, `Culture` controla algunos especificadores de formato. Por ejemplo, en "en-US" "tt" es "AM" or "PM", mientras que en "ar-EG" "tt" es "ص" or "م". Cuando `Format` es `null`, `Culture` controla el formato predeterminado que se va a usar. Cuando `Culture` es `null` o se omite, se usa [Culture.Current](#).

Para admitir flujos de trabajo heredados, `options` y `culture` también puede ser valores de texto. Esto tiene el mismo comportamiento que `options = [Format = options, Culture = culture]`.

Ejemplo 1

Convertir `#time(01, 30, 25)` en un valor `text`. *La salida del resultado puede variar en función de la referencia cultural actual.*

Uso

Power Query M

```
Time.ToText(#time(11, 56, 2))
```

Salida

"11:56 AM"

Ejemplo 2

Convertir con un formato personalizado y la referencia cultural alemana.

Uso

Power Query M

```
Time.ToText(#time(11, 56, 2), [Format="hh:mm", Culture="de-DE"])
```

Salida

"11:56"

Ejemplo 3

Convertir con el formato de hora estándar.

Uso

Power Query M

```
Time.ToText(#time(11, 56, 2), [Format="T", Culture="de-DE"])
```

Salida

"11:56:02"

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)
- [Cadenas de formato de fecha y hora estándar](#)
- [Cadenas de formato de fecha y hora personalizadas](#)

#time

Syntax

```
#time(  
    hour as number,  
    minute as number,  
    second as number  
) as time
```

Acerca de

Crea un valor de hora a partir de números que representan la hora, el minuto y el segundo (fracciones). Genera un error si estas condiciones no son verdaderas:

- $0 \leq \text{hora} \leq 24$
- $0 \leq \text{minuto} \leq 59$
- $0 \leq \text{segundo} < 60$
- si la hora es 24, el minuto y el segundo deben ser 0.

Last updated on 22/11/2025

Funciones de tipo

18/07/2025

Estas funciones crean y manipulan valores de tipo.

 Expandir tabla

Nombre	Descripción
Type.AddTableKey	Agregue una clave a un tipo de tabla.
Type.ClosedRecord	El tipo especificado debe ser un tipo de registro que devuelve una versión cerrada del tipo de registro especificado (o el mismo tipo, si ya está cerrado).
Type.Facets	Devuelve las facetas de un tipo.
Type.ForFunction	Devuelve un tipo que representa funciones con restricciones específicas de parámetro y tipo de valor devuelto.
Type.ForRecord	Devuelve un tipo de registro desde un registro de campos.
Type.FunctionParameters	Devuelve un registro con los valores de campo asociados a los nombres de los parámetros de una función tipo, y sus valores correspondientes a sus tipos respectivos.
Type.FunctionRequiredParameters	Devuelve un número que indica el número mínimo de parámetros necesarios para invocar el tipo de función .
Type.FunctionReturn	Devuelve un tipo devuelto por un tipo de función.
Type.Is	Determina si un valor del primer tipo siempre es compatible con el segundo tipo.
Type.IsNullable	Devuelve true si un tipo es un tipo anulable; de lo contrario, false.
Type.IsOpenRecord	Devuelve si un tipo de registro está abierto.
Type.ListItem	Devuelve un tipo de elemento de un tipo de lista.
Type.NonNullable	Devuelve el tipo no anulable de un tipo dado.
Type.OpenRecord	Devuelve una versión abierta de un tipo de registro o del mismo tipo si ya está abierto.
Type.RecordFields	Devuelve un registro que describe los campos de un tipo de registro con cada campo del tipo de registro devuelto que tiene un nombre correspondiente y un valor que es un registro del formulario <code>[Type = type, Optional = logical]</code> .

Nombre	Descripción
Type.ReplaceFacets	Reemplaza las facetas de un tipo.
Type.ReplaceTableKeys	Reemplaza las claves de un tipo de tabla.
Type.ReplaceTablePartitionKey	Devuelve un nuevo tipo de tabla con la clave de partición reemplazada por la clave de partición especificada.
Type.TableColumn	Devuelve el tipo de una columna de una tabla.
Type.TableKeys	Devuelve claves de un tipo de tabla.
Type.TablePartitionKey	Devuelve la clave de partición para el tipo de tabla especificado si tiene una.
Type.TableRow	Devuelve un tipo de fila de un tipo de tabla.
Type.TableSchema	Devuelve una tabla que contiene una descripción de las columnas (es decir, el esquema) del tipo de tabla especificado.
Type.Union	Devuelve la unión de una lista de tipos.

Type.AddTableKey

16/09/2025

Sintaxis

```
Type.AddTableKey(  
    table as type,  
    columns as list,  
    isPrimary as logical  
) as type
```

Acerca de

Agrega una clave al tipo de tabla especificado.

Type.ClosedRecord

Sintaxis

```
Type.ClosedRecord(type as type) as type
```

Acerca de

Devuelve una versión cerrada de la especificada `record type` (o el mismo tipo, si ya está cerrada).

Ejemplo

Crear una versión cerrada de `type [ A = number, ... ]`.

Uso

Power Query M

```
Type.ClosedRecord(type [A = number, ...])
```

Salida

```
type [A = number]
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Last updated on 26/03/2026

Type.Facets

Syntax

```
Type.Facets(type as type) as record
```

Acerca de

Devuelve un registro que contiene las facetas de `type`.

Last updated on 22/11/2025

Type.ForFunction

16/09/2025

Sintaxis

```
Type.ForFunction(signature as record, min as number) as type
```

Acerca de

Crea un `function type` a partir de `signature`, un registro de `ReturnType` y `Parameters`, y `min`, que es el número mínimo de argumentos necesarios para invocar la función.

Ejemplo 1

Crea el tipo de una función que toma un parámetro numérico denominado X y devuelve un número.

Uso

Power Query M

```
Type.ForFunction([ReturnType = type number, Parameters = [X = type number]], 1)
```

Salida

```
type function (X as number) as number
```

Type.ForRecord

16/09/2025

Sintaxis

```
Type.ForRecord(fields as record, open as logical) as type
```

Acerca de

Devuelve un tipo que representa registros con restricciones de tipo específicas en campos.

Ejemplo 1

Genere dinámicamente un tipo de tabla.

Uso

Power Query M

```
let
    columnNames = {"Name", "Score"},
    columnTypes = {type text, type number},
    rowColumnTypes = List.Transform(columnTypes, (t) => [Type = t, Optional =
false]),
    rowType = Type.ForRecord(Record.FromList(rowColumnTypes, columnNames), false)
in
    #table(type table rowType, {"Betty", 90.3}, {"Carl", 89.5})
```

Salida

Power Query M

```
#table(
    type table [Name = text, Score = number],
    {"Betty", 90.3}, {"Carl", 89.5}
)
```

Type.FunctionParameters

16/09/2025

Sintaxis

```
Type.FunctionParameters(type as type) as record
```

Acerca de

Devuelve un registro con valores de campo asignados a los nombres de los parámetros de `type`, y sus valores asignados a sus tipos correspondientes.

Ejemplo 1

Busque los tipos de los parámetros en la función `(x as number, y as text)`.

Uso

Power Query M

```
Type.FunctionParameters(type function (x as number, y as text) as any)
```

Salida

```
[x = type number, y = type text]
```


Type.FunctionRequiredParameters

16/09/2025

Sintaxis

```
Type.FunctionRequiredParameters(type as type) as number
```

Acerca de

Devuelve un número que indica el número mínimo de parámetros necesarios para invocar la entrada `type` de la función.

Ejemplo 1

Busque el número de parámetros necesarios en la función `(x as number, optional y as text)`.

Uso

Power Query M

```
Type.FunctionRequiredParameters(type function (x as number, optional y as text) as any)
```

Salida

1

Type.FunctionReturn

Syntax

```
Type.FunctionReturn(type as type) as type
```

Acerca de

Devuelve un tipo devuelto por una función `type`.

Ejemplo 1

Busque el tipo de valor devuelto de `() as any`.

Uso

Power Query M

```
Type.FunctionReturn(type function () as any)
```

Salida

```
type any
```

Last updated on 22/11/2025

Type.Is

16/09/2025

Sintaxis

```
Type.Is(type1 as type, type2 as type) as logical
```

Acerca de

Determina si un valor de `type1` siempre es compatible con `type2`. El parámetro `type2` debe ser un valor de tipo primitivo (o primitivo que acepta valores NULL). De lo contrario, el comportamiento de esta función no está definido y no debe basarse en ella.

Ejemplo 1

Determine si un valor de tipo número siempre se puede tratar siempre como de tipo cualquiera.

Uso

Power Query M

```
Type.Is(type number, type any)
```

Salida

true

Ejemplo 2

Determine si un valor de tipo cualquiera siempre se puede tratar como número de tipo.

Uso

Power Query M

```
Type.Is(type any, type number)
```

Salida

false

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Type.IsNullable

Syntax

```
Type.IsNullable(type as type) as logical
```

Acerca de

Devuelve `true` si un tipo es un `nullable` tipo; de lo contrario, `false`.

Ejemplo 1

Determine si `number` acepta valores NULL.

Uso

Power Query M

```
Type.IsNullable(type number)
```

Salida

```
false
```

Ejemplo 2

Determine si `type nullable number` acepta valores NULL.

Uso

Power Query M

```
Type.IsNullable(type nullable number)
```

Salida

```
true
```

Contenido relacionado

- [Tipos y conversión de tipos](#)
-

Last updated on 22/11/2025

Type.IsOpenRecord

16/09/2025

Sintaxis

```
Type.IsOpenRecord(type as type) as logical
```

Acerca de

Devuelve un valor `logical` que indica si un registro `type` está abierto.

Ejemplo 1

Determine si el registro `type [ A = number, ... ]` está abierto.

Uso

Power Query M

```
Type.IsOpenRecord(type [A = number, ...])
```

Salida

```
true
```

Type.ListItem

Syntax

```
Type.ListItem(type as type) as type
```

Acerca de

Devuelve un tipo de elemento de una lista `type`.

Ejemplo 1

Busque el tipo de elemento de la lista `{number}`.

Uso

Power Query M

```
Type.ListItem(type {number})
```

Salida

```
type number
```

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Last updated on 22/11/2025

Type.NonNullable

Syntax

```
Type.NonNullable(type as type) as type
```

Acerca de

Devuelve el tipo distinto `nullable` de `.type`

Ejemplo

Devuelve el tipo que no acepta valores NULL de `type nullable number`.

Uso

Power Query M

```
Type.NonNullable(type nullable number)
```

Salida

```
type number
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 25/03/2026

Type.OpenRecord

Sintaxis

```
Type.OpenRecord(type as type) as type
```

Acerca de

Devuelve una versión abierta del especificado `record type` (o el mismo tipo, si ya está abierto).

Ejemplo

Crear una versión abierta de `type [ A = number ]`.

Uso

Power Query M

```
Type.OpenRecord(type [A = number])
```

Salida

```
type [A = number, ...]
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 25/03/2026

Type.RecordFields

Syntax

```
Type.RecordFields(type as type) as record
```

Acerca de

Devuelve un registro que describe los campos de un registro `type`. Cada campo del tipo de registro devuelto tiene un nombre correspondiente y un valor, en forma de registro `[ Type = type, Optional = logical ]`.

Ejemplo

Busque el nombre y el valor del registro `[ A = number, optional B = any ]`.

Uso

Power Query M

```
Type.RecordFields(type [A = number, optional B = any])
```

Salida

Power Query M

```
[  
  A = [Type = type number, Optional = false],  
  B = [Type = type any, Optional = true]  
]
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Type.ReplaceFacets

Syntax

```
Type.ReplaceFacets(type as type, facets as record) as type
```

Acerca de

Reemplaza las facetas de `type` por las facetas contenidas en el registro `facets`.

Last updated on 22/11/2025

Type.ReplaceTableKeys

16/09/2025

Sintaxis

```
Type.ReplaceTableKeys(tableType as type, keys as list) as type
```

Acerca de

Devuelve un nuevo tipo de tabla con todas las claves reemplazadas por la lista especificada de claves.

Cada clave se define mediante un registro de la forma siguiente:

- **Columns**: una lista de los nombres de columna que definen la clave
- **Primary**: **true** si la clave es la clave principal de la tabla; de lo contrario, **false**

La lista especificada de claves se valida para asegurarse de que no se define más de una clave principal y que todos los nombres de columna de clave existen en el tipo de tabla.

Ejemplo 1

Reemplazar la información clave de un tipo de tabla.

Uso

Power Query M

```
let
    BaseType = type table [ID = number, FirstName = text, LastName = text],
    KeysAdded = Type.ReplaceTableKeys(
        BaseType,
        {
            [Columns = {"ID"}, Primary = true],
            [Columns = {"FirstName", "LastName"}, Primary = false]
        }
    ),
    DetailsOfKeys = Type.TableKeys(KeysAdded)
in
    DetailsOfKeys
```

Salida

Power Query M

```
{  
    [Columns = {"ID"}, Primary = true],  
    [Columns = {"FirstName", "LastName"}, Primary = false]  
}
```

Ejemplo 2

Borre la información de clave definida previamente en un tipo de tabla.

Uso

Power Query M

```
let  
    TypeWithKey = Type.AddTableKey(type table [ID = number, Name = text], {"ID"},  
true),  
    KeyRemoved = Type.ReplaceTableKeys(TypeWithKey, {}),  
    DetailsOfKeys = Type.TableKeys(KeyRemoved)  
in  
    DetailsOfKeys
```

Salida

Power Query M

```
{}
```

Type.ReplaceTablePartitionKey

16/09/2025

Sintaxis

```
Type.ReplaceTablePartitionKey(tableType as type, partitionKey as nullable list) as  
type
```

Acerca de

Devuelve un nuevo tipo de tabla con la clave de partición reemplazada por la clave de partición especificada.

Type.TableColumn

16/09/2025

Sintaxis

```
Type.TableColumn(tableType as type, column as text) as type
```

Acerca de

Devuelve el tipo de la columna `column` en el tipo de tabla `tableType`.

Type.TableKeys

Sintaxis

```
Type.TableKeys(tableType as type) as list
```

Acerca de

Devuelve la lista posiblemente vacía de claves para el tipo de tabla especificado.

Cada clave se define mediante un registro de la forma siguiente:

- **Columns**: una lista de los nombres de columna que definen la clave
- **Primary**: **true** si la clave es la clave principal de la tabla; de lo contrario, **false**

Ejemplo

Devuelve la información clave para un tipo de tabla.

Uso

Power Query M

```
let
    BaseType = type table [ID = number, Name = text],
    AddKey = Type.AddTableKey(BaseType, {"ID"}, true),
    DetailsOfKeys = Type.TableKeys(AddKey)
in
    DetailsOfKeys
```

Salida

```
{[Columns = {"ID"}, Primary = true]}
```

Type.TablePartitionKey

16/09/2025

Sintaxis

```
Type.TablePartitionKey(tableType as type) as nullable list
```

Acerca de

Devuelve la clave de partición para el tipo de tabla especificado si tiene una.

Type.TableRow

Syntax

```
Type.TableRow(table as type) as type
```

Acerca de

Devuelve el tipo de fila del tipo de tabla especificado. El resultado siempre será un tipo de registro.

Ejemplo 1

Devuelve la información de tipo de fila de una tabla simple.

Uso

Power Query M

```
let
    tableRowType = Type.TableRow(Value.Type(#table({"Column1"}, {})))
in
    Type.RecordFields(tableRowType)
```

Salida

```
[Column1 = [Type = type any, Optional = false]]
```

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 22/11/2025

Type.TableSchema

Sintaxis

```
Type.TableSchema(tableType as type) as table
```

Acerca de

Devuelve una tabla que describe las columnas de `tableType`.

Consulte la documentación de [Table.Schema](#) para obtener una descripción de la tabla resultante.

Contenido relacionado

- [Tipos y conversión de tipos](#)

Last updated on 22/11/2025

Type.Union

Syntax

```
Type.Union(types as list) as type
```

Acerca de

Devuelve la unión de los tipos de `types`.

Last updated on 22/11/2025

Funciones de URI

Estas funciones crean y manipulan cadenas de consulta de URI.

 Expandir tabla

Nombre	Description
Uri.BuildQueryString	Ensamblar un registro en una cadena de consulta de URI.
Uri.Combine	Devuelve un URI basado en la combinación de las partes base y relativas.
Uri.EscapeDataString	Codifica caracteres especiales de acuerdo con RFC 3986.
Uri.Parts	Devuelve un valor de registro con los campos establecidos en las partes de un valor de texto de URI.

Last updated on 22/11/2025

Uri.BuildQueryString

Syntax

```
Uri.BuildQueryString(query as record) as text
```

Acerca de

Ensamblar el registro `query` en una cadena de consulta de URI, escape de caracteres según sea necesario.

Ejemplo 1

Codifique una cadena de consulta que contenga algunos caracteres especiales.

Uso

Power Query M

```
Uri.BuildQueryString([a = "1", b = "+$"])
```

Salida

```
"a=1&b=%2B%24"
```

Last updated on 22/11/2025

Uri.Combine

16/09/2025

Sintaxis

```
Uri.Combine(baseUri as text, relativeUri as text) as text
```

Acerca de

Devuelve un URI absoluto que es la combinación del `baseUri` de entrada y el `relativeUri`.

Uri.EscapeDataString

16/09/2025

Sintaxis

```
Uri.EscapeDataString(data as text) as text
```

Acerca de

Codifica caracteres especiales en el `data` de entrada según las reglas de RFC 3986.

Ejemplo 1

Codifique los caracteres especiales en "+money\$".

Uso

Power Query M

```
Uri.EscapeDataString("+money$")
```

Salida

```
"%2Bmoney%24"
```

Uri.Parts

16/09/2025

Sintaxis

```
Uri.Parts(absoluteUri as text) as record
```

Acerca de

Devuelve las partes del `absoluteUri` de entrada como un registro, que contiene valores como Scheme, Host, Port, Path, Query, Fragment, UserName y Password.

Ejemplo 1

Busque las partes del URI absoluto "www.adventure-works.com".

Uso

Power Query M

```
Uri.Parts("www.adventure-works.com")
```

Salida

Power Query M

```
[
    Scheme = "http",
    Host = "www.adventure-works.com",
    Port = 80,
    Path = "/",
    Query = [],
    Fragment = "",
    UserName = "",
    Password = ""
]
```

Ejemplo 2

Descodificar una cadena codificada por porcentaje.

Uso

Power Query M

```
let
    UriUnescapeDataString = (data as text) as text => Uri.Parts("http://contoso?
a=" & data)[Query][a]
in
    UriUnescapeDataString("%2Bmoney%24")
```

Salida

```
" +money$"
```

Funciones de valor

04/09/2025

Estas funciones evalúan y realizan operaciones en valores.

 Expandir tabla

Nombre	Descripción
Value.Alternates	Expresa planes de consulta alternativos.
Value.Compare	Devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo.
Value.Equals	Devuelve si dos valores son iguales.
Value.Expression	Devuelve un árbol de sintaxis abstracta (AST) que representa la expresión del valor.
Value.VersionIdentity	Devuelve la identidad de versión de un valor.
Value.Versions	Devuelve una tabla de navegación que contiene las versiones disponibles de un valor.
Value.NativeQuery	Evalúa una consulta contra un objetivo.
Value.NullableEquals	Devuelve un valor lógico o null basado en dos valores.
Value.Optimize	Si el valor representa una consulta que se puede optimizar, devuelve la consulta optimizada. De lo contrario, devuelve valor.
Value.Type	Devuelve el tipo del valor especificado.

Operaciones aritméticas

 Expandir tabla

Nombre	Descripción
Value.Add	Devuelve la suma de los dos valores.
Value.Divide	Devuelve el resultado de dividir el primer valor por el segundo.
Value.Multiply	Devuelve el producto de los dos valores.
Value.Subtract	Devuelve la diferencia de los dos valores.

Tipos de parámetro

[Expandir tabla](#)

Nombre	Descripción
Value.As	Devuelve el valor si es compatible con el tipo especificado.
Value.Is	Determina si un valor es compatible con el tipo especificado.
Value.ReplaceType	Reemplaza el tipo del valor.

[Expandir tabla](#)

Implementación	Descripción
Action.WithErrorContext	Esta función está pensada solo para uso interno.
DirectQueryCapabilities.From	Esta función está pensada solo para uso interno.
Embedded.Value	Accede a un valor por nombre en un mashup incrustado.
Excel.ShapeTable	Esta función está pensada solo para uso interno.
Module.Versions	Devuelve un registro de las versiones de módulo para el módulo actual y sus dependencias.
Progress.DataSourceProgress	Esta función está pensada solo para uso interno.
SqlExpression.SchemaFrom	Esta función está pensada solo para uso interno.
SqlExpression.ToExpression	Esta función está pensada solo para uso interno.
Value.Firewall	Esta función está pensada solo para uso interno.
Value.ViewError	Esta función está pensada solo para uso interno.
Value.ViewFunction	Esta función está pensada solo para uso interno.
Variable.Value	Devuelve el valor de la variable especificada.
Variable.ValueOrDefault	Devuelve el valor de la variable especificada o el valor predeterminado si la variable no está definida.

Metadatos

[Expandir tabla](#)

Nombre	Descripción
Value.Metadata	Devuelve un registro que contiene los metadatos de la entrada.
Value.RemoveMetadata	Quita los metadatos del valor y devuelve el valor original.
Value.ReplaceMetadata	Reemplaza los metadatos de un valor por el nuevo registro de metadatos proporcionado y devuelve el valor original con los nuevos metadatos adjuntos.

Linaje

 Expandir tabla

Nombre	Descripción
Graph.Nodes	Esta función está pensada solo para uso interno.
Value.Lineage	Esta función está pensada solo para uso interno.
Value.Traits	Esta función está pensada solo para uso interno.

Action.WithErrorContext

16/09/2025

Sintaxis

```
Action.WithErrorContext(action as action, context as text) as action
```

Acercas de

Esta función está pensada solo para uso interno.

DirectQueryCapabilities.From

16/09/2025

Sintaxis

```
DirectQueryCapabilities.From(value as any) as table
```

Acerca de

Esta función está pensada solo para uso interno.

Embedded.Value

Syntax

```
Embedded.Value(value as any, path as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Last updated on 04/04/2026

Excel.ShapeTable

16/09/2025

Sintaxis

```
Excel.ShapeTable(table as table, optional options as nullable record) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Graph.Nodes

16/09/2025

Sintaxis

```
Graph.Nodes(graph as record) as list
```

Acerca de

Esta función está pensada solo para uso interno.

Module.Versions

16/09/2025

Sintaxis

```
Module.Versions() as record
```

Acerca de

Devuelve un registro de las versiones de módulo para el módulo actual y sus dependencias.

Progress.DataSourceProgress

16/09/2025

Sintaxis

```
Progress.DataSourceProgress() as any
```

Acerca de

Esta función está pensada solo para uso interno.

SqlExpression.SchemaFrom

16/09/2025

Sintaxis

```
SqlExpression.SchemaFrom(schema as any) as any
```

Acerca de

Esta función está pensada solo para uso interno.

SqlExpression.ToExpression

16/09/2025

Sintaxis

```
SqlExpression.ToExpression(sql as text, environment as record) as text
```

Acerca de

Convierte la consulta de `sql` proporcionada en código M, con los identificadores disponibles definidos por `environment`. Esta función está pensada solo para uso interno.

Value.Add

16/09/2025

Sintaxis

```
Value.Add(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as any
```

Acerca de

Devuelve la suma de `value1` y `value2`. Se puede especificar un parámetro `precision` opcional, de forma predeterminada `Precision.Double` se usa.

Value.Alternates

Syntax

```
Value.Alternates(alternates as list) as any
```

Acerca de

Expresa planes de consulta alternativos dentro de una expresión de plan de consulta obtenida a través de `Value.Expression(Value.Optimize(...))`. No está pensado para otros usos.

Last updated on 22/11/2025

Value.As

Syntax

```
Value.As(value as any, type as type) as any
```

Acerca de

Devuelve el valor si es compatible con el tipo especificado. Esto equivale al operador "as" de M, con la excepción de que puede aceptar referencias de tipo de identificador como Number.Type.

Ejemplo 1

Convierta un número en un número.

Uso

Power Query M

```
Value.As(123, Number.Type)
```

Salida

```
123
```

Ejemplo 2

Intente convertir un valor de texto en un número.

Uso

Power Query M

```
Value.As("abc", type number)
```

Salida

```
[Expression.Error] We cannot convert the value "abc" to type Number.
```

Value.Compare

16/09/2025

Sintaxis

```
Value.Compare(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as number
```

Acerca de

Devuelve -1, 0 o 1 en función de si el primer valor es menor, igual o mayor que el segundo.

Value.Divide

16/09/2025

Sintaxis

```
Value.Divide(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as any
```

Acerca de

Devuelve el resultado de dividir `value1` por `value2`. Se puede especificar un parámetro `precision` opcional, de forma predeterminada `Precision.Double` se usa.

Value.Equals

16/09/2025

Sintaxis

```
Value.Equals(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as logical
```

Acerca de

Devuelve true si el valor `value1` es igual al valor `value2`, false de lo contrario.

Value.Expression

16/09/2025

Sintaxis

```
Value.Expression(value as any) as nullable record
```

Acerca de

Devuelve un árbol de sintaxis abstracta (AST) que representa la expresión del valor.

Value.Firewall

16/09/2025

Sintaxis

```
Value.Firewall(key as text) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Value.FromText

Sintaxis

```
Value.FromText(text as any, optional culture as nullable text) as any
```

Acerca de

Descodifica un valor de una representación textual e lo interpreta como un valor con un tipo adecuado.

- **text**: texto que se va a interpretar.
- **culture**: (Opcional) Referencia cultural específica utilizada para interpretar el texto (por ejemplo, "en-US").

Esta función toma un valor de texto y devuelve un valor de tipo **number**, **logical**, **null**, **datetime**, **duration** o **text**. Un valor de texto vacío se interpreta como un **null** valor.

Ejemplo 1

Convierta el texto que representa un número en su valor numérico correspondiente.

Uso

Power Query M

```
Value.FromText("12345.6789")
```

Salida

12345.6789

Ejemplo 2

Convertir texto que representa un porcentaje a su valor numérico correspondiente.

Uso

Power Query M

```
Value.FromText("25.4%")
```

Salida

```
0.254
```

Ejemplo 3

Convierte texto que representa un valor de Euro francés en su valor numérico correspondiente.

Uso

Power Query M

```
Value.FromText("€1,190", "fr-FR")
```

Salida

```
1.19
```

Ejemplo 4

Convierte texto que representa una fecha y hora alemanas en su valor de fecha y hora correspondiente.

Uso

Power Query M

```
Value.FromText("24 Dez 2024 14:33:20", "de-DE")
```

Salida

```
#datetime(2024, 12, 24, 14, 33, 20)
```

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

Value.Is

16/09/2025

Sintaxis

```
Value.Is(value as any, type as type) as logical
```

Acerca de

Determina si un valor es compatible con el tipo especificado. Esto equivale al operador "is" en M, con la excepción de que puede aceptar referencias de tipo de identificador como Number.Type.

Ejemplo 1

Compare dos maneras de determinar si un número es compatible con el número de tipo.

Uso

Power Query M

```
Value.Is(123, Number.Type) = (123 is number)
```

Salida

```
true
```

Value.Lineage

16/09/2025

Sintaxis

```
Value.Lineage(value as any) as any
```

Acerca de

Esta función está pensada solo para uso interno.

Value.Metadata

16/09/2025

Sintaxis

```
Value.Metadata(value as any) as any
```

Acerca de

Devuelve un registro que contiene los metadatos de la entrada.

Value.Multiply

Sintaxis

```
Value.Multiply(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as any
```

Acerca de

Devuelve el producto de multiplicar `value1` por `value2`. Se puede especificar un parámetro `precision` opcional, de forma predeterminada [Precision.Double](#) se usa.

Last updated on 02/04/2026

Value.NativeQuery

Syntax

```
Value.NativeQuery(  
  target as any,  
  query as text,  
  optional parameters as any,  
  optional options as nullable record  
) as any
```

Acerca de

`query` Se evalúa con respecto al `target` uso de los parámetros especificados en `parameters` y las opciones especificadas en `options`.

La salida de la consulta se define mediante `target`.

`target` proporciona el contexto para la operación descrita por `query`.

`query` describe la consulta que se va a ejecutar en `target`. `query` se expresa de forma específica de `target` (por ejemplo, una instrucción T-SQL).

El valor opcional `parameters` puede contener una lista o un registro según corresponda para proporcionar los valores de parámetro esperados por `query`.

El registro opcional `options` puede contener opciones que afectan al comportamiento de evaluación de `query` en `target`. Estas opciones son específicas de `target`.

Value.NullableEquals

16/09/2025

Sintaxis

```
Value.NullableEquals(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as nullable logical
```

Acerca de

Devuelve null si cualquiera de los argumentos `value1`, `value2` es NULL; de lo contrario, equivale a [Value.Equals](#).

Value.Optimize

Syntax

```
Value.Optimize(value as any) as any
```

Acerca de

Cuando se usa en Value.Expression, si **value** representa una consulta que se puede optimizar, esta función indica que se debe devolver la expresión optimizada. De lo contrario, **value** se pasará sin ningún efecto.

Last updated on 22/11/2025

Value.RemoveMetadata

16/09/2025

Sintaxis

```
Value.RemoveMetadata(value as any, optional metaValue as any) as any
```

Acerca de

Elimina la entrada de metadatos.

Ejemplo 1

Quite todos los metadatos de un valor de texto.

Uso

Power Query M

```
Value.Metadata(  
    Value.RemoveMetadata("abc" meta [a = 1, b = 2])  
)
```

Salida

```
[[]]
```

Ejemplo 2

Quite solo un campo de metadatos de un valor de texto.

Uso

Power Query M

```
Value.Metadata(  
    Value.RemoveMetadata("abc" meta [a = 1, b = 2], {"a"})  
)
```

Salida

[b = 2]

Value.ReplaceMetadata

16/09/2025

Sintaxis

```
Value.ReplaceMetadata(value as any, metaValue as any) as any
```

Acerca de

Reemplaza la información de metadatos de la entrada.

Value.ReplaceType

16/09/2025

Sintaxis

```
Value.ReplaceType(value as any, type as type) as any
```

Acerca de

Reemplaza el tipo de `value` con el proporcionado `type`.

Ejemplo 1

Reemplace el tipo predeterminado de un registro por un tipo más específico.

Uso

Power Query M

```
Type.RecordFields(  
    Value.Type(  
        Value.ReplaceType(  
            [Column1 = 123],  
            type [Column1 = number]  
        )  
    )  
)[Column1][Type]
```

Salida

```
type number
```

Value.Subtract

16/09/2025

Sintaxis

```
Value.Subtract(  
    value1 as any,  
    value2 as any,  
    optional precision as nullable number  
) as any
```

Acerca de

Devuelve la diferencia de `value1` y `value2`. Se puede especificar un parámetro `precision` opcional, de forma predeterminada `Precision.Double` se usa.

Value.Traits

16/09/2025

Sintaxis

```
Value.Traits(value as any) as table
```

Acerca de

Esta función está pensada solo para uso interno.

Value.Type

16/09/2025

Sintaxis

```
Value.Type(value as any) as type
```

Acerca de

Devuelve el tipo del valor especificado.

- **value**: valor cuyo tipo se devuelve.

Ejemplo 1

Devuelve el tipo del número especificado.

Uso

```
Power Query M
```

```
Value.Type(243.448)
```

Salida

```
type number
```

Ejemplo 2

Devuelve el tipo de la fecha especificada.

Uso

```
Power Query M
```

```
Value.Type(#datetime(2010, 12, 31))
```

Salida

type date

Ejemplo 3

Devuelve el tipo del registro especificado.

Uso

Power Query M

```
Value.Type([a = 1, b = 2])
```

Salida

type record

Contenido relacionado

- [Tipos y de conversión de tipos](#)

Value.VersionIdentity

16/09/2025

Sintaxis

```
Value.VersionIdentity(value as any) as any
```

Acerca de

Devuelve la identidad de versión del `value`, o la del `null` si el `value` no tiene una versión.

Value.Versions

Sintaxis

```
Value.Versions(value as any) as table
```

Acerca de

Devuelve una tabla de navegación que contiene las versiones disponibles de `.value`

Last updated on 04/04/2026

Value.ViewError

16/09/2025

Sintaxis

```
Value.ViewError(errorRecord as record) as record
```

Acerca de

Esta función está pensada solo para uso interno.

Value.ViewFunction

16/09/2025

Sintaxis

```
Value.ViewFunction(function as function) as function
```

Acerca de

Esta función está pensada solo para uso interno.

Variable.Value

16/09/2025

Sintaxis

```
Variable.Value(identifier as text) as any
```

Acerca de

Devuelve el valor de la variable **identifier** especificada definida por el entorno de evaluación actual. Si no se define la variable, se genera un error.

Variable.ValueOrDefault

16/09/2025

Sintaxis

```
Variable.ValueOrDefault(identifier as text, optional defaultValue as any) as any
```

Acerca de

Devuelve el valor de la variable `identifier` especificada definida por el entorno de evaluación actual. Si no se define la variable, se devuelve el valor opcional `defaultValue` .

Enumeraciones

El lenguaje de fórmulas M de Power Query incluye estas enumeraciones.

Lista de enumeraciones

 Expandir tabla

Nombre	Description
AccessControlKind.Type	Especifica el tipo de control de acceso.
BinaryEncoding.Type	Especifica el tipo de codificación binaria.
BinaryOccurrence.Type	Especifica cuántas veces se espera que el elemento aparezca en el grupo.
BufferMode.Type	Describe el tipo de almacenamiento en búfer que se va a realizar.
ByteOrder.Type	Especifica el orden de bytes.
Compression.Type	Especifica el tipo de compresión.
CsvStyle.Type	Especifica la importancia de las comillas de un documento CSV.
Day.Type	Representa el día de la semana.
ExtraValues.Type	Especifica la acción esperada para valores adicionales de una fila que contiene columnas inferiores a las esperadas.
GroupKind.Type	Especifica el tipo de agrupación.
JoinAlgorithm.Type	Especifica el algoritmo de combinación que se va a usar en la operación de combinación.
JoinKind.Type	Especifica el tipo de operación de combinación.
JoinSide.Type	Especifica la tabla izquierda o derecha de una combinación.
LimitClauseKind.Type	Indica las características que admite el dialecto SQL específico.
MissingField.Type	Especifica la acción esperada para los valores que faltan en una fila que contiene columnas inferiores a las esperadas.
Occurrence.Type	Especifica la aparición de un elemento en una secuencia.

Nombre	Description
ODataOmitValues.Type	Especifica los tipos de valores que un servicio OData puede omitir.
Order.Type	Especifica la dirección de ordenación.
PercentileMode.Type	Especifica el tipo de modo percentil.
Precision.Type	Especifica la precisión de la comparación.
QuoteStyle.Type	Especifica el estilo de comillas.
RankKind.Type	Especifica el método de clasificación preciso.
RelativePosition.Type	Indica si la indexación se debe realizar desde el principio o el final de la entrada.
RoundingMode.Type	Especifica la dirección de redondeo cuando hay un empate entre los números posibles a los que redondear.
SapBusinessWarehouseExecutionMode.Type	Especifica las opciones válidas para la opción de modo de ejecución de SAP Business Warehouse.
SapHanaDistribution.Type	Especifica opciones válidas para la opción de distribución de SAP HANA.
SapHanaRangeOperator.Type	Especifica un operador de intervalo para los parámetros de entrada de intervalo de SAP HANA.
TextEncoding.Type	Especifica el tipo de codificación de texto.
TraceLevel.Type	Especifica el nivel de seguimiento.
WebMethod.Type	Especifica un método HTTP.

Last updated on 22/11/2025

AccessControlKind.Type

Definición

Especifica el tipo de control de acceso.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
AccessControlKind.Deny	0	Acceso denegado.
AccessControlKind.Allow	1	Se permite el acceso.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

BinaryEncoding.Type

Definición

Especifica el tipo de codificación binaria.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
BinaryEncoding.Base64	0	Constante que se va a usar como tipo de codificación cuando se requiere la codificación base 64.
BinaryEncoding.Hex	1	Constante que se va a usar como tipo de codificación cuando se requiere codificación hexadecimal.

Se aplica a

- [Funciones binarias](#)

Last updated on 22/11/2025

BinaryOccurrence.Type

Definición

Especifica cuántas veces se espera que el elemento aparezca en el grupo.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
BinaryOccurrence.Optional	0	Se espera que el elemento aparezca cero o una vez en la entrada.
BinaryOccurrence.Required	1	Se espera que el elemento aparezca una vez en la entrada.
BinaryOccurrence.Repeating	2	Se espera que el elemento aparezca cero o más veces en la entrada.

Se aplica a

- [Funciones binarias](#)

Last updated on 22/11/2025

BufferMode.Type

Definición

Describe el tipo de almacenamiento en búfer que se va a realizar.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
BufferMode.Eager	1	El valor completo se almacena inmediatamente en búfer en memoria antes de continuar.
BufferMode.Delayed	2	El tipo del valor se calcula inmediatamente, pero su contenido no se almacena en búfer hasta que se necesitan datos, en cuyo punto se almacena en búfer todo el valor.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

ByteOrder.Type

Definición

Especifica el orden de bytes.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
ByteOrder.LittleEndian	0	El byte menos significativo aparece primero en el orden de bytes Little Endian.
ByteOrder.BigEndian	1	El byte más significativo aparece primero en el orden de bytes big Endian.

Observaciones

Los valores permitidos para esta enumeración son valores posibles para el `byteOrder` parámetro en [BinaryFormat.ByteOrder](#).

Se aplica a

- [Funciones binarias](#)

Last updated on 22/11/2025

Compression.Type

Definición

Especifica el tipo de compresión.

Valores permitidos

[Expandir tabla](#)

Nombre	Importancia	Description
Compression.None	-1	Los datos no están comprimidos.
Compression.GZip	0	Los datos comprimidos tienen el formato "GZip".
Compression.Deflate	1	Los datos comprimidos están en el formato "Deflate".
Compression.Snappy	2	Los datos comprimidos están en el formato "Snappy".
Compression.Brotli	3	Los datos comprimidos están en el formato "Brotli".
Compression.LZ4	4	Los datos comprimidos tienen el formato "LZ4".
Compression.Zstandard	5	Los datos comprimidos están en el formato "Zstandard".

Se aplica a

- [Funciones binarias](#)

Last updated on 22/11/2025

CsvStyle.Type

Definición

Especifica la importancia de las comillas de los documentos CSV.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
CsvStyle.QuoteAfterDelimiter	0	Las comillas de un campo solo son significativas inmediateamente después del delimitador.
CsvStyle.QuoteAlways	1	Las comillas de un campo siempre son significativas independientemente de dónde aparezcan.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

Day.Type

Definición

Representa el día de la semana.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
Day.Sunday	0	Representa domingo.
Day.Monday	1	Representa el lunes.
Day.Tuesday	2	Representa martes.
Day.Wednesday	3	Representa el miércoles.
Día.Jueves	4	Representa jueves.
Day.Friday	5	Representa el viernes.
Day.Saturday	6	Representa sábado.

Se aplica a

- [Funciones de fecha](#)

Last updated on 22/11/2025

ExtraValues.Type

Definición

Especifica la acción esperada para valores adicionales de una fila que contiene columnas inferiores a las esperadas.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
ExtraValues.List	0	Si la función divisora devuelve más columnas de las que espera la tabla, se deben recopilar en una lista.
ExtraValues.Error	1	Si la función divisora devuelve más columnas de las que espera la tabla, se debe generar un error.
ExtraValues.Ignore	2	Si la función divisora devuelve más columnas de las que espera la tabla, se deben omitir.

Se aplica a

- [Acceso a funciones de datos](#)
- [Funciones de tabla](#)

Last updated on 22/11/2025

GroupKind.Type

Definición

Especifica el tipo de agrupación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
GroupKind.Local	0	Un grupo local se forma a partir de una secuencia consecutiva de filas de una tabla de entrada con el mismo valor de clave.
GroupKind.Global	1	Un grupo global se forma a partir de todas las filas de una tabla de entrada con el mismo valor de clave.

Se aplica a

- [Funciones de tabla](#)

Last updated on 22/11/2025

JoinAlgorithm.Type

Definición

Especifica el algoritmo de combinación que se va a usar en la operación de combinación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
JoinAlgorithm.Dynamic	0	Elige automáticamente un algoritmo de combinación basado en inspeccionar las filas iniciales y los metadatos de ambas tablas.
JoinAlgorithm.PairwiseHash	1	Almacena en búfer las filas de las tablas izquierda y derecha hasta que una de las tablas se almacena completamente en búfer y, a continuación, realiza un LeftHash o RightHash, dependiendo de la tabla que se almacena en búfer por completo. Este algoritmo solo se recomienda para tablas pequeñas.
JoinAlgorithm.SortMerge	2	Realiza una combinación de streaming en función de la suposición de que ambas tablas están ordenadas por sus claves de combinación. Aunque es eficaz, devolverá resultados incorrectos si las tablas no están ordenadas según lo previsto.
JoinAlgorithm.LeftHash	3	Almacena en búfer las filas izquierdas en una tabla de búsqueda y transmite las filas derechas. Para cada fila derecha, las filas izquierdas coincidentes se encuentran a través de la tabla de búsqueda almacenada en búfer. Este algoritmo se recomienda cuando la tabla izquierda es pequeña y se espera que la mayoría de las filas de la tabla derecha coincidan con una fila izquierda.
JoinAlgorithm.RightHash	4	Almacena en búfer las filas derechas en una tabla de búsqueda y transmite las filas izquierdas. Para cada fila izquierda, las filas derechas coincidentes se encuentran a través de la tabla de búsqueda almacenada en búfer. Este algoritmo se recomienda cuando la tabla derecha es pequeña y se espera que la mayoría de las filas de la tabla izquierda coincidan con una fila derecha.
JoinAlgorithm.LeftIndex	5	En lotes, usa las claves de la tabla izquierda para realizar consultas basadas en predicados en la tabla derecha. Este

Nombre	Importancia	Description
		algoritmo se recomienda cuando la tabla derecha es grande, admite el plegado de Table.SelectRows y contiene pocas filas que se espera que coincidan con una fila izquierda.
JoinAlgorithm.RightIndex	6	En lotes, usa las claves de la tabla derecha para realizar consultas basadas en predicados en la tabla izquierda. Este algoritmo se recomienda cuando la tabla izquierda es grande, admite el plegado de Table.SelectRows y contiene pocas filas que se espera que coincidan con una fila derecha.

Se aplica a

- [Funciones de tabla](#)

Last updated on 16/09/2025

JoinKind.Type

Definición

Especifica el tipo de operación de combinación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
JoinKind.Inner	0	La tabla resultante de una combinación interna contiene una fila para cada par de filas de las tablas especificadas que se determinaron para que coincidan en función de las columnas de clave especificadas.
JoinKind.LeftOuter	1	Una combinación externa izquierda garantiza que todas las filas de la primera tabla aparezcan en el resultado.
JoinKind.RightOuter	2	Una combinación externa derecha garantiza que todas las filas de la segunda tabla aparezcan en el resultado.
JoinKind.FullOuter	3	Una combinación externa completa garantiza que todas las filas de ambas tablas aparezcan en el resultado. Las filas que no tenían una coincidencia en la otra tabla se combinan con una fila predeterminada que contiene valores NULL para todas sus columnas.
JoinKind.LeftAnti	4	Una combinación anti izquierda devuelve todas las filas de la primera tabla que no tienen una coincidencia en la segunda tabla.
JoinKind.RightAnti	5	Una combinación anti derecha devuelve todas las filas de la segunda tabla que no tienen una coincidencia en la primera tabla.
JoinKind.LeftSemi	6	Una combinación semi izquierda devuelve todas las filas de la primera tabla que tienen una coincidencia en la segunda tabla.
JoinKind.RightSemi	7	Una combinación semi derecha devuelve todas las filas de la segunda tabla que tienen una coincidencia en la primera tabla.

Observaciones

Los campos de esta enumeración son valores posibles para el parámetro opcional `JoinKind` en [Table.Join](#).

Se aplica a

- [Funciones de tabla](#)

Last updated on 22/11/2025

JoinSide.Type

Definición

Especifica la tabla izquierda o derecha de una combinación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
JoinSide.Left	0	Especifica la tabla izquierda de una combinación.
JoinSide.Right	1	Especifica la tabla derecha de una combinación.

Se aplica a

- [Funciones de tabla](#)

Last updated on 22/11/2025

LimitClauseKind.Type

Definición

Describe el tipo de cláusula limit compatible con el dialecto SQL usado por este origen de datos.

Valores permitidos

[Expandir tabla](#)

Nombre	Importancia	Description
LimitClauseKind.None	0	Este dialecto SQL no admite una cláusula limit.
LimitClauseKind.Top	1	Este dialecto SQL admite un especificador TOP para limitar el número de filas devueltas.
LimitClauseKind.LimitOffset	2	Este dialecto SQL admite especificadores LIMIT y OFFSET para limitar el número de filas devueltas.
LimitClauseKind.Limit	3	Este dialecto SQL admite un especificador LIMIT para limitar el número de filas devueltas.
LimitClauseKind.AnsiSql2008	4	Este dialecto SQL admite un especificador LIMIT N ROWS compatible con ANSI SQL para limitar el número de filas devueltas.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

MissingField.Type

Definición

Especifica la acción esperada para los valores que faltan en una fila que contiene columnas inferiores a las esperadas.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
MissingField.Error	0	Indica que los campos que faltan deben producir un error. (Este es el valor predeterminado).
MissingField.Ignore	1	Indica que se deben omitir los campos que faltan.
MissingField.UseNull	2	Indica que los campos que faltan deben incluirse como valores NULL.

Se aplica a

- [Funciones de registro](#)
- [Funciones de tabla](#)

Last updated on 22/11/2025

Occurrence.Type

Definición

Especifica la aparición de un elemento en una secuencia.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
Occurrence.First	0	Se devuelve la posición de la primera aparición del valor encontrado.
Occurrence.Last	1	Se devuelve la posición de la última aparición del valor encontrado.
Occurrence.All	2	Se devuelve una lista de posiciones de todas las apariciones de los valores encontrados.
Occurrence.Optional	0	Se espera que el elemento aparezca cero o una vez en la entrada. Se proporciona para la compatibilidad con versiones anteriores en funciones binarias. En este caso, use BinaryOccurrence.Optional en su lugar.
Occurrence.Required	1	Se espera que el elemento aparezca una vez en la entrada. Se proporciona para la compatibilidad con versiones anteriores en funciones binarias. En este caso, use BinaryOccurrence.Required en su lugar.
Occurrence.Repeating	2	Se espera que el elemento aparezca cero o más veces en la entrada. Se proporciona para la compatibilidad con versiones anteriores en funciones binarias. En este caso, use BinaryOccurrence.Repeating en su lugar.

Se aplica a

- [Funciones de tabla](#)
- [Funciones de texto](#)

ODataOmitValues.Type

Definición

Especifica los tipos de valores que un servicio OData puede omitir.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
ODataOmitValues.Nulls	"nulls"	Permite que el servicio OData omita valores NULL.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

Order.Type

Definición

Especifica la dirección de ordenación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
Order.Ascending	0	Ordena los valores en orden ascendente.
Order.Descending	1	Ordena los valores en orden descendente.

Se aplica a

- [Funciones de tabla](#)

Last updated on 22/11/2025

PercentileMode.Type

Definición

Especifica el tipo de modo percentil.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
PercentileMode.ExcelInc	1	Al interpolar valores para List.Percentile , use un método compatible con el de Excel PERCENTILE.INC .
PercentileMode.ExcelExc	2	Al interpolar valores para List.Percentile , use un método compatible con el de Excel PERCENTILE.EXC .
PercentileMode.SqlDisc	3	Al interpolar valores para List.Percentile , use un método compatible con el de PERCENTILE_DISC SQL Server.
PercentileMode.SqlCont	4	Al interpolar valores para List.Percentile , use un método compatible con el de PERCENTILE_CONT SQL Server.

Se aplica a

- [Funciones de lista](#)

Last updated on 02/04/2026

Precision.Type

Definición

Especifica la precisión de la comparación.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
Precision.Double	0	Parámetro opcional para los operadores aritméticos integrados para especificar precisión doble.
Precision.Decimal	1	Parámetro opcional para los operadores aritméticos integrados para especificar la precisión decimal.

Se aplica a

- [Funciones de valor](#)

Last updated on 22/11/2025

QuoteStyle.Type

Definición

Especifica el estilo de comillas.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
QuoteStyle.None	0	Los caracteres de comillas no tienen significado.
QuoteStyle.Csv	1	Los caracteres de comillas indican el inicio de una cadena entre comillas. Las comillas anidadas se indican entre dos caracteres de comillas.

Se aplica a

- [Funciones divisoras](#)

Last updated on 22/11/2025

RankKind.Type

Definición

Especifica el método de clasificación preciso.

Valores permitidos

[Expandir tabla](#)

Nombre	Importancia	Description
RankKind.Competition	0	Los elementos que se comparan como iguales reciben el mismo número de clasificación y, a continuación, se deja una brecha antes de la siguiente clasificación.
RankKind.Dense	1	Los elementos que se comparan como iguales reciben el mismo número de clasificación y el siguiente elemento se numera consecutivamente sin espacio.
RankKind.Ordinal	2	A todos los elementos se les asigna un número de clasificación único incluso si se comparan como iguales.

Se aplica a

- [Funciones de tabla](#)

Last updated on 22/11/2025

RelativePosition.Type

Definición

Indica si la indexación se debe realizar desde el principio o el final de la entrada.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
RelativePosition.FromStart	0	Indica que la indexación se debe realizar desde el principio de la entrada.
RelativePosition.FromEnd	1	Indica que la indexación se debe realizar desde el final de la entrada.

Se aplica a

- [Funciones de texto](#)

Last updated on 22/11/2025

RoundingMode.Type

Definición

Especifica la dirección de redondeo cuando hay un empate entre los números posibles a los que redondear.

Valores permitidos

[Expandir tabla](#)

Nombre	Importancia	Description
RoundingMode.Up	0	Redondea cuando hay un empate entre los números posibles a los que redondear.
RoundingMode.Down	1	Redondea hacia abajo cuando hay un empate entre los números posibles a los que redondear.
RoundingMode.AwayFromZero	2	Redondea de cero cuando hay un empate entre los números posibles a los que redondear.
RoundingMode.TowardZero	3	Redondea hacia cero cuando hay un empate entre los números posibles a los que redondear.
RoundingMode.ToEven	4	Redondea al número par más cercano cuando hay un empate entre los números posibles a los que redondear.

Se aplica a

- [Funciones de lista](#)

SapBusinessWarehouseExecutionMode.Type

Definición

Especifica las opciones válidas para la opción de modo de ejecución de SAP Business Warehouse.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
SapBusinessWarehouseExecutionMode.BasXml	64	Opción "modo de acoplamiento bXML" para la ejecución MDX en SAP Business Warehouse.
SapBusinessWarehouseExecutionMode.BasXmlGzip	65	Opción "Modo de acoplamiento bXML comprimido de Gzip" para la ejecución MDX en SAP Business Warehouse. Se recomienda para consultas de bajo volumen o latencia baja.
SapBusinessWarehouseExecutionMode.DataStream	66	Opción "Modo de acoplamiento dataStream" para la ejecución MDX en SAP Business Warehouse.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

SapHanaDistribution.Type

Definición

Especifica opciones válidas para la opción de distribución de SAP HANA.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
SapHanaDistribution.Off	0	Opción de distribución "Desactivado" para SAP HANA.
SapHanaDistribution.Connection	1	Opción de distribución "Conexión" para SAP HANA.
SapHanaDistribution.Statement	2	Opción de distribución "Instrucción" para SAP HANA.
SapHanaDistribution.All	3	Muestra los paquetes de una base de datos SAP HANA.

Se aplica a

- [Acceso a funciones de datos](#)

Last updated on 22/11/2025

SapHanaRangeOperator.Type

Definición

Especifica un operador de intervalo para los parámetros de entrada de intervalo de SAP HANA.

Valores permitidos

[Expandir tabla](#)

Nombre	Importancia	Description
SapHanaRangeOperator.GreaterThan	0	Operador de intervalo "Mayor que" para los parámetros de entrada de SAP HANA.
SapHanaRangeOperator.LessThan	1	Operador de intervalo "Menor que" para los parámetros de entrada de SAP HANA.
SapHanaRangeOperator.GreaterThanOrEquals	2	Operador de intervalo "Mayor o igual que" para los parámetros de entrada de SAP HANA.
SapHanaRangeOperator.LessThanOrEquals	3	Operador de intervalo "Menor o igual que" para los parámetros de entrada de SAP HANA.
SapHanaRangeOperator.Equals	4	Operador de intervalo "Equals" para parámetros de entrada de SAP HANA.
SapHanaRangeOperator.NotEquals	5	Operador de intervalo 'Not equals' para los parámetros de entrada de SAP HANA.

Se aplica a

- [Acceso a funciones de datos](#)

TextEncoding.Type

Definición

Especifica el tipo de codificación de texto.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
TextEncoding.Utf16	1200	Use para elegir el formato binario UTF16 little endian.
TextEncoding.Unicode	1200	Use para elegir el formato binario UTF16 little endian.
TextEncoding.BigEndianUnicode	1201	Utilizar para seleccionar el formato binario UTF16 big endian.
TextEncoding.Windows	1252	Utiliza para elegir el formato binario de Windows.
TextEncoding.Ascii	20127	Use para elegir el formulario binario ASCII.
TextEncoding.Utf8	65001	Utilizar para elegir el formato binario UTF8.

 Nota:

Cualquier parámetro de codificación de texto debe aceptar cualquiera de las páginas de códigos de Windows por número. No necesita una enumeración M para ella.

Se aplica a

- [Funciones de texto](#)

Last updated on 22/11/2025

TraceLevel.Type

Definición

Especifica el nivel de seguimiento.

Valores permitidos

 Expandir tabla

Nombre	Importancia	Description
TraceLevel.Critical	1	Especifica el nivel de seguimiento crítico.
TraceLevel.Error	2	Especifica el nivel de seguimiento de errores.
TraceLevel.Warning	4	Especifica el nivel de seguimiento advertencia.
TraceLevel.Information	8	Especifica el nivel de seguimiento de información.
TraceLevel.Verbose	16	Especifica el nivel de seguimiento detallado.

Se aplica a

- [Control de errores](#)

Last updated on 22/11/2025

WebMethod.Type

16/09/2025

Definición

Especifica un método HTTP.

Valores permitidos

[Expandir tabla](#)

Nombre	Valor	Descripción
WebMethod.Delete	"DELETE"	Especifica el método DELETE para HTTP.
WebMethod.Get	"GET"	Especifica el método GET para HTTP.
WebMethod.Head	"HEAD"	Especifica el método HEAD para HTTP.
WebMethod.Patch	"PATCH"	Especifica el método PATCH para HTTP.
WebMethod.Post	"POST"	Especifica el método POST para HTTP.
WebMethod.Put	"PUT"	Especifica el método PUT para HTTP.

Observaciones

Estos campos solo funcionan en el contexto de conectores personalizados.

Se aplica a

- [Acceso a funciones de datos](#)

Constants

16/09/2025

El lenguaje de fórmulas M de Power Query incluye estos valores constantes.

Lista de constantes

 Expandir tabla

Nombre	Description
ItemExpression.Item	Nodo de árbol de sintaxis abstracta (AST) que representa el elemento en una expresión de elemento.
Number.E	Devuelve 2.7182818284590451, el valor de e hasta 16 dígitos decimales.
Number.Epsilon	Devuelve el número más pequeño posible.
Number.NaN	Representa 0/0.
Number.NegativeInfinity	Representa el intervalo de -1 a 0.
Number.Pi	Devuelve 3.1415926535897931, el valor de Pi hasta 16 dígitos decimales.
Number.PositiveInfinity	Representa 1/0.
RowExpression.Row	Nodo de árbol de sintaxis abstracta (AST) que representa una fila en una expresión de tipo fila.

Contenido relacionado

- [Funciones numéricas](#)
- [Construcción de tabla](#)

ItemExpression.Item

16/09/2025

Acerca de

Nodo de árbol de sintaxis abstracta (AST) que representa el elemento en una expresión de elemento.

Este valor es idéntico a [RowExpression.Row](#).

Number.E

16/09/2025

Acerca de

Constante que representa 2.7182818284590451, el valor de e hasta 16 dígitos decimales.

Number.Epsilon

16/09/2025

Acerca de

Valor constante que representa el número positivo más pequeño que puede contener un número de punto flotante.

Number.NaN

16/09/2025

Acerca de

Valor constante que representa 0 dividido por 0.

Number.NegativeInfinity

16/09/2025

Acerca de

Valor constante que representa -1 dividido por 0.

Number.PI

16/09/2025

Acerca de

Constante que representa 3.1415926535897932, el valor de pi hasta 16 dígitos decimales.

Number.PositiveInfinity

16/09/2025

Acerca de

Valor constante que representa 1 dividido por 0.

RowExpression.Row

Acerca de

Nodo de árbol de sintaxis abstracta (AST) que representa una fila en una expresión de tipo fila.

Este valor es idéntico a [ItemExpression.Item](#).

Last updated on 22/11/2025

Valores dinámicos

El lenguaje de fórmulas M de Power Query incluye estos valores dinámicos.

Lista de valores dinámicos

 Expandir tabla

Nombre	Descripción
Culture.Current	Devuelve el nombre de la cultura actual de la aplicación.
TimeZone.Current	Devuelve el nombre de la zona horaria actual de la aplicación.

Consulte también

- [Comparar functions](#)
- [DateTimeZone functions](#)

Last updated on 02/04/2026

Culture.Current

16/09/2025

Acerca de

Devuelve el nombre de la cultura actual de la aplicación.

Contenido relacionado

- [Cómo afecta la referencia cultural al formato de texto](#)

TimeZone.Current

16/09/2025

Acerca de

Devuelve el nombre de la zona horaria actual de la aplicación.